
Fundamentals of Python Programming

Richard L. Halterman

Southern Adventist University

Fundamentals of Python Programming

Copyright 2023 Richard L. Halterman. All rights reserved.

See the preface for the terms of use of this document.

Contents

1	The Context of Software Development	1
1.1	Software	1
1.2	Development Tools	2
1.3	Learning Programming with Python	4
1.4	Writing a Python Program	4
1.5	The Python Interactive Shell	10
1.6	A Longer Python program	12
1.7	Exercises	13
2	Values and Variables	15
2.1	Integer and String Values	15
2.2	Variables and Assignment	19
2.3	Identifiers	27
2.4	Floating-point Numbers	29
2.5	Control Codes within Strings	33
2.6	Simple User Input	34
2.7	Controlling the <code>print</code> Function	37
2.8	Number Formatting	40
2.9	Multi-line Strings	43
2.10	Exercises	44
3	Expressions and Arithmetic	47
3.1	Expressions	47
3.2	Mixed Type Expressions	53
3.3	Operator Precedence and Associativity	53
3.4	Formatting Expressions	55

3.5	Comments	56
3.6	Errors	56
3.6.1	Syntax Errors	57
3.6.2	Run-time Exceptions	58
3.6.3	Logic Errors	60
3.7	Arithmetic Examples	61
3.8	More Arithmetic Operators	62
3.9	Algorithms	64
3.10	Exercises	66
4	Conditional Execution	71
4.1	Boolean Expressions	71
4.2	Boolean Expressions	72
4.3	The Simple if Statement	73
4.4	The if/else Statement	79
4.5	Compound Boolean Expressions	81
4.6	The pass Statement	84
4.7	Floating-point Equality	85
4.8	Nested Conditionals	87
4.9	Multi-way Decision Statements	100
4.10	Multi-way Versus Sequential Conditionals	104
4.11	Structural Pattern Matching	108
4.12	Conditional Expressions	111
4.13	Errors in Conditional Statements	113
4.14	Logic Complexity	115
4.15	Exercises	118
5	Iteration	123
5.1	The while Statement	123
5.2	Definite Loops versus Indefinite Loops	132
5.3	The for Statement	133
5.4	Nested Loops	138
5.5	Abnormal Loop Termination	145
5.5.1	The break statement	145
5.5.2	The continue Statement	149

5.6	while/else and for/else	150
5.7	Infinite Loops	152
5.8	Iteration Examples	156
5.8.1	Computing Square Root	156
5.8.2	Drawing a Tree	157
5.8.3	Printing Prime Numbers	159
5.8.4	Insisting on the Proper Input	163
5.9	Exercises	164
6	Using Functions	171
6.1	Introduction to Using Functions	172
6.2	Functions and Modules	177
6.3	The Built-in Functions	179
6.4	Standard Mathematical Functions	181
6.5	time Functions	186
6.6	Random Numbers	188
6.7	System-specific Functions	192
6.8	The eval and exec Functions	192
6.9	Turtle Graphics	195
6.10	Other Techniques for Importing Functions and Modules	202
6.11	Exercises	208
7	Writing Functions	211
7.1	Function Basics	212
7.2	Parameter Passing	229
7.3	Documenting Functions	231
7.4	Type Hints	233
7.5	Function Examples	237
7.5.1	Better Organized Prime Generator	237
7.5.2	Command Interpreter	239
7.5.3	Restricted Input	241
7.5.4	Better Die Rolling Simulator	242
7.5.5	Tree Drawing Function	244
7.5.6	Floating-point Equality	245
7.6	Refactoring to Eliminate Code Duplication	248

7.7	Custom Functions versus Standard Functions	249
7.8	Exercises	253
8	More on Functions	259
8.1	Global Variables	259
8.2	Default Parameters	264
8.3	Introduction to Recursion	267
8.4	Making Functions Reusable	273
8.5	Executable Modules	277
8.6	Functions as Data	279
8.7	Separating Concerns with Pluggable Modules	284
8.8	Lambda Expressions	305
8.9	Generators	310
8.10	Local Function Definitions	318
8.11	Decorators	326
8.12	Partial Application	340
8.13	Exercises	343
9	Objects	349
9.1	Objects Terminology	349
9.2	String Objects	350
9.3	File Objects	354
9.4	Fraction Objects	361
9.5	Turtle Graphics Objects	363
9.6	Graphics with <code>tkinter</code> Objects	364
9.7	Other Standard Python Objects	370
9.8	Object Mutability and Aliasing	370
9.9	Garbage Collection	374
9.10	Exercises	375
10	Lists	377
10.1	Using Lists	379
10.2	List Traversal	384
10.3	Building Lists	386
10.4	List Membership	392

10.5	Lists and Functions	393
10.6	List Assignment and Equivalence	395
10.7	List Bounds	400
10.8	Slicing	402
10.9	List Element Removal	405
10.10	List Methods	406
10.11	Prime Generation with a List	409
10.12	Command-line Arguments	411
10.13	List Comprehensions	412
10.14	Multidimensional Lists	418
10.15	Summary of List Creation Techniques	428
10.16	Lists versus Generators	429
10.17	Structural Pattern Matching Revisited	429
10.18	Exercises	432
11	Tuples, Dictionaries, and Sets	437
11.1	Tuples	437
11.2	Arbitrary Argument Lists	442
11.3	Dictionaries	448
11.4	Using Dictionaries	451
11.5	Counting with Dictionaries	455
11.6	Grouping with Dictionaries	459
11.7	Keyword Arguments	462
11.8	Sets	465
11.9	Set Quantification with <code>all</code> and <code>any</code>	468
11.10	Enumerating the Elements of a Data Structure	471
11.11	Exercises	473
12	Handling Exceptions	477
12.1	Motivation	477
12.2	Common Standard Exceptions	478
12.3	Handling Exceptions	480
12.4	Handling Multiple Exceptions	483
12.5	The Catch-all Handler	484
12.6	Catching Exception Objects	488

12.7	Exception Handling Scope	489
12.8	Raising Exceptions	500
12.9	The try Statement's Optional else Block	507
12.10	finally block	509
12.11	Using Exceptions Wisely	512
12.12	Exercises	513
13	Custom Types	517
13.1	Structured Data	517
13.2	Circle Objects	521
13.3	Timing Objects	530
13.4	Traffic Light Objects	533
13.5	Automated Testing	540
13.6	Plotting Data	545
13.7	Dynamic Content	550
13.8	Class Variables	553
13.9	Restricting Access to Members	555
13.10	Rational Numbers	558
13.11	Bank Account Objects	563
13.12	Exercises	567
14	Class Design: Composition and Inheritance	573
14.1	Composition	573
14.2	Class Inheritance	578
14.3	Composition versus Inheritance	600
14.4	Multiple Inheritance	603
14.5	Unit Testing	622
14.6	Custom Exceptions	626
14.7	Exercises	629
15	Algorithm Quality	631
15.1	Good Algorithms Versus Bad Algorithms	631
15.2	Sorting	641
15.2.1	Selection Sort	641
15.2.2	Flexible Sorting	645

15.2.3	Quicksort	647
15.3	Search	655
15.3.1	Linear Search	655
15.3.2	Binary Search	657
15.4	Recursion Revisited	668
15.5	List Permutations	674
15.6	Randomly Permuting a List	684
15.7	Reversing a List	689
15.8	Memoization	690
15.9	Exercises	702
16	Representing Relationships with Graphs	705
16.1	Introduction to Graphs	705
16.2	Implementing Graphs in Python	709
16.3	Path Finding	709
16.4	Breadth-first Search	711
16.5	Depth-first Search	722
16.6	Exercises	730
	Index	731

Preface

Legal Notices and Information

This document is copyright 2023 by Richard L. Halterman, all rights reserved.

Permission is hereby granted to make hardcopies and freely distribute the material herein under the following conditions:

- The copyright and this legal notice must appear in any copies of this document made in whole or in part.
- None of material herein can be sold or otherwise distributed for commercial purposes without written permission of the copyright holder.
- Instructors at any educational institution may freely use this document in their classes as a primary or optional textbook under the conditions specified above.

A local electronic copy of this document may be made under the terms specified for hardcopies:

- The copyright and these terms of use must appear in any electronic representation of this document made in whole or in part.
- None of material herein can be sold or otherwise distributed in an electronic form for commercial purposes without written permission of the copyright holder.
- Instructors at any educational institution may freely store this document in electronic form on a local server as a primary or optional textbook under the conditions specified above.

Additionally, a hardcopy or a local electronic copy must contain the uniform resource locator (URL) providing a link to the original content so the reader can check for updated and corrected content. The current URL is

<http://python.cs.southern.edu/pythonbook/pythonbook.pdf>

If you are an instructor using this book in one or more of your courses, please let me know. Keeping track of how and where this book is used helps me justify to my employer that it is providing a useful service to the community and worthy of the time I spend working on it. Simply send a message to halterman@southern.edu with your name, your institution, and the course(s) in which you use it.

Chapter 1

The Context of Software Development

A computer program, from one perspective, is a sequence of instructions that dictate the flow of electrical impulses within a computer system. These impulses affect the computer's memory and interact with the display screen, keyboard, mouse, and perhaps even other computers across a network in such a way as to produce the “magic” that permits humans to perform useful tasks, solve high-level problems, and play games. One program allows a computer to assume the role of a financial calculator, while another transforms the machine into a worthy chess opponent. Note the two extremes here:

- at the lower, more concrete level electrical impulses alter the internal state of the computer, while
- at the higher, more abstract level computer users accomplish real-world work or derive actual pleasure.

So well is the higher-level illusion achieved that most computer users are oblivious to the lower-level activity (the machinery under the hood, so to speak). Surprisingly, perhaps, most programmers today write software at this higher, more abstract level also. An accomplished computer programmer can develop sophisticated software with little or no interest or knowledge of the actual computer system upon which it runs. Powerful software construction tools hide the lower-level details from programmers, allowing them to solve problems in higher-level terms.

1.1 Software

Computing professionals known as *software engineers* develop programs that direct computers to solve particular tasks. A computer program is an example of computer *software*. One can refer to a program as a *piece* of software as if it were a tangible object, but software is actually quite intangible. It is stored on a *medium*. A hard drive, a compact disc (CD), a DVD, and a USB pen drive are all examples of media upon which software can reside. The CD is not the software; the software is a pattern on the CD. In order to be used, a program must reside in the computer's memory. Typically the user loads a computer program into memory from a medium like the computer's hard disk. The user may originally installed the program on the hard disk from a CD or from the Internet. In any case, the essence that was transferred from medium to medium was a pattern of electronic symbols that direct the work of the computer system.

These patterns of electronic symbols are best represented as a sequence of zeroes and ones, digits from the binary (base 2) number system. An example of a binary program sequence is

10001011011000010001000001001110

To the underlying computer hardware, specifically the processor, a zero here and three ones there might mean that certain electrical signals should be sent to the graphics device so that it makes a certain part of the display screen red. Unfortunately, only a minuscule number of people in the world would be able to produce, by hand, the complete sequence of zeroes and ones that represent the program Microsoft Word for an Intel-based computer running the Windows 11 operating system. Further, almost none of those who could produce the binary sequence would claim to enjoy the task.

The Word program for an Apple Mac computer using an M3 processor works similarly to the Windows version. The same company, Microsoft, produced both versions, but the macOS version of the Word program contains a completely different pattern of zeros and ones from the Windows version. An Intel Core i7 processor in the Windows machine accepts a completely different binary language than the M3 processor in the Mac. We say the processors have their own *machine language*.

1.2 Development Tools

If very few humans can (or want) to speak the machine language of the computers' processors, and processors understand only machine language, how is software so ubiquitous in the modern world?

We can represent software by words and symbols that are easier for humans to manage than binary sequences. We then use tools to automatically convert this higher-level description of what we want the computer to do into the required lower-level code. Higher-level programming languages like Python allow programmers to express solutions to programming problems in terms that are much closer to a natural language like English. Some examples of the more popular of the hundreds of higher-level programming languages that have been devised over the past 60 years include FORTRAN, COBOL, Lisp, Haskell, C, Perl, C++, Java, and C#. Most programmers today, especially those concerned with high-level applications, usually do not worry about the details of the underlying hardware platform and its machine language.

Consider the following higher-level program code written in the Python programming language:

```
subtotal = 25
tax = 3
total = subtotal + tax
```

While these three lines do constitute a complete, proper Python program, they more likely are merely a small fragment from a larger program. The lines of text in this program fragment look similar to expressions in algebra. We see no sequence of binary digits. Three words, `subtotal`, `tax`, and `total` are called *variables*, and they represent numeric information. Mathematicians have used variables for hundreds of years before the first digital computer was built. In programming, a variable represents a value stored in the computer's memory. Instead of some cryptic binary instructions meant only for the processor, we see familiar-looking mathematical operators (`=` and `+`). Since this program is expressed in the Python language, not machine language, no computer processor can execute the program directly. A program called an *interpreter* translates the Python code into machine code when a user runs the program. The higher-level language code is called *source code*. The corresponding machine language code is called the *target code*. The interpreter translates the source code into the target machine language.

The beauty of higher-level languages is this: the same Python source code can execute on different target platforms. The target platform must have a Python interpreter available, but multiple Python interpreters are available for all the major computing platforms. This frees the human programmer from being concerned about the details of a specific machine language. Developers can concentrate on writing the solution in Python.

Programmers have a variety of tools available to enhance the software development process. Some common tools include:

- **Editors.** An *editor* allows the programmer to enter the program source code and save it to files. Most programming editors increase programmer productivity by using colors to highlight language features. The *syntax* of a language refers to the way of organizing pieces of the language to make well-formed sentences. To illustrate, the sentence

The tall boy runs quickly to the door.

uses proper English syntax. By comparison, the sentence

Boy the tall runs door to quickly the.

is not correct syntactically. It uses the same words as the original sentence, but their arrangement does not follow the rules of English.

Similarly, programming languages have strict syntax rules that programmers must follow to create well-formed programs. Only well-formed programs are acceptable for translation into executable machine code. Some syntax-aware editors can use colors or other special annotations to alert programmers of syntax errors during the editing process.

- **Compilers.** A *compiler* translates the source code to target code. The target code may be the machine language for a particular platform or embedded device. The target code could be another source language; for example, the earliest C++ compiler translated C++ into C, another higher-level language. A C compiler then translated to C code into executable machine language. (C++ compilers today translate C++ directly into machine language.) Compilers translate the contents of a source file and produce a file containing all the target code. Popular compiled languages include C, C++, Java, and C#.
- **Interpreters.** An *interpreter* is like a compiler, in that it translates higher-level source code into target code (usually machine language). It works differently, however. While a compiler produces an executable program that may run many times with no additional translation needed, an interpreter translates source code statements into machine language each time a user runs the program. A compiled program does not need to be recompiled to run, but an interpreted program must be reinterpreted each time it executes. For this reason interpreted languages are often referred to as *scripting languages*. The interpreter in essence reads the script, where the script is the source code of the program. In general, compiled programs execute more quickly than interpreted programs because the translation activity occurs only once. Interpreted programs, on the other hand, can run as is on any platform with an appropriate interpreter; they do not need to be recompiled to run on a different platform. Python, for example, is used mainly as an interpreted language, but compilers for it are available. Interpreted languages generally are better suited for dynamic, explorative development which many people feel is ideal for beginning programmers. Popular scripting languages include Python, Ruby, Perl, and, for web browsers, Javascript.
- **Debuggers.** A *debugger* allows a programmer to more easily trace a program's execution in order to locate and correct errors in the program's implementation. With a debugger, a developer can simultaneously run a program and see which line in the source code is responsible for the program's current actions. The programmer can watch the values of variables and other program elements to see if their values change as expected. Debuggers are valuable for locating errors (also called *bugs*) and repairing programs that contain errors. (See Section 3.6 for more information about programming errors.)
- **Profilers.** A *profiler* collects statistics about a program's execution allowing developers to tune appropriate parts of the program to improve its overall performance. A profiler indicates how many

times a portion of a program is executed during a particular run and how long that portion takes to execute. Developers also can use profilers for testing purposes to ensure all the code in a program is actually being used somewhere during testing. This is known as *coverage*. It is common for software to fail after its release because users exercise some part of the program that developers failed to test. The main purpose of profiling is to find the parts of a program that can be improved to make the program run faster.

Many developers use integrated development environments (IDEs). An IDE includes editors, debuggers, and other programming aids in one comprehensive program. Python IDEs include Wingware, Enthought, and IDLE.

Despite the wide variety of tools (and tool vendors' claims), the programming process for all but trivial programs is not automatic. Generative artificial intelligence tools are valuable and certainly increase the productivity of developers, but they cannot guarantee correct software. There are no substitutes for sound logical thinking, creativity, common sense, and, of course, programming experience.

1.3 Learning Programming with Python

Guido van Rossum created the Python programming language in the late 1980s. In contrast to other popular languages such as C, C++, Java, and C#, Python strives to provide a simple but powerful syntax.

Companies and organizations such as Google, Yahoo, Facebook, CERN, Industrial Light and Magic, and NASA use Python for software development.

Experienced programmers can accomplish great things with Python, but Python's beauty is that it is accessible to beginning programmers and allows them to tackle interesting problems more quickly than many other more complex languages that have a steeper learning curve.

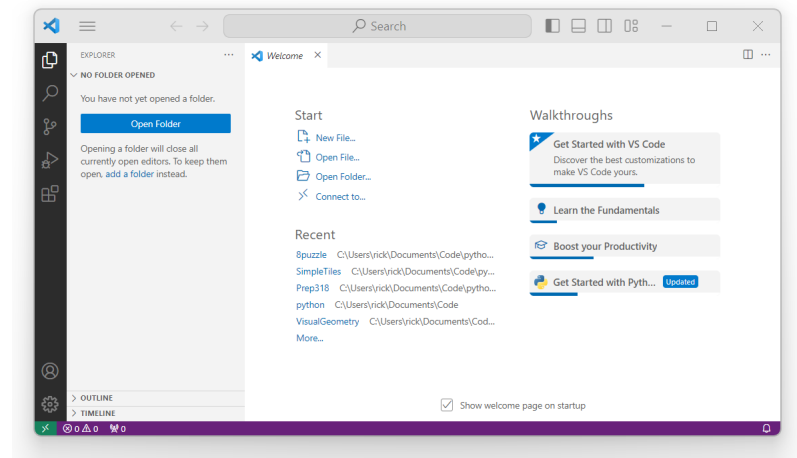
<https://www.python.org> provides more information about Python, including links to download the latest version for Microsoft Windows, Apple macOS, and Linux.

In late 2008, the Python Software Foundation <https://www.python.org/psf-landing/> released Python 3.0. Commonly called Python 3, the current version of Python is incompatible with Python 2, an earlier version of the language. Python developers have almost entirely transitioned to Python 3, but some Python 2 projects still exist. Notably, readers are cautioned that many older Python programming books cover Python 2. All of the code in this book is based on Python 3, specifically version 3.12.0.

This book does not attempt to cover all the facets of the Python programming language. Experienced programmers should look elsewhere for books that cover Python in much more detail. The focus here is on introducing programming techniques and developing good habits. To that end, our approach avoids some of the more esoteric features of Python and concentrates on the programming basics that transfer directly to other imperative programming languages such as Java, C#, and C++. We stick with the basics and explore more advanced features of Python only when necessary to handle the problem at hand.

1.4 Writing a Python Program

The text that makes up a Python program has a particular structure. The syntax must be correct, or the interpreter will generate error messages and not execute the program. This section introduces Python by providing a simple example program.

Figure 1.1 Visual Studio Code running under Microsoft Windows

A program consists of one or more *statements*. A statement is an instruction that the interpreter executes. The following statement invokes the `print` function to display a message:

```
print('This is a simple Python program')
```

We can use the statement in a program. Listing 1.1 (`first.py`) is one of the simplest Python programs that does something:

Listing 1.1: `first.py`

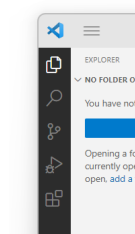
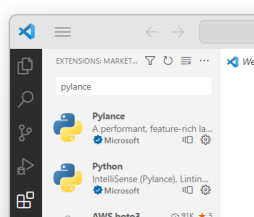
```
print('This is a simple Python program')
```

We need a small collection of tools to write Python programs. Many options exist, but we will use Microsoft's cross-platform Visual Studio Code lightweight IDE. Often abbreviated VS Code, it is available at no cost from <https://code.visualstudio.com/> and runs on Microsoft Windows, Apple macOS, and Linux. The VS Code IDE is programming-language agnostic and by default offers little more than a simple editor and file manager. We will add a couple of extensions to make easy to develop and debug Python programs. VS Code is simple enough for beginning programmers but used by many professional software engineers.

The way you launch VS Code depends on your operating system and how you installed it. Windows users can pin the `VSCode.exe` application to their start menu, and macOS users can pin `VSCode.app` to their dock. Figure 1.1 shows a screenshot of VS Code running on a Windows 11 computer. When first executed, VS Code presents a window similar to the one shown in Figure 1.1.

The left side of the VS Code applications provides several icons arranged vertically. Figure 1.2 zooms in on these icons. The middle section, labeled *Explorer*, will show a list of Python source files once we begin creating them. The large area on the right side that initially presents a welcome page is the editing area in which we will type the code for our Python programs.

VS Code is a generic software development tool and can be customized for many different programming languages. Since we will be using Python, we need to configure it for Python development. This is a simple process of installing two *extensions*. The bottom icon in the vertical strip of icons on the left side—the

Figure 1.2 Icons that determine the contents of the left panel in VS Code**Figure 1.3** Extensions to install for Python development.

one that looks like a set of four small rectangles with the top, right rectangle broken off—provides access to VS Code’s extension manager. If you click on that icon, the *Explorer* panel changes to the *Extensions* panel. In the search box at the top, shown in Figure 1.3, type the word *pylance*. VS Code will offer a number of extensions related to Python programming, but we need just two of them: *Pylance* and *Python*, both provided by Microsoft. VS Code with the Pylance and Python extensions provide all we need for developing software in Python.

VS Code for Windows, macOS, and Linux provide a similar interface and basically work the same way.

To begin entering our program, we will click on the top icon. This changes the left panel back to the *Explorer* view. Click the big button labeled *Open Folder* as shown in Figure 1.4 and navigate to a folder where you wish to store your Python source files. During this process you may open an existing folder or create a brand new, empty folder using the techniques provided by the file manager for your operating system. For this example, we will open (or create, if it does not exist) the folder named *FirstProgram*.

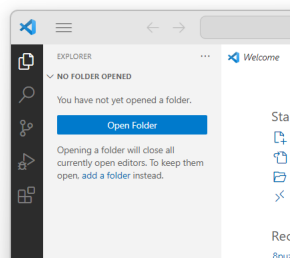
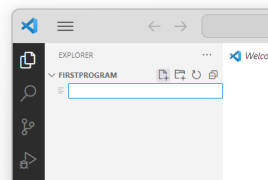
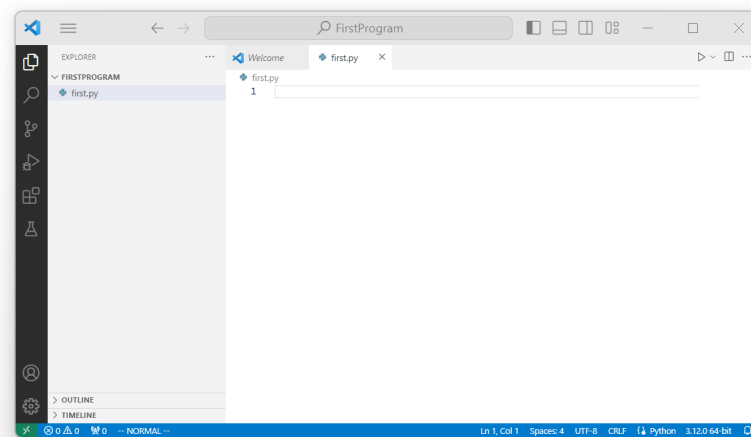
Figure 1.4 Opening or creating a working folder for a Python program.

Figure 1.5 Creating a new Python source file.**Figure 1.6** Editor panel ready for the programmer to enter Python code.

We now are ready to write a Python program. We create a new file by first hovering the mouse cursor over the Explorer panel. This causes a horizontal row of icons to appear near the top of the panel. Select the leftmost icon, which looks like a sheet of paper with its right-top corner folded down. As shown in Figure 1.5, this brings up a text box in which we provide a name for the new file we are creating. Name the file `first.py`. By convention, all Python source files have a `.py` extension. This adds a new tab to the editor panel on the right as shown in Figure 1.6.

We now are ready to type in the code that constitutes the program. Figure 1.7 shows the text to type. The black dot in the tab for our file in the editor indicates that the file is not saved. You make select *Save* from the *File* menu, as shown in Figure 1.8, or simply press the **Ctrl** and **S** keys simultaneously (use the command key on macOS instead of the control key). When the program is safely saved the black dot disappears from the editor tab.

To run the program, click the small triangle in the right top section of the editor panel as shown in Figure 1.9. If the program contains no errors that prevent it from executing, VS Code displays a terminal panel below the editor panel that displays the program's output.

Another way to execute the program is via the *Debug* button in the middle of the vertical icons on the left of the *Explorer* panel as shown in Figure 1.11 (the triangle icon with the little bug). The debugger allows the programmer to see which Python statement the program is executing as it runs. Running via the debugger will be more useful as our programs become more complex.

When you are finished programming and wish to quit the IDE, follow the menu sequence *File*→*Exit* as

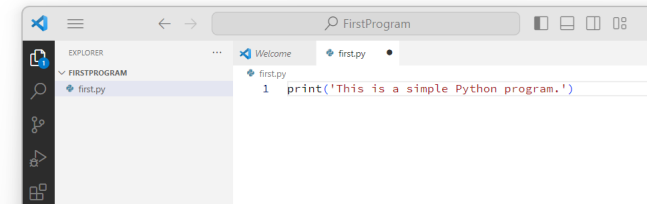
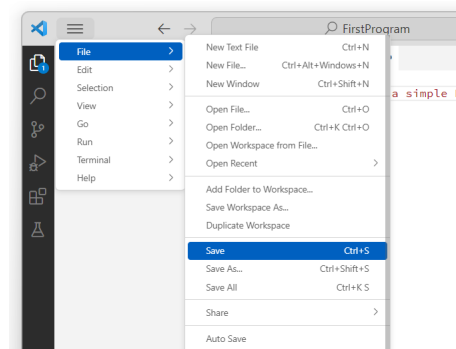
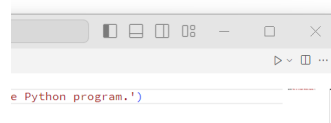
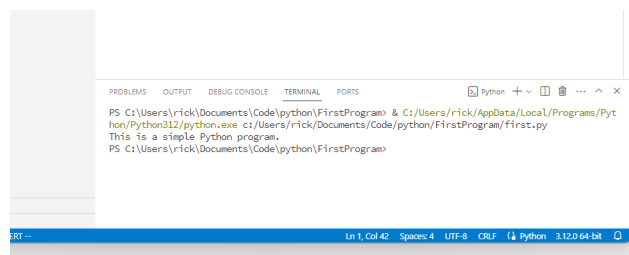
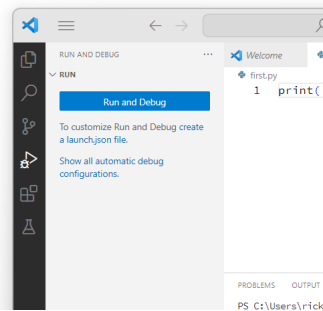
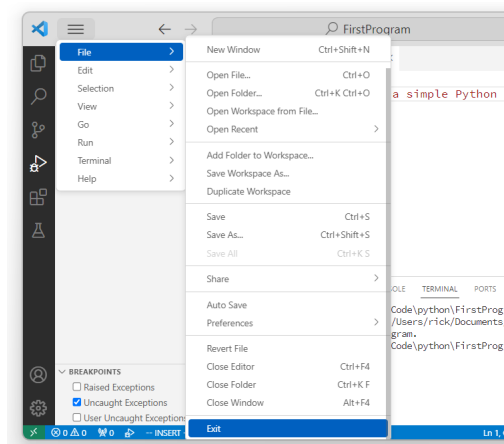
Figure 1.7 The code for the simple program typed into the editor panel.**Figure 1.8** Saving the Python source file.**Figure 1.9** Run the program by clicking the small triangle in the right top section of the editor panel.**Figure 1.10** The VS Code *Terminal* panel displays the program's output.

Figure 1.11 Debugging the program**Figure 1.12** Exiting VS Code.

shown in Figure 1.12. You also may click the window's x icon as you would any application.

Listing 1.1 (first.py) contains only one line of code:

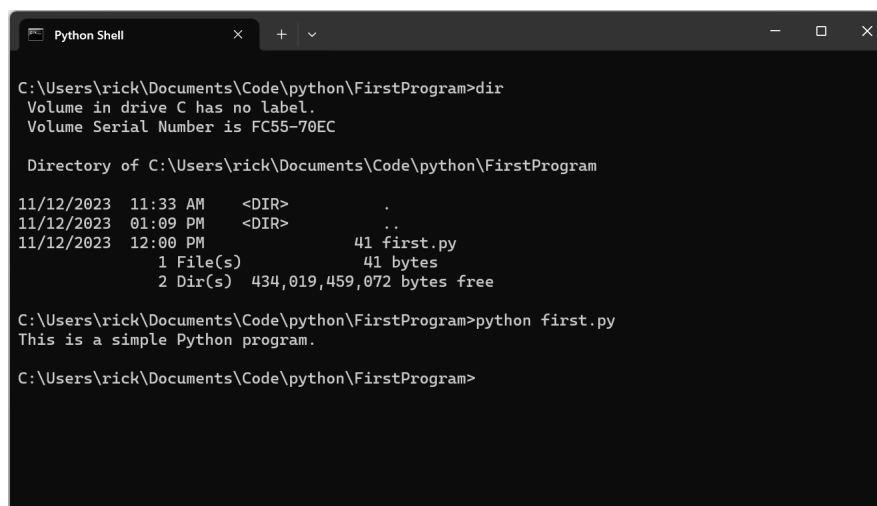
```
print('This is a simple Python program')
```

This is a Python statement. A statement is a command that the interpreter executes. This statement prints the message *This is a simple Python program* on the screen. A statement is the fundamental unit of execution in a Python program. Statements may be grouped into larger chunks called blocks, and blocks can make up more complex statements. Higher-order constructs such as functions and methods are composed of blocks. The statement

```
print('This is a simple Python program')
```

makes use of a built-in function named `print`. Python has a variety of different kinds of statements that we can use to build programs, and the chapters that follow explore these various kinds of statements.

While integrated development environments like VS Code are useful for developing Python programs, we can execute Python programs directly from a command line. In Microsoft Windows, the command console (`cmd.exe`) and PowerShell offer command lines. In Apple macOS, Linux, and Unix, a terminal

Figure 1.13 Running a Python program from the command line

```
C:\Users\rick\Documents\Code\python\FirstProgram>dir
Volume in drive C has no label.
Volume Serial Number is FC55-70EC

Directory of C:\Users\rick\Documents\Code\python\FirstProgram

11/12/2023  11:33 AM    <DIR>          .
11/12/2023  01:09 PM    <DIR>          ..
11/12/2023  12:00 PM                41 first.py
               1 File(s)                41 bytes
               2 Dir(s)  434,019,459,072 bytes free

C:\Users\rick\Documents\Code\python\FirstProgram>python first.py
This is a simple Python program.

C:\Users\rick\Documents\Code\python\FirstProgram>
```

provides a command line. Figure 1.13 shows the Windows command shell running a Python program. In all cases the user's *path* environment variable (%path% on Windows and PATH on macOS and Linux) must be set properly in order for the operating system to find the Python interpreter to run the program.

Figure 1.10 shows that VS Code's terminal panel displays a program's output as black text on a white background. We can change VS Code to use a dark scheme to reverse the colors. In order to better distinguish visually in this text program source code from program output, we will render source code as black text on a white background and the program's output with white text on a black background, as it appears in the command line interpreter as shown in Figure 1.13. This means we would show the output of Listing 1.1 (first.py) as

```
This is a simple Python program
```

1.5 The Python Interactive Shell

We created the program in Listing 1.1 (first.py) and submitted it to the Python interpreter for execution. We can interact with the interpreter directly, typing in Python statements and expressions for its immediate execution. As we saw in Figure 1.10, the VS Code panel labeled *Terminal* is where the executing program directs its output. We also can type commands into the *Terminal* panel, as it is the equivalent to the Windows command shell (technically, it is a PowerShell terminal by default). The `python` command launches the python interactive interpreter. Figure 1.14 shows how the interpreter responds when we enter the program statement directly into the shell. The interpreter prompts the user for input with three greater-than symbols (`>>>`). This means the user typed in the text on the line prefixed with `>>>`. Any lines without the `>>>` prefix represent the interpreter's output, or feedback, to the user.

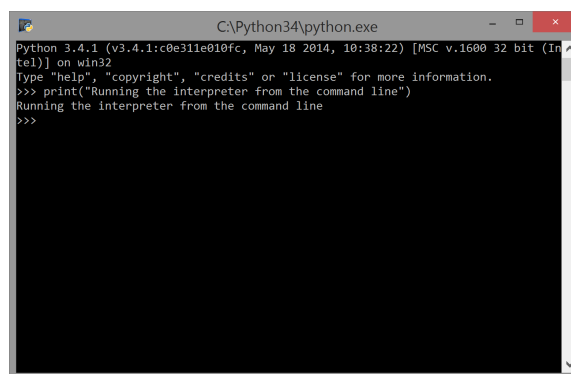
We will find Python's interactive interpreter invaluable for experimenting with various language constructs. We can discover many things about Python without ever writing a complete program.

We can execute the interactive Python interpreter directly from the command line, as Figure 1.15 demonstrates. This means not only can we execute Python programs apart from the VS Code developer environ-

Figure 1.14 Type the command `python` in VS Code's *Terminal* panel to launch the Python interactive interpreter. The interpreter shell allows us to submit Python statements and expressions directly to the interpreter



Figure 1.15 Running the Python interpreter from the command line



ment, we also we can access Python’s interactive interpreter separately from VS Code if we so choose.

Within a Windows command shell, you can exit the Python interpreter by pressing **Ctrl** **Z**. Within a macOS or Linux terminal, **Ctrl** **D** exits the Python interpreter.

In order for readers of this text to better distinguish visually interactive sessions from program output, we will render the user’s direct interaction with the Python interpreter as blue text on a light-gray background. As an example, the following shows a possible interactive session with a user:

```
>>> print('Hello!')
Hello!
```

The interpreter prompt (`>>>`) prefixes all user input in the interactive shell. Lines that do not begin with the `>>>` prompt represent the interpreter’s response.

1.6 A Longer Python program

More interesting programs contain multiple statements. In Listing 1.2 (`arrow.py`), six `print` statements draw an arrow on the screen:

Listing 1.2: `arrow.py`

```
print('  *  ')
print(' *** ')
print('*****')
print('  *  ')
print('  *  ')
print('  *  ')
```

We wish the output of Listing 1.2 (`arrow.py`) to be

```
  *
 ***
*****
  *
  *
  *
```

If you try to enter each line one at a time into the interactive shell, the program’s output will be intermingled with the statements you type. In this case the best approach to achieve the visual effect of an upward pointing arrow is to type the program into an editor, save the code you type to a file, and then execute the program. Most of the time we use an editor to enter and run our Python programs. The interactive interpreter is most useful for experimenting with small snippets of Python code.

In Listing 1.2 (`arrow.py`) each `print` statement “draws” a horizontal slice of the arrow. All the horizontal slices stacked on top of each other results in the picture of the arrow. The statements form a *block* of Python code. It is important that no *whitespace* (spaces or tabs) come before the beginning of each statement. In Python, the indentation of statements is significant and the interpreter generates error messages for improper indentation. If we try to put a single space before a statement in the interactive shell, we get

```
>>> print('hi')
File "<stdin>", line 1
    print('hi')
    ^
IndentationError: unexpected indent
```

The interpreter reports a similar error when we attempt to run a saved Python program if the code contains such extraneous indentation. Consider Listing 1.3 (badindent.py), which provides such an example in a Python program.

Listing 1.3: badindent.py

```
print(' * ')
print(' *** ')
print(' ***** ')
print(' * ')
print(' * ')
print(' * ')
```

An attempt to run Listing 1.3 (badindent.py) results in the following error message:

```
C:\Code\python>python badindent.py
File "C:\Code\python\badindent.py", line 3
    print(' ***** ')
    ^
IndentationError: unexpected indent
```

The third line of Listing 1.3 (badindent.py) has a space at the beginning. All the statements should in this case have no indentation.

Python does allow programmers to place more than one statement on a single line. Semicolons (;) must separate the statements. Listing 1.4 (mulistatementspersline.py), rewrites Listing 1.2 (arrow.py) in just two lines of code.

Listing 1.4: mulistatementspersline.py

```
print(' * '); print(' *** '); print(' ***** ')
print(' * '); print(' * '); print(' * ')
```

It is extremely rare to see Python code that contains more than one statement per line. Writing more than one statement per line makes the code less readable and understandable by humans. The situation worsens as program logic becomes more complicated; therefore, the practice is highly discouraged. Best practices dictate that each Python statement appear on its own line within a program.

1.7 Exercises

1. What is a compiler?
2. What is an interpreter?
3. How is a compiler similar to an interpreter? How are they different?
4. How is compiled or interpreted code different from source code?

5. What tool does a programmer use to produce Python source code?
6. What is necessary to execute a Python program?
7. List several advantages developing software in a higher-level language has over developing software in machine language.
8. How can an IDE improve a programmer's productivity?
9. What is the “official” Python IDE?
10. What is a *statement* in a Python program?

Chapter 2

Values and Variables

In this chapter we explore some building blocks used to develop Python programs. We experiment with the following concepts:

- numeric values
- strings
- variables
- assignment
- identifiers
- reserved words

In the next chapter we will revisit some of these concepts in the context of other data types.

2.1 Integer and String Values

The number four (4) is an example of a *numeric* value. In mathematics, 4 is an *integer* value. Integers are whole numbers, which means they have no fractional parts, and they can be positive, negative, or zero. Examples of integers include 4, -19 , 0, and -1005 . In contrast, 4.5 is not an integer, since it is not a whole number.

Python supports numeric and nonnumeric values. In particular, Python programs can use integer values. The Python statement

```
print(4)
```

prints the value 4. Notice that unlike Listing 1.1 (`first.py`) and Listing 1.2 (`arrow.py`) no quotation marks (`'` or `"`) appear in the statement. The value 4 is an example of an integer *expression*. Python supports other types of expressions besides integer expressions. An expression is a basic building block of a Python statement.

The number 4 by itself is not a complete Python statement and, therefore, cannot be a program. The interpreter, however, can evaluate a Python expression. You may type the enter 4 directly into the interactive interpreter shell:

```
Type "help", "copyright", "credits" or "license" for more information.
>>> 4
4
>>>
```

The interactive shell attempts to evaluate both expressions and statements. In this case, the expression 4 evaluates to 4. The shell executes what is commonly called the *read, eval, print loop*, often abbreviated as REPL. This means the interactive shell's sole activity consists of

1. *reading* the text entered by the user,
2. attempting to *evaluate* the user's input in the context of what the user has entered up that point, and
3. *printing* its evaluation of the user's input.

If the user enters a 4, the shell interprets it as a 4. If the user enters `x = 10`, a statement has no overall value itself, the shell prints nothing. If the user then enters `x`, the shell prints the evaluation of `x`, which is 10. If the user next enters `y`, the shell reports a error because no interaction up to that point defined the expression `y`.

Python uses the `+` symbol with integers to perform normal arithmetic addition, so the interactive shell can serve as a handy adding machine:

```
>>> 3 + 4
7
>>> 1 + 2 + 4 + 10 + 3
20
>>> print(1 + 2 + 4 + 10 + 3)
20
```

The last line evaluated shows how we can use the `+` symbol to add values within a `print` statement that could be part of a Python program.

Consider what happens if we use quote marks around an integer:

```
>>> 19
19
>>> "19"
'19'
>>> '19'
'19'
```

Notice how the output of the interpreter is different. The expression `"19"` is an example of a *string* value. A string is a sequence of characters. Strings most often contain nonnumeric characters:

```
>>> "Fred"
'Fred'
>>> 'Fred'
'Fred'
```

Python recognizes both single quotes (`'`) and double quotes (`"`) as valid ways to delimit a string value. The word *delimit* means to determine the boundaries or limits of something. The left `'` symbol determines the beginning of a string, and the right `'` symbol that follows specifies the end of the string. If a single quote

marks the beginning of a string value, a single quote must delimit the end of the string. Similarly, the double quotes, if used instead, must appear in pairs. You may not mix the two kinds of quotation marks when used to delimit a particular string, as the following interactive sequence shows:

```
>>> 'ABC'
'ABC'
>>> 'ABC"
  File "<stdin>", line 1
    'ABC"
    ^
SyntaxError: unterminated string literal (detected at line 1)
>>> "ABC"
'ABC'
>>> 'ABC'
'ABC'
>>> 'ABC"
  File "<stdin>", line 1
    'ABC"
    ^
SyntaxError: unterminated string literal (detected at line 1)
>>> "ABC"
  File "<stdin>", line 1
    "ABC"
    ^
SyntaxError: unterminated string literal (detected at line 1)
```

The interpreter's output always uses single quotes, but it accepts either single or double quotes as valid input.

Consider the following interaction sequence:

```
>>> 19
19
>>> "19"
'19'
>>> '19'
'19'
>>> "Fred"
'Fred'
>>> 'Fred'
'Fred'
>>> Fred
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
NameError: name 'Fred' is not defined
```

Notice that without the quotation marks the interpreter does not accept the expression `Fred`.

It is important to note that the expressions `4` and `'4'` are different. The expression `4` is an integer, while the expression `'4'` is a string containing the character representing the digit 4. One is an integer expression and the other is a string expression. All expressions in Python have a *type*. The type of an expression indicates the kind of expression it is. An expression's type is sometimes denoted as its *class*. At this point we have considered only integers and strings. The built in `type` function reveals the type of any Python expression:


```
>>> type(4)
<class 'int'>
>>> type('4')
<class 'str'>
```

Python associates the type name `int` with integer expressions and `str` with string expressions.

The built-in `int` function creates an actual integer object from a string that looks like an integer, and the `str` function creates a string object from the digits that make up an integer:

```
>>> 4
4
>>> str(4)
'4'
>>> '5'
'5'
>>> int('5')
5
```

The expression `str(4)` evaluates to the string value `'4'`, and `int('5')` evaluates to the integer value 5. The `int` function applied to an integer evaluates simply to the value of the integer itself, and similarly `str` applied to a string results in the same value as the original string:

```
>>> int(4)
4
>>> str('Judy')
'Judy'
```

As you might guess, there is little reason for a programmer to transform an object into itself—the expression `int(4)` is more easily expressed as 4, so the utility of the `str` and `int` functions will not become apparent until we introduce variables (Section 2.2) and need to process user input (Section 2.6).

The `int` and `str` functions are examples of *constructor* functions; they construct, or create, new objects.

Any integer has a string representation, but not all strings have an integer equivalent:

```
>>> str(1024)
'1024'
>>> int('wow')
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
ValueError: invalid literal for int() with base 10: 'wow'
>>> int('3.4')
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
ValueError: invalid literal for int() with base 10: '3.4'
>>> int(3.4)
3
```

In Python, neither the string `wow` nor the string of characters `3.4` represent valid integer expressions. Observe, however, that Python treats the expression `3.4` without quotes differently. The number 3.4 is not an integer, but Python can convert it to an integer using the `int` function. In short, if the contents of a string (the characters that make it up) look exactly like a valid integer number, you safely can apply the `int` function to produce the represented integer.

The plus operator (+) works differently for strings; consider:

```
>>> 5 + 10
15
>>> '5' + '10'
'510'
>>> 'abc' + 'xyz'
'abcxyz'
```

As you can see, the result of the expression `5 + 10` is very different from `'5' + '10'`. The plus operator splices two strings together in a process known as *concatenation*. Mixing the two types directly is not allowed:

```
>>> '5' + 10
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: can only concatenate str (not "int") to str
```

but the `int` and `str` functions can help:

```
>>> 5 + int('10')
15
>>> '5' + str(10)
'510'
```

The `type` function can determine the type of the most complicated expressions:

```
>>> type(4)
<class 'int'>
>>> type('4')
<class 'str'>
>>> type(4 + 7)
<class 'int'>
>>> type('4' + '7')
<class 'str'>
>>> type(int('3') + int(4))
<class 'int'>
```

Commas may not appear in Python integer values. We must express the number two thousand, four hundred sixty-eight as `2468`, not `2,468`.

In mathematics, integers are unbounded; said another way, the set of mathematical integers is infinite. In Python, integers may be arbitrarily large, but the larger the integer, the more memory required to represent it. This means Python integers theoretically can be as large or as small as needed, but, since a computer has a finite amount of memory (and the operating system may limit the amount of memory allowed for a running program), in practice the range of Python integers are bounded by available memory.

2.2 Variables and Assignment

In algebra, variables represent numbers. The same is true in Python, except Python variables also can represent things other than numbers. Listing 2.1 (`variable.py`) uses a variable to store an integer value and then prints the value of the variable.

Listing 2.1: variable.py

```
x = 10
print(x)
```

Listing 2.1 (variable.py) contains two statements:

- `x = 10`

This is an *assignment* statement. An assignment statement associates a value with a variable. The key to an assignment statement is the symbol `=` which is known as the *assignment operator*. The statement assigns the integer value 10 to the variable `x`. Said another way, this statement binds the variable named `x` to the value 10. At this point the type of `x` is `int` because it is bound to an integer value.

We may assign and reassign a variable as often as necessary. The type of a variable will change if we reassign it to an expression of a different type.

- `print(x)`

This statement prints the variable `x`'s current value. Note that the lack of quotation marks here is very important. If `x` has the value 10, the statement

```
print(x)
```

prints 10, the value of the variable `x`, but the statement

```
print('x')
```

prints `x`, the message containing the single letter `x`.

The meaning of the assignment operator (`=`) is different from equality in mathematics. In mathematics, the symbol `=` asserts that the expression to its left is equal to the expression on its right. In Python, the assignment operator `=` makes the variable on its left take on the value of the expression on its right. It is best to read `x = 5` as “`x` is assigned the value 5,” “`x` gets the value 5,” or “5 is assigned to `x`.” This distinction is important since in mathematics equality is symmetric: if $x = 5$, we know $5 = x$. In Python this symmetry does not exist; the statement

```
5 = x
```

attempts to reassign the value of the literal integer value 5, but this is impossible because 5 is always 5 and cannot be changed. Such a statement will produce an error.

```
>>> x = 5
>>> x
5
>>> 5 = x
File "<stdin>", line 1
    5 = x
    ^
SyntaxError: cannot assign to literal here. Maybe you meant '==' instead of '='?
```

We can reassign different values to a variable as needed, as Listing 2.2 (multipleassignment.py) shows.

Listing 2.2: multipleassignment.py

```
x = 10
print(x)
x = 20
print(x)
x = 30
print(x)
```

Observe that each `print` statement in Listing 2.2 (`multipleassignment.py`) is identical, but when the program runs (as a program, not in the interactive shell) the `print` statements produce different results:

```
10
20
30
```

This program demonstrates that we cannot always predict the behavior of a statement in isolation, especially if that statement involves a variable. This is because the statement's behavior may be dependent on the assigned values of one or more variables that it uses.

The variable `x` in Listing 2.2 (`multipleassignment.py`) has type `int`, since `x` is bound to an integer value. Listing 2.3 (`multipleassignment2.py`) is an enhancement of Listing 2.2 (`multipleassignment.py`) that provides more descriptive `print` statements.

Listing 2.3: multipleassignment2.py

```
x = 10
print('x = ' + str(x))
x = 20
print('x = ' + str(x))
x = 30
print('x = ' + str(x))
```

Listing 2.3 (`multipleassignment2.py`) outputs

```
x = 10
x = 20
x = 30
```

Listing 2.3 (`multipleassignment2.py`) uses the `str` function to treat `x` as a string so the `+` operator will use string concatenation:

```
print('x = ' + str(x))
```

The expression `'x = ' + x` would not be legal; as indicated in Section 2.1, the plus (`+`) operator may not be applied with mixed string and integer operands.

Listing 2.4 (`multipleassignment3.py`) provides a variation of Listing 2.3 (`multipleassignment2.py`) that produces the same output.

Listing 2.4: multipleassignment3.py

```
x = 10
print('x =', x)
x = 20
print('x =', x)
x = 30
print('x =', x)
```

This version of the `print` statement:

```
print('x =', x)
```

illustrates the `print` function accepting two parameters. The first parameter is the string `'x ='`, and the second parameter is the variable `x` bound to an integer value. The `print` function allows programmers to pass multiple expressions to print, each separated by commas. The elements within the parentheses of the `print` function comprise what is known as a *comma-separated list*. The `print` function prints each element in the comma-separated list of parameters. The `print` function automatically prints a space between each element in the list so the printed text elements do not run together.

Modern Python (since Python 3.6) have special strings known as *formatted string literals*, often called *f-strings*, that make printing the values of variables and expressions more convenient. F-strings are string literals, which means their contents must appear within quotes (`'` or `"`), but some of their content is not displayed literally. Consider the following example:

```
>>> x = 10
>>> word = 'Fred'
>>> my_fstring = f'x = {x}, and word = {word}.'
>>> my_fstring
'x = 10, and word = Fred.'
```

F-strings are string literals with an `f` prefix before the opening quote. The symbol `F` also introduces an f-string. Python replaces any expressions within the curly braces (`{}`) with the string representation of their values. In the above interactive session, the `{x}` part of the string becomes the value of the variable `x` (in this case the integer 10), and the `{word}` part of the string becomes the value of the variable `word` (in this case the string `'Fred'`).

The contents within the curly braces in a Python f-string can be any expression convertible to a string representation. Consider the following example:

```
>>> x = 5
>>> y = 8
>>> s = f'The expression is {x + y + 10}'
>>> s
'The expression is 23'
>>> x = 100
>>> s
'Expression: 23'
>>> s = f'The expression is {x + y + 10}'
>>> s
'The expression is 118'
```

The expression `x + y + 10` first evaluates to 23, and the f-string correctly captures its value. Afterward, if one of the value of any of the variables change, it does not affect to f-string. Note that we need to generate a new f-string if we want to update its value after one or more of the variables change.

A programmer may assign multiple variables in one statement using *tuple assignment*. Listing 2.5 (tupleassign.py) shows how:

Listing 2.5: tupleassign.py

```
x, y, z = 10, -45, 0
print(f'x = {x}, y = {y}, z = {z}')
```

The Listing 2.5 (tupleassign.py) program produces

```
x = 10, y = -45, z = 0
```

A *tuple* is a comma-separated list of expressions. If the variables `total` and `s` are defined, the expression `total, 45, s, 0.3`

represents a 4-tuple; that is, a tuple composed of four elements. In the assignment statement

```
x, y, z = 100, -45, 0
```

`x, y, z` is one tuple, and `100, -45, 0` is another tuple. Tuple assignment works as follows: The first variable in the tuple on left side of the assignment operator is assigned the value of the first expression in the tuple on the right side of the assignment operator (effectively `x = 100`). Similarly, the second variable in the tuple on left side of the assignment operator is assigned the value of the second expression in the tuple on the right side (in effect `y = -45`). `z` gets the value `0`.

Tuple assignment works only if the tuple on the left side of the assignment operator contains the same number of elements as the tuple on the right side, as the following example illustrates:

```
>>> x, y, z = 45, 3
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
ValueError: not enough values to unpack (expected 3, got 2)
>>> x, y, z = 45, 3, 0, 8
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
ValueError: too many values to unpack (expected 3)
```

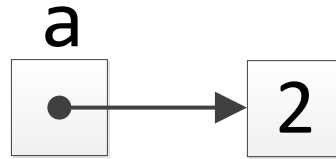
A tuple is a kind of Python type, like `int` or `float`, and we explore tuples in more detail in Chapter 11.

An assignment statement binds a variable name to an object. We can visualize this process with boxes and an arrow as shown in Figure 2.1.

One box represents the variable, so we name the box with the variable's name. The arrow projecting from the box points to the object to which the variable is bound. In this case the arrow points to another box that contains the value `2`. The second box represents a memory location that holds the internal binary representation of the value `2`.

To see how variable bindings can change as the computer executes a sequence of assignment statements, consider the following sequence of Python statements:

```
a = 2
b = 5
a = 3
a = b
b = 7
```

Figure 2.1 Binding a variable to an object**Figure 2.2** How variable bindings change as a program runs: step 1

Figures 2.2–2.6 illustrate how the variable bindings change as the Python interpreter executes each of the above statements. Importantly, the statement

```
a = b
```

means that `a` and `b` both are bound to the same numeric object. Observe that later reassigning `b` does not affect `a`'s value.

Not only may a variable's value change during its use within an executing program; the type of a variable can change as well. Consider Listing 2.6 (`changeabletype.py`).

Listing 2.6: `changeabletype.py`

```
a = 10
print(f'First, variable a has value {a} and type {type(a)}')
a = 'ABC'
print(f'Now, variable a has value {a} and type {type(a)}')
```

Listing 2.6 (`changeabletype.py`) produces the following output:

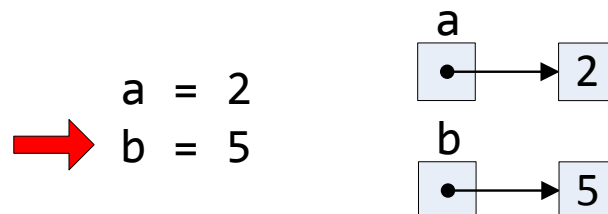
Figure 2.3 How variable bindings change as a program runs: step 2

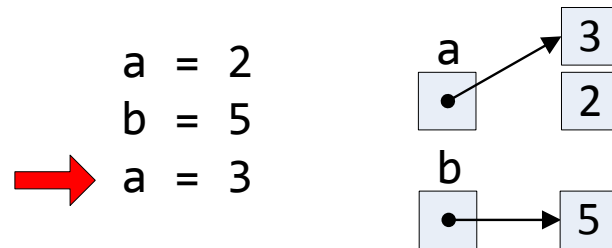
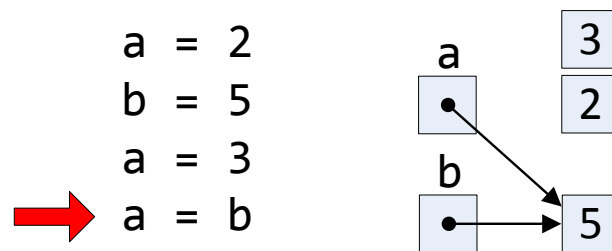
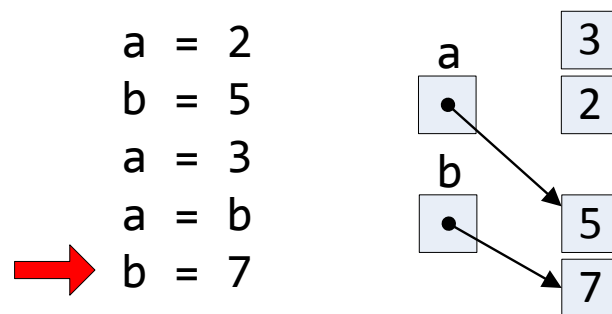
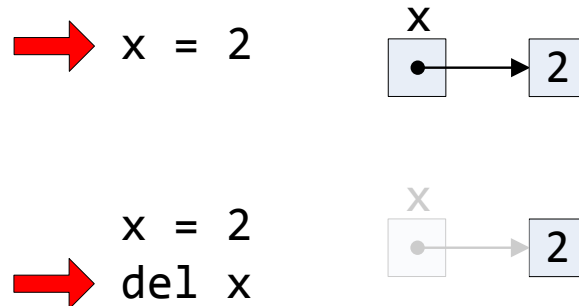
Figure 2.4 How variable bindings change as a program runs: step 3**Figure 2.5** How variable bindings change as a program runs: step 4**Figure 2.6** How variable bindings change as a program runs: step 5

Figure 2.7 Definition and subsequent deletion of variable x

```
First, variable a has value 10 and type <class 'int'>
Now, variable a has value ABC and type <class 'str'>
```

Programmers infrequently perform assignments that change a variable's type. A variable should have a specific meaning within a program, and its meaning should not change during the program's execution. While not always the case, sometimes when a variable's type changes its meaning changes as well.

A variable that has not been assigned is an *undefined variable* or *unbound variable*. Any attempt to use an undefined variable is an error, as the following sequence from Python's interactive shell shows:

```
>>> x = 2
>>> x
2
>>> y
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
NameError: name 'y' is not defined
```

The assignment statement binds 2 to the variable x, and after that the interpreter can evaluate x. The interpreter cannot evaluate the variable y, so it reports an error.

In rare circumstances we may want to undefine a previously defined variable. The `del` statement does that, as the following interactive sequence illustrates:

```
>>> x = 2
>>> x
2
>>> del x
>>> x
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
NameError: name 'x' is not defined
```

The `del` keyword stands for *delete*, and so `del` deletes or removes a variable's definition from the current interpreter session or from an executing Python program. Figure 2.7 illustrates the definition and subsequent deletion of variable x. If variables a, b, and c currently are defined, the statement

```
del a, b, c
```

undefines all three variables in one statement.

2.3 Identifiers

While mathematicians are content with giving their variables one-letter names like `x`, programmers should use longer, more descriptive variable names. Names such as `sum`, `height`, and `sub_total` are much better than the equally permissible `s`, `h`, and `st`. A variable's name should be related to its purpose within the program. Good variable names make programs more readable by humans. Since programs often contain many variables, well-chosen variable names can render an otherwise obscure collection of symbols more understandable.

Python has strict rules for variable names. A variable name is one example of an *identifier*. An identifier is a word used to name things. One of the things an identifier can name is a variable. We will see in later chapters that identifiers name other things such as functions, classes, and methods. Identifiers have the following form:

- An identifiers must contain at least one character.
- The first character of an identifiers must be an alphabetic letter (upper or lower case) or the underscore

ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz_

- The remaining characters (if any) may be alphabetic characters (upper or lower case), the underscore, or a digit

ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz_0123456789

- No other characters (including spaces) are permitted in identifiers.
- A reserved word cannot be used as an identifier (see Table 2.1).

Examples of valid Python identifiers include

- `x`
- `x2`
- `total`
- `port_22`
- `FLAG`.

Table 2.1 Python keywords

and	continue	finally	is	raise
as	def	for	lambda	return
assert	del	from	None	try
async	elif	global	nonlocal	True
await	else	if	not	while
break	except	import	or	with
class	False	in	pass	yield

None of the following words are valid identifiers:

- `sub-total` (dash is not a legal symbol in an identifier)
- `first entry` (space is not a legal symbol in an identifier)
- `4all` (begins with a digit)
- `*2` (the asterisk is not a legal symbol in an identifier)
- `class` (`class` is a reserved word)

Python reserves a number of words for special use that could otherwise be used as identifiers. Called *reserved words* or *keywords*, these words are special and are used to define the structure of Python programs and statements. Table 2.1 lists all the Python reserved words. The purposes of many of these reserved words are revealed throughout this book.

None of the reserved words in Table 2.1 may be used as identifiers. Fortunately, if you accidentally attempt to use one of the reserved words as a variable name within a program, the interpreter will issue an error:

```
>>> class = 15
      File "<stdin>", line 1
        class = 15
            ^
SyntaxError: invalid syntax
```

(see Section 3.6 for more on interpreter generated errors).

To this point we have avoided keywords completely in our programs. This means there is nothing special about the names `print`, `int`, `str`, or `type`, other than they happen to be the names of built-in functions. We are free to reassign these names and use them as variables. Consider the following interactive sequence that reassigns the name `print` to mean something new:

```
>>> print('Our good friend print')
Our good friend print
>>> print
<built-in function print>
>>> type(print)
<class 'builtin_function_or_method'>
>>> print = 77
>>> print
77
>>> print('Our good friend print')
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: 'int' object is not callable
>>> type(print)
<class 'int'>
```

Here we used the name `print` as a variable. In so doing it lost its original behavior as a function to print the console. While we can reassign the names `print`, `str`, `type`, etc., it generally is not a good idea to do so.

Not only can we reassign a function name, but Python allows us to assign a variable to a function.

```
>>> my_print = print
>>> my_print('hello from my_print!')
hello from my_print!
```

After binding the variable `my_print` to `print` we can use `my_print` in exactly as we would use the built-in `print` function.

Python is a case-sensitive language. This means that capitalization matters. `if` is a reserved word, but none of `If`, `IF`, or `iF` is a reserved word. Identifiers also are case sensitive; the variable called `Name` is different from the variable called `name`. Note that only three of Python's 35 reserved words (`False`, `None`, and `True`) are capitalized.

Programmers generally avoid distinguishing between two variables in the same context merely by differences in capitalization. Doing so is more likely to confuse human readers. For the same reason, it is considered poor practice to give a variable the same name as a reserved word with one or more of its letters capitalized.

The most important thing to remember about a variable's name is that it should be well chosen. A variable's name should reflect the variable's purpose within the program. For example, consider a program controlling a point-of-sale terminal (also known as an electronic cash register). The variable keeping track of the total cost of goods purchased might be named `total` or `total_cost`. Variable names such as `a67_99` and `fred` would be poor choices for such a purpose.

2.4 Floating-point Numbers

Many computational tasks require numbers that have fractional parts. For example, to compute the area of a circle given the circle's radius, we use the value π , or approximately 3.14159. Python supports such non-integer numbers, and they are called *floating-point numbers*. The name implies that during mathematical calculations the decimal point can move or "float" to various positions within the number to maintain the

Table 2.2 Characteristics of Floating-point Numbers

Title	Storage	Smallest Magnitude	Largest Magnitude	Minimum Precision
float	64 bits	2.22507×10^{-308}	$1.79769 \times 10^{+308}$	15 digits

proper number of significant digits. The Python name for the floating-point type is `float`. Consider the following interactive session:

```
>>> x = 5.62
>>> x
5.62
>>> type(x)
<class 'float'>
```

The range of floating-points values (smallest value to largest value, both positive and negative) and precision (the number of digits available) depends of the Python implementation for a particular machine. Table 2.2 provides some information about floating point values as commonly implemented on most computer systems. Floating point numbers can be both positive and negative.

As you can see from Table 2.2, unlike Python integers which can be arbitrarily large (or, for negatives, arbitrarily small), floating-point numbers have definite bounds.

Listing 2.7 (`pi-print.py`) prints an approximation of the mathematical value π .

Listing 2.7: `pi-print.py`

```
pi = 3.14159
print(f'Pi = {pi}')
print(f'or {pi:0.3} for short')
print(f'or {3.1416} for short')
```

The first line in Listing 2.7 (`pi-print.py`) assigns an approximation of π to the variable named `pi`, and the second line prints its value. The next line limits the number of digits in the displayed number. The last line prints some text along with a literal floating-point value. Any literal numeric value with a decimal point in a Python program automatically has the type `float`. This means the Python literal `2.0` is a `float`, not an `int`, even though mathematically we would classify it as an integer.

Floating-point numbers are an approximation of mathematical real numbers. The range of floating-point numbers is limited, since each value requires a fixed amount of memory. Floating-point numbers differ from integers in another, very important way. An integer has an exact representation. This is not true necessarily for a floating-point number. Consider the real number π . The mathematical constant π is an irrational number which means it contains an infinite number of digits with no pattern that repeats. Since π contains an infinite number of digits, a Python program can only approximate π 's value. Because of the limited number of digits available to floating-point numbers, Python cannot represent exactly some rational numbers with a finite number of digits; for example, the number 23.3123400654033989 contains too many digits for the `float` type. As the following interaction sequence shows, Python stores 23.3123400654033989 as 23.312340065403397:

```
>>> x = 23.3123400654033989
>>> x
23.312340065403397
```

An example of the problems that can arise due to the inexact nature of floating-point numbers is demonstrated later in Listing 3.2 (*imprecise.py*).

We can express floating-point numbers in scientific notation. Since most programming editors do not provide superscripting and special symbols like $\times$, Python slightly alters the normal scientific notation. The number 6.022×10^{23} is written `6.022e23`. The number to the left of the `e` (we can use capital `E` as well) is the mantissa, and the number to the right of the `e` is the exponent of 10. As another example, -5.1×10^{-4} is expressed in Python as `-5.1e-4`. Listing 2.8 (*scientificnotation.py*) prints some scientific constants using scientific notation.

Listing 2.8: *scientificnotation.py*

```
avogadros_number = 6.022e23
c = 2.998e8
print(f'Avogadro\'s number = {avogadros_number}')
print(f'Speed of light = {c}')
```

Listing 2.8 (*scientificnotation.py*) prints the following:

```
Avogadro's number = 6.022e+23
Speed of light = 299800000.0
```

Note the character sequence `\'` within the f-string of Listing 2.8 (*scientificnotation.py*). The backslash is known as an *escape* character. This prevents the backslash from closing the string that began with a backslash. Another way to include an apostrophe within a string is to use double quotes (`"`) as the string delimiters instead of the single quotes. We could have written the statement as

```
print(f"Avogadro's number = {avogadros_number}")
```

and avoided the bashslash escape of the apostrophe. Section 2.5 provides more information about escape sequences used within Python strings.

The type of any literal expressed scientific notation always has type `float`; for example, the Python expression `2e3` is a `float`, even though conceptually we may consider it the same as integer the 2,000.

Unlike floating-point numbers, integers are whole numbers and cannot store fractional quantities. We can convert a floating-point to an integer in two fundamentally different ways:

- Rounding adds or subtracts a fractional amount as necessary to produce the integer closest to the original floating-point value.
- Truncation simply drops the fractional part of the floating-point number, thus keeping whole number part that remains.

We can see how rounding and truncation differ in Python's interactive shell:

```
>>> 28.71
28.71
>>> int(28.71)
28
>>> round(28.71)
29
>>> round(19.47)
19
>>> int(19.47)
19
```

As we can see, truncation using the `int` function always “rounds down,” while rounding using the `round` function behaves as we would expect.

We also can use the `round` function to round a floating-point number to a specified number of decimal places. The `round` function accepts an optional argument that produces a floating-point rounded to fewer decimal places. The additional argument must be an integer and specifies the desired number of decimal places to round. In the shell we see

```
>>> x
93.34836
>>> round(x)
93
>>> round(x, 2)
93.35
>>> round(x, 3)
93.348
>>> round(x, 0)
93.0
>>> round(x, 1)
93.3
>>> type(round(x))
<class 'int'>
>>> type(round(x, 1))
<class 'float'>
>>> type(round(x, 0))
<class 'float'>
```

As we can see, the single-argument version of `round` produces an integer result, but the two-argument version produces a floating-point result.

The second argument to the `round` function may be a negative integer:

```
>>> x = 28793.54836
>>> round(x)
28794
>>> round(x, 1)
28793.5
>>> round(x, 2)
28793.55
>>> round(x, 0)
28794.0
>>> round(x, 1)
28793.5
>>> round(x, -1)
28790.0
>>> round(x, -2)
28800.0
>>> round(x, -3)
29000.0
```

The expression `round( $n$ ,  $r$ )` rounds floating-point expression n to the 10^{-r} decimal digit; for example, `round( $n$ , -2)` rounds floating-point value n to the hundreds place (10^2). Similarly, `round( $n$ , 3)` rounds floating-point value n to the thousandths place (10^{-3}).

The `round` function can be useful for integer values as well. If the first argument to `round` is an

integer, and the second argument to `round` is a negative integer, the second argument specifies the number decimal places to the *left* of the decimal point to round. Consider the following experiments:

```
>>> round(65535)
65535
>>> round(65535, 0)
65535
>>> round(65535, 1)
65535
>>> round(65535, 2)
65535
>>> round(65535, -1)
65540
>>> round(65535, -2)
65500
>>> round(65535, -3)
66000
>>> round(65535, -4)
70000
>>> round(65535, -5)
100000
>>> round(65535, -6)
0
```

In all of these cases the `round` function produced an integer result. As you can see, if the second argument is a nonnegative integer, the `round` function evaluates to the original value.

2.5 Control Codes within Strings

The characters that can appear within strings include letters of the alphabet (A-Z, a-z), digits (0-9), punctuation (., :, , , etc.), and other printable symbols (#, &, %, etc.). In addition to these “normal” characters, we may embed special characters known as *control codes*. Control codes control the way the console window or a printer renders text. The backslash symbol (\) signifies that the character that follows it is a control code, not a literal character. The string `'\n'` thus contains a single control code. The backslash is known as the *escape symbol*, and in this case we say the `n` symbol is *escaped*. The `\n` control code represents the *newline* control code which moves the text cursor down to the next line in the console window. Other control codes include `\t` for tab, `\f` for a form feed (or page eject) on a printer, `\b` for backspace, and `\a` for alert (or bell). The `\b` and `\a` do not produce the desired results in the interactive shell, but they work properly in a command shell. Listing 2.9 (`specialchars.py`) prints some strings containing some of these control codes.

Listing 2.9: `specialchars.py`

```
print('A\nB\nC')
print('D\tE\tF')
print('WX\bYZ')
print('1\a2\a3\a4\a5\a6')
```

When executed in a command shell, Listing 2.9 (`specialchars.py`) produces


```
A
B
C
D      E      F
WYZ
123456
```

On many systems, the computer's speaker beeps five times when printing the last line.

A string with a single quotation mark at the beginning must be terminated with a single quote; similarly, A string with a double quotation mark at the beginning must be terminated with a double quote. A single-quote string may have embedded double quotes, and a double-quote string may have embedded single quotes. If you wish to embed a single quote mark within a single-quote string, you can use the backslash to escape the single quote (`\'`). An unprotected single quote mark would terminate the string. Similarly, you may protect a double quote mark in a double-quote string with a backslash (`\"`). Listing 2.10 (`escapequotes.py`) shows the various ways in which quotation marks may be embedded within string literals.

Listing 2.10: `escapequotes.py`

```
print("Did you know that 'word' is a word?")
print('Did you know that "word" is a word?')
print('Did you know that \'word\' is a word?')
print("Did you know that \"word\" is a word?")
```

The output of Listing 2.10 (`escapequotes.py`) is

```
Did you know that 'word' is a word?
Did you know that "word" is a word?
Did you know that 'word' is a word?
Did you know that "word" is a word?
```

Since the backslash serves as the escape symbol, in order to embed a literal backslash within a string you must use two backslashes in succession. Listing 2.11 (`printpath.py`) prints a string with embedded backslashes.

Listing 2.11: `printpath.py`

```
filename = 'C:\\Users\\rick'
print(filename)
```

Listing 2.11 (`printpath.py`) displays

```
C:\Users\rick
```

2.6 Simple User Input

The `print` function enables a Python program to display textual information to the user. Programs may use the `input` function to obtain information from the user. The simplest use of the `input` function assigns a string to a variable:

```
x = input()
```

The parentheses are empty because the `input` function does not require any information to do its job. Listing 2.12 (`usinginput.py`) demonstrates that the `input` function produces a string value.

Listing 2.12: `usinginput.py`

```
print('Please enter some text:')
x = input()
print(f'Text entered: {x}')
print(f'Type: {type(x)}')
```

The following shows a sample run of Listing 2.12 (`usinginput.py`):

```
Please enter some text:
My name is Rick
Text entered: My name is Rick
Type: <class 'str'>
```

The user entered the second line shown in the output, and the program printed the first, third, and fourth lines. After the program prints the message *Please enter some text:*, the program's execution stops and waits for the user to type some text using the keyboard. The user can type, backspace to make changes, and type some more. The text the user types is not committed until the user presses the enter (or return) key.

Quite often we want to perform calculations and need to get numbers from the user. The `input` function produces only strings, but we can use the `int` function to convert a properly formed string of digits into an integer. Listing 2.13 (`addintegers.py`) shows how to obtain an integer from the user.

Listing 2.13: `addintegers.py`

```
print('Please enter an integer value:')
x = input()
print('Please enter another integer value:')
y = input()
num1 = int(x)
num2 = int(y)
print(f'{num1} + {num2} = {num1 + num2}')
```

A sample run of Listing 2.13 (`addintegers.py`) shows

```
Please enter an integer value:
2
Please enter another integer value:
17
2 + 17 = 19
```

Lines two and four represent user input, while the program generates the other lines. The program halts after printing the first line and does not continue until the user provides the input. After the program prints the second message it again pauses to accept the user's second entry.

Since user input almost always requires a message to the user about the expected input, the `input` function optionally accepts a string that it prints just before the program stops to wait for the user to respond. The statement

```
x = input('Please enter some text: ')
```

prints the message *Please enter some text:* and then waits to receive the user's input to assign to `x`. This version of the `input` function has the added benefit of allowing the user to provide input on the same line as the prompting message. We can express Listing 2.13 (`addintegers.py`) more compactly using this form of the `input` function as shown in Listing 2.14 (`addintegers2.py`).

Listing 2.14: `addintegers2.py`

```
x = input('Please enter an integer value: ')
y = input('Please enter another integer value: ')
num1 = int(x)
num2 = int(y)
print(f'{num1} + {num2} = {num1 + num2}')
```

Listing 2.15 (`addintegers3.py`) is even shorter. It combines the `input` and `int` functions into one statement.

Listing 2.15: `addintegers3.py`

```
num1 = int(input('Please enter an integer value: '))
num2 = int(input('Please enter another integer value: '))
print(f'{num1} + {num2} = {num1 + num2}')
```

In Listing 2.15 (`addintegers3.py`) the expression

```
int(input('Please enter an integer value: '))
```

uses a technique known as *functional composition*. The `int` function accepts the result of the `input` directly instead of using the intermediate variables shown in Listing 2.14 (`addintegers2.py`). We frequently will use functional composition to make our program code simpler.

Be careful about code such as

```
num = int(input('Please enter a number: '))
```

This statement expects the user to enter an integer value. If the user types 3, for example, all is well. The variable `num` then will refer to the integer object 3. The `int` function can convert the string `'3'` to the integer value 3. The word *number* is ambiguous, however, so the user might attempt to enter 3.4. In this case the `input` statement would return the string `'3.4'`. The `int` function cannot convert the string `'3.4'` to an integer directly, even though it can convert the floating-point number 3.4 to the integer 3. The following interactive sequence demonstrates:

```
>>> num = int(input('Please enter a number: '))
Please enter a number: 3
>>> num
3
>>> num = int(input('Please enter a number: '))
Please enter a number: 3.4
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
ValueError: invalid literal for int() with base 10: '3.4'
>>> int(3.4)
3
>>> int('3.4')
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
ValueError: invalid literal for int() with base 10: '3.4'
```

This example reveals that the `int` can convert a string to an integer only if the string looks exactly like an integer.

We could be more specific and simply request an integer value from the user. If we really want an integer from the user but want to tolerate decimal places, we could use an additional function in our composition of functions, as in the following:

```
>>> num = int(float(input('Please enter a number: ')))
Please enter a number: 3
>>> num
3
>>> num = int(float(input('Please enter a number: ')))
Please enter a number: 3.4
>>> num
3
```

The assignment statement here uses the `input` function to obtain the number from the user as a string. It then uses the `float` function to convert the received string to a floating-point number. Finally it converts the floating-point number to an integer via the `int` function. What if you wish to round the user's input value instead of truncating it? The following function composition would work in that case:

```
>>> num = round(float(input('Please enter a number: ')))
Please enter a number: 3.7
>>> num
4
```

2.7 Controlling the `print` Function

In Listing 2.13 (`addintegers.py`), we would prefer that the cursor remain at the end of the printed line so when the user types a value it appears on the same line as the message prompting for the values. When the user presses the enter key to complete the input, the cursor automatically will move down to the next line. The `print` function as we have seen so far always prints a line of text, and then the cursor moves down to the next line so any future printing appears on the next line. The `print` statement accepts an additional argument that allows the cursor to remain on the same line as the printed text:

```
print('Please enter an integer value:', end='')
```

The expression `end= ''` is known as a *keyword argument*. The term keyword here means something different from the term *keyword* used to mean a *reserved word*. We defer a complete explanation of keyword arguments until we have explored more of the Python language. For now it is sufficient to know that a `print` function call of this form will cause the cursor to remain on the same line as the printed text. Without this keyword argument, the cursor moves down to the next line after printing the text.

The print statement

```
print('Please enter an integer value: ', end='')
```

means “Print the message *Please enter an integer value:*, and then terminate the line with nothing rather than the normal `\n` newline code.” Another way to achieve the same result is

```
print(end='Please enter an integer value: ')
```

This statement means “Print nothing, and then terminate the line with the string `'Please enter an integer value: '` rather than the normal `\n` newline code. The behavior of the two statements is indistinguishable.

The statement

```
print('Please enter an integer value:')
```

is an abbreviated form of the statement

```
print('Please enter an integer value:', end='\n')
```

that is, the default ending for a line of printed text is the string `'\n'`, the newline control code. Similarly, the statement

```
print()
```

is a shorter way to express

```
print(end='\n')
```

Observe closely the output of Listing 2.16 (`printingexample.py`).

Listing 2.16: `printingexample.py`

```
print('A', end='')
print('B', end='')
print('C', end='')
print()
print('X')
print('Y')
print('Z')
```

Listing 2.16 (`printingexample.py`) displays

```
ABC
X
Y
Z
```

The empty `print` statement

```
print()
```

effectively moves the cursor down to next line.

Sometimes it is convenient to divide the output of a single line of printed text over several Python statements. As an example, we may want to compute part of a complicated calculation, print an intermediate result, finish the calculation, and print the final answer with the output all appearing on one line of text. The `end` keyword argument allows us to do so.

Another keyword argument allows us to control how the `print` function visually separates the arguments it displays. By default, the `print` function places a single space in between the items it prints. `print` uses a keyword argument named `sep` to specify the string to use insert between items. The name `sep` stands for *separator*. The default value of `sep` is the string `' '`, a string containing a single space. Listing 2.17 (`printsep.py`) shows the `sep` keyword customizing `print`'s behavior.

Listing 2.17: `printsep.py`

```
w, x, y, z = 10, 15, 20, 25
print(w, x, y, z)
print(w, x, y, z, sep=',')
print(w, x, y, z, sep='')
print(w, x, y, z, sep=':')
print(w, x, y, z, sep='-----')
```

The output of Listing 2.17 (`printsep.py`) is

```
10 15 20 25
10,15,20,25
10152025
10:15:20:25
10-----15-----20-----25
```

The first of the output shows `print`'s default method of using a single space between printed items. The second output line uses commas as separators. The third line runs the items together with an empty string separator. The fifth line shows that the separating string may consist of multiple characters.

2.8 Number Formatting

Consider Listing 2.18 (`powers10left.py`) which prints the first few powers of 10.

Listing 2.18: `powers10left.py`

```
print(0, 10**0)
print(1, 10**1)
print(2, 10**2)
print(3, 10**3)
print(4, 10**4)
print(5, 10**5)
print(6, 10**6)
print(7, 10**7)
print(8, 10**8)
print(9, 10**9)
print(10, 10**10)
print(11, 10**11)
print(12, 10**12)
print(13, 10**13)
print(14, 10**14)
print(15, 10**15)
```

Listing 2.18 (`powers10left.py`) prints

```
0 1
1 10
2 100
3 1000
4 10000
5 100000
6 1000000
7 10000000
8 100000000
9 1000000000
10 10000000000
11 100000000000
12 1000000000000
13 10000000000000
14 100000000000000
15 1000000000000000
```

Observe that each number is left justified.

Next, consider Listing 2.19 (`powers10right.py`) which again prints the first few powers of 10. Each `print` statement uses an f-string to right justify its number.

Listing 2.19: powers10right.py

```
print(f'{0:2} {10**0:17}')
print(f'{1:2} {10**1:17}')
print(f'{2:2} {10**2:17}')
print(f'{3:2} {10**3:17}')
print(f'{4:2} {10**4:17}')
print(f'{5:2} {10**5:17}')
print(f'{6:2} {10**6:17}')
print(f'{7:2} {10**7:17}')
print(f'{8:2} {10**8:17}')
print(f'{9:2} {10**9:17}')
print(f'{10:2} {10**10:17}')
print(f'{11:2} {10**11:17}')
print(f'{12:2} {10**12:17}')
print(f'{13:2} {10**13:17}')
print(f'{14:2} {10**14:17}')
print(f'{15:2} {10**15:17}')
```

Listing 2.19 (powers10right.py) produces the following output:

```
0          1
1          10
2         100
3        1000
4       10000
5      100000
6     1000000
7    10000000
8   100000000
9  1000000000
10 10000000000
11 100000000000
12 1000000000000
13 10000000000000
14 100000000000000
15 1000000000000000
```

Human readers generally prefer a vertical listing of numbers to be aligned at the right, not the left. The positional parameter `{7:2}` within the f-string means “right-justify the integer 7 within a horizontal space of 2 characters.” The positional parameter `{10**7:17}` within the f-string means “right-justify 10^7 within a space of 17 characters.”

Larger numbers are more difficult to read without digit separators. Listing 2.20 (powers10rightcommas.py) adds commas to separate groups of three digits, as is common in English-speaking countries.

Listing 2.20: powers10rightcommas.py

```
print(f'{0:2} {10**0:22,}')
print(f'{1:2} {10**1:22,}')
print(f'{2:2} {10**2:22,}')
print(f'{3:2} {10**3:22,}')
print(f'{4:2} {10**4:22,}')
print(f'{5:2} {10**5:22,}')
print(f'{6:2} {10**6:22,}')
print(f'{7:2} {10**7:22,}')
print(f'{8:2} {10**8:22,}')
print(f'{9:2} {10**9:22,}')
print(f'{10:2} {10**10:22,}')
print(f'{11:2} {10**11:22,}')
print(f'{12:2} {10**12:22,}')
print(f'{13:2} {10**13:22,}')
print(f'{14:2} {10**14:22,}')
print(f'{15:2} {10**15:22,}')
```

Listing 2.20 (powers10rightcommas.py) produces the following output:

```
0          1
1          10
2          100
3         1,000
4        10,000
5       100,000
6      1,000,000
7     10,000,000
8    100,000,000
9   1,000,000,000
10  10,000,000,000
11 100,000,000,000
12 1,000,000,000,000
13 10,000,000,000,000
14 100,000,000,000,000
15 1,000,000,000,000,000
```

Note that we increased the width of the powers of ten from 17 to 22 to account for the added commas.

Listing 2.21 (alignedfloats.py) shows how to align floating-point numbers by their decimal points.

Listing 2.21: alignedfloats.py

```
print(f'{56.02318:10.3f}')
print(f'{9:10.3f}')
print(f'{3.14159:10.3f}')
print(f'{2.25:10.3f}')
```

Listing 2.21 (alignedfloats.py) displays the numbers 56.02318, 9, 3.14159, and 2.25 as

```
56.023
 9.000
 3.142
 2.250
```

Each number occupies 10 spaces, left to right, and each number displays three digits behind the decimal point. Note the correct rounding of the number 3.14159 to three decimal places.

2.9 Multi-line Strings

A Python string ordinarily spans a single line of text. The following statement is illegal:

```
x = 'This is a long string with  
several words'
```

A string literal that begins with a `'` or `"` must be terminated with its matching `'` or `"` on the same line in which it begins. As we saw in Section 2.5), we can add newline control codes to produce line breaks within the string:

```
x = 'This is a long string with\nseveral words'
```

This technique, however, obscures the programmer's view of the string within the source code. Python provides way to represent a string's layout more naturally within source code, using *triple quotes*. The triple quotes (`'''` or `"""`) delimit strings that can span multiple lines in the source code. Consider Listing 2.22 (multilinestring.py) that uses a multi-line string.

Listing 2.22: multilinestring.py

```
x = '''  
This is a multi-line  
    string that goes on  
for three lines!  
'''  
print(x)
```

Listing 2.22 (multilinestring.py) displays

```
This is a multi-line  
    string that goes on  
for three lines!
```

Observe that the multi-line string obeys indentation and line breaks—essentially reproducing the same formatting as in the source code. For a fancier example, consider the following two-dimensional rendition of a three-dimensional cube, rendered with characters:

Listing 2.23: charactercube.py

```
x = '''
    A cube has 8 corners:

        7-----8
       /|       /|
      3-----4 |
      | |       | |
      | 5-----6
      | /       | /
      1-----2

'''
print(x)
```

Listing 2.23 (charactercube.py) displays

```
A cube has 8 corners:

    7-----8
   /|       /|
  3-----4 |
  | |       | |
  | 5-----6
  | /       | /
  1-----2
```

The “picture” in the source code looks like the picture on the screen.

We will see in Section 7.3 how Python’s multi-line strings play a major role in the convenient formatting of source code documentation.

2.10 Exercises

1. Will the following lines of code print the same thing? Explain why or why not.

```
x = 6
print(6)
print("6")
```

2. Will the following lines of code print the same thing? Explain why or why not.

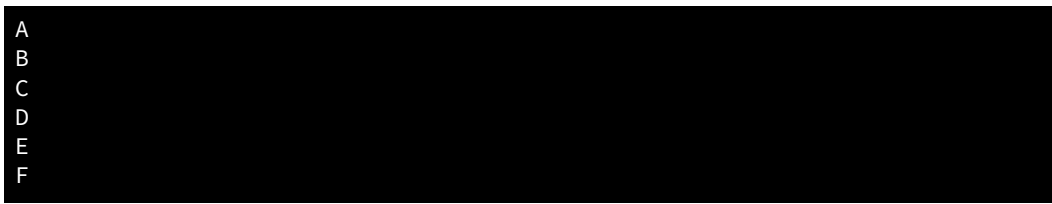
```
x = 7
print(x)
print("x")
```

3. What is the largest floating-point value available on your system?
4. What is the smallest floating-point value available on your system?
5. What happens if you attempt to use a variable within a program, and that variable has not been assigned a value?

6. What is wrong with the following statement that attempts to assign the value ten to variable x?

```
10 = x
```
7. Once a variable has been properly assigned can its value be changed?
8. In Python can you assign more than one variable in a single statement?
9. Classify each of the following as either a *legal* or *illegal* Python identifier:
 - (a) fred
 - (b) `if`
 - (c) 2x
 - (d) -4
 - (e) sum_total
 - (f) sumTotal
 - (g) sum-total
 - (h) sum total
 - (i) sumtotal
 - (j) While
 - (k) x2
 - (l) Private
 - (m) public
 - (n) \$16
 - (o) xTwo
 - (p) _static
 - (q) _4
 - (r) ---
 - (s) 10%
 - (t) a27834
 - (u) wilma's
10. What can you do if a variable name you would like to use is the same as a reserved word?
11. How is the value 2.45×10^{-5} expressed as a Python literal?
12. How can you express the literal value 0.00000000000000000000000000449 as a much more compact Python literal?
13. How can you express the literal value 569923412000000000000000000000000000000000000 as a much more compact Python literal?
14. Can a Python programmer do anything to ensure that a variable's value can never be changed after its initial assignment?
15. Is `"i"` a string literal or variable?
16. What is the difference between the following two strings? `'n'` and `'\n'`?

17. Write a Python program containing exactly one `print` statement that produces the following output:



```
A
B
C
D
E
F
```

18. Write a Python program that simply emits a beep sound when run.

Chapter 3

Expressions and Arithmetic

This chapter uses the Python numeric types introduced in Chapter 2 to build expressions and perform arithmetic. Some other important concepts are covered—more advanced user input, comments, and dealing with errors.

3.1 Expressions

A literal value like 34 and a variable like `x` are examples of simple *expressions*. We can use operators to combine values and variables and form more complex expressions. In Section 2.1 we saw how we can use the `+` operator to add integers and concatenate strings. Listing 3.1 (`add.py`) shows we can use the addition operator (`+`) to add two integers provided by the user.

Listing 3.1: `add.py`

```
value1 = int(input('Please enter a number: '))
value2 = int(input('Please enter another number: '))
sum = value1 + value2
print(f'{value1} + {value2} = {sum}')
```

To review, in Listing 3.1 (`add.py`):

- `value1 = int(input('Please enter a number: '))`

This statement prompts the user to enter some information. After displaying the prompt string *Please enter a number:*, this statement causes the program's execution to stop and wait for the user to type in some text and then press the enter key. The `input` function produces a string, and the `int` function attempts to make an integer value from this string to assign to the variable named `value1`. If the user types the sequence *431* and then presses the enter key, the statement assigns the integer 431 to `value1`.

- `value2 = int(input('Please enter another number: '))`

This statement is similar to the first statement.

- `sum = value1 + value2;`

Table 3.1 Commonly used Python arithmetic binary operators

Expression	Meaning
$x + y$	x added to y , if x and y are numbers x concatenated to y , if x and y are strings
$x - y$	x take away y , if x and y are numbers
$x * y$	x times y , if x and y are numbers x concatenated with itself y times, if x is a string and y is an integer y concatenated with itself x times, if y is a string and x is an integer
x / y	x divided by y , if x and y are numbers
$x // y$	Floor of x divided by y , if x and y are numbers
$x \% y$	Remainder of x divided by y , if x and y are numbers
$x ** y$	x raised to y power, if x and y are numbers

This is an assignment statement because it contains the assignment operator (`=`). The variable `sum` appears to the left of the assignment operator, so `sum` will receive a value when this statement executes. To the right of the assignment operator is an arithmetic expression involving two variables and the addition operator. The Python interpreter *evaluates* the expression by adding together the values bound to the two variables. Once the addition expression's value is determined, the assignment operator binds its value to the `sum` variable.

- `print(f'{value1} + {value2} = {sum}')`

This statement prints the values of the three variables with some additional decoration to make the output clear about what it is showing.

All expressions have a value. The process of determining the expression's value is called *evaluation*. Evaluating simple expressions is easy. The literal value 54 evaluates to 54. The value of a variable named `x` is the value stored in the memory location bound to `x`. The value of a more complex expression is found by evaluating the smaller expressions that make it up and combining them with operators to form potentially new values.

Table 3.1 contains the most commonly used Python arithmetic operators. The common arithmetic operations, addition, subtraction, multiplication, division, and power behave in the expected way. The `//` and `%` operators are not common arithmetic operators in everyday practice, but they are very useful in programming. The `//` operator is called *integer division*, and the `%` operator is the *modulus* or *remainder* operator. `25/3` is 8.3333. Three does not divide into 25 evenly. In fact, three goes into 25 eight times with a remainder of one. Here, eight is the quotient, and one is the remainder. `25//3` is 8 (the quotient), and `25%3` is 1 (the remainder).

All these operators are classified as *binary* operators because they operate on two operands. In the statement

```
x = y + z
```

on the right side of the assignment operator is the addition expression `y + z`. The two operands of the `+` operator are `y` and `z`.

Two operators, `+` and `-`, can be used as *unary* operators. A unary operator has only one operand. The `-` unary operator expects a single numeric expression (literal number, variable, or more complicated numeric expression within parentheses) immediately to its right; it computes the *additive inverse* of its operand. If the operand is positive (greater than zero), the result is a negative value of the same magnitude; if the

operand is negative (less than zero), the result is a positive value of the same magnitude. Zero is unaffected. For example, the following code sequence

```
x, y, z = 3, -4, 0
x = -x
y = -y
z = -z
print(x, y, z)
```

within a program would print

```
-3 4 0
```

The following statement

```
print(-(4 - 5))
```

within a program would print

```
1
```

The unary `+` operator is present only for completeness; when applied to a numeric value, variable, or expression, the resulting value is no different from the original value of its operand. Omitting the unary `+` operator from the following statement

```
x = +y
```

does not change its behavior.

All the arithmetic operators are subject to the limitations of the data types on which they operate; for example, consider the following interaction sequence:

```
>>> 2.0**10
1024.0
>>> 2.0**100
1.2676506002282294e+30
>>> 2.0**1000
1.0715086071862673e+301
>>> 2.0**10000
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
OverflowError: (34, 'Result too large')
```

The expression `2.0**10000` will not evaluate to the correct answer since the correct answer falls outside the range of Python's floating point values.

When we apply the `+`, `-`, `*`, `//`, `%`, or `**` operators to two integers, the result is an integer. The statement `print(25//4, 4//25)`

prints

```
6 0
```

The `//` operator produces an integer result when used with integers. In the first case above 25 divided by 4 is 6 with a remainder of 1, and in the second case 4 divided by 25 is 0 with a remainder of 4. Since integers

Figure 3.1 Integer division and modulus

$$\begin{array}{r} 6 \\ 4 \overline{) 25} \\ \underline{-24} \\ 1 \end{array}$$

$25 // 4$

$25 \% 4$

are whole numbers, the `//` operator discards any fractional part of the answer. The process of discarding the fractional part of a number leaving only the whole number part is called *truncation*. Truncation is not rounding; for example, 13 divided by 5 is 2.6, but 2.6 truncates to 2.



Truncation simply removes any fractional part of the value. It does not round. Both 10.01 and 10.999 truncate to 10.

The modulus operator (%) computes the remainder of integer division; thus,

```
print(25%4, 4%25)
```

prints

```
1 4
```

since 25 divided by 4 is 6 with a remainder of 1, and 4 divided by 25 is 0 with a remainder of 4. Figure 3.1 shows the relationship between integer division and modulus.

The modulus operator is more useful than it may first appear. Listing 3.9 (`timeconv.py`) shows how it can be used to convert a given number of seconds to hours, minutes, and seconds.

The `/` operator applied to two integers produces a floating-point result. The statement

```
print(25/4, 4/25)
```

prints

```
6.25 0.16
```

These results are what we would expect from a hand-held calculator. Floating-point arithmetic always produces a floating-point result.

Recall from Section 2.4 that Python integers have an exact representation, but floating-point numbers are imprecise approximations of real numbers. Listing 3.2 (`imprecise.py`) clearly demonstrates the weakness of floating point numbers.

Listing 3.2: imprecise.py

```

one = 1.0
one_third = 1.0/3.0
zero = one - one_third - one_third - one_third

print('one =', one, ' one_third =', one_third, ' zero =', zero)

```

```

one = 1.0  one_third = 0.3333333333333333  zero = 1.1102230246251565e-16

```

The reported result is $1.1102230246251565 \times 10^{-16}$, or 0.00000000000000011102230246251565. While this number is very small, with real numbers we get

$$1 - \frac{1}{3} - \frac{1}{3} - \frac{1}{3} = 0$$

Floating-point numbers are not real numbers, so the result of `1.0/3.0` cannot be represented exactly without infinite precision. In the decimal (base 10) number system, one-third is a repeating fraction, so it has an infinite number of digits. Even simple nonrepeating decimal numbers can be a problem. One-tenth (0.1) is obviously nonrepeating, so we can express it exactly with a finite number of digits. As it turns out, since numbers within computers are stored in binary (base 2) form, even one-tenth cannot be represented exactly with floating-point numbers, as Listing 3.3 (`imprecise10.py`) illustrates.

Listing 3.3: imprecise10.py

```

one = 1.0
one_tenth = 1.0/10.0
zero = one - one_tenth - one_tenth - one_tenth \
        - one_tenth - one_tenth - one_tenth \
        - one_tenth - one_tenth - one_tenth \
        - one_tenth

print('one =', one, ' one_tenth =', one_tenth, ' zero =', zero)

```

The program's output is

```

one = 1.0  one_tenth = 0.1  zero = 1.3877787807814457e-16

```

Surely the reported answer ($1.3877787807814457 \times 10^{-16}$) is close to the correct answer (zero). If you round our answer to the one-hundred trillionth place (15 places behind the decimal point), it is correct.

In Listing 3.3 (`imprecise10.py`) lines 3–6 make up a single Python statement. If that single statement that performs nine subtractions were written on one line, it would flow well off the page or off the editing window. Ordinarily a Python statement ends at the end of the source code line. A programmer may break up a very long line over two or more lines by using the backslash (`\`) symbol at the end of an incomplete line. When the interpreter is processing a line that ends with a `\`, it automatically joins the line that follows. The interpreter thus sees a very long but complete Python statement.

The Python interpreter also automatically joins long statements spread over multiple lines in the source code if it detects an opening parenthesis `(`, square bracket `[`, or curly brace `{` that is unmatched by its corresponding closing symbol. The following is a legal Python statement spread over two lines in the source code:

```
x = (int(input('Please enter an integer'))
    + (y - 2) + 16) * 2
```

The last closing parenthesis on the second line matches the first opening parenthesis on the first line. No backslash symbol is required at the end of the first line. The interpreter will begin scanning the first line, matching closing parentheses with opening parentheses. When it gets to the end of the line and has not detected a closing parenthesis to match an earlier opening parenthesis, the interpreter assumes it must appear on a subsequent line, and so continues scanning until it completes the long statement. If the interpreter does not find expected closing parenthesis in a program, it issues an error. In the Python interactive shell, the interpreter keeps waiting until the user complies or otherwise types something that causes an error:

```
>>> y = 10
>>> x = (int(input('Please enter an integer: '))
... + (y - 2) + 16
...
...
...
... ) * 2
Please enter an integer: 3
>>> x
54
```

Since computers represent floating-point values internally in binary form, if we choose a binary fractional power, the mathematics will work out precisely. Python can represent the fraction $\frac{1}{4} = 0.25 = 2^{-2}$ exactly. Listing 3.4 (precise4.py) illustrates.

Listing 3.4: precise4.py

```
one = 1.0
one_fourth = 1.0/4.0
zero = one - one_fourth - one_fourth - one_fourth - one_fourth
print('one =', one, ' one-fourth =', one_fourth, ' zero =', zero)
```

Listing 3.4 (precise4.py) behaves much better than the previous examples:

```
one = 1.0    one-fourth = 0.25    zero = 0.0
```

The computed value for the variable `zero` actually is zero.

When should you use integers and when should you use floating-point numbers? A good rule of thumb is this: use integers to count things and use floating-point numbers for quantities obtained from a measuring device. As examples, we can measure length with a ruler or a laser range finder; we can measure volume with a graduated cylinder or a flow meter; we can measure mass with a spring scale or triple-beam balance. In all of these cases, the accuracy of the measured quantity is limited by the accuracy of the measuring device and the competence of the person or system performing the measurement. Environmental factors such as temperature or air density can affect some measurements. In general, the degree of inexactness of such measured quantities is far greater than that of the floating-point values that represent them.

Despite their inexactness, floating-point numbers are used every day throughout the world to solve sophisticated scientific and engineering problems. The limitations of floating-point numbers are unavoidable.

able since values with infinite characteristics cannot be represented in a finite way. Floating-point numbers provide a good trade-off of precision for practicality.

3.2 Mixed Type Expressions

Expressions may contain mixed integer and floating-point elements; for example, in the following program fragment

```
x = 4
y = 10.2
sum = x + y
```

x is an integer and y is a floating-point number. What type is the expression $x + y$? Except in the case of the $/$ operator, arithmetic expressions that involve only integers produce an integer result. All arithmetic operators applied to floating-point numbers produce a floating-point result. When an operator has mixed operands—one operand an integer and the other a floating-point number—the interpreter treats the integer operand as floating-point number and performs floating-point arithmetic. This means $x + y$ is a floating-point expression, and the assignment will make the variable `sum` bind to a floating-point value.

3.3 Operator Precedence and Associativity

When different operators appear in the same expression, the normal rules of arithmetic apply. All Python operators have a *precedence* and *associativity*:

- **Precedence**—when an expression contains two different kinds of operators, which one applies first?
- **Associativity**—when an expression contains two operators with the same precedence, which one applies first?

To see how precedence works, consider the expression

```
2 + 3 * 4
```

Should it be interpreted as

```
(2 + 3) * 4
```

(that is, 20), or rather is

```
2 + (3 * 4)
```

(that is, 14) the correct interpretation? As in normal arithmetic, multiplication and division in Python have equal importance and are performed before addition and subtraction. We say multiplication and division have *precedence* over addition and subtraction. In the following expression:

```
2 + 3 * 4
```

the Python interpreter computes the multiplication result before performing the addition, since multiplication has precedence over addition. The result is 14. The multiplicative operators ($*$, $/$, $//$, and $\%$) have equal precedence with each other, and the additive operators (binary $+$ and $-$) have equal precedence with each other. Every multiplicative operator has precedence over any additive operators.

Table 3.2 Operator precedence and associativity. The operators in each row have a higher precedence than the operators below it. Operators within a row have the same precedence.

Arity	Operators	Associativity
Binary	<code>**</code>	Right
Unary	<code>+</code> , <code>-</code>	
Binary	<code>*</code> , <code>/</code> , <code>//</code> , <code>%</code>	Left
Binary	<code>+</code> , <code>-</code>	Left

As in standard arithmetic, a Python programmer can use parentheses to override the precedence rules and force addition to be performed before multiplication. The expression

```
(2 + 3) * 4
```

evaluates to 20. The parentheses in a Python arithmetic expression may be arranged and nested in any ways that are acceptable in standard arithmetic.

To see how associativity works, consider the expression

```
2 - 3 - 4
```

The two operators are the same, so they have equal precedence. Should the first subtraction operator be applied before the second, as in

```
(2 - 3) - 4
```

(that is, $(-1) - 4 = -5$), or rather is

```
2 - (3 - 4)
```

(that is, $2 - (-1) = 2 + 1 = 3$) the correct interpretation? The former (-5) is the correct interpretation. We say that the subtraction operator is *left associative*, and its evaluation is left to right. This interpretation agrees with standard arithmetic rules. All binary operators except assignment and exponentiation are left associative.

As in the case of precedence, we can use parentheses to override the natural associativity within an expression.

Table 3.2 shows the precedence and associativity rules for some Python operators.

The assignment operator is a different kind of operator from the arithmetic operators. Programmers use the assignment operator only to build assignment *statements*. Python does not allow the assignment operator to be part of a larger expression or part of another statement. As such, the notions of precedence and associativity do not apply in the context of the assignment operator. Python does, however, support a special kind of assignment statement called *chained assignment*. The code

```
w = x = y = z
```

assigns the value of the rightmost variable (in this case `z`) to all the other variables (`w`, `x`, and `y`) to its left. To initialize several variables to zero in one statement, you can write

```
sum = count = 0
```

which is slightly shorter than tuple assignment:

```
sum, count = 0, 0
```

3.4 Formatting Expressions

Python offers considerable flexibility for formatting arithmetic expressions within source code; for example, suppose a program is using the variables x and y to hold data, and we wish to use those variables within a mathematical expression like the following from algebra:

$$3x + 2y - 5$$

Unlike algebra, Python has no implicit multiplication. This means we must write $3x$ as $3*x$. We may not omit the $*$ operator.

We can print the value of the complete expression above as follows:

```
print(3*x + 2*y - 5)
```

This formatting is preferred, as it follows closely the style used in mathematics. The following is equally acceptable to the Python interpreter:

```
print(3*x+2*y-5)
```

Note the lack of spaces. Many agree that this formatting, while legal Python, is less readable by humans. Since people develop and read source code, human readability is important. Some suggest that spaces be used around *all* binary operators, as in the following:

```
print(3 * x + 2 * y - 5)
```

In this case such a practice reduces the readability, since it make it appear as the operators have equal precedence (which they do not—spacing in an expression does not influence operator precedence). The first way we wrote the expression naturally grouped together the factors within a term, just as is common in algebra. The following is computationally equivalent in Python but misleading and should be avoided:

```
print(3 * x+2 * y-5)
```

Again, spacing does not affect operator precedence, but this formatting makes it appear that the addition and the subtraction will take place *before* the multiplication. Psychologically, the lack of space makes it appear that $+$ and $-$ “bind” their operands more tightly than does $*$. This is not true, but a human reader may easily misinterpret the expression formatted this way.

The bottom line is that you should strive to format your Python source code in a way that enhances human readability. Why is this important? Teams of programmers develop commercial software. They must be able to review and revise code written by others. Any coding techniques that make it easier for people to read and understand each other’s code greatly facilitates the development process.

If you really need to add and subtract in the expression above before performing the multiplications, you can use parentheses to override the normal precedence rules, as in:

```
print(3 * (x + 2) * (y - 5))
```

The following formatting:

```
print(3*(x + 2)*(y - 5))
```

better presents the expression as it you would find it in a algebra book.

3.5 Comments

Good programmers annotate their code by inserting remarks that explain the purpose of a section of code or why they chose to write a section of code the way they did. These notes are meant for human readers, not the interpreter. It is common in industry for programs to be reviewed for correctness by other programmers or technical managers. Well-chosen identifiers (see Section 2.3) and comments can aid this assessment process. Also, in practice, teams of programmers develop software. A different programmer may be required to finish or fix a part of the program written by someone else. Well-written comments can help others understand new code quicker and increase their productivity modifying old or unfinished code. While it may seem difficult to believe, even the same programmer working on her own code months later can have a difficult time remembering what various parts do. Comments can help greatly.

Any text contained within comments is ignored by the Python interpreter. The `#` symbol begins a comment in the source code. The comment is in effect until the end of the line of code:

```
# Compute the average of the values
avg = sum / number
```

The first line here is a comment that explains what the statement that follows it is supposed to do. The comment begins with the `#` symbol and continues until the end of that line. The interpreter will ignore the `#` symbol and the contents of the rest of the line. You also may append a short comment to the end of a statement:

```
avg = sum / number # Compute the average of the values
```

Here, an executable statement and the comment appear on the same line. The interpreter will read the assignment statement, but it will ignore the comment.

How are comments best used? Avoid making a remark about the obvious; for example:

```
result = 0 # Assign the value zero to the variable named result
```

The effect of this statement is clear to anyone with even minimal Python programming experience. A comment should add something of value to the reader; generally, “routine” activities require no remarks. Even though the *effect* of the above statement is clear, its *purpose* may need a comment. For example:

```
result = 0 # Ensure 'result' has a well-defined minimum value
```

This remark may be crucial for readers to completely understand how a particular part of the program works. In general, programmers are not prone to providing too many comments. When in doubt, add a remark. The extra time it takes to write good comments is well worth the effort.

3.6 Errors

Beginning programmers make mistakes writing programs because of inexperience in programming in general or due to unfamiliarity with a programming language. Seasoned programmers make mistakes due to carelessness or because the proposed solution to a problem is faulty and the correct implementation of an incorrect solution will not produce a correct program.

In Python, there are three general kinds of errors: syntax errors, run-time exceptions, and logic errors.

3.6.1 Syntax Errors

The interpreter is designed to execute all valid Python programs. The interpreter reads the Python source file and translates it into an executable form. This is the *translation phase*. If the interpreter detects an invalid program statement during the translation phase, it will terminate the program's execution and report an error. Such errors result from the programmer's misuse of the language. A *syntax error* is a common error that the interpreter can detect when attempting to translate a Python statement into machine language. For example, in English one can say

The boy walks quickly.

This sentence uses correct syntax. However, the sentence

The boy walk quickly.

is not correct syntactically: the number of the subject (singular form) disagrees with the number of the verb (plural form). It contains a syntax error. It violates a grammatical rule of the English language. Similarly, the Python statement

```
x = y + 2
```

is syntactically correct because it obeys the rules for the structure of an assignment statement described in Section 2.2. However, consider replacing this assignment statement with a slightly modified version:

```
y + 2 = x
```

If a statement like this one appears in a program, the interpreter will issue an error message; Listing 3.5 (error.py) attempts such an assignment.

Listing 3.5: error.py

```
y = 5
x = y + 2
y + 2 = x
```

When running Listing 3.5 (error.py) the interpreter displays

```
File "C:\Code\python\error.py", line 3
  y + 2 = x
  ^^^^^
SyntaxError: cannot assign to expression here. Maybe you meant '==' instead of
```

The syntax of Python does not allow an expression like `y + 2` to appear on the left side of the assignment operator.

Other common syntax errors arise from simple typographical errors like mismatched parentheses

```
>>> x = )3 + 4)
      File "<stdin>", line 1
        x = )3 + 4)
              ^
SyntaxError: unmatched ')'
```

or mismatched string quotes

```
>>> x = 'hello"
      File "<stdin>", line 1
        x = 'hello"
              ^
SyntaxError: unterminated string literal (detected at line 1)
```

or faulty indentation.

```
>>> x = 2
>>> y = 5
      File "<stdin>", line 1
        y = 5
          ^
IndentationError: unexpected indent
```

These examples illustrate just a few of the ways programmers can write ill-formed code.

The interpreter detects syntax errors before it begins running the program, and so it will not execute any parts of a program that contains syntax errors.

3.6.2 Run-time Exceptions

A syntactically correct Python program still can have problems. Some language errors depend on the context of the program's execution. Such errors are called *run-time exceptions* or *run-time errors*. We say the interpreter *raises* an exception. Run-time exceptions arise after the interpreter's translation phase and during the program's execution phase.

The interpreter may issue an exception for a syntactically correct statement like

```
x = y + 2
```

if the variable `y` has yet to be assigned; for example, if the statement appears at line 12 and by that point `y` has not been assigned, we are informed:

```
>>> x = y + 2
Traceback (most recent call last):
  File "error.py", line 12, in <module>
NameError: name 'y' is not defined
```

Consider Listing 3.6 (dividedanger.py) which contains an error that manifests itself only in certain situations.

Listing 3.6: dividedanger.py

```
# File dividedanger.py

# Get two integers from the user
print('Please enter two numbers to divide.')
dividend = int(input('Please enter the dividend: '))
divisor = int(input('Please enter the divisor: '))
# Divide them and report the result
print(dividend, '/', divisor, "=", dividend/divisor)
```

The expression
dividend/divisor

is potentially dangerous. If the user enters, for example, 32 and 4, the program works nicely

```
Please enter two numbers to divide.
Please enter the dividend: 32
Please enter the divisor: 4
32 / 4 = 8.0
```

If the user instead types the numbers 32 and 0, the program reports an error and terminates:

```
Please enter two numbers to divide.
Please enter the dividend: 32
Please enter the divisor: 0
Traceback (most recent call last):
  File "C:\Code\python\dividedanger.py", line 8, in <module>
    print(dividend, '/', divisor, "=", dividend/divisor)
                                ~~~~~^~~~~~
ZeroDivisionError: division by zero
```

Division by zero is undefined in mathematics, and division by zero in Python is illegal.

As another example, consider Listing 3.7 (halve.py).

Listing 3.7: halve.py

```
# Get a number from the user
value = int(input('Please enter a number to cut in half: '))
# Report the result
print(value/2)
```

Some sample runs of Listing 3.7 (halve.py) reveal

```
Please enter a number to cut in half: 100
50.0
```

and

```
Please enter a number to cut in half: 19.41
9.705
```


The interpreter is of no help at all with logic errors. Such bugs are the major source of frustration for developers. The frustration often arises because in complex programs the bugs sometimes reveal themselves only in certain situations that are difficult to reproduce exactly during testing. You will discover this frustration as your programs become more complicated. The good news is that programming experience and the disciplined application of good programming techniques can help reduce the number of logic errors. The bad news is that since software development is an inherently human intellectual pursuit, logic errors are inevitable. Accidentally introducing and later finding and eliminating logic errors is an integral part of the programming process.

3.7 Arithmetic Examples

Suppose we wish to convert temperature from degrees Fahrenheit to degrees Celsius. The following formula provides the necessary mathematics:

$$^{\circ}\text{C} = \frac{5}{9} \times (^{\circ}\text{F} - 32)$$

Listing 3.8 (tempconv.py) implements the conversion in Python.

Listing 3.8: tempconv.py

```
# File tempconv.py
# Author: Rick Halterman
# Last modified: January 10, 2024
# Converts degrees Fahrenheit to degrees Celsius
# Based on the formula found at
# http://en.wikipedia.org/wiki/Conversion_of_units_of_temperature
# Prompt user for temperature to convert and read the supplied value
degreesF = float(input('Enter the temperature in degrees F: '))
# Perform the conversion
degreesC = 5/9*(degreesF - 32)
# Report the result
print(f'{degreesF} degrees F = {degreesC} degrees C')
```

Listing 3.8 (tempconv.py) contains comments that give an overview of the program's purpose and provide some details about its construction. Comments also document each step explaining the code's logic. Some sample runs show how the program behaves:

```
Enter the temperature in degrees F: 212
212 degrees F = 100.0 degrees C
```

```
Enter the temperature in degrees F: 32
32 degrees F = 0.0 degrees C
```

```
Enter the temperature in degrees F: -40
-40 degrees F = -40.0 degrees C
```

Listing 3.9 (timeconv.py) uses integer division and modulus to split up a given number of seconds to hours, minutes, and seconds.

Listing 3.9: timeconv.py

```
# File timeconv.py

# Get the number of seconds
seconds = int(input('Please enter the number of seconds:'))
# First, compute the number of hours in the given number of seconds
# Note: integer division with possible truncation
hours = seconds // 3600 # 3600 seconds = 1 hour
# Compute the remaining seconds after the hours are accounted for
seconds = seconds % 3600
# Next, compute the number of minutes in the remaining number of seconds
minutes = seconds // 60 # 60 seconds = 1 minute
# Compute the remaining seconds after the minutes are accounted for
seconds = seconds % 60
# Report the results
print(f'{hours}:{minutes:0{2}}:{seconds:0{2}}')
```

The text `{minutes:0{2}}` means render the value of the `minutes` variable in two spaces and use leading zeros if necessary.

If the user enters 10000, the program prints 2:46:40. Notice the assignments to the `seconds` variable, such as

```
seconds = seconds % 3600
```

The right side of the assignment operator (`=`) is first evaluated. The statement assigns back to the `seconds` variable the remainder of `seconds` divided by 3,600. This statement can alter the value of `seconds` if the current value of `seconds` is greater than 3,600. A similar statement that occurs frequently in programs is one like

```
x = x + 1
```

This statement increments the variable `x` to make it one bigger. A statement like this one provides further evidence that the Python assignment operator does not mean mathematical equality. The following statement from mathematics

$$x = x + 1$$

surely is never true; a number cannot be equal to one more than itself. If that were the case, I would deposit one dollar in the bank and then insist that I really had two dollars in the bank, since a number is equal to one more than itself. That two dollars would become \$3.00, then \$4.00, etc., and soon I would be rich. In Python, however, this statement simply means “add one to `x`’s current value and update `x` with the result.”

3.8 More Arithmetic Operators

As Listing 3.9 (timeconv.py) demonstrates, an executing program can alter a variable’s value by performing some arithmetic on its current value. A variable may increase by one or decrease by five. The statement

```
x = x + 1
```

increments `x` by one, making it one bigger than it was before this statement was executed. Python has a shorter statement that accomplishes the same effect:

```
x += 1
```

This is the *increment* statement. A similar *decrement* statement is available:

```
x -= 1      # Same as x = x - 1
```

Python provides a more general way of simplifying a statement that modifies a variable through simple arithmetic. For example, the statement

```
x = x + 5
```

can be shorted to

```
x += 5
```

This statement means “increase `x` by five.” Any statement of the form

$$x \text{ op} = \text{exp}$$

where

- `x` is a variable.
- `op=` is an arithmetic operator combined with the assignment operator; for our purposes, the ones most useful to us are `+=`, `-=`, `*=`, `/=`, `//=`, and `%=`.
- `exp` is an expression compatible with the variable `x`.

Arithmetic reassignment statements of this form are equivalent to

$$x = x \text{ op } \text{exp}$$

This means the statement

```
x *= y + z
```

is equivalent to

```
x = x * (y + z)
```

The version using the arithmetic assignment does not require parentheses. The arithmetic assignment is especially handy if we need to modify a variable with a long name; consider

```
temporary_filename_length = temporary_filename_length / (y + z)
```

versus

```
temporary_filename_length /= y + z
```

Do not accidentally reverse the order of the symbols for the arithmetic assignment operators, like in the statement

```
x =+ 5
```

Notice that the + and = symbols have been reversed. The compiler interprets this statement as if it had been written

```
x = +5
```

that is, assignment and the unary operator. This assigns exactly five to x instead of increasing it by five.

Similarly,

```
x =- 3
```

would assign -3 to x instead of decreasing x by three.

3.9 Algorithms

Have you ever tried to explain to someone how to perform a reasonably complex task? The task could involve how to make a loaf of bread from scratch, how to get to the zoo from city hall, or how to factor an algebraic expression. Were you able to explain all the steps perfectly without omitting any important details critical to the task's solution? Were you frustrated because the person wanting to perform the task obviously was misunderstanding some of the steps in the process, and you believed you were making everything perfectly clear? Have you ever attempted to follow a recipe for your favorite dish only to discover that some of the instructions were unclear or ambiguous? Have you ever faithfully followed the travel directions provided by a friend and, in the end, found yourself nowhere near the intended destination?

Often it is easy to envision the steps to complete a task but hard to communicate precisely to someone else how to perform those steps. We may have completed the task many times, or we even may be an expert on completing the task. The problem is that someone who has never completed the task requires exact, detailed, unambiguous, and complete instructions to complete the task successfully.

Because many real-world tasks involve a number of factors, people sometimes get lucky and can complete a complex task given less-than-perfect instructions. A person often can use experience and common sense to handle ambiguous or incomplete instructions. In fact, humans are so good at dealing with “fuzzy” knowledge that in most instances the effort to produce excruciatingly detailed instructions to complete a task is not worth the effort.

When a computer executes the instructions found in software, it has no cumulative experience and no common sense. It dutifully executes the instructions it receives. While executing a program, a computer cannot fill in the gaps in instructions that a human naturally might be able to do. Further, unlike with humans, executing the same program over and over does not improve the computer's ability to perform the task. The computer has no *understanding*.

An *algorithm* is a finite sequence of steps, with each step taking a finite length of time, that solves a problem or computes a result. A computer program is one example of an algorithm, as is a recipe to make lasagna. In both of these examples, the order of the steps matter. In the case of lasagna, we must cook the noodles in boiling water before layered them into the filling to be baked. It would be inappropriate to place the raw noodles into the pan with all the other ingredients, bake it, and then later remove the already baked noodles to cook them in boiling water separately. In the same way, the ordering of steps is very important in a computer program. While this point may be obvious, consider the following sound argument:

1. The following formula expresses the relationship between degrees Celsius and degrees Fahrenheit:

$$^{\circ}\text{C} = \frac{5}{9} \times (^{\circ}\text{F} - 32)$$

2. Given a temperature in degrees Fahrenheit, we can compute the corresponding temperature in degrees Celsius.

Armed with this knowledge, Listing 3.10 (faultytempconv.py) follows directly.

Listing 3.10: faultytempconv.py

```
# File faultytempconv.py

# Establish some variables
degreesF, degreesC = 0, 0
# Define the relationship between F and C
degreesC = 5/9*(degreesF - 32)
# Prompt user for degrees F
degreesF = float(input('Enter the temperature in degrees F: '))
# Report the result
print(degreesF, 'degrees F =', degreesC, 'degrees C')
```

Unfortunately, when run the program always displays

```
-17.7778
```

regardless of the input provided. The English description provided above is correct. The formula is implemented faithfully. The problem lies simply in statement ordering. The statement

```
degreesC = 5/9*(degreesF - 32)
```

is an *assignment* statement, not a definition of a relationship that exists throughout the program. At the point of the assignment, `degreesF` has the value of zero. The program assigns variable `degreesC` *before* the user provides `degreesF`'s value.

As another example, suppose `x` and `y` are two variables in some program. How would we interchange the values of the two variables? We want `x` to have `y`'s original value and `y` to have `x`'s original value. This code may seem reasonable:

```
x = y
y = x
```

The problem with this section of code is that after the first statement executes, `x` and `y` both have the same value (`y`'s original value). The second assignment is superfluous and does nothing to change the values of `x` or `y`. The solution requires a third variable to remember the original value of one of the variables before reassigning it a new value. The correct code to swap the values is

```
temp = x
x = y
y = temp
```

We can use tuple assignment (see Section 2.2) to make the swap even simpler:

```
x, y = y, x
```

These small examples emphasize the fact that we must specify algorithms precisely. Informal notions about how to solve a problem can be valuable in the early stages of program design, but the coded program requires a correct detailed description of the solution.

The algorithms we have seen so far have been simple. Statement 1, followed by Statement 2, etc. until every statement in the program has been executed. Chapters 4 and 5 introduce some language constructs that permit optional and repetitive execution of some statements. These constructs allow us to build programs that do much more interesting things, but the algorithms that take advantage of them are more complex. We must not lose sight of the fact that a complicated algorithm that is 99% correct is *not* correct. Inattention to the smallest detail can derail any algorithm, however well designed and implemented.

3.10 Exercises

1. Is the literal 4 a valid Python expression?
2. Is the variable `x` a valid Python expression?
3. Is `x + 4` a valid Python expression?
4. What affect does the unary `+` operator have when applied to a numeric expression?
5. Sort the following binary operators in order of high to low precedence: `+`, `-`, `*`, `//`, `/`, `%`, `=`.
6. Given the following assignment:

```
x = 2
```

Indicate what each of the following Python statements would print.

- (a) `print("x")`
- (b) `print('x')`
- (c) `print(x)`
- (d) `print("x + 1")`
- (e) `print('x' + 1)`
- (f) `print(x + 1)`

7. Given the following assignments:

```
i1 = 2
i2 = 5
i3 = -3
d1 = 2.0
d2 = 5.0
d3 = -0.5
```

Evaluate each of the following Python expressions.

- (a) `i1 + i2`
- (b) `i1 / i2`
- (c) `i1 // i2`
- (d) `i2 / i1`
- (e) `i2 // i1`
- (f) `i1 * i3`
- (g) `d1 + d2`

- (h) $d1 / d2$
- (i) $d2 / d1$
- (j) $d3 * d1$
- (k) $d1 + i2$
- (l) $i1 / d2$
- (m) $d2 / i1$
- (n) $i2 / d1$
- (o) $i1/i2*d1$
- (p) $d1*i1/i2$
- (q) $d1/d2*i1$
- (r) $i1*d1/d2$
- (s) $i2/i1*d1$
- (t) $d1*i2/i1$
- (u) $d2/d1*i1$
- (v) $i1*d2/d1$

8. What is printed by the following statement:

```
#print(5/3)
```

9. Given the following assignments:

```
i1 = 2
i2 = 5
i3 = -3
d1 = 2.0
d2 = 5.0
d3 = -0.5
```

Evaluate each of the following Python expressions.

- (a) $i1 + (i2 * i3)$
- (b) $i1 * (i2 + i3)$
- (c) $i1 / (i2 + i3)$
- (d) $i1 // (i2 + i3)$
- (e) $i1 / i2 + i3$
- (f) $i1 // i2 + i3$
- (g) $3 + 4 + 5 / 3$
- (h) $3 + 4 + 5 // 3$
- (i) $(3 + 4 + 5) / 3$
- (j) $(3 + 4 + 5) // 3$
- (k) $d1 + (d2 * d3)$
- (l) $d1 + d2 * d3$
- (m) $d1 / d2 - d3$

- (n) $d1 / (d2 - d3)$
- (o) $d1 + d2 + d3 / 3$
- (p) $(d1 + d2 + d3) / 3$
- (q) $d1 + d2 + (d3 / 3)$
- (r) $3 * (d1 + d2) * (d1 - d3)$

10. What symbol signifies the beginning of a comment in Python?
11. How do Python comments end?
12. Which is better, too many comments or too few comments?
13. What is the purpose of comments?
14. Why is human readability such an important consideration?
15. What circumstances can cause each of the following run-time errors to arise?

- NameError
- ValueError
- ZeroDivisionError
- IndentationError
- OverflowError
- SyntaxError
- TypeError

Hint: Try some of following activities in the interpreter or within a Python program:

- print a variable that has not been assigned
- convert the string `'two'` to an integer
- add an integer to a string
- assign to a variable named `end-point`
- experiment adding spaces and tabs at various places in the code of an error-free Python program
- compute raise a floating-point number to a large power, as in $1.5^{10,000}$.

16. Consider the following program which contains some errors. You may assume that the comments within the program accurately describe the program's intended behavior.

```
# Get two numbers from the user
n1 = float(input())           # 1
n2 = float(input())           # 2
# Compute sum of the two numbers
print(n1 + n2)                 # 3
# Compute average of the two numbers
print(n1+n2/2)                 # 4
# Assign some variables
d1 = d2 = 0                     # 5
# Compute a quotient
print(n1/d1)                    # 6
# Compute a product
n1*n2 = d1                      # 7
# Print result
print(d1)                       # 8
```

For each line listed in the comments, indicate whether or not an interpreter error, run-time exception, or logic error is present. Not all lines contain an error.

17. Write the shortest way to express each of the following statements.

- (a) `x = x + 1`
- (b) `x = x / 2`
- (c) `x = x - 1`
- (d) `x = x + y`
- (e) `x = x - (y + 7)`
- (f) `x = 2*x`
- (g) `number_of_closed_cases = number_of_closed_cases + 2*ncc`

18. What is printed by the following code fragment?

```
x1 = 2
x2 = 2
x1 += 1
x2 -= 1
print(x1)
print(x2)
```

Why does the output appear as it does?

19. Consider the following program that attempts to compute the circumference of a circle given the radius entered by the user. Given a circle's radius, r , the circle's circumference, C is given by the formula:

$$C = 2\pi r$$

```
r = 0
PI = 3.14159
# Formula for the area of a circle given its radius
C = 2*PI*r
# Get the radius from the user
r = float(input('Please enter the circle's radius: '))
# Print the circumference
print('Circumference is', C)
```

- (a) The program does not produce the intended result. Why?
- (b) How can it be repaired so that it works correctly?

20. Write a Python program that ...

21. Write a Python program that ...

Chapter 4

Conditional Execution

All the programs in the preceding chapters execute exactly the same statements regardless of the input, if any, provided to them. They follow a linear sequence: *Statement 1*, *Statement 2*, etc. until the last statement executes and the program terminates. Linear programs like these are very limited in the problems they can solve. This chapter introduces constructs that allow program statements to be optionally executed, depending on the context of the program's execution.

4.1 Boolean Expressions

Arithmetic expressions evaluate to numeric values; a *Boolean* expression, sometimes called a *predicate*, may have only one of two possible values: *false* or *true*. The term Boolean comes from the name of the British mathematician George Boole. A branch of discrete mathematics called Boolean algebra is dedicated to the study of the properties and the manipulation of logical expressions. While on the surface Boolean expressions may appear very limited compared to numeric expressions, they are essential for building more interesting and useful programs.

The simplest Boolean expressions in Python are `True` and `False`. In a Python interactive shell we see:

```
>>> True
True
>>> False
False
>>> type(True)
<class 'bool'>
>>> type(False)
<class 'bool'>
```

We see that `bool` is the name of the class representing Python's Boolean expressions. Listing 4.1 (`boolvars.py`) is a simple program that demonstrates the use of simple Boolean variables.

Table 4.1 The Python relational operators

Expression	Meaning
$x == y$	True if $x = y$ (mathematical equality, not assignment); otherwise, false
$x < y$	True if $x < y$; otherwise, false
$x \leq y$	True if $x \leq y$; otherwise, false
$x > y$	True if $x > y$; otherwise, false
$x \geq y$	True if $x \geq y$; otherwise, false
$x != y$	True if $x \neq y$; otherwise, false

Table 4.2 Examples of some Simple Relational Expressions

Expression	Value
<code>10 < 20</code>	<code>True</code>
<code>10 >= 20</code>	<code>False</code>
<code>x < 100</code>	<code>True</code> if <code>x</code> is less than 100; otherwise, <code>False</code>
<code>x != y</code>	<code>True</code> unless <code>x</code> and <code>y</code> are equal

Listing 4.1: boolvars.py

```
# Assign some Boolean variables
a = True
b = False
print(f'a = {a}, b = {b}')
# Reassign a
a = False
print(f'a = {a}, b = {b}')
```

Listing 4.1 (boolvars.py) produces

```
a = True  b = False
a = False b = False
```

4.2 Boolean Expressions

We have seen that the simplest Boolean expressions are `False` and `True`, the Python Boolean literals. A Boolean variable is also a Boolean expression. An expression comparing numeric expressions for equality or inequality is also a Boolean expression. The simplest kinds of Boolean expressions use *relational operators* to compare two expressions. Table 4.1 lists the relational operators available in Python.

Table 4.2 shows some simple Boolean expressions with their associated values. An expression like `10 < 20` is legal but of little use, since `10 < 20` is always true; the expression `True` is equivalent, simpler, and less likely to confuse human readers. Since variables can change their values during a program's execution, Boolean expressions are most useful when their truth values depend on the values of one or more variables.

In the Python interactive shell we see:

```
>>> x = 10
>>> x
10
>>> x < 10
False
>>> x <= 10
True
>>> x == 10
True
>>> x >= 10
True
>>> x > 10
False
>>> x < 100
True
>>> x < 5
False
```

The first input in the shell binds the variable `x` to the value 10. The other expressions experiment with the relational operators. Exactly matching their mathematical representations, the following expressions all are equivalent:

- `x < 10`
- `10 > x`
- `not (x >= 10)`
- `not (10 <= x)`

The relational operators are binary operators and are all left associative. They all have a lower precedence than any of the arithmetic operators; therefore, Python evaluates the expression

`x + 2 < y / 10`

as if parentheses were placed as so:

`(x + 2) < (y / 10)`

4.3 The Simple if Statement

The Boolean expressions described in Section 4.2 at first may seem arcane and of little use in practical programs. In reality, Boolean expressions are essential for a program to be able to adapt its behavior at run time. Most truly useful and practical programs would be impossible without the availability of Boolean expressions.

The execution errors mentioned in Section 3.6 arise from logic errors. One way that Listing 3.6 (`dividedanger.py`) can fail is when the user enters a zero for the divisor. Fortunately, programmers can take steps to ensure that division by zero does not occur. Listing 4.2 (`betterdivision.py`) shows how it might be done.

Listing 4.2: `betterdivision.py`

```
# File betterdivision.py

# Get two integers from the user
print('Please enter two numbers to divide.')
dividend = int(input('Please enter the first number to divide: '))
divisor = int(input('Please enter the second number to divide: '))
# If possible, divide them and report the result
if divisor != 0:
    print(f'{dividend} / {divisor} = {dividend/divisor}')
```

The program may not always execute the `print` statement. In the following run

```
Please enter two numbers to divide.
Please enter the first number to divide: 32
Please enter the second number to divide: 8
32 / 8 = 4.0
```

the program executes the `print` statement, but if the user enters a zero as the second number:

```
Please enter two numbers to divide.
Please enter the first number to divide: 32
Please enter the second number to divide: 0
```

the program prints nothing after the user enters the values.

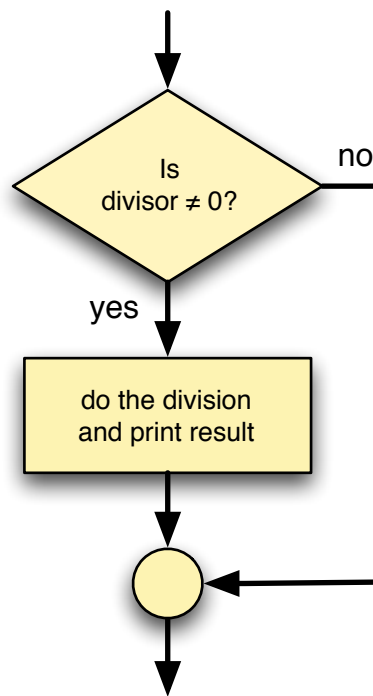
The last non-indented line in Listing 4.2 (`betterdivision.py`) begins with the reserved word `if`. The `if` statement optionally executes the indented section of code. In this case, the `if` statement executes the `print` statement only if the variable `divisor`'s value is not zero.

The Boolean expression

```
divisor != 0
```

determines whether or not the program will execute the statement in the indented block. If `divisor` is not zero, the program prints the message; otherwise, if the user provides a zero divisor, the program displays nothing.

Figure 4.1 shows how program execution flows through the `if` statement. of Listing 4.2 (`betterdivision.py`).

Figure 4.1 if flowchart

The general form of the `if` statement is:

`if` *condition* :
 block

- The reserved word `if` begins a `if` statement.
- The *condition* is a Boolean expression that determines whether or not the body will be executed. A colon (`:`) must follow the condition.
- The *block* is a block of one or more statements to be executed if the condition is true. The statements within the block must all be indented the same number of spaces from the left. The block within an `if` must be indented more spaces than the line that begins the `if` statement. The block technically is part of the `if` statement. This part of the `if` statement is sometimes called the *body* of the `if`.

Python requires the block to be indented. If the block contains just one statement, some programmers will place it on the same line as the `if`; for example, the following `if` statement that optionally assigns `y`:

```
if x < 10:  
    y = x
```

could be written

```
if x < 10: y = x
```

but may *not* be written as

```
if x < 10:  
y = x
```

because the lack of indentation hides the fact that the assignment statement optionally executes. Indentation is how Python determines which statements make up a block.

How many spaces should you indent? Python requires at least one, some programmers consistently use two, four is the most popular number, but some prefer a more dramatic display and use eight. A four space indentation for a block is the recommended Python style. This text uses the recommended four spaces to set off each enclosed block. In most programming editors you can set the `Tab` key to insert spaces automatically so you need not count the spaces as you type. Whichever indent distance you choose, you must use this same distance consistently throughout a Python program.



It is important not to mix spaces and tabs when indenting statements within a block. Python does not allow tabs and spaces to be mixed when indenting statements. In many editors you cannot visually distinguish between a tab and a sequence of spaces. The number of spaces equivalent to the spacing of a tab differs from one editor to another. Generally speaking, a *tab stop* represents a fixed distance from the left side of the editing window. Most editors support multiple tab stops; for example, the first tab stop might be eight characters from the start of the line, followed by a series of tab stops, each one at a distance of eight characters from the previous tab stop (at columns 8, 16, 24, 32, etc. within the line of text). Furthermore, most editors allow the user to reconfigure in some way the locations of the tab stops. Pressing the **Tab** key in the editor causes the cursor within the editing window to jump to the next tab stop within the line. The space bar, on the other hand, always moves the cursor exactly one character to the right.

Why is indentation that mixes tabs and spaces a problem and thus forbidden in Python? Consider creating a Python source file in one editor and then viewing it in a different editor with tab stops set differently. Lines that appear perfectly indented in the original editor could be misaligned in the new editor. Instead, code indented with four spaces within one editor would appear exactly the same in any other editor.

Python does allow the use of tabs for indentation—you just cannot mix them with spaces within the same source file. Most programming editors have a setting to substitute automatically a specified number of spaces when the user presses the **Tab** key. For Python development you should use this feature. Python best practice prefers using spaces over tabs for indentation within Python source.

The `if` block may contain multiple statements for optional execution. Listing 4.3 (alternatedivision.py) optionally executes two statements depending on the input values provided by the user.

Listing 4.3: alternatedivision.py

```
# Get two integers from the user
dividend = int(input('Please enter the number to divide: '))
divisor = int(input('Please enter divisor: '))
# If possible, divide them and report the result
if divisor != 0:
    quotient = dividend/divisor
    print(f'{dividend}/{divisor} = {quotient}')
print('Program finished')
```

The assignment statement and first printing statement are both a part of the block of the `if`. Given the truth value of the Boolean expression `divisor != 0` during a particular program run, either both statements will execute or neither statement will execute. The last statement is not indented, so it is not part of the `if` block. The program always prints *Program finished*, regardless of the user's input.

Remember when checking for equality, as in

```
if x == 10:
    print('ten')
```

to use the relational equality operator (`==`), not the assignment operator (`=`).

As a convenience to programmers, Python's notion of true and false extends beyond what we ordinarily would consider Boolean expressions. The statement

```
if 1:
    print('one')
```

always prints *one*, while the statement

```
if 0:
    print('zero')
```

never prints anything. Python considers the integer value zero to be false and treats every other integer value, positive and negative, to be true. Similarly, the floating-point value 0.0 is false, but any other floating-point value is true. The empty string (`' '` or `''`) is considered false, and any nonempty string is interpreted as true. Any Python expression can serve as the condition for an `if` statement. In later chapters we will explore additional kinds of expressions and see how they relate to Boolean conditions.

Listing 4.4 (`fourdigits.py`) requests an integer value from the user. The program then displays the number using exactly four digits. The program displays leading zeros where necessary to produce four digits. The program treats numbers less than zero as zero and numbers greater than 9,999 as 9999.

Listing 4.4: `fourdigits.py`

```
# Request input from the user
num = int(input('Please enter an integer in the range 0...9999: '))
# Attenuate the number if necessary
if num < 0: # Make sure number is not too small
    num = 0
if num > 9999: # Make sure number is not too big
    num = 9999
print(f'[{num:0{4}}]')
```

A sample run of Listing 4.4 (`fourdigits.py`) produces

```
Please enter an integer in the range 0...9999: 38
[0038]
```

Another run demonstrates the effects of a user entering a negative number:

```
Please enter an integer in the range 0...9999: -450
[0000]
```

The program attenuates numbers that are too large:

```
Please enter an integer in the range 0...9999: 3256670
[9999]
```

In Listing 4.4 (`fourdigits.py`), the two `if` statements at the beginning force the number to be in range.

4.4 The if/else Statement

One undesirable aspect of Listing 4.2 (`betterdivision.py`) is if the user enters a zero divisor, the program prints nothing. It may be better to provide some feedback to the user to indicate that the divisor provided cannot be used. The `if` statement has an optional `else` block that executes only if the Boolean condition is false. Listing 4.5 (`betterfeedback.py`) uses the `if/else` statement to provide the desired effect.

Listing 4.5: `betterfeedback.py`

```
# Get two integers from the user
dividend = int(input('Please enter the number to divide: '))
divisor = int(input('Please enter dividend: '))
# If possible, divide them and report the result
if divisor != 0:
    print(f'{dividend}/{divisor} = {dividend/divisor}')
else:
    print('Division by zero is not allowed')
```

A given run of Listing 4.5 (`betterfeedback.py`) will execute exactly one of either the `if` block or the `else` block. Unlike Listing 4.2 (`betterdivision.py`), this program always displays a message:

```
Please enter the number to divide: 32
Please enter dividend: 0
Division by zero is not allowed
```

The `else` block contains an alternate block of code that the program executes when the condition is false. Figure 4.2 illustrates the program's flow of execution.

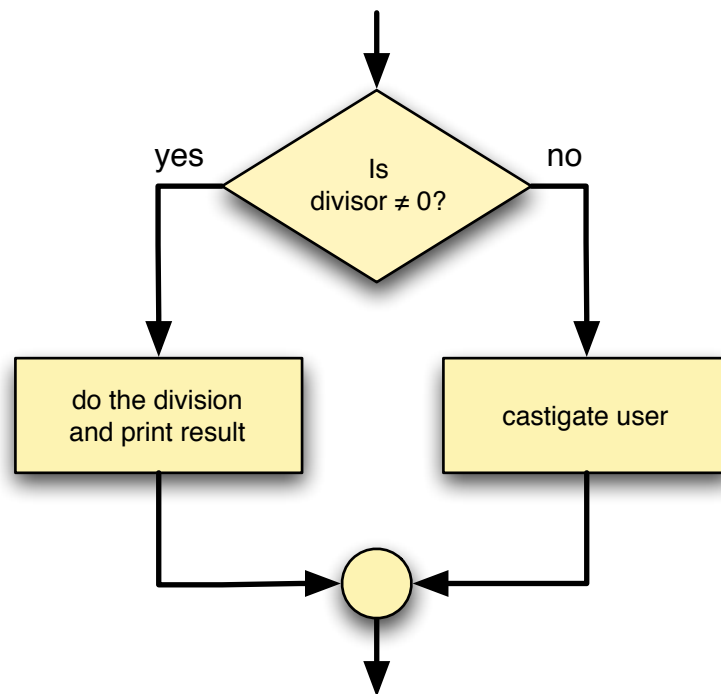
Listing 4.5 (`betterfeedback.py`) avoids the division by zero run-time error that causes the program to terminate prematurely, but it still alerts the user that there is a problem. Another application may handle the situation in a different way; for example, it may substitute some default value for `divisor` instead of zero.

The general form of an `if/else` statement is

```
if condition :
    if-block
else:
    else-block
```

- The reserved word `if` begins the `if/else` statement.
- The *condition* is a Boolean expression that determines whether or not the `if` block or the `else` block executes during a particular program run. A colon (`:`) must follow the condition.

Figure 4.2 if/else flowchart



- The *if-block* is a block of one or more statements to be executed if the condition is true. As with all blocks, it must be indented one level deeper than the `if` line. This part of the `if` statement is sometimes called the body of the `if`.
- The reserved word `else` begins the second part of the `if/else` statement. A colon (`:`) must follow the `else`.
- The *else-block* is a block of one or more statements to execute if the condition is false during a particular program run. It must be indented one level deeper than the line with the `else`. This part of the `if/else` statement is sometimes called the body of the `else`.

The `else` block, like the `if` block, consists of one or more statements indented to the same level.

4.5 Compound Boolean Expressions

We can combine simple Boolean expressions, each involving one relational operator, into more complex Boolean expressions using the logical operators `and`, `or`, and `not`. A combination of two or more Boolean expressions using logical operators is called a *compound Boolean expression*.

To introduce compound Boolean expressions, consider a computer science degree that requires, among other computing courses, *Operating Systems* **and** *Programming Languages*. If we isolate those two courses, we can say a student must successfully complete both *Operating Systems* and *Programming Languages* to qualify for the degree. A student that passes *Operating Systems* but not *Programming Languages* will not have met the requirements. Similarly, *Programming Languages* without *Operating Systems* is insufficient, and a student completing neither *Operating Systems* nor *Programming Languages* surely does not qualify.

The Python logical `and` operator works in exactly the same way. Suppose e_1 and e_2 are two Boolean expressions. e_1 `and` e_2 is true only if e_1 and e_2 are both true; if either one is false or both are false, the compound expression is false.

Related to the logical `and` operator is the logical `or` operator. To illustrate the logical `or` operator, consider two mathematics courses, *Differential Equations* and *Linear Algebra*. A computer science degree requires at least one of those two courses. A student who successfully completes *Differential Equations* but does not take *Linear Algebra* meets the requirement. Similarly, a student may take *Linear Algebra* but not *Differential Equations*. A student that takes neither *Differential Equations* nor *Linear Algebra* certainly has not met the requirement. It is important to note the a student may elect to take both *Differential Equations* and *Linear Algebra* (perhaps on the way to a mathematics minor), but the requirement is no less fulfilled.

Logical `or` works in a similar fashion. Given our Boolean expressions e_1 and e_2 , the compound expression e_1 `or` e_2 is false only if e_1 and e_2 are both false; if either one is true or both are true, the compound expression is true. Note that the `or` operator is an *inclusive or*, not an *exclusive or*. In informal conversation we often imply *exclusive or* in a statement like "Would you like cake **or** ice cream for dessert?" The implication is to choose one or the other, not both. In computer programming the `or` is inclusive; if both subexpressions in an `or` expression are true, the `or` expression evaluates to true.

Logical logical `not` operator reverses the truth value of the expression to which it is applied. If e is a true Boolean expression, `not e` is false; if e is false, `not e` is true. In mathematics, if the expression $x = y$ is false, it must be true that $x \neq y$. In Python, the expression `not (x == y)` is equivalent to the expression `x != y`. If also is the case that the Python expression `not (x != y)` is just a more complicated way of expressing `x == y`. In mathematics, if the expression $x < y$ is false, it must be the case that $x \geq y$. In Python, `not (x < y)` has the same truth value as `x >= y`. The expression `not (x >= y)` is equivalent to `x < y`. You may be able to see from these examples that if e is a Boolean expression, it always

Table 4.3 Logical operators— e_1 and e_2 are Boolean expressions

e_1	e_2	e_1 and e_2	e_1 or e_2	not e_1
False	False	False	False	True
False	True	False	True	True
True	False	False	True	False
True	True	True	True	False

Table 4.4 Precedence of Some Python Operators. Higher precedence operators appear above lower precedence operators.

Arity	Operators	Associativity
binary	**	
unary	+, -	
binary	*, /, //, %	left
binary	+, -	left
binary	>, <, >=, <=, ==, !=	left
unary	not	
binary	and	left
binary	or	left

is true that `not not e` is equivalent to e (this is known as the *double negative* property of mathematical logic).

Table 4.3 is called a *truth table*. It shows all the combinations of truth values for two Boolean expressions and the values of compound Boolean expressions built from applying the `and`, `or`, and `not` Python logical operators.

Both `and` and `or` are binary operators; that is, they require two operands. The `not` operator is a unary operator (see Section 3.1); it requires a single truth expression immediately to its right.

Table 4.4 lists the Python operators we have seen so far. Table 4.4 shows that operator `not` has higher precedence than both `and` and `or`. The `and` operator has higher precedence than `or`. Both the `and` and `or` operators are left associative; `not` is right associative. The `and` and `or` operators have lower precedence than any other binary operator except assignment. This means the expression

```
x <= y and x <= z
```

is evaluated as

```
(x <= y) and (x <= z)
```

Some programmers prefer to use the parentheses as shown here even though they are not required. The parentheses improve the readability of complex expressions, and the interpreted code is no less efficient.

Python allows an expression like

```
x <= y and y <= z
```

which means $x \leq y \leq z$ to be expressed more naturally:

```
x <= y <= z
```

Similarly, Python allows a programmer to test the equivalence of three variables as

```
if x == y == z:
    print('They are all the same')
```

The following section of code assigns the indicated values to a `bool`:

```
x = 10
y = 20
b = (x == 10)           # assigns True to b
b = (x != 10)           # assigns False to b
b = (x == 10 and y == 20) # assigns True to b
b = (x != 10 and y == 20) # assigns False to b
b = (x == 10 and y != 20) # assigns False to b
b = (x != 10 and y != 20) # assigns False to b
b = (x == 10 or y == 20)  # assigns True to b
b = (x != 10 or y == 20)  # assigns True to b
b = (x == 10 or y != 20)  # assigns True to b
b = (x != 10 or y != 20)  # assigns False to b
```

Convince yourself that the following expressions are equivalent:

```
x != y
```

and

```
not (x == y)
```

and

```
x < y or x > y
```

In the expression e_1 **and** e_2 , both subexpressions e_1 and e_2 must be true for the overall expression to be true. Since the **and** operator evaluates left to right, this means that if e_1 is false, there is no need to evaluate e_2 . If e_1 is false, no value of e_2 can make the expression e_1 **and** e_2 true. The **and** operator first tests the expression to its left. If it finds the expression to be false, it does not bother to check the right expression. This approach is called *short-circuit evaluation*. In a similar fashion, in the expression e_1 **or** e_2 , if e_1 is true, then e_2 's value is irrelevant—an **or** expression is true unless both subexpressions are false. The **or** operator uses short-circuit evaluation also.

Why is short-circuit evaluation important? Two situations show why it is important to consider:

- The order of the subexpressions can affect performance. When a program is running, complex expressions require more time for the computer to evaluate than simpler expressions. We classify an expression that takes a relatively long time to evaluate as an *expensive* expression. If a compound Boolean expression is made up of an expensive Boolean subexpression and a less expensive Boolean subexpression, and the order of evaluation of the two expressions does not effect the behavior of the program, then place the more expensive Boolean expression second. In the context of the **and** operator, if its left operand is **False**, the more expensive right operand need not be evaluated. In the context of the **or** operator, if the left operand is **True**, the more expensive right operand may be ignored.

As a simple example, consider the following Python code snippet that could be part of a larger program:

```
if x < 10 and input('Print value (y/n)?') == 'y':
    print(x)
```

If x is a numeric value less than 10, this statement will query the user to print or not print the value of x . If $x \geq 10$, the program need not stop and wait for the user's input. If $x \geq 10$, the user's input

is superfluous anyway. Now consider the statement with the Boolean expressions ordered the other way:

```
if input('Print value (y/n)?') == 'y' and x < 10:
    print(x)
```

In this case as well, both subconditions must be true to print the value of x . The difference here is that the program always pauses its execution to accept the user's input regardless of x 's value. This statement bothers the user for input even when the second subcondition ensures the user's answer will make no difference.

- Subexpressions may be ordered to prevent run-time errors. This is especially true when one of the subexpressions depends on the other in some way. Consider the following expression:

```
(x != 0) and (z/x > 1)
```

Here, if x is zero, the program will not perform the division by zero. If the order of the subexpressions were switched, the program would generate a run-time error when x is zero.

4.6 The pass Statement

Some beginning programmers attempt to use an `if/else` statement when a simple `if` statement is more appropriate; for example, in the following code fragment the programmer wishes to do nothing if the value of the variable x is less than zero; otherwise, the programmer wishes to print x 's value:

```
if x < 0:
    # Do nothing    (This will not work!)
else:
    print(x)
```

If the value of x is less than zero, this section of code should print nothing. Unfortunately, the code fragment above is not legal Python. The `if/else` statement contains an `else` block, but it does not contain an `if` block. The comment does not count as a Python statement. Both `if` and `if/else` statements require an `if` block that contains at least one statement. Additionally, an `if/else` statement requires an `else` block that contains at least one statement.

Python has a special statement, `pass`, that means *do nothing*. We may use the `pass` statement in our code in places where the language requires a statement to appear but we wish the program to take no action whatsoever. We can make the above code fragment legal by adding a `pass` statement:

```
if x < 0:
    pass # Do nothing
else:
    print(x)
```

While the `pass` statement makes the code legal, we can express its logic better by using a simple `if` statement. In mathematics, if the expression $x < y$ is false, it must be the case that $x \geq y$. If we invert the truth value of the relation within the condition, we can express the above code more succinctly as

```
if x >= 0:
    print(x)
```

So, if you ever feel the need to write an `if/else` statement with an empty `if` body, do the following instead:

1. invert the truth value of the condition
2. make the proposed `else` body the `if` body
3. eliminate the `else`

In situations where you may be tempted to use a non-functional `else` block, as in the following:

```
if x == 2:
    print(x)
else:
    pass      # Do nothing if x is not equal to 2
```

do not alter the condition but simply eliminate the `else` and the `else` block altogether:

```
if x == 2:
    print(x)    # Print only if x is equal to 2
```

The `pass` statement in Python is useful for holding the place for code to appear in the future; for example, consider the following code fragment:

```
if x < 0:
    pass      # TODO: print an appropriate warning message to be determined
else:
    print(x)
```

In this code fragment the programmer intends to provide an `if` block, but the exact nature of the code in the `if` block has yet to be determined. The `pass` statement serves as a suitable placeholder for the future code. The included comment documents what is expected to appear eventually in place of the `pass` statement.

We will see other uses of the `pass` statement as we explore Python more deeply.

4.7 Floating-point Equality

The equality operator (`==`) checks for *exact* equality. This can be a problem with floating-point numbers, since floating-point numbers inherently are imprecise. Listing 4.6 (`samedifferent.py`) demonstrates the perils of using the equality operator with floating-point numbers.

Listing 4.6: `samedifferent.py`

```
d1 = 0.11 - 0.10    # Expecting d1 to be 0.01
d2 = 1.11 - 1.10    # Expecting d2 to be 0.01
print(f'd1 = {d1}, d2 = {d2}')
if d1 == d2:
    print('Same')
else:
    print('Different')
```

In mathematics, we expect the following equality to hold:

$$1.11 - 1.10 = 0.01 = 2.11 - 2.10$$

The output of the first `print` statement in Listing 4.6 (`samedifferent.py`) reminds us of the imprecision of floating-point numbers:

```
d1 = 0.009999999999999995, d2 = 0.010000000000000009
Different
```

Since the expression

```
d1 == d2
```

checks for exact equality, the program reports that `d1` and `d2` are different. The bits (0s and 1s) stored in memory for `d1` are different from the bits making up `d2`.

The solution is not to check floating-point numbers for exact equality, but rather see if the values are “close enough” to each other to be considered the same. If `d1` and `d2` are two floating-point numbers, we need to check if the absolute value of the `d1 - d2` is a very small number. Listing 4.7 (`floatequals.py`) adapts Listing 4.6 (`samedifferent.py`) using this approximately equal concept.

Listing 4.7: `floatequals.py`

```
d1 = 0.11 - 0.10 # Expecting d1 to be 0.01
d2 = 1.11 - 1.10 # Expecting d2 to be 0.01
print(f'd1 = {d1}, d2 = {d2}')
diff = d1 - d2 # Compute difference
if diff < 0: # Compute absolute value
    diff = -diff
if diff < 0.0000001: # Are the values close enough?
    print('Same')
else:
    print('Different')
```

Listing 4.7 (`floatequals.py`) prints

```
d1 = 0.009999999999999995, d2 = 0.010000000000000009
Same
```

While the numbers are not exactly the same, our application can consider them equal. If we were working with mathematical real numbers, the two variables would be exactly the same.

Listing 4.8 (`floatequals2.py`) is a variation of Listing 4.7 (`floatequals.py`) that does not compute the absolute value but instead checks to see if the difference is between two numbers that are very close to zero: one negative and the other positive.

Listing 4.8: `floatequals2.py`

```
d1 = 0.11 - 0.10 # Expecting d1 to be 0.01
d2 = 1.11 - 1.10 # Expecting d2 to be 0.01
print(f'd1 = {d1}, d2 = {d2}')
diff = d1 - d2 # Compute difference
if -0.0000001 < d1 - d2 < 0.0000001:
    print('Same')
else:
    print('Different')
```

In Section 7.5.6 we will see how to encapsulate this floating-point equality code within a function to make it more convenient for general use.

4.8 Nested Conditionals

The statements in the block of the `if` or the `else` may be any Python statements, including other `if/else` statements. We can use these nested `if` statements to develop arbitrarily complex program logic. Consider Listing 4.9 (`checkrange.py`) that determines if a number is between 0 and 10, inclusive.

Listing 4.9: `checkrange.py`

```
value = int(input('Please enter an integer value in the range 0...10: '))
if value >= 0:           # First check
    if value <= 10:      # Second check
        print('In range')
print('Done')
```

Listing 4.9 (`checkrange.py`) behaves as follows:

- The executing program checks first condition. If `value` is less than zero, the program does not evaluate the second condition and it continues its execution with the statement following the outer `if`. The statement after the outer `if` simply prints *Done*.
- If the executing program finds the `value` variable to be greater than or equal to zero, it executes the statement within the `if`-block. This statement is itself an `if` statement. The program thus checks the second (inner) condition. If the second condition is satisfied, the program displays the *In range* message; otherwise, it does not. Regardless, the program eventually prints the *Done* message.

We say that the second `if` (with the comment *Second check*) is *nested* within the first `if` (*First check*). We call the first `if` the *outer if* and the second `if` the *inner if*. Notice the entire inner `if` statement is indented one level relative to the outer `if` statement. This means the inner `if`'s block, the `print('In range')` statement, is indented two levels deeper than the outer `if` statement. Remember that if you use the recommended four spaces as the distance for a indentation level, you must consistently use this four space distance for each indentation level throughout the program.

Both conditions of this nested `if` construct must be met for the program to print the *In range* message. Said another way, the first condition *and* the second condition must be met for the program to print the *In range* message. From this perspective, we can rewrite the program to behave the same way with only *one* `if` statement, as Listing 4.10 (`newcheckrange.py`) shows.

Listing 4.10: `newcheckrange.py`

```
value = int(input('Please enter an integer value in the range 0...10: '))
if value >= 0 and value <= 10: # Only one, slightly more complicated check
    print('In range')
print('Done')
```

Listing 4.10 (`newcheckrange.py`) uses the `and` operator to check both conditions at the same time. Its logic is simpler, using only one `if` statement, at the expense of a slightly more complex Boolean expression in its condition. The second version is preferable here because simpler logic is usually a desirable goal.

We may express the condition the `if` within Listing 4.10 (`newcheckrange.py`):

`value >= 0 and value <= 10`

more compactly as

```
0 <= value <= 10
```

This expression is true only when the `value` variable is in the range 0...10, inclusive.

Sometimes we cannot simplify a program's logic as readily as in Listing 4.10 (`newcheckrange.py`). Listing 4.11 (`enhancedcheckrange.py`) would be impossible to rewrite with only one `if` statement.

Listing 4.11: `enhancedcheckrange.py`

```
value = int(input('Please enter an integer value in the range 0...10: '))
if value >= 0:      # First check
    if value <= 10: # Second check
        print(value, 'is in range')
    else:
        print(value, 'is too large')
else:
    print(value, 'is too small')
print('Done')
```

Listing 4.11 (`enhancedcheckrange.py`) provides a more specific message instead of a simple notification of acceptance. The program prints exactly one of three messages based on the value of the variable. A single `if` or `if/else` statement cannot choose from among more than two different execution paths.

Computers store all data internally in binary form. The binary (base 2) number system is much simpler than the familiar decimal (base 10) number system because it uses only two digits: 0 and 1. The decimal system uses 10 digits: 0, 1, 2, 3, 4, 5, 6, 7, 8, and 9. Despite the lack of digits, every decimal integer has an equivalent binary representation. Binary numbers use a place value system not unlike the decimal system. Figure 4.3 shows how the familiar base 10 place value system works.

Figure 4.3 The base 10 place value system

...	4	7	3	4	0	6
...	10^5	10^4	10^3	10^2	10^1	10^0
...	100,000	10,000	1,000	100	10	1

$$\begin{aligned}
 473,406 &= 4 \times 10^5 + 7 \times 10^4 + 3 \times 10^3 + 4 \times 10^2 + 0 \times 10^1 + 6 \times 10^0 \\
 &= 400,000 + 70,000 + 3,000 + 400 + 0 + 6 \\
 &= 473,406
 \end{aligned}$$

With 10 digits to work with, the decimal number system distinguishes place values with powers of 10. Compare the base 10 system to the base 2 place value system shown in Figure 4.4.

With only two digits to work with, the binary number system distinguishes place values by powers of two. Since both binary and decimal numbers share the digits 0 and 1, we will use the subscript 2 to indicate a binary number; therefore, 100 represents the decimal value *one hundred*, while 100_2 is the binary number *four*. Sometimes to be very clear we will attach a subscript of 10 to a decimal number, as in 100_{10} .

Listing 4.12 (`binaryconversion.py`) uses an `if` statement containing a series of nested `if` statements to print a 10-bit binary string representing the binary equivalent of a decimal integer supplied by the user. We

Figure 4.4 The base 2 place value system

$$\begin{array}{cccccc}
 \dots & \boxed{1} & \boxed{0} & \boxed{0} & \boxed{1} & \boxed{1} & \boxed{1} \\
 \dots & 2^5 & 2^4 & 2^3 & 2^2 & 2^1 & 2^0 \\
 \dots & 32 & 16 & 8 & 4 & 2 & 1
 \end{array}$$

$$\begin{aligned}
 100111_2 &= 1 \times 2^5 + 0 \times 2^4 + 0 \times 2^3 + 1 \times 2^2 + 1 \times 2^1 + 1 \times 2^0 \\
 &= 32 + 0 + 0 + 4 + 2 + 1 \\
 &= 39
 \end{aligned}$$

use `if/else` statements to print the individual digits left to right, essentially assembling the sequence of bits that represents the binary number.

Listing 4.12: binaryconversion.py

```

# Get number from the user
value = int(input('Please enter an integer value in the range 0...1023: '))
# Create an empty binary string to build upon
binary_string = ''
# Integer must be less than 1024
if 0 <= value < 1024:
    if value >= 512:
        binary_string += '1'
        value %= 512
    else:
        binary_string += '0'
    if value >= 256:
        binary_string += '1'
        value %= 256
    else:
        binary_string += '0'
    if value >= 128:
        binary_string += '1'
        value %= 128
    else:
        binary_string += '0'
    if value >= 64:
        binary_string += '1'
        value %= 64
    else:
        binary_string += '0'
    if value >= 32:
        binary_string += '1'
        value %= 32
    else:
        binary_string += '0'
    if value >= 16:
        binary_string += '1'
        value %= 16

```



```
    else:
        binary_string += '0'
    if value >= 8:
        binary_string += '1'
        value %= 8
    else:
        binary_string += '0'
    if value >= 4:
        binary_string += '1'
        value %= 4
    else:
        binary_string += '0'
    if value >= 2:
        binary_string += '1'
        value %= 2
    else:
        binary_string += '0'
    binary_string += str(value)

# Display the results
if binary_string != '':
    print(binary_string)
else:
    print('Cannot convert')
```

In Listing 4.12 (binaryconversion.py):

- The outer `if` checks to see if the value the user provides is in the proper range. The program works only for numbers in the range $0 \leq \text{value} < 1,024$.
- Each inner `if` compares the user-supplied entered integer against decreasing powers of two. If the number is large enough, the program:
 - appends the digit (actually character) 1 to the binary string under construction, and
 - removes via the remainder operator that power of two's contribution to the value.

If the number is not at least as big as the given power of two, the program instead concatenates a 0 and moves on without modifying the input value.

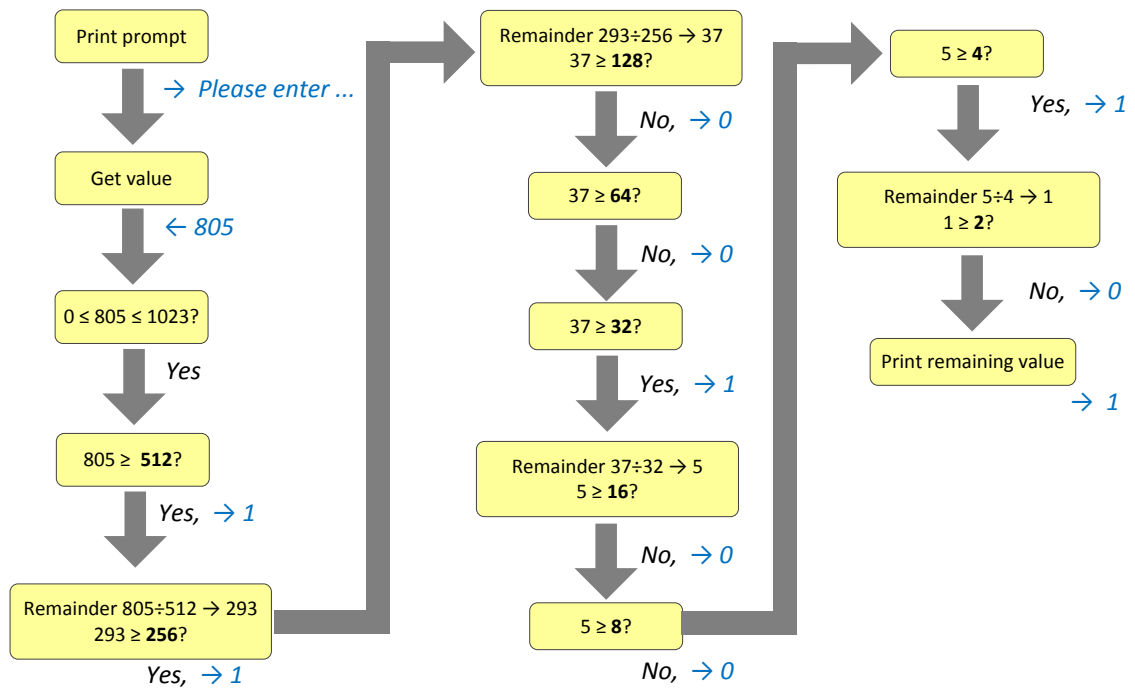
- For the ones place at the end, no check is necessary—the remaining value will be 0 or 1 and so the program appends the string version of 0 or 1.

The following shows a sample run of Listing 4.12 (binaryconversion.py):

```
Please enter an integer value in the range 0...1023: 805
1100100101
```

Figure 4.5 illustrates the execution of Listing 4.12 (binaryconversion.py) when the user enters 805.

Figure 4.5 The process of the binary number conversion program when the user supplies 805 as the input value.



Listing 4.13 (`simplerbinaryconversion.py`) simplifies the logic of Listing 4.12 (`binaryconversion.py`) at the expense of some additional arithmetic. It uses only one `if` statement.

Listing 4.13: `simplerbinaryconversion.py`

```
# Get number from the user
value = int(input('Please enter an integer value in the range 0...1023: '))
# Initial binary string is empty
binary_string = ''
# Integer must be less than 1024
if 0 <= value < 1024:
    binary_string += str(value//512)
    value %= 512
    binary_string += str(value//256)
    value %= 256
    binary_string += str(value//128)
    value %= 128
    binary_string += str(value//64)
    value %= 64
    binary_string += str(value//32)
    value %= 32
    binary_string += str(value//16)
    value %= 16
    binary_string += str(value//8)
    value %= 8
    binary_string += str(value//4)
    value %= 4
    binary_string += str(value//2)
    value %= 2
    binary_string += str(value)
# Report results
if binary_string != '':
    print(binary_string)
else:
    print('Unable to convert')
```

The sole `if` statement in Listing 4.13 (`simplerbinaryconversion.py`) ensures that the user provides an integer in the proper range. The other `if` statements that originally appeared in Listing 4.12 (`binaryconversion.py`) are gone. A clever sequence of integer arithmetic operations replace the original conditional logic. The two programs—`binaryconversion.py` and `simplerbinaryconversion.py`—behave identically but `simplerbinaryconversion.py`'s logic is simpler.

Python provides a built-in function named `bin` that produces a binary string representation of an integer. We can use it in the Python interpreter to check the results of our binary conversion:

```
>>> bin(805)
'0b1100100101'
```

Note that the `bin` function includes a `'0b'` prefix. In Python source, we can express an integer in binary form if we prefix it with `0b`. Consider the following experiment in the interpreter:

```
>>> 805
805
>>> 0b1100100101
805
```

```
>>> x = 0b1100100101
>>> print(x)
805
>>> print(bin(x))
0b1100100101
```

It is easy to remove the `'0b'` prefix to express just the binary number string. We will cover the basics of *slicing* later, but for now, the following shows a one-line statement that accomplishes the same effect as Listing 4.12 (`binaryconversion.py`) and Listing 4.13 (`simplerbinaryconversion.py`):

```
>>> bin(805)[2:]
'1100100101'
```

The `[2:]` notation effectively cuts off the first two characters the string that the `bin` function produces. This process is called *slicing*, and it is a technique available to Python sequence types like strings. Section 10.8 introduces the concept of slicing in more detail.

Listing 4.14 (`troubleshoot.py`) implements a very simple troubleshooting program that an (equally simple) computer technician might use to diagnose an ailing computer.

Listing 4.14: troubleshoot.py

```

print("Help! My computer doesn't work!")
print('Does the computer make any sounds (fans, etc.)')
choice = input('or show any lights? (y/n):')
# The troubleshooting control logic
if choice == 'n': # The computer does not have power
    choice = input('Is it plugged in? (y/n):')
    if choice == 'n': # It is not plugged in, plug it in
        print('Plug it in. If the problem persists, ')
        print('please run this program again.')
    else: # It is plugged in
        choice = input('Is the switch in the \'on\' position? (y/n):')
        if choice == 'n': # The switch is off, turn it on!
            print('Turn it on. If the problem persists, ')
            print('please run this program again.')
        else: # The switch is on
            choice = input('Does the computer have a fuse? (y/n):')
            if choice == 'n': # No fuse
                choice = input('Is the outlet OK? (y/n):')
                if choice == 'n': # Fix outlet
                    print("Check the outlet's circuit ")
                    print('breaker or fuse. Move to a')
                    print('new outlet, if necessary. ')
                    print('If the problem persists, ')
                    print('please run this program again.')
                else: # Beats me!
                    print('Please consult a service technician.')
            else: # Check fuse
                print('Check the fuse. Replace if ')
                print('necessary. If the problem ')
                print('persists, then ')
                print('please run this program again.')
        else: # The computer has power
            print('Please consult a service technician.')

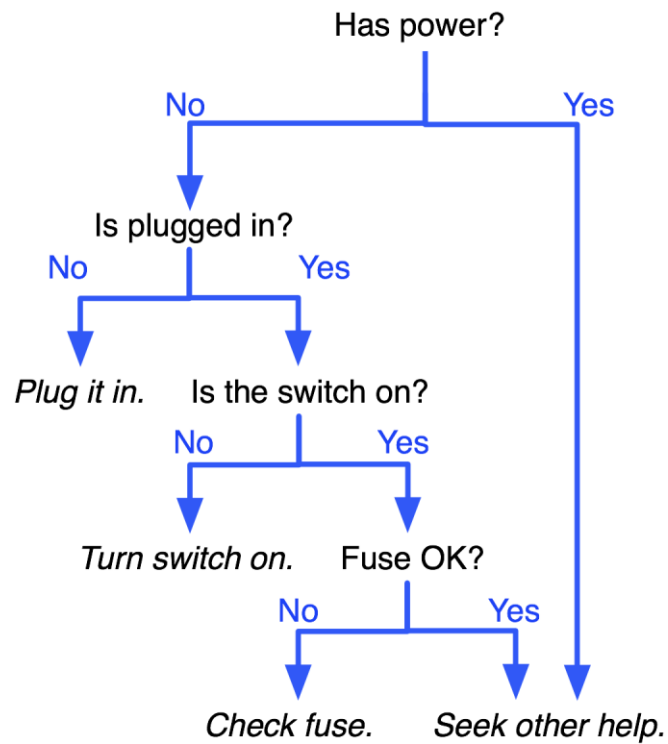
```

This very simple troubleshooting program attempts to diagnose why a computer does not work. The potential for enhancement is unlimited, but this version deals only with power issues that have simple fixes. Notice that if the computer has power (fan or disk drive makes sounds or lights are visible), the program indicates that help should be sought elsewhere! Figure 4.6 shows the decision tree that represents the logic of the program. The steps the program performs are:

1. Is it plugged in? This simple fix is sometimes overlooked.
2. Is the switch in the *on* position? This is another simple fix.
3. If applicable, is the fuse blown? Some computer systems have a user-serviceable fuse that can blow out during a power surge. (Most newer computers have power supplies that can handle power surges and have no user-serviceable fuses.)
4. Is there power at the receptacle? Perhaps the wall outlet's circuit breaker or fuse has a problem.

This algorithm performs the easiest checks first. It adds progressively more difficult checks as the program continues. Based on your experience with troubleshooting computers that do not run properly, you may be able to think of many enhancements to this simple program.

Figure 4.6 Decision tree for troubleshooting a computer system



Note the various blocks of code and how the blocks are indented within Listing 4.14 (troubleshoot.py). Visually, programmers quickly can determine the logical structure of the program by the arrangement and indentation of the blocks.

Recall the time conversion program in Listing 3.9 (timeconv.py). If the user enters 10000, the program runs as follows:

```
Please enter the number of seconds:10000
2:46:40
```

Suppose we wish to express this as *2 hours, 46 minutes, and 40 seconds*. If we spell out *hours*, *minutes*, and *seconds*, we must be careful to use the singular form *hour*, *minute*, or *second* when the corresponding value is one. Listing 4.15 (timeconvcond1.py) uses *if/else* statements to express to time units with the correct number.

Listing 4.15: timeconvcond1.py

```
# File timeconvcond1.py

# Some useful conversion factors
seconds_per_minute = 60
seconds_per_hour   = 60*seconds_per_minute # 3600

# Get user input in seconds
seconds = int(input('Please enter the number of seconds:'))

# First, compute the number of hours in the given number of seconds
hours = seconds // seconds_per_hour # 3600 seconds = 1 hour
# Compute the remaining seconds after the hours are accounted for
seconds = seconds % seconds_per_hour
# Next, compute the number of minutes in the remaining number of seconds
minutes = seconds // seconds_per_minute # 60 seconds = 1 minute
# Compute the remaining seconds after the minutes are accounted for
seconds = seconds % seconds_per_minute
# Report the results
print(hours, end='')
# Decide between singular and plural form of hours
if hours == 1:
    print(' hour, ', end='')
else:
    print(' hours, ', end='')
print(minutes, end='')
# Decide between singular and plural form of minutes
if minutes == 1:
    print(' minute, and ', end='')
else:
    print(' minutes, and ', end='')
print(seconds, end='')
# Decide between singular and plural form of seconds
if seconds == 1:
    print(' second')
else:
    print(' seconds')
```

The `if/else` statements within Listing 4.15 (`timeconvcond1.py`) are responsible for printing the correct version—singular or plural—for each time unit. One run of Listing 4.15 (`timeconvcond1.py`) produces

```
Please enter the number of seconds:10000
2 hours, 46 minutes, and 40 seconds
```

All the words are plural since all the value are greater than one. Another run produces

```
Please enter the number of seconds:9961
2 hours, 46 minutes, and 1 second
```

Note the word *second* is singular as it should be.

```
Please enter the number of seconds:3601
1 hour, 0 minutes, and 1 second
```

Here again the printed words agree with the number of the value they represent.

An improvement to Listing 4.15 (`timeconvcond1.py`) would not print a value and its associated time unit if the value is zero. Listing 4.16 (`timeconvcond2.py`) adds this feature.

Listing 4.16: `timeconvcond2.py`

```
# File timeconvcond2.py

# Some useful conversion constants
seconds_per_minute = 60
seconds_per_hour   = 60*seconds_per_minute # 3600
seconds = int(input('Please enter the number of seconds:'))
# First, compute the number of hours in the given number of seconds
hours = seconds // seconds_per_hour # 3600 seconds = 1 hour
# Compute the remaining seconds after the hours are accounted for
seconds = seconds % seconds_per_hour
# Next, compute the number of minutes in the remaining number of seconds
minutes = seconds // seconds_per_minute # 60 seconds = 1 minute
# Compute the remaining seconds after the minutes are accounted for
seconds = seconds % seconds_per_minute
# Report the results
result = '' # Start with empty string
if hours > 0: # Print hours at all?
    result += str(hours)
    # Decide between singular and plural form of hours
    if hours == 1:
        result += ' hour'
    else:
        result += ' hours'
    if (minutes > 0 and seconds == 0) or (minutes == 0 and seconds > 0):
        result += ' and '
    elif minutes > 0 and seconds > 0:
        result += ', '
if minutes > 0: # Print minutes at all?
    result += str(minutes)
    # Decide between singular and plural form of minutes
    if minutes == 1:
        result += ' minute'
```



```
    else:
        result += ' minutes'
    if hours > 0 and seconds > 0:
        result += ', and '
    elif hours == 0 and seconds > 0:
        result += ' and '
    # Print seconds at all?
    if seconds > 0 or (hours == 0 and minutes == 0 and seconds == 0):
        result += str(seconds)
        # Decide between singular and plural form of seconds
        if seconds == 1:
            result += ' second'
        else:
            result += ' seconds'
    print(result) # Finally print the newline
```

In Listing 4.16 (timeconvcond2.py), conditional logic controls the concatenation of time values, the English words for time units, commas, and *and*.

One run of Listing 4.16 (timeconvcond2.py) produces

```
Please enter the number of seconds:10000
2 hours, 46 minutes, and 40 seconds
```

All the words are plural since all the value are greater than one. Another run produces

```
Please enter the number of seconds:9961
2 hours, 46 minutes, and 1 second
```

Note the word *second* is singular as it should be.

```
Please enter the number of seconds:3601
1 hour and 1 second
```

Here again the printed words agree with the number of the value they represent.

```
Please enter the number of seconds:7200
2 hours
```

Another run produces:

```
Please enter the number of seconds:60
1 minute
```

Finally, the following run shows that the program handles zero seconds properly:

```
Please enter the number of seconds:0
0 seconds
```

When dealing with nested conditional logic, proper indentation is especially important. The Python interpreter will detect illegal indentation, as shown in Listing 1.3 (badindent.py), but sometimes improper indentation simply changes the logic of the program. Consider Listing 4.17 (indentation.py).

Listing 4.17: indentation.py

```

x = int(input('Please enter a number: '))
y = int(input('Please enter another number: '))

print(f'x = {x}, y = {y}')

print('-----')

if x == 1:                                # 1a
    if y == 2:                            # 2a
        print('A', end=' ')              # 3a
    else:                                 # 4a
        print('B', end=' ')              # 5a
print('C')                                # 6a

print('-----')

if x == 1:                                # 1b
    if y == 2:                            # 2b
        print('A', end=' ')              # 3b
else:                                     # 4b
    print('B', end=' ')                  # 5b
print('C')                                # 6b

```

The lines labeled *#1a* through *#6a* contain the same code as lines *#1b* through *#6b*, but note that lines *#4b* and *#5b* are not indented at the same level as lines *#4a* and *#5a*. Both sections of the code represent legal Python indentation, so the interpreter will not indicate any indentation errors. Running the program shows that the two sections of code behave differently, depending on the user's input. When the user enters the values 1 and 2, the program prints the following:

```

Please enter a number: 1
Please enter another number: 2
x = 1, y = 2
-----
A C
-----
A C

```

It seems the two sections of code behave identically; however, consider the user's entry of 2 and 1:

```

Please enter a number: 2
Please enter another number: 1
x = 2, y = 1
-----
C
-----
B C

```

Here, the first section prints just C, but the second section prints B C. There definitely is a difference. Next, consider the entries 1 and 1:

```

Please enter a number: 1
Please enter another number: 1
x = 1, y = 1
-----
B C
-----
C

```

In this case, the first section prints B C, but the second section prints just C. The indentation changes the program's logic. The first section of code is an `if` statement that has an `if/else` statement in its `if` block. The second section of code is an `if/else` statement that has an `if` statement in its `if` block. While these may seem like the same thing at first reading, the program's execution clearly shows that the logic is different. The two sections of code behave differently under identical inputs.

4.9 Multi-way Decision Statements

A simple `if/else` statement can select from between two execution paths. Listing 4.11 (`enhancedcheckrange.py`) showed how to select from among three options. What if exactly one of many actions should be taken? This requires nested `if/else` statements, and Listing 4.18 (`digittoword.py`) shows the forms of these nested `if/else` statements.

Listing 4.18: `digittoword.py`

```

value = int(input('Please enter an integer in the range 0...5: '))
if value < 0:
    print('Too small')
else:
    if value == 0:
        print('zero')
    else:
        if value == 1:
            print('one')
        else:
            if value == 2:
                print('two')
            else:
                if value == 3:
                    print('three')
                else:
                    if value == 4:
                        print('four')
                    else:
                        if value == 5:
                            print('five')
                        else:
                            print('Too large')
print('Done')

```

Observe the following about Listing 4.18 (`digittoword.py`):

- It prints exactly one of eight messages depending on the user's input.

- Notice that each `if` block contains a single printing statement and each `else` block, except the last one, contains an `if` statement. The control logic forces the program execution to check each condition in turn. The first condition that matches wins, and its corresponding `if` body executes. If none of the conditions are true, the program prints the last `else`'s *Too large* message.

As a consequence of the required formatting of Listing 4.18 (`digittoword.py`), the mass of text drifts to the right as the program requires more conditions to be checked. Python provides a multi-way conditional construct called `if/elif/else` that permits a more manageable textual structure for programs that must check many conditions. Listing 4.19 (`restyleddigittoword.py`) uses the `if/elif/else` statement to avoid the rightward code drift.

Listing 4.19: `restyleddigittoword.py`

```
value = int(input('Please enter an integer in the range 0...5: '))
if value < 0:
    print('Too small')
elif value == 0:
    print('zero')
elif value == 1:
    print('one')
elif value == 2:
    print('two')
elif value == 3:
    print('three')
elif value == 4:
    print('four')
elif value == 5:
    print('five')
else:
    print('Too large')
print('Done')
```

The word `elif` is a contraction of `else` and `if`; if you read `elif` as *else if*, you can see how we can transform the code fragment

```
else:
    if value == 2:
        print('two')
```

in Listing 4.18 (`digittoword.py`) into

```
elif value == 2:
    print('two')
```

in Listing 4.19 (`restyleddigittoword.py`).

The `if/elif/else` statement is valuable for selecting exactly one block of code to execute from several different options. The `if` part of an `if/elif/else` statement is mandatory. The `else` part is optional. After the `if` part and before `else` part (if present) you may use as many `elif` blocks as necessary.

The general form of an `if/elif/else` statement is

```
if condition-1 :  
    block-1  
elif condition-2 :  
    block-2  
elif condition-3 :  
    block-3  
elif condition-4 :  
    block-4  
    .  
    .  
    .  
else:  
    default-block
```

Listing 4.20 (`datetransformer.py`) uses an `if/elif/else` statement to transform a numeric date in month/day format to an expanded US English form and an international Spanish form; for example, 2/14 would be converted to February 14 and 14 febrero.

Listing 4.20: datetransformer.py

```
month = int(input('Please enter the month as a number (1-12): '))
day = input('Please enter the day of the month: ')
# Translate month into English
if month == 1:
    english = 'January'
elif month == 2:
    english = 'February'
elif month == 3:
    english = 'March'
elif month == 4:
    english = 'April'
elif month == 5:
    english = 'May'
elif month == 6:
    english = 'June'
elif month == 7:
    english = 'July'
elif month == 8:
    english = 'August'
elif month == 9:
    english = 'September'
elif month == 10:
    english = 'October'
elif month == 11:
    english = 'November'
else:
    english = 'December'
# Add the day
date = english + ' ' + day + ' or ' + day
# Translate month into Spanish
if month == 1:
    spanish = ' de enero'
elif month == 2:
    spanish = ' de febrero'
elif month == 3:
    spanish = ' de marzo'
elif month == 4:
    spanish = ' de abril'
elif month == 5:
    spanish = ' de mayo'
elif month == 6:
    spanish = ' de junio'
elif month == 7:
    spanish = ' de julio'
elif month == 8:
    spanish = ' de agosto'
elif month == 9:
    spanish = ' de septiembre'
elif month == 10:
    spanish = ' de octubre'
elif month == 11:
    spanish = ' de noviembre'
else:
```

```
spanish = ' de diciembre'
date += spanish

print(date)
```

A sample run of Listing 4.20 (datetransformer.py) is shown here:

```
Please enter the month as a number (1-12): 5
Please enter the day of the month: 20
May 20 or 20 de mayo
```

An `if/elif/else` statement that includes the optional `else` will execute exactly one of its blocks. The first condition that evaluates to true selects the block to execute. An `if/elif/else` statement that omits the `else` block may fail to execute the code in any of its blocks if none of its conditions evaluate to `True`.

4.10 Multi-way Versus Sequential Conditionals

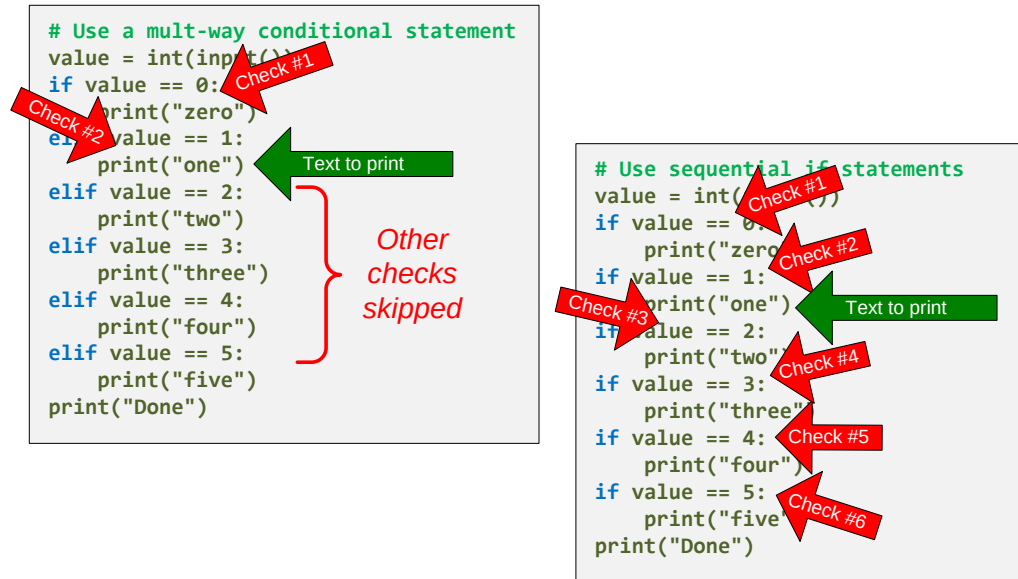
Beginning programmers sometimes confuse multi-way `if/elif/else` statement with a sequence of simple statements. Consider Listing 4.21 (simplerdigittoword.py). It does not print anything if the user enters an integer out of range; otherwise, it prints the equivalent English word.

Listing 4.21: simplerdigittoword.py

```
# Use a multi-way conditional statement
value = int(input())
if value == 0:
    print('zero')
elif value == 1:
    print('one')
elif value == 2:
    print('two')
elif value == 3:
    print('three')
elif value == 4:
    print('four')
elif value == 5:
    print('five')
print('Done')
```

Next, consider Listing 4.22 (simplerdigittoword2.py). It replaces the `elif`s with `ifs`. It behaves identically to Listing 4.21 (simplerdigittoword.py).

Figure 4.7 A trace of the execution of Listing 4.21 (`simplerdigittoword.py`) (left) and Listing 4.22 (`simplerdigittoword2.py`) (right) when the user enters the value 1.



Listing 4.22: `simplerdigittoword2.py`

```
# Use sequential conditional statements
value = int(input())
if value == 0:
    print('zero')
if value == 1:
    print('one')
if value == 2:
    print('two')
if value == 3:
    print('three')
if value == 4:
    print('four')
if value == 5:
    print('five')
print('Done')
```

Despite their equivalent behavior, Listing 4.21 (`simplerdigittoword.py`) is the better program. Figure 4.7 traces the two programs' progress when the user enters the value 1. Listing 4.21 (`simplerdigittoword.py`) performs only two comparisons before making its choice and then finally printing the message *Done*. In contrast, Listing 4.22 (`simplerdigittoword2.py`) must check every conditional expression in the program before it prints *Done*. For a program of this size and complexity, the difference is imperceptible. In some more complex situations, however, the extra computation may noticeably affect the application. The bigger

issue is the impression that it can make on new programmers that the sequential `ifs` are equivalent to the multi-way `if/elifs`. To dispel this idea, compare the behavior of Listing 4.23 (`multivsseq1.py`) to Listing 4.24 (`multivsseq2.py`).

Listing 4.23: `multivsseq1.py`

```
num = int(input('Enter a number: '))
if num == 1:
    print('You entered one')
elif num == 2:
    print('You entered two')
elif num > 5:
    print('You entered a number greater than five')
elif num == 7:
    print('You entered seven')
else:
    print('You entered some other number')
```

In Listing 4.23 (`multivsseq1.py`) if the user enters 7, the program prints *You entered a number greater than five*, as shown in the following program run:

```
Enter a number: 7
You entered a number greater than five
```

Listing 4.24 (`multivsseq2.py`) merely changes all the `elifs` to `ifs`.

Listing 4.24: `multivsseq2.py`

```
num = int(input('Enter a number: '))
if num == 1:
    print('You entered one')
if num == 2:
    print('You entered two')
if num > 5:
    print('You entered a number greater than five')
if num == 7:
    print('You entered seven')
else:
    print('You entered some other number')
```

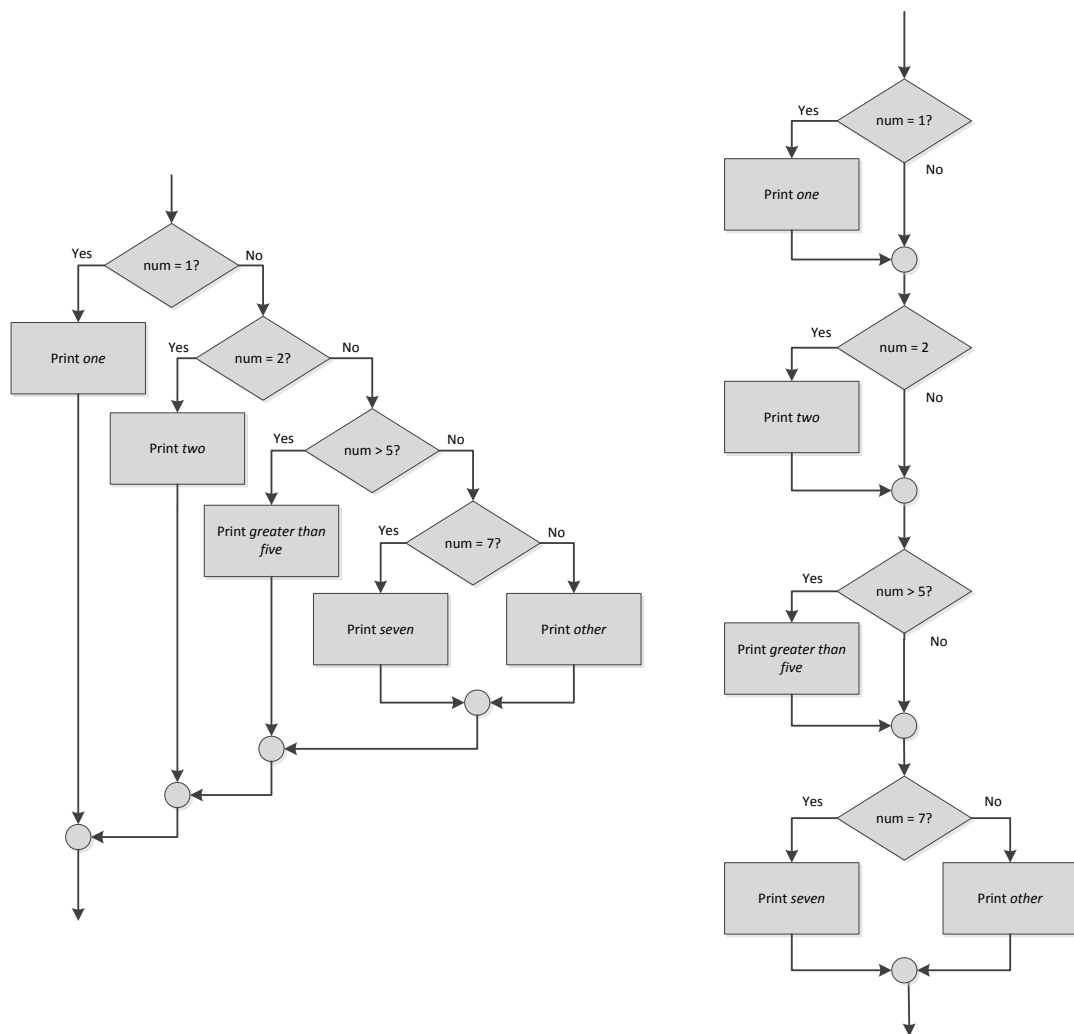
In this case if the user enters 7, the program's output is different:

```
Enter a number: 7
You entered a number greater than five
You entered seven
```

The program prints the same message as does Listing 4.23 (`multivsseq1.py`), but it additionally will print the message *You entered seven*. These two programs clearly demonstrate that multi-way conditional statements behave differently from sequential `ifs`.

Figure 4.8 compares the structure of the conditional statements in Listing 4.23 (`multivsseq1.py`) to Listing 4.24 (`multivsseq2.py`).

Figure 4.8 The structure of the conditional statements in Listing 4.23 (multisseq1.py) (left) versus Listing 4.24 (multisseq2.py) (right).



4.11 Structural Pattern Matching

Listing 4.20 (datetransformer.py) used a multi-way `if/elif/else` statement to transform a date into both English and Spanish. `pgrefmatchdatetransform.py` uses a Python `match` statement to accomplish the same behavior.

Listing 4.25: matchdatetransform.py

```
month = int(input('Please enter the month as a number (1-12): '))
day = input('Please enter the day of the month: ')
# Translate month into English
match month:
    case 1 :
        english = 'January'
    case 2 :
        english = 'February'
    case 3 :
        english = 'March'
    case 4 :
        english = 'April'
    case 5 :
        english = 'May'
    case 6 :
        english = 'June'
    case 7 :
        english = 'July'
    case 8 :
        english = 'August'
    case 9 :
        english = 'September'
    case 10:
        english = 'October'
    case 11:
        english = 'November'
    case 12:
        english = 'December'
    case _:
        english = 'Error'
# Add the day
date = english + ' ' + day + ' or ' + day
# Translate month into Spanish
match month:
    case 1:
        spanish = ' de enero'
    case 2:
        spanish = ' de febrero'
    case 3:
        spanish = ' de marzo'
    case 4:
        spanish = ' de abril'
    case 5:
        spanish = ' de mayo'
    case 6:
        spanish = ' de junio'
    case 7:
```

```
    spanish = ' de julio'
case 8:
    spanish = ' de agosto'
case 9:
    spanish = ' de septiembre'
case 10:
    spanish = ' de octubre'
case 11:
    spanish = ' de noviembre'
case _:
    spanish = ' de diciembre'
date += spanish

print(date)
```

As we can see, the `match` statement in its simplest form can substitute for a multi-way `if/elif/else` statement. The `match` statement, however, has capabilities beyond multi-way conditional logic.

Within a `match` statement, both `match` and `case` are known as *soft keywords*. This means Python treats them as keywords within the context of the `match` statement. Outside of a `match` statement, programmers are free to use the words `match` and `case` as regular identifiers for variable names, class names, function names, etc. Why are `match` and `case` not real reserved words? The `match` statement is a relatively recent addition to the Python programming language, introduced in Python 3.10 in 2021 (Guido van Rossum released Python 1 in 1991). Making `match` and `case` first-class keywords would break many pre-existing Python programs written over the previous 30 years that use these words as identifiers.

The general form of the `match` statement is:

```
match subject :
case pattern 1 :
    action 1
case pattern 2 :
    action 2
case pattern 3 :
    action 3
    ⋮
case pattern n :
    action n
case _ :
    default action
```

The soft keyword `match` begins a match statement. The *subject* is an expression that potentially contains a pattern to match. Each section labeled with `case` defines a pattern. When reading from top to bottom, the first *case pattern* that matches the *subject* triggers its associated *action*.

What differentiates the `match` statement from a multi-way `if/elif/else` statement is its ability to match the subject with a pattern, rather than simply making decisions based on comparing expressions.

Consider Listing 4.26 (`classifypoint.py`) that determines where a mathematical point object, (x,y) , lies relative to the coordinate axes.

Listing 4.26: `classifypoint.py`

```
x = float(input('Please enter the x corrdinate: '))
y = float(input('Please enter the y corrdinate: '))

if x == 0.0 and y == 0.0:
    print('The point lies on the origin')
elif x == 0.0:
    print(f'The point lies on the y axis at {y}')
elif y == 0.0:
    print(f'The point lies on the x axis at {x}')
else:
    print(f'The point lies on neither the x-axis nor the y-axis')
```

Compare Listing 4.26 (`classifypoint.py`) to Listing 4.27 (`classifypointmatch.py`) that uses a `match` statement.

Listing 4.27: `classifypointmatch.py`

```
x = float(input('Please enter the x coordinate: '))
y = float(input('Please enter the y coordinate: '))

match (x, y):
    case (0, 0):
        print('The point lies on the origin')
    case (0, y):
        print(f'The point lies on the y axis at {y}')
    case (x, 0):
        print(f'The point lies on the x axis at {x}')
    case _:
        print(f'The point lies on neither the x-axis nor the y-axis')
```

The `==` and `and` operators are missing from Listing 4.27 (`classifypointmatch.py`). To understand the logic, consider the point the user provides. If it matches the tuple $(0,0)$, the program prints the first message. If the x coordinate is zero and the first pattern did not match, the program prints the second message. If the y coordinate is zero and the first pattern did not match, the program prints the third message. Finally, if the user provided (x,y) does not match any of the provided patterns, the underscore pattern (`_`) represents the wildcard “catch all” pattern that plays a role similar to `else` in an `if/elif/else` statement. The `_` matches any subject if none of the other patterns do.

Once you understand how the `match` statement works, the logic of Listing 4.27 (`classifypointmatch.py`) is a little simpler than Listing 4.26 (`classifypoint.py`). The `==` operator is missing, as the pattern matching handles it implicitly.

If you need to match multiple patterns with a single action, combine the patterns with the `|` (read as “or”) operator.

```

match (x, y):
    case (0, 0):
        print('The point lies on the origin')
    case (0, _) | (_, 0):
        print(f'The point lies on an axis')
    case _:
        print(f'The point lies on neither the x-axis nor the y-axis')

```

The pattern `(0, _)` matches any tuple with a first element of zero. The pattern `(0, _) | (_, 0)` matches any point with a zero *x* or zero *y* coordinate that the first pattern `(0, 0)` did not match.

Section 10.17 shows how to use the `match` statement with more advanced Python features.

4.12 Conditional Expressions

Consider the following code fragment:

```

if a != b:
    c = d
else:
    c = e

```

This code assigns to variable `c` one of two possible values. As purely a syntactical convenience, Python provides an alternative to the `if/else` construct called a *conditional expression*. A conditional expression evaluates to one of two values depending on a Boolean condition. We can rewrite the above code as

```
c = d if a != b else e
```

The general form of the conditional expression is

expression-1 **if** *condition* **else** *expression-2*

where

- *expression-1* is the overall value of the conditional expression if *condition* is true.
- *condition* is a normal Boolean expression that might appear in an `if` statement.
- *expression-2* is the overall value of the conditional expression if *condition* is false.

In the above code fragment, *expression-1* is the variable `d`, *condition* is `a != b`, and *expression-2* is `e`.

Listing 4.28 (safedivide.py) uses our familiar `if/else` statement to check for division by zero.

Listing 4.28: safedivide.py

```
# Get the dividend and divisor from the user
dividend = int(input('Enter dividend: '))
divisor = int(input('Enter divisor: '))
# We want to divide only if divisor is not zero; otherwise,
# we will print an error message
if divisor != 0:
    print(dividend/divisor)
else:
    print('Error, cannot divide by zero')
```

Using a conditional expression, we can rewrite Listing 4.28 (safedivide.py) as Listing 4.29 (safedivideconditional.py).

Listing 4.29: safedivideconditional.py

```
# Get the dividend and divisor from the user
dividend = int(input('Enter dividend: '))
divisor = int(input('Enter divisor: '))
# We want to divide only if divisor is not zero; otherwise,
# we will print an error message
msg = dividend/divisor if divisor != 0 else 'Error, cannot divide by zero'
print(msg)
```

Notice that in Listing 4.29 (safedivideconditional.py) the type of the msg variable depends which expression is assigned; msg can be a floating-point value or a string.

As another example, the *absolute value* of a number is defined in mathematics by the following formula:

$$|n| = \begin{cases} n, & \text{when } n \geq 0 \\ -n, & \text{when } n < 0 \end{cases}$$

In other words, the absolute value of a positive number or zero is the same as that number; the absolute value of a negative number is the additive inverse (negative of) of that number. The following Python expression represents the *absolute value* of the variable n:

```
-n if n < 0 else n
```

An equally valid way to express it is

```
n if n >= 0 else -n
```

The expression itself is not statement. Listing 4.30 (absvalueconditional.py) is a small program that provides an example of the conditional expression's use in a statement.

Listing 4.30: absvalueconditional.py

```
# Acquire a number from the user and print its absolute value.
n = int(input('Enter a number: '))
print(f'|{n}| = {-n if n < 0 else n}')
```

Some sample runs of Listing 4.30 (absvalueconditional.py) show

```
Enter a number: -34
|-34| = 34
```

and

```
Enter a number: 0
|0| = 0
```

and

```
Enter a number: 100
|100| = 100
```

Some argue that the conditional expression is not as readable as a normal `if/else` statement. Regardless, many Python programmers use it sparingly because of its very specific nature. Standard `if/else` blocks can contain multiple statements, but contents in the conditional expression are limited to single, simple expressions.

4.13 Errors in Conditional Statements

Carefully consider each compound conditional used, such as

```
value > 0 and value <= 10
```

found in Listing 4.10 (`newcheckrange.py`). Confusing logical `and` and logical `or` is a common programming error. Consider the Boolean expression

```
x > 0 or x <= 10
```

What values of `x` make the expression true, and what values of `x` make the expression false? This expression is always true, no matter what value is assigned to the variable `x`. A Boolean expression that is always true is known as a *tautology*. Think about it. If `x` is a number, what value could the variable `x` assume that would make this Boolean expression false? Regardless of its value, one or both of the subexpressions will be true, so the compound `or` expression is always true. This particular `or` expression is just a complicated way of expressing the value `True`.

Another common error is accidentally writing compound Boolean expressions that are always false, known as *contradictions*. Suppose you wish to *exclude* values from a given range; for example, reject values in the range 0...10 and accept all other numbers. Is the Boolean expression in the following code fragment up to the task?

```
# All but 0, 1, 2, ..., 10
if value < 0 and value > 10:
    print(value)
```

A closer look at the condition reveals it can *never* be true. What number can be both less than zero and greater than ten *at the same time*? None can, of course, so the expression is a contradiction and a complicated way of expressing `False`. To correct this code fragment, replace the `and` operator with `or`.

Table 4.4 indicates that the logical operators `and`, `or`, and `not` have lower precedence than the comparison operators. Suppose you wish to print the word *OK* if a variable `x` is 1, 2, or 3. An informal translation from English might yield:

```
if x == 1 or 2 or 3:
    print('OK')
```


Unfortunately, x 's value is irrelevant; the code always prints the word *OK* regardless of the value of x . Since the `==` operator has higher precedence than `or`, Python interprets the expression

```
x == 1 or 2 or 3
```

as if it were expressed as

```
(x == 1) or 2 or 3
```

The expression `x == 1` is either true or false, but integer 2 is always interpreted as true, and integer 3 is interpreted as true as well. Any appearance of a true value within a sequence of subexpressions all connected by `or` will make the overall expression true.

The the most correct way express the original statement would be

```
if x == 1 or x == 2 or x == 3:
    print('OK')
```

The revised Boolean expression is more verbose and less similar to the English rendition, but it is the correct formulation for Python. Note that each of the subexpressions involves a Boolean expression, not an integer value. If x is known to be an integer and not a floating-point number, the expression

```
1 <= x <= 3
```

also would be equivalent.

Consider Listing 4.31 (`digittoworderror.py`), which is a variation of Listing 4.19 (`restyleddigittoword.py`). Listing 4.31 (`digittoworderror.py`) assigns a word to a string variable instead of printing it immediately.

Listing 4.31: `digittoworderror.py`

```
value = int(input('Please enter an integer in the range 0...5: '))
answer = 'not in range' # Default answer
if value == 0:
    answer = 'zero'
elif value == 1:
    answer = 'one'
elif value == 2:
    answer = 'two'
elif value == 3:
    answer = 'three'
elif value == 4:
    answer = 'four'
elif value == 5:
    answer = 'five'
print('The number you entered was', answer)
```

The following shows a sample run of Listing 4.31 (`digittoworderror.py`):

```
Please enter an integer in the range 0...5: 2
The number you entered was two
```

Another run shows

```
Please enter an integer in the range 0...5: 7
The number you entered was not in range
```

Of course the program will crash with an error if the user enters a noninteger value (the `int` function will raise an exception), but the program should accept any integer value and always do the right thing. In fact, Listing 4.31 (`digittoworderror.py`) always works as expected *except* when the user enters 3:

```
Please enter an integer in the range 0...5: 3
The number you entered was not in range
```

The program has a subtle spelling error; can you find it?

On an English keyboard the **M** and **N** keys are next to each other. It is a common typographical error to press one in place of the other by mistake. Note the variable named `amswer` in the condition that checks for `value` equaling 3. The program reassigns the original `answer` variable for each option except when the user submits the value 3. When the user enters 3 the program creates a new variable which goes unreported in the final printing statement. This kind of error is not restricted to conditional statements but is due to the way Python manages variables. A programmer can create a new variable at any time using the simple assignment operator. The statement

```
amswer = 'three'
```

cannot reassign the original `answer` variable but only create a brand new variable with a different name. Unfortunately the interpreter cannot understand what we *mean* by the code we write; it only can dutifully *execute* the code we provide to it.

Listing 4.31 (`digittoworderror.py`) is small and focussed, and so the logic error introduced by the misspelled variable was easy to track down. As our programs grow in size and our logic becomes more complex, a misspelling error that creates a new variable sometimes can be difficult to track down without assistance. Fortunately tools exist that can help. Programmers can use *Pylint* (<http://www.pylint.org/>) to analyze Python source code. Among other useful information, Pylint can detect variables defined within a program that have no later use.

4.14 Logic Complexity

Python provides the tools to construct some very complicated conditional statements. It is important to resist the urge to make things overly complex. Composing Boolean expressions with `and`, `or`, and `not` allows us to build conditions with arbitrarily complex logic. There often are many ways to achieve the same effect; for example, the following four Boolean expressions are equivalent:

```
not (a == b and c != d)
not (a == b and not (c == d))
not (a == b) or not (c != d)
a != b or c == d
```

While the four expressions are logically equivalent, they exhibit various degrees of complication.

Some situations require complicated logic to achieve the desired program behavior, but, in general, the simplest logic that works is better than more complex equivalent logic for several reasons:

- Simpler logic usually is easier to understand.
- Simpler logic is easier to write and get working. Longer, more complicated expressions increase the risk of typographical errors that can manifest themselves as logic errors. These errors are more difficult to find and correct in complex code.

- Simpler logic is can be more efficient. Every relational comparison and Boolean operation executed by a running program requires machine cycles. The expression

```
not (a == b and not (c == d))
```

performs up to five separate operations: the `==` operation twice, the `not` operation twice, and the `and` operation once. If `a` does not equal `b`, the expression will perform only two operations (`==` and the outer `not`) due to short-circuit Boolean evaluation of the `and`. Compare this expression to the following equivalent expression:

```
a != b or c == d
```

An executing program can evaluate this expression faster, as it involves at most three operations: `!=`, `==`, and `or`. If `a` is not equal to `b`, the execution short-circuits the `or`, and evaluates only the `!=` operation.

- Simpler logic is easier to modify and extend.

Consider the task of computing the maximum value of four integers entered by the user. Listing 4.32 (max4a.py) provides one solution.

Listing 4.32: max4a.py

```
# Get input values from user
print('Please enter four integer values.')
num1 = int(input('Enter number 1: '))
num2 = int(input('Enter number 2: '))
num3 = int(input('Enter number 3: '))
num4 = int(input('Enter number 4: '))

# Compute the maximum value
if num1 >= num2 and num1 >= num3 and num1 >= num4:
    max = num1
elif num2 >= num1 and num2 >= num3 and num2 >= num4:
    max = num2
elif num3 >= num1 and num3 >= num2 and num3 >= num4:
    max = num3
else: # The maximum must be num4 at this point
    max = num4

# Report result
print(f'The maximum number entered was: {max}')
```

Listing 4.32 (max4a.py) uses a multi-way `if/elif/else` construct to select the correct value to assign to its `max` variable.

Listing 4.32 (max4a.py) works correctly for any four integers a user may enter, but its logic is somewhat complicated. We will consider a different strategy for our selection logic. All variables have a meaning, and their names should reflect their meaning in some way. We'll let our `max` variable mean “maximum I have determined so far.” The following outlines our new approach to the solution:

1. Set `max` equal to `n1`. This means as far as we know at the moment, `n1` is the biggest number because `max` and `n1` have the same value.

2. Compare `max` to `n2`. If `n2` is larger than `max`, change `max` to have `n2`'s value to reflect the fact that we determined `n2` is larger; if `n2` is not larger than `max`, we have no reason to change `max`, so do not change it.
3. Compare `max` to `n3`. If `n3` is larger than `max`, change `max` to have `n3`'s value to reflect the fact that we determined `n3` is larger; if `n3` is not larger than `max`, we have no reason to change `max`, so do not change it.
4. Compare `max` to `n4`. If `n4` is larger than `max`, change `max` to have `n4`'s value to reflect the fact that we determined `n4` is larger; if `n4` is not larger than `max`, we have no reason to change `max`, so do not change it.

In the end the meaning of the `max` variable remains the same—"maximum I have determined so far," but, after comparing `max` to all the input variables, we now know it is *the* maximum value of all four input numbers. The extra variable `max` is not strictly necessary, but it makes thinking about the problem and its solution easier.

Listing 4.33 (`max4b.py`) uses our revised selection logic to provide an alternative to Listing 4.32 (`max4a.py`).

Listing 4.33: `max4b.py`

```
# Get input values from user
print('Please enter four integer values.')
num1 = int(input('Enter number 1: '))
num2 = int(input('Enter number 2: '))
num3 = int(input('Enter number 3: '))
num4 = int(input('Enter number 4: '))

# Compute the maximum value
max = num1
if num2 > max:
    max = num2
if num3 > max:
    max = num3
if num4 > max:
    max = num4

# Report result
print(f'The maximum number entered was: {max}')
```

Listing 4.33 (`max4b.py`) uses simple, sequential `if` statements to eventually assign the correct value to its `max` variable.

Listing 4.33 (`max4b.py`) always performs three `>` comparisons and at most four assignments regardless of the values provided by the user. The source code for Listing 4.32 (`max4a.py`) contains many more Boolean operations than Listing 4.33 (`max4b.py`), but, due to short-circuit evaluation, Listing 4.32 (`max4a.py`) actually can outperform Listing 4.33 (`max4b.py`) in some situations. Simpler logic is no guarantee of better performance. Both programs are so simple, however, that users will perceive no difference in execution speeds.

What changes would we need to make to both Listing 4.32 (`max4a.py`) and Listing 4.33 (`max4b.py`) if we must extend them to handle five input values instead of four? Adding this capability to Listing 4.32 (`max4a.py`) forces us to modify **every** condition in the program, adding a check against a new `num5` variable. We also must provide an additional `elif` check since we will need to select from among five possible

assignments to the `max` variable. In Listing 4.33 (`max4b.py`), however, we need only add an extra sequential `if` statement with a new simple condition to check.

Chapter 5 introduces loops, the ability to execute statements repeatedly. You easily can adapt the sequential `if` approach to allow users to type in as many numbers as they like and then have the program report the maximum number the user entered. It is not possible to adapt in this manner the multi-way `if/elif/else` approach with the more complex logic. Not only is our sequential `if` version cleaner and simpler; we more easily can extend its capabilities.

4.15 Exercises

1. What possible values can a Boolean expression have?
2. Where does the term Boolean originate?
3. What is an integer equivalent to `True` in Python?
4. What is the integer equivalent to `False` in Python?
5. Is the value `-16` interpreted as `True` or `False`?
6. Given the following definitions:

```
x, y, z = 3, 5, 7
```

evaluate the following Boolean expressions:

- (a) `x == 3`
 - (b) `x < y`
 - (c) `x >= y`
 - (d) `x <= y`
 - (e) `x != y - 2`
 - (f) `x < 10`
 - (g) `x >= 0 and x < 10`
 - (h) `x < 0 and x < 10`
 - (i) `x >= 0 and x < 2`
 - (j) `x < 0 or x < 10`
 - (k) `x > 0 or x < 10`
 - (l) `x < 0 or x > 10`
7. Given the following definitions:

```
x, y = 3, 5  
b1, b2, b3, b4 = True, False, x == 3, y < 3
```

evaluate the following Boolean expressions:

 - (a) `b3`
 - (b) `b4`

- (c) `not b1`
 - (d) `not b2`
 - (e) `not b3`
 - (f) `not b4`
 - (g) `b1 and b2`
 - (h) `b1 or b2`
 - (i) `b1 and b3`
 - (j) `b1 or b3`
 - (k) `b1 and b4`
 - (l) `b1 or b4`
 - (m) `b2 and b3`
 - (n) `b2 or b3`
 - (o) `b1 and b2 or b3`
 - (p) `b1 or b2 and b3`
 - (q) `b1 and b2 and b3`
 - (r) `b1 or b2 or b3`
 - (s) `not b1 and b2 and b3`
 - (t) `not b1 or b2 or b3`
 - (u) `not (b1 and b2 and b3)`
 - (v) `not (b1 or b2 or b3)`
 - (w) `not b1 and not b2 and not b3`
 - (x) `not b1 or not b2 or not b3`
 - (y) `not (not b1 and not b2 and not b3)`
 - (z) `not (not b1 or not b2 or not b3)`
8. Express the following Boolean expressions in simpler form; that is, use fewer operators or fewer symbols. `x` is an integer.
- (a) `not (x == 2)`
 - (b) `x < 2 or x == 2`
 - (c) `not (x < y)`
 - (d) `not (x <= y)`
 - (e) `x < 10 and x > 20`
 - (f) `x > 10 or x < 20`
 - (g) `x != 0`
 - (h) `x == 0`
9. Express the following Boolean expressions in an equivalent form *without* the `not` operator. `x` and `y` are integers.
- (a) `not (x == y)`
 - (b) `not (x > y)`

- (c) `not (x < y)`
- (d) `not (x >= y)`
- (e) `not (x <= y)`
- (f) `not (x != y)`
- (g) `not (x != y)`
- (h) `not (x == y and x < 2)`
- (i) `not (x == y or x < 2)`
- (j) `not (not (x == y))`

10. What is the simplest tautology?
11. What is the simplest contradiction?
12. Write a Python program that requests an integer value from the user. If the value is between 1 and 100 inclusive, print "OK;" otherwise, do not print anything.
13. Write a Python program that requests an integer value from the user. If the value is between 1 and 100 inclusive, print "OK;" otherwise, print "Out of range."
14. Write a Python program that allows a user to type in an English day of the week (*Sunday, Monday, etc.*). The program should print the Spanish equivalent, if possible.
15. Consider the following Python code fragment:

```
# i, j, and k are numbers
if i < j:
    if j < k:
        i = j
    else:
        j = k
else:
    if j > k:
        j = i
    else:
        i = k
print("i =", i, " j =", j, " k =", k)
```

What will the code print if the variables *i*, *j*, and *k* have the following values?

- (a) *i* is 3, *j* is 5, and *k* is 7
 - (b) *i* is 3, *j* is 7, and *k* is 5
 - (c) *i* is 5, *j* is 3, and *k* is 7
 - (d) *i* is 5, *j* is 7, and *k* is 3
 - (e) *i* is 7, *j* is 3, and *k* is 5
 - (f) *i* is 7, *j* is 5, and *k* is 3
16. Consider the following Python program that prints one line of text:

```

val = int(input())
if val < 10:
    if val != 5:
        print("wow ", end='')
    else:
        val += 1
else:
    if val == 17:
        val += 10
    else:
        print("whoa ", end='')
print(val)

```

What will the program print if the user provides the following input?

- (a) 3
- (b) 21
- (c) 5
- (d) 17
- (e) -5

17. Consider the following two Python programs that appear very similar:

<pre> n = int(input()) if n < 1000: print('*', end='') if n < 100: print('*', end='') if n < 10: print('*', end='') if n < 1: print('*', end='') print() </pre>	<pre> n = int(input()) if n < 1000: print('*', end='') elif n < 100: print('*', end='') elif n < 10: print('*', end='') elif n < 1: print('*', end='') print() </pre>
---	---

How do the two programs react when the user provides the following inputs?

- (a) 0
- (b) 1
- (c) 5
- (d) 50
- (e) 500
- (f) 5000

Why do the two programs behave as they do?

18. Write a Python program that requests five integer values from the user. It then prints the maximum and minimum values entered. If the user enters the values 3, 2, 5, 0, and 1, the program would indicate that 5 is the maximum and 0 is the minimum. Your program should handle ties properly; for example, if the user enters 2, 4, 2, 3, and 3, the program should report 2 as the minimum and 4 as maximum.

19. Write a Python program that requests five integer values from the user. It then prints one of two things: if any of the values entered are duplicates, it prints **"DUPLICATES"**; otherwise, it prints **"ALL UNIQUE"**.

Chapter 5

Iteration

Iteration repeats the execution of a sequence of code. Iteration is useful for solving many programming problems. Iteration and conditional execution form the basis for algorithm construction.

5.1 The while Statement

Listing 5.1 (counttofive.py) counts to five by printing a number on each output line.

Listing 5.1: counttofive.py

```
print(1)
print(2)
print(3)
print(4)
print(5)
```

When executed, this program displays

```
1
2
3
4
5
```

How would you write the code to count to 10,000? Would you copy, paste, and modify 10,000 printing statements? You could, but that would be impractical! Counting is such a common activity, and computers routinely count up to very large values, so there must be a better way. What we really would like to do is print the value of a variable (call it `count`), then increment the variable (`count += 1`), and repeat this process until the variable is large enough (`count == 5` or maybe `count == 10000`). This process of executing the same section of code over and over is known as *iteration*, or *looping*. Python has two different statements, `while` and `for`, that enable iteration.

Listing 5.2 (iterativecounttofive.py) uses a `while` statement to count to five:

Listing 5.2: iterativecounttofive.py

```
count = 1           # Initialize counter
while count <= 5:    # Should we continue?
    print(count)     # Display counter, then
    count += 1       # Increment counter
```

The `while` statement in Listing 5.2 (`iterativecounttofive.py`) repeatedly displays the variable `count`. The program executes the following block of statements five times:

```
print(count)
count += 1
```

After each redisplay of the variable `count`, the program increments it by one. Eventually (after five iterations), the condition `count <= 5` will no longer be true, and the block is no longer executed.

Unlike the approach taken in Listing 5.1 (`counttofive.py`), it is trivial to modify Listing 5.2 (`iterativecounttofive.py`) to count up to 10,000—just change the literal value 5 to 10000.

The line

```
while count <= 5:
```

begins the `while` statement. The expression following the `while` keyword is the condition that determines if the statement block executes or continues to execute. As long as the condition is true, the program executes the code block over and over again. When the condition becomes false, the loop is finished. If the condition is false initially, the program will not execute the code block within the body of the loop at all.

The `while` statement has the general form:

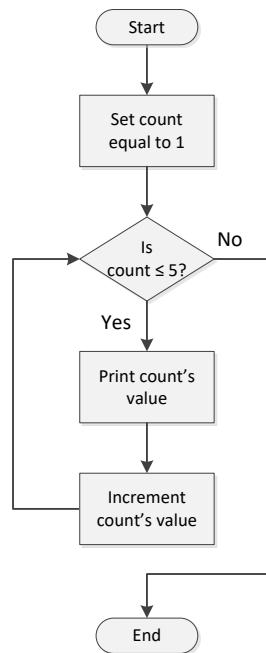
```
while condition :
```

```
    block
```

- The reserved word `while` begins the `while` statement.
- The *condition* determines whether the body will execute or will continue to execute. A colon (`:`) must follow the condition.
- *block* is a block of one or more statements to execute as long as the condition is true. As a block, all the statements that comprise the block must be indented one level deeper than the line that begins the `while` statement.

Except for the reserved word `while` instead of `if`, `while` statements look identical to `if` statements. Sometimes beginning programmers confuse the two or accidentally type `if` when they mean `while` or vice-versa. Usually the very different behavior of the two statements reveals the problem immediately; however, sometimes, especially in nested, complex logic, this mistake can be hard to detect.

Figure 5.1 shows how program execution flows through Listing 5.2 (`iterativecounttofive.py`).

Figure 5.1 while flowchart for Listing 5.2 (iterativecounttofive.py)

The executing program checks the condition before executing the `while` block and then checks the condition again after executing the `while` block. As long as the condition remains true, the program repeatedly executes the code in the `while` block. If the condition initially is false, the program will not execute the code within the `while` block. If the condition initially is true, the program executes the block repeatedly until the condition becomes false, at which point the loop terminates.

Listing 5.3 (`countup.py`) counts up from zero as long as the user wishes to do so.

Listing 5.3: `countup.py`

```
# Counts up from zero. The user continues the count by entering
# 'Y'. The user discontinues the count by entering 'N'.

count = 0 # The current count
entry = 'Y' # Count to begin with
while entry != 'N' and entry != 'n':
    # Print the current value of count
    print(count)
    entry = input('Please enter "Y" to continue or "N" to quit: ')
    if entry == 'Y' or entry == 'y':
        count += 1 # Keep counting
    # Check for "bad" entry
    elif entry != 'N' and entry != 'n':
        print(f'"{entry}" is not a valid choice')
    # else must be 'N' or 'n'
```

A sample run of Listing 5.3 (`countup.py`) produces

```
0
Please enter "Y" to continue or "N" to quit: y
1
Please enter "Y" to continue or "N" to quit: y
2
Please enter "Y" to continue or "N" to quit: y
3
Please enter "Y" to continue or "N" to quit: q
"q" is not a valid choice
3
Please enter "Y" to continue or "N" to quit: r
"r" is not a valid choice
3
Please enter "Y" to continue or "N" to quit: W
"W" is not a valid choice
3
Please enter "Y" to continue or "N" to quit: Y
4
Please enter "Y" to continue or "N" to quit: y
5
Please enter "Y" to continue or "N" to quit: n
```

In Listing 5.3 (`countup.py`) the expression

```
entry != 'N' and entry != 'n'
```

is true if `entry` is neither `N` nor `n`.

Listing 5.4 (addnonnegatives.py) is a program that allows a user to enter any number of nonnegative integers. When the user enters a negative value, the program no longer accepts input, and it displays the sum of all the nonnegative values. If a negative number is the first entry, the sum is zero.

Listing 5.4: addnonnegatives.py

```
# Allow the user to enter a sequence of nonnegative
# integers. The user ends the list with a negative
# integer. At the end the sum of the nonnegative
# numbers entered is displayed. The program prints
# zero if the user provides no nonnegative numbers.

entry = 0      # Ensure the loop is entered
sum = 0        # Initialize sum

# Request input from the user
print('Enter numbers to sum, negative number ends list:')

while entry >= 0:      # A negative number exits the loop
    entry = int(input()) # Get the value
    if entry >= 0:      # Is number nonnegative?
        sum += entry    # Only add it if it is nonnegative
    print('Sum =', sum) # Display the sum
```

Listing 5.4 (addnonnegatives.py) uses two variables, `entry` and `sum`:

- `entry`

In the beginning we initialize `entry` to zero for the sole reason that we want the condition `entry >= 0` of the `while` statement to be true initially. If we fail to initialize `entry`, the program will produce a run-time error when it attempts to compare `entry` to zero in the `while` condition. The `entry` variable holds the number entered by the user. Its value can change each time through the loop.

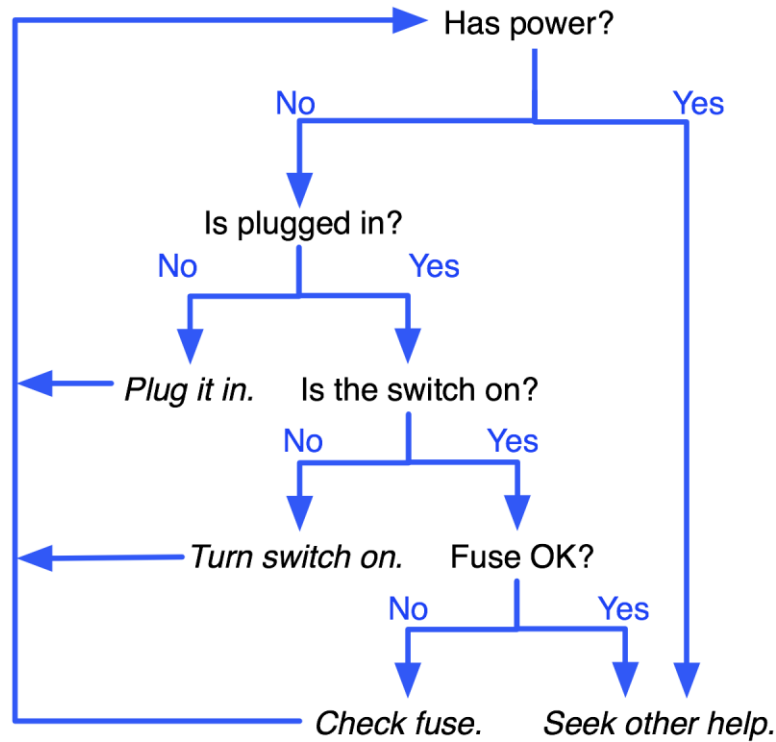
- `sum`

The variable `sum` is known as an *accumulator* because it accumulates each value the user enters. We initialize `sum` to zero in the beginning because a value of zero indicates that it has not accumulated anything. If we fail to initialize `sum`, the program will generate a run-time error when it attempts to use the `+=` operator to modify the (non-existent) variable. Within the loop we repeatedly add the user's input values to `sum`. When the loop finishes (because the user entered a negative number), `sum` holds the sum of all the nonnegative values entered by the user.

The initialization of `entry` to zero coupled with the condition `entry >= 0` of the `while` guarantees that the program will execute the body of the `while` loop at least once. The `if` statement ensures that the program will not add a negative `entry` to `sum`. (Could the `if` condition have used `>` instead of `>=` and achieved the same results?) When the user enters a negative value, the executing program will not update the `sum` variable, and the condition of the `while` will no longer be true. The loop then terminates and the program executes the print statement.

Listing 5.4 (addnonnegatives.py) shows that a `while` loop can be used for more than simple counting. The program does not keep track of the number of values entered. The program simply accumulates the entered values in the variable named `sum`.

We can use a `while` statement to make Listing 4.14 (troubleshoot.py) more convenient for the user. Recall that the computer troubleshooting program forces the user to rerun the program once it detects a

Figure 5.2 Decision tree for troubleshooting a computer system

potential problem (for example, turn on the power switch, then run the program again to see what else might be wrong). A more desirable Figure 5.2 shows a more desirable decision logic.

Listing 5.5 (`troubleshootloop.py`) incorporates a `while` statement so that the program's execution continues until the problem is resolved or its resolution is beyond the capabilities of the program.

Listing 5.5: troubleshootloop.py

```

print("Help! My computer doesn't work!")
done = False    # Not done initially
while not done:
    print("Does the computer make any sounds (fans, etc.) ")
    choice = input("or show any lights? (y/n):")
    # The troubleshooting control logic
    if choice == 'n': # The computer does not have power
        choice = input("Is it plugged in? (y/n):")
        if choice == 'n': # It is not plugged in, plug it in
            print("Plug it in.")
        else: # It is plugged in
            choice = input("Is the switch in the \"on\" position? (y/n):")
            if choice == 'n': # The switch is off, turn it on!
                print("Turn it on.")
            else: # The switch is on
                choice = input("Does the computer have a fuse? (y/n):")
                if choice == 'n': # No fuse
                    choice = input("Is the outlet OK? (y/n):")
                    if choice == 'n': # Fix outlet
                        print("Check the outlet's circuit ")
                        print("breaker or fuse. Move to a")
                        print("new outlet, if necessary. ")
                    else: # Beats me!
                        print("Please consult a service technician.")
                        done = True # Nothing else I can do
                else: # Check fuse
                    print("Check the fuse. Replace if ")
                    print("necessary.")
    else: # The computer has power
        print("Please consult a service technician.")
        done = True # Nothing else I can do

```

A `while` block makes up the bulk of Listing 5.5 (`troubleshootloop.py`). The Boolean variable `done` controls the loop; as long as `done` is false, the loop continues. A Boolean variable like `done` used in this fashion is often called a *flag*. You can think of the flag being down when the value is false and raised when it is true. In this case, a raised flag signals that the loop should terminate.

It is important to note that the expression

```
not done
```

of the `while` statement's condition evaluates to the opposite truth value of the variable `done`; the expression does *not* affect the value of `done`. In other words, the `not` operator applied to a variable does not modify the variable's value. In order to actually change the variable `done`, you would need to reassign it, as in

```
done = not done # Invert the truth value
```

For Listing 5.5 (`troubleshootloop.py`) we have no need to invert `done`'s value. We ensure that `done`'s value is `False` initially and then make it `True` when the user has exhausted the program's options.

In Python, sometimes it is convenient to use a simple value as conditional expression in an `if` or `while` statement. Python interprets the integer value 0 and floating-point value 0.0 both as `False`. It considers

all other integer and floating-point values, both positive and negative, as `True`. This means the following code:

```
x = int(input())  # Get integer from user
while x:
    print(x)      # Print x only if x is nonzero
    x -= 1        # Decrement x
```

is equivalent to

```
x = int(input())  # Get integer from user
while x != 0:
    print(x)      # Print x only if x is nonzero
    x -= 1        # Decrement x
```

The `bool` function converts an expression to its Boolean equivalent. The following interactive session further demonstrates how Python treats non-Boolean values as truth values:

```
>>> bool(1)
True
>>> bool(0)
False
>>> int(True)
1
>>> int(False)
0
>>> bool(-17)
True
>>> bool(0.0)
False
>>> bool(0.000000001)
True
>>> bool('')    # Empty string
False
>>> bool(' ')   # String containing one space
True
>>> bool('good')
True
>>> bool(20 - (2 * 10))
False
>>> float(False)
0.0
>>> float(True)
1.0
```

Speaking of floating-point values, Section 3.1 demonstrated how the limited precision of floating-point numbers can be a problem. You should use caution when using a floating-point number to control a loop, as

Listing 5.6 (stopatone.py) shows

Listing 5.6: stopatone.py

```
x = 0.0
while x != 1.0:
    print(x)
    x += 0.1
print("Done")
```

The programmer's intention is for Listing 5.6 (stopatone.py) to begin at 0.0 and count up to 1.0 by tenths. In other words, the program should stop looping when the variable `x` becomes 1.0. With true mathematical real numbers, accumulating $\frac{1}{10}$ over ten iterations results in 1. Unfortunately, Listing 5.6 (stopatone.py) bypasses 1 and keeps going! The first few lines of its execution are

```
0.0
0.1
0.2
0.30000000000000004
0.4
0.5
0.6
0.7
0.7999999999999999
0.8999999999999999
0.9999999999999999
1.0999999999999999
1.2
1.3
1.4000000000000001
1.5000000000000002
```

The program never stops printing numbers. Note that $\frac{1}{10}$ does not have an exact internal binary floating-point representation. If you change the statement

```
x += 0.1
```

to be

```
x += 0.125
```

the program stops as expected. This is because $\frac{1}{8}$ has an exact internal binary floating-point representation. Since such exact representations of floating-point values are rare, you should avoid using the `==` and `!=` operators with floating-point numbers to control the number of loop iterations. Listing 5.7 (stopatonefixed.py) uses `<=` to control the loop instead of `!=`, and it behaves as expected.

Listing 5.7: stopatonefixed.py

```
x = 0.0
while x <= 1.0:
    print(x)
    x += 0.1
print("Done")
```

5.2 Definite Loops versus Indefinite Loops

In Listing 5.8 (definite1.py), code similar to Listing 5.1 (counttofive.py), prints the integers from one to 10.

Listing 5.8: definite1.py

```
n = 1
while n <= 10:
    print(n)
    n += 1
```

We can inspect the code and determine the exact number of iterations the loop will perform. This kind of loop is known as a *definite loop*, since we can predict exactly how many times the loop repeats. Consider Listing 5.9 (definite2.py).

Listing 5.9: definite2.py

```
n = 1
stop = int(input())
while n <= stop:
    print(n)
    n += 1
```

Looking at the source code of Listing 5.9 (definite2.py), we cannot predict how many times the loop will repeat. The number of iterations depends on the input provided by the user. However, at the program's point of execution after obtaining the user's input and before the start of the execution of the loop, we would be able to determine the number of iterations the `while` loop would perform. Because of this, the loop in Listing 5.9 (definite2.py) is considered to be a definite loop as well.

Compare these programs to Listing 5.10 (indefinite.py).

Listing 5.10: indefinite.py

```
done = False                # Enter the loop at least once
while not done:
    entry = int(input())     # Get value from user
    if entry == 999:        # Did user provide the magic number?
        done = True        # If so, get out
    else:
        print(entry)       # If not, print it and continue
```

In Listing 5.10 (`indefinite.py`), we cannot predict at any point during the loop's execution how many iterations the loop will perform. We know the value to match (999) before and during the loop, but the variable `entry` can be anything the user enters. The user could choose to enter 0 exclusively or enter 999 immediately and be done with it. The `while` statement in Listing 5.10 (`indefinite.py`) is an example of an *indefinite loop*.

Listing 5.5 (`troubleshootloop.py`) is another example of an indefinite loop.

The `while` statement is ideal for indefinite loops. Although we have used the `while` statement to implement definite loops, Python provides a better alternative for definite loops: the `for` statement.

5.3 The for Statement

The `while` loop is ideal for indefinite loops. As Listing 5.5 (`troubleshootloop.py`) demonstrated, a programmer cannot always predict how many times a `while` loop will execute. We have used a `while` loop to implement a definite loop, as in

```
n = 1
while n <= 10:
    print(n)
    n += 1
```

The `print` statement in this code executes exactly 10 times every time this code runs. This code requires three crucial pieces to manage the loop:

- initialization: `n = 1`
- check: `n <= 10`
- update: `n += 1`

Python provides a more convenient way to express a definite loop. The `for` statement iterates over a sequence of values. One way to express a sequence is via a tuple, as shown here:

```
for n in 1, 2, 3, 4, 5, 6, 7, 8, 9, 10:
    print(n)
```

This code behaves identically to the `while` loop above. The `print` statement here executes exactly 10 times. The code first prints 1, then 2, then 3, etc. The final value it prints is 10. During the iteration the variable `n` thus assumes, in order, all the values that make up the tuple.

It usually is cumbersome to explicitly list all the elements of a tuple, and often it is impractical. Consider iterating over all the integers from 1 to 1,000—writing out all the tuple's elements would be unwieldy. Fortunately, Python provides a convenient way to express a sequence of integers that follow a regular pattern. The following code uses a `range` expression to print the integers 1 through 10:

```
for n in range(1, 11):
    print(n)
```

The expression `range(1, 11)` creates a `range` object that allows the `for` loop to assign to the variable `n` the values 1, 2, ..., 10. Conceptually, the expression `range(1, 11)` represents the sequence of integers 1, 2, 3, 4, 5, 6, 7, 8, 9, 10.

The line

```
for n in range(1, 11):
```

is best read as “For each integer n in the range $1 \leq n < 11$.” During the first iteration of the loop, n ’s value is 1 within the block. In the loop’s second iteration, n has the value of 2. Each time through the loop, n ’s value increases by one. The code within the block will use the values of n up to 10.

The general form of the `range` expression is

`range( begin, end, step )`

where

- *begin* is the first value in the range; if omitted, the default value is 0
- *end* is **one past** the last value in the range; the *end* value is always required and may **not** be omitted
- *step* is the amount to increment or decrement; if the *step* parameter is omitted, it defaults to 1 (counts up by ones)

begin, *end*, and *step* must all be integer expressions; floating-point expressions and other types are not allowed. The arguments in the `range` expression may be literal numbers (like 10), variables (like x , if x is bound to an integer), and arbitrarily complex integer expressions.

The `range` expression is very flexible. Consider the following loop that counts down from 21 to 3 by threes:

```
for n in range(21, 0, -3):
    print(n, end=' ')
```

It prints

```
21 18 15 12 9 6 3
```

Thus `range(21, 0, -3)` represents the sequence 21, 18, 15, 12, 9, 3.

The expression `range(1000)` produces the sequence 0, 1, 2, ..., 999.

The following code computes and prints the sum of all the positive integers less than 100:

```
sum = 0      # Initialize sum
for i in range(1, 100):
    sum += i
print(sum)
```

The following examples show how to use `range` to produce a variety of sequences:

- `range(10)` $\rightarrow$ 0, 1, 2, 3, 4, 5, 6, 7, 8, 9
- `range(1, 10)` $\rightarrow$ 1, 2, 3, 4, 5, 6, 7, 8, 9
- `range(1, 10, 2)` $\rightarrow$ 1, 3, 5, 7, 9
- `range(10, 0, -1)` $\rightarrow$ 10, 9, 8, 7, 6, 5, 4, 3, 2, 1
- `range(10, 0, -2)` $\rightarrow$ 10, 8, 6, 4, 2

- `range(2, 11, 2) → 2,4,6,8,10`
- `range(-5, 5) → -5,-4,-3,-2,-1,0,1,2,3,4`
- `range(1, 2) → 1`
- `range(1, 1) → (empty)`
- `range(1, -1) → (empty)`
- `range(1, -1, -1) → 1,0`
- `range(0) → (empty)`

In a `range` expression with one argument, as in `range(x)`, the `x` represents the *end* of the range, with 0 being the implied *begin* value, and 1 being the *step* value.

In a `range` expression with two arguments, as in `range(x, y)`, the `x` represents the *begin* value, and `y` represents the *end* of the range. The implied *step* value is 1.

In a `range` expression with three arguments, as in `range(x, y, z)`, the `x` represents the *begin* value, `y` represents the *end* of the range, and `z` is the *step* value.

Loops allow us to rewrite an expanded form of Listing 2.20 (`powers10rightcommas.py`) more compactly. Listing 5.11 (`powers10loop.py`) uses a `for` loop to print the first 16 powers of 10.

Listing 5.11: `powers10loop.py`

```
for i in range(16):  
    print(f'{i:3} {10**i:22,}')
```

Listing 5.11 (`powers10loop.py`) prints

```
0          1  
1          10  
2         100  
3        1,000  
4       10,000  
5      100,000  
6     1,000,000  
7    10,000,000  
8   100,000,000  
9  1,000,000,000  
10 10,000,000,000  
11 100,000,000,000  
12 1,000,000,000,000  
13 10,000,000,000,000  
14 100,000,000,000,000  
15 1,000,000,000,000,000
```

In a `for` loop the `range` object has complete control over determining the loop variable each time through the loop. To prove this, Listing 5.12 (`abusefor.py`) attempts to thwart the `range`'s loop variable by changing its value inside the loop.

Listing 5.12: abusefor.py

```
# Abuse the for statement
for i in range(10):
    print(i, end= ' ') # Print i as served by the range object
    if i == 5:
        i = 20 # Change i inside the loop?
    print(f'({i})')
print()
```

Listing 5.12 (abusefor.py) prints the following:

```
0 (0)
1 (1)
2 (2)
3 (3)
4 (4)
5 (20)
6 (6)
7 (7)
8 (8)
9 (9)
```

The first number is `i`'s value at the beginning of the block, and the parenthesized number is `i`'s value at the end of the block before the next iteration. The code within the block can reassign `i`, but this binds `i` to a different integer object (20). The next time through the loop the `for` statement obtains the next integer served by the `range` object and binds `i` to this new integer.



If you look in older Python books or at online examples of Python code, you probably will encounter the `xrange` expression. Python 2 has both `range` and `xrange`, but Python 3 (the version we use in this text) does not have the `xrange` expression. The `range` expression in Python 3 is equivalent to the `xrange` expression in Python 2. The `range` expression in Python 2 creates a data structure called a *list*, and this process can involve considerable overhead for an executing program. The `xrange` expression in Python 2 avoids this overhead, making it more efficient than `range`, especially for a large sequence. When building loops with the `for` statement, Python 2 programmers usually use `xrange` rather than `range` to improve their code's efficiency. In Python 3, we can use `range` without compromising run-time performance. In Chapter 10 we will see it is easy to make a list out of a Python 3 `range` expression, so Python 3 does not need two different range expressions that do almost exactly the same thing.

We initially emphasize the `for` loop's ability to iterate over integer sequences because this is a useful and common task in software construction. The `for` loop, however, can iterate over any iterable object. As we have seen, a tuple is an iterable object, and a `range` object is an iterable object. A string also is an iterable object. We can use a `for` loop to iterate over the characters that comprise a string. Listing 5.13 (stringletters.py) uses a `for` loop to print the individual characters of a string.

Listing 5.13: stringletters.py

```
word = input('Enter a word: ')
for letter in word:
    print(letter)
```

In the following sample execution of Listing 5.13 (stringletters.py) shows how the program responds when the user enters the word *tree*:

```
Enter a word: tree
t
r
e
e
```

At each iteration of its `for` loop Listing 5.13 (stringletters.py) assigns to the `letter` variable a string containing a single character.

Listing 5.14 (stringliteralletters.py) uses a `for` loop to iterate over a literal string.

Listing 5.14: stringliteralletters.py

```
for c in 'ABCDEF':
    print(f'[{c}]', end='', sep='')
print()
```

Listing 5.14 (stringliteralletters.py) prints

```
[A][B][C][D][E][F]
```

Listing 5.15 (countvowels.py) counts the number of vowels in the text provided by the user.

Listing 5.15: countvowels.py

```
word = input('Enter text: ')
vowel_count = 0
for c in word:
    if c == 'A' or c == 'a' or c == 'E' or c == 'e' \
        or c == 'I' or c == 'i' or c == 'O' or c == 'o':
        print(c, end=' ') # Print the vowel
        vowel_count += 1 # Count the vowel
print(f' ({vowel_count} vowels)')
```

Listing 5.14 (stringliteralletters.py) prints vowels it finds and then reports how many it found:

```
Enter text: Mary had a little lamb.
a a a i e a (6 vowels)
```

Chapter 10 and beyond use the `for` statement to traverse data structures such as lists and dictionaries.

5.4 Nested Loops

As in the case of `if` statements, `while` and `for` blocks can contain arbitrary Python statements, including other loops. A loop can therefore be nested within another loop. To see how nested loops work, consider a program that prints out a *multiplication table*. Elementary school students use multiplication tables, or times tables, as they learn the products of integers up to 10 or even 12. Figure 5.3 shows a 10×10 multiplication table. We want our multiplication table program to be flexible and allow the user to specify the table's

Figure 5.3 A 10×10 multiplication table

×	1	2	3	4	5	6	7	8	9	10
1	1	2	3	4	5	6	7	8	9	10
2	2	4	6	8	10	12	14	16	18	20
3	3	6	9	12	15	18	21	24	27	30
4	4	8	12	16	20	24	28	32	36	40
5	5	10	15	20	25	30	35	40	45	50
6	6	12	18	24	30	36	42	48	54	60
7	7	14	21	28	35	42	49	56	63	70
8	8	16	24	32	40	48	56	64	72	80
9	9	18	27	36	45	54	63	72	81	90
10	10	20	30	40	50	60	70	80	90	100

size. We will begin our development work with a simple program and add features as we go. First, we will not worry about printing the table's row and column titles, nor will we print the lines separating the titles from the contents of the table. Initially we will print only the contents of the table. We will see we need a nested loop to print the table's contents, but that still is too much to manage in our first attempt. In our first attempt we will print the rows of the table in a very rudimentary manner. Once we are satisfied that our simple program works, we can add more features. Listing 5.16 (`timestable1.py`) shows our first attempt at a multiplication table.

Listing 5.16: `timestable1.py`

```
# Get the number of rows and columns in the table
size = int(input("Please enter the table size: "))
# Print a size x size multiplication table
for row in range(1, size + 1):
    print("Row #", row)
```

The output of Listing 5.16 (timestable1.py) is somewhat underwhelming:

```
Please enter the table size: 10
Row #1
Row #2
Row #3
Row #4
Row #5
Row #6
Row #7
Row #8
Row #9
Row #10
```

Listing 5.16 (timestable1.py) does indeed print each row in its proper place—it just does not supply the needed detail for each row. Our next step is to refine the way the program prints each row. Each row should contain `size` numbers. Each number within each row represents the product of the current row and current column; for example, the number in row 2, column 5 should be $2 \times 5 = 10$. In each row, therefore, we must vary the column number from 1 to `size`. Listing 5.17 (timestable2.py) contains the needed refinement.

Listing 5.17: timestable2.py

```
# Get the number of rows and columns in the table
size = int(input("Please enter the table size: "))
# Print a size x size multiplication table
for row in range(1, size + 1):
    for column in range(1, size + 1):
        product = row*column      # Compute product
        print(product, end=' ')   # Display product
    print()                       # Move cursor to next row
```

We use a loop to print the contents of each row. The outer loop controls how many total rows the program prints, and the inner loop, executed in its entirety each time the program prints a row, prints the individual elements that make up a row.

The result of Listing 5.17 (timestable2.py) is

```
Please enter the table size: 10
1 2 3 4 5 6 7 8 9 10
2 4 6 8 10 12 14 16 18 20
3 6 9 12 15 18 21 24 27 30
4 8 12 16 20 24 28 32 36 40
5 10 15 20 25 30 35 40 45 50
6 12 18 24 30 36 42 48 54 60
7 14 21 28 35 42 49 56 63 70
8 16 24 32 40 48 56 64 72 80
9 18 27 36 45 54 63 72 81 90
10 20 30 40 50 60 70 80 90 100
```

The numbers within each column are not lined up nicely, but the numbers are in their correct positions relative to each other. We can use an f-string to right justify the numbers within a four-digit area.

Listing 5.18 (timestable3.py) contains this alignment adjustment.

Listing 5.18: timestable3.py

```
# Get the number of rows and columns in the table
size = int(input("Please enter the table size: "))
# Print a size x size multiplication table
for row in range(1, size + 1):
    for column in range(1, size + 1):
        product = row*column                # Compute product
        print(f'{product:4}', end='') # Display product
    print()                                # Move cursor to next row
```

Listing 5.18 (timestable3.py) produces the table's contents in an attractive form:

```
Please enter the table size: 10
 1  2  3  4  5  6  7  8  9 10
 2  4  6  8 10 12 14 16 18 20
 3  6  9 12 15 18 21 24 27 30
 4  8 12 16 20 24 28 32 36 40
 5 10 15 20 25 30 35 40 45 50
 6 12 18 24 30 36 42 48 54 60
 7 14 21 28 35 42 49 56 63 70
 8 16 24 32 40 48 56 64 72 80
 9 18 27 36 45 54 63 72 81 90
10 20 30 40 50 60 70 80 90 100
```

Notice that the table presentation adjusts to the user's input:

```
Please enter the table size: 5
 1  2  3  4  5
 2  4  6  8 10
 3  6  9 12 15
 4  8 12 16 20
 5 10 15 20 25
```

A multiplication table of size 15 looks like

```
Please enter the table size: 15
 1  2  3  4  5  6  7  8  9 10 11 12 13 14 15
 2  4  6  8 10 12 14 16 18 20 22 24 26 28 30
 3  6  9 12 15 18 21 24 27 30 33 36 39 42 45
 4  8 12 16 20 24 28 32 36 40 44 48 52 56 60
 5 10 15 20 25 30 35 40 45 50 55 60 65 70 75
 6 12 18 24 30 36 42 48 54 60 66 72 78 84 90
 7 14 21 28 35 42 49 56 63 70 77 84 91 98 105
 8 16 24 32 40 48 56 64 72 80 88 96 104 112 120
 9 18 27 36 45 54 63 72 81 90 99 108 117 126 135
10 20 30 40 50 60 70 80 90 100 110 120 130 140 150
11 22 33 44 55 66 77 88 99 110 121 132 143 154 165
12 24 36 48 60 72 84 96 108 120 132 144 156 168 180
13 26 39 52 65 78 91 104 117 130 143 156 169 182 195
14 28 42 56 70 84 98 112 126 140 154 168 182 196 210
15 30 45 60 75 90 105 120 135 150 165 180 195 210 225
```

All that is left is to add the row and column titles and the lines that bound the edges of the table. Listing 5.19 (timestable4.py) adds the necessary code.

Listing 5.19: timestable4.py

```
# Get the number of rows and columns in the table
size = int(input("Please enter the table size: "))
# Print a size x size multiplication table
# First, print heading: 1 2 3 4 5 etc.
print('      ', end='')

# Print column heading
for column in range(1, size + 1):
    print(f'{column:4}', end='') # Display column number
print() # Go down to the next line

# Print line separator: +-----print(" +", end='')
print("  +", end='')
for column in range(1, size + 1):
    print('----', end='') # Display line
print() # Drop down to next line

# Print table contents
for row in range(1, size + 1):
    print(f'{row:3} |', end='') # Print heading for this row
    for column in range(1, size + 1):
        print(f'{row*column:4}', end='') # Display product
    print() # Move cursor to next row
```

When the user supplies the value 10, Listing 5.19 (timestable4.py) produces the following output:

```
Please enter the table size: 10
      1  2  3  4  5  6  7  8  9 10
    +-----+
  1 |  1  2  3  4  5  6  7  8  9 10
  2 |  2  4  6  8 10 12 14 16 18 20
  3 |  3  6  9 12 15 18 21 24 27 30
  4 |  4  8 12 16 20 24 28 32 36 40
  5 |  5 10 15 20 25 30 35 40 45 50
  6 |  6 12 18 24 30 36 42 48 54 60
  7 |  7 14 21 28 35 42 49 56 63 70
  8 |  8 16 24 32 40 48 56 64 72 80
  9 |  9 18 27 36 45 54 63 72 81 90
 10 | 10 20 30 40 50 60 70 80 90 100
```

An input of 15 yields

```

Please enter the table size: 15
      1  2  3  4  5  6  7  8  9 10 11 12 13 14 15
+-----+
1 | 1  2  3  4  5  6  7  8  9 10 11 12 13 14 15
2 | 2  4  6  8 10 12 14 16 18 20 22 24 26 28 30
3 | 3  6  9 12 15 18 21 24 27 30 33 36 39 42 45
4 | 4  8 12 16 20 24 28 32 36 40 44 48 52 56 60
5 | 5 10 15 20 25 30 35 40 45 50 55 60 65 70 75
6 | 6 12 18 24 30 36 42 48 54 60 66 72 78 84 90
7 | 7 14 21 28 35 42 49 56 63 70 77 84 91 98 105
8 | 8 16 24 32 40 48 56 64 72 80 88 96 104 112 120
9 | 9 18 27 36 45 54 63 72 81 90 99 108 117 126 135
10 | 10 20 30 40 50 60 70 80 90 100 110 120 130 140 150
11 | 11 22 33 44 55 66 77 88 99 110 121 132 143 154 165
12 | 12 24 36 48 60 72 84 96 108 120 132 144 156 168 180
13 | 13 26 39 52 65 78 91 104 117 130 143 156 169 182 195
14 | 14 28 42 56 70 84 98 112 126 140 154 168 182 196 210
15 | 15 30 45 60 75 90 105 120 135 150 165 180 195 210 225

```

If the user enters 7, the program prints

```

Please enter the table size: 7
      1  2  3  4  5  6  7
+-----+
1 | 1  2  3  4  5  6  7
2 | 2  4  6  8 10 12 14
3 | 3  6  9 12 15 18 21
4 | 4  8 12 16 20 24 28
5 | 5 10 15 20 25 30 35
6 | 6 12 18 24 30 36 42
7 | 7 14 21 28 35 42 49

```

The user even can enter a 1:

```

Please enter the table size: 1
      1
+----+
1 | 1

```

As we can see, the table automatically adjusts to the size and spacing required by the user's input.

This is how Listing 5.19 (timestable4.py) works:

- It is important to distinguish what is done only once (outside all loops) from that which is done repeatedly. The column heading across the top of the table is outside of all the loops; therefore, the program uses a loop to print it one time.
- The work to print the heading for the rows is distributed throughout the execution of the outer loop. This is because the heading for a given row cannot be printed until all the results for the previous row have been printed.
- The printing statement

```
print(f'{row*column:4}', end='') # Display product
```

right justifies the value of the product of the current row and current column in a field that is four characters wide. This technique properly aligns the columns within the times table.

- In the nested loop, `row` is the control variable for the outer loop; `column` controls the inner loop.
- The inner loop executes `size` times on every single iteration of the outer loop. This means the innermost statement

```
print(f'{row*column:4}', end='') # Display product
```

executes `size × size` times, one time for every product in the table.

- The program prints a newline after it displays the contents of each row; thus, all the values printed in the inner (`column`) loop appear on the same line.

Nested loops are necessary when an iterative process itself must be repeated. In our times table example, a `for` loop prints the contents of each row, and an enclosing `for` loop prints out each row.

Listing 5.20 (`permuteabc.py`) uses a triply-nested loop to print all the different arrangements of the letters *A*, *B*, and *C*. Each string printed is a *permutation* of *ABC*. A permutation is a possible ordering of the elements in a sequence.

Listing 5.20: `permuteabc.py`

```
# File permuteabc.py

# The first letter varies from A to C
for first in 'ABC':
    for second in 'ABC': # The second varies from A to C
        if second != first: # No duplicate letters allowed
            for third in 'ABC': # The third varies from A to C
                # Don't duplicate first or second letter
                if third != first and third != second:
                    print(first + second + third)
```

Notice how the `if` statements prevent duplicate letters within a given string. The output of Listing 5.20 (`permuteabc.py`) is all six permutations of *ABC*:

```
ABC
ACB
BAC
BCA
CAB
CBA
```

Listing 5.21 (`permuteabcd.py`) uses a four-deep nested loop to print all the different arrangements of the letters *A*, *B*, *C*, and *D*. Each string it prints is a permutation of *ABCD*.

Listing 5.21: permuteabcd.py

```
# File permuteabcd.py

# The first letter varies from A to D
for first in 'ABCD':
    for second in 'ABCD': # The second varies from A to D
        if second != first: # No duplicate letters allowed
            for third in 'ABCD': # The third varies from A to D
                # Don't duplicate first or second letter
                if third != first and third != second:
                    for fourth in 'ABCD': # The fourth varies from A to D
                        if fourth != first and fourth != second \
                            and fourth != third:
                            print(first + second + third + fourth)
```

When dealing with nested loops, proper indentation is especially important. The Python interpreter will detect illegal indentation, as shown in Listing 1.3 (badindent.py), but sometimes improper indentation simply changes the logic of the program. Consider Listing 5.22 (indentationloop.py).

Listing 5.22: indentationloop.py

```
for i in range(3):          #1a
    print(i, end=' ')       #2a
    for j in range(3):      #3a
        print(j, end=' ')   #4a
print()                    #5a

#-----

for i in range(3):          #1b
    print(i, end=' ')       #2b
for j in range(3):          #3b
    print(j, end=' ')       #4b
print()                    #5b
```

The lines labeled *#1a* through *#5a* contain the same code as lines *#1b* through *#5b*, but note that lines *#3b* and *#4b* are not indented at the same level as lines *#3a* and *#4a*. Both sections of the code represent legal Python indentation, so the interpreter will not indicate any indentation errors. Running the program shows that the two sections of code behave differently:

```
0 0 1 2 1 0 1 2 2 0 1 2
-----
0 1 2 0 1 2
```

The first section of code uses a nested loop—the second `for` statement is in the body of the first `for` statement. The second section of code contains two sequential `for` statements—the first `for` statement runs to completion before the second `for` statement begins executing.

Nested loops are powerful, and some novice programmers attempt to use nested loops where a single loop is more appropriate. Before you attempt to write a nested loop, ensure that a single, non-nested loop is insufficient to solve the problem. Nested loops are more difficult to write correctly and, when not necessary, they are less efficient than a simple loop.

5.5 Abnormal Loop Termination

Normally, a `while` statement executes until its condition becomes false. A running program checks this condition first to determine if it should execute the statements in the loop's body. It then re-checks this condition only after executing all the statements in the loop's body. Ordinarily a `while` loop will not immediately exit its body if its condition becomes false before completing all the statements in its body. The `while` statement is designed this way because usually the programmer intends to execute all the statements within the body as an indivisible unit. Sometimes, however, it is desirable to immediately exit the body or recheck the condition from the middle of the loop instead. Said another way, a `while` statement checks its condition only at the “top” of the loop. It is not the case that a `while` loop finishes immediately whenever its condition becomes true. Listing 5.23 (`whileexitatop.py`) demonstrates this top-exit behavior.

Listing 5.23: `whileexitatop.py`

```
x = 10
while x == 10:
    print('First print statement in the while loop')
    x = 5      # Condition no longer true; do we exit immediately?
    print('Second print statement in the while loop')
```

Listing 5.23 (`whileexitatop.py`) prints

```
First print statement in the while loop
Second print statement in the while loop
```

Even though the condition for continuing in the loop (`x` being equal to 10) changes in the middle of the loop's body, the `while` statement does not check the condition until it completes all the statements in its body and execution returns to the top of the loop.

Sometimes it is more convenient to exit a loop from the middle of its body; that is, quit the loop before all the statements in its body execute. This means if a certain condition becomes true in the loop's body, exit right away.

Similarly, a `for` statement typically iterates over all the values in its range or over all the characters in its string. Sometimes, however, it is desirable to exit the `for` loop prematurely. Python provides the `break` and `continue` statements to give programmers more flexibility designing the control logic of loops.

5.5.1 The `break` statement

As we noted above, sometimes it is necessary to exit a loop from the middle of its body; that is, quit the loop before all the statements in its body execute. This means if a certain condition becomes true in the loop's body, exit right away. This “middle-exiting” condition could be the same condition that controls the `while` loop (that is, the “top-exiting” condition), but it does not need to be.

Python provides the `break` statement to implement middle-exiting loop control logic. The `break` statement causes the program's execution to immediately exit from the body of the loop. Listing 5.24 (`addmiddleexit.py`) is a variation of Listing 5.4 (`addnonnegatives.py`) that illustrates the use of `break`.

Listing 5.24: addmiddleexit.py

```
# Allow the user to enter a sequence of nonnegative
# numbers. The user ends the list with a negative
# number. At the end the sum of the nonnegative
# numbers entered is displayed. The program prints
# zero if the user provides no nonnegative numbers.

entry = 0      # Ensure the loop is entered
sum = 0        # Initialize sum

# Request input from the user
print("Enter numbers to sum, negative number ends list:")

while True:    # Loop forever? Not really
    entry = int(input()) # Get the value
    if entry < 0:        # Is number negative number?
        break           # If so, exit the loop
    sum += entry         # Add entry to running sum
print("Sum =", sum)     # Display the sum
```

The condition of the `while` statement in Listing 5.24 (`addmiddleexit.py`) is a tautology, so when the program runs it is guaranteed to begin executing the statements in its `while` block at least once. Since the condition of the `while` can never be false, the `break` statement is the only way to get out of the loop. Here, the `break` statement executes only when the executing program determines that the number the user entered is negative. When the program encounters the `break` statement during its execution, it skips any statements that follow in the loop's body and exits the loop immediately. The keyword `break` means “break out of the loop.” The placement of the `break` statement in Listing 5.24 (`addmiddleexit.py`) makes it impossible to add a negative number to the `sum` variable.

Listing 5.5 (`troubleshootloop.py`) uses a variable named `done` that controls the duration of the loop. Listing 5.25 (`troubleshootloop2.py`) uses `break` statements in place of the Boolean `done` variable.

Listing 5.25: troubleshootloop2.py

```

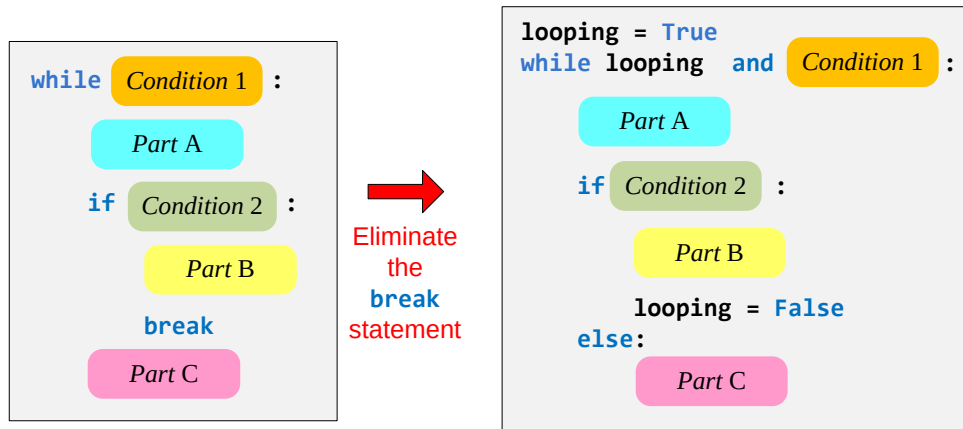
print("Help! My computer doesn't work!")
while True:
    print("Does the computer make any sounds (fans, etc.)")
    choice = input(" or show any lights? (y/n):")
    # The troubleshooting control logic
    if choice == 'n': # The computer does not have power
        choice = input("Is it plugged in? (y/n):")
        if choice == 'n': # It is not plugged in, plug it in
            print("Plug it in.")
        else: # It is plugged in
            choice = input("Is the switch in the \"on\" position? (y/n):")
            if choice == 'n': # The switch is off, turn it on!
                print("Turn it on.")
            else: # The switch is on
                choice = input("Does the computer have a fuse? (y/n):")
                if choice == 'n': # No fuse
                    choice = input("Is the outlet OK? (y/n):")
                    if choice == 'n': # Fix outlet
                        print("Check the outlet's circuit ")
                        print("breaker or fuse. Move to a")
                        print("new outlet, if necessary. ")
                    else: # Beats me!
                        print("Please consult a service technician.")
                        break # Nothing else I can do, exit loop
                else: # Check fuse
                    print("Check the fuse. Replace if ")
                    print("necessary.")
            else: # The computer has power
                print("Please consult a service technician.")
                break # Nothing else I can do, exit loop

```

Some software designers believe that programmers should use the `break` statement sparingly because it deviates from the normal loop control logic. Ideally, every loop should have a single entry point and single exit point. While Listing 5.24 (`addmiddleexit.py`) has a single exit point (the `break` statement), some programmers commonly use `break` statements within `while` statements in the which the condition for the `while` is not a tautology. Adding a `break` statement to such a loop adds an extra exit point (the top of the loop where the condition is checked is one point, and the `break` statement is another). Most programmers find two exits point perfectly acceptable, but much above two `break` points within a single loop is particularly dubious and you should avoid that practice.

The `break` statement is not absolutely required for full control over a `while` loop; that is, we can rewrite any Python program that contains a `break` statement within a `while` loop so that it behaves the same way but does not use a `break`. Figure 5.4 shows how we can transform any `while` loop that uses a `break` statement into a `break`-free version. The no-`break` version introduces a Boolean variable (`looping`), and the loop control logic is a little more complicated. The no-`break` version uses more memory (an extra variable) and more time to execute (requires an extra check in the loop condition during every iteration of the loop). This extra memory is insignificant, and except for rare, specialized applications, the extra execution time is imperceptible. In most cases, the more important issue is that the more complicated the control logic for a given section of code, the more difficult the code is to write correctly. In some situations, even though it violates the “single entry point, single exit point” principle, a simple `break` statement is a desirable loop control option.

Figure 5.4 The code on the left generically represents any `while` loop that uses a `break` statement. The code on the right shows how we can transform the loop into a functionally equivalent form that does not use `break`.



We can use the `break` statement inside a `for` loop as well. Listing 5.26 (`countvowelsnox.py`) shows how we can use a `break` statement to exit a `for` loop prematurely, provided by the user.

Listing 5.26: `countvowelsnox.py`

```
word = input('Enter text (no X\'s, please): ')
vowel_count = 0
for c in word:
    if c == 'A' or c == 'a' or c == 'E' or c == 'e' \
       or c == 'I' or c == 'i' or c == 'O' or c == 'o':
        print(c, ' ', end='') # Print the vowel
        vowel_count += 1 # Count the vowel
    elif c == 'X' or c == 'x':
        break
print(f' ({vowel_count} vowels)')
```

If the program detects an `X` or `x` anywhere in the user's input string, it immediately exits the `for` even though it may not have considered all the characters in the string. Consider the following sample run:

```
Enter text (no X's, please): Mary had a lixtle lamb.
a a a i (4 vowels)
```

The program breaks out of the loop when it attempts to process the `x` in the user's input.

The `break` statement is handy when a situation arises that requires immediate exit from a loop. The `for` loop in Python behaves differently from the `while` loop, in that it has no explicit condition that it checks to continue its iteration. We must use a `break` statement if we wish to prematurely exit a `for` loop before it completes its specified iterations. The `for` loop is a *definite loop*, which means programmers can determine up front the number of iterations the loop will perform. The `break` statement has the potential to disrupt this predictability. For this reason, programmers use `break` statements in `for` loops less frequently, and they often serve as an escape from a bad situation that continued iteration might make worse.

5.5.2 The continue Statement

When a program's execution encounters a `break` statement inside a loop, it skips the rest of the body of the loop and exits the loop. The `continue` statement is similar to the `break` statement, except the `continue` statement does not necessarily exit the loop. The `continue` statement skips the rest of the body of the loop and immediately checks the loop's condition. If the loop's condition remains true, the loop's execution resumes at the top of the loop. Listing 5.27 (`continueexample.py`) shows the `continue` statement in action.

Listing 5.27: `continueexample.py`

```
sum = 0
done = False
while not done:
    val = int(input("Enter positive integer (999 quits):"))
    if val < 0:
        print(f'Negative value {val} ignored')
        continue # Skip rest of body for this iteration
    if val != 999:
        print(f'Tallying {val}')
        sum += val
    else:
        done = (val == 999) # 999 entry exits loop
print(f'sum = {sum}')
```

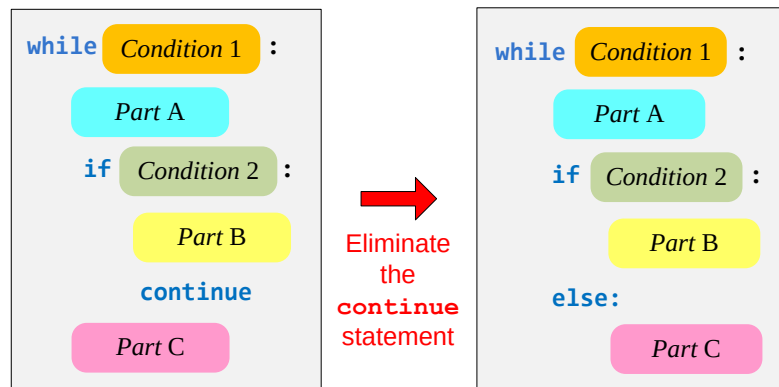
Programmers do not use the `continue` statement as frequently as the `break` statement since it is very easy to transform the code that uses `continue` into an equivalent form that does not. Listing 5.28 (`nocontinueexample.py`) works exactly like Listing 5.27 (`continueexample.py`), but it avoids the `continue` statement.

Listing 5.28: `nocontinueexample.py`

```
sum = 0
done = False
while not done:
    val = int(input("Enter positive integer (999 quits):"))
    if val < 0:
        print(f'Negative value {val} ignored')
    else:
        if val != 999:
            print(f'Tallying {val}')
            sum += val
        else:
            done = (val == 999) # 999 entry exits loop
print(f'sum = {sum}')
```

Figure 5.5 shows how we can rewrite any program that uses a `continue` statement into an equivalent form that does not use `continue`. The transformation is simpler than for `break` elimination (see Figure 5.4) since the loop's condition remains the same, and no additional variable is needed. The logic of the `else` version is no more complex than the `continue` version. Therefore, unlike the `break` statement above, there is no compelling reason to use the `continue` statement. Sometimes a programmer may add a `continue` statement at the last minute to an existing loop body to handle an exceptional condition (like ignoring negative numbers in the example above) that initially went unnoticed. If the body of the loop

Figure 5.5 The code on the left generically represents any loop that uses a `continue` statement. It is possible to transform the code on the left to eliminate the `continue` statement, as the code on the right shows.



is lengthy, the programmer can add a conditional statement with a `continue` near the top of the loop body without touching the logic of the rest of the loop. The `continue` statement thus merely provides a convenient alternative for the programmer. The `else` version is preferred.

5.6 while/else and for/else

Python loops support an optional `else` block. The `else` block in the context of a loop provides code to execute when the loop exits normally. Said another way, the code in a loop's `else` block does not execute if the loop terminates due to a `break` statement.

When a `while` loop exits due to its condition being false during its normal check, its associated `else` block executes. This is true even if its condition is found to be false before its body has had a chance to execute. Listing 5.29 (`whileelse.py`) shows how the `while/else` statement works.

Listing 5.29: `whileelse.py`

```
# Add five nonnegative numbers supplied by the user
count = sum = 0
print('Please provide five nonnegative numbers when prompted')
while count < 5:
    # Get value from the user
    val = float(input('Enter number: '))
    if val < 0:
        print('Negative numbers not acceptable! Terminating')
        break
    count += 1
    sum += val
else:
    print('Average =', sum/count)
```

When the user behaves and supplies only nonnegative values to Listing 5.29 (whileelse.py), it computes the average of the values provided:

```
Please provide five nonnegative numbers when prompted
Enter number: 23
Enter number: 12
Enter number: 14
Enter number: 10
Enter number: 11
Average = 14.0
```

When the user does not comply with the instructions, the program will print a corrective message and not attempt to compute the average:

```
Please provide five nonnegative numbers when prompted
Enter number: 23
Enter number: 12
Enter number: -4
Negative numbers not acceptable! Terminating
```

It may be more natural to read the `else` keyword for the `while` statement as “if no `break`,” meaning execute the code in the `else` block if the program’s execution of code in the `while` block did not encounter the `break` statement.

The `else` block is not essential; Listing 5.30 (whilenouse.py) uses `if/else` statement to achieve the same effect as Listing 5.29 (whileelse.py).

Listing 5.30: whilenouse.py

```
# Add five nonnegative numbers supplied by the user
count = sum = 0
print('Please provide five nonnegative numbers when prompted')
while count < 5:
    # Get value from the user
    val = float(input('Enter number: '))
    if val < 0:
        break
    count += 1
    sum += val
if count < 5:
    print('Negative numbers not acceptable! Terminating')
else:
    print('Average =', sum/count)
```

Listing 5.30 (whilenouse.py) uses two distinct Python constructs, the `while` statement followed by an `if/else` statement, whereas Listing 5.29 (whileelse.py) uses only one, a `while/else` statement. Listing 5.30 (whilenouse.py) also must check the `count < 5` condition twice, once in the `while` statement and again in the `if/else` statement.

A `for` statement with an `else` block works similarly to the `while/else` statement. When a `for/else` loop exits because it has considered all the values in its range or all the characters in its string, it executes the code in its associated `else` block. If a `for/else` statement exits prematurely due to a `break` statement, it does not execute the code in its `else` block. Listing 5.31 (countvowelse.py) shows how the `else` block works with a `for` statement.

Listing 5.31: countvowelselse.py

```
word = input('Enter text (no X\'s, please): ')
vowel_count = 0
for c in word:
    if c == 'A' or c == 'a' or c == 'E' or c == 'e' \
       or c == 'I' or c == 'i' or c == 'O' or c == 'o':
        print(c, end=' ') # Print the vowel
        vowel_count += 1 # Count the vowel
    elif c == 'X' or c == 'x':
        print('X not allowed')
        break
else:
    print(f' ({vowel_count} vowels)')
```

Unlike Listing 5.15 (`countvowels.py`), Listing 5.31 (`countvowelselse.py`), does not print the number of vowels if the user supplies text containing `X` or `x`.

5.7 Infinite Loops

In an infinite loop, the program's execution enters the loop but cannot escape. The user must resort to extraordinary means to terminate the program's execution. Programmers sometimes intentionally design infinite loops; for example, a web server application needs to run continuously checking for incoming connections. The Web server can perform this checking within a loop that runs indefinitely. Beginning programmers, unfortunately, all too often create infinite loops by accident, and these infinite loops represent logic errors in their programs.

Intentional infinite loops should be made obvious. For example,

```
while True:
    # Do something forever. . .
```

The Boolean literal `True` is always true, so it is impossible for the loop's condition to be false. The only ways to exit the loop is via a `break` statement, `return` statement (see Chapter 7), or a `sys.exit` call (see Chapter 6) embedded somewhere within its body.

Intentional infinite loops are easy to write correctly. Accidental infinite loops are quite common, but can be puzzling for beginning programmers to diagnose and repair. Consider Listing 5.32 (`findfactors.py`) that attempts to print all the integers with their associated factors from 1 to 20.

Listing 5.32: findfactors.py

```

# List the factors of the integers 1...MAX
MAX = 20 # MAX is 20
n = 1 # Start with 1
while n <= MAX: # Do not go past MAX
    factor = 1 # 1 is a factor of any integer
    print(f'{n}: ', end='') # Which integer are we examining?
    while factor <= n: # Factors are <= the number
        if n % factor == 0: # Test to see if factor is a factor of n
            print(factor, end=' ') # If so, display it
            factor += 1 # Try the next number
    print() # Move to next line for next n
    n += 1

```

It displays

```

1: 1
2: 1 2

```

and then "freezes up" or "hangs," ignoring any user input (except the key sequence **Ctrl C** on most systems which interrupts and terminates the running program). This type of behavior is a frequent symptom of an unintentional infinite loop. The factors of 1 display properly, as do the factors of 2. Before the program can display the next number (3), it hangs. Since the program is short, the problem may be easy to locate. In some programs, though, the error may be challenging to find. Even in Listing 5.32 (findfactors.py) the debugging task is nontrivial since the program involves nested loops. (Can you find and fix the problem in Listing 5.32 (findfactors.py) before reading further?)

In order to avoid infinite loops, we must ensure that the loop exhibits certain properties:

- The loop's condition must not be a tautology (a Boolean expression that can never be false). For example, the statement

```

while i >= 1 or i <= 10:
    # Block of code follows ...

```

would produce an infinite loop since any value chosen for *i* will satisfy one or both of the two subconditions. Perhaps the programmer intended to use **and** instead of **or** to stay in the loop as long as *i* remains in the range 1...10.

In Listing 5.32 (findfactors.py) the outer loop condition is

```
n <= MAX
```

If *n* is 21 and *MAX* is 20, then the condition is false. Since we can find values for *n* and *MAX* that make this expression false, it cannot be a tautology. Checking the inner loop condition:

```
factor <= n
```

we see that if *factor* is 3 and *n* is 2, then the expression is false; therefore, this expression also is not a tautology.

- The condition of a **while** must be true initially to gain access to its body. The code within the body must modify the state of the program in some way so as to influence the outcome of the condition that is checked at each iteration. This usually means the body must be able to modify one of the variables

used in the condition. Eventually the variable assumes a value that makes the condition false, and the loop terminates.

In Listing 5.32 (`findfactors.py`) the outer loop's condition involves the variables `n` and `MAX`. We observe that we assign 20 to `MAX` before the loop and never change it afterward, so to avoid an infinite loop it is essential that `n` be modified within the loop. Fortunately, the last statement in the body of the outer loop increments `n`. `n` is initially 1 and `MAX` is 20, so unless the circumstances arise to make the inner loop infinite, the outer loop eventually should terminate.

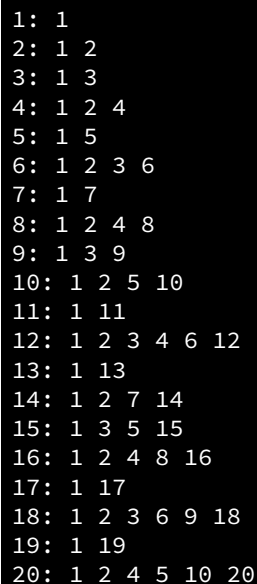
The inner loop's condition involves the variables `n` and `factor`. No statement in the inner loop modifies `n`, so it is imperative that `factor` be modified in the loop. The good news is `factor` is incremented in the body of the inner loop, but the bad news is the increment operation is protected within the body of the `if` statement. The inner loop contains one statement, the `if` statement. That `if` statement in turn has two statements in its body:

```
while factor <= n:
    if n % factor == 0:
        print(factor, end=' ')
        factor += 1
```

If the condition of the `if` is ever false, the variable `factor` will not change. In this situation if the expression `factor <= n` is ever true, it will remain true. This effectively creates an infinite loop. We must move the statement that modifies `factor` outside of the `if` statement's body:

```
while factor <= n:
    if n % factor == 0:
        print(factor, end=' ')
    factor += 1
```

This new version runs correctly:



```
1: 1
2: 1 2
3: 1 3
4: 1 2 4
5: 1 5
6: 1 2 3 6
7: 1 7
8: 1 2 4 8
9: 1 3 9
10: 1 2 5 10
11: 1 11
12: 1 2 3 4 6 12
13: 1 13
14: 1 2 7 14
15: 1 3 5 15
16: 1 2 4 8 16
17: 1 17
18: 1 2 3 6 9 18
19: 1 19
20: 1 2 4 5 10 20
```

We can use a tool called a debugger to step through a program to see where and why an infinite loop arises. Another common technique is to put print statements in strategic places to examine the values of the variables involved in the loop's control. We can augment the original inner loop in this way:

```

while factor <= n:
    print(f'factor={factor}, n={n}')
    if n % factor == 0:
        print(factor, end=' ')
        factor += 1    # <-- Note, still has original error here

```

It produces the following output:

```

1: factor=1, n=1
1
2: factor=1, n=2
1 factor=2, n=2
2
3: factor=1, n=3
1 factor=2, n=3
factor=2, n=3
factor=2, n=3
factor=2, n=3
.
.
.

```

The program continues to print the same line until the user interrupts its execution. The output demonstrates that once `factor` becomes equal to 2 and `n` becomes equal to 3 the program's execution becomes trapped in the inner loop. Under these conditions:

1. $2 < 3$ is true, so the loop continues and
2. $3 \% 2$ is equal to 1, so the `if` statement will not increment `factor`.

It is imperative that the program increment `factor` each time through the inner loop; therefore, the statement incrementing `factor` must be moved outside of the `if`'s guarded body. Moving it outside means removing it from the `if` statement's block, which means unindenting it.

Listing 5.33 (`findfactorsfor.py`) is a different version of our factor finder program that uses nested `for` loops instead of nested `while` loops. Not only is it slightly shorter, but it avoids the potential for the misplaced increment of the `factor` variable. This is because the `for` statement automatically handles the loop variable update.

Listing 5.33: `findfactorsfor.py`

```

# List the factors of the integers 1...MAX
MAX = 20 # MAX is 20
for n in range(1, MAX + 1): # Consider numbers 1...MAX
    print(f'{n}: ', end='') # Which integer are we examining?
    for factor in range(1, n + 1): # Try factors 1...n
        if n % factor == 0: # Test to see if factor is a factor of n
            print(factor, end=' ') # If so, display it
    print() # Move to next line for next n

```

5.8 Iteration Examples

We can implement some sophisticated algorithms in Python now that we are armed with conditional statements (`if`, `if/else`, and `if/elif/else`) and loops (`while` and `for`). This section provides several examples that show off the power of conditional execution and iteration.

5.8.1 Computing Square Root

Suppose you must write a Python program that computes the square root of a number supplied by the user. We can compute the square root of a number by using the following simple strategy:

1. Guess the square root.
2. Square the guess and see how close it is to the original number; if it is close enough to the correct answer, stop.
3. Make a new guess that will produce a better result and proceed with step 2.

Step 3 is a little vague, but Listing 5.34 (`computesquareroot.py`) implements the above strategy in Python, providing the missing details.

Listing 5.34: `computesquareroot.py`

```
# File computesquareroot.py

# Get value from the user
val = float(input('Enter number: '))

# Compute a provisional square root
root = 1.0

# How far off is our provisional root?
diff = root*root - val

# Loop until the provisional root is close enough to the actual root
while diff > 0.00000001 or diff < -0.00000001:
    print(f'{root} squared is {root*root}') # Report how we are doing
    root = (root + val/root) / 2 # Compute new provisional root
    # How bad is our current approximation?
    diff = root*root - val

# Report approximate square root
print(f'Square root of {val} = {root}')
```

The program is based on a simple algorithm that uses successive approximations to zero in on an answer that is within 0.00000001 of the true answer.

The following shows the program's output when the user enters the value 2:

```

Enter number: 2
1.0 squared is 1.0
1.5 squared is 2.25
1.416666666666665 squared is 2.006944444444444
1.4142156862745097 squared is 2.0000060073048824
Square root of 2 = 1.4142135623746899

```

The actual square root is approximately 1.4142135623730951 and so the result is within our accepted tolerance (0.00000001). Another run yields

```

Enter number: 100
1.0 squared is 1.0
50.5 squared is 2550.25
26.24009900990099 squared is 688.542796049407
15.025530119986813 squared is 225.76655538663093
10.840434673026925 squared is 117.51502390016438
10.032578510960604 squared is 100.6526315785885
10.000052895642693 squared is 100.0010579156518
Square root of 100 = 10.000000000139897

```

The real answer, of course, is 10, but our computed result again is well within our programmed tolerance.

While Listing 5.34 (`computesquareroot.py`) is a good example of the practical use of a loop, if we really need to compute the square root, Python has a library function that is more accurate and more efficient. We examine it and other handy mathematical functions in Chapter 6.

5.8.2 Drawing a Tree

Suppose we wish to draw a triangular tree with its height provided by the user. A tree that is five levels tall would look like

```

  *
 ***
*****
*****
*****

```

whereas a three-level tree would look like

```

  *
 ***
*****

```

If the height of the tree is fixed, we can write the program as a simple variation of Listing 1.2 (`arrow.py`) which uses only printing statements and no loops. It is more interesting, however, to vary the tree's height and width based on input from the user.

Listing 5.35 (`startree.py`) provides the necessary functionality.

Listing 5.35: startree.py

```

# Get tree height from user
height = int(input('Enter height of tree: '))

# Draw one row for every unit of height
row = 0
while row < height:
    # Print leading spaces; as row gets bigger, the number of
    # leading spaces gets smaller
    count = 0
    while count < height - row:
        print(end=' ')
        count += 1

    # Print out stars, twice the current row plus one:
    # 1. number of stars on left side of tree
    #    = current row value
    # 2. exactly one star in the center of tree
    # 3. number of stars on right side of tree
    #    = current row value
    count = 0
    while count < 2*row + 1:
        print(end='*')
        count += 1
    # Move cursor down to next line
    print()
    row += 1    # Consider next row

```

The following shows a sample run of Listing 5.35 (startree.py) where the user enters 7:

```

Enter height of tree: 7
      *
     ***
    *****
   *********
  ***********
 *****
*****

```

Listing 5.35 (startree.py) uses two sequential `while` loops nested within a `while` loop. The outer `while` loop draws one row of the tree each time its body executes:

- As long as the user enters a value greater than zero, the body of the outer `while` loop will execute; if the user enters zero or less, the program terminates and does nothing. This is the expected behavior.
- The last statement in the body of the outer `while`:

```
row += 1
```

ensures that the variable `row` increases by one each time through the loop; therefore, it eventually will equal `height` (since it initially had to be less than `height` to enter the loop), and the loop will terminate. There is no possibility of an infinite loop here.

The two inner loops play distinct roles:

- The first inner loop prints spaces. The number of spaces it prints is equal to the height of the tree the first time through the outer loop and decreases each iteration. This is the correct behavior since each succeeding row moving down contains fewer leading spaces but more asterisks.
- The second inner loop prints the row of asterisks that make up the tree. The first time through the outer loop, `row` is zero, so it prints no left side asterisks, one central asterisk, and no right side asterisks. Each time through the loop the number of left-hand and right-hand stars to print both increase by one, but there remains just one central asterisk to print. This means the tree grows one wider on each side for each line moving down. Observe how the $2 * \text{row} + 1$ value expresses the needed number of asterisks perfectly.
- While it seems asymmetrical, note that no third inner loop is required to print trailing spaces on the line after the asterisks are printed. The spaces would be invisible, so there is no reason to print them!

For comparison, Listing 5.36 (`startreefor.py`) uses `for` loops instead of `while` loops to draw our star trees. The `for` loop is a better choice for this program since once the user provides the height, the program can calculate exactly the number of iterations required for each loop. This number will not change during the rest of the program's execution, so the definite loop (`for`) is better a better choice than the indefinite loop (`while`).

Listing 5.36: `startreefor.py`

```
# Get tree height from user
height = int(input('Enter height of tree: '))

# Draw one row for every unit of height
for row in range(height):
    # Print leading spaces; as row gets bigger, the number of
    # leading spaces gets smaller
    for count in range(height - row):
        print(end=' ')

    # Print out stars, twice the current row plus one:
    # 1. number of stars on left side of tree
    #    = current row value
    # 2. exactly one star in the center of tree
    # 3. number of stars on right side of tree
    #    = current row value
    for count in range(2*row + 1):
        print(end='*')
    # Move cursor down to next line
    print()
```

5.8.3 Printing Prime Numbers

A *prime number* is an integer greater than one whose only factors (also called divisors) are one and itself. For example, 29 is a prime number (only 1 and 29 divide into 29 with no remainder), but 28 is not (1, 2, 4, 7, and 14 are factors of 28). Prime numbers were once merely an intellectual curiosity of mathematicians, but now they play an important role in cryptography and computer security.

The task is to write a program that displays all the prime numbers up to a value entered by the user. Listing 5.37 (`printprimes.py`) provides one solution.

Listing 5.37: printprimes.py

```

max_value = int(input('Display primes up to what value? '))
value = 2  # Smallest prime number
while value <= max_value:
    # See if value is prime
    is_prime = True  # Provisionally, value is prime
    # Try all possible factors from 2 to value - 1
    trial_factor = 2
    while trial_factor < value:
        if value % trial_factor == 0:
            is_prime = False  # Found a factor
            break  # No need to continue; it is NOT prime
        trial_factor += 1  # Try the next potential factor
    if is_prime:
        print(value, end= ' ')  # Display the prime number
        value += 1  # Try the next potential prime number
    print()  # Move cursor down to next line

```

Listing 5.37 (printprimes.py), with an input of 90, produces:

```

Display primes up to what value? 90
2 3 5 7 11 13 17 19 23 29 31 37 41 43 47 53 59 61 67 71 73 79 83 89

```

The logic of Listing 5.37 (printprimes.py) is a little more complex than that of Listing 5.35 (starttree.py). The user provides a value for `max_value`. The main loop (outer `while`) iterates over all the values from two to `max_value`:

- The program initializes the `is_prime` variable to true, meaning it assumes `value` is a prime number unless later tests prove otherwise. `trial_factor` takes on all the values from two to `value - 1` in the inner loop:

```

trial_factor = 2
while trial_factor < value:
    if value % trial_factor == 0:
        is_prime = False  # Found a factor
        break  # No need to continue; it is NOT prime
    trial_factor += 1  # Try the next potential factor

```

The expression `value % trial_factor` is zero when `trial_factor` divides into `value` with no remainder—exactly when `trial_factor` is a factor of `value`. If the program discovers a value of `trial_factor` that actually is a factor of `value`, then it sets `is_prime` false and exits the loop via the `break` statement. If the loop continues to completion, the program will not set `is_prime` to false, which means it found no factors, and, so, `value` is indeed prime.

- The `if` statement after the inner loop:

```

if is_prime:
    print(value, end= ' ')  # Display the prime number

```

simply checks the status of `is_prime`. If `is_prime` is true, then `value` must be prime, so the program prints `value` followed by a space to separate it from other factors that it may print during the remaining iterations.

Some important questions must be answered:

1. If the user enters a 2, what will the program print?

In this case `max_value = value = 2`, so the condition of the outer loop

```
value <= max_value
```

is true, since $2 \leq 2$. The executing program sets `is_prime` to true, but the condition of the inner loop

```
trial_factor < value
```

is not true (2 is not less than 2). Thus, the program skips the inner loop, it does not change `is_prime` from true, and so it prints 2. This behavior is correct because 2 is the smallest prime number (and the only even prime).

2. If the user enters a number less than 2, what will the program print?

The `while` condition ensures that values less than two are not considered. The program will never enter the body of the `while`. The program prints only the newline, and it displays no numbers. This behavior is correct, as there are no primes numbers less than 2.

3. Is the inner loop guaranteed to always terminate?

In order to enter the body of the inner loop, `trial_factor` must be less than `value`. `value` does not change anywhere in the loop. `trial_factor` is not modified anywhere in the `if` statement within the loop, and it is incremented within the loop immediately after the `if` statement. `trial_factor` is, therefore, incremented during each iteration of the loop. Eventually, `trial_factor` will equal `value`, and the loop will terminate.

4. Is the outer loop guaranteed to always terminate?

In order to enter the body of the outer loop, `value` must be less than or equal to `max_value`. `max_value` does not change anywhere in the loop. The last statement within the body of the outer loop increases `value`, and no where else does the program modify `value`. Since the inner loop is guaranteed to terminate as shown in the previous answer, eventually `value` will exceed `max_value` and the loop will end.

We can rearrange slightly the logic of the inner `while` to avoid the `break` statement. The current version is:

```
while trial_factor < value:
    if value % trial_factor == 0:
        is_prime = False      # Found a factor
        break                 # No need to continue; it is NOT prime
    trial_factor += 1         # Try the next potential factor
```

We can rewrite it as:

```
while is_prime and trial_factor < value:
    is_prime = (value % trial_factor != 0)  # Update is_prime
    trial_factor += 1                       # Try the next potential factor
```

This version without the `break` introduces a slightly more complicated condition for the `while` but removes the `if` statement within its body. `is_prime` is initialized to true before the loop. Each time through the loop it is reassigned. `trial_factor` will become false if at any time `value % trial_factor`

is zero. This is exactly when `trial_factor` is a factor of `value`. If `is_prime` becomes false, the loop cannot continue, and if `is_prime` never becomes false, the loop ends when `trial_factor` becomes equal to `value`. Because of operator precedence, the parentheses in

```
is_prime = (value % trial_factor != 0)
```

are not necessary. The parentheses do improve readability, since an expression including both `=` and `!=` is awkward for humans to parse. When parentheses are placed where they are not needed, as in

```
x = (y + 2)
```

the interpreter simply ignores them, so there is no efficiency penalty in the executing program.

We can shorten the code of Listing 5.37 (`printprimes.py`) a bit by using `for` statements instead of `while` statements as shown in Listing 5.38 (`printprimesfor.py`).

Listing 5.38: printprimesfor.py

```
max_value = int(input('Display primes up to what value? '))
# Try values from 2 (smallest prime number) to max_value
for value in range(2, max_value + 1):
    # See if value is prime
    is_prime = True # Provisionally, value is prime
    # Try all possible factors from 2 to value - 1
    for trial_factor in range(2, value):
        if value % trial_factor == 0:
            is_prime = False # Found a factor
            break # No need to continue; it is NOT prime
    if is_prime:
        print(value, end= ' ') # Display the prime number
print() # Move cursor down to next line
```

We can simplify Listing 5.38 (printprimesfor.py) even further by using the `for/else` statement as Listing 5.39 (printprimesforelse.py) illustrates.

Listing 5.39: printprimesforelse.py

```
max_value = int(input('Display primes up to what value? '))
# Try values from 2 (smallest prime number) to max_value
for value in range(2, max_value + 1):
    # See if value is prime: try all possible factors from 2 to value - 1
    for trial_factor in range(2, value):
        if value % trial_factor == 0:
            break # Found a factor, no need to continue; it is NOT prime
    else:
        print(value, end= ' ') # Display the prime number
print() # Move cursor down to next line
```

If the inner `for` loop completes its iteration over all the values in its range, it will execute the print statement in its `else` block. The only way the inner `for` loop can be interrupted is if it discovers a factor of `value`. If it does find a factor, the premature exit of the inner `for` loop prevents the execution of its `else` block. This logic enables it to print only prime numbers—exactly the behavior we want.

5.8.4 Insisting on the Proper Input

Listing 5.40 (betterinputonly.py) traps the user in a loop until the user provides an acceptable integer value.

Listing 5.40: betterinputonly.py

```
# Require the user to enter an integer in the range 1-10
in_value = 0    # Ensure loop entry
attempts = 0    # Count the number of tries

# Loop until the user supplies a valid number
while in_value < 1 or in_value > 10:
    in_value = int(input("Please enter an integer in the range 0-10: "))
    attempts += 1

# Make singular or plural word as necessary
tries = "try" if attempts == 1 else "tries"
# in_value at this point is guaranteed to be within range
print(f'It took you {attempts} {tries} to enter a valid number')
```

A sample run of Listing 5.40 (betterinputonly.py) produces

```
Please enter an integer in the range 0-10: 11
Please enter an integer in the range 0-10: 12
Please enter an integer in the range 0-10: 13
Please enter an integer in the range 0-10: 14
Please enter an integer in the range 0-10: -1
Please enter an integer in the range 0-10: 5
It took you 6 tries to enter a valid number
```

We initialize the variable `in_value` at the top of the program only to make sure the loop's body executes at least one time. A definite loop (`for`) is inappropriate for a program like Listing 5.40 (betterinputonly.py) because the program cannot determine ahead of time how many attempts the user will make before providing a value in range.

5.9 Exercises

1. In Listing 5.4 (addnonnegatives.py) could the condition of the `if` statement have used `>` instead of `>=` and achieved the same results? Why?
2. In Listing 5.4 (addnonnegatives.py) could the condition of the `while` statement have used `>` instead of `>=` and achieved the same results? Why?
3. In Listing 5.4 (addnonnegatives.py) what would happen if the statement

```
entry = int(input()) # Get the value
```

were moved out of the loop? Is moving the assignment out of the loop a good or bad thing to do? Why?
4. How many asterisks does the following code fragment print?

```
a = 0
while a < 100:
    print('*', end='')
    a += 1
print()
```

5. How many asterisks does the following code fragment print?

```
a = 0
while a < 100:
    print('*', end='')
    print()
```

6. How many asterisks does the following code fragment print?

```
a = 0
while a > 100:
    print('*', end='')
    a += 1
    print()
```

7. How many asterisks does the following code fragment print?

```
a = 0
while a < 100:
    b = 0
    while b < 55:
        print('*', end='')
        b += 1
    print()
    a += 1
```

8. How many asterisks does the following code fragment print?

```
a = 0
while a < 100:
    if a % 5 == 0:
        print('*', end='')
    a += 1
    print()
```

9. How many asterisks does the following code fragment print?

```
a = 0
while a < 100:
    b = 0
    while b < 40:
        if (a + b) % 2 == 0:
            print('*', end='')
        b += 1
    print()
    a += 1
```

10. How many asterisks does the following code fragment print?

```
a = 0
while a < 100:
    b = 0
    while b < 100:
```

```
    c = 0
    while c < 100:
        print('*', end='')
        c += 1
    b += 1
    a += 1
    print()
```

11. What is minimum number of arguments acceptable to the `range` expression?
12. What is maximum number of arguments acceptable to the `range` expression?
13. Provide the exact sequence of integers specified by each of the following `range` expressions.
 - (a) `range(5)`
 - (b) `range(5, 10)`
 - (c) `range(5, 20, 3)`
 - (d) `range(20, 5, -1)`
 - (e) `range(20, 5, -3)`
 - (f) `range(10, 5)`
 - (g) `range(0)`
 - (h) `range(10, 101, 10)`
 - (i) `range(10, -1, -1)`
 - (j) `range(-3, 4)`
 - (k) `range(0, 10, 1)`
14. What is a shorter way to express `range(0, 5, 1)`?
15. Provide an equivalent Python `range` expression for each of the following integer sequences.
 - (a) 1,2,3,4,5
 - (b) 5,4,3,2,1
 - (c) 5,10,15,20,25,30
 - (d) 30,25,20,15,10,5
 - (e) -3,-2,-1,0,1,2,3
 - (f) 3,2,1,0,-1,-2,-3
 - (g) -50,-40,-30,-20,-10
 - (h) *Empty sequence*
16. If `x` is bound to the integer value 2, what integer sequence does `range(x, 10*x, x)` represent?
17. If `x` is bound to the integer value 2 and `y` is bound to the integer 5, what integer sequence does `range(x, x + y)` represent?
18. Is it possible to represent the following sequence with a Python `range` expression: 1, -1, 2, -2, 3, -3, 4, -4?
19. How many asterisks does the following code fragment print?

```
for a in range(100):  
    print('*', end='')  
print()
```

20. How many asterisks does the following code fragment print?

```
for a in range(20, 100, 5):  
    print('*', end='')  
print()
```

21. How many asterisks does the following code fragment print?

```
for a in range(100, 0, -2):  
    print('*', end='')  
print()
```

22. How many asterisks does the following code fragment print?

```
for a in range(1, 1):  
    print('*', end='')  
print()
```

23. How many asterisks does the following code fragment print?

```
for a in range(-100, 100):  
    print('*', end='')  
print()
```

24. How many asterisks does the following code fragment print?

```
for a in range(-100, 100, 10):  
    print('*', end='')  
print()
```

25. Rewrite the code in the previous question so it uses a `while` instead of a `for`. Your code should behave identically.

26. What does the following code fragment print?

```
a = 0  
while a < 100:  
    print(a)  
    a += 1  
print()
```

27. Rewrite the code in the previous question so it uses a `for` instead of a `while`. Your code should behave identically.

28. What is printed by the following code fragment?

```
a = 0  
while a > 100:  
    print(a)  
    a += 1  
print()
```

29. Rewrite the following code fragment using a `break` statement and eliminating the `done` variable. Your code should behave identically to this code fragment.

```
done = False
n, m = 0, 100
while not done and n != m:
    n = int(input())
    if n < 0:
        done = True
    print("n =", n)
```

30. Rewrite the following code fragment so it does not use a `break` statement. Your code should behave identically to this code fragment.

```
remaining = 100
sum = 0
while True:
    in_val = int(input(f'Remaining {remaining}:'))
    if in_val == -1:
        break
    sum += in_val
    remaining -= 1
    print('sum =', sum)
# Code with break ...
```

31. Rewrite the following code fragment so it eliminates the `continue` statement. Your new code's logic should be simpler than the logic of this fragment.

```
x = 5
while x > 0:
    y = int(input())
    if y == 25:
        continue
    x -= 1
    print('x =', x)
```

32. What is printed by the following code fragment?

```
a = 0
while a < 100:
    print(a, end=' ')
    a += 1
print()
```

33. Write a Python program that accepts a single integer value entered by the user. If the value entered is less than one, the program prints nothing. If the user enters a positive integer, n , the program prints an $n \times n$ box drawn with $*$ characters. If the user enters 1, for example, the program prints

```
*
```

If the user enters a 2, it prints

```
**
**
```

An entry of three yields

```
***
***
***
```

and so forth. If the user enters 7, it prints

```
*****
*****
*****
*****
*****
*****
*****
```

that is, a 7×7 box of $*$ symbols.

34. Write a Python program that allows the user to enter exactly twenty floating-point values. The program then prints the sum, average (arithmetic mean), maximum, and minimum of the values entered.
35. Write a Python program that allows the user to enter any number of nonnegative floating-point values. The user terminates the input list with any negative value. The program then prints the sum, average (arithmetic mean), maximum, and minimum of the values entered. The terminating negative value is **not** used in the computations.
36. Redesign Listing 5.35 (startree.py) so that it draws a sideways tree pointing right; for example, if the user enters 7, the program would print

```
*
**
***
****
*****
*****
*****
*****
*****
*****
****
***
**
*
```


37. Redesign Listing 5.35 (`startree.py`) so that it draws a sideways tree pointing left; for example, if the user enters 7, the program would print

```
      *
     **
    ***
   ****
  *****
 *****
*****
*****
 *****
  *****
   ****
    ***
     **
      *
```

Chapter 6

Using Functions

Recall the square root code we wrote in Listing 5.34 (`computesquareroot.py`). In it we used a loop to compute the approximate square root of a value provided by the user.

While this code may be acceptable for many applications, better algorithms exist that work faster and produce more precise answers. Another problem with the code is this: What if you are working on a significant scientific or engineering application and must use different formulas in various parts of the source code, and each of these formulas involve square roots in some way? In mathematics, for example, we use square root to compute the distance between two geometric points (x_1, y_1) and (x_2, y_2) as

$$\sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

and, using the quadratic formula, the solution to the equation $ax^2 + bx + c = 0$ is

$$\frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$$

In electrical engineering and physics, the root mean square of a set of values $\{a_1, a_2, a_3, \dots, a_n\}$ is

$$\sqrt{\frac{a_1^2 + a_2^2 + a_3^2 + \dots + a_n^2}{n}}$$

Suppose we are writing one big program that, among many other things, needs compute distances and solve quadratic equations. Must we copy and paste the relevant portions of our square root code found in Listing 5.34 (`computesquareroot.py`) to each location in our source code that requires a square root computation? Also, what if we develop another program that requires computing a root mean square? Will we need to copy the code from Listing 5.34 (`computesquareroot.py`) into every program that needs to compute square roots, or is there a better way to package the square root code and reuse it?

One way to make code more reusable is by packaging it in *functions*. A function is a unit of reusable code. In Chapter 7 we will see how to write our own reusable functions, but in this chapter we examine some of the functions available in the Python standard library. Python provides a collection of standard functions stored in libraries called *modules*. Programmers can use the functions from these libraries within their own code to build sophisticated programs.

Figure 6.1 Conceptual view of the square root function. The function is like a black box—callers do not need to know the details of the code inside the function in order to use it.



6.1 Introduction to Using Functions

We have been using functions in Python since the first chapter. Functions in earlier chapters include `print`, `input`, `int`, `float`, `str`, `round`, `bin`, and `type`. The Python standard library includes many other functions useful for common programming tasks.

In mathematics, a *function* computes a result from a given value; for example, from the function definition $f(x) = 2x + 3$ we can compute $f(5) = 2 \cdot 5 + 3 = 13$ and $f(0) = 2 \cdot 0 + 3 = 3$. A function in Python works like a mathematical function. To introduce the function concept, we will look at the standard Python function that implements mathematical square root.

In Python, a function is a named block of code that performs a specific task. If an executing program needs to perform such a task, it calls upon the function to do the work. One example of a function is the mathematical square root function. Python has a function in its standard library named `sqrt` (see Section 6.4). The square root function accepts one numeric (integer or floating-point) value and produces a floating-point result; for example, $\sqrt{16} = 4$, so when presented with 16.0, `sqrt` responds with 4.0. Figure 6.1 illustrates the conceptual view of the `sqrt` function. The square root function is like a black box to the code that uses it. Callers do not need to know the details of the code inside the function in order to use it. Programmers are concerned more about *what* the function does, not *how* it does it.

This `sqrt` function is exactly what we need for our square root program, Listing 5.34 (`computesquareroot.py`). The new version, Listing 6.1 (`standardsquareroot.py`), uses the library function `sqrt`, eliminating the complex logic of the original code.

Listing 6.1: `standardsquareroot.py`

```
from math import sqrt

# Get value from the user
num = float(input('Enter number: '))

# Compute the square root
root = sqrt(num)

# Report result
print(f'Square root of {num} = {root}')
```

The expression
`sqrt(num)`

is a *function invocation*, also known as a *function call*. A function provides a service to the code that uses it. Here, our code in Listing 6.1 (`standardsquareroot.py`) is the *calling code*, or *client code*. Our code is the client that uses the service provided by the `sqrt` function. We say our code *calls*, or *invokes*, `sqrt` passing it the value of `num`. The expression `sqrt(num)` evaluates to the square root of the value of the variable `num`.

The functions we used in earlier chapters—`print`, `input`, `int`, `float`, `str`, `round`, `bin`, and `type`—are part of a standard library module named `__builtins__`. The `__builtins__` module contains many more functions that we have yet considered. All functions in the `__builtins__` module are automatically available to any Python program. The `sqrt` function is not part of the `__builtins__` module, so it is not automatically available to Python programs. The `sqrt` function is part of the `math` module in the standard library. One way to make the `sqrt` function available to a Python program is to import it as shown in Listing 6.1 (`standardsquareroot.py`):

```
from math import sqrt
```

This statement makes the `sqrt` function available for use in the program. The `import` keyword makes code in a module available to the interpreter. The `math` module has many other mathematical functions. These include trigonometric, logarithmic, hyperbolic, and other mathematical functions. This `import` statement will make *only* the `sqrt` function available to the program.

When calling a function, a pair of parentheses follow the function's name. Information that the function requires to perform its task must appear within these parentheses. In the expression

```
sqrt(num)
```

`num` is the information the function needs to do its work. We say `num` is the *argument*, or *parameter*, passed to the function. We also can say “we are passing `num` to the `sqrt` function.” The function uses the variable `num`'s value to perform the computation. Parameters enable callers to communicate information to a function during the function's execution.

The program could call the `sqrt` function in many other ways, as Listing 6.2 (`usingsqrt.py`) illustrates.

Listing 6.2: usingsqrt.py

```
# This program shows the various ways the
# sqrt function can be used.

from math import sqrt

x = 16
# Pass a literal value and display the result
print(sqrt(16.0))
# Pass a variable and display the result
print(sqrt(x))
# Pass an expression
print(sqrt(2 * x - 5))
# Assign result to variable
y = sqrt(x)
print(y)
# Use result in an expression
y = 2 * sqrt(x + 16) - 4
print(y)
# Use result as argument to a function call
y = sqrt(sqrt(256.0))
print(y)
print(sqrt(int('45')))
```

The `sqrt` function accepts a single numeric argument. As Listing 6.2 (`usingsqrt.py`) shows, the parameter that a caller can pass to `sqrt` can be a literal number, a numeric variable, an arithmetic expression, or even a function invocation that produces a numeric result.

Some functions, like `sqrt`, compute a value that it returns to the caller. The caller can use this returned value in various ways, as shown in Listing 6.2 (`usingsqrt.py`). The statement

```
print(sqrt(16.0))
```

directly prints the result of computing the square root of 16. The statement

```
y = sqrt(x)
```

assigns the result of the function call to the variable `y`. The statement

```
y = sqrt(sqrt(256.0))
```

computes $\sqrt{256}$ via the inner `sqrt` call and immediately passes the result to the outer `sqrt` call. This composition of function calls computes $\sqrt{\sqrt{256}} = \sqrt{16} = 4$, and the assignment operator binds the variable `y` to 4. The statement

```
print(sqrt(int('45')))
```

prints the result of computing the square root of the integer created from the string `'45'`.

If the calling code attempts to pass a parameter to a function that is incompatible with type expected by that function, the interpreter issues an error. Consider:

```
print(sqrt("16")) # Illegal, a string is not a number
```

In the interactive shell we get

```
>>> from math import sqrt
>>>
>>> sqrt(16)
4.0
>>> sqrt('16')
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: must be real number, not str
```

The `sqrt` function can process only numbers: integers and floating-point numbers. Even though we know we could convert the string parameter `'16'` to the integer 16 (with the `int` function) or to the floating-point value 16.0 (with the `float` function), the `sqrt` function does not automatically do this for us.

Listing 6.2 (`usingsqrt.py`) shows that a program can call the `sqrt` function as many times and in as many places as needed.

As noted in Figure 6.1, the square root function is a black box to the caller. The caller is concerned strictly about *what* the function does, not *how* the function accomplishes its task. We safely can treat all functions like black boxes. We can use the service that a function provides without being concerned about its internal details. We ordinarily can influence the function's behavior only via the parameters that we pass to it, and that nothing else we do can affect what the function does or how it does it. Furthermore, for the types of objects we have considered so far (integers, floating-point numbers, and strings), when a caller passes data to a function, the function cannot affect the caller's copy of that data. The caller is, however, free to use the return value of a function to modify any of its variables. The important distinction is that the caller is modifying its own variables—the function is not modifying the caller's variables. The following interactive sequence demonstrates that the `sqrt` function does not affect the object passed to it:

```
>>> from math import sqrt
>>> x = 2
>>> sqrt(x)
1.4142135623730951
>>> x
2
>>> x = sqrt(x)
>>> x
1.4142135623730951
```

Observe that passing `x` to the `sqrt` function did not change `x`, it still refers to the integer object 2. We must reassign `x` to the value that `sqrt` returns if we really wish to change `x` to be its square root.

Some functions take more than one parameter; for example, `print` can accept multiple parameters separated by commas.

From the caller's perspective a function has three important parts:

- **Name.** Every function has a name that identifies the code to be executed. Function names follow the same rules as variable names; a function name is another example of an identifier (see Section 2.3).
- **Parameters.** A function must be called with a certain number of parameters, and each parameter must be the correct type. Some functions like `print` permit callers to pass a variable number of arguments, but most functions, like `sqrt`, specify an exact number. If a caller attempts to call a function with too many or too few parameters, the interpreter will issue an error message and refuse to run the program. Consider the following misuse of `sqrt` in the interactive shell:

```
>>> sqrt(10)
3.1622776601683795
>>> sqrt()
Traceback (most recent call last):
  File "<pyshell#14>", line 1, in <module>
    sqrt()
TypeError: sqrt() takes exactly one argument (0 given)
>>> sqrt(10, 20)
Traceback (most recent call last):
  File "<pyshell#15>", line 1, in <module>
    sqrt(10, 20)
TypeError: math.sqrt() takes exactly one argument (2 given)
```

Similarly, if the parameters the caller passes are not compatible with the types specified for the function, the interpreter reports appropriate error messages:

```
>>> sqrt(16)
4.0
>>> sqrt("16")
Traceback (most recent call last):
  File "<pyshell#3>", line 1, in <module>
    sqrt("16")
TypeError: must be real number, not str
```

- **Result type.** A function returns a value to its caller. Generally a function will compute a result and return the value of the result to the caller. The caller's use of this result must be compatible with the function's specified result type. A function's result type and its parameter types can be completely unrelated. The `sqrt` function computes and returns a floating-point value; the interactive shell reports

```
>>> type(sqrt(16.0))
<class 'float'>
```

Some functions do not accept any parameters; for example, the function to generate a pseudorandom floating-point number, `random`, requires no arguments:

```
>>> from random import random
>>> random()
0.3557364474144479
```

The `random` function is part of the `random` module. The `random` function returns a floating-point value, but the caller does not pass the function any information to do its task. Any attempts to do so will fail:

```
>>> random(20)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: Random.random() takes no arguments (1 given)
```

Like mathematical functions that must produce a result, a Python function always produces a value to return to the caller. Some functions are not designed to produce any useful results. Clients call such a function for the effects provided by the executing code within a function, not for any value that the function computes. The `print` function is one such example. The `print` function displays text in the console

window; it does not compute and return a value to the caller. Since Python requires that all functions return a value, `print` must return something. Functions that are not meant to return anything return the special object `None`. We can show this in the Python shell:

```
>>> print(print(4))
4
None
```

The inner `print` call prints 4, and the outer `print` displays the return value of the inner `print` call. The following shows the exact type of the `None` object:

```
>>> type(print(4))
4
<class 'NoneType'>
```

We can assign the value `None` to any variable. It represents “nothing” or “no object.”

6.2 Functions and Modules

A Python *module* is simply a file that contains Python code. The name of the file dictates the name of the module; for example, a file named `math.py` contains the functions available from the standard `math` module. The modules of immediate interest to us are the standard modules that contain functions that our programs can use. The Python standard library contains thousands of functions distributed throughout more than 230 modules. These modules cover a wide range of application domains. One of the modules, known as the built-ins module (actual name `__builtins__`), contains all the functions we have been using in earlier chapters: `print`, `input`, etc. These built-in functions make up only a very small fraction of all the functions the standard library provides. Programmers must use one or more `import` statements within a program or within the interactive interpreter to gain access to the remaining standard functions.

The Python distribution for a given platform stores these standard modules somewhere on the computer’s hard drive. The interpreter knows where to locate these standard modules when an executing program needs to import them. It is not uncommon for a complex program to import a dozen separate modules to obtain all functions it needs to do its job.

Python provides a number of ways to import functions from a module. We will concentrate on the two most commonly used techniques. Section 6.10 briefly examines other, more specialized `import` statements.

Listing 6.1 (`standardsquareroot.py`) imported the square root function as follows:

```
from math import sqrt
```

A program that needs to compute square roots, common logarithms, and the trigonometric cosine function could use the following import statement:

```
from math import sqrt, log10, cos
```

This statement makes only the `sqrt`, `log10`, and `cos` functions from the `math` module available to the program. The `math` module offers many other mathematical functions—for example, the `atan` function that computes the arctangent—but this limited import statement does not provide these other definitions to the interpreter. Figure 6.2 shows the general form of this kind of import statement. Such an import statement is appropriate for smaller Python programs that use a small number of functions from a module. This kind of import statement allows callers to use an imported function’s simple name, as in

Figure 6.2 General form of a statement that imports a subset of a module’s available functions. The *function list* is a comma-separated list of function names to import.

`from` *module* `import` *function list*

Figure 6.3 General form of a statement that imports an entire module. The *module list* is a comma-separated list of module names to import.

`import` *module list*

```
y = sqrt(x)
```

If a program requires many different functions from a module, listing them all individually can become unwieldy. Python provides a way to import everything a module has to offer, as we will see in Section 6.10, but we also will see why this practice is not desirable.

Rather than importing one or more components of a module, we can import the entire module, as shown here:

```
import math
```

Figure 6.3 shows the general form of this kind of import statement. This import statement makes all of the functions of the module available to the program, but in order to use a function the caller must attach the module’s name during the call. The following code demonstrates the call notation:

```
y = math.sqrt(x)
print(math.log10(100))
```

Note the `math.` prefix attached to the calls of the `sqrt` and `log10` functions. We call a composite name (*module-name.function-name*) like this a *qualified name*. The qualified name includes the module name and function name connected by a dot (`.`). Many programmers prefer this approach because the complete name unambiguously identifies the function with its module. A large, complex program could import the `math` module and a different, third-party module called `extramath`. Suppose the `extramath` module provided its own `sqrt` function. There can be no mistaking the fact that the `sqrt` being called in the expression `math.sqrt(16)` is the one provided by the `math` module. It is impossible for a program to import the `sqrt` functions separately from both modules and use their simple names simultaneously within a program. Does

```
y = sqrt(x)
```

intend to use `math`’s `sqrt` or `extramath`’s `sqrt`?

Note that a statement such as

```
from math import sqrt
```

does not import the entire module; specifically, code under this import statement may use only the simple name, `sqrt`, and cannot use the qualified name, `math.sqrt`.

As programs become larger and more complex, the import entire module approach becomes more compelling. The qualified function names improve the code's readability and avoids name clashes between modules that provide functions with identical names. Soon we will be writing our own, custom functions. Qualified names ensure that names we create ourselves will not clash with any names that modules may provide.

6.3 The Built-in Functions

Section 6.1 observed that we have been using functions in Python since the first chapter. These functions include `print`, `input`, `int`, `float`, `str`, `round`, `bin`, and `type`. These functions and many others reside in a module named `__builtins__`. The `__builtins__` module is special because its components are automatically available to any Python program—no `import` statement is required. The full name of the `print` function is `__builtins__.print`, although chances are you will never see its full name written in a Python program. We can verify its fully qualified name in the interpreter:

```
>>> print('Hi')
Hi
>>> __builtins__.print('Hi')
Hi
>>> id(print)
1914442507472
>>> id(__builtins__.print)
1914442507472
```

This interactive sequence verifies that the names `print` and `__builtins__.print` refer to the same function object. The `id` function is another `__builtins__` function. The expression `id(x)` evaluates to the address in memory of object named `x`. Since `id(print)` and `id(__builtins__.print)` evaluate to the same value, we know both names correspond to the same function object.

The `dir` function, which stands for *directory*, reveals all the components that a module has to offer. The following interactive sequence prints the `__builtins__` components:

```
>>> dir(__builtins__)
['ArithmeticError', 'AssertionError', 'AttributeError', 'BaseException',
'BaseExceptionGroup', 'BlockingIOError', 'BrokenPipeError', 'BufferError',
'BytesWarning', 'ChildProcessError', 'ConnectionAbortedError',
'ConnectionError', 'ConnectionRefusedError', 'ConnectionResetError',
'DeprecationWarning', 'EOFError', 'Ellipsis', 'EncodingWarning',
'EnvironmentError', 'Exception', 'ExceptionGroup', 'False',
'FileExistsError', 'FileNotFoundError', 'FloatingPointError',
'FutureWarning', 'GeneratorExit', 'IOError', 'ImportError',
'ImportWarning', 'IndentationError', 'IndexError', 'InterruptedError',
'IsADirectoryError', 'KeyError', 'KeyboardInterrupt', 'LookupError',
'MemoryError', 'ModuleNotFoundError', 'NameError', 'None',
'NotADirectoryError', 'NotImplemented', 'NotImplementedError',
'OSError', 'OverflowError', 'PendingDeprecationWarning', 'PermissionError',
'ProcessLookupError', 'RecursionError', 'ReferenceError',
'ResourceWarning', 'RuntimeError', 'RuntimeWarning', 'StopAsyncIteration',
'StopIteration', 'SyntaxError', 'SyntaxWarning', 'SystemError',
'SystemExit', 'TabError', 'TimeoutError', 'True', 'TypeError',
'UnboundLocalError', 'UnicodeDecodeError', 'UnicodeEncodeError',
'UnicodeError', 'UnicodeTranslateError', 'UnicodeWarning', 'UserWarning',
'ValueError', 'Warning', 'WindowsError', 'ZeroDivisionError',
'__build_class__', '__debug__', '__doc__', '__import__', '__loader__',
'__name__', '__package__', '__spec__', 'abs', 'aiter', 'all', 'anext',
'any', 'ascii', 'bin', 'bool', 'breakpoint', 'bytearray', 'bytes',
'callable', 'chr', 'classmethod', 'compile', 'complex', 'copyright',
'credits', 'delattr', 'dict', 'dir', 'divmod', 'enumerate', 'eval',
'exec', 'exit', 'filter', 'float', 'format', 'frozenset', 'getattr',
'globals', 'hasattr', 'hash', 'help', 'hex', 'id', 'input', 'int',
'isinstance', 'issubclass', 'iter', 'len', 'license', 'list', 'locals',
'map', 'max', 'memoryview', 'min', 'next', 'object', 'oct', 'open', 'ord',
'pow', 'print', 'property', 'quit', 'range', 'repr', 'reversed', 'round',
'set', 'setattr', 'slice', 'sorted', 'staticmethod', 'str', 'sum', 'super',
'tuple', 'type', 'vars', 'zip']
```

A module can contain other things besides functions. Most of the names in the first 18 lines or so of the `__builtins__` module's directory listing are types the module defines, and most of the names in the last 11 lines are functions it provides. The list contains many recognizable names: `dir`, `bool`, `float`, `id`, `input`, `int`, `print`, `range`, `round`, `str`, and `type` functions.

The `__builtins__` module provides a common core of general functions useful to any Python program regardless of its application area. The other standard modules that Python provides are aimed at specific application domains, such as mathematics, text processing, file processing, system administration, graphics, networking, and multimedia. Programs that require more domain-specific functionality must import the appropriate modules that provide the needed services.

The `__builtins__` module includes a `help` function. In the interactive interpreter we can use the `help` function to print human-readable information about specific functions in the current namespace. The following interactive sequence shows how `help` works:

```
>>> help(print)
Help on built-in function print in module builtins:

print(*args, sep=' ', end='\n', file=None, flush=False)
    Prints the values to a stream, or to sys.stdout by default.

    sep
        string inserted between values, default a space.
    end
        string appended after the last value, default a newline.
    file
        a file-like object (stream); defaults to the current sys.stdout.
    flush
        whether to forcibly flush the stream.
>>> help(math.sqrt)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
NameError: name 'math' is not defined. Did you forget to import 'math'?
>>> import math
>>> help(math.sqrt)
Help on built-in function sqrt in module math:

sqrt(x, /)
    Return the square root of x.
```

Notice that `help` was powerless to provide information about `math.sqrt` function until we imported the `math` module.

6.4 Standard Mathematical Functions

The standard `math` module provides much of the functionality of a scientific calculator. Table 6.1 lists only a few of the available functions.

Table 6.1 A few of the functions from the `math` module

math Module	
<code>sqrt</code>	Computes the square root of a number: $\text{sqrt}(x) = \sqrt{x}$
<code>exp</code>	Computes e raised a power: $\text{exp}(x) = e^x$
<code>log</code>	Computes the natural logarithm of a number: $\text{log}(x) = \log_e x = \ln x$
<code>log10</code>	Computes the common logarithm of a number: $\text{log}(x) = \log_{10} x = \log x$
<code>log2</code>	Computes the base-2 logarithm of a number: $\text{log2}(x) = \log_2 x$
<code>cos</code>	Computes the cosine of a value specified in radians: $\text{cos}(x) = \cos x$; other trigonometric functions include sine, tangent, arc cosine, arc sine, arc tangent, hyperbolic cosine, hyperbolic sine, and hyperbolic tangent
<code>pow</code>	Raises one number to a power of another: $\text{pow}(x, y) = x^y$
<code>degrees</code>	Converts a value in radians to degrees: $\text{degrees}(x) = \frac{\pi}{180}x$
<code>radians</code>	Converts a value in degrees to radians: $\text{radians}(x) = \frac{180}{\pi}x$
<code>fabs</code>	Computes the absolute value of a number: $\text{fabs}(x) = x $

The `math` module also defines the values `pi` (π) and `e` (e):

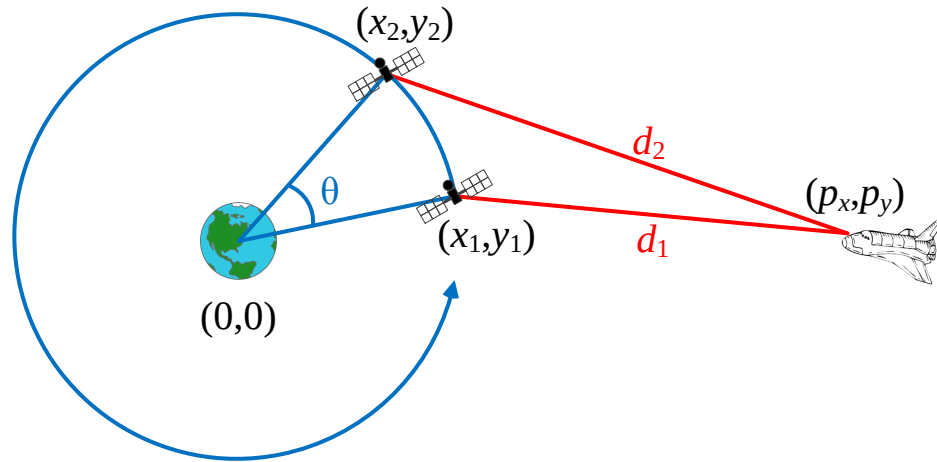
```
>>> import math
>>> math.pi
3.141592653589793
>>> math.e
2.718281828459045
```

The following interactive sequence reveals the `math` module's full directory of components:

```
>>> import math
>>> dir(math)
['__doc__', '__loader__', '__name__', '__package__', '__spec__', 'acos',
'acosh', 'asin', 'asinh', 'atan', 'atan2', 'atanh', 'ceil', 'copysign',
'cos', 'cosh', 'degrees', 'e', 'erf', 'erfc', 'exp', 'expm1', 'fabs',
'factorial', 'floor', 'fmod', 'frexp', 'fsum', 'gamma', 'hypot',
'isfinite', 'isinf', 'isnan', 'ldexp', 'lgamma', 'log', 'log10', 'log1p',
'log2', 'modf', 'pi', 'pow', 'radians', 'sin', 'sinh', 'sqrt', 'tan',
'tanh', 'trunc']
>>>
```

All of the names in the directory except those that begin with `__` represent mathematical functions. A Python program that uses any of these mathematical functions must import the `math` module and use the fully-qualified function names or use the `from...import` syntax to import individual functions.

Figure 6.4 Orbital distance problem. In this diagram, the satellite begins at point (x_1, y_1) , a distance of d_1 from the spacecraft. The satellite's orbit takes it to point (x_2, y_2) after an angle of θ rotation. The distance to its new location is d_2 .



The parameter passed by the caller is known as the *actual* parameter. The parameter specified by the function is called the *formal* parameter. During a function call the first actual parameter is assigned to the first formal parameter, the second actual parameter is assigned to the second formal parameter, etc. Callers must be careful to put the arguments they pass in the proper order when calling a function; for example, the call `math.pow(10, 2)` computes $10^2 = 100$, but the call `math.pow(2, 10)` computes $2^{10} = 1,024$.

The functions in the `math` module are ideal for solving problems like the one shown in Figure 6.4. Suppose a spacecraft is at a fixed location in space some distance from a planet. A satellite is orbiting the planet in a circular orbit. We wish to compute how much farther away the satellite will be from the spacecraft when it has progressed θ degrees along its orbital path.

We will let the origin of our coordinate system $(0,0)$ be located at the center of the planet. This location corresponds also to the center of the satellite's circular orbital path. The satellite is located as some point, (x, y) and the spacecraft is stationary at point (p_x, p_y) . The spacecraft is located in the same plane as the satellite's orbit. We wish to compute the distances between the moving point (satellite) and the fixed point (spacecraft) as the satellite orbits the planet.

Facts from mathematics provide solutions to the following two problems:

1. **Problem:** We must recompute the location of the moving point as it moves along the circle.

Solution: Given an initial position (x, y) of a point, a rotation of θ degrees around the origin will yield a new point at (x', y') , where

$$\begin{aligned} x' &= x \cos \theta - y \sin \theta \\ y' &= x \sin \theta + y \cos \theta \end{aligned}$$

2. **Problem:** We must recalculate the distance between the moving point and the fixed point as the moving point moves to a new position.

Solution: The distance d in Figure 6.4 between the two points (p_x, p_y) and (x, y) is given by the

formula

$$d = \sqrt{(x - p_x)^2 + (y - p_y)^2}$$

Listing 6.3 (orbitdist.py) uses these mathematical results to compute a table of distances that span a complete orbit of the satellite.

Listing 6.3: orbitdist.py

```
# Use some functions and values from the math module
from math import sqrt, sin, cos, pi, radians

# Get coordinates of the stationary spacecraft, (px, py)
px = float(input("Enter x coordinate of spacecraft: "))
py = float(input("Enter y coordinate of spacecraft: "))

# Get starting coordinates of satellite, (x1, y1)
x = float(input("Enter initial satellite x coordinate: "))
y = float(input("Enter initial satellite y coordinate: "))

# Convert 60 degrees to radians to be able to use the
# trigonometric functions
rads = radians(60)

# Precompute the cosine and sine of the angle
COS_theta = cos(rads)
SIN_theta = sin(rads)

# Make a complete revolution (6*60 = 360 degrees)
for increment in range(0, 7):
    # Compute the distance to the satellite
    dist = sqrt((px - x)*(px - x) + (py - y)*(py - y))
    print(f'Distance to satellite {dist:10.2f} km')
    # Compute the satellite's new (x, y) location after rotating by 60 degrees
    x, y = x*COS_theta - y*SIN_theta, x*SIN_theta + y*COS_theta
```

Listing 6.3 (orbitdist.py) prints the distances from the spacecraft to the satellite in 60-degree orbit increments. A sample run of Listing 6.3 (orbitdist.py) looks like

```
Enter x coordinate of spacecraft: 100000
Enter y coordinate of spacecraft: 0
Enter initial satellite x coordinate: 20000
Enter initial satellite y coordinate: 0
Distance to satellite 80000.00 km
Distance to satellite 91651.51 km
Distance to satellite 111355.29 km
Distance to satellite 120000.00 km
Distance to satellite 111355.29 km
Distance to satellite 91651.51 km
Distance to satellite 80000.00 km
```

Here, the user first enters the coordinates for the point (100,000,0) and then the coordinates for the point (20,000,0). Observe that the satellite begins 80,000 km away from the spacecraft and the distance increases to a maximum of 120,000 km when it is at the far side of its orbit. Eventually the satellite returns to its starting place ready for the next orbit.

Listing 6.3 (orbitdist.py) uses tuple assignment to update the *x* and *y* variables:

```
# Uses tuple assignment
x, y = x*COS_theta - y*SIN_theta, x*SIN_theta + y*COS_theta
```

If we instead used two separate assignment statements, we must be careful—the following code does *not* work the same way:

```
# Does not work correctly
x = x*COS_theta - y*SIN_theta
y = x*SIN_theta + y*COS_theta
```

This is because the value of *x* used in the second assignment statement is the *new* value of *x* computed by the first assignment statement. The tuple assignment version uses the original *x* value in both computations. If we really wanted to use two assignment statements rather than a single tuple assignment, we would need to introduce an extra variable so we do not lose *x*'s original value:

```
new_x = x*COS_theta - y*SIN_theta # Compute new x value
y = x*SIN_theta + y*COS_theta    # Compute new y value using original x
x = new_x                       # Update x
```

We can use the square root function to improve the efficiency of Listing 5.37 (printprimes.py). Instead of trying all the potential factors of *n* up to *n* − 1, we need only try potential factors up to $\sqrt{n}$. Listing 6.4 (moreefficientprimes.py) uses the `sqrt` function to reduce the number of potential factors the program needs to consider.

Listing 6.4: moreefficientprimes.py

```
from math import sqrt

max_value = int(input('Display primes up to what value? '))
value = 2 # Smallest prime number

while value <= max_value:
    # See if value is prime
    is_prime = True # Provisionally, value is prime
    # Try all possible factors from 2 to the square root of value
    trial_factor = 2
    root = round(sqrt(value))
    while trial_factor <= root:
        if value % trial_factor == 0:
            is_prime = False # Found a factor
            break # No need to continue; it is NOT prime
        trial_factor += 1 # Try the next potential factor
    if is_prime:
        print(value, end= ' ') # Display the prime number
        value += 1 # Try the next potential prime number

print() # Move cursor down to next line
```

In the case of 1,000,003, for example, the inner loop of Listing 5.37 (printprimes.py) performs 1,000,001 iterations, while the inner loop of Listing 6.4 (moreefficientprimes.py) iterates only 999 times. The version using the square root function is approximately 1,001 times faster!

Note that we can achieve the same effect without the `sqrt` function if we change the condition of the `while` statement from


```
trial_factor <= root
```

to

```
trial_factor * trial_factor <= value
```

This version performs a multiplication each time it checks the loop condition, while the square root version does not.

6.5 time Functions

The `time` module contains a number of functions that relate to time. We will consider two: `perf_counter_ns` and `sleep`.

time.perf_counter_ns. The `time.perf_counter_ns` function allows us measure elapsed time as if we were using a stopwatch or looking at a clock. The `time.perf_counter_ns` function returns an integer value representing a point in time. Its return value by itself is not too useful; it is only after a second call to `time.perf_counter_ns` that we can extract any useful information. The difference between the first call to `time.perf_counter_ns` and the second call to `time.perf_counter_ns` represents an elapsed time in nanoseconds; thus, with two calls to the `time.perf_counter_ns` function we can measure *elapsed time*.

Listing 6.5 (`timeit.py`) measures how long it takes a user to enter a character from the keyboard.

Listing 6.5: `timeit.py`

```
from time import perf_counter_ns

print("Enter your name: ", end="")
start_time = perf_counter_ns()
name = input()
elapsed = (perf_counter_ns() - start_time)/1_000_000_000
print(f'{name}, it took you {elapsed} seconds to respond')
```

The following represents the program's interaction with a particularly slow typist:

```
Enter your name: Rick
Rick, it took you 2.802725699999428 seconds to respond
```

Listing 6.6 (`timeaddition.py`) measures the time it takes for a Python program to add up all the integers from 1 to 100,000,000.

Listing 6.6: `timeaddition.py`

```
from time import perf_counter_ns

sum = 0                # Initialize sum accumulator
start = perf_counter_ns() # Start the stopwatch
for n in range(1, 100000001): # Sum the numbers
    sum += n
elapsed = perf_counter_ns() - start # Stop the stopwatch
print(f'sum: {sum}, time: {elapsed/1_000_000_000} seconds') # Report results
```

On one system Listing 6.6 (timeaddition.py) reports

```
sum: 5000000050000000, time: 11.5352314 seconds
```

Listing 6.7 (measureprimespeed.py) measures how long it takes a program to count all the prime numbers up to 10,000 using the same algorithm as Listing 5.38 (printprimesfor.py).

Listing 6.7: measureprimespeed.py

```
from time import perf_counter_ns

max_value = 10000
count = 0

start_time = perf_counter_ns() # Start timer
# Try values from 2 (smallest prime number) to max_value
for value in range(2, max_value + 1):
    # See if value is prime
    is_prime = True # Provisionally, value is prime
    # Try all possible factors from 2 to value - 1
    for trial_factor in range(2, value):
        if value % trial_factor == 0:
            is_prime = False # Found a factor
            break # No need to continue; it is NOT prime
    if is_prime:
        count += 1 # Count the prime number
elapsed = (perf_counter_ns() - start_time)/1_000_000_000 # Stop the timer

print(f'Count: {count}, Elapsed time: {elapsed} sec')
```

On one system, the program produces

```
Count: 1229, Elapsed time: 0.5670676 sec
```

Repeated runs consistently report an execution time of approximately 0.5 seconds to count all the prime numbers up to 10,000. By comparison, Listing 6.8 (timemoreefficientprimes.py), based on the algorithm in Listing 6.4 (moreefficientprimes.py) using the square root optimization runs on average over 20 times faster. A sample run shows

```
Count: 1229, Elapsed time: 0.0268069 sec
```

Exact times will vary depending on the speed of the computer.

Listing 6.8: timemoreefficientprimes.py

```

from math import sqrt
from time import perf_counter_ns

max_value = 10000
count = 0
value = 2          # Smallest prime number
start = perf_counter_ns() # Start the stopwatch
while value <= max_value:
    # See if value is prime
    is_prime = True # Provisionally, value is prime
    # Try all possible factors from 2 to value - 1
    trial_factor = 2
    root = round(sqrt(value))
    while trial_factor <= root:
        if value % trial_factor == 0:
            is_prime = False # Found a factor
            break           # No need to continue; it is NOT prime
        trial_factor += 1   # Try the next potential factor
    if is_prime:
        count += 1          # Count the prime number
        value += 1          # Try the next potential prime number
    elapsed = (perf_counter_ns() - start)/1_000_000_000 # Stop the stopwatch

print(f'Count: {count}, Elapsed time: {elapsed} sec')
```

An even faster prime generator appears in Listing 10.24 (`fasterprimes.py`); it uses a completely different algorithm to generate prime numbers.

time.sleep. The `time.sleep` function suspends the program's execution for a specified number of seconds. Listing 6.9 (`countdown.py`) counts down from 10 with one second intervals between numbers.

Listing 6.9: countdown.py

```

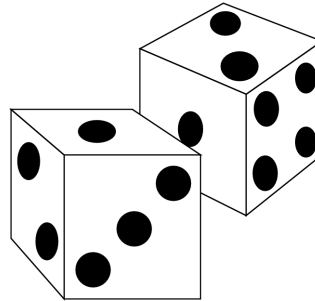
from time import sleep

for count in range(10, -1, -1): # Range 10, 9, 8, ..., 0
    print(count)               # Display the count
    sleep(1)                   # Suspend execution for 1 second
```

The `time.sleep` function is useful for controlling the speed of graphical animations.

6.6 Random Numbers

Some applications require behavior that appears random. Random numbers are particularly useful in games and simulations. For example, many board games use a die (one of a pair of dice—see Figure 6.5) to determine how many places a player is to advance. Other games of chance use a die or pair of dice. A die is a cube containing spots on each of its six faces. The number of spots range from one to six. A player rolls a die or sometimes a pair of dice, and the side that faces up has meaning in the game being played. The value of a face after a roll is determined at random by the complex tumbling of the die. A software adaptation of a game that involves dice would need a way to simulate the random roll of a die.

Figure 6.5 A pair of dice

All algorithmic random number generators actually produce *pseudorandom* numbers, not true random numbers. A pseudorandom number generator has a particular period, based on the nature of the algorithm used. If the generator is used long enough, the pattern of numbers produced repeats itself exactly. A sequence of true random numbers would not contain such a repeating subsequence. All practical algorithmic pseudorandom number generators have periods that are large enough for most applications.

In addition to a long period, a good pseudorandom generator would be equally likely to generate any number in its range; that is, it would not be biased toward a subset of its possible values. Ideally, the numbers the generator produces will be uniformly distributed across its range of values.

The good news is that the Python standard library has a very good pseudorandom number generator based the *Mersenne Twister* algorithm. See http://en.wikipedia.org/wiki/Mersenne_twister for more information about the algorithm.

The Python `random` module contains a number of standard functions that programmers can use for working with pseudorandom numbers. Table 6.2 lists a few of these functions.

Table 6.2 A few of the functions from the `random` module

randomfunctions Module	
<code>random</code>	Returns a pseudorandom floating-point number x in the range $0 \leq x < 1$
<code>randrange</code>	Returns a pseudorandom integer value within a specified range.
<code>seed</code>	Sets the random number seed.
<code>choice</code>	Selects an element at random from a collection of elements.

The `random.seed` function establishes the initial value from which the sequence of pseudorandom numbers is generated. Each call to `random.random` or `random.randrange` returns the next value in the sequence of pseudorandom values. Listing 6.10 (`simplerandom.py`) prints 100 pseudorandom integers in the range $1 \dots 100$.

Listing 6.10: simplerandom.py

```

from random import randrange, seed

seed(23)                                # Set random number seed
for i in range(0, 100):                  # Print 100 random numbers
    print(randrange(1, 1001), end=' ')    # Range 1...1,000, inclusive
print()                                  # Print newline

```

The numbers Listing 6.10 (`simplerandom.py`) prints appear to be random. The program begins its pseudorandom number generation with a seed value, 23. The seed value determines the exact sequence of numbers the program generates; identical seed values generate identical sequences. If you run the program again, it displays the same sequence. In order for the program to display different sequences, the seed value must be different for each run.

If we omit the call to the `random.seed` function, the program derives its initial value in the sequence from the time kept by the operating system. This usually is adequate for simple pseudorandom number sequences. Being able to specify a seed value is useful during development and testing when we want program executions to exhibit reproducible results.

We now have all we need to write a program that simulates the rolling of a die. Listing 6.11 (`die.py`) simulates rolling die.

Listing 6.11: die.py

```

from random import randrange

# Roll the die three times
for i in range(0, 3):
    # Generate random number in the range 1...7
    value = randrange(1, 7)

    # Show the die
    print("+-----+")
    if value == 1:
        print("|      |")
        print("|  *  |")
        print("|      |")
    elif value == 2:
        print("| *   |")
        print("|      |")
        print("|    *|")
    elif value == 3:
        print("|   * |")
        print("|  *  |")
        print("| *   |")
    elif value == 4:
        print("| * * |")
        print("|      |")
        print("| * * |")
    elif value == 5:
        print("| * * |")
        print("|  *  |")
        print("| * * |")
    elif value == 6:

```

```

        print("| * * * |")
        print("|      |")
        print("| * * * |")
    else:
        print(" *** Error: illegal die value ***")
    print("+-----+")

```

The output of one run of Listing 6.11 (die.py) is

```

+-----+
|  *  *  |
|  *  *  |
+-----+
+-----+
| * * * |
| * * * |
+-----+
+-----+
|      *  |
|      *  |
+-----+

```

Since the program generates the values pseudorandomly, actual output will vary from one run to the next.

The random module provides a `randint` function that works similarly to `random.randrange`. The call `random.randint(a, b)` is equivalent to `random.randrange(a, b + 1)`.

We will find the `choice` function more useful after we examine Python's more advanced data structures like lists. As a preview, Listing 6.12 (randomstring.py) shows how we can select a random element from a tuple of strings.

Listing 6.12: randomstring.py

```

from random import choice

for i in range(10):
    print(choice(("one", "two", "three", "four", "five", "six",
                 "seven", "eight", "nine", "ten")))

```

One run of Listing 6.12 (randomstring.py) produces

```

three
nine
five
nine
three
seven
nine
two
eight
ten

```

We must enclose within parentheses the tuple of strings passed to the `choice` function so the Python interpreter treats as one parameter (a tuple consisting of 10 strings) rather than 10 separate string parameters.

6.7 System-specific Functions

The `sys` module provides a number of functions and variables that give programmers access to system-specific information. One useful function is `exit` that terminates an executing program. Listing 6.13 (`exitprogram.py`) uses the `sys.exit` function to end the program's execution after it prints 10 numbers.

Listing 6.13: `exitprogram.py`

```
import sys

sum = 0
while True:
    x = int(input('Enter a number (999 ends):'))
    if x == 999:
        sys.exit(0)
    sum += x
    print('Sum is', sum)
```

The `sys.exit` function accepts a single integer argument, which it passes back to the operating system when the program completes. The value zero indicates that the program completed successfully; a nonzero value represents the program terminating due to an error of some kind.

6.8 The `eval` and `exec` Functions

The `input` function produces a string from the user's keyboard input. If we wish to treat that input as a number, we can use the `int` or `float` function to make the necessary conversion:

```
x = float(input('Please enter a number: '))
```

Here, whether the user enters 2 or 2.0, `x` will be a variable with type floating point. What if we wish `x` to be of type integer if the user enters 2 and `x` to be floating point if the user enters 2.0?

The `__builtins__` module provides an interesting function named `eval` that attempts to evaluate a string in the same way that the interactive shell would evaluate it. Listing 6.14 (`evalfunc.py`) illustrates the use of `eval`.

Listing 6.14: `evalfunc.py`

```
x1 = eval(input('Entry x1? '))
print('x1 =', x1, ' type:', type(x1))
x2 = eval(input('Entry x2? '))
print('x2 =', x2, ' type:', type(x2))
x3 = eval(input('Entry x3? '))
print('x3 =', x3, ' type:', type(x3))
x4 = eval(input('Entry x4? '))
print('x4 =', x4, ' type:', type(x4))
x5 = eval(input('Entry x5? '))
print('x5 =', x5, ' type:', type(x5))
```

A sample run of Listing 6.14 (evalfunc.py) produces

```
Enter x1? 4
x1 = 4  type: <class 'int'>
Enter x2? 4.0
x2 = 4.0  type: <class 'float'>
Enter x3? 'x1'
x3 = x1  type: <class 'str'>
Enter x4? x1
x4 = 4  type: <class 'int'>
Enter x5? x6
Traceback (most recent call last):
  File "C:\Code\python\evalfunc.py", line 9, in <module>
    x5 = eval(input('Enter x5? '))
          ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "<string>", line 1, in <module>
NameError: name 'x6' is not defined. Did you mean: 'x1'?
```

Notice that when the user enters the text consisting of the integer 4, the `eval` function interprets it as integer 4 and assigns an integer to the variable `x1`. When the user enters the text 4.0, the assigned variable is a floating-point variable. For `x3`, the user supplies the string `'x3'` (note the quotes), and the variable's type is string. The more interesting situation is `x4`. The user enters `x1` (no quotes). The `eval` function evaluates the unquoted text as a reference to the name `x1` established by the first assignment statement. The program bound the name `x1` to the integer value 4 when executing the first line of the program. This statement thus binds `x4` to the same integer value; that is, 4. Finally, the user enters `x6` (no quotes). Since the quotes are missing, the `eval` function does not interpret `x6` as a literal string; instead `eval` treats `x6` as a name and attempts to evaluate it. Since no variable named `x6` exists, the `eval` function prints an error message.

The `eval` function dynamically translates the text provided by the user into an executable form that the program can process. This allows users to provide input in a variety of flexible ways; for example, users could enter multiple entries separated by commas, and the `eval` function would evaluate the text typed by the user as Python tuple. As Listing 6.15 (addintegers4.py) shows, this makes tuple assignment (see Section 2.2) from the `input` function possible.

Listing 6.15: addintegers4.py

```
num1, num2 = eval(input('Please enter number 1, number 2: '))
print(f'{num1} + {num2} = {num1 + num2}')
```

The following sample run shows how the user now must enter the two numbers at the same time separated by a comma:

```
Please enter number 1, number 2: 23, 10
23 + 10 = 33
```

Listing 6.16 (enterarith.py) is a simple, one line Python program that behaves like Python's interactive shell, except that it accepts only one expression from the user.

Listing 6.16: enterarith.py

```
print(eval(input()))
```

A sample run of Listing 6.16 (enterarith.py) shows that the user may enter an arithmetic expression, and `eval` handles it properly:


```
4 + 10
14
```

The user enters the text `4 + 10`, and the program prints `14`. Notice that no addition operation appears in Listing 6.16 (`enterarith.py`); as the program runs the `eval` function compiles the user-supplied text into executable code and executes it to produce `14`.

The `exec` function, also from the `__builtins__` module, is similar to the `eval` function. The `exec` function accepts a string parameter that consists of a Python source statement. The `exec` function interprets the statement and executes it. Listing 6.17 (`myinterpreter.py`) plays the role of a rudimentary Python interpreter.

Listing 6.17: `myinterpreter.py`

```
while True:
    exec(input(">>>"))
```

The following shows a sample run of Listing 6.17 (`myinterpreter.py`):

```
>>>from sys import exit
>>>x = 45
>>>print(x)
45
>>>x += 1000
>>>print(x)
1045
>>>exit(0)
```

While the `eval` and `exec` functions may seem to open up a number of interesting possibilities, their use actually is very limited. The `eval` and `exec` functions demand much caution on the part of the programmer. In fact, the examples above that use the `eval` and `exec` functions are not advisable in practice. This is because they enable the user to make the program do things the programmer never intended. Python contains functions that call on the operating system to perform tasks. This functionality includes the possibility of erasing files or formatting entire disk drives. If the user knows the required Python code to accomplish such devious tasks, he or she could hijack the program and cause havoc. As simple, harmless example, consider the following example run of Listing 6.14 (`evalfunc.py`):

```
Entry x1? 100
x1 = 100  type: <class 'int'>
Entry x2? exec("import sys; sys.exit(0)")
```

During the execution of Listing 6.14 (`evalfunc.py`) the user entered the text

```
exec("import sys; sys.exit(0)")
```

The `eval` function then interprets and evaluates the call to `exec`. The evaluation of the `sys.exit` function will, of course, terminate the program. The program is written to interact with the user a while longer, but the user cleverly found a way to terminate it early.

In programs that use `eval` or `exec`, the programmer must preprocess the user input to defend against unwanted and unwelcome program behavior.

6.9 Turtle Graphics

One of the simplest ways to draw pictures is the way we do it by hand with pen and paper. We place the pen on the paper and move the pen, leaving behind a mark on the paper. The length and shape of the mark depends on the movement of the pen. We then can lift the pen from the paper and place it elsewhere on the paper to continue our graphical composition. We may have pens of various colors at our disposal.

Turtle graphics on a computer display mimics these actions of placing, moving, and turning a pen on a sheet of paper. It is called Turtle graphics because originally the pen moving within the display window was shaped like a turtle. Seymour Papert originated the concept of Turtle graphics in his Logo programming language in the late 1960s (see http://en.wikipedia.org/wiki/Turtle_graphics for more information about Turtle graphics). Python includes a Turtle graphics library that is relatively easy to use.

In the simplest Turtle graphics program we need only issue commands to a pen (turtle) object. We must import the `turtle` module to have access to Turtle graphics. Listing 6.18 (`boxturtle.py`) draws a rectangular box.

Listing 6.18: `boxturtle.py`

```
# Draws a rectangular box in the window

import turtle

turtle.pencolor('red')    # Set pen color to red
turtle.forward(200)       # Move pen forward 200 units (create bottom)
turtle.left(90)           # Turn pen by 90 degrees
turtle.pencolor('blue')  # Change pen color to blue
turtle.forward(150)       # Move pen forward 150 units (create right wall)
turtle.left(90)           # Turn pen by 90 degrees
turtle.pencolor('green')  # Change pen color to green
turtle.forward(200)       # Move pen forward 200 units (create top)
turtle.left(90)           # Turn pen by 90 degrees
turtle.pencolor('black')  # Change pen color to black
turtle.forward(150)       # Move pen forward 150 units (create left wall)
turtle.hideturtle()       # Make pen invisible
turtle.exitonclick()      # Wait for user input
```

Figure 6.6 shows the result of running Listing 6.18 (`boxturtle.py`). By default, the pen starts at the center of the window facing to the right. Figure 6.7 shows the default Turtle graphics coordinate system. Listing 6.18 (`boxturtle.py`) reveals a few of the functions provided by the `turtle` module. The following provides a more complete list functions available to beginning Turtle graphics programmers:

- `setheading`: sets the pen to face in a particular direction
- `pencolor`: sets pen's current drawing color.
- `forward`: moves the pen forward a given distance at its current heading
- `left`: turns the pen to the left by an angle specified in degrees.
- `right`: turns the pen to the right by an angle specified in degrees.
- `setposition`: moves the pen to a given (x,y) coordinate within the graphics window. Draws a line from the previous position unless the pen is up.

Figure 6.6 A very simple drawing made with Turtle graphics

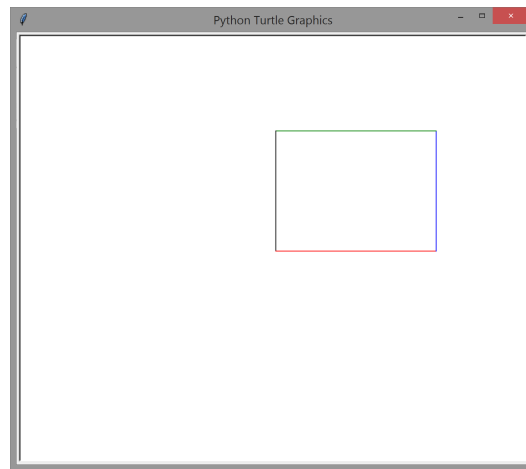
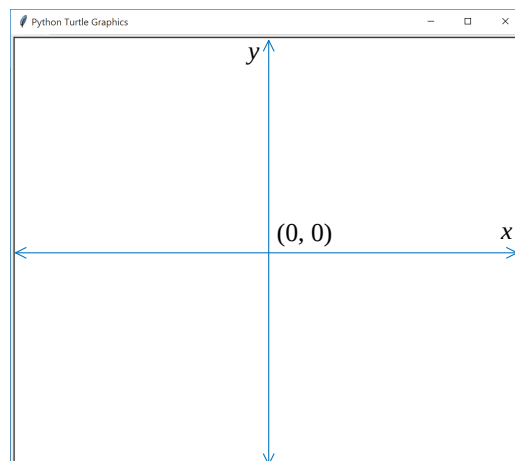


Figure 6.7 The default coordinate system for a Turtle graphics picture. The x and y axes do not appear in an actual image. As in the Cartesian coordinate system from algebra, the x values increase from left to right, and the y values increase from bottom to top.



- `title`: sets the text to appear in the window's title bar
- `penup`: disables drawing until the pen is put back down (allows turtle movements without drawing)
- `pendown`: enables the pen to draw when moved
- `hideturtle`: makes the pen object (turtle) itself invisible; this does not affect its ability to draw.
- `tracer`: turns off tracing (drawing animation) when its argument is zero—this speeds up rendering.
- `update`: renders all pending drawing actions (necessary when tracing is disabled).
- `done`: ends the drawing activity and waits on the user to close the window.
- `exitonclick`: directs the program to terminate when the user clicks the mouse over the window.
- `mainloop`: used in place of `done` to enable the graphics framework to handle events such as user mouse clicks and keystrokes.

Listing 6.19 (`octagon.py`) draws in the display window a blue spiral within a red octagon.

Listing 6.19: `octagon.py`

```
# Draws in the window a spiral surrounded with an octagon

import turtle

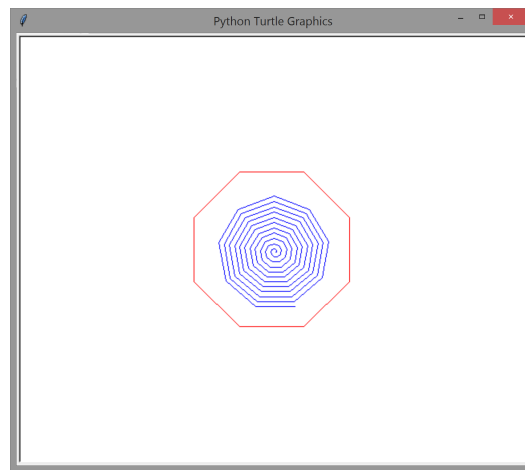
# Draw a red octagon centered at (-45, 100)
turtle.pencolor('red')      # Set pen color
turtle.penup()              # Lift pen to move it
turtle.setposition(-45, 100) # Move the pen to coordinates (-45, 100)
turtle.pendown()            # Place pen to begin drawing
for i in range(8):          # Draw the eight sides
    turtle.forward(80)       # Each side is 80 units long
    turtle.right(45)         # Each vertex is 45 degrees

# Draw a blue spiral centered at (0, 0)
distance = 0.2
angle = 40
turtle.pencolor('blue')     # Set pen color
turtle.penup()              # Lift pen to move it
turtle.setposition(0, 0)    # Position the pen at coordinates (0, 0)
turtle.pendown()            # Set pen down to begin drawing
for i in range(100):
    turtle.forward(distance)
    turtle.left(angle)
    distance += 0.5

turtle.hideturtle()         # Make pen invisible
turtle.exitonclick()        # Quit program when user clicks mouse button
```

Listing 6.19 (`octagon.py`) uses the `turtle.penup`, `turtle.setposition`, and `turtle.pendown` functions to move the pen to a particular location without leaving a mark within the display window. The center of the display window is at coordinates (0, 0). Figure 6.8 shows the result of running Listing 6.19 (`octagon.py`).

Figure 6.8 More Turtle graphics fun: a spiral within an octagon



After each drawing command such as `forward` or `left` the graphics environment updates the image, thus showing the effect of the command. If you are annoyed that the drawing is too slow, you can speed up the rendering process in several ways. You can add the following statement before moving the pen:

```
turtle.speed(0)    # Fastest turtle actions
```

The `speed` function accepts an integer in the range 0...10. The value 1 represents the slowest speed, and the turtle's speed increases as the arguments approach 10. Counterintuitively, 0 represents the fastest turtle speed. The `speed` function will accept a string argument in place of an integer value; the permissible strings correspond to the following numeric values:

- `"fastest"` is equivalent to 0
- `"fast"` is equivalent to 10
- `"normal"` is equivalent to 6
- `"slow"` is equivalent to 3
- `"slowest"` is equivalent to 1

The `delay` function provides another way to affect the time it takes to render an image. Rather than controlling the overall speed of the turtle's individual movements and/or turns, the `delay` function specifies the time delay in milliseconds between drawing incremental updates of the image to the screen. Listing 6.20 (`speedvsdelay.py`) demonstrates the subtle difference in the way `speed` and `delay` functions effect the time to render an image.

Listing 6.20: speedvsdelay.py

```
import turtle

y = -200 # Initial y value

# Default speed and default delay
turtle.color("red")
for x in range(10):
    turtle.penup()
    turtle.setposition(-200, y)
    turtle.pendown()
    turtle.forward(400)
    y += 10

# Slowest speed, but no delay
turtle.speed("slowest")
turtle.delay(0)
turtle.update()
turtle.color("blue")
for x in range(10):
    turtle.penup()
    turtle.setposition(-200, y)
    turtle.pendown()
    turtle.forward(400)
    y += 10

# Fastest speed with a 500 millisecond delay
turtle.speed("fastest")
turtle.delay(500)
turtle.update()
turtle.color("green")
for x in range(10):
    turtle.penup()
    turtle.setposition(-200, y)
    turtle.pendown()
    turtle.forward(400)
    y += 10

# Fastest speed with no delay
turtle.speed("fastest")
turtle.delay(0)
turtle.update()
turtle.color("black")
for x in range(10):
    turtle.penup()
    turtle.setposition(-200, y)
    turtle.pendown()
    turtle.forward(400)
    y += 10

turtle.done()
```

Figure 6.9 Listing 6.20 (`speedvsdelay.py`) demonstrates the different effects of the `turtle.speed` versus `turtle.delay` functions. The program renders the bottom 10 lines using the default speed and delay settings. In the next 10 lines the program draws with the slowest speed but without a delay. The program next draws 10 lines at the fastest speed with a 100 millisecond delay between screen updates. The top 10 lines represent the fastest speed with no delay. Note that the animation is still smooth at a slower speed, but the `delay` function makes the animation choppy at regular time intervals.

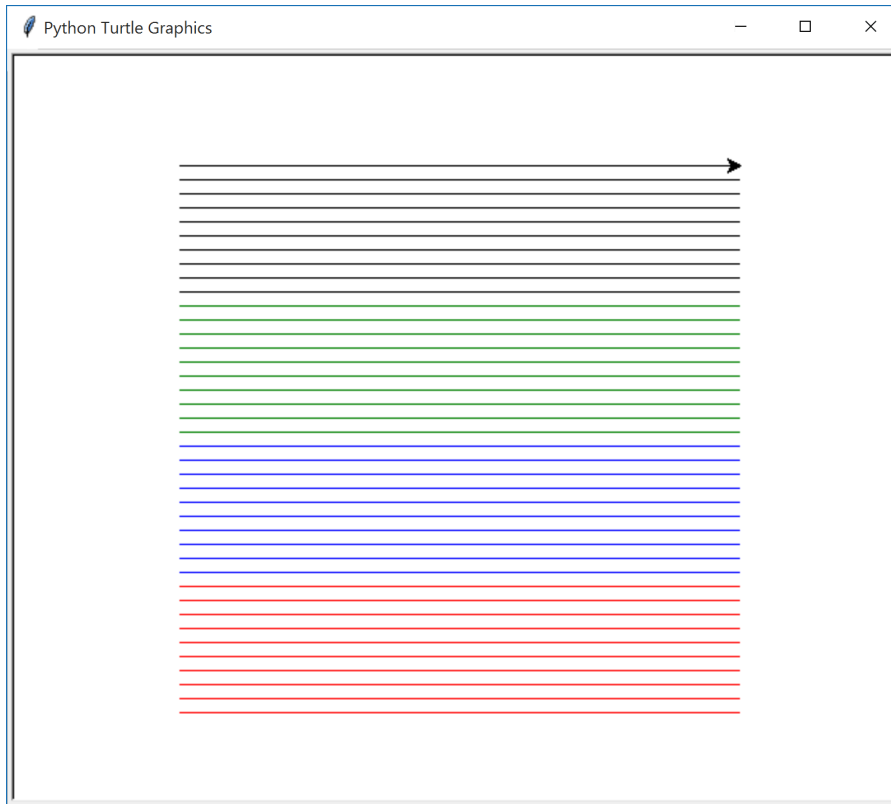


Figure 6.9 shows the eventual output of Listing 6.20 (`speedvsdelay.py`). The `speed` function does not sacrifice the smoothness of the animation, while the `delay` function can make the animation choppy at regular time intervals.

You can use the `tracer` function to achieve the fastest rendering possible. Listing 6.21 (`noanimation.py`) produces the same drawing as Listing 6.20 (`speedvsdelay.py`), but the picture appears almost instantly.

Listing 6.21: noanimation.py

```
import turtle

y = -200 # Initial y value

# Turn off animation
turtle.tracer(0)
turtle.color("red")
for x in range(10):
    turtle.penup()
    turtle.setposition(-200, y)
    turtle.pendown()
    turtle.forward(400)
    y += 10

turtle.color("blue")
for x in range(10):
    turtle.penup()
    turtle.setposition(-200, y)
    turtle.pendown()
    turtle.forward(400)
    y += 10

turtle.color("green")
for x in range(10):
    turtle.penup()
    turtle.setposition(-200, y)
    turtle.pendown()
    turtle.forward(400)
    y += 10

turtle.color("black")
for x in range(10):
    turtle.penup()
    turtle.setposition(-200, y)
    turtle.pendown()
    turtle.forward(400)
    y += 10

turtle.update() # Ensure all of image is drawn
turtle.done()
```

When animation is disabled with the `tracer` function, you should call `update` at the point the complete image should appear. When the tracer is active, it explicitly draws the penstrokes to the screen as the turtle moves. With the tracer turned off, the programmer must ensure all the image becomes visible by calling the `update` function. Turning the tracer off is the ultimate way to speed up Turtle graphics rendering in Python.

6.10 Other Techniques for Importing Functions and Modules

Section 6.2 introduced the two most common ways programmers use to import functions into Python code. For the sake of completeness, we briefly examine three other importing techniques that you may encounter in published Python code.

Recall that we can use the `from ... import ...` notation to import some of the functions that a module has to offer. This allows callers to use the base names of the functions without prepending the module name. This technique becomes unwieldy when a programmer wishes to use a large number of functions from a particular module.

The following statement:

```
from math import *
```

makes all the code in the `math` module available to the program. The `*` symbol is the wildcard symbol that represents “everything.” Some programmers use an `import` statement like this one for programs that need to use many different functions from a module. As we will see below, however, best Python programming practice discourages this “import everything” approach.

This “import all” statement is in some ways the easiest to use. The mindset is, “Import everything because we may need some things in the module, but we are not sure exactly what we need starting out.” The source code is shorter: `*` is quicker to type than a list of function names, and, within the program, short function names are easier to type than the longer, qualified function names. While in the short term the “import all” approach may appear to be attractive, in the long term it can lead to problems. As an example, suppose a programmer is writing a program that simulates a chemical reaction in which the rate of the reaction is related logarithmically to the temperature. The statement

```
from math import log10
```

may cover all that this program needs from the `math` module. If the programmer instead uses

```
from math import *
```

this statement imports everything from the `math` module, including a function named `degrees` which converts an angle measurement in radians to degrees (from trigonometry, $360^\circ = 2\pi$ radians). Given the nature of the program, the word `degrees` is a good name to use for a variable that represents temperature. The two words are the same, but their meanings are very different. Even though the `import` statement brings in the `degrees` function, the programmer is free to redefine `degrees` to be a floating-point variable (recall redefining the `print` function in Section 2.3). If the program does redefine `degrees`, the `math` module’s `degrees` function is unavailable if the programmer later discovers its need. A *name collision* results if the programmer tries to use the same name for both the angle conversion and temperature representation. The same name cannot be used simultaneously for both purposes.

Most significant Python programs must import multiple modules to obtain all the functionality they need. It is possible for two different modules to provide one or more functions with the same name. This is another example of a name collision.

The names of variables and functions available to a program live in that program’s *namespace*. The `dir` function prints a program’s namespace, and we can experiment with it in Python’s interactive interpreter:

```

>>> dir()
['__annotations__', '__builtins__', '__doc__', '__loader__', '__name__',
 '__package__', '__spec__']
>>> x = 2
>>> dir()
['__annotations__', '__builtins__', '__doc__', '__loader__', '__name__',
 '__package__', '__spec__', 'x']
>>> from math import sqrt
>>> dir()
['__annotations__', '__builtins__', '__doc__', '__loader__', '__name__',
 '__package__', '__spec__', 'sqrt', 'x']
>>> from math import *
>>> dir()
['__annotations__', '__builtins__', '__doc__', '__loader__', '__name__',
 '__package__', '__spec__', 'acos', 'acosh', 'asin', 'asinh', 'atan',
 'atan2', 'atanh', 'cbrt', 'ceil', 'comb', 'copysign', 'cos', 'cosh',
 'degrees', 'dist', 'e', 'erf', 'erfc', 'exp', 'exp2', 'expm1', 'fabs',
 'factorial', 'floor', 'fmod', 'frexp', 'fsum', 'gamma', 'gcd', 'hypot',
 'inf', 'isclose', 'isfinite', 'isinf', 'isnan', 'isqrt', 'lcm', 'ldexp',
 'lgamma', 'log', 'log10', 'log1p', 'log2', 'modf', 'nan', 'nextafter',
 'perm', 'pi', 'pow', 'prod', 'radians', 'remainder', 'sin', 'sinh',
 'sqrt', 'sumprod', 'tan', 'tanh', 'tau', 'trunc', 'ulp', 'x']

```

Observe how assigning the variable `x` adds the name `x` to the interpreter's namespace. Importing just `math.sqrt` adds `sqrt` to the namespace. Finally, importing everything from the `math` module adds many more names. If we attempt to use any of these names in a different way, they will lose their original purpose; for example, the following continues the above interactive sequence:

```

>>> help(exp)
Help on built-in function exp in module math:

exp(x, /)
    Return e raised to the power of x.

```

Now let us redefine `exp`:

```

>>> exp = None
>>> help(exp)
Help on NoneType object:

class NoneType(object)
|   Methods defined here:
|
|   __bool__(self, /)
|       True if self else False
|
|   __hash__(self, /)
|       Return hash(self).
|
|   __repr__(self, /)
|       Return repr(self).
|
|   -----
|   Static methods defined here:
|
|   __new__(*args, **kwargs) from builtins.type
|       Create and return a new object.  See help(type) for accurate signature.

```

If we reassign the `exp` name to refer to `None`, we no longer can use it to compute e^x .

We say that the “import everything” statement *pollutes* the program’s namespace. This kind of import adds many names (variables, functions, and other objects) to the collection of names managed by the program. This can cause name collisions as in the example above with the name `degrees`, and it makes it more difficult to work with larger programs. When adding new functionality to such a program we must be careful not to tread on any names that already exist in the program’s namespace.

Python best programming practices discourages the use of the “import everything” statement:

```
from math import *
```

since this provides more opportunities for name collisions and renders code less maintainable.

Most Python programmers agree that the best approach imports the whole module, as in

```
import math
```

and uses qualified names for the functions the module provides. In the above example, this module import approach solves the name collision problem: `math.degrees` is a different name than plain `degrees`. Also, if, for example, modules `mod1` and `mod2` both contain a function named `process`, the module import statement forces programmers to write `mod1.process` and `mod2.process`, thus avoiding the name clash.

We have seen the compromise: import only the functions needed, as in

```
from math import sqrt, log10
```

This does not impact the program’s namespace very much, and it allows the program to use short function names. Also, by explicitly naming the functions to import, the programmer is more aware of how the names will impact the program. Many programmers find this approach acceptable for smaller programs where the probability of name clashes is low within a program.

You can think of a module as a toolbox. The `math` module is a box containing mathematics tools. The statement

```
from math import *
```

is like bringing the math toolbox into your workroom and dumping everything out on the floor. It may be handy at times, but it makes a mess and can lead to problems; for example, you need a wrench from toolbox *A*, but a completely different wrench from toolbox *B* looks just like the wrench you need. The wrench from toolbox *B* actually works completely differently from toolbox *A*'s, but you cannot tell them apart (name clash).

The statement

```
import math
```

is like bringing the math toolbox into your workroom. When you need a mathematics tool you take it out of the box and use it. When you are finished with it, even if you may need it later, you put it back in the toolbox. If you need it later, you can take it out again because you know right where it is. It is a little more work (longer, qualified names), but it is more organized.

The statement

```
from math import sqrt, log10
```

is like bringing the math toolbox into your workroom and taking out the two mathematics tools you need for a project. You don't put the tools back until you are finished with them completely. It is not as messy as dumping out all the tools, and you are less likely to confuse one of the tools with another.

Python provides a way to import a module under a different name. The following statement imports the standard `math` module as `m`:

```
import math as m
```

In this case, the caller would invoke the `math` module's functions in the following way:

```
y = m.sqrt(x)
print(m.log10(100))
```

Note the `m.` prefix attached to the calls of the `sqrt` and `log10` functions. Programmers sometimes use this module renaming import to simplify typing when a module name is long. Listing 6.22 (`octagon2.py`) is a rewrite of Listing 6.19 (`octagon.py`) that introduces a new name for the `turtle` module: `t`. We say that `turtle` and `t` are *aliases* for the same module. This effectively shortens the qualified names for each of the function calls. The fact that Listing 6.22 (`octagon2.py`) runs faster than Listing 6.19 (`octagon.py`) has nothing to do with the module name aliasing or shorter qualified function names; Listing 6.22 (`octagon2.py`) runs much faster because it calls the `turtle.delay` function to speed up the drawing.

Listing 6.22: octagon2.py

```

# Draws in the window a spiral surrounded with an octagon

import turtle as t          # Use a shorter name for turtle module

t.delay(0)                  # Draw as quickly as possible
t.speed(0)                  # Turtle's action as fast as possible
t.hideturtle()              # Do not show the pen
# Draw a red octagon centered at (-45, 100)
t.pencolor('red')           # Set pen color
t.penup()                   # Lift pen to move it
t.setposition(-45, 100)     # Move the pen to coordinates (-45, 100)
t.pendown()                 # Place pen to begin drawing
for i in range(8):          # Draw the eight sides
    t.forward(80)            # Each side is 80 units long
    t.right(45)              # Each vertex is 45 degrees

# Draw a blue spiral centered at (0, 0)
distance = 0.2
angle = 40
t.pencolor('blue')          # Set pen color
t.penup()                   # Lift pen to move it
t.setposition(0, 0)          # Position the pen at coordinates (0, 0)
t.pendown()                 # Set pen down to begin drawing
for i in range(100):
    t.forward(distance)
    t.left(angle)
    distance += 0.5

t.exitonclick()             # Quit program when user clicks the mouse button

```

When a program imports a module this way the module's original name is not automatically available. This means the name `turtle` is not available to the code in Listing 6.22 (`octagon2.py`). Code must access the `turtle` module's function via `t`. There are two ways to enable a program to have simultaneous access to a module's original and new names:

1. Import the module as usual and then make an alias:

```

import math

m = math
print(m.sqrt(16), math.sqrt(16))  # Both work

```

2. Import the module under a new name and make an alias to its original name:

```

import math as m

math = m
print(m.sqrt(16), math.sqrt(16))  # Both work

```

While the Python language permits such aliasing, you should avoid it. Having multiple names for the same

module within the same program is highly discouraged. It makes the code confusing to human readers—even to the original programmer!

The practice of using an alias merely to shorten module name is questionable. It essentially hides a recognized standard name and so renders the code less readable. Fortunately, the ability to alias module names is convenient for other purposes besides shortening module names. Suppose a third-party software vendor develops an alternative to Python's standard `math` module. It includes all the functions provided by the `math` module, with exactly the same names. The company markets it as a drop-in replacement for the `math` module. The vendor names this module `fastmath` because the algorithms it uses to implement the mathematical functions are more efficient than those used in the `math` module. As an example, when invoked with the same arguments, the `fastmath.sqrt` and `math.sqrt` functions compute identical results, but `fastmath.sqrt` returns its result quicker than `math.sqrt`.

Suppose, too, that we have a large application that performs many mathematical calculations using functions from the `math` module. We would like to try the third-party `fastmath` module to see if it indeed speeds up our application. Among other possible module imports, our application includes the following statement:

```
import math
```

This means our application is sprinkled with statements like

```
y = math.sqrt(max - alpha)
while value < max:
    value += math.log10(inc)
```

calling `math` functions in hundreds of places throughout the code. Note the qualified function names. If we change the import statement to

```
import fastmath
```

we will have to edit the function invocations in hundreds of places within our code. Instead, we can change the import statement to read

```
import fastmath as math
```

Adding just two words to the original import statement enables us to leave everything else in our code alone. Every mention of `math` will refer actually to the `fastmath` module.

In addition to importing an entire module under a new name, Python allows programmers to alias individual functions. Consider the following code:

```
from math import sqrt as sq

print(sq(16))
```

The names of standard functions are well known to experienced Python developers, so such renaming renders a program immediately less less readable. We should not consider renaming standard functions unless we have a very good reason. Some have found this technique useful for resolving name clashes between two modules that define one or more functions with the same name. Returning to our example from above, suppose we wish to compare directly the performance of `math.sqrt` to `fastmath.sqrt`. The process of measuring the relative performance of software is known as *benchmarking*. We need to have both `math.sqrt` and `fastmath.sqrt` available in the same program. The following code performs the benchmark and avoids qualified function names:

```
from time import perf_counter_ns
from math import sqrt as std_sqrt
from fastmath import sqrt as fast_sqrt

start_time = perf_counter_ns()
for n in range(100000):
    std_sqrt(n)
print('Standard:', perf_counter_ns() - start_time)

start_time = perf_counter_ns()
for n in range(100000):
    fast_sqrt(n)
print('Fast:', perf_counter_ns() - start_time)
```

The renamed functions may be confusing. The better approach imports the modules themselves rather than the individual functions:

```
import math, fastmath, time

start_time = time.perf_counter_ns()
for n in range(100000):
    math.sqrt(n)
print(time.perf_counter_ns() - start_time)

start_time = time.perf_counter_ns()
for n in range(100000):
    fastmath.sqrt(n)
print(time.perf_counter_ns() - start_time)
```

This version arguably is better. Without looking elsewhere in the program's source code, the programmer immediately can see exactly which function from which module the program is calling. The qualified names make it perfectly clear which function is which.

While using module and function aliases can be convenient at times, you should use this feature only if necessary. Standard names are well known and recognizable by experienced Python programmers. Giving new names to standard functions and modules unnecessarily can make programs less readable and confuse programmers attempting to modify or extend an existing application.

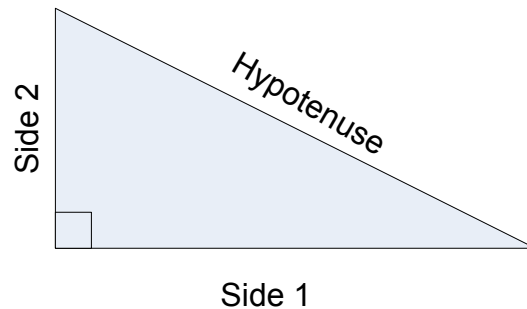
When in doubt, follow best practice and use the module import statement. This makes each function call more self-documenting by unambiguously indicating its module of origin.

6.11 Exercises

1. Suppose you need to compute the square root of a number in a Python program. Would it be a good idea to write the code to perform the square root calculation? Why or why not?
2. Which of the following values could be produced by the call `random.randrange(0, 100)` function (circle all that apply)?

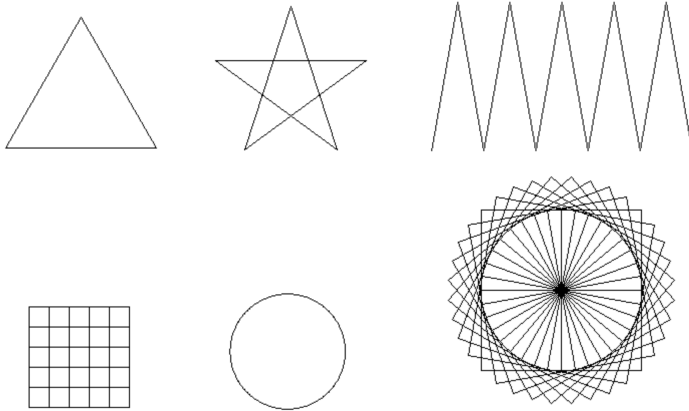
4.5 34 -1 100 0 99

Figure 6.10 Right triangle

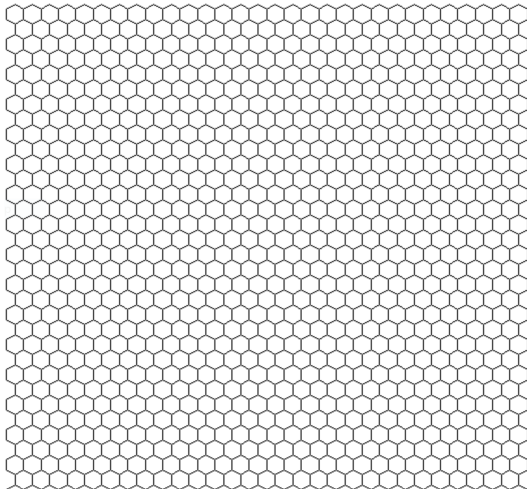


-
3. Classify each of the following expressions as *legal* or *illegal*. Each expression represents a call to a standard Python library function.
 - (a) `math.sqrt(4.5)`
 - (b) `math.sqrt(4.5, 3.1)`
 - (c) `random.rand(4)`
 - (d) `random.seed()`
 - (e) `random.seed(-1)`
 4. From geometry: Write a computer program that, given the lengths of the two sides of a right triangle adjacent to the right angle, computes the length of the hypotenuse of the triangle. (See Figure 6.10.) If you are unsure how to solve the problem mathematically, do a web search for the *Pythagorean theorem*.
 5. Write a guessing game program in which the computer chooses at random an integer in the range $1 \dots 100$. The user's goal is to guess the number in the least number of tries. For each incorrect guess the user provides, the computer provides feedback whether the user's number is too high or too low.
 6. Extend Problem 5 by keeping track of the number of guesses the user needed to get the correct answer. Report the number of guesses at the end of the game.
 7. Extend Problem 6 by measuring how much time it takes for the user to guess the correct answer. Report the time and number of guesses at the end of the game.

8. For each of the drawings below write a program that draws the shape using functions from Python's Turtle graphics module.



9. Write a program that uses functions from Python's Turtle graphics module to draw a grid of hexagons as shown in the picture below.



Chapter 7

Writing Functions

As programs become more complex, programmers must structure their programs in such a way as to effectively manage their complexity. Most humans have a difficult time keeping track of too many pieces of information at one time. It is easy to become bogged down in the details of a complex problem. The trick to managing complexity is to break down the problem into more manageable pieces. Each piece has its own details that must be addressed, but these details are hidden as much as possible within that piece. These pieces assemble to form the problem's complete solution.

So far all of the code we have written has been placed within a single block of code. That single block may have contained sub-blocks for the bodies of structured statements like `if` and `while`, but the program's execution begins with the first statement in the block and ends when the last statement in that block is finished. Even though all of the code we have written has been limited to one, sometimes big, block, our programs all have executed code outside of that block. All the functions we have used—`print`, `input`, `sqrt`, `randrange`, etc.—represent blocks of code that some other programmers have written for us. These blocks of code have a structure that makes them reusable by any Python program.

As the number of statements within our block of code increases, the code becomes more difficult to manage. A single block of code (like in all our programs to this point) that does all the work itself is called *monolithic code*. Monolithic code that is long and complex is undesirable for several reasons:

- **It is difficult to write correctly.** Complicated monolithic code attempts to do everything that needs to be done within the program. The indivisible nature of the code divides the programmer's attention amongst all the tasks the block must perform. In order to write a statement within a block of monolithic code, the programmer must be completely familiar with the details of *all* the code in that block. For instance, we must use care when introducing a new variable to ensure that variable's name is not already being used within the block.
- **It is difficult to debug.** If the sequence of code does not work correctly, it may be difficult to find the source of the error. The effects of an erroneous statement that appears earlier in a block of monolithic code may not become apparent until a possibly correct statement later uses the erroneous statement's incorrect result. Programmers naturally focus their attention first to where they observe the program's misbehavior. Unfortunately, when the problem actually lies elsewhere, it takes more time to locate and repair the problem.
- **It is difficult to extend.** Much of the time software developments spend is modifying and extending existing code. As in the case of originally writing the monolithic block of code, a programmer must

understand all the details in the entire sequence of code before attempting to modify it. If the code is complex, this may be a formidable task.

We can write our own functions to divide our code into more manageable pieces. Using a divide and conquer strategy, we can decompose a complicated block of code into several simpler functions. The original code then can do its job by delegating the work to these functions. This process is known as *functional decomposition*. Besides their code organization aspects, functions allow us to bundle functionality into reusable parts. In Chapter 6 we saw how library functions can dramatically increase the capabilities of our programs. While we should capitalize on library functions as much as possible, often we need a function exhibiting custom behavior unavailable in any standard function. Fortunately, we *can* create our own functions. Once created, we can use (call) these functions in numerous places within a program. If the function's purpose is general enough and we write the function properly, we can reuse the function in other programs as well.

7.1 Function Basics

There are two aspects to every Python function:

- **Function definition.** The definition of a function contains the code that determines the function's behavior.
- **Function invocation.** A function is used within a program via a function invocation. In Chapter 6, we invoked standard functions that we did not have to define ourselves.

Every function has exactly one definition but may have many invocations.

An ordinary function definition consists of four parts:

- **def**—The `def` keyword introduces a function definition.
- **Name**—The name is an identifier (see Section 2.3). As with variable names, the name chosen for a function should accurately portray its intended purpose or describe its functionality. (Python provides for specialized anonymous functions called `lambda` expressions, but we defer their introduction until Section 8.8.)
- **Parameters**—every function definition specifies the parameters that it accepts from callers. The parameters appear in a parenthesized comma-separated list. The list of parameters is empty if the function requires no information from code that calls the function. A colon follows the parameter list.
- **Body**—every function definition has a block of indented statements that constitute the function's body. The body contains the code to execute when callers invoke the function. The code within the body is responsible for producing the result, if any, to return to the caller.

Figure 7.1 shows the general form of a function definition.

A function usually accepts some values from its caller and returns a result back to its caller. Listing 7.1 (`doublenumber.py`) provides a very simple function definition to illustrate the process.

Figure 7.1 General form of a function definition

```
def name ( parameter list ) :  
    block
```

Listing 7.1: doublenumber.py

```
def double(n):  
    return 2 * n # Return twice the given number  
  
# Call the function with the value 3 and print its result  
x = double(3)  
print(x)
```

The program prints the following:

6

Listing 7.1 (doublenumber.py) is admittedly not very useful, but it illustrates several important points:

- The two lines

```
def double(n):  
    return 2 * n # Return twice the given number
```

constitute the definition of the `double` function.
- The `def` keyword marks the beginning of the function's definition.
- The function definition establishes the function's name to be `double`.
- The function definition specifies that the function accepts one value from the user. This value will go by the name `n` within the function's body. This variable `n` is the function's only parameter.
- The statement(s) that constitute the working part of the function definition (in this case just one statement) are indented relative to the line containing `def`.
- The `return` keyword indicates the value the function is to communicate by to its caller. In this case, the function simply returns the product of its parameter `n` and 2.
- The code that follows the `double` function's definition:

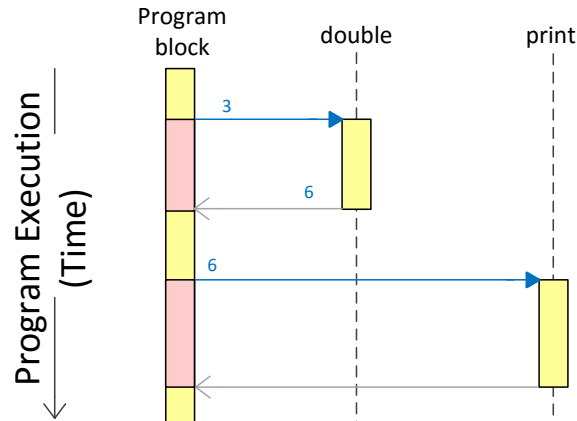
```
# Call the function with the value 3 and print its result  
x = double(3)  
print(x)
```

calls (or invokes) the function sending it the value 3. The assignment statement binds the function's computed return value to the variable `x`. The program can use `x` as it sees fit; in this case the program simply prints `x`'s value.

The program runs as follows:

1. The program's execution begins with the first line in the "naked" block; that is, the block that is not part of the function definition. The program thus executes the assignment statement that calls the `double` function with the argument 3. Before the assignment can happen, the program's execution transfers to the body of the `double` function. The code within `double` executes, which simply returns the product of 2 and the parameter passed in (in this case 3).
2. When `double` is finished, control is passed back to the point in the code where it was called, in this case effectively evaluating the expression `double(3)` to be 6. The assignment operator then assigns the value 6 to the variable `x`.

Figure 7.2 Calling relationships among functions during the execution of Listing 7.1 (doublenumber.py). Time flows from top to bottom. A vertical bar represents the time in which a block of code is active. Observe that functions are active only during their call. The shaded area within a block represents the time that block is idle, waiting for a function call to complete. Right arrows (→) represent function calls. Function calls show parameters, where applicable. Left arrows (←) represent function returns. Function returns show return values, if applicable.



3. The program finally prints the value of `x`, calculated to be 6.

Figure 7.2 contains a diagram illustrating the execution of Listing 7.1 (doublenumber.py) as control passes among the various functions. The interaction amongst functions can be quite elaborate, even for relatively simple programs.

It is important to note that a program executes code within a function only when the function is called. If a program contains a function definition, but no code executing within that program calls that function, that function's code will not execute.

As another simple example, consider Listing 7.2 (countto10.py).

Listing 7.2: countto10.py

```
# Counts to ten
for i in range(1, 11):
    print(i, end=' ')
print()
```

which simply counts to ten:

```
1 2 3 4 5 6 7 8 9 10
```

If counting to ten in this way is something we want to do frequently within a program, we can write a function as shown in Listing 7.3 (countto10func.py) and call it as many times as necessary.

Listing 7.3: countto10func.py

```
# Count to ten and print each number
def count_to_10():
    for i in range(1, 11):
        print(i, end=' ')
    print()

print("Going to count to ten . . .")
count_to_10()
print("Going to count to ten again. . .")
count_to_10()
```

Listing 7.3 (countto10func.py) prints

```
Going to count to ten . . .
1 2 3 4 5 6 7 8 9 10
Going to count to ten again. . .
1 2 3 4 5 6 7 8 9 10
```

The empty parentheses in `count_to_10`'s definition indicates that the function does not accept any parameters from its caller. Also, the absence of a `return` statement indicates that this function communicates no information back to its caller. Such functions that get no information in and provide no results can be useful for the effects they achieve (in this case just printing the numbers 1 to 10).

Our `double` and `count_to_10` functions are a bit underwhelming. We can eliminate the `double` function and replace each call to `double` with the arithmetic it performs. The same could be said for the `count_to_10` function, although it is convenient to have the simple one-line statement that hides the complexity of the loop. These examples serve simply to familiarize us with the mechanics of function definitions and invocations. Functions really shine when our problems become more complex.

Our experience using a simple function like `print` shows us that we can alter the behavior of some functions by passing different parameters. The following successive calls to the `print` function produces different results:

```
print('Hi')
print('Bye')
```

The two statements produce different results, of course, because we pass to the `print` function two different strings. Our `double` function from above will compute a different result if we pass it a different parameter. If a function is written to accept information from the caller, the caller must supply the information in order to use the function. The caller communicates the information via one or more parameters as required by the function. The `count_to_10` function does us little good if we sometimes want to count up to a different number. Listing 7.4 (countton.py) generalizes Listing 7.3 (countto10func.py) to count as high as the caller needs.

Listing 7.4: countton.py

```
# Count to n and print each number
def count_to_n(n):
    for i in range(1, n + 1):
        print(i, end=' ')
    print()

print("Going to count to ten . . .")
count_to_n(10)
print("Going to count to five . . .")
count_to_n(5)
```

Listing 7.4 (countton.py) displays

```
Going to count to ten . . .
1 2 3 4 5 6 7 8 9 10
Going to count to five . . .
1 2 3 4 5
```

When the caller code issues the call

```
count_to_n(10)
```

the argument 10 is known as the *actual* parameter. In the function definition, the parameter named *n* is called the *formal* parameter. During the call

```
count_to_n(10)
```

the actual parameter 10 is assigned to the formal parameter *n* before the function's statements begin executing.

The actual parameter may be a literal value (such as 10 in the expression `count_to_n(10)`), or it may be a variable, as Listing 7.5 (countwithvariable.py) illustrates.

Listing 7.5: countwithvariable.py

```
def count_to_n(n):
    for i in range(1, n + 1):
        print(i, end=' ')
    print()

for i in range(1, 10):
    count_to_n(i)
```



```
1
1 2
1 2 3
1 2 3 4
1 2 3 4 5
1 2 3 4 5 6
1 2 3 4 5 6 7
1 2 3 4 5 6 7 8
1 2 3 4 5 6 7 8 9
```

The actual parameter a caller sends to the `count_to_n` function may in fact be any expression that evaluates to an integer.

A caller must pass exactly one integer parameter to `count_to_n` during a call. An attempt to pass no parameters or more than one integer parameter results in a syntax error:

```
count_to_n()      # Error, missing parameter during the call
count_to_n(3, 5)  # Error, too many parameters during the call
```

An attempt to pass a non-integer results in a run-time exception because the `count_to_n` function passes its parameter on to the `range` expression, and `range` requires all of its arguments to be integers.

```
count_to_n(3.2)   # Run-time error, actual parameter not an integer
```

To recap, in the first line of the function definition:

```
def double(n):
```

we refer to `n` as the formal parameter. A formal parameter is used like a variable within the function's body, and it is local to the function. A formal parameter is the parameter from the perspective of the function definition. During an invocation of `double`, such as `double(2)`, the caller passes actual parameter 2. The actual parameter is the parameter from the caller's point of view. A function invocation, therefore, binds the actual parameters sent by the caller to their corresponding formal parameters.

A caller can pass multiple pieces of information into a function via multiple parameters. Consider Listing 7.6 (`regularpolygon.py`) that uses Turtle graphics (see Section 6.9) to draw regular polygons with varying numbers of sides. In a regular polygon all sides have the same length, and all angles between the sides are the same (see https://en.wikipedia.org/wiki/Regular_polygon).

Listing 7.6: regularpolygon.py

```
import turtle
import random

# Draws a regular polygon with the given number of sides.
# The length of each side is length.
# The pen begins at point(x, y).
# The color of the polygon is color.
def polygon(sides, length, x, y, color):
    turtle.penup()
    turtle.setposition(x, y)
    turtle.pendown()
    turtle.color(color)
    turtle.begin_fill()
    for i in range(sides):
        turtle.forward(length)
        turtle.left(360//sides)
    turtle.end_fill()

# Disable rendering to speed up drawing
turtle.hideturtle()
turtle.tracer(0)

# Draw 20 random polygons with 3-11 sides, each side ranging
# in length from 10-50, located at random position (x, y).
# Select a color at random from red, green, blue, black, or yellow.
for i in range(20):
    polygon(random.randrange(3, 11), random.randrange(10, 51),
            random.randrange(-250, 251), random.randrange(-250, 251),
            random.choice(("red", "green", "blue", "black", "yellow")))

turtle.update() # Render image
turtle.exitonclick() # Wait for user's mouse click
```

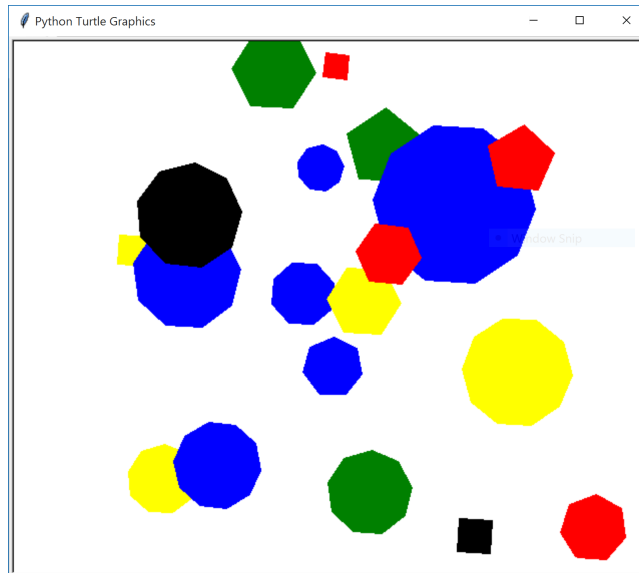
The polygon function accepts four parameters:

- `sides` specifies the number of sides,
- `length` specifies the length of each side,
- `x` specifies the *x* component of the polygon's (*x,y*) location.
- `y` specifies the *y* component of the polygon's (*x,y*) location.
- `color` specifies the polygon's filled color.

The polygon function does not have a `return` statement, so it does not communicate a result back to its caller. This program introduces the `begin_fill` and `end_fill` functions from the `turtle` module. When placed around code that draws a closed figure, these functions fill the shape with the current drawing color. Figure 7.3 shows a screenshot of a sample run of Listing 7.6 (`regularpolygon.py`).

While a caller can pass multiple pieces of information into a function via multiple parameters, a function ordinarily passes back to the caller one piece of information via a `return` statement. If necessary, a

Figure 7.3 A sample run of Listing 7.6 (regularpolygon.py). The number of sides, length of each side, location, and color of each polygon varies based on the arguments passed by the caller.



function may return multiple pieces of information packed up in a tuple or other data structure. Listing 7.7 (midpoint.py) uses a custom function to compute the midpoint between two mathematical points.

Listing 7.7: midpoint.py

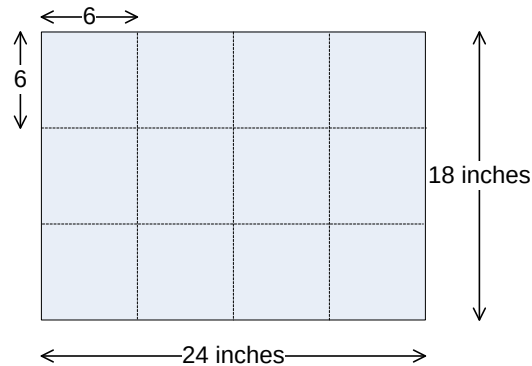
```
def midpoint(pt1, pt2):
    x1, y1 = pt1    # Extract x and y components from the first point
    x2, y2 = pt2    # Extract x and y components from the second point
    return (x1 + x2)/2, (y1 + y2)/2

# Get two points from the user
point1 = float(input("Enter first point's x: ")), \
          float(input("Enter first point's y: "))
point2 = float(input("Enter second point's x: ")), \
          float(input("Enter second point's y: "))
# Compute the midpoint
mid = midpoint(point1, point2)
# Report result to user
print(f'Midpoint of {point1} and {point2} is {mid}')
```

Listing 7.7 (midpoint.py) accepts two parameters, each of which is a tuple containing two values: the x and y components of a point. Given two mathematical points (x_1, y_1) and (x_2, y_2) , the function uses the following formula to compute (x_m, y_m) , the midpoint of (x_1, y_1) and (x_2, y_2) :

$$(x_m, y_m) = \left(\frac{x_1 + x_2}{2}, \frac{y_1 + y_2}{2} \right)$$

A sample run of Listing 7.7 (midpoint.py) looks like the following:

Figure 7.4 Cutting plywood

```

Enter first point's x: 0
Enter first point's y: 0
Enter second point's x: 1
Enter second point's y: 1
Midpoint of (0.0, 0.0) and (1.0, 1.0) is (0.5, 0.5)

```

The `midpoint` function returns only one result, but that result is a tuple containing two pieces of data. The `mid` variable in Listing 7.7 (`midpoint.py`) refers to a single tuple object. We will examine tuples in more detail in Chapter 11, but for now it is useful to note that we also can extract the components of the returned tuple into individual numeric variables as follows:

```
mx, my = midpoint(point1, point2) # Unpack the returned tuple
```

Here, the `midpoint` function still is returning just one value (a tuple), but the assignment statement “unpacks” the individual values stored in the tuple and assigns them to the variables `mx` and `my`. The process of extracting the pieces of a tuple object into separate variables is formally called *tuple unpacking*.

Recall the greatest common divisor (also called greatest common factor) function from elementary mathematics. To determine the GCD of 24 and 18 we list all of their common factors and select the largest one:

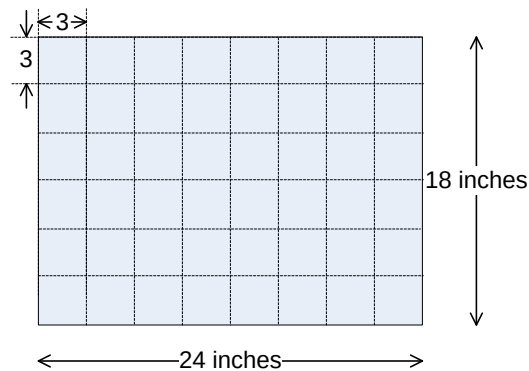
```

24: 1, 2, 3, 4, 6, 8, 12, 24
18: 1, 2, 3, 6, 9, 18

```

The greatest common divisor function is useful for reducing fractions to lowest terms; for example, consider the fraction $\frac{18}{24}$. The greatest common divisor of 18 and 24 is 6, and we can compute the reduced

fraction by dividing the numerator and the denominator by 6: $\frac{18 \div 6}{24 \div 6} = \frac{3}{4}$. The GCD function has applications in other areas besides reducing fractions to lowest terms. Consider the problem of dividing a piece of plywood 24 inches long by 18 inches wide into square pieces of maximum size in integer dimensions, without wasting any material. Since the $\text{GCF}(24, 18) = 6$, we can cut the plywood into twelve 6 inch $\times$ 6 inch square pieces as shown in Figure 7.4. If we cut the plywood into squares of any other size without wasting any of the material, the squares would have to be smaller than 6 inches $\times$ 6 inches; for example, we could make forty-eight 3 inch $\times$ 3 inch squares as shown in pieces as shown in Figure 7.5. If we cut squares larger than 6 inches $\times$ 6 inches, not all the plywood can be used to make the squares. Figure 7.6. shows

Figure 7.5 Squares too small

how some larger squares would fare. In addition to basic arithmetic and geometry, the GCD function plays a vital role in cryptography, enabling secure communication across an insecure network.

We can write a program that computes the GCD of two integers supplied by the user. Listing 7.8 (gcdprog.py) is one such program.

Listing 7.8: gcdprog.py

```
# Compute the greastest common factor of two integers
# provided by the user

# Prompt user for input
num1 = int(input('Please enter an integer: '))
num2 = int(input('Please enter another integer: '))

# Determine the smaller of num1 and num2
min = num1 if num1 < num2 else num2

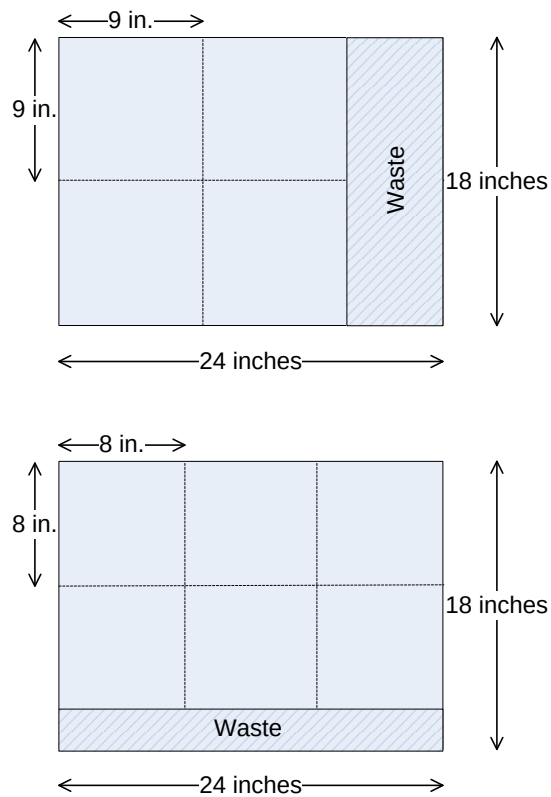
# 1 definitely is a common factor to all ints
largest_factor = 1
for i in range(1, min + 1):
    if num1 % i == 0 and num2 % i == 0:
        largest_factor = i    # Found larger factor
# Print the GCD
print(largest_factor)
```

Listing 7.8 (gcdprog.py) implements a straight-forward but naive algorithm that seeks potential factors by considering every integer less than the smaller of the two values provided by the user. This algorithm is not very efficient, especially for larger numbers. Its logic is easy to follow, with no deep mathematical insight required. Soon we will see a better algorithm for computing GCD.

If we need to compute the GCD from several different places within our program, we should package the code in a function rather than copying it to multiple places. The following code fragment defines a Python function that computes the greatest common divisor of two integers. It determines the largest factor (divisor) common to its parameters:

```
def gcd(num1, num2):
```

Figure 7.6 Squares too large



```
# Determine the smaller of num1 and num2
min = num1 if num1 < num2 else num2
# 1 definitely is a common factor to all ints
largest_factor = 1
for i in range(1, min + 1):
    if num1 % i == 0 and num2 % i == 0:
        largest_factor = i # Found larger factor
return largest_factor
```

This function is named `gcd` and expects two integer arguments. Its formal parameters are named `num1` and `num2`. It returns an integer result. The function uses three local variables: `min`, `largest_factor`, and `i`. Local variables have meaning only within their scope. The scope of a local variable is the point within the function's block after its first assignment until the end of that block. This means that when you write a function you can name a local variable without concern that its name may be used already in another part of the program. Two different functions can use local variables named `x`, and these are two different variables that have no influence on each other. Anything local to a function definition is hidden to all code outside that function definition. Since a formal parameter also is local to its function, you can reuse the names of formal parameters in different functions without a problem.

Listing 7.9 (`localplay.py`) illustrates how function definitions provide protection for local variables.

Listing 7.9: `localplay.py`

```
x = 2
print('1. x =', x) # Print variable x's current value

def fun1():
    x = 10
    print('2. x =', x) # Print this variable x's current value

print('3. x =', x) # Print variable x's current value

def fun2():
    x = 20
    print('4. x =', x) # Print this variable x's current value

print('5. x =', x) # Print variable x's current value

fun1() # Invoke function fun1
fun2() # Invoke function fun2

print('6. x =', x) # Print variable x's current value
```

Notice that Listing 7.9 (`localplay.py`) numbers each print statement in the order of its appearance in the program's source code. When executing the program the interpreter will execute the source code line by line, top to bottom. It executes each statement in turn, but function definitions are special—a function definition packages code into an executable unit to be executed later. The code within a function definition executes only when invoked by a caller. Listing 7.9 (`localplay.py`), therefore, will not execute the print statements in the order listed in the source code. Listing 7.9 (`localplay.py`) prints

```
1. x = 2
3. x = 2
5. x = 2
2. x = 10
4. x = 20
6. x = 2
```

The printing statements 1, 2, 5, and 6 all refer to the variable `x` defined outside of functions `fun1` and `fun2`. When `fun1` and `fun2` assign to a variable named `x`, this `x` is local to its respective function. The assignments within `fun1` and `fun2` do not affect the variable involved in printing statements 1, 2, 5, or 6. Note that the printing statements within `fun1` and `fun2` do not execute until the program actually calls the functions. That is why printing statements 2 and 4 appear out of numerical order in the program's execution. In the end, the last printing statement, number 6, prints the value of the original `x` variable that the program assigned in its first line. The code within `fun1` and `fun2`, as they currently are written, cannot disturb the value of this external variable. (In Section 8.1 we shall see how a function can gain access to a variable defined outside the function's definition.)

In the code we have considered in earlier chapters, the name of a variable uniquely identified it and distinguished that variable from another variable. It may seem strange that now we can use the same name in two different functions within the same program to refer to two distinct variables. The block of statements that makes up a function definition constitutes a context for local variables. A simple analogy may help. In the United States, many cities have a street named *Main Street*; for example, there is a thoroughfare named Main Street in San Francisco, California. Dallas, Texas also has a street named Main Street. Each city and town provides its own context for the use of the term *Main Street*. A person in San Francisco asking "How do I get to Main Street?" will receive the directions to San Francisco's Main Street, while someone in Dallas asking the same question will receive Dallas-specific instructions. In a similar manner, assigning a variable within a function block localizes its identity to that function. We can think of a program's execution as a person traveling around the U.S. When in San Francisco, all references to *Main Street* mean San Francisco's Main Street, but when the traveler arrives in Dallas, the term *Main Street* means Dallas' Main Street. A program's thread of execution cannot execute more than one statement at a time, which means it uses its current context to interpret any names it encounters within a statement. Similarly, at the risk of overextending the analogy, a person cannot be physically located in more than one city at a time. Furthermore, Main Street may be a bustling, multi-lane boulevard in one large city, but a street by the same name in a remote, rural township may be a narrow dirt road! Similarly, two like-named variables may mean two completely different things. A variable named `x` in one function may represent an integer, while a different function may use a string variable named `x`.

Another advantage of local variables is that they occupy space in the computer's memory only when the function is executing. The run-time environment allocates space in the computer's memory for local variables and parameters when the function begins executing. When a function invocation is complete and control returns to the caller, the function's variables and parameters go out of scope, and the run-time environment ensures that the memory used by the local variables is freed up for other purposes within the running program. This process of local variable allocation and deallocation happens each time a caller invokes the function.

Once we have written a complete function definition we can use the function within our program. We invoke a programmer-defined function in exactly the same way as a standard library function like `sqrt` (6.4) or `randrange` (6.6). If the function returns a value, we can use its invocation anywhere an expression of its return type can be used. We can call the function `gcd` as part of an assignment statement:

```
factor = gcd(val, 24)
```


This call uses the variable `val` as its first actual parameter and the literal value 24 as its second actual parameter. As with the standard Python functions, we can pass variables, expressions, and literals as actual parameters. The function then computes and returns its result. Here, the statement assigns the function call's result to the variable `factor`.

How does the function call and parameter mechanism work? It's actually quite simple. The executing program binds the actual parameters, in order, to each of the formal parameters in the function definition and then passes control to the body of the function. When the function's body is finished executing, control passes back to the point in the program where the function was called. The value returned by the function, if any, replaces the function call expression. The statement

```
factor = gcd(val, 24)
```

assigns an integer value to `factor`. The expression on the right is a function call, so the executing program invokes the function to determine what to assign. The value of the variable `val` is assigned to the formal parameter `num1`, and the literal value 24 is assigned to the formal parameter `num2`. The body of the `gcd` function then executes. When the `return` statement in the body of `gcd` executes, program control returns back to where the function was called. The argument of the return statement becomes the value assigned to `factor`.

Note that we can call `gcd` from many different places within the same program, and, since we can pass different parameter values at each of these different invocations, `gcd` could compute a different result at each invocation.

Other invocation examples include:

- `print(gcd(36, 24))`

This example simply prints the result of the invocation. The function invocation binds the value 36 to `num1` and binds 24 to `num2`. The statement prints 12, since 12 is the greatest common divisor of 36 and 24.

- `x = gcd(x - 2, 24)`

The execution of this statement would evaluate `x - 2` and bind its value to `num1`. `num2` is bound to 24. The result of the call is then assigned to `x`. Since the right side of the assignment statement is evaluated *before* being assigned to the left side, the original value of `x` is used when calculating `x - 2`, and the function return value then updates `x`.

- `x = gcd(x - 2, gcd(10, 8))`

This example shows two invocations in one statement. Since the function returns an integer value, its result can itself be used as an actual parameter in a function call. Passing the result of one function call as an actual parameter to another function call is called *function composition*. Function composition is nothing new to us, consider the following statement which prints the square root of 16:

```
print(sqrt(16))
```

The actual parameter passed to the `print` function is the result of the `sqrt` function call.

Listing 7.10 (`gcdfunc.py`) is a complete Python program that uses the `gcd` function.

Listing 7.10: `gcdfunc.py`

```
# Compute the greatest common factor of two integers
# provided by the user

def gcd(n1, n2):
    # Determine the smaller of n1 and n2
    min = n1 if n1 < n2 else n2
    # 1 definitely is a common factor to all ints
    largest_factor = 1
    for i in range(1, min + 1):
        if n1 % i == 0 and n2 % i == 0:
            largest_factor = i    # Found larger factor
    return largest_factor

# Exercise the gcd function

# Prompt user for input
num1 = int(input('Please enter an integer: '))
num2 = int(input('Please enter another integer: '))

# Determine the smaller of num1 and num2
min = num1 if num1 < num2 else num2

# Print the GCD
print(gcd(num1, num2))
```

The following shows a sample run of Listing 7.10 (`gcdfunc.py`):

```
Please enter an integer: 24
Please enter another integer: 18
6
```

Note that the program first defines the `gcd` function and then uses (calls) it in the program's last line. As usual, we can tell where the function definition ends and the rest of the program begins since the function's block of statements is indented relative to the rest of the program.

Within a program, a function's definition must appear before its use. Consider Listing 7.11 (`gcdfuncbad.py`), which moves the `gcd`'s definition to the end of the source code.

Listing 7.11: gcdfuncbad.py

```
# NOTE: This program will not run to completion because it
# calls the gcd function before defining it!

# [Attempt to] Compute the greatest common factor of two integers
# provided by the user

# Exercise the gcd function

# Prompt user for input
num1 = int(input('Please enter an integer: '))
num2 = int(input('Please enter another integer: '))

# Print the GCD
print(gcd(num1, num2))

def gcd(n1, n2):
    # Determine the smaller of n1 and n2
    min = n1 if n1 < n2 else n2
    # 1 definitely is a common factor to all ints
    largest_factor = 1
    for i in range(1, min + 1):
        if n1 % i == 0 and n2 % i == 0:
            largest_factor = i    # Found larger factor
    return largest_factor
```

The execution of Listing 7.11 (gcdfuncbad.py) produces an error:

```
Please enter an integer: 12
Please enter another integer: 24
Traceback (most recent call last):
  File "C:\Code\python\gcdfuncbad.py", line 12, in <module>
    print(gcd(num1, num2))
          ^^^
NameError: name 'gcd' is not defined
```

The Python interpreter executes the code, line by line, until it encounters the call to `gcd`. If it has not yet seen `gcd`'s definition, it will terminate the program with an error

Functions help us organize our code. It is not uncommon for programmers to write a main controlling function that calls other functions to accomplish the work of the application. Listing 7.12 (gcdwithmain.py) illustrates this organization.

Listing 7.12: gcdwithmain.py

```

# Computes the greatest common divisor of m and n
def gcd(m, n):
    # Determine the smaller of m and n
    min = m if m < n else n
    # 1 is definitely a common factor to all ints
    largest_factor = 1
    for i in range(1, min + 1):
        if m % i == 0 and n % i == 0:
            largest_factor = i    # Found larger factor
    return largest_factor

# Get an integer from the user
def get_int():
    return int(input("Please enter an integer: "))

# Main code to execute
def main():
    n1 = get_int()
    n2 = get_int()
    print(f'gcd({n1}, {n2}) = {gcd(n1, n2)}')

# Run the program
main()

```

The single free statement at the end:

```
main()
```

calls the `main` function which in turn directly calls several other functions (`get_int`, `print`, and `gcd`). The `get_int` function itself directly calls `int` and `input`. In the course of its execution the `gcd` function calls `range`. Figure 7.7 contains a diagram that shows the calling relationships among the function executions during a run of Listing 7.12 (`gcdwithmain.py`).

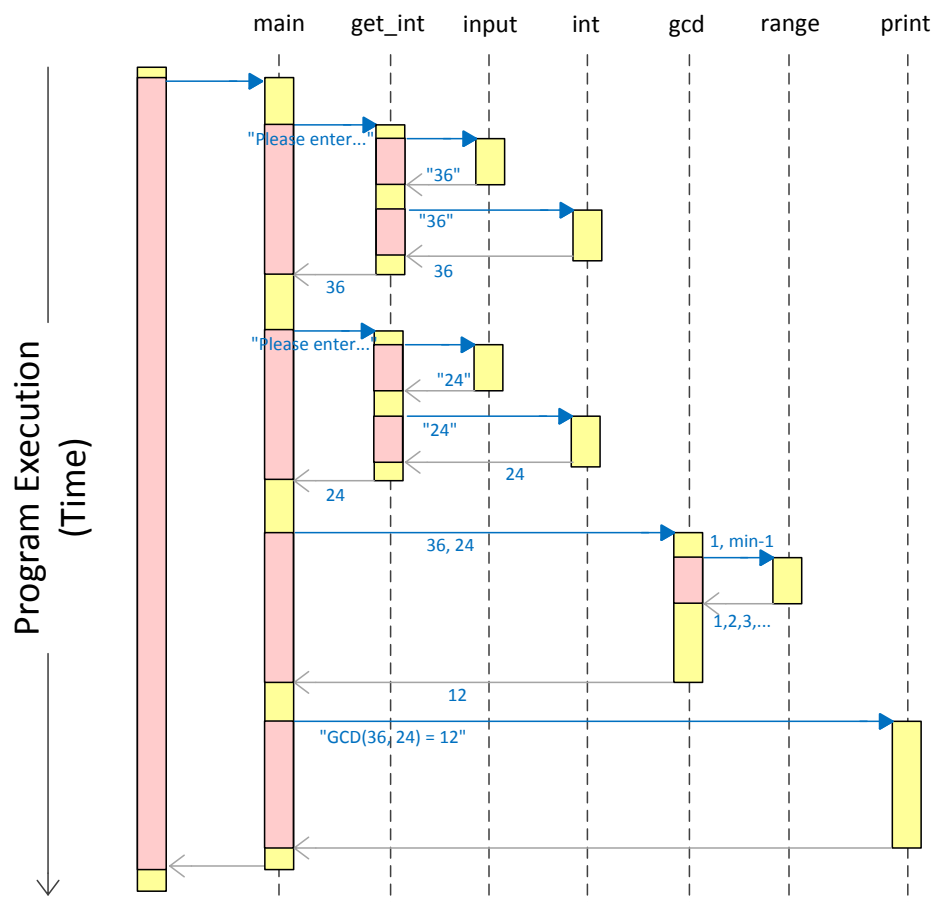
The name `main` for the controlling function is arbitrary but traditional; several other popular programming languages (C, C++, Java, C#, Objective-C) require such a function and require it to be named `main`.

7.2 Parameter Passing

When a caller invokes a function that expects a parameter, the caller must pass a parameter to the function. The process behind parameter passing in Python is simple: the function call binds to the formal parameter the object referenced by the actual parameter. The kinds of objects we have considered so far—integers, floating-point numbers, and strings—are classified as *immutable* objects. This means a programmer cannot change the value of the object. For example, the assignment

```
x = 4
```

binds the variable named `x` to the integer 4. We may change `x` by reassigning it, but we cannot change the integer 4. Four is always four. Similarly, we may assign a string literal to a variable, as in

Figure 7.7 Calling relationships among functions during the execution of Listing 7.12 (gcdwithmain.py)

```
word = 'great'
```

but we cannot change the string object to which `word` refers. If the caller's actual parameter references an immutable object, the function's activity cannot affect the value of the actual parameter. Listing 7.13 (`parampassing.py`) illustrates the consequences of passing an immutable type to an function.

Listing 7.13: `parampassing.py`

```
def increment(x):
    print("Beginning execution of increment, x =", x)
    x += 1    # Increment x
    print("Ending execution of increment, x =", x)

def main():
    x = 5
    print("Before increment, x =", x)
    increment(x)
    print("After increment, x =", x)

main()
```

For additional drama we chose to name the actual parameter the same as the formal parameter, but, of course, the names do not matter; the variables live in two completely different contexts. Listing 7.13 (`parampassing.py`) produces

```
Before increment, x = 5
Beginning execution of increment, x = 5
Ending execution of increment, x = 6
After increment, x = 5
```

The `increment` method cannot not affect the variable `x` in `main` because `x` references an integer, and all integers are immutable. Inside the `increment` function the statement

```
x += 1
```

is short for

```
x = x + 1
```

The expression `x + 1` refers to $5 + 1 = 6$, a different object from 5. The assignment statement re-binds `increment`'s `x` variable to 6. At this point `increment`'s `x` variable and `main`'s `x` variable refer to two different integer objects.

7.3 Documenting Functions

It is good practice to document a function's definition with information that aids programmers who may need to use or extend the function. The essential information includes:

- **The purpose of the function.** The function's purpose is not always evident merely from its name. This is especially true for functions that perform complex tasks. A few sentences explaining what the function does can be helpful.

- **The role of each parameter.** A parameter's name is obvious in the definition, but the expected type and the purpose of a parameter may not be apparent merely from its name.
- **The nature of the return value.** While the function may do a number of interesting things as indicated in the function's purpose, what exactly does it return to the caller? It is helpful to clarify exactly what value the function produces, if any.

We can use comments to document our functions, but Python provides a way that allows developers and tools to extract more easily the needed information.

Recall Python's multi-line strings, introduced in Section 2.9. When such a string appears as the first line in the block of a function definition, the string is known as a *documentation string*, or *docstring* for short. We can document our gcd function as shown in Listing 7.14 (docgcd.py).

Listing 7.14: docgcd.py

```
def gcd(n1, n2):  
    """  
    Computes the greatest common divisor of integers n1 and n2.  
    """  
    # Determine the smaller of n1 and n2  
    min = n1 if n1 < n2 else n2  
    # 1 definitely is a common factor to all ints  
    largest_factor = 1  
    for i in range(1, min + 1):  
        if n1 % i == 0 and n2 % i == 0:  
            largest_factor = i    # Found larger factor  
    return largest_factor
```

Note that Listing 7.14 (docgcd.py) is not executable Python program; it provides only the definition of the gcd function. We can start a Python interactive shell and import the gcd code:

```
>>> from docgcd import gcd  
>>> gcd(18, 24)  
6  
>>>
```

With the docgcd code loaded into the interactive shell as so, we can type:

```
>>> help(gcd)  
Help on function gcd in module docgcd:  
  
gcd(n1, n2)  
    Computes the greatest common divisor of integers n1 and n2.  
>>>
```

The normal # comments serve as *internal documentation* for developers of the gcd function, while the function's docstring serves as *external documentation* for callers of the function.

Other information often is required in a commercial environment:

- **Author of the function.** Specify exactly who wrote the function. An email address can be included. If questions about the function arise, this contact information can be invaluable.

- **Date that the function's implementation was last modified.** An additional comment can be added each time a programmer updates the code in the function. Each update should specify the exact changes that were made and the person responsible for the update.
- **References.** If the code was adapted from another source, list the source. The reference may consist of a Web URL.

Some or all of this additional information may appear as internal documentation rather than appear in a docstring.

The official Python style guide recommends using `"""` for docstrings rather than `' '`—see <https://www.python.org/dev/peps/pep-0008/>. In fact, since the docstring for our `gcd` function above is only one line of text, the normal `'` and `"` quotation marks are adequate to specify its docstring. We will follow the convention of using `"""` to delimit our docstrings, even when expressing a single-line documentation.

The following fragment shows the beginning of a well-commented function definition:

```
# Author: Joe Algori (joe@eng-sys.net)
# Last modified: 2024-01-08
# Adapted from a formula published at
# http://en.wikipedia.org/wiki/Distance
def distance(x1, y1, x2, y2):
    """
        Computes the distance between two geometric points
        x1 is the x coordinate of the first point
        y1 is the y coordinate of the first point
        x2 is the x coordinate of the second point
        y2 is the y coordinate of the second point
        Returns the distance between (x1,y1) and (x2,y2)
    """
    ...
```

From the information provided

- callers know what the function can do for them (via the docstring),
- callers know how to use the function (via the docstring),
- subsequent programmers that must maintain the function can contact the original author (via the comment) if questions arise about its use or implementation,
- subsequent programmers that must maintain the function can check the Wikipedia reference (via the comment) if questions arise about its implementation, and
- subsequent programmers can evaluate the quality of the algorithm based upon the quality of its source of inspiration (Wikipedia, via the comment).

7.4 Type Hints

Python is a *dynamically-typed* language. Unlike many other programming languages, programmers do not need to declare within their code that a variable has a specific type. Most other mainstream programming

languages, including C, C++, Java, C#, and Rust, are *statically-typed* languages. Statically-typed languages require programmers to write their code in such a way so that their compiler can unambiguously determine the type of every variable within the program. Furthermore, that variable's type is fixed and cannot change as the program executes. The type of a Python variable can change during its lifetime. The following code illustrates:

```
x = 2           # x is an integer
x = 'ABC'       # x is a string
x = range(10)   # x is range object
```

Dynamically-type languages like Python offer greater flexibility and freedom to programmers. This flexibility and freedom comes with some costs, however. Programs written in statically-typed programming languages generally use less memory and execute much faster than equivalent dynamically-typed programs. Static typing provides more information to tools that analyze the correctness of programs.

Consider Listing 7.15 (typeerror.py) which exercises our gcd function.

Listing 7.15: typeerror.py

```
def gcd(n1, n2):
    """ Computes the greatest common divisor of integers n1 and n2. """
    # Determine the smaller of n1 and n2
    min = n1 if n1 < n2 else n2
    # 1 definitely is a common factor to all ints
    largest_factor = 1
    for i in range(1, min + 1):
        if n1 % i == 0 and n2 % i == 0:
            largest_factor = i # Found larger factor
    return largest_factor

def main():
    num1 = input('Enter a number: ')
    num2 = input('Enter another number: ')
    ans = gcd(num1, num2)
    print(f'gcd({num1}, {num2}) = {ans}.')

if __name__ == '__main__':
    main()
```

The following execution of Listing 7.15 (typeerror.py) reveals a problem:

```
Enter a number: 24
Enter another number: 18
Traceback (most recent call last):
  File "C:\Code\typeerror.py", line 47, in <module>
    main()
  File "C:\Code\typeerror.py", line 42, in main
    ans = gcd(num1, num2)
          ^^^^^^^^^^^^^^^^^
  File "C:\Code\typeerror.py", line 34, in gcd
    for i in range(1, min + 1):
                    ~~~~~^~
TypeError: can only concatenate str (not "int") to str
```

The error report indicates there is a run-time error at Line 34 in the `gcd` function. The program's execution arrived at Line 34 from Line 42—`main`'s call to the `gcd` function. Line 47 started the whole process when it invoked the `main` function. The displayed chain of function calls is known as a *stack trace*. (See Section 12.7 for more information about stack traces.)

The error correctly shows the line with the problem:

```
for i in range(1, min + 1):
```

Initially, we would suspect there is a problem with this statement, as its execution produced the error and terminated the program's execution. In fact, this statement is fine. The `gcd` function itself is correct. The `gcd` function is not to blame. The problem lies in the following three statements within the `main` function:

```
num1 = input('Enter a number: ')
num2 = input('Enter another number: ')
ans = gcd(num1, num2)
```

The first statement binds the variable `num1` to a string value, as the `input` function returns a string result. The second statement similarly binds `num2` to a string. The third statement passes the two strings as is to the `gcd` function. The `gcd` function expects the actual parameters to be numbers (`int` or `float`), not strings (`str`). Within the `gcd` function, the following statement:

```
min = n1 if n1 < n2 else n2
```

works with no problems because the `<` operator can compare two strings lexicographically (commonly known as alphabetically). This statement assigns to the `min` variable the string parameter that is the lexicographically lesser of the two. The problem arises in the following expression:

```
min + 1
```

At this point, `min` is a string. The `+` operator can concatenate two strings or add two numbers, but it cannot combine a string and a number in any meaningful way. The interpreter issues a `TypeError` run-time error at this point.

The problem is this: The programmer may spend a lot of time attempting to repair the `gcd` function, as that is where the error manifested itself. No work in `gcd` will help the situation, as the problem really lies in the `main` function. The `gcd` function expects two numbers, but the caller passed two strings. The expected formal parameter types do not match the caller's actual parameter types. For a small program like Listing 7.15 (`typeerror.py`), this issue is not so serious. A skilled programmer can follow the stack trace and find the issue quickly. The problem arises in larger programs consisting 100,000 or more lines of code. Stack traces can be lengthy. Worse yet, a potential type error like this one may evade testing and lurk in the code, only to be discovered by users after a program's release.

Statically-typed languages check for such formal parameter/actual parameter type mismatches, and they report the error before the program executes. Their very nature makes this easy, as statically-typed languages keep close accounting of all the involved types. From the code we have seen thus far, dynamically-typed Python is prone to a class of programming errors to which statically-typed languages are immune.

Python 3.5 introduced *type hints* into the language. Programmers can use type hints to declare the types of formal parameters and return types for functions. These type hints allow external tools to verify the type-correctness of Python programs. This brings to Python some of the type safety available to statically-typed languages. The following code fragment shows how to add type hints to our `gcd` function:

```
def gcd(n1: int, n2: int) -> int:
    """ Computes the greatest common divisor of integers n1 and n2. """
    # Everything else remains the same in the function definition
```

Figure 7.8 Visual Studio Code reporting a type error. The Pylance extension determines that the variables `num1` and `num2` are strings, but the `gcd` function expects callers to supply two integer values.



Note the colon (`:`) and `int` appearing after the formal parameters `n1` and `n2`. This indicates that the `gcd` function expects the caller to pass two objects that are assignment-compatible with the `int` type. The arrow (`->`) after the right parenthesis of the parameter list points to the type that the function returns. Together, we can see that this `gcd` function expects the caller to pass two `int` arguments, and the function will return an `int` value as its result. Type hints serve as additional documentation help programmers use functions correctly. Note the comment in the docstring identifying `n1` and `n2` as integers is no longer necessary; the type hints makes it explicit.

The good news is external tools, such as VS Code with the *Pylance* extension understand type hints. They can detect and report type errors in a Python program before the program executes. The VS Code editor flags such errors with red squiggly lines below the offending part of the Python source code. Figure 7.8 shows how VS Code responds to a portion of Listing 7.15 (`typeerror.py`).

Note that VS Code does not need to execute the program to locate the problem. The Pylance extension analyzes the Python source code statically, using the programmer-supplied type hints to judge compliance. Developers do not need to see if the program crashes and then step through a potentially lengthy stack trace in an attempt to diagnose the error.

The bad news (to some) is the current Python interpreter ignores type hints. The interpreter scans the Python source code and can detect syntax errors before executing the program. If the code is syntactically correct, the interpreter translates each statement of the Python source code into executable machine language while the program executes. Since type errors are not syntax errors, the interpreter cannot detect a type error until it attempts to execute a statement that misuses the type. In the greatest common divisor example above, that happens when the statement within the `gcd` function attempts to use the `+` operator with incompatible operands.

Type hints are optional. A programmer can use type hints in some functions and not in others within the same program. The tools perform type checking on functions with type hints and ignore checking those

Table 7.1 Associated type hints for the basic Python types.

Type Name	Python Type	Type Hint
Integer	<code>int</code>	<code>int</code>
Floating-point number	<code>float</code>	<code>float</code>
String	<code>str</code>	<code>str</code>
Boolean	<code>bool</code>	<code>bool</code>
Nothing	<code>None</code>	<code>None</code>
Any type	—	Any

without type information.

Many Python developers, including the language’s creator, Guido van Rossum, agree that the liberal use of type hints lead to more robust code. Type hints help programmers more quickly understand existing code written by someone else when they need to repair or extend it.

Table 7.1 provides a list of the basic types we have seen so far and their associated type hints. When a situation where a parameter is expected to be a number (that is, an `int` or a `float`), use `float` for the type hint. There are many more ways use type hints than listed in Table 7.1. Typing hinting for the basic types is simple, but more complicated types are more involved.

Type hints make functions a little more verbose, but the value they add to developers in type-hint-aware IDEs like VS Code is worth the extra effort. The remainder of the examples in this book will use type hints to better document function definitions.

7.5 Function Examples

This section contains a number of examples of code organization with functions.

7.5.1 Better Organized Prime Generator

Listing 7.16 (`primefunc.py`) is a simple enhancement of Listing 6.4 (`moreefficientprimes.py`). It uses the square root optimization and adds a separate `is_prime` function.

Listing 7.16: `primefunc.py`

```

from math import sqrt

def is_prime(n: int) -> bool:
    """
    Determines the primality of a given value.
    n an integer to test for primality.
    Returns true if n is prime; otherwise, returns false.
    """
    if n == 2:
        return True      # 2 is the only even prime number
    if n % 2 == 0 or n < 2:
        return False     # Exclude other even numbers
    root = round(sqrt(n)) + 1
    # Try all potential factors from 2 to the square root of n
    # Skip any even numbers
    for trial_factor in range(3, root, 2):
        if n % trial_factor == 0: # Is it a factor?
            return False        # Found a factor
    return True              # No factors found

def main() -> None:
    """
    Tests for primality each integer from 2 up to a value provided by the user.
    If an integer is prime, it prints it; otherwise, the number is not printed.
    """
    max_value = int(input("Display primes up to what value? "))
    for value in range(2, max_value + 1):
        if is_prime(value):      # See if value is prime
            print(value, end=" ") # Display the prime number
    print() # Move cursor down to next line

main() # Run the program

```

On an algorithmic note, the `is_prime` function takes advantage of the fact that two is the only even prime number and so no prime number except two may have a factor that is even. Applying this fact allows us to cut in half the potential factors to consider within the loop.

Listing 7.16 (`primefunc.py`) illustrates several important points about well-organized programs:

- The complete work of the program is no longer limited to one block of code. The `main` function is responsible for generating prime candidates and printing the numbers that are prime. `main` delegates the task of testing for primality to the `is_prime` function. Both `main` and `is_prime` individually are simpler than the original monolithic code. Also, each function is more logically *coherent*. A function is coherent when it is focused on a single task. Coherence is a desirable property of functions. If a function becomes too complex by trying to do too many different things, it can be more difficult to write correctly and debug when problems are detected. A complex function usually can be decomposed into several, smaller, more coherent functions. The original function would then call these new simpler functions to accomplish its task. Here, `main` is not concerned about *how* to determine if a given number is prime; `main` simply delegates the work to `is_prime` and makes use of the `is_prime` function's findings. For `is_prime` to do its job it does not need to know anything about

the history of the number passed to it, nor does it need to know the caller's intentions with the result it returns.

- A thorough docstring describing the nature of the function precedes each function. The docstring explains the meaning of each parameter, and it indicates what the function should return.
- While the function's docstring indicates *what* the function is to do, comments within each function explain in more detail *how* the function accomplishes its task. The docstring is meant for developers using the function, and the comments are meant for programmers tasks with maintaining the function.

A call to `is_prime` returns `True` or `False` depending on the value passed to it. We can express a condition like

```
if is_prime(value) == True:
```

more compactly as

```
if is_prime(value):
```

because if `is_prime(value)` is `True`, `True == True` is `True`, and if `is_prime(value)` is `False`, `False == True` is `False`. The expression `is_prime(value)` all by itself suffices.

Observe that a `return` statement in the `is_prime` function immediately exits the function. In the `for` loop the `return` statement acts like a `break` statement because it immediately exits the loop on the way to immediately exiting the function.

Some purists contend that just as it is better for a loop to have exactly one exit point, it is better for a function to have a single `return` statement. The following code rewrites the `is_prime` function so it uses only one return statement:

```
def is_prime(n: int) -> bool:
    result = False
    if n == 2:
        result = True      # 2 is the only even prime number
    elif n % 2 == 0 or n < 2:
        result = False     # Exclude any other even number
    else:
        result = True      # Provisionally, n is prime
        root = round(sqrt(n)) + 1
        # Try all potential factors from 3 to the square root of n
        trial_factor = 3
        while result and trial_factor <= root:
            result = (n % trial_factor != 0) # Is it a factor?
            trial_factor += 2 # Try next candidate
    return result
```

This version adds a local variable (`result`) and complicates the logic a little, so we can make a strong case for the original, multiple `return` version. The two `return` statements in the original `is_prime` function are close enough textually in the code that the logic is easy to follow.

7.5.2 Command Interpreter

Some functions are useful even if they accept no information from the caller and return no result. Listing 7.17 (`calculator.py`) uses such a function.

Listing 7.17: calculator.py

```
def help_screen() -> None:
    """
    Displays information about how the program works.
    Accepts no parameters.
    Returns nothing.
    """
    print('Add:      Adds two numbers')
    print('Subtract: Subtracts two numbers')
    print('Print:     Displays the result of the latest operation')
    print('Help:      Displays this help screen')
    print('Quit:      Exits the program')

def menu() -> str:
    """
    Displays a menu
    Accepts no parameters
    Returns the string entered by the user.
    """
    return input('=== A)dd S)ubtract P)rint H)elp Q)uit ==> ')

def get_operands() -> tuple[float, float]:
    """
    Prompts the user for two floating-point numbers and
    returns them in a tuple.
    """
    num1 = float(input('Enter arg 1: '))
    num2 = float(input('Enter arg 2: '))
    return num1, num2

def main() -> None:
    """
    Runs a command loop that allows users to perform simple arithmetic.
    """
    result = 0.0
    done = False # Initially not done
    while not done:
        choice = menu() # Get user's choice

        if choice == 'A' or choice == 'a': # Addition
            arg1, arg2 = get_operands()
            result = arg1 + arg2
            print(result)
        elif choice == 'S' or choice == 's': # Subtraction
            arg1, arg2 = get_operands()
            result = arg1 - arg2
            print(result)
        elif choice == 'P' or choice == 'p': # Print
            print(result)
        elif choice == 'H' or choice == 'h': # Help
            help_screen()
```

```

        elif choice == 'Q' or choice == 'q': # Quit
            done = True

main()

```

The `help_screen` function needs no information from `main`, nor does it return a result. It behaves exactly the same way each time it is called.

Note the type hint for a tuple of floating-point numbers, the type the `get_operands` function returns:

```
tuple[float, float]
```

7.5.3 Restricted Input

Listing 5.40 (`betterinputonly.py`) forces the user to enter a value within a specified range. We now can easily adapt that concept to a function. Listing 7.18 (`betterinputfunc.py`) uses a function named `get_int_in_range` that does not return until the user supplies a proper value.

Listing 7.18: `betterinputfunc.py`

```

def get_int_in_range(first: int, last: int) -> int:
    """
    Forces the user to enter an integer within a specified range.
    first is either a minimum or maximum acceptable value.
    last is the corresponding other end of the range, either a
    maximum or minimum value.
    Returns an acceptable value from the user.
    """
    # If the larger number is provided first, switch the parameters
    if first > last:
        first, last = last, first
    # Insist on values in the range first...last
    prompt = f'Please enter values in the range {first}...{last}: '
    in_value = int(input(prompt))
    while in_value < first or in_value > last:
        print(f'{in_value} is not in the range {first}...{last}')
        in_value = int(input('Please try again: '))
    # in_value at this point is guaranteed to be within range
    return in_value

def main() -> None:
    """
    Tests the get_int_in_range function
    """
    print(get_int_in_range(10, 20))
    print(get_int_in_range(20, 10))
    print(get_int_in_range(5, 5))
    print(get_int_in_range(-100, 100))

main() # Run the program

```


Listing 7.18 (`betterinputfunc.py`) forces the user to enter a value within a specified range, as shown in this sample run:

```
Please enter values in the range 10...20: 4
4 is not in the range 10...20
Please try again: 21
21 is not in the range 10...20
Please try again: 16
16
Please enter values in the range 10...20: 10
10
Please enter values in the range 5...5: 4
4 is not in the range 5...5
Please try again: 6
6 is not in the range 5...5
Please try again: 5
5
Please enter values in the range -100...100: -101
-101 is not in the range -100...100
Please try again: 101
101 is not in the range -100...100
Please try again: 0
0
```

This functionality could be useful in many programs. In Listing 7.18 (`betterinputfunc.py`)

- Parameters delimit the high and low values. This makes the function more flexible since it can be used elsewhere in the program with a completely different range specified and still work correctly.
- Programmers are supposed to call the function with the lower number passed as the first parameter and the higher number passed as the second parameter. The function also will accept the parameters out of order and automatically swap them to work as expected; thus,

```
num = get_int_in_range(20, 50)
```

will work exactly like

```
num = get_int_in_range(50, 20)
```

7.5.4 Better Die Rolling Simulator

Listing 7.19 (`betterdie.py`) reorganizes Listing 6.11 (`die.py`) into functions.

Listing 7.19: betterdie.py

```

from random import randrange

def show_die(spots: int) -> None:
    """
    Draws a picture of a die with number of spots indicated.
    spots is the number of spots on the top face.
    """
    print('+-----+')
    if spots == 1:
        print('|       |')
        print('|    *   |')
        print('|       |')
    elif spots == 2:
        print('| *     |')
        print('|       |')
        print('|    *  |')
    elif spots == 3:
        print('|     * |')
        print('|    *  |')
        print('|  *    |')
    elif spots == 4:
        print('| * *   |')
        print('|       |')
        print('| * *   |')
    elif spots == 5:
        print('| * * * |')
        print('|    *  |')
        print('| * *   |')
    elif spots == 6:
        print('| * * * |')
        print('|       |')
        print('| * * * |')
    else:
        print(' *** Error: illegal die value ***')
    print('+-----+')

def roll() -> int:
    """ Returns a pseudorandom number in the range 1...6, inclusive """
    return randrange(1, 7)

def main() -> None:
    """ Simulates the roll of a die three times """
    # Roll the die three times
    for i in range(0, 3):
        show_die(roll())

main()    # Run the program

```

In Listing 7.19 (betterdie.py), the `main` function is oblivious to the details of pseudorandom number

generation. Also, `main` is not responsible for drawing the die. These important components of the program are now in functions, so programmers can perfect their details independently from the `main` function.

Note how we pass the result of the call of the `roll` function directly as an argument to `show_die`:

```
show_die(roll())
```

This is another example of *function composition*. Function composition is not new to us; we have been using with the standard functions `input` and `int` in statements like: statements like

```
x = int(input())
```

7.5.5 Tree Drawing Function

Listing 7.20 (`treefunc.py`) reorganizes Listing 5.35 (`startree.py`) into functions.

Listing 7.20: `treefunc.py`

```
def tree(height: int) -> None:
    """
    Draws a tree of a given height.
    """
    row = 0                # First row, from the top, to draw
    while row < height:    # Draw one row for every unit of height
        # Print leading spaces
        count = 0
        while count < height - row:
            print(end=' ')
            count += 1
        # Print out stars, twice the current row plus one:
        # 1. number of stars on left side of tree
        #    = current row value
        # 2. exactly one star in the center of tree
        # 3. number of stars on right side of tree
        #    = current row value
        count = 0
        while count < 2*row + 1:
            print(end='*')
            count += 1
        # Move cursor down to next line
        print()
        # Change to the next row
        row += 1

def main() -> None:
    """ Allows users to draw trees of various heights """
    height = int(input('Enter height of tree: '))
    tree(height)

main()
```

Observe that Listing 7.20 (`treefunc.py`) uses the name `height` as a local variable in `main` and as a formal parameter name in the `tree` function. There is no conflict here, and the two `height` variables represent two distinct quantities. Furthermore, the fact that the statement

```
tree(height)
```

uses `main`'s `height` as an actual parameter and `height` happens to be the name as the formal parameter is simply a coincidence. The function call binds the value of `main`'s `height` variable to the formal parameter in `tree` also named `height`. The interpreter can keep track of which `height` is which based on the function in which it appears in the Python source code.

7.5.6 Floating-point Equality

Recall from Listing 3.2 (`imprecise.py`) that floating-point numbers are not mathematical real numbers; a floating-point number is finite and is represented internally as a quantity with a binary mantissa and exponent. Just as we cannot represent $1/3$ as a finite decimal in the base 10 number system, we cannot represent $1/10$ exactly in the binary (base 2) number system with a fixed number of digits. Often, no problems arise from this imprecision, and in fact for decades programmers have developed many software applications performing precise calculations using floating-point numbers. The stakes can be high, however. Consider the task of directing a spacecraft to a distant planet. Due to the distances involved, even a small error in its trajectory computation can result in the mission's complete failure. We can use floating-point numbers safely and effectively, but we must use appropriate care.

To build our confidence with floating-point numbers, consider Listing 7.21 (`simplefloataddition.py`), which adds two double-precision floating-point numbers and checks for a given value.

Listing 7.21: `simplefloataddition.py`

```
def main() -> None:
    x = 0.9
    x += 0.1
    if x == 1.0:
        print('OK')
    else:
        print('NOT OK')

main()
```

Listing 7.21 (`simplefloataddition.py`) reports

```
OK
```

All seems well judging from the behavior of Listing 7.21 (`simplefloataddition.py`). Next, consider Listing 7.22 (`badfloatcheck.py`) which attempts to control a loop with a double-precision floating-point number.

Listing 7.22: badfloatcheck.py

```
def main() -> None:
    # Count to ten by tenths
    i = 0.0
    while i != 1.0:
        print("i =", i)
        i += 0.1

main()
```

When executed, Listing 7.22 (badfloatcheck.py) begins as expected, but it does not end as expected:

```
i = 0.0
i = 0.1
i = 0.2
i = 0.30000000000000004
i = 0.4
i = 0.5
i = 0.6
i = 0.7
i = 0.7999999999999999
i = 0.8999999999999999
i = 0.9999999999999999
i = 1.0999999999999999
i = 1.2
i = 1.3
i = 1.4000000000000001
i = 1.5000000000000002
i = 1.6000000000000003
i = 1.7000000000000004
i = 1.8000000000000005
i = 1.9000000000000006
i = 2.0000000000000004
```

We expect the program to stop when the loop variable `i` equals 1, but the program continues executing until the user types **Ctrl C** or otherwise interrupts the program's execution. We are adding 0.1, just as in Listing 7.21 (simplefloataddition.py), but now there is a problem. Since 0.1 has no exact representation within the constraints of the binary double-precision floating-point number systems, the repeated addition of 0.1 leads to round off errors that accumulate over time. Whereas 0.1 + 0.9 rounded off may equal 1, we see that 0.1 added to itself 10 times yields 0.9999999999999999 which is not exactly 1.

Listing 7.22 (badfloatcheck.py) demonstrates that the `==` and `!=` operators are of questionable worth when comparing floating-point values. The better approach is to check to see if two floating-point values are *close enough*, which means they differ by only a very small amount. When comparing two floating-point numbers x and y , we essentially must determine if the absolute value of their difference is small; for example, $|x - y| < 0.00001$. We can construct an `equals` function and incorporate the `fabs` function introduced in 6.4. Listing 7.23 (floatequalsfunction.py) provides such an `equals` function.

Listing 7.23: floatequalsfunction.py

```

from math import fabs

def equals(a: float, b: float, tolerance: float) -> bool:
    """
    Returns true if a = b or |a - b| < tolerance.
    If a and b differ by only a small amount (specified by tolerance),
    a and b are considered "equal." Useful to account for floating-point
    round-off error. The == operator is checked first since some special
    floating-point values such as floating-point infinity require an
    exact equality check.
    """
    return a == b or fabs(a - b) < tolerance

def main() -> None:
    """ Try out the equals function """
    i = 0.0
    while not equals(i, 1.0, 0.0001):
        print("i =", i)
        i += 0.1

main()

```

The third parameter, named `tolerance`, specifies how close the first two parameters must be in order to be considered equal. The `==` operator must be used for some special floating-point values such as the floating-point representation for infinity, so the function checks for `==` equality as well. Since Python uses short-circuit evaluation for Boolean expressions involving logical *OR* (see 4.2), if the `==` operator indicates equality, the executing program will not perform the more elaborate check. The program performs the second check with `fabs` and `tolerance` only if the `==` expression evaluates to false.

The output of Listing 7.23 (`floatequalsfunction.py`) is

```

i = 0.0
i = 0.1
i = 0.2
i = 0.30000000000000004
i = 0.4
i = 0.5
i = 0.6
i = 0.7
i = 0.7999999999999999
i = 0.8999999999999999

```

You should use a function like `equals` when comparing two floating-point values for equality. The Python `math` module provides the `is_close` that tests for floating-point equality. Programmers can specify the tolerance for `math.is_close`, but the default 1×10^{-9} generally is good enough.

7.6 Refactoring to Eliminate Code Duplication

Recall Listing 6.21 (noanimation.py) that uses Turtle graphics to draw a collection of lines of various colors. Consider the following code fragment, snipped from Listing 6.21 (noanimation.py):

```
turtle.color("blue")
for x in range(10):
    turtle.penup()
    turtle.setposition(-200, y)
    turtle.pendown()
    turtle.forward(400)
    y += 10

turtle.color("green")
for x in range(10):
    turtle.penup()
    turtle.setposition(-200, y)
    turtle.pendown()
    turtle.forward(400)
    y += 10
```

What is interesting to note about this code snippet is the similarities between the two sections. These two sections of code are performing exactly the same work, except for the variation of color. Listing 6.21 (noanimation.py) contains two additional sections not shown here that exhibit the same similarities. This code represents duplication of coding effort, and this is known as *code duplication*.

Code duplication is undesirable for several reasons:

- The extra code to write requires more work on the part of the programmer. Copying and pasting with a good editor can save typing, but the code fragments are not identical in all ways, so a programmer must be sure to make all of the little changes required to produce the desired variance in behavior.
- Code duplication results in code that is more difficult to maintain. Most commercial software is not static; developers constantly work on newer versions. Newer versions typically provide bug fixes and add features. Upon repairing a logic error or enhancing a section of duplicated code, a programmer must track down all the other code sections within the system that duplicate the modified section. Failure to do so can introduce inconsistency into the program's behavior.

Functions are ideal for consolidating duplicate code. Through a process known as code *refactoring*, a programmer can place the common code within a function definition and tune the minor variance in behavior of the duplicated code via parameters. Listing 7.24 (noduplication.py) refactors the duplicated code in Listing 6.21 (noanimation.py) into one function that accepts a color and y value as a parameter.

Listing 7.24: noduplication.py

```
import turtle

def draw_lines(color: str, y: int) -> None:
    """ Draws 10 horizontal lines of a given color stacked
        on top of each other with the lowest line appearing
        at position y on the y axis. """
    turtle.color(color)
    for x in range(10):
        turtle.penup()
        turtle.setposition(-200, y)
        turtle.pendown()
        turtle.forward(400)
        y += 10

# Turn off animation
turtle.tracer(0)
draw_lines("red", -200)
draw_lines("blue", -100)
draw_lines("green", 0)
draw_lines("black", 100)

turtle.update()      # Ensure all of image is drawn
turtle.done()
```

Now that the originally duplicated code appears in only one place in Listing 7.24 (noduplication.py), any correction or enhancement made to the code within the function automatically will be available to all the callers of that code. This removes the possibility of introducing an inconsistency into the application.

7.7 Custom Functions versus Standard Functions

Recall the custom square root code we saw in Listing 5.34 (computesquareroot.py). We can package this code in a function. Just like the standard `math.sqrt` function, our custom square root function will accept a single numeric value and return a numeric result. Listing 7.25 (customsquareroot.py) contains the definition of our custom `square_root` function.

Listing 7.25: customsquareroot.py

```

# File customsquareroot.py

def square_root(val: float) -> float:
    """ Compute an approximation of the square root of x """
    # Compute a provisional square root
    root = 1.0

    # How far off is our provisional root?
    diff = root*root - val

    # Loop until the provisional root
    # is close enough to the actual root
    while diff > 0.00000001 or diff < -0.00000001:
        root = (root + val/root) / 2    # Compute new provisional root
        # How bad is our current approximation?
        diff = root*root - val

    return root

# Use the standard square root function to compare with our custom function
from math import sqrt

d = 1.0
while d <= 10.0:
    print(f'{d:6.1f}: {square_root(d):16.8f} {sqrt(d):16.8f}')
    d += 0.5    # Next d

```

Listing 7.25 (customsquareroot.py) compares the behavior of our custom `square_root` function to the `sqrt` library function. The program produces the following output:

1.0:	1.00000000	1.00000000
1.5:	1.22474487	1.22474487
2.0:	1.41421356	1.41421356
2.5:	1.58113883	1.58113883
3.0:	1.73205081	1.73205081
3.5:	1.87082869	1.87082869
4.0:	2.00000000	2.00000000
4.5:	2.12132034	2.12132034
5.0:	2.23606798	2.23606798
5.5:	2.34520788	2.34520788
6.0:	2.44948974	2.44948974
6.5:	2.54950976	2.54950976
7.0:	2.64575131	2.64575131
7.5:	2.73861279	2.73861279
8.0:	2.82842713	2.82842712
8.5:	2.91547595	2.91547595
9.0:	3.00000000	3.00000000
9.5:	3.08220700	3.08220700
10.0:	3.16227766	3.16227766

Observe there is only a slight difference for $\sqrt{8}$, and return value of the two functions differ by only 0.00000001.

The fact that we found one difference in this small collection of test cases justifies using the standard `math.sqrt` function instead of our custom function. Generally speaking, if you have the choice of using a standard library function or writing your own custom function that provides the same functionality, choose the standard library routine. The advantages of using the standard library routine include:

- You eliminate entirely your effort to produce the custom code, and so you can devote more effort to other parts of the application's development.
- If you write your own custom code, you must thoroughly test it to ensure its correctness; standard library code, while not immune to bugs, generally has been subjected to a complete test suite. Additionally, many developers use functions from the Python standard library, and this large volume of use usually exposes any lurking errors early. By contrast, only you exercise your custom functions by the programs you write, and any errors in the functions may not become apparent immediately. If a wide audience does not use your custom functions, its bugs may lie dormant for a long time. Standard library routines are well known and trusted; custom code, due to its limited exposure, is suspect until it gains wider exposure and adoption.
- Standard routines typically are tuned to be very efficient; it takes a great deal of time and effort to make custom code efficient.
- Standard routines are well-documented; it takes time to write good documentation for custom functions.

Listing 7.26 (`squarerootcomparison.py`) tests a slightly simplified version of our custom square root function over a range of 100,000,000 floating point values. It begins with zero, goes to 10,000, and adds 0.0001 each iteration.

Listing 7.26: squarerootcomparison.py

```

from math import fabs, sqrt

def equals(a: float, b: float, tolerance: float) -> bool:
    """
    Consider two floating-point numbers equal when the
    difference between them is very small.
    Returns true if  $a = b$  or  $|a - b| < \text{tolerance}$ .
    If  $a$  and  $b$  differ by only a small amount (specified
    by tolerance),  $a$  and  $b$  are considered "equal."
    Useful to account for floating-point round-off error.
    Uses the == operator first since some special
    floating-point values such as floating-point infinity
    require an exact equality check.
    """
    return a == b or fabs(a - b) < tolerance

def square_root(val: float) -> float:
    """ Computes the approximate square root of val. """
    # Compute an initial provisional square root
    root = 1.0

    # Loop until the provisional root is close enough to the actual root
    while fabs(root*root - val) > 0.00000001:
        root = (root + val/root) / 2 # Compute new provisional root
    return root

def main() -> None:
    d = 0.0
    while d < 10000.0:
        if not equals(square_root(d), sqrt(d), 0.00000001):
            print('*** Difference detected for', d)
            print(' Expected', sqrt(d))
            print(' Computed', square_root(d))
            d += 0.0001 # Consider next value

main()

```

Listing 7.26 (squarerootcomparison.py) uses our `equals` function from Listing 4.7 (floatequals.py). Observe that the tolerance used within the square root computation is smaller than the tolerance `main` uses to check the result. The `main` function, therefore, uses a less strict notion of equality. The output of Listing 7.26 (squarerootcomparison.py) is

```

*** Difference detected for 0.0
Expected 0.0
Computed 6.103515625e-05
*** Difference detected for 0.0006000000000000000001
Expected 0.024494897427831782
Computed 0.024495072155655266

```

shows that our custom square root function produces results outside of `main`'s acceptable tolerance for just two values. Two wrong answers out of one-hundred million tests represents a 0.000002% error rate. While this error rate is very small, it indicates a difference in behavior between our `square_root` function and the trusted `math.sqrt` function. Testing a different range of values may reveal other differences. The best approach is to use the standard library functions

Note that we can use the `math.isclose` function to eliminate our custom `equals` function and simplify our custom square root function even further, as Listing 7.27 (`squarerootwithisclose.py`) shows.

Listing 7.27: `squarerootwithisclose.py`

```
from math import isclose, sqrt

def square_root(val: float) -> float:
    """ Computes the approximate square root of val. """
    # Compute an initial provisional square root
    root = 1.0

    # Loop until the provisional root is close enough to the actual root
    while not isclose(root*root, val):
        root = (root + val/root) / 2 # Compute new provisional root
    return root

def main() -> None:
    d = 0.0
    while d < 10000.0:
        if not isclose(square_root(d), sqrt(d)):
            print('*** Difference detected for', d)
            print(' Expected', sqrt(d))
            print(' Computed', square_root(d))
        d += 0.0001 # Consider next value

main()
```

The `math.isclose` uses a tighter tolerance by default, 0.000000001. The only difference it detects over this range of values is at zero, and the difference is miniscule:

```
*** Difference detected for 0.0
Expected 0.0
Computed 1.1113793747425387e-162
```

7.8 Exercises

1. Is the following a legal Python program?

```
def proc(x):
    return x + 2
```

```
def proc(n):  
    return 2*n + 1
```

```
def main():  
    x = proc(5)
```

```
main()
```

2. Is the following a legal Python program?

```
def proc(x):  
    return x + 2
```

```
def main():  
    x = proc(5)  
    y = proc(4)
```

```
main()
```

3. Is the following a legal Python program?

```
def proc(x):  
    print(x + 2)
```

```
def main():  
    x = proc(5)
```

```
main()
```

4. Is the following a legal Python program?

```
def proc(x, y):  
    return 2*x + y*y
```

```
def main():  
    print(proc(5, 4))
```

```
main()
```

5. Is the following a legal Python program?

```
def proc(x, y):  
    return 2*x + y*y
```

```
def main():  
    print(proc(5))
```

```
main()
```

6. Is the following a legal Python program?

```
def proc(x):  
    return 2*x
```

```
def main():  
    print(proc(5, 4))
```

```
main()
```

7. Is the following a legal Python program?

```
def proc(x):  
    print(2*x*x)
```

```
def main():  
    proc(5)
```

```
main()
```

8. The programmer was expecting the following program to print 200. What does it print instead? Why does it print what it does?

```
def proc(x):  
    x = 2*x*x
```

```
def main():  
    num = 10  
    proc(num)  
    print(num)
```

```
main()
```

9. Is the following program legal since the variable `x` is used in two different places (`proc` and `main`)? Why or why not?

```
def proc(x):  
    return 2*x*x
```

```
def main():
    x = 10
    print(proc(x))
```

```
main()
```

10. Is the following program legal since the actual parameter has a different name from the formal parameter (y versus x)? Why or why not?

```
def proc(x):
    return 2*x*x
```

```
def main():
    y = 10
    print(proc(y))
```

```
main()
```

11. Complete the following distance function that computes the distance between two geometric points (x_1, y_1) and (x_2, y_2) :

```
def distance(x1, y1, x2, y2):
    ...
```

Test it with several points to convince yourself that is correct.

12. What happens if a caller passes too many parameters to a function?
13. What happens if a caller passes too few parameters to a function?
14. What are the rules for naming a function in Python?
15. Consider the following function definitions:

```
from random import randrange
```

```
def fun1(n):
    result = 0
    while n:
        result += n
        n -= 1
    return result
```

```
def fun2(stars):
    for i in range(stars + 1):
        print(end="★")
    print()
```

```
def fun3(x, y):  
    return 2*x*x + 3*y  
  
def fun4(n):  
    return 10 <= n <= 20  
  
def fun5(a, b, c):  
    return a <= b if b <= c else false  
  
def fun6():  
    return randrange(0, 2)
```

Examine each of the following statements. If the statement is illegal, explain why it is illegal; otherwise, indicate what the statement will print.

- (a) `print(fun1(5))`
- (b) `print(fun1())`
- (c) `print(fun1(5, 2))`
- (d) `print(fun2(5))`
- (e) `fun2(5)`
- (f) `fun2(0)`
- (g) `fun2(-2)`
- (h) `print(fun3(5, 2))`
- (i) `print(fun3(5.0, 2.0))`
- (j) `print(fun3('A', 'B'))`
- (k) `print(fun3(5.0))`
- (l) `print(fun3(5.0, 0.5, 1.2))`
- (m) `print(fun4(15))`
- (n) `print(fun4(5))`
- (o) `print(fun4(5000))`
- (p) `print(fun5(2, 4, 6))`
- (q) `print(fun5(4, 2, 6))`
- (r) `print(fun5(2, 2, 6))`
- (s) `print(fun5(2, 6))`
- (t)

```
if fun5(2, 2, 6):  
    print("Yes")  
else:  
    print("No")
```
- (u) `print(fun6())`
- (v) `print(fun6(4))`

(w) `print(fun3(fun1(3), 3))`
(x) `print(fun3(3, fun1(3)))`
(y) `print(fun1(fun1(fun1(3))))`
(z) `print(fun6(fun6()))`

Chapter 8

More on Functions

This chapter covers some additional aspects of functions in Python. It introduces recursion, a key concept in computer science.

8.1 Global Variables

Variables defined within functions are local variables. Local variables have some very desirable properties:

- A local variable exists in the computer's memory only when the variable is in scope; that is, the variable exists only during the function's execution. When the program's execution leaves the scope of a local variable, the program frees up the memory for that variable. The program then can reuse this same memory for the local variables of other functions when they execute.
- Programmers can use the same variable name in different functions without any conflict. The interpreter derives all of its information about a local variable from that variable's definition within the function. If the interpreter attempts to execute a statement that uses a variable that has not been defined by a previous statement within the function, the interpreter issues a run-time error. When executing code in one function the interpreter will not look for a variable definition in another function. Thus, there is no way a local variable in one function can interfere with a local variable defined in another function.

A local variable is transitory, so it disappears in between function invocations. Sometimes it is desirable to have a variable that exists apart from any function executions. In contrast to a local variable, a *global variable* lives outside of all functions and is not local to any particular function. This means any function is capable of accessing and/or modifying a global variable.

A variable is defined when it is assigned to an object. Any variable assigned within a function is local to that function, unless the variable is declared to be a global variable using the `global` reserved word. Listing 8.1 (`globalcalculator.py`) is a modification of Listing 7.17 (`calculator.py`) that uses global variables named `result`, `arg1`, and `arg2` that are shared by several functions in the program.

Listing 8.1: globalcalculator.py

```
def help_screen() -> None:
    """
    Displays information about how the program works.
    Accepts no parameters.
    Returns nothing.
    """
    print('Add:      Adds two numbers')
    print('Subtract:  Subtracts two numbers')
    print('Print:      Displays the result of the latest operation')
    print('Help:       Displays this help screen')
    print('Quit:       Exits the program')

def menu() -> str:
    """
    Display a menu.
    Accepts no parameters.
    Returns the string entered by the user.
    """
    # Display a menu
    return input('=== A)dd S)ubtract P)rint H)elp Q)uit ==> ')

# Global variables used by several functions
result = 0.0
arg1 = 0.0
arg2 = 0.0

def get_input() -> None:
    """
    Assigns the globals arg1 and arg2 from user keyboard input.
    """
    global arg1, arg2 # arg1 and arg2 are globals
    arg1 = float(input('Enter argument #1: '))
    arg2 = float(input('Enter argument #2: '))

def report() -> None:
    """ Reports the value of the global result """
    # Not assigning to result, global keyword not needed
    print(result)

def add() -> None:
    """
    Assigns the sum of the globals arg1 and arg2
    to the global variable result.
    """
    global result # Assigning to result, global keyword needed
    result = arg1 + arg2

def subtract() -> None:
```

```

"""
Assigns the difference of the globals arg1 and arg2
to the global variable result
"""
global result  # Assigning to result, global keyword needed
result = arg1 - arg2

def main() -> None:
    """
    Runs a command loop that allows users to
    perform simple arithmetic.
    """
    done = False  # Initially not done
    while not done:
        choice = menu()  # Get user's choice

        if choice == 'A' or choice == 'a':  # Addition
            get_input()
            add()
            report()
        elif choice == 'S' or choice == 's':  # Subtraction
            get_input()
            subtract()
            report()
        elif choice == 'P' or choice == 'p':  # Print
            report()
        elif choice == 'H' or choice == 'h':  # Help
            help_screen()
        elif choice == 'Q' or choice == 'q':  # Quit
            done = True

main()

```

Listing 8.1 (globalcalculator.py) uses global variables `result`, `arg1`, and `arg2`. These names no longer appear in the `main` function. The program accesses and/or modifies these global variables in four different functions: `get_input`, `report`, `add`, and `subtract`. The `global` keyword within a function's block of code identifies the variables which are global variables. Notice that if a function uses a global variable without assigning its value, the `global` declaration is not necessary. This is because variable assignment is variable definition, and a local variable must be defined within a function.



A function may use a global variable without declaring it with the `global` keyword if the function does not assign a variable of that name anywhere in its body. A function that assigns a global variable must declare that variable as global with the `global` keyword.

If a function defines a local variable with the same name as a global variable, the global variable becomes inaccessible to code within the function. We say the local variable *hides* the like-named global variable from code in the function's body.

When is it acceptable to use global variables, and when is it better to use local variables? In general, local variables are preferable to global variables for several reasons:

- When a function uses local variables exclusively and performs no other input operations (like calling the `input` function), only parameters passed in by the caller can influence the function's behavior. If a global variable appears in a function, the function's behavior potentially is affected by every other function that can modify that global variable. As a simple example, consider the following trivial function that appears in a program:

```
def increment(n: int) -> int:
    return n + 1
```

Can you predict what the following statement within that program will print?

```
print(increment(12))
```

If your guess is 13, you are correct. The `increment` function simply returns the result of adding one to its argument. The `increment` function behaves the same way each time it is called with the same argument.

Next, consider the following three functions that appear in some program:

```
def process(n: int) -> int:
    return n + m      # m is a global integer variable
```

```
def assign_m() -> None:
    global m
    m = 5
```

```
def inc_m() -> None:
    global m
    m += 1
```

Can you predict what the following statement within the program will print?

```
print(process(12))
```

We cannot predict what this statement in isolation will print. The following scenarios all produce different results:

```
assign_m()
print(process(12))
```

prints 17,

```
m = 10      # m is the global
print(process(12))
```

prints 22,

```
m = 0      # m is the global
inc_m()
inc_m()
print(process(12))
```

```
prints 14, and
assign_m()
inc_m()
inc_m()
print(process(12))
```

prints 19. The identical printing statements print different values depending on the cumulative effects of the program's execution up to that point.

It may be difficult to locate an error if a function that uses a global variable fails because it may be the fault of *another* function that assigned an incorrect value to the global variable. The situation may be more complicated than the one portrayed in the simple examples above; consider:

```
assign_m() -> None
.
.   # 30 statements in between, some of which may change a,
.   # b, and m
.
if a < 2 and b <= 10:
    m = a + b - 100
.
.   # 20 statements in between, some of which may change m
.
print(process(12))
```

- A nontrivial program that uses global variables will be more difficult for a human reader to understand than one that does not. When examining the contents of a function, a global variable requires the reader to look elsewhere (outside the function) for its meaning:

```
# Linear function
def f(x: float) -> float:
    return m*x + b
```

What are *m* and *b*? How, where, and when are they assigned or re-assigned?

- If a function uses only local variables, we can test it for correctness in isolation from other functions, since other functions cannot affect the behavior of this function. Only the function's parameters, if any, can influence the function's behavior.

The exclusion of global variables from a function leads to *functional independence*. A function that depends on information outside of its scope to correctly perform its task is a *dependent function*. When a function operates on a global variable it depends on that global variable being in the correct state for the function to complete its task correctly. Nontrivial programs that contain many dependent functions are more difficult debug and extend. A truly independent function that use no global variables and uses no programmer-defined functions to help it out can be tested for correctness in isolation. Additionally, we can copy an independent function from one program and paste it into another program, and it will work without modification. Functional independence is a desirable quality.

The exclusion of global variables from a function's definition does not guarantee that the function always will produce the same results given the same parameter values; consider

```
def compute(n: int) -> int:
    favorite = int(input("Please enter your favorite number: "))
    return n + favorite
```

The `compute` function avoids global variables, yet we cannot predict the value of the expression `compute(12)`. Recall the `increment` function from above:

```
def increment(n: int) -> int:
    return n + 1
```

Its behavior is totally predictable. Furthermore, `increment` does not modify any global variables, meaning its code all by itself cannot in any way influence the overall program's behavior. We say that `increment` is a *pure function*. A pure function cannot perform any input or output (for example, use the `print` or `input` statements), nor may it use global variables. While `increment` is pure, the `compute` function is impure. The following function is impure also, since it performs output:

```
def increment_and_report(n: int) -> int:
    print("Incrementing", n)
    return n + 1
```

A pure function simply computes its return value and has no other observable side effects.

A function that calls only other pure functions and otherwise would be considered pure is itself a pure function; for example:

```
def double_increment(n: int) -> int:
    return increment(n) + 1
```

`double_increment` is a pure function since `increment` is pure; however, `double_increment_with_report`:

```
def double_increment_with_report(n: int) -> int:
    return increment_and_report(n) + 1
```

is not a pure function since it calls `increment_and_report`, which is impure.

8.2 Default Parameters

We have seen how callers may invoke some Python functions with differing numbers of parameters. Compare

```
a = input()
```

to

```
a = input('Enter your name: ')
```

We can define our own functions that accept a varying number of parameters by using a technique known as *default parameters*. Consider the following function that counts down:

```
def countdown(n: int=10):
    for count in range(n, -1, -1): # Count down from n to zero
        print(count)
```

The formal parameter expressed as `n: int=10` represents a default parameter or default argument. If the caller does not supply an actual parameter, the executing program assigns 10 to the formal parameter `n`. The following call

```
countdown()
```

prints

```
10
9
8
7
6
5
4
3
2
1
0
```

but the invocation

```
countdown(5)
```

displays

```
5
4
3
2
1
0
```

As we can see, when the caller does not supply a parameter specified by a function, and that parameter has a default value, the function invocation uses the default value.

We may mix non-default and default parameters in the parameter lists of a function declaration, but all default parameters within the parameter list must appear after all the non-default parameters. This means the following definitions are acceptable:

```
# OK, default follows non-default
def sum_range(n: int, m: int=100) -> int:
    sum = 0
    for val in range(n, m + 1):
        sum += val
    return sum
```

and

```
# OK, both parameters default
def sum_range(n: int=0, m: int=100) -> int:
    sum = 0
    for val in range(n, m + 1):
        sum += val
    return sum
```

but the following definition is illegal, since a default parameter precedes a non-default parameter in the function's parameter list:

```
# Illegal, non-default follows default
def sum_range(n: int=0, m: int) -> int:
```



```

sum = 0
for val in range(n, m + 1):
    sum += val
return sum

```

Default arguments allow programmers to provide a highly tunable function that offer a simpler interface for its typical uses. Recall Listing 7.6 (`regularpolygon.py`) that uses Turtle graphics to draw a regular polygon with specified number of sides, of a given size, location, and color. Listing 8.2 (`enhancedpolygon.py`) enhances Listing 7.6 (`regularpolygon.py`) by adding the ability to optionally fill the polygon with a color. Since the number of parameters is becoming unwieldy for a caller to manage, the color and fill directives are specified as optional.

Listing 8.2: `enhancedpolygon.py`

```

import turtle
import random

# Draws a regular polygon with the given number of sides.
# The length of each side is length.
# The pen begins at point(x, y).
# The color of the polygon is color (defaults to black).
# The polygon is rendered solid if fill is True (defaults to False).
def polygon(sides:int, length: int, x: int, y: int,
            color: str='black', fill: bool=False) -> None:
    turtle.penup()
    turtle.setposition(x, y)
    turtle.pendown()
    turtle.color(color)
    if fill:
        turtle.begin_fill()
    for i in range(sides):
        turtle.forward(length)
        turtle.left(360//sides)
    if fill:
        turtle.end_fill()

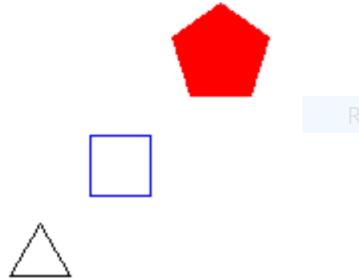
# Disable rendering to speed up drawing
turtle.hideturtle()
turtle.tracer(0)

# Draw a few polygons
polygon(3, 30, 10, 10)                # Back triangle outline
polygon(4, 30, 50, 50, 'blue')        # Blue square outline
polygon(5, 30, 100, 100, 'red', True)  # Red solid pentagon

turtle.update() # Render image
turtle.exitonclick() # Wait for user's mouse click

```

Observe that the caller must pass at least four parameters and optionally can pass one or two more. The idea here is that when drawing a polygon it is important to know how many sides it has, the length of each side, and its location. Such a polygon will consist of a black outline of the shape. Callers can enhance the figure by specifying a color for the outline. Finally, a caller can indicate that the polygon should be filled.

Figure 8.1 The shapes drawn by Listing 8.2 (enhancedpolygon.py)

The color and fill are optional extras that are not required just to draw a basic polygon. Figure 8.1 shows the shapes drawn by Listing 8.2 (enhancedpolygon.py).

Note that it is not possible to produce a filled polygon without also providing a color; for example, the call

```
polygon(5, 30, 150, 150, True) # Black solid pentagon?
```

would attempt to assign 5 to the sides formal parameter (which is fine), 30 to length (also fine), 150 to x (so far, so good), 150 to y (again, okay), and `True` to color (what about this?). Inside the definition of the `polygon` function, the statement

```
turtle.color(color)
```

with `color` bound to `True` will generate an exception because `True` is not one of the color strings like `'red'`, `'blue'`, etc. that the `turtle.color` function expects. Such a call of the `polygon` function, therefore, will fail. Default parameters always substitute for missing parameters from back to front in a function's parameter list.

8.3 Introduction to Recursion

The *factorial* function is widely used in combinatorial analysis (counting theory in mathematics), probability theory, and statistics. The expression $n!$ represents the factorial function applied to n . Factorial is defined for nonnegative integers as

$$n! = n \cdot (n-1) \cdot (n-2) \cdot (n-3) \cdots 3 \cdot 2 \cdot 1$$

and $0!$ is defined to be 1. Thus $6! = 6 \cdot 5 \cdot 4 \cdot 3 \cdot 2 \cdot 1 = 720$. Mathematicians precisely define factorial in this way:

$$n! = \begin{cases} 1, & \text{if } n = 0 \\ n \cdot (n-1)!, & \text{otherwise.} \end{cases}$$

This definition is *recursive* since the `!` function is being defined, but `!` also appears in the definition. We can define a Python function recursively. Listing 8.3 (factorialtest.py) includes a factorial function that exactly models the mathematical definition.

Listing 8.3: factorialtest.py

```
def factorial(n: int) -> int:
    """ Computes n! Returns the factorial of n. """
    if n == 0:
        return 1
    else:
        return n * factorial(n - 1)

def main() -> None:
    """ Try out the factorial function """
    print(' 0! = ', factorial(0))
    print(' 1! = ', factorial(1))
    print(' 6! = ', factorial(6))
    print('10! = ', factorial(10))

main()
```

Listing 8.3 (factorialtest.py) produces

```
0! = 1
1! = 1
6! = 720
10! = 3628800
```

Observe that the `factorial` function in Listing 8.3 (factorialtest.py) uses no loop to compute its result. The `factorial` function simply calls itself. The call `factorial(6)` is computed as follows:

```
factorial(6) = 6 * factorial(5)
              = 6 * 5 * factorial(4)
              = 6 * 5 * 4 * factorial(3)
              = 6 * 5 * 4 * 3 * factorial(2)
              = 6 * 5 * 4 * 3 * 2 * factorial(1)
              = 6 * 5 * 4 * 3 * 2 * 1 * factorial(0)
              = 6 * 5 * 4 * 3 * 2 * 1 * 1
              = 6 * 5 * 4 * 3 * 2 * 1
              = 6 * 5 * 4 * 3 * 2
              = 6 * 5 * 4 * 6
              = 6 * 5 * 24
              = 6 * 120
              = 720
```

Note that we can optimize the `factorial` function slightly by changing the `if`'s condition from `n == 0` to `n < 2`. This change results in a function execution trace that eliminates one function call at the end:

```
factorial(6) = 6 * factorial(5)
             = 6 * 5 * factorial(4)
             = 6 * 5 * 4 * factorial(3)
             = 6 * 5 * 4 * 3 * factorial(2)
             = 6 * 5 * 4 * 3 * 2 * factorial(1)
             = 6 * 5 * 4 * 3 * 2 * 1
             = 6 * 5 * 4 * 3 * 2
             = 6 * 5 * 4 * 6
             = 6 * 5 * 24
             = 6 * 120
             = 720
```

Figure 8.2 illustrates the chain of recursive `factorial` invocations when executing the statement `print(factorial(6))`.

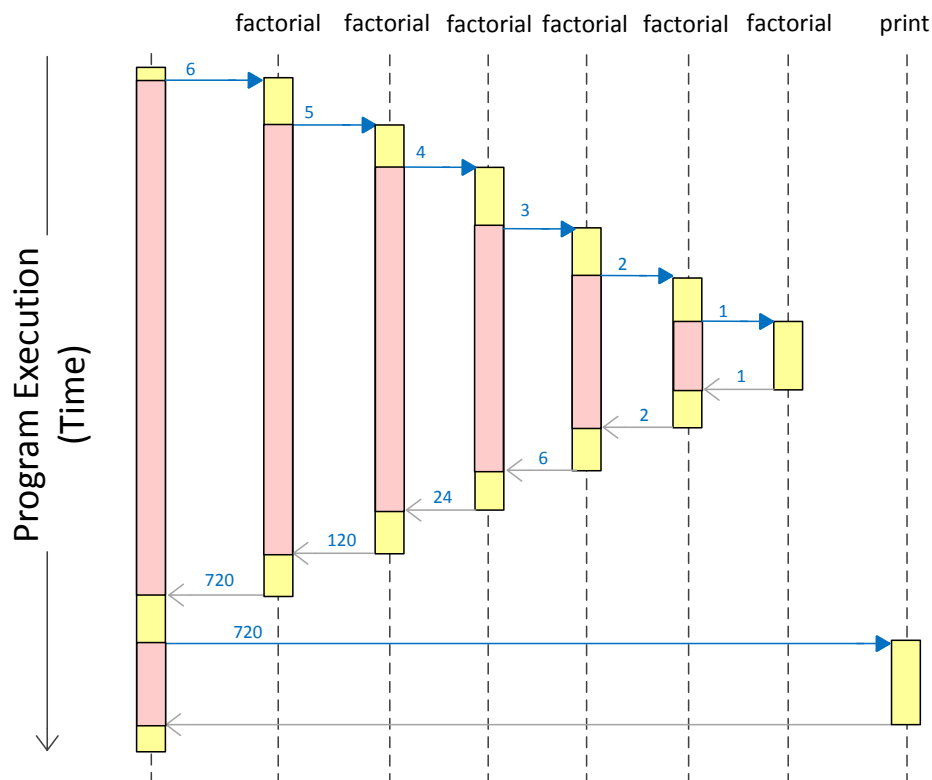
A correct simple recursive function definition is based on four key concepts:

1. The function optionally must call itself within its definition; this is the *recursive case*.
2. The function optionally must *not* call itself within its definition; this is the *base case*.
3. Some sort of conditional execution (such as an `if/else` statement) selects between the recursive case and the base case based on one or more parameters passed to the function.
4. Each invocation that does correspond to the base case must call itself with parameter(s) that move the execution closer to the base case. The function's recursive execution must converge to the base case.

Each recursive invocation must bring the function's execution closer to its base case. The `factorial` function calls itself in the `else` block of the `if/else` statement. Its base case executes if the condition of the `if` statement is true. Since the factorial is defined only for nonnegative integers, a caller's initial invocation of `factorial` would pass a value of zero or greater. A zero parameter (the base case) results in no recursive call. Any other positive parameter results in a recursive call with a parameter that is closer to zero than the one before. The nature of the recursive process progresses towards the base case, upon which the recursion terminates.

Recursion is not our only option when computing a factorial. Listing 8.4 (`nonrecursfact.py`) provides a nonrecursive factorial function.

Figure 8.2 A graphical representation of the chain of recursive `factorial` invocations when executing the statement `print(factorial(6))`, where the `factorial` function is from Listing 8.3 (`factorialtest.py`) with the condition optimized to $n < 2$. The vertical bars represent the time a function invocation is active. The shaded area within each bar represents the time that the function, while still active, is idle, waiting for a function it calls to complete. Note that during the process of recursion all earlier function invocations in the call chain remain active (but idle) until all the functions further down the call chain return.



Listing 8.4: nonrecursfact.py

```
def factorial(n: int) -> int:
    """ Computes n! Returns the factorial of n. """
    product = 1
    while n:
        product *= n
        n -= 1
    return product

def main() -> None:
    """ Try out the factorial function """
    print(' 0! = ', factorial(0))
    print(' 1! = ', factorial(1))
    print(' 6! = ', factorial(6))
    print('10! = ', factorial(10))

main()
```

Which `factorial` function is better, the recursive or non-recursive version? Generally, if both the recursive and non-recursive functions implement the same basic algorithm, the non-recursive function will be more efficient. A function call is a relatively expensive operation compared to a variable assignment or comparison. The body of the non-recursive `factorial` function invokes no functions, but the recursive version calls itself during all but the last recursive invocation. The iterative version of `factorial` is therefore more efficient than the recursive version.

Even though the iterative version of the factorial function technically is more efficient than the recursive version, on most systems you could not tell the difference. The reason is the factorial function “grows” fast, meaning it returns fairly large results for relatively small arguments.

Recall the `gcd` function from Section 7.5. It computed the greatest common divisor (also known as greatest common factor) of two integer values. It works, but it is not very efficient. Listing 8.5 (`gcd.py`) uses a better algorithm. It is based on one of the oldest algorithms known, attributed to Euclid around 300 B.C.

Listing 8.5: gcd.py

```

def gcd(m: int, n: int) -> int:
    """
    Uses Euclid's method to compute the greatest common divisor
    (also called greatest common factor) of m and n.
    Returns the GCD of m and n.
    """
    if n == 0:
        return m
    else:
        return gcd(n, m % n)

def iterative_gcd(num1: int, num2: int) -> int:
    """
    Uses a naive algorithm to compute the greatest common divisor
    (also called greatest common factor) of m and n.
    Returns the GCD of m and n.
    """
    # Determine the smaller of num1 and num2
    min = num1 if num1 < num2 else num2
    # 1 is definitely a common factor to all integers
    largest_factor = 1;
    for i in range(1, min + 1):
        if num1 % i == 0 and num2 % i == 0:
            largest_factor = i # Found larger factor
    return largest_factor

def main() -> None:
    """ Try out the gcd function """
    for num1 in range(1, 101):
        for num2 in range(1, 101):
            rect_result = gcd(num1, num2)
            iter_result = iterative_gcd(num1, num2)
            if rect_result != iter_result:
                print(f'gcd({num1}, {num2}) is {rect_result} or {iter_result}.')

main()

```

Note that this gcd function is recursive. The algorithm it uses is very different from our original iterative version. Because of the difference in the algorithms, this recursive version actually is much more efficient than our original iterative version. A recursive function, therefore, cannot be dismissed as inefficient just because it is recursive. We will revisit recursion in Section 15.4.

Listing 8.5 (gcd.py) prints nothing because the two greatest common divisor functions compute the same values over the given range of integers.

A recursive function may call itself within its definition multiple times. Consider the following sequence of integer values:

0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144, ...

This is beginning of the infinite *Fibonacci sequence* (see https://en.wikipedia.org/wiki/Fibonacci_

number). It is a sequence of integers beginning with 0 followed by 1. Subsequent elements of the sequence are the sum of their two immediately preceding elements; thus, the third number is $0 + 1 = 1$, the fourth number is $1 + 1 = 2$, the fifth number is $1 + 2 = 3$, etc. The numbers that comprise the Fibonacci sequence are known as *Fibonacci numbers*. Note that 3 is a Fibonacci number but 4 is not.

The mathematical properties of Fibonacci numbers have bearing in such diverse fields as biology, economics, and art.

A common problem is computing the n^{th} Fibonacci number. Zero is the 0^{th} , 1 is the 1^{st} , 1 is also the 2^{nd} , 2 is the 3^{rd} , 3 is the 4^{th} , 5 is the 5^{th} , etc.

A recursive Python function to compute the n^{th} Fibonacci number follows easily from the definition of the Fibonacci sequence:

```
def fibonacci(n: int) -> int:
    """ Returns the nth Fibonacci number. """
    if n <= 0:
        return 0
    elif n == 1:
        return 1
    else:
        return fibonacci(n - 2) + fibonacci(n - 1)
```

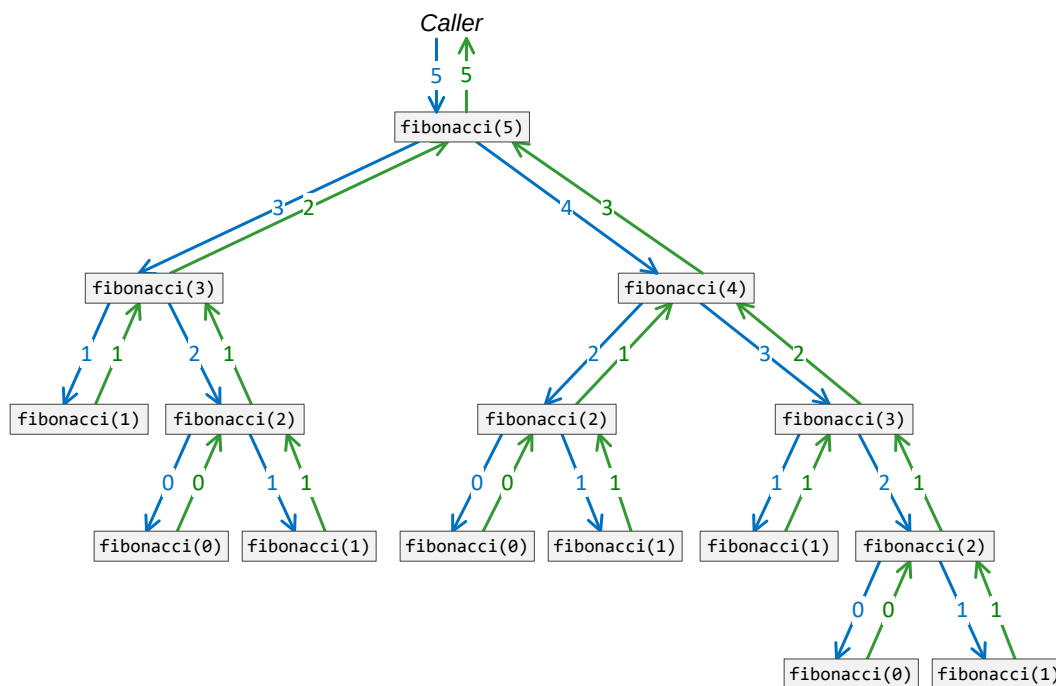
Figure 8.3 illustrates the recursive computation of `fibonacci(5)` using this `fibonacci` function. Note that the call `fibonacci(5)` invokes the `fibonacci` function a total of 15 times. While this factorial function computes the correct result, this tendency to call itself so many times makes it impractical for larger values. Section 15.8 introduces an algorithm design technique that greatly improves the performance of this recursive function.

8.4 Making Functions Reusable

In a function definition, we can package functionality that can be used in many different places within a program. We have yet to see how we can easily reuse our function definitions in other *programs*. For example, our `is_prime` function in Listing 7.16 (`primefunc.py`) works well within Listing 7.16 (`primefunc.py`), and we could put it to good use in other programs that need to test primality (encryption software, for example, makes heavy use of prime numbers). We could use the copy-and-paste feature of our favorite text editor to copy the `is_prime` function definition from Listing 7.16 (`primefunc.py`) into the new encryption program we are developing. It is possible to reuse a function in this way only if the function definition does not use any programmer-defined global variables nor any other programmer-defined functions. If a function does use any of these programmer-defined external entities, we must include these dependencies as well in the new code for the function to be viable. Said another way, the code in the function definition ideally will use only local variables and parameters. Such a function truly is independent and easily transportable to other programs.

The notion of copying source code from one program to another is not ideal, however. It is too easy for the copy to be incomplete or otherwise incorrect. Furthermore, such code duplication is wasteful. If 100 programs on a particular system all need to use the `is_prime` function, under this scheme they must all include the `is_prime` code. This redundancy wastes space. Finally, in perhaps the most compelling demonstration of the weakness of this copy-and-paste approach, what if someone discovers a bug in the `is_prime` function that all 100 programs are built around? When the error is discovered and fixed in one program, the other 99 programs will still contain the bug. Programmers must update the code in the other 99 programs, and it may be difficult to determine which files need to be fixed. The problem becomes much

Figure 8.3 The recursive computation of `fibonacci(5)`. Each rectangle represents an invocation of the `fibonacci` function. The call at the top of the diagram represents the initial call of `fibonacci(5)`. An arrow pointing down indicates the argument being passed into an invocation of `fibonacci`, and an arrow pointing up represents the value returned by that invocation. An invocation of `fibonacci` with no arrow pointing down away from the invocation represents a base case; observe that any invocation receiving a 0 or 1 is a base case. We see that the recursive process for `fibonacci(5)` invokes the function a total of 15 times.



worse if the code has been released to the general public. It may be impossible to track down and correct all the copies of the faulty function. The situation would be the same if a correct `is_prime` function were updated to be made more efficient. The problem is this: all the programs using `is_prime` define their *own* `is_prime` function; while the function definitions are meant to be identical, there is no mechanism tying all these common definitions together. We really would like to reuse the function as is without copying it.

Fortunately, Python makes it easy for developers to package their functions into *modules*. Programmers can build modules independently from the programs that use them. Software engineers did exactly that when developing Python's standard modules. They did so without the foreknowledge of exactly how we would use the functions they provide.

A Python source file constitutes a module. Consider the module Listing 8.6 (`primecode.py`).

Listing 8.6: `primecode.py`

```
""" Contains the definition of the is_prime function """
from math import sqrt

def is_prime(n: int) -> bool:
    """
    Returns True if nonnegative integer n is prime; otherwise,
    returns false
    """
    if n == 2:
        return True      # 2 is the only even prime number
    if n % 2 == 0 or n < 2:
        return False     # Exclude any other even number
    trial_factor = 3
    root = round(sqrt(n))

    while trial_factor <= root:
        if n % trial_factor == 0:    # Is trial factor a factor?
            return False           # Yes, return right away
        trial_factor += 2           # Consider next potential factor

    return True                  # Tried them all, must be prime
```

Other Python programs can use the code within the Listing 8.6 (`primecode.py`) file. In the simplest case, this module (file) appears in the same directory (folder) as the calling code file that uses it. Listing 8.7 (`usingprimecode.py`) contains a sample program that uses our packaged `is_prime` function.

Listing 8.7: `usingprimecode.py`

```
from primecode import is_prime

num = int(input('Enter an integer: '))
if is_prime(num):
    print(num, 'is prime')
else:
    print(num, 'is NOT prime')
```

The statement

```
from primecode import is_prime
```

directs the interpreter to import the `is_prime` function from the file `primecode.py`, which is the `primecode` module. A program that imports `is_prime` this way can use the name without any additional qualification.

Listing 8.8 (usingprimecode2.py) illustrates the alternative way of importing code from a module.

Listing 8.8: usingprimecode2.py

```
import primecode

num = int(input('Enter an integer: '))
if primecode.is_prime(num):
    print(num, 'is prime')
else:
    print(num, 'is NOT prime')
```

The statement

```
import primecode
```

directs the interpreter to import the entire module. Any access to names defined in the module require the module name prefix. In this example, the call to the `is_prime` function requires the module qualification: `primecode.is_prime`.

If we want our Listing 8.6 (`primecode.py`) module to be more widely available, we can place the file in a special Python library folder. This makes it available to all users on the system.

Observe the docstring at the top of the Listing 8.6 (`primecode.py`) module. This provides external documentation that serves as an overview to the functions the module provides. As we saw in Section 7.3, we can use the `help` function to reveal the information developers have provided in their docstrings. The following interactive sequence demonstrates how the `help` function accesses information in the `primecode` module's docstring:

```
>>> import primecode
>>> help(primecode)
Help on module primecode:

NAME
    primecode - Contains the definition of the is_prime function

FUNCTIONS
    is_prime(n: int) -> bool
        Returns True if nonnegative integer n is prime; otherwise,
        returns false

    sqrt(x, /)
        Return the square root of x.

FILE
    c:\code\primecode.py
```

Notice that our `primecode` module provides access to the `math` module's `sqrt` function as well as our `is_prime` function. This is because the `primecode` module directly imports `math.sqrt`.

8.5 Executable Modules

The Listing 8.6 (primecode.py) module contains the function `is_prime` that determines if an integer is prime or composite. Consider the following interactive sequence:

```
>>> import primecode
>>> dir(primecode)
['__builtins__', '__cached__', '__doc__', '__file__', '__loader__',
 '__name__', '__package__', '__spec__', 'is_prime', 'sqrt']
```

We see that the `primecode` module contains an element named `__name__`. We can inspect it:

```
>>> primecode.__name__
'primecode'
```

This is not surprising. The name of a module is a string representing its name! Listing 8.9 (primecode2.py) adds a statement at the end of Listing 8.6 (primecode.py) that reveals the module's name.

Listing 8.9: primecode2.py

```
""" Contains the definition of the is_prime function """
from math import sqrt

def is_prime(n: int) -> bool:
    """
    Returns True if nonnegative integer n is prime; otherwise,
    returns false
    """
    if n == 2:
        return True    # 2 is the only even prime number
    if n % 2 == 0 or n < 2:
        return False   # Exclude any other even number
    trial_factor = 3
    root = round(sqrt(n))

    while trial_factor <= root:
        if n % trial_factor == 0:    # Is trial factor a factor?
            return False            # Yes, return right away
            trial_factor += 2        # Consider next potential factor

    return True                    # Tried them all, must be prime

print(__name__)
```

The `print` statement is not part of the definition of the `is_prime` function. Consider interacting with this new module:

```
>>> import primecode2
primecode2
```

The mere action of importing the module executes the `print` statement to display the module's `__name__` string.

Listing 8.10 (useprimecode2.py) is a Python program that imports the `primecode2` module.

Listing 8.10: useprimecode2.py

```
import primecode2
```

Running Listing 8.10 (useprimecode2.py) provides an experience like that in the Python interpreter:

```
primecode2
```

If we attempt to execute the `primecode2` module as a program rather than importing it as a module, the result is different. Its `print` statement displays

```
__main__
```

This demonstrates that the `__name__` of a module depends on the context of its use:

- The `__name__` string for an imported module is the name of the module.
- The `__name__` string for a module being executed as a Python program is `'__main__'`.

This means we can write code within a module that can check the value of the module's `__name__` variable and optionally execute code depending on how the module is being used. It is common to provide some test code for a module within the module itself. Listing 8.11 (primecode3.py) checks the module's `__name__` string to optionally execute some simple test code.

Listing 8.11: primecode3.py

```
""" Contains the definition of the is_prime function """
from math import sqrt

def is_prime(n: int) -> bool:
    """
    Returns True if nonnegative integer n is prime; otherwise,
    returns false
    """
    if n == 2:
        return True      # 2 is the only even prime number
    if n % 2 == 0 or n < 2:
        return False     # Exclude any other even number
    trial_factor = 3
    root = round(sqrt(n))

    while trial_factor <= root:
        if n % trial_factor == 0:      # Is trial factor a factor?
            return False              # Yes, return right away
        trial_factor += 2              # Consider next potential factor

    return True                      # Tried them all, must be prime

if __name__ == '__main__':
    # Test the integers from 0 to 20 for primeness
    for val in range(21):
        print(f'{val:2}: {is_prime(val)}')
```

If we run Listing 8.11 (primecode3.py) as a Python program, it prints

```
0: False
1: False
2: True
3: True
4: False
5: True
6: False
7: True
8: False
9: False
10: False
11: True
12: False
13: True
14: False
15: False
16: False
17: True
18: False
19: True
20: False
```

If we import `primecode3` as a module, it prints nothing:

```
>>> import primecode3
>>>
```

Checking the module's `__name__` string gives us the ability to combine function definitions and function tests or other demonstration code in the same Python source file.

8.6 Functions as Data

In Python, a function is special kind of object, just as integers, floating-point numbers, and strings are objects. Consider the following sequence in the interactive shell:

```
>>> type(2)
<class 'int'>
>>> type('Rick')
<class 'str'>
>>> from math import sqrt
>>> type(sqrt)
<class 'builtin_function_or_method'>
```

The `sqrt` function has the Python type `builtin_function_or_method`. The word *class* indicates that `builtin_function_or_method` is an object type, just as `int` and `str` are object types. As such, we can treat a function as if it were a data value; for example, we can assign a variable to a function, as shown here:

```
from math import sqrt
```

```
x = sqrt          # Assign x to sqrt function object
print(x(16))      # Prints 4.0
```

We also can pass a function as a parameter to another function. Listing 8.12 (arithmeticeval.py) includes a function that accepts a function as a parameter.

Listing 8.12: arithmeticeval.py

```
from typing import Callable

def add(x: float, y: float) -> float:
    """ Adds the parameters x and y and returns the result """
    return x + y

def multiply(x: float, y: float) -> float:
    """ Multiplies the parameters x and y and returns the result """
    return x * y

def evaluate(f: Callable[[float, float], float], x: float, y: float) -> float:
    """ Calls the function f with parameters x and y: f(x, y) """
    return f(x, y)

def main() -> None:
    """ Tests the add, multiply, and evaluate functions """
    print(add(2, 3))
    print(multiply(2, 3))
    print(evaluate(add, 2, 3))
    print(evaluate(multiply, 2, 3))

if __name__ == '__main__':
    main()
```

Listing 8.12 (arithmeticeval.py) prints

```
5
6
5
6
```

The first parameter of `evaluate`, named `f`, is a function. The following type hint:

`Callable[[float, float], float]`

specifies that the function the caller passes should be one that accepts two floating-point parameters (the types in the inner square brackets) and return a floating-point result. Examining the code in the body of `evaluate`, we can see that indeed `f` must be a function that can be called with two arguments. We also see that `evaluate` calls `f` passing it the second and third parameters it receives from its caller. The expression

`evaluate(add, 2, 3)`

passes the `add` function and the literal values 2 and 3 to `evaluate`. The `evaluate` function then invokes the `add` function with arguments 2 and 3.

Table 8.1 provides a more comprehensive list of Python type hints.

We can simplify the name of a complex type hint using a technique known as *type aliasing*. Type aliasing creates new type hint name for another type hint. Listing 8.13 (arithmeticaltypealias.py) introduces the name `ArithmeticFunction` for the more verbose callable type hint.

Table 8.1 Type hints for various Python types. Type hints that begin with a capital letter require import from the `typing` module.

Type Name	Python Type	Type Hint
Integer	<code>int</code>	<code>int</code>
Floating-point number	<code>float</code>	<code>float</code>
String	<code>str</code>	<code>str</code>
Boolean	<code>bool</code>	<code>bool</code>
Any type	—	<code>Any</code>
Integer, string tuple	—	<code>tuple[int, str]</code>
Float, integer tuple	—	<code>tuple[float, int]</code>
Float or nothing	—	<code>float None</code>
Function that accepts an <code>int</code> parameter and returns an <code>int</code>	—	<code>Callable[[int], int]</code>
Function that accepts two double parameters and returns a <code>bool</code>	—	<code>Callable[[double, double], bool]</code>
Function that accepts an <code>int</code> parameter and returns a <code>str</code>	—	<code>Callable[[int], str]</code>
Function that accepts an <code>int</code> parameter and returns an <code>int</code> or nothing	—	<code>Callable[[int], int None]</code>
Function that accepts an <code>int</code> or <code>str</code> parameter and returns a <code>str</code>	—	<code>Callable[[int str], str]</code>
Function that accepts any parameters and returns an <code>int</code>	—	<code>Callable[..., str]</code>
Function that accepts any parameters and returns any type	—	<code>Callable[..., Any]</code>

Listing 8.13: arithevaltypealias.py

```

from typing import Callable

ArithmeticFunction = Callable[[float, float], float]

def add(x: float, y: float) -> float:
    """ Adds the parameters x and y and returns the result """
    return x + y

def multiply(x: float, y: float) -> float:
    """ Multiplies the parameters x and y and returns the result """
    return x * y

def evaluate(f: ArithmeticFunction, x: float, y: float) -> float:
    """ Calls the function f with parameters x and y: f(x, y) """
    return f(x, y)

def main() -> None:
    """ Tests the add, multiply, and evaluate functions """
    print(add(2, 3))
    print(multiply(2, 3))
    print(evaluate(add, 2, 3))
    print(evaluate(multiply, 2, 3))

if __name__ == '__main__':
    main()

```

Consider a function that is supposed to perform a computation and return a floating-point value:

```

def compute(x: float) -> float:
    """ Compute the answer. """
    # Details omitted . . .

```

Under some circumstances the function may not have enough information to compute and return a valid result. In such a case, the function should return `None`. The following definition represents the programmer's intent:

```

def compute(x: float) -> float | None:
    """ Compute the answer if possible. """
    # Details omitted . . .

```

In this case, callers must check that the return value is not `None` before attempting to use the result.

The ability to pass a function object to another function is a powerful concept. It provides one mechanism that enables programmers to develop code that interoperates with a larger software framework. Python's Turtle graphics environment is one such framework. Our Turtle graphics examples to this point have involved drawing pictures in the graphics window with no provision for user interaction. With our knowledge of functions as parameters we now have the tools to write graphical programs that respond to mouse clicks and key presses.

The `turtle` module contains the function `onscreenclick` that accepts a function as a parameter. It expects the function the caller passes to it to accept two numeric parameters. Listing 8.14 (`trackmouse.py`) is a very simple program that shows how `onscreenclick` works.

Listing 8.14: trackmouse.py

```
import turtle

def report_position(x: float, y: float) -> None:
    """ Simply prints the values of x and y. """
    print(f'x = {x}, y = {y}')

# Establish a function the framework should call when
# the user clicks the mouse
turtle.onscreenclick(report_position)

# Start the graphics loop that listens for user input
turtle.mainloop()
```

Listing 8.14 (trackmouse.py) presents an empty, white graphical window to the user. Move the mouse within the graphical window and press the mouse button at different locations. You will see no feedback in the graphical window, but the `print` statement will print the mouse's location within the non-graphical console.

As a more interesting, yet still relatively simple example, Listing 8.15 (placesquares.py) draws a square centered at each location the user clicks the mouse.

Listing 8.15: placesquares.py

```
import turtle

def draw_square(x: float, y: float) -> None:
    """ Draws a 10 x 10 square centered at the point (x, y). """
    turtle.penup()
    turtle.setheading(0) # Ensure the pen is pointed the proper direction
    turtle.setposition(x - 5, y - 5) # Center square
    turtle.pendown()
    for i in range(4):
        turtle.forward(10)
        turtle.left(90)
    turtle.update()

# Turn off animation
turtle.tracer(0)
turtle.hideturtle()

# Allow user to place 10 x 10 squares using the mouse
turtle.onscreenclick(draw_square)

# Start the graphics loop that listens for user input
turtle.mainloop()
```

The `turtle.onscreenclick` function allows a programmer to register a function with the graphical framework. The registered function (for example, `report_position` in Listing 8.14 (`trackmouse.py`) or `draw_square` in Listing 8.15 (`placesquares.py`)) is known as a *callback function* because the graphical framework will *call back* the function when a mouse click event occurs.

It is important to note that no code within Listing 8.14 (`trackmouse.py`) or Listing 8.15 (`placesquares.py`) actually calls the callback function. Neither does the `turtle.onscreenclick` function call the callback function; the call to `onscreenclick` merely registers the callback function with the graphical framework. It is the responsibility of the graphical framework to call the callback function. The graphical framework tracks mouse events, and so it is the framework that must call the callback function with the appropriate (x,y) location.

We will see in Section ?? that the ability to pass function objects around enables us to develop flexible algorithms that we can adapt at run time.

8.7 Separating Concerns with Pluggable Modules

Consider a very simple connect-the-dots game. In such a game two players compete on a two-dimensional rectangular grid of dots, as shown in Figure 8.4. Typically played with pen and paper, the number of rows and columns can vary, sometimes limited only by the size of the paper. The rules of the game are simple. Two players take turns drawing a line between adjacent dots. A player can draw a horizontal or vertical connecting line; diagonal lines are forbidden. Turns alternate until a player's line completes a square. At that point the player that drew the line becomes the square's owner and marks the square with an initial. As a bonus for winning the square, the square owner can immediately take another turn. Figure 8.5 shows a game in progress. The player owning the most squares at the end of the game is the winner.

Figure 8.4 The initial playing field for a connect-the-dots game.

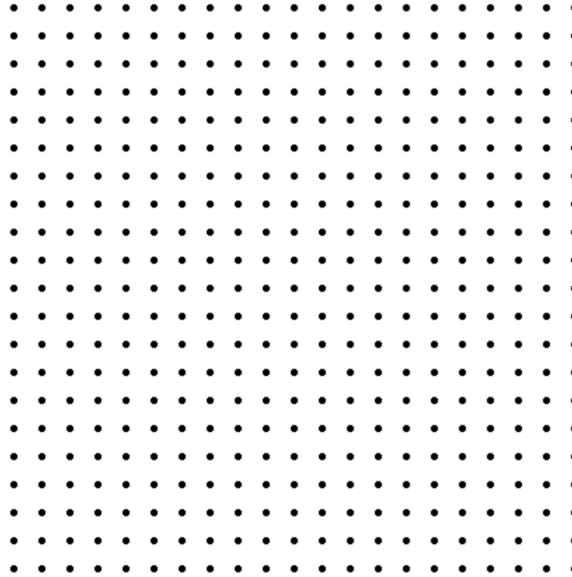
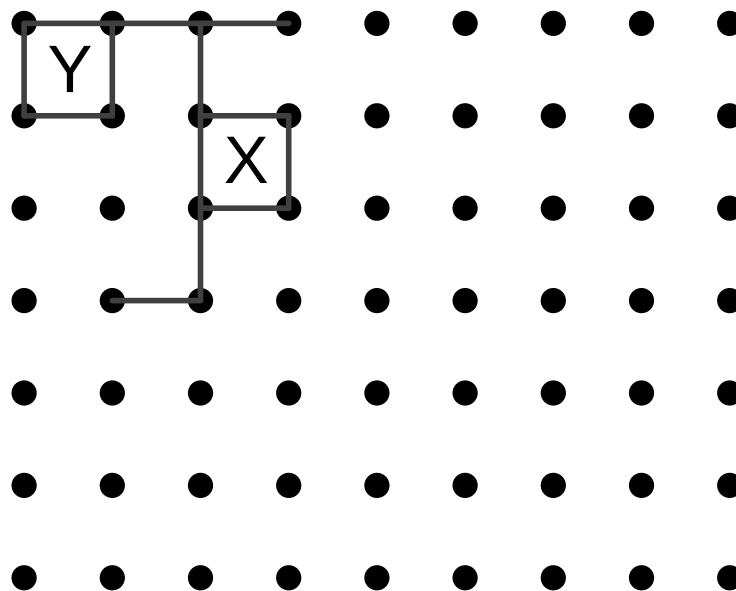


Figure 8.5 A connect-the-dots game in progress.



We will implement a simplified version of the connect-the-dots game so that we can exercise some of the concepts introduced in this chapter. We will limit our game to a 3×3 grid because we have yet to cover the concepts required to maintain a large amount of data at one time. (Lists in Chapter 10 provide a way to build connect-the-dot grids of arbitrary size.) Even in its simplified form, our program will be the most complex we have considered so far. This is because instead of writing the application in one large source file, we intentionally will divide our implementation into two parts:

- **Game engine.** The game engine controls the logic of the game. It enforces the game rules, dictates whose turn it is, determines who wins a square, and declares a winner or draw when no more moves are possible.
- **Presentation.** The presentation communicates the status of the game to the players and allows players to make moves. The presentation could provide a graphical representation and respond to mouse clicks, or it could print to a text-based console and accept typed information from the keyboard.

This kind of design depends on several important points:

- Developers must precisely specify the interface that connects both parts of the system. The interface specifies how the two parts communicate with each other. The interface includes function names, expected function parameters, and purpose of the function.
- Developers working on each part must conscientiously follow the requirements of the interface. Failure to comply with the agreed upon interface diminishes the ability of the two parts to interoperate.
- The presentation must be aware of the services the game engine provides, but the game engine will have no detailed knowledge of the presentation component.

If implemented properly, software engineers may develop both parts of the system independently of each other.

Listing 8.16 (`dot3x3logic.py`) provides the game engine for our 3×3 connect-the-dots game.

Listing 8.16: `dot3x3logic.py`

```
"""
dot3x3_logic.py

Provides the logic for a simplified connect the dots game.
The code in this module determines and enforces the rules of the
game. Callers are responsible for the game's presentation component,
that is, the user experience.

The playing surface consists of a 3x3 grid of dots:

    @   @   @

    @   @   @

    @   @   @

The game allows two players, X and Y, to alternately add horizontal and
vertical lines to connect the dots. The following shows a game in
progress:
```

```

a---a    a
  |      |
a    a    a
  |      |
a    a---a

```

When a player completes a square, that player wins the square and retains control of the next turn. The following shows a game in which player y has completed a square:

```

@---@      @
  |        |
@      @---@
  |      Y  |
@      @---@

```

If a player connects two dots and does not complete a square, control passes to the other player. A player must add a line during his/her turn. The game ends when all dots have been connected. The player with more squares wins the game, and the game is a draw if both players have two squares each.

This game engine manages 12 lines. Each line is distinguished by its name (a string) as shown here:

```

graph TD
    subgraph Row1
        direction LR
        A["@---'Northwest_North'---@"]
        B["@---'North_Northeast'---@"]
        C["@"]
    end
    subgraph Row2
        direction LR
        D["'Northwest_West'"]
        E["'North_Center'"]
        F["'Northeast_East'"]
    end
    subgraph Row3
        direction LR
        G["@---'West_Center'-----@"]
        H["@-----'Center_East'-----@"]
        I["@"]
    end
    subgraph Row4
        direction LR
        J["'West_Southwest'"]
        K["'Center_South'"]
        L["'East_Southeast'"]
    end
    subgraph Row5
        direction LR
        M["@---'Southwest_South'---@"]
        N["@---'South_Southeast'---@"]
        O["@"]
    end
    A --- D
    B --- E
    C --- F
    D --- G
    E --- H
    F --- I
    G --- J
    H --- K
    I --- L
    J --- M
    K --- N
    L --- O

```

The game engine manages four squares. Each square is distinguished by its name (a string) as shown here:

Diagram illustrating a 2x2 grid of rectangles. The top-left rectangle is labeled 'LeftTop', the top-right is 'RightTop', the bottom-left is 'LeftBottom', and the bottom-right is 'RightBottom'. Dashed lines connect the corners of the rectangles, forming a larger rectangle. The corners are labeled 'a' at the top-left, 'b' at the top-right, 'c' at the bottom-left, and 'd' at the bottom-right.

```

| 'LeftBottom' | 'RightBottom' |
|              |              |
|              |              |
@-----@-----@

```

The string 'X' represents player X, and the string 'Y' represents player Y.

```

"""

#-----
#  Global variables that maintain the state of the current game.
#  These variables are meant to be used only by code within this file;
#  the underscore prefix discourages their access outside of this
#  module.
#-----

#  Boolean variables that keep track of whether or not a line
#  exists between two dots; for example, if
#  _nw_n is true, this means the line identified as
#  'Northwest_North' exists.

#
#      @---_nw_n---@---_n_ne---@
#      |           |           |
#      _nw_w       _n_c       _ne_e
#      |           |           |
#      @---_w_c---@---_c_e---@
#      |           |           |
#      _w_sw       _c_s       _e_se
#      |           |           |
#      @---_sw_s---@---_se_e---@
#
#  Initially, no lines exist anywhere
_nw_n, _n_ne = False, False
_nw_w, _n_c, _ne_e = False, False, False
_w_c, _c_e = False, False
_w_sw, _c_s, _e_se = False, False, False
_sw_s, _s_se = False, False

#  The player whose turn it is currently (player X or player Y)
#  The value of this variable should be either the string 'X' or the
#  string 'Y'
_current_player = 'X'

#  Stores the owner of the given squares.
#  If _leftbottom_owner equals the string 'X', that means
#  player X owns the left-bottom square. Initially none of the
#  squares have an owner, hence their owners are each None.
_lefttop_owner, _righttop_owner = None, None
_leftbottom_owner, _rightbottom_owner = None, None

#-----
#  Functions used only by functions within this file.

```

```

# Note the names begin with an underscore to discourage access
# outside of this module.
#-----

def _update_square(sq: str) -> bool:
    """ Updates the owner of square sq, if possible.
        sq must be one of the strings 'LeftTop', 'RightTop',
        'LeftBottom', or 'RightBottom'.
        The function checks to see
            1) if the square currently is not owned by a player and
            2) if all the lines are in place to complete the square.
        If both conditions are met, it marks the square with the
        current player and returns True. If one of the players
        already owns the square, or if not all four lines exist to
        complete the square, the function simply returns False. """

    # The function may affect one of the following global variables:
    global _lefttop_owner, _righttop_owner, _leftbottom_owner, \
           _rightbottom_owner

    if sq == 'LeftTop' and _lefttop_owner == None \
        and _nw_n and _nw_w and _n_c and _w_c:
        _lefttop_owner = _current_player
        return True
    elif sq == 'RightTop' and _righttop_owner == None \
        and _n_ne and _ne_e and _c_e and _n_c:
        _righttop_owner = _current_player
        return True
    elif sq == 'LeftBottom' and _leftbottom_owner == None \
        and _w_c and _c_s and _sw_s and _w_sw:
        _leftbottom_owner = _current_player
        return True
    elif sq == 'RightBottom' and _rightbottom_owner == None \
        and _c_e and _e_se and _s_se and _c_s:
        _rightbottom_owner = _current_player
        return True
    else:
        return False # Owners unchanged

def _update_squares() -> bool:
    """ Attempts to update the owners of all squares that a new line
        might affect. Returns True if one or more squares receives a
        new owner; otherwise, the function returns False. """
    lt = _update_square('LeftTop')
    rt = _update_square('RightTop')
    lb = _update_square('LeftBottom')
    rb = _update_square('RightBottom')
    return lt or rt or lb or rb

#-----
# Functions that reveal or control the state of the current game.
# These functions are meant to be used outside of this
# module.
#-----

```



```

def add_line(line: str) -> bool:
    """ Attempts to add a line between the two dots.
        The parameter line must be one of 'Northwest_North',
        'North_Northeast', etc.; that is, a string representing
        a line on the game board.
        If the line is not present, the function adds the line
        and returns True.
        If the line already is present, the function
        does not change the state of the board and returns false. """

    # The function can change one of the following global variables
    # maintaining the state of the game.
    global _nw_n, _n_ne, _nw_w, _n_c, _ne_e, _w_c, _c_e, \
           _w_sw, _c_s, _e_se, _sw_s, _s_se, _current_player

    line_added = False # Unsuccessful by default
    if line == 'Northwest_North' and not _nw_n:
        _nw_n = True
        line_added = True
    elif line == 'North_Northeast' and not _n_ne:
        _n_ne = True
        line_added = True
    elif line == 'Northwest_West' and not _nw_w:
        _nw_w = True
        line_added = True
    elif line == 'North_Center' and not _n_c:
        _n_c = True
        line_added = True
    elif line == 'Northeast_East' and not _ne_e:
        _ne_e = True
        line_added = True
    elif line == 'West_Center' and not _w_c:
        _w_c = True
        line_added = True
    elif line == 'Center_East' and not _c_e:
        _c_e = True
        line_added = True
    elif line == 'West_Southwest' and not _w_sw:
        _w_sw = True
        line_added = True
    elif line == 'Center_South' and not _c_s:
        _c_s = True
        line_added = True
    elif line == 'East_Southeast' and not _e_se:
        _e_se = True
        line_added = True
    elif line == 'Southwest_South' and not _sw_s:
        _sw_s = True
        line_added = True
    elif line == 'South_Southeast' and not _s_se:
        _s_se = True
        line_added = True
    # If the line was added successfully,
    # check to see if it completes a square

```

```

    if line_added and not _update_squares():
        # Turn moves to other players upon a successful move
        if _current_player == 'X':
            _current_player = 'Y'
        else:
            _current_player = 'X'
    return line_added

def square_owner(sq: str) -> str | None:
    """ Returns the player who owns the given square.
        sq must be one of the strings 'LeftTop', 'RightTop',
        'LeftBottom', or 'RightBottom'.
        Returns None if the square has no owner. """
    if sq == 'LeftTop':
        return _lefttop_owner
    elif sq == 'RightTop':
        return _righttop_owner
    elif sq == 'LeftBottom':
        return _leftbottom_owner
    elif sq == 'RightBottom':
        return _rightbottom_owner
    else:
        return None

def check_line(line: str) -> bool:
    """ Returns True if the line exists on the game board.
        The parameter line must be one of 'Northwest_North',
        'North_Northeast', etc.; that is, a string representing
        a line on the game board.
        If the function returns False if the line does not yet
        exist. """
    if line == 'Northwest_North':
        return _nw_n
    elif line == 'North_Northeast':
        return _n_ne
    elif line == 'Northwest_West':
        return _nw_w
    elif line == 'North_Center':
        return _n_c
    elif line == 'Northeast_East':
        return _ne_e
    elif line == 'West_Center':
        return _w_c
    elif line == 'Center_East':
        return _c_e
    elif line == 'West_Southwest':
        return _w_sw
    elif line == 'Center_South':
        return _c_s
    elif line == 'East_Southeast':
        return _e_se
    elif line == 'Southwest_South':
        return _sw_s

```

```

elif line == 'South_Southeast':
    return _s_se
else:
    return False

def winner() -> str | None:
    """ Returns the winner, 'X' or 'Y', or 'Draw' if the game
        board is full and X and Y both own two squares. Returns
        None if open squares still exist, and so the game can
        continue. """
    # Count the player squares
    x_count, y_count = 0, 0
    if _lefttop_owner == 'X':
        x_count += 1
    elif _lefttop_owner == 'Y':
        y_count += 1
    if _righttop_owner == 'X':
        x_count += 1
    elif _righttop_owner == 'Y':
        y_count += 1
    if _rightbottom_owner == 'X':
        x_count += 1
    elif _rightbottom_owner == 'Y':
        y_count += 1
    if _leftbottom_owner == 'X':
        x_count += 1
    elif _leftbottom_owner == 'Y':
        y_count += 1
    if x_count + y_count == 4: # All squares filled
        if x_count > y_count:
            return 'X' # Player X won
        elif x_count < y_count:
            return 'Y' # Player Y won
        else:
            return 'Draw' # Tie game, no winner
    else:
        return None # No winner or draw; the game continues

def initialize_board() -> None:
    """ Makes the playing board ready for a new game:
        1) clears all the lines from the board,
        2) makes all the squares empty, and
        3) sets the current player to X
        This function does not return a value to the caller. """
    # All of the following global variables are affected:
    global _nw_n, _n_ne, _nw_w, _n_c, _ne_e, _w_c, _c_e, \
        _w_sw, _c_s, _e_se, _sw_s, _s_se, _current_player, \
        _lefttop_owner, _righttop_owner, \
        _leftbottom_owner, _rightbottom_owner
    _nw_n = _n_ne = _nw_w = _n_c = _ne_e = _w_c = _c_e = _w_sw \
        = _c_s = _e_se = _sw_s = _s_se = False

    # Clear all the squares (neither player owns any squares)

```

```

_leftttop_owner = _rightttop_owner = _leftbottom_owner \
    = _rightbottom_owner = None

# X always begins
_current_player = 'X'

def current_player() -> str:
    """ Returns the player whose turn it is to move """
    return _current_player

```

The docstring at the beginning of Listing 8.16 (`dot3x3logic.py`) explains overall design of the game engine. The game engine module consists of a collection of 17 global variables and eight functions:

- **Global variables:** The comments that follow the docstring reveal the purpose of the global variables that maintain the state of the game. The game engine's global variables fall into three categories:
 - The first set of global variables (`_nw_n`, `_n_ne`, etc.) store Boolean values that indicate if a line exists in a given location. If the global variable `_nw_n` is `False`, no line connects the top left and top center dots. If `_nw_n` is `True`, however, a line connects the top left and top center dots.
 - The global variable `player` keeps track of the player (indicated by the string `"X"` or string `"Y"`) whose turn it is.
 - The final set of global variables (`_leftttop_owner`, `_rightttop_owner`, etc.) store the owners of the squares. They begin with the special value `None`, which means no one owns any squares at the start of a game. When a player completes a square, the game engine assigns `"X"` or `"Y"` to the appropriate variable.
- **Functions:** The presentation part of the game code is not meant to access directly the global variables of the game engine; instead, the presentation should influence the global variables only via six of the supplied functions. The functions in the `dot3x3logic` module fall into three categories:
 - The `_update_square` and `_update_squares` functions are meant to be internal to the `dot3x3logic` module, meaning that only the functions within this file should call these two functions. These two functions determine which, if any squares become complete by a player's move. The only caller of the `_update_square` function is `_update_squares`. The docstrings within these two functions provide more information about their purpose and use.
 - The `add_line` and `initialize_board` functions are available to the presentation and can modify the state of the game. The presentation calls `add_line` when a player attempts to draw a line. The `add_line` function calls the `_update_squares` function to check whether a new line completes a square. The `initialize_board` function resets all the global variables in preparation of a new game. The presentation calls the `add_line` function when a player provides input representing a move. The presentation calls the `initialize_board` function to begin a new game. The docstrings within these two functions provide more information about their purpose and use.
 - The `check_line`, `square_owner`, `winner`, and `current_player` functions cannot change the state of the game. They serve to report the state of the game to the presentation. The presentation can call these functions to determine how to change its appearance. The docstrings within these functions provide more information about their purpose and use.

Note the type hint for the return value for the functions `square_owner` and `winner`:

`str | None`

This notation indicates these functions can return a value of type `str` or `None`. Read the vertical bar as “or.” To satisfy a type checker like Pylance, a caller must explicitly check for the return type and handle each case appropriately.

Listing 8.17 (`dot3x3turtle.py`) provides the graphical presentation using Python’s `turtle` module.

Listing 8.17: `dot3x3turtle.py`

```
import turtle
from tkinter import messagebox

# Gain access to the game engine functions
from dot3x3logic import *

# Some global variables supporting the graphical interface

# The first dot selected by a player when creating a line
initial_dot = None

dot_radius = 10 # Can adjust for larger or smaller dots

def square_to_point(sq: str) -> tuple[float, float] | None:
    """ Compute the (x, y) coordinates of the center of a square.
        Used to properly place the square's owner when it becomes
        captured. """
    if sq == 'LeftTop':
        return (200, 400)
    elif sq == 'RightTop':
        return (400, 400)
    elif sq == 'LeftBottom':
        return (200, 200)
    elif sq == 'RightBottom':
        return (400, 200)
    else:
        return None

def hit(x: float, y: float) -> str | None:
    """ Returns the dot (if any) that the point (x, y) is within.
        Returns None if the point (x, y) does not overlap any dot.
        Used when a player clicks the mouse over the board to
        determine which dot (if any) was selected. """
    dot = None # Default result
    if 90 < x < 110 and 490 < y < 510:
        dot = 'Northwest'
    elif 290 < x < 310 and 490 < y < 510:
        dot = 'North'
    elif 490 < x < 510 and 490 < y < 510:
        dot = 'Northeast'
    elif 90 < x < 110 and 290 < y < 310:
        dot = 'West'
```

```

elif 290 < x < 310 and 290 < y < 310:
    dot = 'Center'
elif 490 < x < 510 and 290 < y < 310:
    dot = 'East'
elif 90 < x < 110 and 90 < y < 110:
    dot = 'Southwest'
elif 290 < x < 310 and 90 < y < 110:
    dot = 'South'
elif 490 < x < 510 and 90 < y < 110:
    dot = 'Southeast'
return dot

def dot_to_point(dot: str) -> tuple[float, float] | None:
    """ Maps a dot to its location on the board. Used to
        render a dot in its proper place. """
    if dot == 'Northwest':
        return 100, 500
    elif dot == 'North':
        return 300, 500
    elif dot == 'Northeast':
        return 500, 500
    elif dot == 'West':
        return 100, 300
    elif dot == 'Center':
        return 300, 300
    elif dot == 'East':
        return 500, 300
    elif dot == 'Southwest':
        return 100, 100
    elif dot == 'South':
        return 300, 100
    elif dot == 'Southeast':
        return 500, 100
    else:
        return None

def draw_dot(name: str, col: str='black') -> None:
    """ Draws a graphical dot within the graphical window.
        col specifies the dot's color (black by default). """
    global dot_radius

    turtle.pensize(1)
    turtle.penup()
    position = dot_to_point(name)
    if position:
        x, y = position
        turtle.setposition(x, y - dot_radius)
        turtle.pendown()
        turtle.color(col)
        turtle.begin_fill()
        turtle.circle(dot_radius)
        turtle.end_fill()
        turtle.update()

```

```
def draw_line(x1: float, y1: float, x2: float, y2: float) -> None:
    """ Draws a line segment with endpoints (x1, y1) and (x2, y2). """
    turtle.penup()
    turtle.setposition(x1, y1)
    turtle.pendown()
    turtle.setposition(x2, y2)
    turtle.update()

def draw_X(x: float, y: float) -> None:
    """ Draws player X's mark at position (x,y). """
    turtle.color('blue')
    turtle.pensize(10)
    draw_line(x - 40, y - 40, x + 40, y + 40)
    draw_line(x - 40, y + 40, x + 40, y - 40)

def draw_Y(x: float, y: float) -> None:
    """ Draws player Y's mark at position (x,y). """
    turtle.color('blue')
    turtle.pensize(10)
    draw_line(x - 40, y + 40, x, y)
    draw_line(x + 40, y + 40, x, y)
    draw_line(x, y, x, y - 40)

def draw_square(sq: str, owner: str | None) -> None:
    """ Draws the owner of a square in the square, if the square
        has an owner. """
    # Coordinates the square
    position = square_to_point(sq)
    if position:
        x, y = position
        if owner == 'X':
            draw_X(x, y)
        elif owner == 'Y':
            draw_Y(x, y)
    # Else do not draw anything--the square has no owner

def check_squares() -> None:
    """ Checks all squares to determine if any are complete.
        Draws the square owner's mark in a completed square.
        If all squares are owned, the function declares a winner
        or announces a draw. """
    # Check the ownership of each potential square
    draw_square('LeftTop', square_owner('LeftTop'))
    draw_square('RightTop', square_owner('RightTop'))
    draw_square('LeftBottom', square_owner('LeftBottom'))
    draw_square('RightBottom', square_owner('RightBottom'))
    # Determine
    win = winner()
    if win == 'X':
        messagebox.showinfo('Game Over', 'X wins')
```

```

elif win == 'Y':
    messagebox.showinfo('Game Over', 'Y wins')
elif win == 'Draw':
    messagebox.showinfo('Game Over', 'Tie Game')

def mouse_click(x: float, y: float) -> None:
    """ Responds to the user clicking the mouse when the cursor
        is over the window. Determines if the user clicked on
        a dot. Activates an initial dot if the initial dot is not
        already active. Establishes a line to a terminal dot if
        possible. If the move is illegal, the function produces a
        message box alerting the player. """
    global initial_dot
    print('initial_dot =', initial_dot)

    print('clicked at x =', x, ' y =', y)
    dot = hit(x, y) # Did the player click on a dot?
    if dot:
        print(dot)
        if not initial_dot:
            initial_dot = dot
            draw_dot(initial_dot, 'red')
        elif dot != initial_dot:
            ln = line_name(initial_dot, dot)
            if ln and add_line(ln):
                # Draw the added line
                turtle.color('black')
                turtle.pensize(5)
                dot1_position = dot_to_point(initial_dot)
                dot2_position = dot_to_point(dot)
                if dot1_position and dot2_position:
                    x1, y1 = dot1_position
                    x2, y2 = dot2_position
                    draw_line(x1, y1, x2, y2)
                    # Adjust title bar to current player
                    turtle.title('3x3 Connect the Dots   Current Player: '
                                + current_player())
                    # Check to see if the move captured a square
                    # and/or ended the game
                    check_squares()
                    # Clear the initial dot and redraw both connecting dots
                    draw_dot(initial_dot)
                    initial_dot = None # Initial dot no longer in play
                    draw_dot(dot)
            else:
                if initial_dot:
                    draw_dot(initial_dot)
                    initial_dot = None

def reset_game() -> None:
    """ Reinitialize the game's state for the start of a new game. """
    turtle.clearscreen()
    initialize()

```



```

def line_name(dot1: str, dot2: str) -> str | None:
    """ Returns the name of the line that connects the
        Dot objects dot1 and dot2. Derives the line's name
        from the names of the two dot objects. Displays
        an error message box if the two dots do not
        participate in a valid line. """
    if (dot1 == 'Northwest' and dot2 == 'North') or \
        (dot1 == 'North' and dot2 == 'Northwest'):
        return 'Northwest_North'
    elif (dot1 == 'North' and dot2 == 'Northeast') or \
        (dot1 == 'Northeast' and dot2 == 'North'):
        return 'North_Northeast'
    elif (dot1 == 'Northwest' and dot2 == 'West') or \
        (dot1 == 'West' and dot2 == 'Northwest'):
        return 'Northwest_West'
    elif (dot1 == 'North' and dot2 == 'Center') or \
        (dot1 == 'Center' and dot2 == 'North'):
        return 'North_Center'
    elif (dot1 == 'Northeast' and dot2 == 'East') or \
        (dot1 == 'East' and dot2 == 'Northeast'):
        return 'Northeast_East'
    elif (dot1 == 'West' and dot2 == 'Center') or \
        (dot1 == 'Center' and dot2 == 'West'):
        return 'West_Center'
    elif (dot1 == 'Center' and dot2 == 'East') or \
        (dot1 == 'East' and dot2 == 'Center'):
        return 'Center_East'
    elif (dot1 == 'West' and dot2 == 'Southwest') or \
        (dot1 == 'Southwest' and dot2 == 'West'):
        return 'West_Southwest'
    elif (dot1 == 'Center' and dot2 == 'South') or \
        (dot1 == 'South' and dot2 == 'Center'):
        return 'Center_South'
    elif (dot1 == 'East' and dot2 == 'Southeast') or \
        (dot1 == 'Southeast' and dot2 == 'East'):
        return 'East_Southeast'
    elif (dot1 == 'Southwest' and dot2 == 'South') or \
        (dot1 == 'South' and dot2 == 'Southwest'):
        return 'Southwest_South'
    elif (dot1 == 'South' and dot2 == 'Southeast') or \
        (dot1 == 'Southeast' and dot2 == 'South'):
        return 'South_Southeast'
    else:
        print('Not a valid dot link')
        messagebox.showerror('Invalid Link', 'You may not connect those two dots')
        return None

def initialize() -> None:
    """ Initializes the graphical presentation and game engine. """
    # Global dot objects

    # Initialize game engine

```

```

initialize_board()

# Specify the dimensions of the window
turtle.screensize(600, 600)
# Move origin (0, 0) to the left bottom of the window
turtle.setworldcoordinates(0, 0, 599, 599)

# Apply the usual tricks to speed up the image rendering
turtle.tracer(0)
turtle.hideturtle()

# Register callback functions with the Turtle graphics framework
turtle.onscreenclick(mouse_click) # What to do on a mouse click
turtle.onkeyrelease(reset_game, 'q') # What to do when user presses the Q key

# Permit window to receive keypress events
turtle.listen()

# Set initial widow title
turtle.title('3x3 Connect the Dots   Current Player: '
            + current_player())

# No initial dot has been selected
initial_dot = None

# Draw all the dots
for dot in ('Northwest', 'North', 'Northeast',
            'West', 'Center', 'East',
            'Southwest', 'South', 'Southeast'):
    draw_dot(dot)

turtle.update() # Force the window to show drawing

if __name__ == '__main__':
    initialize() # Set up the game
    turtle.mainloop() # Start game running

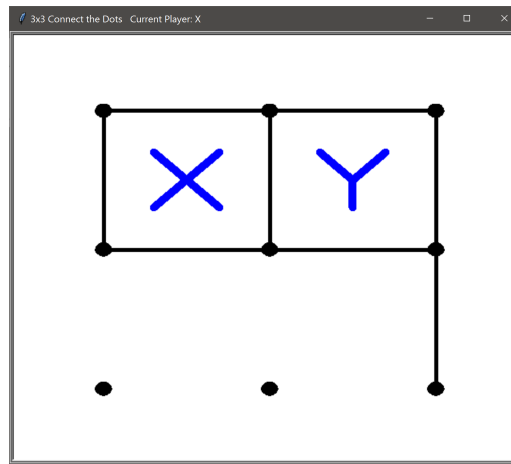
```

Listing 8.17 (dot3x3turtle.py) uses its own global variables to determine which dots the user selects when making a line. Note that the presentation registers the function `mouse_click` for the graphical framework to call when the user clicks the mouse button. The `mouse_click` function must determine if the click occurred over a graphical dot. It uses the `initial_dot` global to keep track if this is the initial endpoint of a line or the terminal endpoint of a line. The `mouse_click` function then calls the `line_name` function to translate the information about the two graphical dots into the proper line string to send to the game engine's `add_line` function. If the player attempts an illegal move, the presentation pops up a message box alerting the player.

This program registers the `reset_game` function via the Turtle library's `onkeyrelease` function. This allows a player to quit the current game and start a new one at any time by pressing the **Q** key. In addition to initializing the Turtle graphics environment, the `reset_game` calls the game engine's `initialize_board` function to ensure the game engine removes any remnants of the abandoned game.

Figure 8.6 shows a graphical 3×3 connect-the-dots game in progress.

This separation of concerns, dividing the game control logic from the presentation, requires more effort

Figure 8.6 A graphical 3 × 3 connect-the-dots game in progress.

to implement properly than does putting all the code together in one module. The payoff of this extra work is increased flexibility. This design allows us to decouple the engine from the presentation. We can “unplug” this graphical user interface from the game engine and “plug in” a completely different presentation without touching the game engine code. Listing 8.18 (`dot3x3text.py`) uses the exact same game engine as Listing 8.17 (`dot3x3turtle.py`) but provides a text-output, keyboard-only-input interface to the players.

Listing 8.18: `dot3x3text.py`

```
from dot3x3logic import *

def show_player(p: str | None) -> str:
    ''' Determines the string to print in the context of a
        square's owner. p is a player. '''
    if p:
        return ' ' + p
    else:
        return ' '

def draw_game() -> None:
    ''' Draws the connect the dots game using text graphics '''
    # 0---1---2
    print(' 0', end='')
    if check_line('Northwest_North'):
        print('---', end='')
    else:
        print(' ', end='')
    print('1', end='')
    if check_line('North_Northeast'):
        print('---', end='')
    else:
        print(' ', end='')
    print('2')
```

```

# | | | | -----
if check_line('Northwest_West'):
    print(' |', end='')
else:
    print(' ', end='')
print(show_player(square_owner('LeftTop')), end='')
if check_line('North_Center'):
    print(' |', end='')
else:
    print(' ', end='')
print(show_player(square_owner('RightTop')), end='')
if check_line('Northeast_East'):
    print(' |')
else:
    print(' ')

# 3---4---5 -----
print(' 3', end='')
if check_line('West_Center'):
    print('---', end='')
else:
    print(' ', end='')
print('4', end='')
if check_line('Center_East'):
    print('---', end='')
else:
    print(' ', end='')
print('5')

# | | | | -----
if check_line('West_Southwest'):
    print(' |', end='')
else:
    print(' ', end='')
print(show_player(square_owner('LeftBottom')), end='')
if check_line('Center_South'):
    print(' |', end='')
else:
    print(' ', end='')
print(show_player(square_owner('RightBottom')), end='')
if check_line('East_Southeast'):
    print(' |')
else:
    print(' ')

# 6---7---8 -----
print(' 6', end='')
if check_line('Southwest_South'):
    print('---', end='')
else:
    print(' ', end='')
print('7', end='')
if check_line('South_Southeast'):
    print('---', end='')

```

```

else:
    print(' ', end='')
print('8')

def make_line(move: str | None) -> str:
    ''' Determines the line between two dots specified in the string
        move. The move parameter is one of '01', '12', '03', etc.
        representing a line connecting the dots in the pattern
            0---1---2
            |   |   |
            3---4---5
            |   |   |
            6---7---8
        The function returns the string that represents the
        line position between the dots, one of 'Northwest_North',
        'North_Center', etc. If the dot1, dot2 pair represents an
        invalid combination, for example, (0, 4), the function
        returns the string 'NoLine'. '''
    if move == '01' or move == '10':
        return 'Northwest_North'
    elif move == '12' or move == '21':
        return 'North_Northeast'
    elif move == '03' or move == '30':
        return 'Northwest_West'
    elif move == '14' or move == '41':
        return 'North_Center'
    elif move == '25' or move == '52':
        return 'Northeast_East'
    elif move == '34' or move == '43':
        return 'West_Center'
    elif move == '45' or move == '54':
        return 'Center_East'
    elif move == '36' or move == '63':
        return 'West_Southwest'
    elif move == '47' or move == '74':
        return 'Center_South'
    elif move == '58' or move == '85':
        return 'East_Southeast'
    elif move == '67' or move == '76':
        return 'Southwest_South'
    elif move == '78' or move == '87':
        return 'South_Southeast'
    else:
        return 'NoLine' # Default return value

# Start a new game
initialize_board()
draw_game()
winning_player = None # No winner at start
while not winning_player:
    prompt = 'Move for player ' + current_player() + ': '
    move = input(prompt) # Get the dots to connect
    if move == 'Q' or move == 'q': # 'Q' quits the program

```

```

        break
    print()
    # Add the line, if possible
    new_line = make_line(move)
    if new_line != 'NoLine' and add_line(new_line):
        draw_game()
    else:
        print('Connection not possible')
    winning_player = winner() # Check for win

# Determine winner, if either player X or player Y
if winning_player == 'X':
    print('X won')
elif winning_player == 'Y':
    print('Y won')
else:
    print('Draw')
print('\n***  G A M E    O V E R   ***')
```

Listing 8.17 (dot3x3turtle.py) had to maintain a few global variables to keep track of the user's graphical dot selections. Observe that Listing 8.18 (dot3x3text.py) needs no global variables of its own. This is because the user (players) simply type in the numbers of the dots to connect. The following shows an example of a complete game in one execution of Listing 8.18 (dot3x3text.py):

```

0   1   2
3   4   5
6   7   8
Move for player X: 12

0   1---2
3   4   5
6   7   8
Move for player Y: 52

0   1---2
    |
3   4   5
6   7   8
Move for player X: 47

0   1---2
    |
3   4   5
    |
6   7   8
Move for player Y: 34

0   1---2
    |
3---4   5
```

```

      |
  6   7   8
Move for player X: 45

  0   1---2
      |
  3---4---5
      |
  6   7   8
Move for player Y: 14

  0   1---2
      | Y |
  3---4---5
      |
  6   7   8
Move for player Y: 58

  0   1---2
      | Y |
  3---4---5
      | |
  6   7   8
Move for player X: 87

  0   1---2
      | Y |
  3---4---5
      | X |
  6   7---8
Move for player X: 01

  0---1---2
      | Y |
  3---4---5
      | X |
  6   7---8
Move for player Y: 76

  0---1---2
      | Y |
  3---4---5
      | X |
  6---7---8
Move for player X: 63

  0---1---2
      | Y |
  3---4---5
  | X | X |
  6---7---8
Move for player X: 03

  0---1---2
  | X | Y |

```

```

  3---4---5
  | X | X |
  6---7---8
X won

***  G A M E    O V E R  ***

```

The text-based version is not as natural for humans to play as the graphical version, but it is playable. As far as the game logic is concerned, it behaves identically to our graphical version. This is because they share the same game engine. One benefit of this modularity is that if we detect and correct a logic error in our game engine, the change will repair both the graphical- and text-based versions of the game.

8.8 Lambda Expressions

One of the primary benefits of functions is that we can write a function's code once and invoke it from many different places within the program (and even invoke it from other programs). Ordinarily, in order to call a function, we must know its name. Almost all the examples we have seen have invoked a function via its name. Listing 8.12 (`arithmeticeval.py`) in Section 8.6 provided examples of invoking functions without using their names directly. There we saw a function named `evaluate` that accepts a function as a parameter:

```

def evaluate(f: Callable[[float, float], float],
            x: float, y: float) -> float:
    return f(x, y)

```

The `evaluate` function calls `f`. The question is, what function does `evaluate` call? The name `f` refers to one of `evaluate`'s parameters; there is no separate function named `f` specified by `def` within Listing 8.12 (`arithmeticeval.py`). The answer, of course, is that `evaluate` invokes the function passed in from the caller. The function named `main` in Listing 8.12 (`arithmeticeval.py`) calls `evaluate` passing the `add` function on one occasion and the `multiply` function on another.

The code in the `evaluate` function demands that callers send a function as the first parameter. Does this mean we have to write a separate function using `def` in order to call `evaluate`? What if we want to ensure that our function will execute exactly one time and only when invoked by `evaluate`?

Python supports the definition of simple, anonymous functions via `lambda` expressions. The general form of a `lambda` expression is

lambda *parameterlist* : *expression*

where:

- `lambda` is a reserved word that introduces a `lambda` expression.
- *parameterlist* is a comma-separated list of parameters as you would find in the function definition (but notice the lack of parentheses).
- *expression* is a single Python expression. The *expression* cannot be a complete statement, nor can it be a block of statements.

The term *lambda* comes from *lambda calculus* (see http://en.wikipedia.org/wiki/Lambda_calculus), a function-based mathematical system developed by Alonzo Church in the 1930s. Concepts from lambda calculus led to the development of the modern computer.

We can use a `lambda` expression in a call to the `evaluate` function:

```
evaluate(lambda x, y: x * y, 2, 3)
```

The `lambda` keyword signifies the definition of an unnamed function; thus, the first argument being passed to `evaluate` is indeed a function. Notice that result of passing the `lambda` expression here is the same as passing the `multiply` function from Listing 8.12 (`arithmeticval.py`)—both compute the product of the two parameters.

The following interactive session shows some additional examples of `lambda` expressions:

```
>>> def evaluate(f, x, y):
...     return f(x, y)
...
>>> evaluate(lambda x, y: 3*x + y, 10, 2)
32
>>> evaluate(lambda x, y: print(x, y), 10, 2)
10 2
>>> evaluate(lambda x, y: 10 if x == y else 2, 5, 5)
10
>>> evaluate(lambda x, y: 10 if x == y else 2, 5, 3)
2
```

The expression following the colon (`:`) in a `lambda` expression cannot be a Python statement. The conditional expression, for example, is acceptable, but an `if/else` statement is illegal. The expression's value is what the anonymous function *returns*, but the keyword `return` itself may not appear within the expression. Assignments are not possible within `lambda` expressions, and loops are not allowed. Note that a `lambda` expression can involve one or more function *invocations*, so the `lambda` expression in the following statement is legal:

```
evaluate(lambda x, y: max(x, y) + x - sqrt(y), 2, 3)
```

if the `max` and `sqrt` functions are available to the program.

A `lambda` expression does not support type hints for its parameters.

One interesting aspect of `lambda` functions is that they can form what is known as a *closure*. A closure is a function that can capture the context of its surrounding environment. Listing 8.19 (`closurein.py`) demonstrates a simple closure.

Listing 8.19: `closurein.py`

```
from typing import Callable

def evaluate(f: Callable[[float], bool], num: int) -> bool:
    return f(num)

def main() -> None:
    a = int(input('Enter an integer:'))
    print(evaluate((lambda x: False if x == a else True), 2))

main()
```

Note that `main` creates a function (the `lambda` expression) that it passes to `evaluate`. It is important to note that the statement

```
print(evaluate(lambda x, y: False if x == a else True, 2, 3))
```

passes just three parameters to `evaluate`: a function definition and two integer values. The `main` function's local variable `a` is not passed as a parameter; instead, it is embedded within the `lambda` code of the first parameter. The variable `a` is encoded into the `lambda` expression. We say the function definition (`lambda` expression) *captures* the variable `a`. When `evaluate` invokes the function sent by the caller, `evaluate` has no access to a variable named `a`. The `a` involved in the conditional expression is captured from `main`. This is an example of a closure transporting a captured variable into a function call.

For an example of a closure transporting a captured local variable out of a function, consider Listing 8.20 (`makeadder.py`) which includes a function that returns a function (`lambda` expression) to its caller.

Listing 8.20: `makeadder.py`

```
from typing import Callable

def make_adder() -> Callable[[int], int]:
    loc_val = 2 # Local variable definition
    return lambda x: x + loc_val # Returns a function

def main():
    f = make_adder()
    print(f(10))
    print(f(2))

main()
```

Ordinarily when a function returns, all of its local variables disappear. This means that after the following statement in the `main` function executes:

```
f = make_adder()
```

`make_adder`'s `loc_val` local variable should no longer exist. The function that `make_adder` returns, however, uses `loc_val` in its computation. This means the function that `make_adder` returns forms a closure that captures `make_adder`'s local variable `loc_val`. In the output of Listing 8.20 (`makeadder.py`) we can see that function `f` still has knowledge of `loc_val`'s value:

```
12
4
```

We can assign a variable to a `lambda` expression:

```
>>> f = lambda x: 2*x
>>> f(10)
20
```

This gives the function a name, so the statement

```
f = lambda x: 2*x
```

is equivalent to the function definition

```
def f(x):
    return 2*x
```

The function definition with `def` is much more powerful, however, because it can contain any number of arbitrary Python statements.

While not a particularly useful application of `lambda`, to demonstrate the regularity of the Python language, we can define an anonymous function and invoke it immediately; for example, the statement

```
print((lambda x, y: x * y)(2, 3))
```

prints 6. In this case the statement `print(2*3)` or, even better, `print(6)` produces the same result much more simply.

Listing 8.21 (`plotter.py`) combines the concepts of lambda functions, functions as data (Section 8.6), and Turtle graphics (Section 6.9). It defines several functions:

- `initialize_plotter`—calls appropriate Turtle graphics functions to initial the window size and coordinate system based on the parameters passed by its caller. Draws an x - and y -axis to simulate a Cartesian coordinate plane.
- `plot`—draws the graph of a two-dimensional mathematical function on the Cartesian coordinate plane established by `initialize_plotter`. It accepts a mathematical function, expressed as a Python function, as one of its parameters. Its other parameter is the drawing color for the curve.
- `finish_plotting`—finishes rendering the Turtle graphics.
- `quad`—represents the mathematical quadratic function $f(x) = \frac{1}{2}x^2 + 3$. A caller can pass this to the `plot` function for rendering.

The comments within Listing 8.21 (`plotter.py`) provide additional details about the functions' operation.

Listing 8.21: `plotter.py`

```
""" Provides the plot function that draws the graph of a mathematical
    function on a Cartesian coordinate plane. """
from typing import Callable
import turtle

def initialize_plotter(width: int, height: int,
                      min_x: float, max_x:
                      float, min_y: float, max_y: float) -> None:
    """ Initializes the plotter with a window with dimensions
        width x height, the x-axis ranging from
        min_x to max_x, and the y-axis ranging from
        min_y to max_y. Establishes the global beginning and ending
        x values for the plot and the global x_increment value.
        Draws the x- and y-axes. """

    # Global variables that the plot function must access
    global x_begin, x_end, x_increment

    #turtle.tracer(False)          # Render immediately
```

```

turtle.delay(0)                # Speed up rendering

turtle.title('Curve plotter')

# Establish global x and y ranges
x_begin, x_end = min_x, max_x
# Set up window size, in pixels
turtle.setup(width=width, height=height)
# Set up screen size, in pixels
turtle.screensize(width, height)
turtle.setworldcoordinates(min_x, min_y, max_x, max_y)

# x-axis distance that corresponds to one pixel in window distance
x_increment = (max_x - min_x)/width

turtle.hideturtle()            # Make pen invisible
# Draw x axis
turtle.pencolor('black')
turtle.penup()
turtle.setposition(min_x, 0)    # Move the pen to the left, center
turtle.setheading(0)            # Aim the pen right
turtle.pendown()
turtle.forward(max_x - min_x)   # Draw a line left to right

# Draw y axis
turtle.penup()
turtle.setposition(0, min_y)    # Move the pen to the center, bottom
turtle.setheading(90)           # Aim the pen up
turtle.pendown()                # Draw line bottom to top
turtle.forward(max_y - min_y)

def plot(f: Callable[[float], float], color: str) -> None:
    """ Plots function f on the Cartesian coordinate plane
        established by initialize_plotter. Plots (x, f(x)),
        for all x in the range x_begin <= x < x_end.
        The color parameter dictates the curve's color. """

    # Move pen to starting position
    turtle.penup()
    turtle.setposition(x_begin, f(x_begin))
    turtle.pencolor(color)
    turtle.pendown()
    # Iterate over the range of x values for x_begin <= x < x_end
    x = x_begin
    while x < x_end:
        turtle.setposition(x, f(x))
        x += x_increment    # Next x

def finish_plotting() -> None:
    turtle.exitonclick()

def quad(x: float) -> float:    # Quadratic function (parabola)

```

```

    return 1/2 * x**2 + 3

def main() -> None:
    """ Provides a simple test of the plot function. """
    from math import sin
    initialize_plotter(600, 600, -10, 10, -10, 10)
    # Plot f(x) = 1/2*x + 3, for -10 <= x < 100
    plot(quad, 'red')
    # Plot f(x) = x, for -10 <= x < 100
    plot(lambda x: x, 'blue')
    # Plot f(x) = 3 sin x, for -10 <= x < 100
    plot(lambda x: 3*sin(x), 'green')
    finish_plotting()

if __name__ == '__main__':
    main()

```

Listing 8.21 (plotter.py) contains a test function, `main`, that plots the following mathematical functions:

$$\begin{aligned}
 f(x) &= \frac{1}{2}x^2 + 3 \\
 f(x) &= x \\
 f(x) &= 3\sin x
 \end{aligned}$$

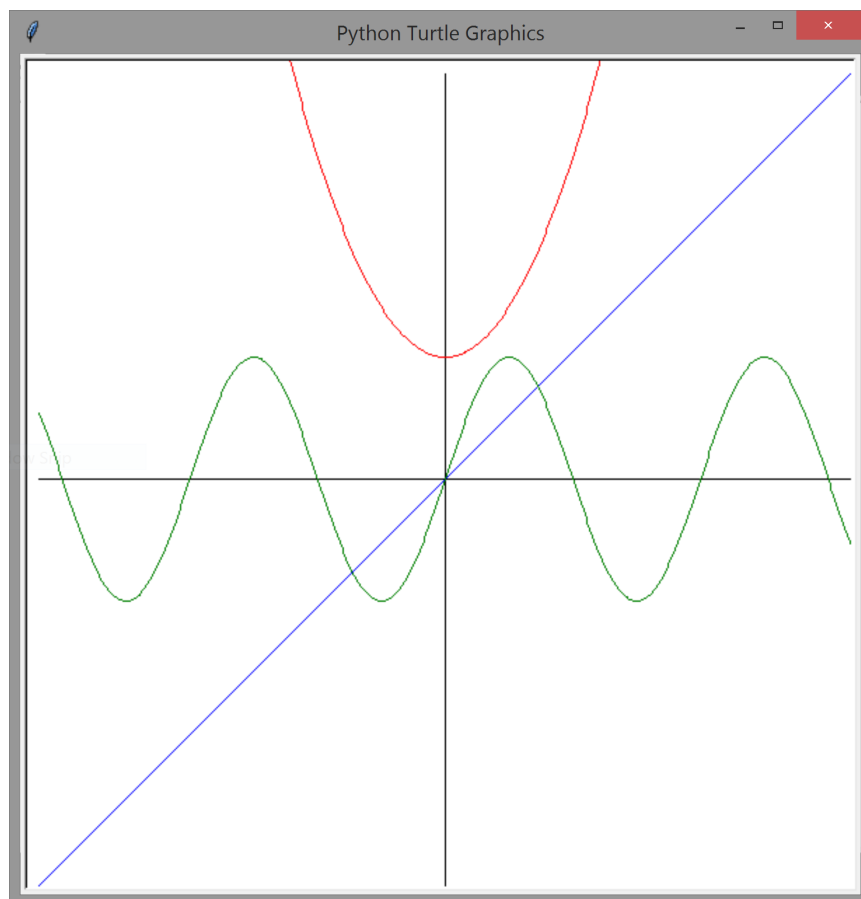
The `main` function passes to `plot` the latter two functions as `lambda` functions. Observe that `plot` accepts both named functions and `lambda` functions with equal ease. Figure 8.7 provides a screenshot of the program's execution.

8.9 Generators

Ordinarily when a function returns to its caller, the function relinquishes all of the memory for its local variables and parameters. The executing program then reuses this memory during calls to other functions, including re-calls to the same function. As a consequence, a function begins fresh during every invocation with no traces of its past execution. This means a function normally cannot remember anything about past invocations.

We could use global variables to allow a function to remember some information. The function `remember` in Listing 8.22 (funcmemory.py) uses a global variable to keep track of the number of times the program has invoked it during a particular run.

Figure 8.7 A screenshot of the `plot` function rendering the mathematical functions $f(x) = \frac{1}{2}x^2 + 3$ (red), $f(x) = x$ (blue), and $f(x) = 3 \sin x$ (green), all over the range $-10 \leq x < 10$.



Listing 8.22: funcmemory.py

```
count = 0          # A global count variable

def remember() -> None:
    global count
    count += 1      # Count this invocation
    print('Calling remember (#' + str(count) + ')')

print('Beginning program')
remember()
remember()
remember()
remember()
remember()
print('Ending program')
```

Listing 8.22 (funcmemory.py) prints

```
Beginning program
Calling remember (#1)
Calling remember (#2)
Calling remember (#3)
Calling remember (#4)
Calling remember (#5)
Ending program
```

Functions that access no global variables have precisely predictable behavior, which is a very desirable quality. In isolation we have no way to predict what the following statement will print:

```
remember()
```

We need to know the complete context. Certainly we need to know how many times the executing program previously called `remember`. Even that knowledge is insufficient in the context of a larger program that involves other functions. Other functions conceivably could manipulate the global `count` variable in between invocations to `remember`.

In order to write functions with persistence we need to use programming *objects*. We consider objects in depth in Chapters 9 and beyond, but for now we will consider a Python programming feature that invisibly uses an object behind the scenes.

A *generator* is a programming object that produces (that is, *generates*) a sequence of values. Code that uses a generator may obtain one value in the sequence at a time without the possibility of revisiting earlier values. We say the code that uses the generator *consumes* the generator's product.

Given only our current knowledge of functions, we can easily make and use generator objects. We create a generator within a function and “return” it. We do not use the `return` keyword; instead, we use the `yield` keyword. The code within the function definition specifies the behavior of the generator. A few simple examples illustrate how this works.

First, consider the module defined in Listing 8.23 (yieldsequence.py).

Listing 8.23: yieldsequence.py

```
from typing import Any, Generator

def gen() -> Generator[Any, None, None]:
    yield 3
    yield 'wow'
    yield -1
    yield 1.2
```

The following interactive sequence reveals some information about the `gen` function within Listing 8.23 (`yieldsequence.py`):

```
>>> from yieldsequence import gen
>>> gen
<function gen at 0x00FA14B0>
>>> type(gen)
<class 'function'>
>>> gen()
<generator object gen at 0x00FAA300>
>>> type(gen())
<class 'generator'>
>>> x = gen()
>>> x
<generator object gen at 0x00FAA300>
>>> type(x)
<class 'generator'>
```

We see that `gen` is just a function, and, when invoked, `gen` returns a generator object. The type hint for the simple generator object here is `Generator[Any, None, None]`. The first type in the square brackets is the type of value the generator will return. The other two types can be `None` for the simple generators we will consider. What can we do with a generator object?

Python has a built-in function named `next` that accepts a generator object and returns the next value in the generator's sequence. Consider the following interactive sequence:

```
>>> from yieldsequence import gen
>>> x = gen()
>>> next(x)
3
>>> next(x)
'wow'
>>> next(x)
-1
>>> next(x)
1.2
>>> next(x)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
StopIteration
```

The statement

```
x = gen()
```


binds variable `x` to the generator object that `gen` returns. Once we have a generator object we can use the `next` function to extract the values in its sequence. Observe that we get an error if we ask the generator to provide a value after the final value in its sequence.

Programmers generally do not use the `next` function directly. Instead, they leave it to the `for` statement to call `next` behind the scenes. A generator object is one example of an iterable object. We learned in Section 5.3 that Python's `for` statement is built to work with an iterable object. The `for` statement, therefore, can iterate over the sequence of values produced by a generator object. Listing 8.24 (`forgenerator.py`) shows that the `for` statement works naturally with the generator our `gen` function produces.

Listing 8.24: `forgenerator.py`

```
from typing import Any, Generator

def gen() -> Generator[Any, None, None]:
    yield 3
    yield 'wow'
    yield -1
    yield 1.2

for i in gen():
    print(i)
```

Listing 8.24 (`forgenerator.py`) prints

```
3
wow
-1
1.2
```

The `for` statement basically receives the next value from the generator each time through the loop. The loop stops automatically when the generator has no more values left.

The `yield` statement within the function generates the values of the sequence. It is uncommon to provide separate `yield` statements for each value the generator is to produce. More likely, one `yield` statement executes multiple times within a loop. Listing 8.25 (`regulargenerator.py`) shows a more common scenario.

Listing 8.25: `regulargenerator.py`

```
from typing import Generator

def generate_multiples(m: int, n: int) -> Generator[int, None, None]:
    count = 0
    while count < n:
        yield m * count
        count += 1

if __name__ == '__main__':
    for mult in generate_multiples(3, 6):
        print(mult, end=' ')
    print()
```

Listing 8.25 (`regulargenerator.py`) prints the first six multiples of three:

```
0 3 6 9 12 15
```

The `generate_multiples` function in Listing 8.25 (`regulargenerator.py`) contains only one `yield` statement, but the loop executes the `yield` statement `n` times.

Listing 8.26 (`myrange.py`) shows how we can use a generator to simulate the behavior of the built-in `range` expression.

Listing 8.26: `myrange.py`

```
from typing import Generator

def myrange(arg1: int, arg2=None, step: int=1) -> Generator[int, None, None]:
    if arg2 != None:    # Do we have at least two arguments?
        begin = arg1
        end = arg2
    else:              # We must have just one argument
        begin = 0      # Begin value is zero by default
        end = arg1
    i = begin
    while i != end:
        yield i
        i += step

print('0 to 9:', end=' ')
for i in myrange(10):
    print(i, end=' ')
print()

print('1 to 10:', end=' ')
for i in myrange(1, 11):
    print(i, end=' ')
print()

print('2 to 18 by twos:', end=' ')
for i in myrange(2, 20, 2):
    print(i, end=' ')
print()

print('20 down to 2 by twos:', end=' ')
for i in myrange(20, 0, -2):
    print(i, end=' ')
print()
```

Listing 8.26 (`myrange.py`) prints

```
0 to 9: 0 1 2 3 4 5 6 7 8 9
1 to 10: 1 2 3 4 5 6 7 8 9 10
2 to 18 by twos: 2 4 6 8 10 12 14 16 18
20 down to 2 by twos: 20 18 16 14 12 10 8 6 4 2
```

While our `myrange` function works like Python's built-in `range` expression, in fact, `range` is different. A simple exercise with the interactive shell reveals:

```
>>> range
<class 'range'>
>>> range(10)
range(0, 10)
>>> type(range)
<class 'type'>
>>> type(range(10))
<class 'range'>
```

The expression `range(0, 10)` does not return a generator object but instead creates and returns a `range` object. Furthermore, the interactive sequence shows that `range` is not a function at all; it is a *class*. In reality, the expression `range(0, 10)` calls the `range` class *constructor*. We will not be concerned with such details about objects until Chapter 13. For now we will be content with the understanding that the `for` statement is designed to work with iterable objects, and generators and `range` objects are both instances of iterable objects.

Our `myrange` function may be interesting, but it offers no advantage over the built-in `range` expression. It is time to use a generator in a more interesting way. Recall Listing 7.16 (`primefunc.py`) that uses a function named `is_prime` in the course of printing the prime numbers within a range specified by the user. What if we wish to print only the prime numbers within a range that end with the digit 3? What if we wish to add up all the prime numbers within a given range? A generator is ideal for implementing the more modular and flexible code required to generate prime numbers independent of how a program uses them. Listing 8.27 (`generatedprimes.py`) uses a generator function to produce the sequence of prime numbers. The caller (`main`) then can select which values to print and sum in a sequence.

Listing 8.27: `generatedprimes.py`

```
# Contains the definition of the is_prime function
from typing import Generator
from math import sqrt

def is_prime(n: int) -> bool:
    """ Returns True if nonnegative integer n is prime;
        otherwise, returns false """
    if n == 2: # 2 is the only even prime number
        return True
    if n < 2 or n % 2 == 0: # Handle simple cases immediately
        return False # No evens and nothing less than 2
    trial_factor = 3
    root = round(sqrt(n))
    while trial_factor <= root:
        if n % trial_factor == 0: # Is trial factor a factor?
            return False # Yes, return right away
        trial_factor += 2 # Next potential factor, skip evens
    return True # Tried them all, must be prime

def prime_sequence(begin: int, end: int) -> Generator[int, None, None]:
    """ Generates the sequence of prime numbers between begin and end. """
    for value in range(begin, end + 1):
        if is_prime(value): # See if value is prime
            yield value # Produce the prime number
```

```

def main() -> None:
    """ Experiments with the prime number generator """
    min_value = int(input("Enter start of range: "))
    max_value = int(input("Enter last of range: "))

    print('Print all the primes from', min_value, 'to', max_value)
    for value in prime_sequence(min_value, max_value):
        print(value, end=' ') # Display the prime number
    print() # Move cursor down to next line

    print('Print all the primes in that range that end with digit 3')
    for value in prime_sequence(min_value, max_value):
        if value % 10 == 3: # See if value's ones digit is 3
            print(value, end=' ') # Display the number
    print() # Move cursor down to next line

    # Add up all the primes in the range
    sum = 0
    for value in prime_sequence(min_value, max_value):
        sum += value
    print('The sum of the primes in that range is', sum)

    # Decorate the output
    print('Fancier display')
    for value in prime_sequence(min_value, max_value):
        print('<' + str(value) + '>', end='')

if __name__ == '__main__':
    main() # Run the program

```

Listing 8.27 (generatedprimes.py) prints

```

Enter start of range: 20
Enter last of range: 50
Print all the primes from 20 to 50
23 29 31 37 41 43 47
Print all the primes in that range that end with digit 3
23 43
The sum of the primes in that range is 251
Fancier display
<23><29><31><37><41><43><47>

```

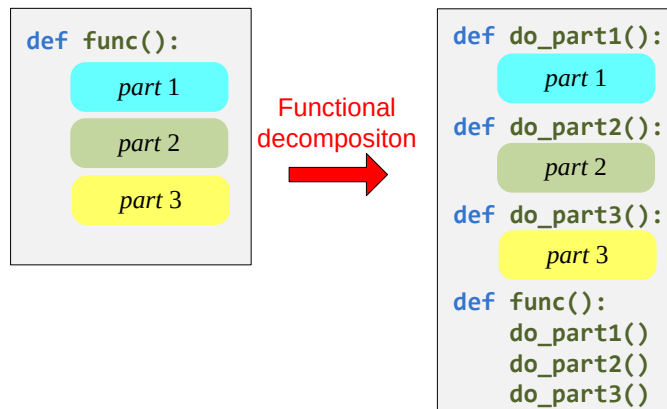
Each call of the expression

```
prime_sequence(min_value, max_value)
```

creates a new generator object ready to use.

As Listing 8.27 (generatedprimes.py) demonstrates, a generator is useful when you need to centralize and reuse code that produces a sequence of values and you cannot predict how consumers of the sequence will use those values.

Figure 8.8 Decomposing a larger function into a collection of smaller functions. Callers now have access to the individual functions `do_part1`, `do_part2`, and `do_part3`, as well as to `func`.



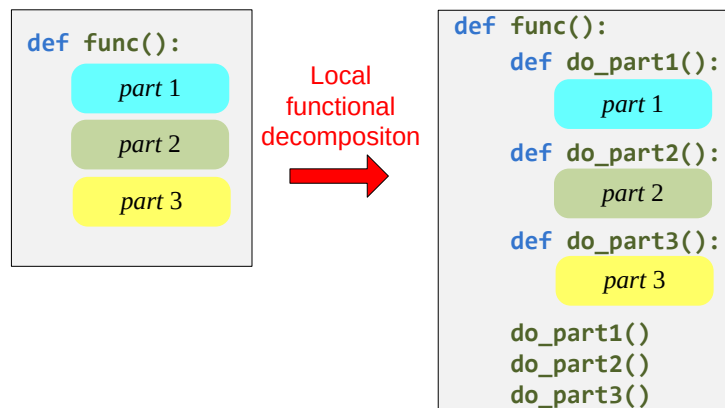
8.10 Local Function Definitions

In Chapter 7 we introduced functional decomposition—a fancy term that means programmers can use functions to break up a large, complex, monolithic program into smaller, more manageable pieces. The code within a function is somewhat insulated from the rest of the program, in that a caller can influence the behavior of a function only via the function’s parameters and any global variables the function may use. Any local variables the function uses are invisible to code outside of the function.

Suppose we develop a function that itself becomes large and unwieldy. We further can break down the large function into smaller pieces as Figure 8.8 shows. The perhaps unintended consequence of this functional decomposition is that callers now can bypass the original function and access its pieces directly and individually. Sometimes this more fine-grained access is desirable, but at other times programmers do not want to expose that level of detail to callers.

Generalizing the concept of local variables, Python permits programmers to define functions within other function definitions. These local functions are available to the code *within* their enclosing function but are inaccessible *outside* their enclosing function. Figure 8.9 shows how to restructure the `func` function definition using local functions to enable functional decomposition without violating the code encapsulation of the original monolithic function. Listing 8.28 (`boxmeasure.py`) includes a function named `surface_area` that computes the surface area of a rectangular box. The function expects the eight points that represent the corners of the box. Note that the `surface_area` function uses a local function to compute the area of each of its sides. Listing 8.28 (`boxmeasure.py`) also includes a function named `volume` also computes the box’s volume.

Figure 8.9 The local functions `do_part1`, `do_part2`, and `do_part3` are available to function `func` but are inaccessible outside the `func` function.



Listing 8.28: boxmeasure.py

```

from math import fabs

def surface_area(x1, y1, z1, x2, y2, z2, x3, y3, z3, x4, y4, z4,
                x5, y5, z5, x6, y6, z6, x7, y7, z7, x8, y8, z8):
    """ Computes the surface area of a rectangular box
        (cuboid) defined by the 3D points (x,y,z) of
        its eight corners:

            7-----8
           /|       /|
          3-----4 |
          | |       | |
          | 5-----6
          | /       | /
          1-----2

    """

    # Local helper function to compute area
    def area(length, width):
        """ Computes the area of a length x width rectangle """
        return length * width

    # Main code for surface_area function
    # Compute area of front face
    length = fabs(x2 - x1)
    height = fabs(y3 - y1)
    front_area = area(length, height)
    # Compute area of side face
    width = fabs(z5 - z1)
    side_area = area(width, height)
    # Compute area of top face
    top_area = area(length, width)
    # Compute and return surface area: front/back,
    # left side/right side, and top/bottom faces
    return 2*front_area + 2*side_area + 2*top_area

def volume(length, width, height):
    """ Computes the volume of a rectangular box
        (cuboid) defined by its length, width, and height """
    return length * width * height

def get_point(msg):
    """ Prints a message specified by msg and allows the user to
        enter the (x, y, z) coordinates of a point. Returns the
        point as a tuple. """
    print(msg)
    x = float(input("Enter x coordinate: "))
    y = float(input("Enter y coordinate: "))
    z = float(input("Enter z coordinate: "))
    return x, y, z

```

```

# Get the coordinates of the box's corners from the user
print('Enter the coordinates of each of the box\'s corners')
print(''
      7-----8
      /|      /|
      3-----4 |
      | |      | |
      | 5-----6
      | /      | /
      1-----2
      '')
x1, y1, z1 = get_point('Corner 1')
x2, y2, z2 = get_point('Corner 2')
x3, y3, z3 = get_point('Corner 3')
x4, y4, z4 = get_point('Corner 4')
x5, y5, z5 = get_point('Corner 5')
x6, y6, z6 = get_point('Corner 6')
x7, y7, z7 = get_point('Corner 7')
x8, y8, z8 = get_point('Corner 8')

# Compute the surface area of the box
print('Surface area:', surface_area(x1, y1, z1, x2, y2, z2,
                                   x3, y3, z3, x4, y4, z4,
                                   x5, y5, z5, x6, y6, z6,
                                   x7, y7, z7, x8, y8, z8))

# Compute the volume of the box
ln = fabs(x2 - x1) # Compute length
wd = fabs(z5 - z1) # Compute width
ht = fabs(y3 - y1) # Compute height
print('Volume:', volume(ln, wd, ht))

```


The following shows a sample run of Listing 8.28 (boxmeasure.py):

```
Enter the coordinates of each of the box's corners
```

```

  7-----8
 /|      /|
3-----4 |
| |      | |
| 5----|-6
|/      |/
1-----2

```

```

Corner 1
Enter x coordinate: 0
Enter y coordinate: 0
Enter z coordinate: 0
Corner 2
Enter x coordinate: 2
Enter y coordinate: 0
Enter z coordinate: 0
Corner 3
Enter x coordinate: 0
Enter y coordinate: 2
Enter z coordinate: 0
Corner 4
Enter x coordinate: 2
Enter y coordinate: 2
Enter z coordinate: 0
Corner 5
Enter x coordinate: 0
Enter y coordinate: 0
Enter z coordinate: 2
Corner 6
Enter x coordinate: 2
Enter y coordinate: 0
Enter z coordinate: 2
Corner 7
Enter x coordinate: 0
Enter y coordinate: 2
Enter z coordinate: 2
Corner 8
Enter x coordinate: 2
Enter y coordinate: 2
Enter z coordinate: 2
Surface area: 24.0
Volume: 8.0

```

Only code within the `surface_area` function can use the `area` function. If we wanted to, we could define a function named `area` local to the `volume` function, and, if we did, the two like-named functions would be completely separate functions.

Local functions can access the local variables and other local functions defined by enclosing function. As with the global functions we have seen before this section, any variable defined within a local function is a local variable of that function. If we need a local function to modify a variable defined in its outer scope (its enclosing function), we must declare the variable as `nonlocal`. The `global` keyword declares

a variable as truly global, so we cannot use the `global` keyword in place of `nonlocal` in this situation.

Listing 8.29 (localcounter.py) uses a local function to mimic a generator.

Listing 8.29: localcounter.py

```
from typing import Callable

def create_counter(n: int) -> Callable[[], int]:
    """ Creates a counting function that counts up to n """
    count = 0

    def counter() -> int:        # Local function
        """ Increments the outer variable unless it
            has reached its limit """
        nonlocal count
        if count < n:
            count += 1
        return count

    return counter    # Returns a function

ctr = create_counter(4)
print(ctr())
print(ctr())
print(ctr())
print(ctr())
print(ctr())
print(ctr())
```

Listing 8.29 (localcounter.py) prints

```
1
2
3
4
4
4
```

Note that the `create_counter` function returns a function; it returns its own local function. This returned local function “remembers” the value of its enclosing function’s local variable `count`; thus, it represents a closure (see Section 8.8). Since the `count` variable in the enclosing function is not global, no outside code can modify the `count` variable except by calling the function that `create_counter` returns.

It may appear that `create_counter` is similar to a generator function (see Section 8.9). Unfortunately the `create_counter` function in Listing 8.29 (localcounter.py) does not create a generator object, as it has no `yield` statement. This means we cannot use it in a `for` statement, and it does not work with the global `next` function.

As another example of a function returning a local closure, consider the calculation of a *derivative*.

Those familiar with basic calculus will recall the derivative of a function f at a is defined to be

$$f'(a) = \lim_{h \rightarrow 0} \frac{f(a+h) - f(a)}{h}$$

If you are unfamiliar with calculus, all you need to know is that the derivative of a function is itself a function; the above formula shows how to transform a function into its derivative. The process of computing a derivative is known as *differentiation*. The $\lim_{h \rightarrow 0}$ notation indicates that the answer becomes more precise as the value h gets closer to zero. Letting h be exactly zero would result in division by zero, which is undefined. The trick is to make h as small as possible, keeping in mind that the computer's floating-point numbers have limitations.

Based on the mathematical definition we can define a Python function that computes the derivative of another function, as shown here:

```
def derivative(f: Callable[[float], float],
              h: float) -> Callable[[float], float]:
    return lambda x: (f(x + h) - f(x)) / h
```

Note that the `derivative` function returns a function—a `lambda` expression is a simple function definition (see Section 8.8). The function that `derivative` returns is a closure because it captures the function parameters `f` and `h`.

Our `derivative` function allows us to compute the derivative of a function at a given value. This is known as *numerical differentiation*. Another approach (the one emphasized in calculus courses) uses *symbolic differentiation*. Symbolic differentiation transforms the formula for a function into a different formula. The details of symbolic differentiation are beyond the scope of this text, but we will use one of its results for a particular function to check our computed numerical results.

A particular function f is defined as follows:

$$f(x) = 3x^2 + 5$$

If you have studied calculus, you can confirm that f 's derivative, f' , is:

$$f'(x) = 6x$$

Without a knowledge of calculus, we can just accept this as the correct answer so we can test our `derivative` function.

Listing 8.30 (`derivative.py`) uses the `derivative` function on $f(x) = 3x^2 + 5$ and compares its results with the known solution, $f'(x) = 6x$.

Listing 8.30: derivative.py

```

from typing import Callable

def derivative(f: Callable[[float], float],
              h: float) -> Callable[[float], float]:
    """ Approximates the derivative of function f
        given an h value. The closer h is to zero,
        the better the estimate. """
    return lambda x: (f(x + h) - f(x)) / h

def fun(x: float) -> float:    # The function we wish to differentiate
    return 3*x**2 + 5

def ans(x: float) -> float:    # The known derivative to function fun
    return 6*x

# Difference: Approximation better as h -> 0
h = 0.0000001

# Compute the function representing an approximation
# of the derivative of function fun
der = derivative(fun, h)

# Compare the computed derivative to the exact derivative
# derived symbolically
x = 5.0
print('-----')
print('          Approx.      Actual')
print('    x          f(x)      h      f\'(x)      f\'(x)')
print('-----')
while x < 5.1:
    print(f'{x:.5f}    {fun(x):.5f}    {h:.8f}    {der(x):.5f}    {ans(x):.5f}')
    x += 0.01

```

With $h = 0.0000001$, Listing 8.30 (derivative.py) produces good results to the fifth decimal place:

x	f(x)	h	Approx. f'(x)	Actual f'(x)
5.00000	80.00000	0.00000010	30.00000	30.00000
5.01000	80.30030	0.00000010	30.06000	30.06000
5.02000	80.60120	0.00000010	30.12000	30.12000
5.03000	80.90270	0.00000010	30.18000	30.18000
5.04000	81.20480	0.00000010	30.24000	30.24000
5.05000	81.50750	0.00000010	30.30000	30.30000
5.06000	81.81080	0.00000010	30.36000	30.36000
5.07000	82.11470	0.00000010	30.42000	30.42000
5.08000	82.41920	0.00000010	30.48000	30.48000
5.09000	82.72430	0.00000010	30.54000	30.54000
5.10000	83.03000	0.00000010	30.60000	30.60000

Even with h as large as 0.01, the results are not too bad:

x	f(x)	h	Approx. f'(x)	Actual f'(x)
5.00000	80.00000	0.01000000	30.03000	30.00000
5.01000	80.30030	0.01000000	30.09000	30.06000
5.02000	80.60120	0.01000000	30.15000	30.12000
5.03000	80.90270	0.01000000	30.21000	30.18000
5.04000	81.20480	0.01000000	30.27000	30.24000
5.05000	81.50750	0.01000000	30.33000	30.30000
5.06000	81.81080	0.01000000	30.39000	30.36000
5.07000	82.11470	0.01000000	30.45000	30.42000
5.08000	82.41920	0.01000000	30.51000	30.48000
5.09000	82.72430	0.01000000	30.57000	30.54000
5.10000	83.03000	0.01000000	30.63000	30.60000

In Listing 8.30 (derivative.py), the statement

```
der = derivative(fun, h)
```

assigns `der` to the function returned by `derivative`; thus, `der(x)` returns the value of the derivative of `fun` at `x`. In order for `der` to compute its answer, it must have access to function `fun`. It has this access because `der` is a closure that captured `fun` during the call to `derivative`.

8.11 Decorators

In Section 8.10 we saw how we can define a function within another function. That section also showed how we can write a function that returns a function as its result. We can use these two abilities in a clever way to add functionality to arbitrary functions, even functions that we ourselves have not written.

As a simple example, suppose we are building an application that uses a number of functions that accept two integer parameters and return an integer; Listing 8.31 (undecorated.py) provides some code to illustrate.

Listing 8.31: undecorated.py

```

from random import randrange

def max(x: int, y: int) -> int:
    """ Determine the maximum of x and y """
    return x if x > y else y

def gcd(m: int, n: int) -> int:
    """
    Uses Euclid's method to compute the greatest common divisor
    (also called greatest common factor) of m and n.
    Returns the GCD of m and n.
    """
    if n == 0:
        return m
    else:
        return gcd(n, m % n)

def summation(begin: int, end: int) -> int:
    """ Add up the integers in the range begin...end - 1, inclusive """
    sum = 0
    while begin != end:
        sum += begin
        begin += 1
    return sum

def star_rect(length: int, width: int) -> None:
    """ Draw a length x width rectangle with asterisks """
    for row in range(length):
        print('* ' * width)

print("The maximum of 20 and 30 is", max(20, 30))
print("The maximum of 30 and 20 is", max(30, 20))
print("The maximum of 20 and 20 is", max(20, 20))
print('-----')

print("The GCD of 20 and 30 is", gcd(20, 30))
print("The GCD of 30 and 20 is", gcd(30, 20))
print("The GCD of 20 and 20 is", gcd(20, 20))
print('-----')

print("The summation from 20 to 30 is", summation(20, 30))
print('-----')

star_rect(7, 3)
print('-----')
star_rect(4, 4)
print('-----')
star_rect(2, 5)
print('-----')

print(randrange(10, 21),

```

```
"is a pseudorandom integer in the range 10 to 20, inclusive")
```

When executed, Listing 8.31 (undecorated.py) displays

```
The maximum of 20 and 30 is 30
The maximum of 30 and 20 is 30
The maximum of 20 and 20 is 20
-----
The GCD of 20 and 30 is 10
The GCD of 30 and 20 is 10
The GCD of 20 and 20 is 20
-----
The summation from 20 to 30 is 245
-----
* * *
* * *
* * *
* * *
* * *
* * *
* * *
-----
* * * *
* * * *
* * * *
* * * *
-----
* * * * *
* * * * *
-----
14 is a pseudorandom integer in the range 10 to 20, inclusive
```

The pseudorandom number displayed will vary, but the rest of the output will be the same whenever the program runs.

During testing we would like every call to any of these functions (and any of the possibly hundreds more functions that make up the application) to report the function's arguments and return value. Ideally we would write this information to a log file, but, here we simply will print the information. One approach would be to modify each function temporarily for testing purposes, as Listing 8.32 (poorlydecorated.py) demonstrates

Listing 8.32: poorlydecorated.py

```
from random import randrange

def max(x: int, y: int) -> int:
    """ Determine the maximum of x and y """
    call_string = f'max({x}, {y})'
    print('>>> Calling ' + call_string)
    result = x if x > y else y
    print('<<< Returning {result} from ' + call_string)
    return result

def gcd(m: int, n: int) -> int:
```

```

"""
Uses Euclid's method to compute the greatest common divisor
(also called greatest common factor) of m and n.
Returns the GCD of m and n.
"""
call_string = f'gcd({m}, {n})'
print('>>> Calling ' + call_string)
if n == 0:
    result = m
else:
    result = gcd(n, m % n)
print('<<< Returning {result} from ' + call_string)
return result

def summation(begin: int, end: int) -> int:
    """ Add up the integers in the range begin...end - 1, inclusive """
    call_string = f'summation({begin}, {end})'
    print('>>> Calling ' + call_string)
    sum = 0
    while begin != end:
        sum += begin
        begin += 1
    print('<<< Returning {sum} from ' + call_string)
    return sum

def star_rect(length: int, width: int) -> None:
    """ Draw a length x width rectangle with asterisks """
    call_string = f'star_rect({length}, {width})'
    print('>>> Calling ' + call_string)
    for row in range(length):
        print('* ' * width)
    print('<<< Returning {None} from ' + call_string)

# This time just call the functions, and let each function print its
# parameter and result details.
max(20, 30)
print('-----')
max(30, 20)
print('-----')
max(20, 20)
print('-----')

gcd(20, 30)
print('-----')
gcd(30, 20)
print('-----')
gcd(20, 20)
print('-----')

summation(20, 30)
print('-----')

star_rect(7, 3)
print('-----')
star_rect(4, 4)

```



```

print('-----')
star_rect(2, 5)
print('-----')

print(randrange(10, 21),
      "is a pseudorandom integer in the range 10 to 20, inclusive")

```

Note that we print information at the beginning of the function's execution, indicated by the prefix >>> (for entering the function). We also print some information prefixed with <<< at the end of the function's execution (for leaving the function). This allows our output to be readable even for functions that do their own printing like `star_rect`. It also allows us to better track recursive function invocations like `gcd`.

Listing 8.32 (`poorlydecorated.py`) displays

```

>>> Calling max(20, 30)
<<< Returning {result} from max(20, 30)
-----
>>> Calling max(30, 20)
<<< Returning {result} from max(30, 20)
-----
>>> Calling max(20, 20)
<<< Returning {result} from max(20, 20)
-----
>>> Calling gcd(20, 30)
>>> Calling gcd(30, 20)
>>> Calling gcd(20, 10)
>>> Calling gcd(10, 0)
<<< Returning {result} from gcd(10, 0)
<<< Returning {result} from gcd(20, 10)
<<< Returning {result} from gcd(30, 20)
<<< Returning {result} from gcd(20, 30)
-----
>>> Calling gcd(30, 20)
>>> Calling gcd(20, 10)
>>> Calling gcd(10, 0)
<<< Returning {result} from gcd(10, 0)
<<< Returning {result} from gcd(20, 10)
<<< Returning {result} from gcd(30, 20)
-----
>>> Calling gcd(20, 20)
>>> Calling gcd(20, 0)
<<< Returning {result} from gcd(20, 0)
<<< Returning {result} from gcd(20, 20)
-----
>>> Calling summation(20, 30)
<<< Returning {sum} from summation(20, 30)
-----
>>> Calling star_rect(7, 3)
* * *
* * *
* * *
* * *
* * *
* * *
* * *

```

```

<<< Returning {None} from star_rect(7, 3)
-----
>>> Calling star_rect(4, 4)
* * * *
* * * *
* * * *
* * * *
<<< Returning {None} from star_rect(4, 4)
-----
>>> Calling star_rect(2, 5)
* * * * *
* * * * *
<<< Returning {None} from star_rect(2, 5)
-----
14 is a pseudorandom integer in the range 10 to 20, inclusive

```

Listing 8.32 (poorlydecorated.py) displays the information we need, but it comes at considerable cost:

1. We must modify every function with the augmented functionality, and it may involve hundreds of functions in the full application.
2. We must ensure we modify every one of the functions correctly; since each function has different parameter names we simply cannot copy and paste code and use it as is. For functions with `return` statements that involve an expression with multiple components (as in the original `max` function), we must introduce a local variable to print and then return that variable. We must be particularly careful, as mistakes here can alter the function's logic.
3. Once we are finished testing we must put the functions back to their original state. This act of modifying the functions and then changing them back provides ample opportunities for introducing errors into a function that originally contained correct code.

This process is labor intensive and too problematic to use as a general technique.

Another serious problem involves functions over which we have no control. In our code, for example, we cannot (or, better, should not) modify the `randrange` function to provide the call details we seek. The `randrange` function is part of a library function maintained by someone else. If we attempt to change the contents of the `random` module's source code, there is a good chance we could introduce an error. Modifying code within a library, therefore, reduces its trustworthiness. Enhancing `randrange` by modifying its source code is not a viable option.

We really need a way to augment the functions in a uniform way without modifying them. This may seem like an impossible task, but Python's ability to pass functions as arguments and define local functions makes it possible.

Consider the following function:

```
def show_call_and_return_details(f):
    """ f is a function that accepts two arguments. """
    func_name = f.__name__ # Get the function's name
    def execute_augmented(x, y):
        call_string = f'{func_name}({x}, {y})'
        print(">>> Calling " + call_string)
        result = f(x, y)
        print('<<< Returning {result} from ' + call_string)
        return result
    return execute_augmented
```

This `show_call_and_return_details` accepts a single function as a parameter. It “wraps” the function in some additional code within a local function. The `show_call_and_return_details` function finally returns this local function to its caller. Listing 8.33 (`simpledecorator.py`) shows how we can use the `show_call_and_return_details` function to achieve our goal of printing details about function calls without modifying the code of the individual functions.

Listing 8.33: `simpledecorator.py`

```
from typing import Callable, Any
from random import randrange

def show_call_and_return_details(f: Callable[..., Any]) -> Callable[..., Any]:
    """ Decorates a function f so its call will display the
        parameter values and return value. """
    func_name = f.__name__ # Get the function's name
    def execute_augmented(x: int, y: int) -> Callable[..., Any]:
        call_string = f'{func_name}({x}, {y})'
        print(">>> Calling " + call_string)
        result = f(x, y)
        print('<<< Returning {result} from ' + call_string)
        return result
    return execute_augmented

def max(x: int, y: int) -> int:
    """ Determine the maximum of x and y """
    return x if x > y else y

def gcd(m: int, n: int) -> int:
    """
    Uses Euclid's method to compute the greatest common divisor
    (also called greatest common factor) of m and n.
    Returns the GCD of m and n.
    """
    if n == 0:
        return m
    else:
        return gcd(n, m % n)

def summation(begin: int, end: int) -> int:
    """ Add up the integers in the range begin...end - 1, inclusive """
    sum = 0
```

```

    while begin != end:
        sum += begin
        begin += 1
    return sum

def star_rect(length: int, width: int) -> None:
    """ Draw a length x width rectangle with asterisks """
    for row in range(length):
        print('* ' * width)

# Decorate the functions to provide information about their calls
# We can make up a new name
augmented_max = show_call_and_return_details(max)
# Or, more typically, simply redirect the original name to a new function!
gcd = show_call_and_return_details(gcd)
summation = show_call_and_return_details(summation)
star_rect = show_call_and_return_details(star_rect)
randrange = show_call_and_return_details(randrange)

augmented_max(20, 30)
print('-----')
augmented_max(30, 20)
print('-----')
augmented_max(20, 20)
print('-----')

gcd(20, 30)
print('-----')
gcd(30, 20)
print('-----')
gcd(20, 20)
print('-----')

summation(20, 30)
print('-----')

star_rect(7, 3)
print('-----')
star_rect(4, 4)
print('-----')
star_rect(2, 5)
print('-----')

print(randrange(10, 21),
      "is a pseudorandom integer in the range 10 to 20, inclusive")

```

Listing 8.33 (simpledecorator.py) defines the original, unmodified functions from Listing 8.31 (undecorated.py) and adds five statements that redirect each of the functions to their augmented equivalents. The first statement creates a new function name, while the other four reassign the original function's name. Listing 8.33 (simpledecorator.py) displays

```

>>> Calling max(20, 30)
<<< Returning 30 from max(20, 30)
-----

```

```

>>> Calling max(30, 20)
<<< Returning 30 from max(30, 20)
-----
>>> Calling max(20, 20)
<<< Returning 20 from max(20, 20)
-----
>>> Calling gcd(20, 30)
>>> Calling gcd(30, 20)
>>> Calling gcd(20, 10)
>>> Calling gcd(10, 0)
<<< Returning 10 from gcd(10, 0)
<<< Returning 10 from gcd(20, 10)
<<< Returning 10 from gcd(30, 20)
<<< Returning 10 from gcd(20, 30)
-----
>>> Calling gcd(30, 20)
>>> Calling gcd(20, 10)
>>> Calling gcd(10, 0)
<<< Returning 10 from gcd(10, 0)
<<< Returning 10 from gcd(20, 10)
<<< Returning 10 from gcd(30, 20)
-----
>>> Calling gcd(20, 20)
>>> Calling gcd(20, 0)
<<< Returning 20 from gcd(20, 0)
<<< Returning 20 from gcd(20, 20)
-----
>>> Calling summation(20, 30)
<<< Returning 245 from summation(20, 30)
-----
>>> Calling star_rect(7, 3)
* * *
* * *
* * *
* * *
* * *
* * *
* * *
<<< Returning None from star_rect(7, 3)
-----
>>> Calling star_rect(4, 4)
* * * *
* * * *
* * * *
* * * *
<<< Returning None from star_rect(4, 4)
-----
>>> Calling star_rect(2, 5)
* * * * *
* * * * *
<<< Returning None from star_rect(2, 5)
-----
>>> Calling randrange(10, 21)
<<< Returning 14 from randrange(10, 21)
17 is a pseudorandom integer in the range 10 to 20, inclusive

```

In Listing 8.33 (simpledecorator.py), the `show_call_and_return_details` function is known as a *decorator*. A decorator does not change the way a function works; it simply adds some “decoration” to the function, usually to augment the function’s behavior. A decorator “wraps” a function passed to it. Observe that a decorator cannot modify the inner workings of a function. Our example shows that a decorator can do some preprocessing (activity before calling the function it wraps) and postprocessing (activity after calling the function it wraps). A particular decorator could call a different function instead of the function it wraps.

Note the type hint for a function that accepts an unspecified number of parameters of any type and that can return any type of value:

```
Callable[..., Any]
```

This ability to decorate functions is so useful that Python provides a special syntax to support it. We can use this special syntax for functions that we write. To see how it works, we will define our `show_call_and_return_details` function exactly as before. In the line before each function we wish to decorate we will add the following decoration:

```
@show_call_and_return_details
```

Our `max` function definition from Listing 8.33 (simpledecorator.py) would become

```
@show_call_and_return_details
def max(x: int, y: int) -> int:
    """ Determine the maximum of x and y """
    return x if x > y else y
```

With the definition adjusted as shown the caller can use `max` as is without first calling the `show_call_return_details` function. Listing 8.34 (decorator.py) updates Listing 8.33 (simpledecorator.py) with decorated functions.

Listing 8.34: decorator.py

```
from typing import Callable, Any
from random import randrange

def show_call_and_return_details(f: Callable[..., Any]) -> Callable[..., Any]:
    """ Decorates a function f so its call will display the
        parameter values and return value. """
    func_name = f.__name__ # Get the function's name
    def execute_augmented(x: int, y: int) -> Callable[..., Any]:
        call_string = f'{func_name}({x}, {y})'
        print(">>> Calling " + call_string)
        result = f(x, y)
        print(f'<<< Returning {result} from ' + call_string)
        return result
    return execute_augmented

@show_call_and_return_details
def max(x: int, y: int) -> int:
    """ Determine the maximum of x and y """
    return x if x > y else y

@show_call_and_return_details
def gcd(m: int, n: int) -> int:
```

```

    """
    Uses Euclid's method to compute the greatest common divisor
    (also called greatest common factor) of m and n.
    Returns the GCD of m and n.
    """
    if n == 0:
        return m
    else:
        return gcd(n, m % n)

@show_call_and_return_details
def summation(begin: int, end: int) -> int:
    """ Add up the integers in the range begin...end - 1, inclusive """
    sum = 0
    while begin != end:
        sum += begin
        begin += 1
    return sum

@show_call_and_return_details
def star_rect(length: int, width: int) -> None:
    """ Draw a length x width rectangle with asterisks """
    for row in range(length):
        print('* ' * width)

max(20, 30)
print('-----')
max(30, 20)
print('-----')
max(20, 20)
print('-----')

gcd(20, 30)
print('-----')
gcd(30, 20)
print('-----')
gcd(20, 20)
print('-----')

summation(20, 30)
print('-----')

star_rect(7, 3)
print('-----')
star_rect(4, 4)
print('-----')
star_rect(2, 5)
print('-----')

print(randrange(10, 21),
      "is a pseudorandom integer in the range 10 to 20, inclusive")

@show_call_and_return_details
def star_rect(length, width):
    """ Draw a length x width rectangle with asterisks """

```

```

    for row in range(length):
        print('* ' * width)

# Decorate the randrange function as before; we cannot alter its
# source code
randrange = show_call_and_return_details(randrange)

max(20, 30)
print('-----')
max(30, 20)
print('-----')
max(20, 20)
print('-----')

gcd(20, 30)
print('-----')
gcd(30, 20)
print('-----')
gcd(20, 20)
print('-----')

summation(20, 30)
print('-----')

star_rect(7, 3)
print('-----')
star_rect(4, 4)
print('-----')
star_rect(2, 5)
print('-----')

print(randrange(10, 21),
      "is a pseudorandom integer in the range 10 to 20, inclusive")

```

When executed, Listing 8.34 (decorator.py) behaves exactly as Listing 8.33 (simpledecorator.py).

Since we do not control the definition of the `randrange` function we cannot decorate its definition using the special `@` syntax. As Listing 8.34 (decorator.py) shows, however, we can decorate it in the manner shown earlier in Listing 8.33 (simpledecorator.py).

Decorators open the door to many interesting possibilities. Suppose you wish remove all functionality from a function that accepts two parameters. The following decorator would work:

```

def nullify(f: Callable[..., Any]) -> Callable[..., Any]:
    """ A decorator that guts the functionality of a function that
        requires two parameters. """
    def empty_func(x: Any, y: Any) -> Any:
        """ This function does nothing and returns None """
        pass
    return empty_func

```

Our `max` function decorated as such:

```
@nullify
```



```
def max(x, y):  
    """ Determine the maximum of x and y """  
    return x if x > y else y
```

would do nothing when called! If in Listing 8.34 (decorator.py) we changed all the `show_call_and_result_details` decorators to be `nullify` decorators, the program would print

```
-----  
-----  
-----  
-----  
-----  
-----  
-----  
-----  
-----  
-----  
-----  
-----  
-----  
-----  
-----  
None is a pseudorandom integer in the range 10 to 20, inclusive
```

We can use a decorator to measure the speed of a function's execution. Listing 8.35 (timeprimesdecorated.py) decorates a function that counts the number of primes in a given range.

Listing 8.35: timeprimesdecorated.py

```

from typing import Callable
from math import sqrt
from time import perf_counter_ns

def time_decorator(f: Callable[[int, int], int]) \
    -> Callable[... , tuple[int, float]]:
    """
    Decorates a function that accepts two parameters.
    Returns in a tuple the function's original result
    along with an elapsed time in seconds.
    """
    def timer(x: int, y: int) -> tuple[int, float]:
        start_time = perf_counter_ns()
        result = f(x, y)
        stop_time = perf_counter_ns()
        return result, (stop_time - start_time)/1_000_000_000

    return timer

def is_prime(n: int) -> int:
    """
    Returns True if nonnegative integer n is prime;
    otherwise, returns false
    """
    if n == 2:                                # 2 is the only even prime number
        return True
    if n < 2 or n % 2 == 0:                   # Handle simple cases immediately
        return False                         # No evens and nothing less than 2
    trial_factor = 3
    root = round(sqrt(n))
    while trial_factor <= root:
        if n % trial_factor == 0:             # Is trial factor a factor?
            return False                     # Yes, return right away
        trial_factor += 2                    # Next potential factor, skip evens
    return True                               # Tried them all, must be prime

@time_decorator
def count_primes(begin: int, end: int) -> int:
    count = 0
    for i in range(begin, end + 1):
        if is_prime(i):
            count += 1
    return count

if __name__ == '__main__':
    starting_value = 2
    ending_value = 1_000_000
    c, t = count_primes(starting_value, ending_value)
    print(f'Number of primes from {starting_value} ', end='')
    print(f'to {ending_value}: {c}')
    print(f'Execution time: {t} sec')

```

The `count_primes` function in Listing 8.35 (`timeprimesdecorated.py`) counts the number of primes between a starting and ending value, inclusive. The decorator, named `time_decorator`, wraps `count_primes` and times how long it takes to complete its task. The decorator also augments the return value of `count_primes`. The `count_primes` function returns a single integer value, the number of prime numbers in the range. The decorated version returns a tuple consisting of a count of the number of primes and an elapsed time in seconds. One run of Listing 8.35 (`timeprimesdecorated.py`) displays

```
Number of primes from 2 to 1000000: 78498
Execution time: 2.8406591 sec
```

The necessity to define a particular number of parameters for the function that the decorator wraps significantly reduces the decorator's generality. Fortunately, we are limited now only by our current knowledge of Python; Chapter 11 introduces the Python concepts that enable us to design decorators that apply to functions accepting any number of parameters.

8.12 Partial Application

In Section 8.6 we saw how we can pass functions as arguments to other functions. Section 8.8 showed how a function could serve as a return value from another function. The `functools` module provides an interesting function named `partial` that accepts a function as its first parameter and one or more other parameters. The `partial` function returns a new function that is behaviorally related to the original function passed to it. To see what `partial` does, consider the function `add`:

```
def add(x: int, y: int) -> int:
    return x + y
```

The `add` function expects two arguments. The interpreter will not allow a caller to pass fewer than two or more than two parameters. Given the following import statement:

```
from functools import partial
```

we can use the `partial` function to make a new addition function that requires only one argument:

```
add5 = partial(add, 5)
```

This new `add5` function accepts a single parameter. An example call would be

```
print(add5(3))    # Works like print(add(5, 3))
```

The `add5` invocation calls the original `add` function with 5 as the first argument and `add5`'s parameter, 3, as the second argument.

This technique is known as *partial application*. Partial application allows us to make a new function from an existing function with one or more of the original function's actual parameters “hardwired” into the definition. This new function exhibits the same behavior as the original function but requires fewer parameters during its call.

Partial application can predetermine only *leading parameters*. It is not possible to predetermine a parameter that follows a non-predetermined parameter.

Partial application is a rarely used, advanced Python feature, but we can illustrate its utility in a simple program. Consider a program meant to compare a dice throw simulation to the behavior of a pair of real, physical dice. Suppose a person performs an experiment with a pair of dice, rolling them hundreds of times

and recording the result each time. The number recorded is the sum of the numbers on both dice. Since each die face contains one, two, three, four, five, or six spots, the outcome of a roll of two dice can be any number in the range 2–12, inclusive. The person conducting the experiment records the results in a text file named `dicedata.data`.

The following function opens a text file expecting a list of integer values representing the outcomes of hundreds of dice rolls.

```
def read_file(filename: str, n: int, val: int) -> int:
    """ Reads n integers from the text file named filename.
        Returns the number of times val appears in the file. """
    count, read = 0, 0
    with open(filename, 'r') as f:
        for value in f.readlines():
            read += 1
            # Have we read enough values in yet?
            if read > n:
                break
            # Convert text integer into an actual integer
            if int(value) == val:
                count += 1
    return count
```

The `read_file`'s parameters consist of the name of the file to process (`filename`), the number of outcomes to consider (`n`), and the outcome value to count (`val`). By passing in the file name, `read_file` can flexibly handle any data files of the proper format. The expectation is that the file will be very large, and the parameter `n` limits how much of the data the function processes.

The `read_file` function uses two new keywords: `with` and `as`. Section 9.3 covers these keywords in more detail.

Now we need our program to perform the same experiment, simulating dice rolls using a pseudorandom number generator (see Section 6.6 for a review of pseudorandom number generation). The following function simulates rolling a pair of dice a given number of times and counts the number of times a particular value appears:

```
def roll(n: int, val: int):
    """ Simulates the roll of a pair of dice n times.
        Returns the number of times a roll resulted in val. """
    count = 0
    for i in range(n):
        roll = randint(1, 6) + randint(1, 6)
        if roll == val:
            count += 1
    return count
```

The `roll` function does not need any file name since it generates the numbers itself. The function otherwise behaves similarly to `read_file`.

The following `run_trials` function performs the experiment with a given number of throws and counts the number of times each outcome occurs:

```
def run_trials(f: Callable[[], int], n: int):
    """ Performs n experiments using function f as the source of
        outcomes. Counts the number of occurrences of each possible
        outcome. """
    for value in range(2, 13):
        print(f'{value:>3}:{f(n, value):>5}')
```

Observe that `run_trials` invokes function `f` passing to it two values.

Our `roll` function is perfectly compatible as a potential `f` parameter for `run_trials`, but `read_file` is not. The `read_file` function requires three parameters, not two. Is there any way we can make `read_file` compatible with `run_trials`?

Partial application comes to our rescue. The `read_file` function flexibly works with any appropriate data file, but `run_trials` cannot by itself handle this flexibility. The expression

```
partial(read_file, 'dicedata.data')
```

evaluates to a new function that, when invoked, will invoke `read_file` with the string `'dicedata.data'` hardwired as the first parameter. This new function accepts just two parameters (the remaining parameters `n` and `val` left over from `read_file`) and so will work perfectly as the first argument to `run_trials`.

Listing 8.36 (`comparerolls.py`) pulls everything together into an executable program.

Listing 8.36: `comparerolls.py`

```
from typing import Callable
from random import randint
from functools import partial

def read_file(filename: str, n: int, val: int) -> int:
    """ Reads n integers from the text file named filename.
        Returns the number of times val appears in the file. """
    count, read = 0, 0
    with open(filename, 'r') as f:
        for value in f.readlines():
            read += 1
            # Have we read enough values in yet?
            if read > n:
                break
            # Convert text integer into an actual integer
            if int(value) == val:
                count += 1
    return count

def roll(n: int, val: int) -> int:
    """ Simulates the roll of a pair of dice n times.
        Returns the number of times a roll resulted in val. """
    count = 0
    for i in range(n):
        roll = randint(1, 6) + randint(1, 6)
        if roll == val:
            count += 1
```

```

        return count

def run_trials(f: Callable[[int, int], int], n: int) -> None:
    """ Performs n experiments using function f as the source of
        outcomes. Counts the number of occurrences of each possible
        outcome. """
    for value in range(2, 13):
        print(f'{value:>3}:{f(n, value):>5}')

# Compare the actual experiments to the simulation
number_of_trials = 100
print('--- Pseudorandom number rolls ---')
run_trials(roll, number_of_trials)
print('--- Actual experimental data ---')
run_trials(partial(read_file, 'dicedata.data'), number_of_trials)

```

Given the simplicity of Listing 8.36 (comparerolls.py), you may be thinking of ways to rewrite the code to avoid using partial application. Restructuring this code is indeed an option, since we have total control over it. Partial application really shines when we need to interface with library functions over which we have no control. Partial application is a tool that sometimes is handy for quickly adapting an existing function to the requirements of a library developed by others.

8.13 Exercises

1. Consider the following Python code:

```

def sum1(n):
    s = 0
    while n > 0:
        s += 1
        n -= 1
    return s

def sum2():
    global val
    s = 0
    while val > 0:
        s += 1
        val -= 1
    return s

def sum3():
    s = 0
    for i in range(val, 0, -1):
        s += 1
    return s

```

```
def main():
    # See each question below for details

main()    # Call main function
```

- (a) What is printed if `main` is written as follows?

```
def main():
    global val
    val = 5
    print(sum1(5))
    print(sum2())
    print(sum3())
```

- (b) What is printed if `main` is written as follows?

```
def main():
    global val
    val = 5
    print(sum1(5))
    print(sum3())
    print(sum2())
```

- (c) What is printed if `main` is written as follows?

```
def main():
    global val
    val = 5
    print(sum2())
    print(sum1(5))
    print(sum3())
```

- (d) Which of the functions `sum1`, `sum2`, and `sum3` produce a side effect? What is the side effect?
- (e) Which function may not use the `val` variable?
- (f) What is the scope of the variable `val`? What is its lifetime?
- (g) What is the scope of the variable `i`? What is its lifetime?
- (h) Which of the functions `sum1`, `sum2`, and `sum3` demonstrate good functional independence? Why?

2. Consider the following Python code:

```
def next_int1():
    cnt = 0
    cnt += 1
    return cnt

global_count = 0

def next_int2():
    global_count += 1
```

```
        return global_count

def main():
    for i = range(0, 5):
        print(next_int1(), next_int2())

main()
```

- (a) What does the program print?
 - (b) Which of the functions `next_int1` and `next_int2` is the best function for the intended purpose? Why?
 - (c) What is a better name for the function named `next_int1`?
 - (d) The `next_int2` function works in this context, but why is it not a good implementation of function that always returns the next largest integer?
3. When is the `global` statement required?
4. What does the following Python program print?

```
def sum(m=0, n=0, r=0):
    return m + n + r

def main():
    print(sum())
    print(sum(4))
    print(sum(4, 5))
    print(sum(5, 4))
    print(sum(1, 2, 3))
    print(sum(2.6, 1.0, 3))

main()
```

5. Consider the following function:

```
def proc(n):
    if n < 1:
        return 1
    else:
        return proc(n/2) + proc(n - 1)
```

Evaluate each of the following expressions:

- (a) `proc(0)`
- (b) `proc(1)`
- (c) `proc(2)`
- (d) `proc(3)`

- (e) `proc(5)`
- (f) `proc(10)`

6. Rewrite the `gcd` function so that it implements Euclid's method but uses iteration instead of recursion.

7. Classify the following functions as pure or impure. `x` is a global variable.

- (a)

```
def f1(m, n):
    return 2*m + 3*n
```
- (b)

```
def f2(n):
    return n - 2
```
- (c)

```
def f3(n):
    return n - x
```
- (d)

```
def f4(n):
    print(2*n)
```
- (e)

```
def f5(n):
    m = int(input())
    return m * n
```
- (f)

```
def f6(n):
    m = 2*n
    p = 2*m - 5
    return p - n
```

8. Consider the following very simple module, found in the file `mymod.py`:

```
""" Provides the increment function, increment. """

def increment(x):
    """ Increments x by 1 and returns the result. """
    return x + 1
```

A programmer wishes to use the `increment` function from the `mymod.py` module. Indicate which, if any, of the following code snippets would work:

- (a) `import mymod`

```
print(increment(4))    # Supposed to print 5
```

```
import from mymod import increment
```

```
print(increment(4))    # Supposed to print 5
```

- (b) `import mymod`

```
print(mymod.increment(4))    # Supposed to print 5
```

```
from mymod import increment
```

```
print(mymod.increment(4))    # Supposed to print 5
```

- (9) Modify the main function in Listing 8.27 (generatedprimes.py) that so that it prints all the prime numbers less than 1,000 that contain the digit 2 or digit 3 (or both).

10. Write a generator function named `evens` that enables the following code:

```
for n in evens_less_than(12):  
    print(n, end=' ')  
print()
```

to print

```
2 4 6 8 10
```

that is, all positive even numbers less than 12.

11. Functions as data TODO Consider the following function definition:

```
def f():  
    pass
```

12. Lambda expressions TODO

13. Write a generator function named `oscillate` that enables the following code:

```
for n in oscillate(-3, 5):  
    print(n, end=' ')  
print()
```

to print

```
-3 3 -2 2 -1 1 0 0 1 -1 2 -2 3 -3 4 -4
```

14. Local functions TODO

15. Partial application TODO

Chapter 9

Objects

From the hardware perspective, a computer consists of a motherboard (a circuit board containing sockets for a processor and assorted support chips), a processor, memory, a video card, etc. The video card by itself is a sophisticated piece of hardware containing a video processor chip, memory, and other electronic components. A technician does not need to assemble the card but uses it as is off the shelf during computer assembly. The video card provides a substantial amount of functionality in a standard package. One video card can be replaced with another card from a different vendor or with different capabilities. The computer will work with either card (subject to availability of drivers for the operating system) because standard interfaces allow the components to work together.

Software development today is increasingly *component based*. Developers use software components like computer engineers use hardware components. Developers can build large parts of a software system by assembling pre-existing software building blocks. Python supports various kinds of software building blocks. The simplest of these is the *function*, that we investigated in Chapter 6 and Chapter 7. A more powerful technique uses software *objects*.

Python is *object oriented*. Most modern programming languages support object-oriented (OO) development to one degree or another. An OO programming language allows the programmer to define, create, and manipulate objects. Objects bundle together data and functions. Like other variables, each Python object has a type, or *class*. The terms *class* and *type* are synonymous.

In this chapter we explore some of the classes available in the Python standard library.

9.1 Objects Terminology

An object is an instance of a class. We have been using objects since the beginning, but we have not taken advantage of all the capabilities that objects provide. Integers, floating-point numbers, strings, and functions are all objects in Python. With the exception of function objects, we have treated these objects as passive data. We can assign an integer value to a variable and then use that variable's value. We can add two floating-point numbers and concatenate two strings with the + operator. We can pass objects to functions and functions can return objects.

In object-oriented programming, rather than treating data as passive values and functions as active agents that manipulate data, we fuse data and functions together into software units called *objects*. A typical object consists of two parts: *data* and *methods*. An object's data consists of its *instance variables*. The term

instance variable comes from the fact that the data is represented by a *variable* owned by an object, and an object is an *instance* of a class. Other names for instance variables include *attributes* and *fields*. Methods are like functions, and they are known also as *operations*. The instance variables and methods of an object constitute the object's *members*. The code that uses an object is called the object's *client*. We say that an object provides a service to its clients. The services provided by an object can be more elaborate than those provided by simple functions because objects make it easy to store persistent data in their instance variables.

9.2 String Objects

We have been using string objects—instances of class `str`—for some time. Objects bundle data and functions together, and the data that comprise a string consist of the sequence of characters that make up the string. We now turn our attention to string methods. Listing 9.1 (`stringupper.py`) shows how a programmer can use the `upper` method available to string objects.

Listing 9.1: `stringupper.py`

```
name = input("Please enter your name: ")
print(f'Hello {name.upper()}, how are you?')
```

Listing 9.1 (`stringupper.py`) capitalizes (converts to uppercase) all the letters in the string the user enters:

```
Please enter your name: Rick
Hello RICK, how are you?
```

The expression

`name.upper()`

within the `print` statement represents a *method call*. The general form of a method call is

object . *method name* (*parameter list*)

- *object* is an expression that represents an object. In the example in Listing 9.1 (`stringupper.py`), `name` is a reference to a string object.
- The period, pronounced *dot*, associates an object expression with the method to be called.
- *method name* is the name of the method to execute on behalf of the object.
- *parameter list* is comma-separated list of parameters to the method. For some methods the parameter list may be empty, but the parentheses always are required. The parameter list for a method call works exactly like the parameter list for a function call.

Except for the object prefix, a method call works like a function call. It can return a value to its caller.

The `upper` method returns a new string. The `upper` method does not affect the object upon which it is called; this means `name.upper()` does not modify the `name` object.

A method may accept parameters. Listing 9.2 (`rjustprog.py`), uses the `rjust` string method to right justify a string padded with a specified character.

Listing 9.2: `rjustprog.py`

```
word = "ABCD"
print(word.rjust(10, "*"))
print(word.rjust(3, "*"))
print(word.rjust(15, ">"))
print(word.rjust(10))
```

The output of Listing 9.2 (`rjustprog.py`):

```
*****ABCD
ABCD
>>>>>>>>>>ABCD
      ABCD
```

shows

- `word.rjust(10, "*")` right justifies the string `"ABCD"` within a 10-character horizontal area padded with `*` characters.
- `word.rjust(3, "*")` does not return a different string from the original `"ABCD"` since the specified width (3) is less than or equal to the length of the original string (4).
- `word.rjust(10)` shows that the default padding character is a space.

The following interactive sequence shows that we can call a method from a string literal:

```
>>> 'aBcDeFgHiJ'.upper()
'ABCDEFGHij'
>>> 'This is a sentence.'.rjust(25, '-')
'-----This is a sentence.'
```

Table 9.1 lists some of the methods available to `str` objects.

Table 9.1 A few of the methods available to `str` objects

str Methods	
<code>upper</code>	Returns a copy of the original string with all the characters converted to uppercase
<code>lower</code>	Returns a copy of the original string with all the characters converted to lower case
<code>rjust</code>	Returns a string right justified within an area padded with a specified character which defaults to a space
<code>ljust</code>	Returns a string left justified within an area padded with a specified character which defaults to a space
<code>center</code>	Returns a copy of the string centered within an area of a given width and optional fill characters; fill characters default to spaces
<code>strip</code>	Returns a copy of the given string with the leading and trailing whitespace removed; if given an optional string, the <code>strip</code> function strips leading and trailing characters found in the parameter string
<code>startswith</code>	Determines if the string parameter is a prefix of the invoking string
<code>endswith</code>	Determines if the string parameter is a suffix of the invoking string
<code>count</code>	Determines the number times the string parameter is found as a substring within the invoking string; the count includes only non-overlapping occurrences
<code>find</code>	Returns the lowest index where the string parameter is found as a substring of the invoking string; returns <code>-1</code> if the parameter is not a substring of the invoking string

Listing 9.3 (`stripandcount.py`) demonstrates two of the string methods.

Listing 9.3: `stripandcount.py`

```
# Strip leading and trailing whitespace and count substrings
s = '   ABCDEFGHBCDIJKLMNOPQRSBCDTUVWXYZ   '
print(f'[{s}]')
s = s.strip()
print(f'[{s}]')

# Count occurrences of the substring "BCD"
print(s.count("BCD"))
```

Listing 9.3 (`stripandcount.py`) displays:

```
[   ABCDEFGHBCDIJKLMNOPQRSBCDTUVWXYZ   ]
[ABCDEFGHIBCDIJKLMNOPQRSBCDTUVWXYZ]
3
```

The `str` class provides a `__getitem__` method that returns the character at a given position within the string. Since the method's name begins with two underscores (`__`), the method is meant for internal class use and not for clients. The `__getitem__` method is special, as clients can access it via a special syntax:

```
>>> s = 'ABCEFGHI'
>>> s
'ABCEFGHI'
>>> s.__getitem__(0)
'A'
>>> s.__getitem__(1)
'B'
>>> s.__getitem__(2)
'C'
>>> s[0]
'A'
>>> s[1]
'B'
>>> s[2]
'C'
```

When used with an object in the manner shown above, the square bracket syntax invokes that object's `__getitem__` method. In the case of string `s`, the expression `s[0]` represents the character at a distance of zero from the beginning of the string. If `s` is the string `'ABCDEFGHI'`, `s[0]` is the first character: A. The expression `s[1]` is the character in the second position: B. The integer within the square brackets is known as an *index*. The index can be any valid integer expression. For string `s`, `s[0]` is the first character in the string, `s[1]` is the second character, as so forth. If string `s` contains `n` characters, the last character is `s[n - 1]`.

Strings also provide a `__len__` method that returns the number of characters that make up the string. Again, since the name `__len__` begins with two underscores, clients are supposed to invoke it in a different way. The following shows the preferred way of determining a string's length:

```
>>> s
'ABCEFGHI'
>>> s = 'ABCEFGHI'
>>> s
'ABCEFGHI'
>>> len(s)
8
>>> s.__len__()
8
```

The expressions `len(s)` and `s.__len__()` are functionally equivalent. Instead of calling the `__len__` method directly, clients should use the global `len` function. Listing 9.4 (`printcharacters.py`) uses the `len` function and `[]` index operator to print the individual characters that make up a string.

Listing 9.4: printcharacters.py

```
s = "ABCDEFGHIJK"
print(s)
for i in range(len(s)):
    print(f'[{s[i]}]', end='')
print() # Print newline

for ch in s:
    print(f'<{ch}>', end='')
print() # Print newline
```

Listing 9.4 (printcharacters.py) displays the following:

```
ABCDEFGHIJK
[A][B][C][D][E][F][G][H][I][J][K]
<A><B><C><D><E><F><G><H><I><J><K>
```

Strings are immutable objects. This means we cannot modify the contents of a string object:

```
s = 'ABCDEFGHIJKLMN'
s[3] = 'S' # Illegal, strings are immutable
```

String immutability means a method such as `strip` may not change a given string:

```
s = '   ABC   '
s.strip() # Does not change s
print(f'<{s}>') # Prints <   ABC   >, not <ABC>
```

In order to strip the leading and trailing whitespace as far as the string bound to the variable `s` is concerned, we must reassign `s`:

```
s = '   ABC   '
s = s.strip() # Note the reassignment
print(f'<{s}>') # Prints <ABC>
```

The `strip` method returns a new string and does not modify the instance on which it is invoked. To effectively strip the whitespace from a string, a client must, as in this example, reassign its variable to the string passed back by the `strip` method.

When treated as a Boolean expression, the empty string (`''`) is interpreted as `False`, and all other strings are considered `True`.

9.3 File Objects

So far all the programs we have seen lose their data at the end of their execution. Most useful applications, however, require greater data persistence. Imagine using a word processor that does *not* allow you to save your document and retrieve it later for further editing. Most modern operating systems store persistent data in *files*. A word processor, for example, could store one of its documents in a file named `thesis.doc`.

Fortunately, Python's standard library has a file class that makes it easy for programmers to make objects that can store data to, and retrieve data from, disk. The formal name of the class of file objects we will be using is `TextIOWrapper`, and it is found in the `io` module. Since file processing is such a common

activity, the functions and classes defined in the `io` module are available to any program, and no `import` statement is required.

The `open` function in the statement

```
f = open('myfile.txt', 'r')
```

creates and returns a file object (literally a `TextIOWrapper` object) named `f`. The first argument to `open` is the name of the file, and the second argument is a mode. The `open` function supports the following modes:

- `'r'` opens the file for reading
- `'w'` opens the file for writing; creates a new file
- `'a'` opens the file to append data to it

The statement

```
f = open('myfile.txt', 'r')
```

creates a file object named `f` capable of reading the contents of the text file named `myfile.txt`. If the file does not exist or the user of the program does not have adequate permissions to open the file, the `open` function will raise an exception.

The statement

```
f = open('myfile.txt', 'w')
```

creates and returns a file object named `f` capable of writing data to the text file named `myfile.txt`. If the file does not exist, the function creates the file on disk. If a file by that name currently exists, new data will replace the current data stored in the file. This means any pre-existing data in the file will be lost.

The statement

```
f = open('myfile.txt', 'a')
```

creates and returns a file object named `f` capable of writing data to the text file named `myfile.txt`. If the file does not exist, the function creates the file on disk. If a file by that name currently exists, new data will be appended after the pre-existing data in that file. This means that the original data in the file is not lost.

If the second argument to the `open` function is missing, it defaults to `'r'`, so the statement

```
f = open(fname)
```

is equivalent to

```
f = open(fname, 'r')
```

Once you have a file object capable of writing (opened with `'w'` or `'a'`) you can save data to the file associated with that file object using the `write` method. For a file object named `f`, the statement

```
f.write('data')
```

stores the string `'data'` to the file. The three statements

```
f.write('data')
f.write('compute')
f.write('process')
```

writes the text `'datacomputeprocess'` to the file. If our intention is to retrieve the three separate original strings, we must add delimiters to separate the pieces of data. Newline characters serve as good delimiters:

```
f.write('data\n')
f.write('compute\n')
f.write('process\n')
```

This places each word on its own line in the text file. The advantage of storing each piece of data on its own line of text is that it makes it easier to read the data from the file with a `for` statement. If `f` is a file object created for reading, the following code:

```
for line in f:
    print(line.strip())
```

reads in each line of text from the file and prints it out. The variable `line` is a string, and we use the `strip` method to remove the trailing newline (`'\n'`) character.

We also can read the contents of the entire file into a single string using the file object's `read` method:

```
contents = f.read()
```

Given the text file from above, the code

```
in = open('comterms.txt', 'r')
s = in.read()
```

assigns to `s` the string `'data\ncompute\nprocess\n'`.

The `open` method opens a file for reading or writing, and the `read`, `write`, and other such methods enable the program to interact with the file. When the executing program is finished with its file processing, it must call the `close` method to close the file properly. Failure to close a file can have serious consequences when writing to a file, as data meant to be saved could be lost. Every call to the `open` function should have a corresponding call to the file object's `close` method.

Listing 9.5 (`simplefileread.py`) opens a file named `data.dat` for reading and reads in and prints out each line of text:

Listing 9.5: `simplefileread.py`

```
f = open('data.dat')           # f is a file object
for line in f:                 # Read each line as text
    print(line.strip())        # Remove trailing newline character and print a line
f.close()                     # Close the file
```

If the file `data.dat` does not exist or there are issues such as the user does not have sufficient permissions to read the file, the executing program will raise an exception.

Since it is important to always close a file after opening it, Python offers a simpler way to express Listing 9.5 (`simplefileread.py`) that automatically closes the file when finished. Listing 9.6 (`simplerread.py`) uses the `with/as` statement to create what is known as a *context manager* that ensures the file is closed.

Listing 9.6: simplerread.py

```
with open('data.dat') as f:      # f is a file object
    for line in f:               # Read each line as text
        print(line.strip())      # Remove trailing newline character
    # No need to close the file
```

The general form of the `with/as` statement is

`with` *object-creation* `as` *object* :
block

- The reserved word `with` begins the `with/as` statement.
- The expression *object-creation* attempts to create and return an object. If the *object-creation* expression is unsuccessful, the statement's execution does not continue.
- The reserved word `as` binds the object created by the *object-creation* expression to a variable.
- *object* is bound to the object created by the *object-creation* expression.
- *block* contains the code that uses the object bound to *object*.

The `with/as` statement can work with classes like `TextIOWrapper` that provide a particular protocol for initialization and finalization. In the case of our file object, if the call to `open` proceeds without an error, the program will execute the code in the `with/as` block. When the code in the `with/as` has finished executing, the statement executes any finalization actions the class requires. In this case, the finalization code of the `TextIOWrapper` class closes the file associated with file object `f`. Only certain classes support the initialization/finalization protocol in a way that is compatible with the `with/as` statement. Such classes provide a method named `__enter__` that performs the initialization and a method named `__exit__` that performs the finalization.

Listing 9.7 (`numbersaver.py`) allows the user to enter numbers from the keyboard and save them to a file. It also allows the user retrieve the values previously saved to a file. The user can specify the name of the file and, thus, work with multiple files.

Listing 9.7: numbersaver.py

```

"""
Uses Python's file class to store data to and retrieve data from
a text file.
"""
def load_data(filename: str) -> None:
    """ Print the elements stored in the text file named filename. """
    # Open file to read
    with open(filename) as f:    # f is a file object
        for line in f:          # Read each line as text
            print(int(line))     # Convert to integer and append to the list

def store_data(filename: str) -> None:
    """ Allows the user to store data to the text file named filename. """
    with open(filename, 'w') as f:    # f is a file object
        number = 0
        while number != 999:          # Loop until user provides magic number
            number = int(input('Please enter number (999 quits):'))
            if number != 999:
                f.write(str(number) + '\n')
            else:
                break                  # Exit loop

def main() -> None:
    """ Interactive function that allows the user to
    create or consume files of numbers. """
    done = False

    while not done:
        cmd = input('S)ave L)oad Q)uit: ')
        if cmd == 'S' or cmd == 's':
            store_data(input('Enter file name:'))
        elif cmd == 'L' or cmd == 'l':
            load_data(input('Enter filename:'))
        elif cmd == 'Q' or cmd == 'q':
            done = True

if __name__ == '__main__':
    main()

```

The following shows one particular run of Listing 9.7 (numbersaver.py):

```

S)ave L)oad Q)uit: s
Enter file name:numbers.txt
Please enter number (999 quits):10
Please enter number (999 quits):20
Please enter number (999 quits):30
Please enter number (999 quits):40
Please enter number (999 quits):50
Please enter number (999 quits):999
S)ave L)oad Q)uit: q

```

This run creates a file named `numbers.txt`. We can run the program again to retrieve the previously entered values:

```
S)ave L)oad Q)uit: l
Enter filename:numbers.txt
10
20
30
40
50
S)ave L)oad Q)uit: q
```

The literal name of Python's file class is `TextIOWrapper` from the `io` module. The kind of files processed by this file class are known as *text files*. Text files store character data, and we can use a simple editor to create and modify text files. Many applications prevent the easy modification of data files outside of the application by encoding the data in a special way. Depending on the data, the application also may encode the files to save space.

We can combine Python's string objects and file objects to create some powerful file processing programs. In particular we can open one file, read its contents, and write a modified form of its contents to a second file. Listing 9.8 (`convertupper.py`) is a module providing a simple utility function, `capitalize`, that capitalizes the text within a file.

Listing 9.8: `convertupper.py`

```
"""
convertupper.py
"""
def capitalize(filename: str) -> None:
    """ Creates a new file with the prefix 'upper_'
        added to the name of the original file.
        All the alphabetic characters in the new
        are capitalized. This function does not
        disturb the contents of the original file. """
    with open(filename, 'r') as infile:
        with open('upper_' + filename, 'w') as outfile:
            for line in infile:
                line = line.strip().upper()
                print(line, file=outfile)
```

The `capitalize` function creates a new file, named with the `upper_` prefix added to the name of the original file.

Suppose we have a text file named `declaration.txt` containing the first part of the introduction and preamble to the United States Declaration of Independence, as shown here:

```
When in the Course of human events, it becomes necessary for one people
to dissolve the political bands which have connected them with another,
and to assume among the powers of the earth, the separate and equal sta-
tion to which the Laws of Nature and of Nature's God entitle them, a de-
cent respect to the opinions of mankind requires that they should declare
the causes which impel them to the separation.
```

Listing 9.9 (`runconvert.py`) produces a new file named `upper_declaration.txt`.

Listing 9.9: runconvert.py

```
from convertupper import capitalize
capitalize('declaration.txt')
```

The file upper_declaration.txt will contain the following text:

```
WHEN IN THE COURSE OF HUMAN EVENTS, IT BECOMES NECESSARY FOR ONE PEOPLE
TO DISSOLVE THE POLITICAL BANDS WHICH HAVE CONNECTED THEM WITH ANOTHER,
AND TO ASSUME AMONG THE POWERS OF THE EARTH, THE SEPARATE AND EQUAL STA-
TION TO WHICH THE LAWS OF NATURE AND OF NATURE'S GOD ENTITLE THEM, A DE-
CENT RESPECT TO THE OPINIONS OF MANKIND REQUIRES THAT THEY SHOULD DECLARE
THE CAUSES WHICH IMPEL THEM TO THE SEPARATION.
```

Table 9.2 summarizes some of the functions and methods available to file objects.

Table 9.2 A few of the functions and methods available to file objects

TextIOWrapper Methods	
<code>open</code>	A function that returns a file object (instance of <code>io.TextIOWrapper</code>).
<code>read</code>	A method that reads the contents of a text file into a single string.
<code>write</code>	A method that writes a string to a text file.
<code>close</code>	A method that closes the file from further processing. When writing to a file, the <code>close</code> method ensures that all data sent to the file is saved to the file.

Objects usually contain data in addition to methods. `TextIOWrapper` objects store integer, string, and Boolean information. The following interactive session reveals some of the data stored in file objects:

```
>>> f = open('temp.dat', 'w')
>>> f.name
'temp.dat'
>>> f._CHUNK_SIZE
8192
>>> f.mode
'w'
>>> f.encoding
'cp1252'
>>> f.line_buffering
False
```

We say that `name`, `_CHUNK_SIZE`, `encoding`, and `line_buffering` are all instance variables of the object `f`. These are just like the variables we have been using, except that we must prefix their name with their associated object and a dot (`.`). Since these names refer to data, not methods, no parentheses appear at the end. If we have two different file objects, `f` and `g`, `f.name` may be different from `g.name`. The statement

```
x = 2
```

binds the variable `x` to the value 2, whereas the statement

```
obj.x = 2
```

binds the `x` instance variable of object `obj` to the value 2.

9.4 Fraction Objects

The `fractions` module provides the `Fraction` class. `Fraction` objects model mathematical rational numbers; that is, the ratio of two integers. Rational numbers contain a *numerator* and *denominator*. Listing 9.10 (`fractionplay.py`) makes and uses some `Fraction` objects.

Listing 9.10: `fractionplay.py`

```
from fractions import Fraction

f1 = Fraction(3, 4)      # Make the fraction 3/4
print(f1)                # Print it
print(f1.numerator)     # Print numerator
print(f1.denominator)   # Print denominator
print(float(f1))         # Floating-point equivalent
f2 = Fraction(1, 8)      # Make another fraction, 1/8
print(f2)                # Print the second fraction
f3 = f1 + f2             # Add the two fractions
print(f3)                # 3/4 + 1/8 = 6/8 + 1/8 = 7/8
```

Listing 9.10 (`fractionplay.py`) prints

```
3/4
3
4
0.75
1/8
7/8
```

The statement

```
f1 = Fraction(3, 4)
```

creates a `Fraction` object and assigns the variable `f1` to the object. The expression `Fraction(3, 4)` calls a *class constructor*. Class constructors allow clients to supply data used in the formation of a new object. In this case, the first parameter represents the numerator of the new fraction object, and the second parameter represents the denominator of the object. The `Fraction(3, 4)` expression returns a reference to the newly created fraction object, and the statement

```
f1 = Fraction(3, 4)
```

binds the variable `f1` to this object.

We see from Listing 9.10 (`fractionplay.py`) that `Fraction` objects contain `numerator` and `denominator` instance variables. The expression `f1.numerator` represents the `numerator` instance variable of object `f1`.

The program appears to be devoid of any method calls, but the addition statement involves a method call behind the scenes. Python reserves special names for some methods. The `Fraction` class provides a method named `__add__`. This allows clients to add together two fraction objects as shown in Listing 9.10 (`fractionplay.py`):

```
f3 = f1 + f2
```

This is a nicer way of expressing

```
f3 = f1.__add__(f2)
```

We say the former statement using the `+` operator is *syntactic sugar* for latter statement that uses the explicit `__add__` method. Most human readers prefer the version with `+`, but, in reality, both statements are identical to the Python interpreter. The `str` class of string objects also provides an `__add__` method. If `s` and `t` are string objects, the string concatenation expression `s + t` is syntactic sugar for `s.__add__(t)`.

The `Fraction` class includes a number of these special methods that exploit syntactic sugar; examples include the following (`f` and `g` reference `Fraction` objects):

- `__mul__`, multiplication: `f.__mul__(g)` is equivalent to `f * g`
- `__eq__`, relational quality: `f.__eq__(g)` is equivalent to `f == g`
- `__gt__`, greater than: `f.__gt__(g)` is equivalent to `f > g`
- `__sub__`, subtraction: `f.__sub__(g)` is equivalent to `f - g`
- `__neg__`, unary minus: `f.__neg__()` is equivalent to `-f`

Any type that supports these operations must have these special methods. In fact, the familiar `int`, `float`, and `str` types all have these methods, and we merely have been using their syntactic sugar-coatings. Consider the following interactive sequence:

```
>>> (20).__add__(4)
24
>>> 0.25.__mul__(4)
1.0
>>> 'hello'.__add__('there')
'hellothere'
>>> 'hello'.__mul__(3)
'hellohellohello'
```

In the first line of the interactive sequence we put `20` in parentheses so the interpreter interprets the period as a *dot* instead of a numerical *decimal point*. We also could have put a space in front of the dot, as in

```
20 .__add__(4) # Note space before the dot
```

Calling `__add__` instead of using the `+` operator offers no performance advantages and makes the code less readable for humans. You therefore should use the more readable operator syntax in your code. We mention the alternate syntax at this point more as interesting curiosity, but later in Chapter 13 we can take advantage of these special methods when we are designing our own custom types.

9.5 Turtle Graphics Objects

In Section 6.9, we introduced Python's Turtle graphics functional interface. We saw how it is possible to draw pictures within a graphical window via function calls. Behind the scenes the `turtle` module creates a global Turtle object which models the pen doing the drawing. The Turtle graphics functions such as `left` and `pencolor` manipulate this hidden Turtle object. We can create and use our own Turtle objects. This is useful if we wish to manage multiple pens simultaneously.

To illustrate the use of Turtle objects, we will rewrite the programs that appeared in Section 6.9 but use a Turtle object directly. We will call methods associated with our Turtle object instead of using the global functions that operate on the hidden Turtle object.

Listing 9.11 (`boxturtleobject.py`) is the object version of Listing 6.18 (`boxturtle.py`), which draws a rectangular box within a graphical window.

Listing 9.11: `boxturtleobject.py`

```
# Draws a rectangular box in the window

from turtle import Turtle, mainloop

t = Turtle()           # Create a turtle object named t
t.pencolor('red')      # t's pen color is red
t.forward(200)         # Move turtle t forward 200 units (create bottom of rectangle)
t.left(90)             # Turn turtle left by 90 degrees
t.pencolor('blue')     # Change t's pen color to blue
t.forward(150)         # Move turtle t forward 150 units (create right wall)
t.left(90)             # Turn turtle left by 90 degrees
t.pencolor('green')    # Change t's pen color to green
t.forward(200)         # Move turtle t forward 200 units (create top)
t.left(90)             # Turn turtle left by 90 degrees
t.pencolor('black')    # Change t's pen color to black
t.forward(150)         # Move turtle t forward 150 units (create left wall)
t.hideturtle()         # Make turtle t invisible
mainloop()             # Await user input
```

The behavior of Listing 9.11 (`boxturtleobject.py`) is identical to that of Listing 6.18 (`boxturtle.py`) (see Figure 6.6).

Observe that the methods of the `Turtle` class correspond exactly to the global functions we used before, both in name and expected parameters.

Listing 9.12 (`octagonobject.py`) is an enhanced version of Listing 6.19 (`octagon.py`). Since introducing Turtle graphics in Section 6.9 we have introduced writing our own functions. Besides using a `Turtle` object instead of the global Turtle graphics functions, Listing 9.12 (`octagonobject.py`) organizes the code into functions for greater modularity and the potential for reuse.

Listing 9.12: octagonobject.py

```

"""  Draws in the window a spiral surrounded with an octagon  """

import turtle

def octagon(t: turtle.Turtle, x: float, y: float, color: str) -> None:
    """  Draws with turtle t an octagon centered at (x, y)
        with the specified color  """
    t.pencolor(color)    # Set pen color
    t.penup()            # Lift pen to move it
    t.setposition(x, y)  # Move the pen to coordinates (x, y)
    t.pendown()          # Place pen to begin drawing
    for i in range(8):   # Draw the eight sides
        t.forward(80)
        t.right(45)

def spiral(t: turtle.Turtle, x: float, y: float, color: str) -> None:
    """  Draws with turtle t a spiral centered at (x, y)
        with the specified color  """
    distance = 0.2
    angle = 40
    t.pencolor(color)    # Set pen color
    t.penup()            # Lift pen to move it
    t.setposition(x, y)  # Position the pen at coordinates (x, y)
    t.pendown()          # Set pen down to begin drawing
    for i in range(100):
        t.forward(distance)
        t.left(angle)
        distance += 0.5

t = turtle.Turtle()      # Create a turtle object named t
octagon(t, -45, 100, 'red')
spiral(t, 0, 0, 'blue')
t.hideturtle()          # Make turtle t invisible
turtle.done()

```

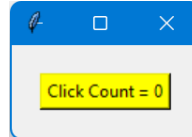
While Listing 9.12 (octagonobject.py) may look significantly different from Listing 6.19 (octagon.py), it produces exactly the same picture with exactly the same algorithms (see Figure 6.8).

9.6 Graphics with tkinter Objects

The `tkinter` module provides classes for building graphical user interfaces via the cross-platform Tk toolkit. Tk is available for the Microsoft Windows, Apple macOS, and Linux operating systems. The `tkinter` module is much larger and more complex than the `turtle` module. In fact, the `turtle` module is built from components the `tkinter` module provides.

Listing 9.13 (buttontester.py) is a simple example that exercises a graphical button. Figure 9.1 shows a picture of the application's window after the user has pressed the button three times.

Figure 9.1 An very simple interactive graphical application using the Tk toolkit. The button keeps track of how many times the user presses it. In this case the user has pressed the button three times.



Listing 9.13: buttontester.py

```
from tkinter import Tk, Button

count = 0    # Records the number of button presses

def update() -> None:
    """ Updates the count label within the graphical button """
    global count, b
    count += 1
    b.config(text='Click Count = ' + str(count))
    print('Updating')

root = Tk()
b = Button(root)
b.configure(background='yellow', text='Click Count = 0', command=update)
b.pack(padx=20, pady=20)
root.mainloop()
```

Listing 9.13 (buttontester.py) uses two classes from the `tkinter` package:

- **Tk:** This class represents a graphical window. The statement

```
root = Tk()
```

creates a `Tk` object named `root`. The `root` object corresponds to the application's main graphical window.

The statement

```
root.mainloop()
```

calls the `mainloop` method on behalf of the `root` window object to start the graphical program. This method starts the process in motion that enables the user to provide input to the application and allows the application to provide visual feedback to the user.

- **Button:** This class represents a graphical button that a user can press. A `Button` is one many graphical user interface widgets the Tk toolkit provides. (Other Tk widgets include checkboxes, text labels, radio buttons, text entry fields, and list boxes, scroll bars, progress bars, and spin boxes.) The statement

```
b = Button(root)
```

creates a `Button` object named `b` that is associated with the `root` window.

The statement

```
b.configure(background="yellow", text="Click Count = 0",
             command=update)
```

configures the graphical button with a yellow background color, sets its displayed text, and assigns a function to be called when the user presses the button. Recall from Section 6.3 that the `print` function can accept up to four optional keyword arguments. We commonly use the `end` and `sep` keyword arguments with `print`. The code here calls the `configure` method of the `Button` class with `background`, `text`, and `command` keyword arguments. These names are only three of 35 different keyword argument names that the `configure` method can accept. A few other characteristics of a `Button` object that the `configure` method enables programmers to adjust include text font and color, horizontal and vertical padding, text justification, and border thickness.

The statement

```
b.pack(padx=20, pady=20)
```

ensures the button will be placed within the root window with 20 pixels of padding (empty space) around it on all sides.

The code in Listing 9.13 (`buttontester.py`) calls the `Button` class constructor with one argument, `root`, the main window of the application. Instead, it could have used the following statement:

```
b = Button(root, background="yellow", text="Click Count = 0",
           command=update)
```

and omitted the call to `configure` entirely. Here we see the class constructor accepting the required parent window parameter (`root`) plus the three keyword arguments.

Note that in the `update` function of Listing 9.13 (`buttontester.py`) we must declare that `count` is a global variable because the function reassigns it. Without the `global` declaration `update` would treat the name `count` as a local variable. Since the code within `update` does not attempt to assign the `b` variable, the name `b` is implicitly global within the function. We technically do not need to declare `b` to be `global` within the `update` function, but it does hurt to do so.

To explore the capabilities of Tk graphics a little more, Listing 9.14 (`tkinterlight.py`) provides a fully functioning graphical interactive program that models a traffic light. The light changes from red to green to yellow to red ..., when the user presses the graphical button labeled *Change*.

Listing 9.14: `tkinterlight.py`

```
from tkinter import Tk, Canvas
from tkinter.ttk import Button, Frame

def do_button_press() -> None:
    """ The window manager will call this function when the user
        presses the graphical button.
        The variables color, canvas, red_lamp, yellow_lamp,
        and green_lamp are global variables that this function
        expects to exist.
        Since we assign only to the color variable, it is the only
        variable that needs explicit global declaration. """
    global color
    if color == 'red':
        color = 'green'
```

```

        canvas.itemconfigure(red_lamp, fill='black')      # Turn red off
        canvas.itemconfigure(green_lamp, fill='green')    # Turn green on
    elif color == 'green':
        color = 'yellow'
        canvas.itemconfigure(green_lamp, fill='black')    # Turn green off
        canvas.itemconfigure(yellow_lamp, fill='yellow')  # Turn yellow on
    elif color == 'yellow':
        color = 'red'
        canvas.itemconfigure(yellow_lamp, fill='black')   # Turn yellow off
        canvas.itemconfigure(red_lamp, fill='red')        # Turn red on

# Create and initialize global variables
color = 'red'      # The light's current color
root = Tk()        # Create the main window
root.title("Traffic Light")

frame = Frame(root) # Create a frame to hold the widgets
frame.pack()        # Make the frame fill the entire window

# Create a drawing surface within the frame
canvas = Canvas(frame, width=150, height=300)

# Set up the visual aspects of the traffic light
# Traffic light frame
canvas.create_rectangle(50, 20, 150, 280, fill='gray')
# Red lamp
red_lamp = canvas.create_oval(70, 40, 130, 100, fill='red')
# Yellow lamp
yellow_lamp = canvas.create_oval(70, 120, 130, 180, fill='black')
# Green lamp
green_lamp = canvas.create_oval(70, 200, 130, 260, fill='black')

# Create a graphical button and ensure it calls the do_button_press
# function when the user presses it
button = Button(frame, text='Change', command=do_button_press)

# Position button in the containing frame's first row, first column,
# and position canvas in the first row, second column (zero origin,
# like list indices).
button.grid(row=0, column=0)
canvas.grid(row=0, column=1)

# Start the GUI event loop
root.mainloop()

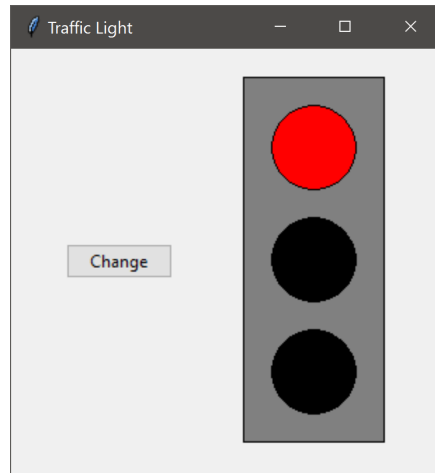
```

Figure 9.2 shows the result of running Listing 9.14 (tkinterlight.py).

Listing 9.14 (tkinterlight.py) uses two classes from the tkinter package and two classes from the tkinter.ttk package:

- Tk: As before, this class represents a graphical window. The statement
`root = Tk()`

Figure 9.2 An interactive graphical traffic light. The user presses the *Change* button to cycle the traffic signals.



creates a Tk object named `root`. The `root` object corresponds to the application's main graphical window. The statement

```
root.title("Traffic Light")
```

sets the text in the window's title bar. The statement

```
root.mainloop()
```

calls the `mainloop` method on behalf of the `root` window object to start the graphical program.

- **Frame:** This class represents an invisible container that can hold other graphical objects. The statement

```
frame = Frame(root)
```

creates a frame object and associates it with the graphical window. The statement

```
frame.pack()
```

makes this container widget fill the whole graphical window.

- **Canvas:** This class represents a drawing area within a graphical window. The statement

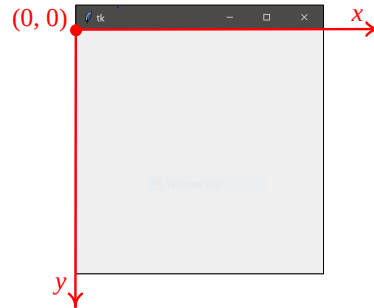
```
canvas = Canvas(frame, width=150, height=300)
```

creates a Canvas object named `canvas` placed in the `frame` widget container. The canvas object's dimensions are 150 pixels wide and 300 pixels tall, as specified by the `width` and `height` keyword arguments. The origin of the coordinate system of the canvas, the point (0,0), is found at the left-top corner of the drawing area within the window, and the `y` axis is inverted, meaning it points down instead of up. This means that while `x` values increase from left to right as expected, `y` increase from top to bottom within the canvas. Figure 9.3 illustrates.

The statement

```
canvas.create_rectangle(50, 20, 150, 280, fill='gray')
```

Figure 9.3 The Tk canvas coordinate system superimposed on a graphics window. x values increase from left to right, and y values increase from top to bottom. The y axis is upside down compared to the Cartesian coordinate system commonly used in mathematics.



invokes the `create_rectangle` method to add a gray rectangle of the given size to the canvas. The first two parameters set the left-top corner of the rectangle to the location (50,20). The third and fourth parameters locate the rectangle's right-bottom corner to be exactly one pixel to the left and one pixel above the point (150,280). The `fill` keyword argument specifies the rectangle's background color. This rectangle represents the traffic light's frame that holds the lamps. The `create_oval` method works similarly for creating the circles representing the lamps of the traffic light. Figure 9.4 shows how the arguments passed to `create_oval` bounds the oval shape within a rectangular area.

The statement

```
canvas.itemconfigure(red_lamp, fill='black')
```

calls the `itemconfigure` method to change the color of the circle named `red_lamp`.

- **Button:** This class represents a graphical button the user can press. Unlike in Listing 9.13 (`buttontester.py`), this class comes from the `tkinter.ttk` package. The visual appearance and functionality differs slightly from `Button` objects from the `tkinter` package. Buttons from the `tkinter.ttk` package have a more modern look and feel.

The statement

```
button = Button(canvas, text='Change', command=do_button_press)
```

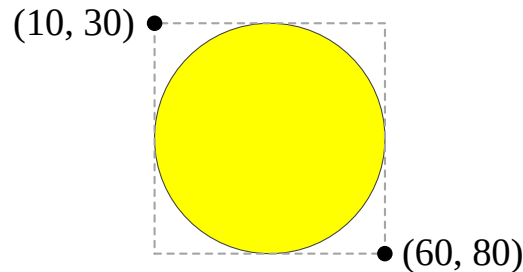
creates a `Button` object named `button` attached to the `canvas` object. The button is labeled *Change*, and it reacts to a user's press by calling the `do_button_press` function. The statements

```
button.grid(row=0, column=0)
canvas.grid(row=0, column=1)
```

impose a one-row, two-column grid widget layout upon the `root` graphical window. The Tk toolkit determines this configuration from the code, and we can too. Note that we add only the `button` and `canvas` widgets to the `root` window. The two calls to the `grid` method specify only one row (both are 0) but two columns (one is 0 and the other is 1). This means the `button` and `canvas` objects will appear side by side (same row 0), and the `button` will be to the left of the `canvas` that draws the traffic light image (0 is the first column, and 1 is the second column).

Figure 9.4 The `create_oval` method of the Tk Canvas class adds an oval shape to a canvas object. The statement in the figure creates a circle shape on the canvas object referenced by `c`. The first two parameters specify the left-top corner of a bounding rectangle, and the third and fourth parameters specify the bounding rectangle's right-bottom corner.

```
c.create_oval(10, 30, 60, 80, fill="yellow")
```



Note that in the `do_button_press` function of Listing 9.14 (`tkinterlight.py`) we must declare that `color` is a global variable because the function reassigns it. Without the `global` declaration, `do_button_press` would treat `color` as a local variable. Since the code within `do_button_press` does not assign the `canvas` variable, `canvas` is implicitly global within the function.

9.7 Other Standard Python Objects

Python provides many other standard classes. Among these, lists, tuples, dictionaries, and sets are particularly useful. These classes are so general, useful, and widely-used that we devote the next few chapters to exploring them in detail.

9.8 Object Mutability and Aliasing

Recall that a variable is a name that labels an object. We informally have treated a variable as the object it represents; for example, given the following statement

```
frac1 = Fraction(1, 2)
```

we often refer to the object `frac1`. In fact, the object is the `Fraction` object representing the rational number $\frac{1}{2}$, and `frac1` is merely a name by which we can access the `Fraction` object. This informality has not been a problem so far, but as we explore objects more deeply we need to be more careful about how we talk about and use objects.

Consider Listing 9.15 (`fractionassign.py`) which uses `Fraction` variables.

Listing 9.15: fractionassign.py

```
from fractions import Fraction

# Assign some Fraction variables
f1 = Fraction(1, 2)
f2 = Fraction(1, 2)
f3 = f1

# Examine the objects involved
print('f1 =', f1)
print('f2 =', f2)
print('f3 =', f3)

# Examine the numerators and denominators separately
print('f1 numerator, denominator:', f1.numerator, f1.denominator)
print('f2 numerator, denominator:', f2.numerator, f2.denominator)
print('f3 numerator, denominator:', f3.numerator, f3.denominator)

# Compare the fractions
print('f1 == f2?', f1 == f2)
print('f1 == f3?', f1 == f3)
print('f1 is f2?', f1 is f2)
print('f1 is f3?', f1 is f3)
```

Listing 9.15 (fractionassign.py) prints

```
f1 = 1/2
f2 = 1/2
f3 = 1/2
f1 numerator, denominator: 1 2
f2 numerator, denominator: 1 2
f3 numerator, denominator: 1 2
f1 == f2? True
f1 == f3? True
f1 is f2? False
f1 is f3? True
```

To better understand the behavior of Listing 9.15 (fractionassign.py) we will examine each of its parts in detail. The statement

```
f1 = Fraction(1, 2)
```

calls the `Fraction` class constructor which creates a new `Fraction` object with its `numerator` instance variable set to 1 and its `denominator` instance variable set to 2. It assigns the variable `f1` to this new fraction object. The statement

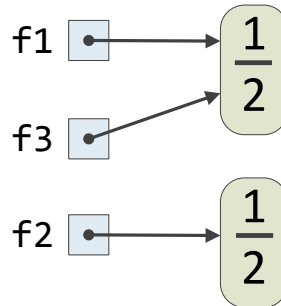
```
f2 = Fraction(1, 2)
```

creates a new `Fraction` object with its `numerator` instance variable set to 1 and its `denominator` instance variable set to 2. It assigns the variable `f2` to this fraction object. The statement

```
f3 = f1
```

assigns the variable `f3` to the same fraction object to which `f1` is assigned. Note that since the statement does not involve the `Fraction` class constructor, it does not create a new fraction object. At this point we

Figure 9.5 An illustration of the relationships among the fraction objects and variables resulting from the assignment statements in Listing 9.15 (fractionassign.py).



have two `Fraction` objects and three variables bound to `Fraction` objects. Figure 9.5 illustrates the relationships among the `Fraction` objects and variables that result from these three assignment statements. The variables `f1` and `f3` refer to the same object. We say that `f1` *aliases* `f3`. Said another way, the variables `f1` and `f3` are aliases. The `Fraction` class implements a method named `__eq__` which enables clients to compare two fraction objects for logical equality using the familiar `==` operator. The statements

```
print('f1 == f2?', f1 == f2)
print('f1 == f3?', f1 == f3)
```

reveal that all three variables refer to objects that are logically equal to each other. The `Fraction.__eq__` method (`==`) compares the `numerator` and `denominator` instance variables of two fraction objects to determine equality.

Sometimes logical equality is not sufficient, and we may need to know if two variables refer to the same object. Python's `is` operator tests to see if two variables refer to the same object. The statements

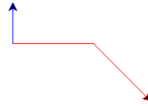
```
print('f1 is f2?', f1 is f2)
print('f1 is f3?', f1 is f3)
```

show that the variables `f1` and `f2` reference two different objects, but `f1` and `f3` refer to the same object. This proves that `f1` and `f3` are aliases. Python also has an `id` function that returns an integer that is unique to a particular object. (For most Python implementations this number is the starting address in memory where the executing program has placed the object.) If `a` and `b` are objects, `a is b` is true exactly when `id(a) == id(b)`.

Prior to this chapter we have restricted our attention to the classes `int`, `float`, `str`, and `bool`. Object aliasing has no practical consequences for programmers restricted to these data types. Instances of these classes are all immutable objects, which means an object of any these of these types cannot change its state after its creation. The integer 3 always is 3, for example, and the string object `'Fred'` cannot change to `'Free'`. Instances of the `Fraction` class are immutable also.

Aliasing can be an issue for mutable objects. In Python's Turtle graphics library (Section 9.5), `Turtle` objects are mutable. Programmers can move a turtle object, change its orientation, and change its pen color. Each of these actions changes the state of the turtle and affects the way the turtle draws within the graphics window.

Listing 9.16 (multiturtle.py) uses two `Turtle` objects.

Figure 9.6 The drawing that results from executing the first part of Listing 9.16 (multiturtle.py).**Figure 9.7** The final drawing rendered by Listing 9.16 (multiturtle.py).**Listing 9.16: multiturtle.py**

```

""" Uses two Turtle objects to draw on the screen """

from turtle import *

# Part 1
t1 = Turtle()           # Create a turtle object named t1
t2 = Turtle()           # Create a second turtle object named t2
t1.pencolor('red')      # t1's pen color is red
t2.pencolor('blue')     # t2's pen color is blue
t2.left(90)             # Point t2 up (t1 automatically points to the right)
t1.forward(100)         # Move turtle t1 forward 100 units
t2.forward(50)          # Move turtle t2 forward 50 units

# Part 2
t2 = t1                 # Make the second turtle just like the first?
t2.right(45)            # Turn turtle 2 (but not turtle 1?)
t2.forward(50)          # Move turtle 2 (why does turtle 1 move instead?)

done()

```

The first part of Listing 9.16 (multiturtle.py) draws the picture shown in Figure 9.6. The beginning of Part 2 of Listing 9.16 (multiturtle.py) reassigns `t2` to `t1`. At this point turtles `t1` and `t2` are aliases, and any further actions taken either through `t1` or `t2` affect only one of the turtles. Figure 9.7 shows the final results of Listing 9.16 (multiturtle.py). The turning and moving of `t2` does not affect the original turtle 2. Instead, changing `t2` affects the `t1`'s turtle, as `t2` and `t1` refer to the same `Turtle` object.

Aliasing of mutable types can be a problem for beginning programmers because it is easy to believe the statement

```
t2 = t1
```

makes `t2` a copy of `t1` and that they remain distinct objects. If the situation arises where your program is managing what you believe to be similar but separate objects and changing the characteristics of one object unexpectedly changes one or more of the other objects in exactly the same way, you likely have an unintended aliasing problem.

9.9 Garbage Collection

The following statement:

```
f = Fraction(2, 3)
```

creates a `Fraction` object representing $\frac{2}{3}$ and assigns the variable `f` to the object. The Python interpreter acts on this statement by reserving enough space in the computer's memory to hold the object. It also performs any initialization that the object requires; in this case it sets `f.numerator` to 2 and `f.denominator` to 3.

What happens to the object associated with `f` when we reassign the variable `f`? The following statement:

```
# f is the Fraction object created above
f = None
```

redirects `f` to refer to the special object `None`, which represents no object. At this point *no* variable references the $\frac{2}{3}$ fraction object created earlier. This means the object effectively is cut off from the remainder of the program's execution or the remainder of the interactive session. This abandoned object is classified as *garbage*. The term *garbage* is a technical term used in computer science that refers to memory allocated by an executing program that the program no longer can access. The Python interpreter cleans up garbage through a process called *garbage collection*. Python uses a reference counting garbage collector that automatically reclaims the space occupied by abandoned objects.

Reference counting garbage collection works as follows. All objects have an associated reference count. When the executing program creates a new object and assigns a variable to it, it sets the object's reference count to 1. The following statement:

```
f = Fraction(2, 3) # The 2/3 object initially has a reference count of 1
```

assigns the variable `f` to a newly created `Fraction` object with a reference count of 1. A reference count of 1 means that exactly one variable (in this case `f`) is assigned to the object. Making an alias, as in

```
g = f # The 2/3 object now has a reference count of 1 + 1 = 2
```

increments the $\frac{2}{3}$ object's reference count by one. If we make another alias, as in

```
h = f # The 2/3 object now has a reference count of 2 + 1 = 3
```

the assignment statement increases the object's reference count to 3. If reassign `f`, `g`, or `h`, as in

```
g = Fraction(9/10) # Reassign a new object to g
```

the reference count of the $\frac{2}{3}$ object decreases by one, and the reference count of the new $\frac{9}{10}$ object becomes 1. If we reassign `f`:

```
f = None
```

this leaves only variable `h` referencing the $\frac{2}{3}$ object, so the object's reference count is 1. If we finally reassign `h`:

`h = 15`

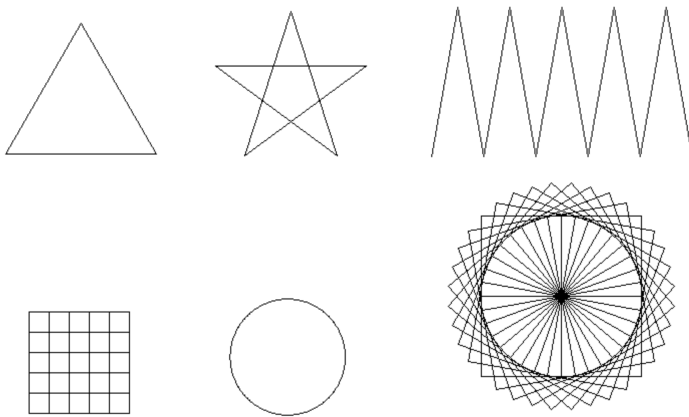
the $\frac{2}{3}$ object's reference count drops to zero. An object with a reference count of zero is garbage, and the garbage collector automatically will reclaim the space held by the object so its memory can be recycled and used elsewhere.

9.10 Exercises

1. What is the difference between a *class* and an *object*?
2. What are some other names for the term *instance variable*?
3. What is another name for the term *method*?
4. What symbol associates an object with a method invocation?
5. How does a method differ from a function?
6. What method from the `string` class returns a new string with no leading or trailing whitespace?
7. What function returns the length of its string argument?
8. What type of object does the `open` function return?
9. What does the second parameter of the `open` function represent?
10. Write a program that stores the first 100 integers to a text file named `numbers.txt`. Each number should appear on a line all by itself.
11. Complete the following function that reads a collection of integers from a text file named `numbers.txt`. Each number in the file appears on a line all by itself. The function accepts a single parameter, a string text file name. The function returns the sum of the integers in the file.

```
def sumfile(filename):  
    # Add your code here . . .
```

12. Provide the syntactic sugar for each of the following methods of the `Fraction` class:
 - (a) `__sub__`
 - (b) `__eq__`
 - (c) `__neg__`
 - (d) `__gt__`
13. How is using a `Turtle` object from Python's Turtle graphics module different from using the free functions; for example, `t.penup()` versus `penup()`?
14. For each of the drawings below write a program that draws the shape using a `Turtle` object from Python's Turtle graphics module.



15. Does Python permit a programmer to change one symbol in a `string` object? If so, how?
16. What would be the consequences if a `turtle.Turtle` object were immutable?
17. In the context of programming, what is *garbage*?
18. What is *garbage collection*, and how does it work in Python?
19. Consider the following code:

```
a = "ABC"  
b = a  
c = b  
a = "XYZ"
```

- (a) At the end of this code's execution what is the reference count for the string object `"ABC"`?
- (b) At the end of this code's execution is `b` an alias of `a`?
- (c) At the end of this code's execution is `b` an alias of `c`?

Chapter 10

Lists

With the exception of tuples, any variable we have used to this point has bound to only one object at a time. As we have seen, we can use individual variables to create some interesting and useful programs; however, variables that can represent only one value at a time do have their limitations. Consider Listing 10.1 (averagenumbers.py) which averages five numbers entered by the user.

Listing 10.1: averagenumbers.py

```
def main() -> None:
    print('Please enter 5 numbers:')
    # Allow the user to enter in the five values.
    n1 = float(input("Please enter number 1: "))
    n2 = float(input("Please enter number 2: "))
    n3 = float(input("Please enter number 3: "))
    n4 = float(input("Please enter number 4: "))
    n5 = float(input("Please enter number 5: "))
    print("Numbers entered:", n1, n2, n3, n4, n5)
    print("Average:", (n1 + n2 + n3 + n4 + n5)/5)

if __name__ == '__main__':
    main()
```

A sample run of Listing 10.1 (averagenumbers.py) looks like:

```
Please enter five numbers:
Please enter number 1: 34.2
Please enter number 2: 10.4
Please enter number 3: 18.0
Please enter number 4: 29.3
Please enter number 5: 15.1
Numbers entered: 34.2 10.4 18.0 29.3 15.1
Average: 21.4
```

The program conveniently displays the values the user entered and then computes and displays their average.

Suppose the number of values to average must increase from five to 25. If we use Listing 10.1 (averagenumbers.py)

as a guide, we would need to introduce twenty additional variables, and the overall length of the program necessarily would grow. Averaging 1,000 numbers using this approach would be impractical.

Listing 10.2 (averagenumbers2.py) provides an alternative approach for averaging numbers that uses a loop.

Listing 10.2: averagenumbers2.py

```
def main() -> None:
    sum = 0.0
    NUMBER_OF_ENTRIES = 5
    print(f'Please enter {NUMBER_OF_ENTRIES} numbers:')
    for i in range(1, NUMBER_OF_ENTRIES + 1):
        num = float(input(f'Enter number {i}: '))
        sum += num
    print("Average:", sum/NUMBER_OF_ENTRIES)

if __name__ == '__main__':
    main()
```

Listing 10.2 (averagenumbers2.py) behaves slightly differently from Listing 10.1 (averagenumbers.py), as the following sample run using the same data shows:

```
Please enter 5 numbers:
Enter number 0: 34.2
Enter number 1: 10.4
Enter number 2: 18.0
Enter number 3: 29.3
Enter number 4: 15.1
Average: 21.4
```

We can modify Listing 10.2 (averagenumbers2.py) to average 25 values much more easily than Listing 10.1 (averagenumbers.py) that must use 25 separate variables—just change the value of `NUMBER_OF_ENTRIES`. In fact, the coding change to average 1,000 numbers is no more difficult. However, unlike the original average program, this new version does not at the end display all the numbers entered. This is a significant difference; it may be necessary to retain all the values entered for various reasons:

- The program can redisplay all the values after the user enters them so the user can visually verify their correctness.
- The program may need to displayed the values in some creative way; for example, they may appear in a graphical user interface component, like a visual grid (spreadsheet).
- The program may need to processed the numbers in a different way after their entry; for example, we may wish to redisplay just the values entered above a certain value (like greater than zero), but the program cannot determine the limit until after all the numbers are available.

In all of these situations we must retain all the values for future recall.

We need to combine the advantages of both of the above programs; specifically we want

- the ability to retain individual values, and

- the ability to dispense with creating separate variables to store all the individual values

These may seem like contradictory requirements, but Python provides a standard data structure that simultaneously provides both of these advantages—the list.

10.1 Using Lists

A list is an object that holds a collection of objects; it represents a sequence of data. In that sense, a list is similar to a string, except a string can hold only characters. A list can hold any Python object. A list need not be homogeneous; that is, the elements of a list do not all have to be of the same type.

Like any other variable, a list variable can be local or global, and it must be defined (assigned) before its use. The following code fragment defines a list named `lst` that holds the integer values 2, −3, 0, 4, −1:

```
lst = [2, -3, 0, 4, -1]
```

The right-hand side of the assignment statement is a literal list. The elements of the list appear within square brackets (`[ ]`), and commas separate the elements. The following statement assigns the empty list to a variable named `a`:

```
a = []
```

We can print list literals and lists referenced through variables:

```
lst = [2, -3, 0, 4, -1]    # Assign the list
print([2, -3, 0, 4, -1])  # Print a literal list
print(lst)                # Print a list via a variable
```

The above code prints

```
[2, -3, 0, 4, -1]
[2, -3, 0, 4, -1]
```

We may access the elements contained in a list via their position within the list. We access individual elements of a list using square brackets with an integer index:

```
lst = [2, -3, 0, 4, -1]    # Assign the list
lst[0] = 5                # Make the first element 5
print(lst[1])             # Print the second element
lst[4] = 12               # Make the last element 12
print(lst)                # Print a list variable
print([10, 20, 30][1])    # Print second element of literal list
```

This code prints

```
-3
[5, -3, 0, 4, 12]
20
```

The number within the square brackets is called the *index*. A nonnegative index indicates the distance from the beginning of the list. The expression `lst[0]` therefore indicates the element at the very beginning (a distance of zero from the beginning) of `lst`, and `lst[1]` is the second element (a distance of one away

from the beginning). We can read aloud the expression `a[3]` as “a sub three,” where the index 3 represents a *subscript*. The subscript terminology is borrowed from mathematicians who use subscripts to reference elements in a mathematical vector or matrix; for example, V_2 represents the second element in vector V . Unlike the convention often used in mathematics, however, the first element in a Python list is at position *zero*, not one. As mentioned above, the index indicates the distance from the beginning; thus, the very first element is at a distance of zero from the beginning of the list. The first element of list `a` is `a[0]`. As a consequence of a zero beginning index, if list `a` holds n elements, the last element in `a` is `a[n - 1]`, not `a[n]`.

If `a` is a list with n elements, and i is an integer such that $0 \leq i < n$, then `a[i]` is an element in the list.

A negative list index represents a negative offset from an imaginary element one past the end of the list. For list `a`, the expression `a[-1]` represents the last element in `a`. The expression `a[-2]` represents the next to the last element, and so forth. If `a` contains n elements, the expression `a[0]` corresponds to `lst[-n]`. Listing 10.3 (`negindex.py`) illustrates the use of negative indices to print a list in reverse.

Listing 10.3: `negindex.py`

```
def main() -> None:
    data = [10, 20, 30, 40, 50, 60]

    # Print the individual elements with negative indices
    print(data[-1])
    print(data[-2])
    print(data[-3])
    print(data[-4])
    print(data[-5])
    print(data[-6])

if __name__ == '__main__':
    main() # Execute main
```

Listing 10.3 (`negindex.py`) prints

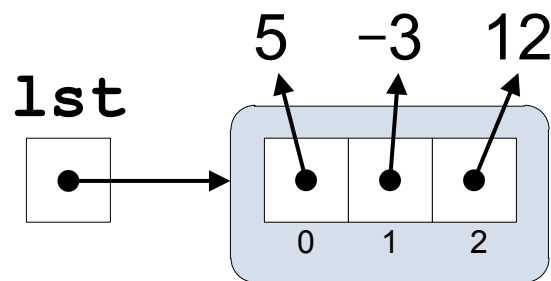
```
60
50
40
30
20
10
```

Figure 10.1 visualizes the list assigned as

```
lst = [5, -3, 12]
```

Listing 10.4 (`heterolist.py`) demonstrates that lists may be heterogeneous; that is, a list can hold elements of varying types.

Figure 10.1 A simple list with three elements. The small number below a list element represents the index of that element.



Listing 10.4: heterolist.py

```
collection = [24.2, 4, 'word', print, 19, -0.03, 'end']
print(collection[0])
print(collection[1])
print(collection[2])
print(collection[3])
print(collection[4])
print(collection[5])
print(collection[6])
print(collection)
```

Listing 10.4 (heterolist.py) prints

```
24.2
4
word
<built-in function print>
19
-0.03
end
[24.2, 4, 'word', <built-in function print>, 19, -0.03, 'end']
```

We clearly see that a single list can hold integers, floating-point numbers, strings, and even functions. A list can hold other lists; the following code

```
col = [23, [9.3, 11.2, 99.0], [23], [], 4, [0, 0]]
print(col)
```

prints

```
[23, [9.3, 11.2, 99.0], [23], [], 4, [0, 0]]
```

Four of the elements of the list `col` are themselves lists.

The following interactive sequence shows how placing variables in a list actually copies the variable's values into the list:

```
>>> x = 5
>>> y = 'ABC'
>>> z = [x, y]
>>> seq = [x, y, z]
>>> seq
[5, 'ABC', [5, 'ABC']]
>>> x = 0
>>> y = 10
>>> seq
[5, 'ABC', [5, 'ABC']]
```

As this sequence demonstrates, changing the variable `x` does not affect the list built from `x`'s original value.

We can treat the elements of a list we access via `[]` as any other variable; for example,

```

nums = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15]
# Print the fourth element
print(nums[3])
# Make the third element the average of two other elements
nums[2] = (nums[0] + nums[9])/2;
# Assign elements at indices 1 and 4 using tuple assignment
nums[1], nums[4] = sqrt(x), x + 2*y

```

The fact that we can write an assignment statement such as

```
lst[4] = 12
```

indicates that lists are *mutable* objects. In contrast, strings are immutable objects, so it is not possible to modify the contents of a string object. We can, however, use the `[]` subscripting operator to access the individual characters (actually strings of length one) that comprise a string; for example,

```

>>> s = 'ABCEFGHI'
>>> s[0]
'A'
>>> s[1]
'B'

```

We know that string objects are immutable, so the following code generates an exception:

```

>>> s = 'ABCEFGHI'
>>> s[0] = 'a'
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: 'str' object does not support item assignment

```

When an expression involving the subscripting operator appears on the left side of the assignment operator, as in

```
lst[0] = 4
```

the interpreter calls the object's `__setitem__` method. An equivalent statement is

```
lst.__setitem__(0, 4)
```

The `str` class has no `__setitem__` method. All other situations that use the `[]` operator within an expression that do not attempt to modify the associated object correspond to the object's `__getitem__` method. For example, the following two statements are equivalent:

```

# The following two statements are equivalent:
x = lst[3]
x = lst.__getitem__(3)

```

String and list objects both have a `__getitem__` method. Programmers, of course, ordinarily should use the syntactically sugared `[]` expressions and not invoke directly an object's `__setitem__` and `__getitem__` methods.

The expression within `[]` must evaluate to an integer; some examples include

- an integer literal: `a[34]`

- an integer variable: `a[x]` (`x` must be an integer)
- an integer arithmetic expression: `a[x + 3]` (`x` must be an integer)
- an integer result of a function call that returns an integer: `a[max(x, y)]` (`max` must return an integer when called with `x` and `y`)
- an element of another or the same list: `a[b[3]]` (element `b[3]` must be an integer)

The following shows an attempt to retrieve an element from the list using a floating-point number:

```
>>> lst = [1, 2, 3, 4, 5]
>>> lst[1]
2
>>> lst[1.2]
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: list indices must be integers or slices, not float
```

10.2 List Traversal

The action of moving through a list visiting each element is known as *traversal*. A list is a kind of iterable object, so we can use a `for` loop to visit each element in order within a list. Listing 10.5 (`heterolistfor.py`) uses a `for` loop and behaves identically to Listing 10.4 (`heterolist.py`).

Listing 10.5: `heterolistfor.py`

```
collection = [24.2, 4, 'word', print, 19, -0.03, 'end']
for item in collection:
    print(item)          # Print each element
```

The built-in function `len` returns the number of elements in a list: The code segment

```
print(len([2, 4, 6, 8]))
a = [10, 20, 30]
print(len(a))
```

prints

```
4
3
```

The name `len` stands for *length*. The index of the last element in list `lst` is `lst[len(lst) - 1]`.

If you have some experience in other programming languages, you may be tempted to use `len` and an explicit list index with a `for` loop as shown in Listing 10.6 (`heterolistforindex.py`) instead of the code in Listing 10.5 (`heterolistfor.py`).

Listing 10.6: `heterolistforindex.py`

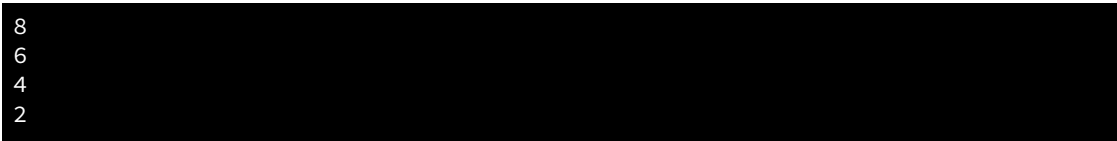
```
collection = [24.2, 4, 'word', print, 19, -0.03, 'end']
for i in range(len(collection)):    # Not the preferred way to traverse a list
    print(collection[i])           # Print each element
```

Listing 10.6 (`heterolistforindex.py`) works identically to Listing 10.5 (`heterolistfor.py`), but Listing 10.5 (`heterolistfor.py`) demonstrates the preferred way to iterate over the elements of a list. Listing 10.5 (`heterolistfor.py`) exploits a list object's iterable property, and it more efficient and elegant than the technique used in Listing 10.6 (`heterolistforindex.py`).

How could we print the elements in a list in reverse order? We could use our familiar `for i in range...` construct and use an explicit index to count backwards:

```
nums = [2, 4, 6, 8]
# Print last element to first (zero index) element
for i in range(len(nums) - 1, -1, -1):
    print(nums[i])
```

This fragment prints



```
8
6
4
2
```

An even better way to iterate over list elements from back to front uses Python's built-in function named `reversed`:

```
nums = [2, 4, 6, 8]
# Print last element to first (zero index) element
for i in reversed(nums):
    print(nums[i])
```

Not only is this version shorter, it actually is more efficient than the version that uses `range` and `len`. The `reversed` expression creates an iterable object that enables the `for` statement to traverse the elements of the list in reverse. The expression `reversed(nums)` does not affect the contents of the list `nums`; it simply enables a backwards traversal of the elements. The `reversed` function returns a generator that works like the following:

```
def my_reversed(lst):
    """ Generate the elements of list lst from back to front.
        Works like the built-in reversed generator function. """
    for i in range(len(lst) - 1, -1, -1):
        yield(lst[i])
```

Within the `range` expression, the first argument, `len(lst) - 1`, is the index of the last element in the list `lst`. The second argument, `-1`, indicates `-1` terminates, but is not included in, the range. The last argument, `-1` indicates the range counts backwards. Taken all together we see that the range spans the indices of all the elements in the list, from the last to the first.

While the `reversed` function behaves exactly like the `my_reversed` generator shown here, its implementation is much more efficient than the code in the `my_reversed`.

The `reversed` function is found in the `__builtins__` module, so its use requires no special `import` statement.

10.3 Building Lists

Python supports several other ways of building a list besides enumerating all the list's elements in a list literal. We can construct a new list from two existing lists using concatenation. The plus (+) operator concatenates lists in the same way it concatenates strings. The following shows some experiments in the interactive shell with list concatenation:

```
>>> a = [2, 4, 6, 8]
>>> a
[2, 4, 6, 8]
>>> a + [1, 3, 5]
[2, 4, 6, 8, 1, 3, 5]
>>> a
[2, 4, 6, 8]
>>> a = a + [1, 3, 5]
>>> a
[2, 4, 6, 8, 1, 3, 5]
>>> a += [10]
>>> a
[2, 4, 6, 8, 1, 3, 5, 10]
>>> a += 20
Traceback (most recent call last):
  File "<pyshell#14>", line 1, in <module>
    a += 20
TypeError: 'int' object is not iterable
```

The statement

```
a = [2, 4, 6, 8]
```

assigns the given list literal to the variable `a`. The expression

```
a + [1, 3, 5]
```

evaluates to the list `[2, 4, 6, 8, 1, 3, 5]`, but the statement does not change the list to which `a` refers. The statement

```
a = a + [1, 3, 5]
```

actually reassigns `a` to the new list `[2, 4, 6, 8, 1, 3, 5]`. The statement

```
a += [10]
```

updates `a` to be the new list `[2, 4, 6, 8, 1, 3, 5, 10]`. Observe that the `+` will concatenate two lists, but it cannot join a list and something that is not a list. The following statement

```
a += 20
```

is illegal since `a` refers to a list, and `20` is an integer, not a list. If used within a program under these conditions, this statement will produce a run-time exception.

If we wish to append a variable's value to a list, we similarly must first enclose it within square brackets:

```
>>> x = 2
>>> a = [0, 1]
>>> a += [x]
>>> a
[0, 1, 2]
```

Listing 10.7 (builduserlist.py) shows how to build lists as the program executes.

Listing 10.7: builduserlist.py

```
# Build a custom list of nonnegative integers specified by the user

def make_list() -> list[int]:
    """
    Builds a list from input provided by the user.
    """
    result = []      # List to return is initially empty
    in_val = 0       # Ensure loop is entered at least once
    while in_val >= 0:
        in_val = int(input("Enter integer (-1 quits): "))
        if in_val >= 0:
            result += [in_val]    # Add item to list
    return result

def main() -> None:
    col = make_list()
    print(col)

if __name__ == '__main__':
    main()
```

A sample run of Listing 10.7 (builduserlist.py) produces

```
Enter integer (-1 quits): 23
Enter integer (-1 quits): 100
Enter integer (-1 quits): 44
Enter integer (-1 quits): 19
Enter integer (-1 quits): 19
Enter integer (-1 quits): 101
Enter integer (-1 quits): 98
Enter integer (-1 quits): -1
[23, 100, 44, 19, 19, 101, 98]
```

If the user enters a negative number initially, we get:

```
Enter integer (-1 quits): -1
[]
```

Listing 10.7 (builduserlist.py) reveals the type hint for a list that contains integer values: `list[int]`. Lists ordinarily may contain elements of mixed types. The type hint `list` (notice the missing square brackets) indicates a list with no restrictions on its content.

There are several ways to build a list without explicitly listing every element in the list. We can use `range` to produce a regular sequence of integers. The `range` object returned by `range` is not itself a list, but we can make a list from a range using the `list` function, as Listing 10.8 (`makeintegerlists.py`) demonstrates.

Listing 10.8: `makeintegerlists.py`

```
def main() -> None:
    a = list(range(0, 10))
    print(a)
    a = list(range(10, -1, -1))
    print(a)
    a = list(range(0, 100, 10))
    print(a)
    a = list(range(-5, 6))
    print(a)

main()
```

Listing 10.8 (`makeintegerlists.py`) prints

```
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
[10, 9, 8, 7, 6, 5, 4, 3, 2, 1, 0]
[0, 10, 20, 30, 40, 50, 60, 70, 80, 90]
[-5, -4, -3, -2, -1, 0, 1, 2, 3, 4, 5]
```

We can use the `list` conversion function to make a list out of any generator (see Section 8.9 for information about generators). Listing 10.9 (`generator2list.py`) uses the `prime_sequence` generator we saw in Listing 8.27 (`generatedprimes.py`) to make a list containing all the prime number in the range 20 to 50.

Listing 10.9: `generator2list.py`

```
from typing import Generator
from math import sqrt

def is_prime(n: int) -> bool:
    """ Returns True if nonnegative integer n is prime;
        otherwise, returns false """
    if n == 2:                                # 2 is the only even prime number
        return True
    if n < 2 or n % 2 == 0:                   # Handle simple cases immediately
        return False                         # No evens and nothing less than 2
    trial_factor = 3
    root = round(sqrt(n))
    while trial_factor <= root:
        if n % trial_factor == 0:             # Is trial factor a factor?
            return False                    # Yes, return right away
        trial_factor += 2                   # Next potential factor, skip evens
    return True                             # Tried them all, must be prime

def prime_sequence(begin: int, end: int) -> Generator[int, None, None]:
    """ Generates the sequence of prime numbers between begin and end. """
```

```

    for value in range(begin, end + 1):
        if is_prime(value):           # See if value is prime
            yield value               # Produce the prime number

def main():
    """ Make a list from a generator """
    # Build the list of prime numbers in the range 20 to 50
    primes = list(prime_sequence(20, 50))
    print(primes)

if __name__ == '__main__':
    main()    # Run the program

```

Listing 10.9 (generator2list.py) displays the list built from our custom prime number generator:

```
[23, 29, 31, 37, 41, 43, 47]
```

Note the type hint for the generator in this case is `Generator[int, None, None]`. The first type in the type hint indicates the generator will produce integers; the final two types in the type hint can be `None` for the simple generator objects we use. In order to use the generator type hint, we must import it, as shown at the top of Listing 10.9 (generator2list.py).

It is easy to make a list in which all the elements are the same or a pattern of elements repeat. The `*` operator, when applied to a list and an integer, “multiplies” the elements of a list. The expression

```
[0] * 5
```

produces the list `[0, 0, 0, 0, 0]`. The integer multiplier may be any valid integer expression. Listing 10.10 (makeuniformlists.py) builds several lists using the `*` list multiplication operator.

Listing 10.10: makeuniformlists.py

```

def main() -> None:
    a = [0] * 10
    print(a)

    a = [3.4] * 5
    print(a)

    a = 3 * ['ABC']
    print(a)

    a = 4 * [10, 20, 30]
    print(a)

    n = 3          # Use a variable multiplier
    a = n * ['abc', 22, 8.7]
    print(a)

main()

```

The output of Listing 10.10 (makeuniformlists.py) is

```
[0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
[3.4, 3.4, 3.4, 3.4, 3.4]
['ABC', 'ABC', 'ABC']
[10, 20, 30, 10, 20, 30, 10, 20, 30, 10, 20, 30]
['abc', 22, 8.7, 'abc', 22, 8.7, 'abc', 22, 8.7]
```

Observe that the integer multiplier may appear either to left or the right of the `*` operator, and the effects are the same. This means the list multiplication `*` operator is *commutative*.

The `*` multiplier operator works similarly with strings. Consider the following interactive sequence:

```
>>> 'abc' * 3
'abccabccabc'
```

List multipliers create aliases of the copied elements. This is not a problem for immutable objects, such as integers, floating-point numbers, and strings, as the following interactive session shows:

```
>>> a = 3 * [5]
>>> a
[5, 5, 5]
>>> a[1] = 7
>>> a
[5, 7, 5]
```

Consider, however, creating a list of lists:

```
>>> a = 3 * [[1, 2, 3]]
>>> a
[[1, 2, 3], [1, 2, 3], [1, 2, 3]]
>>> a[0]
[1, 2, 3]
>>> a[0][0] = 9
>>> a
[[9, 2, 3], [9, 2, 3], [9, 2, 3]]
```

In the original list, `[[1, 2, 3], [1, 2, 3], [1, 2, 3]]`, each of the `[1, 2, 3]` elements are aliases of the same list. Changing an element in one of those lists changes the same element in all the lists.

Strings are immutable, so such an attempt is not possible:

```
>>> b = 3 * ['ABC']
>>> b
['ABC', 'ABC', 'ABC']
>>> b[0][0] = 'x'
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: 'str' object does not support item assignment
```

We can, of course, replacing one of the strings with a completely different string object:

```
>>> b[1] = 'xyz'
>>> b
['ABC', 'xyz', 'ABC']
```

In a similar manner, we could replace one of our aliased inner list objects with a different list:

```
>>> a = 3 * [[1, 2, 3]]
>>> a
[[1, 2, 3], [1, 2, 3], [1, 2, 3]]
>>> a[1] = [7, 8, 9]
>>> a
[[1, 2, 3], [7, 8, 9], [1, 2, 3]]
```

In this case, the first and last element alias the same list object, but the middle element is a different list object.

The list multiplier constructor is useful, but it is not as helpful for creating lists of lists or lists of other mutable types.

We now have all the tools we need to build a program that flexibly averages numbers while retaining all the values the user enters. Listing 10.11 (`listaverage.py`) uses a list and a loop to achieve the generality of Listing 10.2 (`averagenumbers2.py`) with the ability to retain all input for later redisplay.

Listing 10.11: `listaverage.py`

```
def main() -> None:
    # Set up variables
    sum = 0.0
    NUMBER_OF_ENTRIES = 5
    numbers = []    # Start with empty list

    # Get input from user
    print(f'Please enter {NUMBER_OF_ENTRIES} numbers:')
    for i in range(1, NUMBER_OF_ENTRIES + 1):
        num = float(input(f'Enter number {i}: '))
        numbers += [num]
        sum += num

    # Print the numbers entered
    print(end="Numbers entered: ")
    for num in numbers:
        print(num, end=" ")
    print()    # Print newline

    # Print average
    print("Average:", sum/NUMBER_OF_ENTRIES)

if __name__ == '__main__':
    main()    # Execute main
```

The output of Listing 10.11 (`listaverage.py`) is similar to the original Listing 10.1 (`averagenumbers.py`) program:

```
Please enter 5 numbers:
Enter number 0: 9.0
Enter number 1: 3.5
Enter number 2: 0.2
Enter number 3: 100.0
Enter number 4: 15.3
Numbers entered: 9.0 3.5 0.2 100.0 15.3
Average: 25.6
```

Unlike the original program, however, we now conveniently can extend this program to handle as many values as we wish. We need only change the definition of the `NUMBER_OF_ENTRIES` variable to allow the program to handle any number of values. This centralization of the definition of the list's size eliminates duplicating a literal numeric value and leads to a program that is more maintainable. Suppose every occurrence of `NUMBER_OF_ENTRIES` were replaced with the literal value 5. The program would work exactly the same way, but changing the size would require touching many places within the program. When duplicate information is scattered throughout a program, it is a common mistake to update some but not all of the information when a change is to be made. If all of the duplicate information is not updated to agree, the inconsistencies result in logic errors within the program. By faithfully using the `NUMBER_OF_ENTRIES` variable throughout the program instead of the literal numeric value, we can avoid the problems of these potential inconsistencies.

The first loop in Listing 10.11 (`listaverage.py`) collects all five input values from the user. The second loop prints all the numbers the user entered.

When treated as a Boolean expression, the empty list (`[]`) is interpreted as `False`, and all other lists are considered `True`. This means `bool([])` evaluates to `True`, since `[]` is not empty; it contains one element—the empty list.

10.4 List Membership

We can use the Python `in` operator to determine if an object is an element in a list. If `lst` is a list, the expression `x in lst` evaluates to `True` if `x` is an element in `lst`; otherwise, the expression is `False`. Similarly, the expression `x not in lst` evaluates to `True` if `x` is not an element in `lst`; otherwise, the expression is `False`. The expression `x not in lst` is equivalent to `not(x in lst)`. Listing 10.12 (`listmembership.py`) exercises Python's `in` operator.

Listing 10.12: `listmembership.py`

```
lst = list(range(0, 21, 2))
for i in range(-2, 23):
    if i in lst:
        print(i, 'is a member of', lst)
    if i not in lst:
        print(i, 'is NOT a member of', lst)
```

Listing 10.12 (listmembership.py) prints

```
-2 is NOT a member of [0, 2, 4, 6, 8, 10, 12, 14, 16, 18, 20]
-1 is NOT a member of [0, 2, 4, 6, 8, 10, 12, 14, 16, 18, 20]
0 is a member of [0, 2, 4, 6, 8, 10, 12, 14, 16, 18, 20]
1 is NOT a member of [0, 2, 4, 6, 8, 10, 12, 14, 16, 18, 20]
2 is a member of [0, 2, 4, 6, 8, 10, 12, 14, 16, 18, 20]
3 is NOT a member of [0, 2, 4, 6, 8, 10, 12, 14, 16, 18, 20]
4 is a member of [0, 2, 4, 6, 8, 10, 12, 14, 16, 18, 20]
5 is NOT a member of [0, 2, 4, 6, 8, 10, 12, 14, 16, 18, 20]
6 is a member of [0, 2, 4, 6, 8, 10, 12, 14, 16, 18, 20]
7 is NOT a member of [0, 2, 4, 6, 8, 10, 12, 14, 16, 18, 20]
8 is a member of [0, 2, 4, 6, 8, 10, 12, 14, 16, 18, 20]
9 is NOT a member of [0, 2, 4, 6, 8, 10, 12, 14, 16, 18, 20]
10 is a member of [0, 2, 4, 6, 8, 10, 12, 14, 16, 18, 20]
11 is NOT a member of [0, 2, 4, 6, 8, 10, 12, 14, 16, 18, 20]
12 is a member of [0, 2, 4, 6, 8, 10, 12, 14, 16, 18, 20]
13 is NOT a member of [0, 2, 4, 6, 8, 10, 12, 14, 16, 18, 20]
14 is a member of [0, 2, 4, 6, 8, 10, 12, 14, 16, 18, 20]
15 is NOT a member of [0, 2, 4, 6, 8, 10, 12, 14, 16, 18, 20]
16 is a member of [0, 2, 4, 6, 8, 10, 12, 14, 16, 18, 20]
17 is NOT a member of [0, 2, 4, 6, 8, 10, 12, 14, 16, 18, 20]
18 is a member of [0, 2, 4, 6, 8, 10, 12, 14, 16, 18, 20]
19 is NOT a member of [0, 2, 4, 6, 8, 10, 12, 14, 16, 18, 20]
20 is a member of [0, 2, 4, 6, 8, 10, 12, 14, 16, 18, 20]
21 is NOT a member of [0, 2, 4, 6, 8, 10, 12, 14, 16, 18, 20]
22 is NOT a member of [0, 2, 4, 6, 8, 10, 12, 14, 16, 18, 20]
```

Note that the `in` operator produces a Boolean result; it can reveal whether or not an object is a member of a list. It cannot reveal the location (index) of the object it finds. We consider options for locating elements in Section 15.3.

10.5 Lists and Functions

We can pass a list as an argument to a function, as shown in Listing 10.13 (listfunc.py)

Listing 10.13: `listfunc.py`

```
# NumberList is a type hint for a list that holds numbers:
# mixed ints/floats, all ints, or all floats
NumberList = list[int | float] | list[int] | list[float]

def sum(lst: NumberList):
    """
    Adds up the contents of a list of numeric values.
    lst is the list to sum.
    Returns the sum of all the elements or zero if the list is empty.
    """
    result = 0
    for item in lst:
        result += item
    return result
```



```
def make_zero(lst: NumberList) -> None:
    """
    Makes every element in list lst zero
    """
    for i in range(len(lst)):
        lst[i] = 0

def random_list(n: int) -> list[int]:
    """
    Builds a list of n integers, where each integer
    is a pseudorandom number in the range 0...99.
    Returns the random list.
    """
    import random
    result = []
    for i in range(n):
        rand = random.randrange(100)
        result += [rand]
    return result

def main() -> None:
    my_list = [2, 4, 6, 8]
    print(f'{my_list} sums to {sum(my_list)}')

    # Zero out all the elements of list
    make_zero(my_list)
    print(f'{my_list} sums to {sum(my_list)}')

    # Test empty list
    my_list = []
    print(f'{my_list} sums to {sum(my_list)}')

    # Test pseudorandom list with 10 elements
    my_list = random_list(10)
    print(f'{my_list} sums to {sum(my_list)}')

    # Test list with floats
    my_list = [0.5, 1.2, 2.2]
    print(f'{my_list} sums to {sum(my_list)}')

    # Test list with mixed ints and floats
    my_list = [0.5, 2, 2.2]
    print(f'{my_list} sums to {sum(my_list)}')

    # # Test list with strings
    # my_list = ['ABC', 'DEF', 'GHI']
    # print(f'{my_list} sums to {sum(my_list)}')

if __name__ == '__main__':
    main()
```

In Listing 10.13 (`listfunc.py`) the functions `sum` and `make_zero` accept a parameter of type `list`. The type hint

```
list[int | float] | list[int] | list[float]
```

means the functions will accept a list of mixed integers and floating-point numbers, a list of all integers, or a list of all floating-point numbers. The first line creates a new type hint name by combining other type names with the `|` symbol. We can read the `|` symbol as “or.” The `sum` function will accept a list of numbers, but it will not accept a list of string objects. Section 7.2 addressed the consequences of passing immutable types like integers and strings to functions. Since list objects are mutable, passing to a function a reference to a list object binds the formal parameter to the list object. This means the formal parameter becomes an alias of the actual parameter. The `sum` method does not attempt to modify its parameter, but the `make_zero` method changes every element in the list to zero. This means the `make_zero` function will modify the `my_list` list object in `main`.

The output of Listing 10.13 (`listfunc.py`):

```
[2, 4, 6, 8] sums to 20
[0, 0, 0, 0] sums to 0
[] sums to 0
[94, 30, 22, 60, 39, 80, 96, 55, 88, 31] sums to 595
[0.5, 1.2, 2.2] sums to 3.9000000000000004
[0.5, 2, 2.2] sums to 4.7
```

demonstrates that the `make_zero` function indeed changes the `main` method’s `my_list` object.

10.6 List Assignment and Equivalence

Given the assignment

```
lst = [2, 4, 6, 8]
```

the expression `lst` is very different from the expression `lst[2]`. The expression `lst` is a reference to the list, while `lst[2]` is a reference to a particular element in the list, in this case the integer 6. The integer 6 is immutable (see Section 7.2); a literal integer cannot change to be another value. Six is always six. A variable, of course, can change its value and its type through assignment. Variable assignment changes the object to which the variable is bound. Recall Figures 2.2–2.6, and consider the Listing 10.14 (`listassignment.py`).

Listing 10.14: `listassignment.py`

```
a = [10, 20, 30, 40]
b = [10, 20, 30, 40]
print('a =', a)
print('b =', b)
b[2] = 35
print('a =', a)
print('b =', b)
```

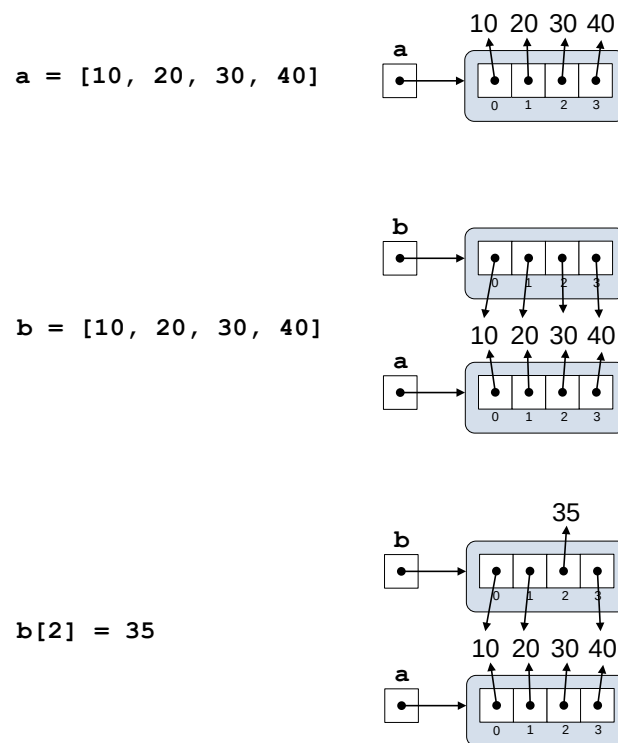
Figure 10.2 State of Listing 10.14 (listassignment.py) as the assignment statements execute

Figure 10.2 shows the consequences of each of the assignment statements in Listing 10.14 (`listassignment.py`),

As Figure 10.2 illustrates, variables `a` and `b` refer to two different list objects; however, the elements of both lists bind to the same (immutable) values. Reassigning an element of list `b` does not affect list `a`. The output of Listing 10.14 (`listassignment.py`) verifies this analysis:

```
a = [10, 20, 30, 40]
b = [10, 20, 30, 40]
a = [10, 20, 30, 40]
b = [10, 20, 35, 40]
```

Now consider Listing 10.15 (`listalias.py`), a subtle variation of Listing 10.14 (`listassignment.py`). At first glance, the code in Listing 10.15 (`listalias.py`) looks like it may behave exactly like Listing 10.14 (`listassignment.py`).

Listing 10.15: `listalias.py`

```
a = [10, 20, 30, 40]
b = a
print('a =', a)
print('b =', b)
b[2] = 35
print('a =', a)
print('b =', b)
```

As Figure 10.3 illustrates, the second assignment statement causes variables `a` and `b` to refer to the *same* list object. We say that `a` and `b` are *aliases*. Reassigning `b[2]` changes `a[2]` as well, as Listing 10.15 (`listalias.py`)’s output shows:

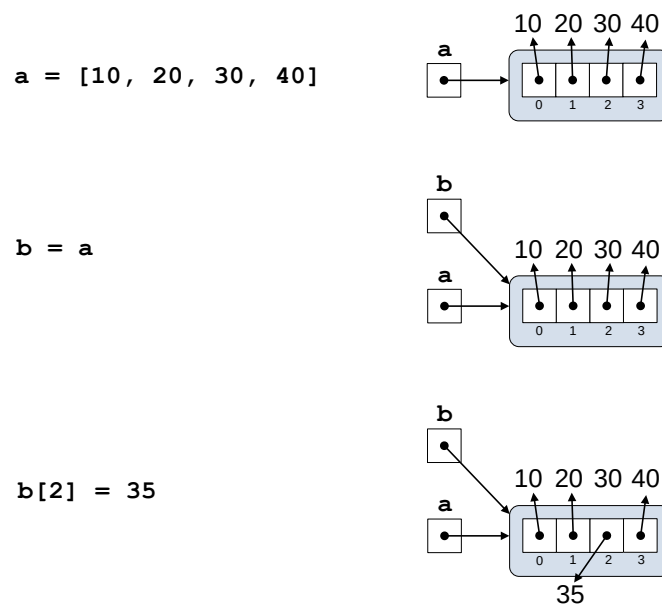
```
a = [10, 20, 30, 40]
b = [10, 20, 30, 40]
a = [10, 20, 35, 40]
b = [10, 20, 35, 40]
```

If `a` refers to a list, the statement

```
b = a
```

does not make a copy of `a`’s list. Instead it makes `a` and `b` aliases to the same list. Lists are *mutable* data structures. We may reassign individual list elements via `[]`. If we bind more than one variable to the same list, any element modification through one of the variables will affect the list from the point of view of all the aliased variables.

The familiar `==` equality operator determines if two lists contain the same elements. The `is` operator determines if two variables alias the same list. Listing 10.16 (`listequivalence.py`) demonstrates the difference between the two operators.

Figure 10.3 State of Listing 10.15 (listalias.py) as the assignment statements execute

Listing 10.16: `listequivalence.py`

```

# a and b are distinct lists that contain the same elements
a = [10, 20, 30, 40]
b = [10, 20, 30, 40]
print(f'Is {a} equal to {b}? {a == b}')
print(f'Are {a} and {b} aliases? {a is b}')

print()

# c and d alias are distinct lists that contain the same elements
c = [100, 200, 300, 400]
d = c      # Makes d an alias of c
print(f'Is {c} equal to {d}? {c == d}')
print(f'Are {c} and {d} aliases? {c is d}')

```

Listing 10.16 (`listequivalence.py`) prints

```

Is [10, 20, 30, 40] equal to [10, 20, 30, 40]? True
Are [10, 20, 30, 40] and [10, 20, 30, 40] aliases? False

Is [100, 200, 300, 400] equal to [100, 200, 300, 400]? True
Are [100, 200, 300, 400] and [100, 200, 300, 400] aliases? True

```

When comparing lists `lst1` and `lst2`, if the expression `lst1 is lst2` evaluates to `True`, the expression `lst1 == lst2` is guaranteed to be `True`.

What if we wish to make a copy of an existing list? Listing 10.17 (`listcopy.py`) shows one way to accomplish this.

Listing 10.17: `listcopy.py`

```

def list_copy(lst: list) -> list:
    """ Makes a copy of a list. """
    result = []
    for item in lst:
        result += [item]
    return result

def main() -> None:
    # a and b are distinct lists that contain the same elements
    a = [10, 20, 30, 40]
    b = list_copy(a)      # Make a copy of a
    print(f'a = {a}    b = {b}')
    print(f'Is {a} equal to {b}? {a == b}')
    print(f'Are {a} and {b} aliases? {a is b}')
    b[2] = 35             # Change an element of b
    print(f'a = {a}    b = {b}')

if __name__ == '__main__':
    main()

```

The output of Listing 10.17 (`listcopy.py`) reveals:

```
a = [10, 20, 30, 40]    b = [10, 20, 30, 40]
Is [10, 20, 30, 40] equal to [10, 20, 30, 40]? True
Are [10, 20, 30, 40] and [10, 20, 30, 40] aliases? False
a = [10, 20, 30, 40]    b = [10, 20, 35, 40]
```

The `list_copy` function in Listing 10.17 (`listcopy.py`) makes an actual copy of `a`. Changing an element of `b` does not affect list `a`. Note that `list` is the type hint for a list that can hold any type of elements.

In Section 10.8 we will see a more effective way to copy a list.

We can use `range` to create a range of values that the `for` statement can consume, but this `range` object is not a list. The following interactive sequence shows how we can use the `list` function to make a list out of a `range` object:

```
>>> r = range(10)
>>> r
range(0, 10)
>>> type(r)
<class 'range'>
>>> list(r)
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
>>> lst = list(r)
>>> lst
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
>>> type(lst)
<class 'list'>
```

The expression `list(range(5))`, therefore, creates the list `[0, 1, 2, 3, 4]`.

The flexibility of the `range` expression makes it easy to create a variety of different kinds of lists with regular structure. Listing 10.18 (`rangetolist.py`) explores several possibilities.

Listing 10.18: `rangetolist.py`

```
print(list(range(11)))
print(list(range(10, 101, 10)))
print(list(range(10, -1, -1)))
```

Listing 10.18 (`rangetolist.py`) produces

```
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
[10, 20, 30, 40, 50, 60, 70, 80, 90, 100]
[10, 9, 8, 7, 6, 5, 4, 3, 2, 1, 0]
```

10.7 List Bounds

In the following code fragment:

```
a = [10, 20, 30, 40]
```

All of the following expressions are valid: `a[0]`, `a[1]`, `a[2]` and `a[3]`. The expression `a[4]` does not represent a valid element in the list. An attempt to use this expression, as in

```
a = [10, 20, 30, 40]
print(a[4])      # Out-of-bounds access
```

results in a run-time exception. The interpreter will insist that the programmer use an integral value for an index, but in order to prevent a run-time exception the programmer must ensure that the index used is within the bounds of the list. Consider the following code:

```
# Make a list containing 100 zeros
v = [0] * 100
# User enters x at run time
x = int(input("Enter an integer: "))
v[x] = 1      # Is this OK? What is x?
```

Since a list index may consist of an arbitrary integer expression, the interpreter checks every attempt to access a list. If the interpreter detects an out-of-bounds index, the interpreter raises an `IndexError` (list index out-of-bounds) exception. The programmer must ensure the provided index is in bounds to prevent such a run-time error.

We can help the above unreliable code with conditional access:

```
# Make a list containing 100 zeros
v = [0] * 100
# User enters x at run time
x = int(input("Enter an integer: "))
# Ensure index is within list bounds
if 0 <= x < len(v):
    v[x] = 1      # This should be fine
else:
    print("Value provided is out of range")
```

Listing 10.19 (`badreverse.py`) attempts to print the list's elements in reverse order, but it fails to stay inside the bounds of the list.

Listing 10.19: `badreverse.py`

```
def make_list() -> list:
    """ Builds a reversed list from input provided by the user. """
    result = []      # List to return is initially empty
    in_val = 0       # Ensure loop is entered at least once
    while in_val >= 0:
        in_val = int(input('Enter integer (-1 quits): '))
        if in_val >= 0:
            result = result + [in_val]      # Add item to list
    return result

def main() -> None:
    col = make_list()
    # Print the list in reverse
    for i in range(len(col), 0, -1):
        print(col[i], end=' ')
    print()

if __name__ == '__main__':
    main()
```


The `for` statement

```
for i in range(len(col), 0, -1):
    print(col[i], end=' ')
```

considers first the element at `col[len(col)]`, which is one index past the end of the list. The corrected `for` statement is

```
for i in range(len(col) - 1, -1, -1):
    print(col[i], end=' ')
```

10.8 Slicing

We can make a new list from a portion of an existing list using a technique known as *slicing*. A list slice is an expression of the form

list [*begin* : *end* : *step*]

where

- *list* is a list—a variable referring to a list object, a literal list, or some other expression that evaluates to a list,
- *begin* is an integer representing the starting index of a subsequence of the list,
- *end* is an integer that is one larger than the index of the last element in a subsequence of the list, and
- *step* is an integer that specifies the stride size through the list. A step size of three, for example, would include every third element in the list within the specified range. Negative step values reverse the direction of the slice.

If missing, the *begin* value defaults to 0. A *begin* value less than zero is treated as zero. If the *end* value is missing, it defaults to the length of the list. An *end* value greater than the length of the list is treated as the length of the list. The default *step* value is 1. The examples provided in Listing 10.20 (`listslice.py`) illustrate how list slicing works.

Listing 10.20: `listslice.py`

```
lst = [10, 20, 30, 40, 50, 60, 70, 80, 90, 100, 110, 120]
print(lst)           # [10, 20, 30, 40, 50, 60, 70, 80, 90, 100, 110, 120]
print(lst[0:3])      # [10, 20, 30]
print(lst[4:8])      # [50, 60, 70, 80]
print(lst[2:5])      # [30, 40, 50]
print(lst[-5:-3])    # [80, 90]
print(lst[:3])       # [10, 20, 30]
print(lst[4:])       # [50, 60, 70, 80, 90, 100, 110, 120]
print(lst[:])        # [10, 20, 30, 40, 50, 60, 70, 80, 90, 100, 110, 120]
print(lst[-100:3])   # [10, 20, 30]
print(lst[4:100])    # [50, 60, 70, 80, 90, 100, 110, 120]
print(lst[2:-2:2])   # [30, 50, 70, 90]
print(lst[::2])       # [10, 30, 50, 70, 90, 110]
print(lst[::-1])     # [120, 110, 100, 90, 80, 70, 60, 50, 40, 30, 20, 10]
```

When a slice involves a negative step, value the first argument in the slice represents the end of the reverse slice, and the second argument is the beginning of the slice.

Suppose variable `a` refers to a nonempty list. Note the difference between the expression `a[0:1]` and the expression `a[0]`. The expression `a[0:1]` represents a new list that contains only the first element of the original list `a`. The expression `a[0]` refers to the first element in `a`, which is not necessarily a list. The following interactive sequence illustrates the difference:

```
>>> a = [34, -19, 20, 8, 12]
>>> print(a)
[34, -19, 20, 8, 12]
>>> print(a[0:1])
[34]
>>> print(a[0])
34
>>> b = [[2, 5], 7, [11, 4]]
>>> print(b)
[[2, 5], 7, [11, 4]]
>>> print(b[0:1])
[[2, 5]]
>>> print(b[0])
[2, 5]
```

Slicing is the easiest way to make a copy of a list. The expression `lst[:]` evaluates to a copy of the entire list `lst`. The `list_copy` function we saw in Listing 10.17 (`listcopy.py`) made for an interesting exercise, but list slicing is a shorter, simpler way to achieve the same result. The last statement in Listing 10.20 (`listslice.py`) shows the expression `lst[::-1]` makes a copy of list `lst` with all of its elements appearing in reverse order.

Listing 10.21 (`prefixsuffix.py`) prints all the prefixes and suffixes of the list `[1, 2, 3, 4, 5, 6, 7, 8]`.

Listing 10.21: `prefixsuffix.py`

```
a = [1, 2, 3, 4, 5, 6, 7, 8]
print('Prefixes of', a)
for i in range(0, len(a) + 1):
    print(f'<{a[0:i]}>', sep='')
print('-----')
print('Suffixes of', a)
for i in range(0, len(a) + 1):
    print(f'<{a[i:len(a) + 1]}>', sep='')
```

Listing 10.21 (`prefixsuffix.py`) prints

```
Prefixes of [1, 2, 3, 4, 5, 6, 7, 8]
<>
<[1]>
<[1, 2]>
<[1, 2, 3]>
<[1, 2, 3, 4]>
<[1, 2, 3, 4, 5]>
<[1, 2, 3, 4, 5, 6]>
<[1, 2, 3, 4, 5, 6, 7]>
<[1, 2, 3, 4, 5, 6, 7, 8]>
-----
```

```
Suffixes of [1, 2, 3, 4, 5, 6, 7, 8]
<[1, 2, 3, 4, 5, 6, 7, 8]>
<[2, 3, 4, 5, 6, 7, 8]>
<[3, 4, 5, 6, 7, 8]>
<[4, 5, 6, 7, 8]>
<[5, 6, 7, 8]>
<[6, 7, 8]>
<[7, 8]>
<[8]>
<[]>
```

When the slicing expression appears on the left side of the assignment operator it can modify the contents of the list. This is known as *slice assignment*. A slice assignment can modify a list by removing or adding a subrange of elements in an existing list. Listing 10.22 (listslicemod.py) demonstrates how to use slice assignment to modify a list.

Listing 10.22: listslicemod.py

```
lst = [10, 20, 30, 40, 50, 60, 70, 80]
print(lst)           # Print the list
lst[2:5] = ['a', 'b', 'c'] # Replace [30, 40, 50] segment with ['a', 'b', 'c']
print(lst)
print('=====')
lst = [10, 20, 30, 40, 50, 60, 70, 80]
print(lst)           # Print the list
lst[2:6] = ['a', 'b'] # Replace [30, 40, 50, 60] segment with ['a', 'b']
print(lst)
print('=====')
lst = [10, 20, 30, 40, 50, 60, 70, 80]
print(lst)           # Print the list
lst[2:2] = ['a', 'b', 'c'] # Insert ['a', 'b', 'c'] segment at index 2
print(lst)
print('=====')
lst = [10, 20, 30, 40, 50, 60, 70, 80]
print(lst)           # Print the list
lst[2:5] = [] # Replace [30, 40, 50] segment with [] (delete the segment)
print(lst)
```

Listing 10.22 (listslicemod.py) displays:

```
[10, 20, 30, 40, 50, 60, 70, 80]
[10, 20, 'a', 'b', 'c', 60, 70, 80]
=====
[10, 20, 30, 40, 50, 60, 70, 80]
[10, 20, 'a', 'b', 70, 80]
=====
[10, 20, 30, 40, 50, 60, 70, 80]
[10, 20, 'a', 'b', 'c', 30, 40, 50, 60, 70, 80]
=====
[10, 20, 30, 40, 50, 60, 70, 80]
[10, 20, 60, 70, 80]
```

Observe that when the begin and end index of an assignment slice are the same, the assignment inserts a new segment into the list:

```
>>> a = [1, 2, 3, 4, 5]
>>> a
[1, 2, 3, 4, 5]
>>> a[1:1] = [100, 200]
>>> a
[1, 100, 200, 2, 3, 4, 5]
```

When the end index is different, the assignment slice replaces a segment of the list with new elements:

```
>>> a
[1, 2, 3, 4, 5]
>>> a[1:3] = [100, 200]
>>> a
[1, 100, 200, 4, 5]
```

or possibly removes the elements:

```
>>> a = [1, 2, 3, 4, 5]
>>> a
[1, 2, 3, 4, 5]
>>> a[1:3] = []
>>> a
[1, 4, 5]
```

The next section describes a different technique for removing elements from a list.

10.9 List Element Removal

We have seen how to append elements to a list using the list concatenation operator (+). We can use `del` to remove a specific element from a list via its index. The following sequence uses `range` to build a list and `del` to remove one of the list's elements:

```
>>> a = list(range(10, 51, 10))
>>> a
[10, 20, 30, 40, 50]
>>> del a[2]
>>> a
[10, 20, 40, 50]
```

We can remove a contiguous range of elements of a list using `del` with a slice, as shown here:

```
>>> b = list(range(20))
>>> b
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19]
>>> del b[5:15]
>>> b
[0, 1, 2, 3, 4, 15, 16, 17, 18, 19]
```

In Section 2.2 we saw how to use `remove` (actually, *undefine*) multiple scalar values in one `del` statement. We can delete multiple list elements in one `del` statement, but it requires much more care. Consider the following:

```
>>> c = list(range(20))
>>> c
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19]
>>> del c[1], c[18]
>>> c
[0, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18]
```

You might expect the `del` statement to remove element 1 and element 18. Instead, it removed 1 and 19. The deletion progresses from left to right, so the statement first removes 1, leaving the following list:

```
[0, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19]
```

The subsequent removal of `c[18]` occurs on this new list, not the original list. Element 19 now is at index 18, so the statement deletes element 19 instead of element 18. Consider the following list:

```
d = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19]
```

suppose we wish to delete elements 2, 3, 4, 5, 6, and 18. We might try the following `del` statement:

```
del c[2:7], c[18]
```

This statement, however, produces a run-time exception. The error arises because removing the slice of elements shortens the list so index 18 is out of range. Reordering the statement produces the desired results:

```
del c[18], c[2:7]
```

since it removes elements from the back to the front.

Because faulty attempts at multiple list element deletions can lead to surprising results it is best to restrict a list `del` statement to a single element or a single slice.

10.10 List Methods

All Python lists are instances of the `list` class. Table 10.1 lists some of the methods available to `list` objects.

Table 10.1 A few of the methods available to `list` objects

list Methods	
<code>count</code>	Returns the number of times a given element appears in the list. Does not modify the list.
<code>insert</code>	Inserts a new element before the element at a given index. Increases the length of the list by one. Modifies the list.
<code>append</code>	Adds a new element to the end of the list. Modifies the list.
<code>index</code>	Returns the lowest index of a given element within the list. Produces an error if the element does not appear in the list. Does not modify the list.
<code>remove</code>	Removes the first occurrence (lowest index) of a given element from the list. Produces an error if the element is not present in the list. Modifies the list if the item to remove is in the list.
<code>reverse</code>	Physically reverses the elements in the list. Modifies the list.
<code>sort</code>	Sorts the elements of the list in ascending order. Modifies the list.

Since lists are mutable data structures, the `list` class has both `__getitem__` and `__setitem__` methods. The statement

```
x = lst[2]
```

behind the scenes becomes the method call

```
x = list.__getitem__(lst, 2)
```

and the statement

```
lst[2] = x
```

maps to

```
list.__setitem__(lst, 2, x)
```

The `str` class does not have a `__setitem__` method, since strings are immutable.

The code

```
lst = ["one", "two", "three"]
lst += ["four"]
```

is equivalent to

```
lst = ["one", "two", "three"]
lst.append("four")
```

but the version using `append` is more efficient.

Section 10.8 showed that if `lst` is a list, the slice `lst[::-1]` returns a copy of the list with all the elements in reverse order. The expression `lst.reverse()` modifies `lst`, reversing the elements within `lst` itself. Just to complicate the situation a bit more, Section 10.2 used the `reversed` function to create an iterable object that allows programs to traverse a list in reverse order. Three ways different techniques that involve reversing a list can be confusing. The following summarizes the differences:

- `reversed` is a function (not a method) that accepts a list (or, as we will see, any sequence object); it returns an iterable object compatible with the `for` statement. The `reversed` function does not modify the list, and it does not create a new list.
- `list.reverse` is a method of the `list` class (not a function) that mutates an existing list; it does not return anything. The `list.reverse` method modifies the existing list.
- The `[::-1]` slice notation returns a new list containing a copy of the original list with its elements in reverse order. The slice does not modify the original list.

Listing 10.23 (`reversals.py`) exposes the major differences among the three list reversal techniques.

Listing 10.23: `reversals.py`

```
lst = [1, 2, 3, 4, 5, 6, 7]
print("---- Original list ----")
print("lst =", lst)
print("---- reversed function----")
obj1 = reversed(lst)
print("lst =", lst)
print("obj1 =", obj1)
print("---- Slice ----")
obj2 = lst[::-1]
print("lst =", lst)
print("obj2 =", obj2)
print("---- list.reverse method ----")
obj3 = lst.reverse()
print("lst =", lst)
print("obj3 =", obj3)
```

Listing 10.23 (`reversals.py`) produces the following output:

```
---- Original list ----
lst = [1, 2, 3, 4, 5, 6, 7]
---- reversed function----
lst = [1, 2, 3, 4, 5, 6, 7]
obj1 = <list_reverseiterator object at 0x000000000281A320>
---- Slice ----
lst = [1, 2, 3, 4, 5, 6, 7]
obj2 = [7, 6, 5, 4, 3, 2, 1]
---- list.reverse method ----
lst = [7, 6, 5, 4, 3, 2, 1]
obj3 = None
```

Notice that neither the `reversed` function nor the slice alter the original list. Also observe that the `list.reverse` method does not return a value (it implicitly returns `None`).

10.11 Prime Generation with a List

Listing 10.24 (`fasterprimes.py`) uses an algorithm developed by the Greek mathematician Eratosthenes who lived from 274 B.C. to 195 B.C. Called the Sieve of Eratosthenes, the principle behind the algorithm is simple: Make a list of all the integers two and larger. Two is a prime number, but any multiple of two cannot be a prime number since a multiple of two has two as a factor. Go through the rest of the list and mark out all multiples of two (4, 6, 8, ...). Move to the next number in the list (in this case, three). If it is not marked out, it must be prime, so go through the rest of the list and mark out all multiples of that number (6, 9, 12, ...). Continue this process until you have listed all the primes you want.

Listing 10.24 (`fasterprimes.py`) implements the Sieve of Eratosthenes in Python.

Listing 10.24: `fasterprimes.py`

```
# Display the prime numbers between 2 and 500

# Largest potential prime considered
MAX = 500

# Each position in the Boolean list indicates
# if the number of that position is not prime:
# false means "prime," and true means "composite."
# Initially all numbers are prime until proven otherwise
nonprimes = MAX * [False] # Initialize to all False

# First prime number is 2; 0 and 1 are not prime
nonprimes[0] = nonprimes[1] = True

# Start at the first prime number, 2.
for i in range(2, MAX):
    # See if i is prime
    if not nonprimes[i]:
        print(i, end=' ')
        # It is prime, so eliminate all of its
        # multiples that cannot be prime
        for j in range(2*i, MAX, i):
            nonprimes[j] = True
print() # Move cursor down to next line
```

How much better is the algorithm in Listing 10.24 (`fasterprimes.py`) than the square-root-optimized version we saw in Listing 6.8 (`timemoreefficientprimes.py`)? Listing 10.25 (`timeprimes.py`) compares the execution speed of the two algorithms.

Listing 10.25: `timeprimes.py`

```
""" Count the number of prime numbers less than 5 million
    and time how long it takes. Compares the performance of
    two different algorithms. """

from time import perf_counter_ns
from math import sqrt

def count_primes(n: int) -> None:
    """
```



```

Generates all the prime numbers from 2 to n - 1.
n - 1 is the largest potential prime considered.
"""
start = perf_counter_ns()    # Record start time

count = 0
for val in range(2, n):
    root = round(sqrt(val)) + 1
    # Try all potential factors from 2 to the square root of n
    for trial_factor in range(2, root):
        if val % trial_factor == 0: # Is it a factor?
            break                 # Found a factor
    else:
        count += 1                # No factors found

stop = perf_counter_ns()    # Stop the clock
elapsed = (stop - start)/1_000_000_000
print("Count =", count, "Elapsed time:", elapsed, "seconds")

def sieve(n: int) -> None:
    """
    Generates all the prime numbers from 2 to n - 1.
    n - 1 is the largest potential prime considered.
    Algorithm originally developed by Eratosthenes.
    """

    start = perf_counter_ns()    # Record start time

    # Each position in the Boolean list indicates
    # if the number of that position is not prime:
    # false means "prime," and true means "composite."
    # Initially all numbers are prime until proven otherwise
    nonprimes = n * [False]    # Global list initialized to all False

    count = 0

    # First prime number is 2; 0 and 1 are not prime
    nonprimes[0] = nonprimes[1] = True

    # Start at the first prime number, 2.
    for i in range(2, n):
        # See if i is prime
        if not nonprimes[i]:
            count += 1
            # It is prime, so eliminate all of its
            # multiples that cannot be prime
            for j in range(2*i, n, i):
                nonprimes[j] = True
    # Print the elapsed time
    stop = perf_counter_ns()
    elapsed = (stop - start)/1_000_000_000
    print("Count =", count, "Elapsed time:", elapsed, "seconds")

```

```
def main() -> None:
    LIMIT = 5000000
    count1 = count_primes(LIMIT)
    count2 = sieve(LIMIT)

if __name__ == '__main__':
    main()
```

Since printing to the screen takes up the majority of the time, Listing 10.25 (timeprimes.py) counts the number of primes rather than printing each one. This allows us to better compare the behavior of the two approaches.

The output of Listing 10.25 (timeprimes.py) on one system reveals

```
Count = 348513 Elapsed time: 45.3336438 seconds
Count = 348513 Elapsed time: 1.620349 seconds
```

Our previous “optimized” version requires approximately 45 seconds to count the number of primes less than five million, while the version based on the Sieve of Eratosthenes takes only about one and one-half seconds. The Sieve of Eratosthenes function is 28 times faster.

10.12 Command-line Arguments

The `sys` module provides a global variable named `argv` that is a list of extra text that the user can supply when launching an application from the operating system shell (normally called the *command prompt* in Windows and *terminal* in macOS and Linux). To run a program stored in the file `myprog.py`, the user would type the command

```
python myprog.py
```

Some programs expect or allow the user to provide extra information. Listing 10.26 (cmdlineargs.py) is a program meant to be executed from the command line with extra arguments. It simply reports the extra information the user supplied.

Listing 10.26: cmdlineargs.py

```
import sys

for arg in sys.argv:
    print(f'[{arg}]', end=' ')
print()
```

The following shows one run of Listing 10.26 (cmdlineargs.py):

```
C:\Code\python>python cmdlineargs.py -h 45 extra
[cmdlineargs.py] [-h] [45] [extra]
```

The shell printed `C:\Code\python>`, and the user typed the remainder of the first line. The program printed the second line. We see that `argv[0]` is simply the name of the file containing the program. `argv[1]` is the string `'-h'`, `argv[2]` is the string `'45'`, and `argv[3]` is the string `'extra'`.

In Listing 10.27 (`sqrtcmdline.py`), the user supplies a range of integers on the command line. The program then prints all the integers in that range along with their square roots.

Listing 10.27: `sqrtcmdline.py`

```
from sys import argv
from math import sqrt

if len(argv) < 3:
    print('Supply range of values')
else:
    for n in range(int(argv[1]), int(argv[2]) + 1):
        print(n, sqrt(n))
    print()
```

The following shows some sample runs of Listing 10.27 (`sqrtcmdline.py`):

```
C:\Code\python>python sqrtcmdline.py
Supply range of values

C:\Code\python>python sqrtcmdline.py 2
Supply range of values

C:\Code\python>python sqrtcmdline.py 2 10
2 1.4142135623730951
3 1.7320508075688772
4 2.0
5 2.23606797749979
6 2.449489742783178
7 2.6457513110645907
8 2.8284271247461903
9 3.0
10 3.1622776601683795
```

As Listing 10.27 (`sqrtcmdline.py`) shows, each command line argument is a string. If the program must interpret an argument as a number, it must convert it using the appropriate conversion function, `int` or `double`.

10.13 List Comprehensions

Mathematicians often represent sets in two different ways:

1. set roster notation, which enumerates the elements in the set, and
2. set builder notation, which describes the contents of the set using a rule for constructing the set's elements

We can express P , the set of perfect squares less than 50, using both of these methods:

1. set roster notation: $P = \{0, 1, 4, 9, 16, 25, 36, 49\}$
2. set builder notation: $P = \{x^2 \mid x \in \{0, 1, 2, 3, 4, 5, 6, 7\}\}$

The set roster notation example is obvious—it just lists the elements of the set. We read the set builder notation example as “ P is the set of all squares of x , such that x is taken from the set $\{0, 1, 2, 3, 4, 5, 6, 7\}$.” Set builder notation in mathematics is essential for representing very large sets and infinite sets; for example, consider $S = \{x^2 \mid x \in \mathbb{Z}\}$, the set of *all* perfect squares ($\mathbb{Z}$ is the infinite set of integers). A perfect square is the square of an integer: $0^2 = 0, 1^2 = 1, 2^2 = 4, 3^2 = 9, 4^2 = 16, \dots$. Listing all the elements is impossible. We could try to list the first few elements followed by an ellipsis (...), but the pattern may not be obvious to all readers.

We can compare Python’s lists to mathematical sets (with the important caveat that the order of the elements in a list matters, but element order is irrelevant in a mathematical set). We have seen how to express lists in Python in a manner similar to set roster notation in mathematics:

```
P = [1, 4, 9, 16, 25, 36, 49]
```

Python also supports a technique similar to set builder notation, called *list comprehension*. In Python, we can express list P above as

```
P = [x**2 for x in [0, 1, 2, 3, 4, 5, 6, 7]]
```

or, using a `range` expression as

```
P = [x**2 for x in range(8)]
```

Observe the use of the keywords `for` and `in` within the square brackets. The `for` keyword takes the place of the mathematical $\mid$ symbol (usually pronounced “such that” or “where”), and `in` is the $\in$ membership relation. All list comprehensions use `for` within the square brackets.

We have seen that the `range` expression is useful for building lists with a regular, predictable ordering. As another example, consider the following code that builds a list of the multiples of four from 4 to 40:

```
>>> lst = list(range(4, 41, 4))
>>> lst
[4, 8, 12, 16, 20, 24, 28, 32, 36, 40]
```

One limitation of `range` is that its arguments all must be integers. Suppose we wish to create succinctly a list of floating-point numbers in a regular sequence. We cannot use the `range` expression by itself to express such a list, but a list comprehension is ideal for the task. The following interactive sequence creates a list containing the first ten multiples of one-half:

```
>>> halves = [x/2 for x in range(10)]
>>> halves
[0.0, 0.5, 1.0, 1.5, 2.0, 2.5, 3.0, 3.5, 4.0, 4.5]
```

The single slash (/) performs floating-point division, populating the list with floating-point values.

We can use a list comprehension to filter certain elements out of a list. In the following example we take a list containing integers and produce a new list with all the negative values omitted:

```
>>> lst = [9, -44, 0, 30, 2, -6, -8, 9, -8, 23]
>>> lst
[9, -44, 0, 30, 2, -6, -8, 9, -8, 23]
>>> [x for x in lst if x >= 0]
[9, 0, 30, 2, 9, 23]
```

The expression `[x for x in lst if x >= 0]` selects all the elements from list `lst` except for those less than zero. If we instead want a list of the negative values, invert the logic of the condition:

```
>>> lst
[9, -44, 0, 30, 2, -6, -8, 9, -8, 23]
>>> [x for x in lst if x < 0]
[-44, -6, -8, -8]
```

The next example begins with a mixed list of numeric values and strings. The following interactive sequence shows how we can use list comprehensions to selectively filter out of the list all the strings or all the numbers:

```
>>> lst = ['ABC', 23.4, 7, 'Wow', 16, 'xyz', 10]
>>> lst
['ABC', 23.4, 7, 'Wow', 16, 'xyz', 10]
>>> [x for x in lst if type(x) != str]
[23.4, 7, 16, 10]
>>> lst
['ABC', 23.4, 7, 'Wow', 16, 'xyz', 10]
>>> [x for x in lst if type(x) == str]
['ABC', 'Wow', 'xyz']
```

Neither list comprehension expression affects the original list `lst`. A list comprehension expression evaluates to a new list, and we can assign a variable to this list if we wish to retain it for later use:

```
>>> lst = ['ABC', 23.4, 7, 'Wow', 16, 'xyz', 10]
>>> lst2 = [x for x in lst if type(x) != str]
>>> lst
['ABC', 23.4, 7, 'Wow', 16, 'xyz', 10]
>>> lst2
[23.4, 7, 16, 10]
```

The lists we build with list comprehension are not limited to elements with simple, single values. The following interactive sequence builds a list containing tuples of the first 10 positive integers paired with their squares:

```
>>> squares = [(x, x**2) for x in range(1, 11)]
>>> squares
[(1, 1), (2, 4), (3, 9), (4, 16), (5, 25), (6, 36), (7, 49), (8, 64),
(9, 81), (10, 100)]
```

We can use multiple `for` expressions within a single list comprehension. This is useful for combining elements from different lists. In the following example we make tuples out of the elements of two different lists:

```
>>> pairs = [(x, y) for x in [1, 2, 3] for y in ['a', 'b']]
>>> pairs
[(1, 'a'), (1, 'b'), (2, 'a'), (2, 'b'), (3, 'a'), (3, 'b')]
```

We can use the `if` keyword to add conditions for list membership. To start with, suppose we wish to express the list containing the ordered pairs of elements taken from the list `[1, 2, 3]`. That is easy enough:

```
>>> points = [(x, y) for x in [1, 2, 3] for y in [1, 2, 3]]
>>> points
[(1, 1), (1, 2), (1, 3), (2, 1), (2, 2), (2, 3), (3, 1), (3, 2), (3, 3)]
```

We can use variables within our list comprehension to shorten it a bit:

```
>>> V = [1, 2, 3]
>>> points = [(x, y) for x in V for y in V]
>>> points
[(1, 1), (1, 2), (1, 3), (2, 1), (2, 2), (2, 3), (3, 1), (3, 2), (3, 3)]
```

Now suppose we wish to exclude pairs that contain the same elements. We can add an `if` expression to the end of the list comprehension to impose additional conditions on the list members:

```
>>> V = [1, 2, 3]
>>> points = [(x, y) for x in V for y in V if x != y]
>>> points
[(1, 2), (1, 3), (2, 1), (2, 3), (3, 1), (3, 2)]
```

Here we see no number is paired with itself. The following list comprehension excludes pairs with components that sum to an even number:

```
>>> points = [(x, y) for x in [1, 2, 3] for y in [1, 2, 3]
               if (x + y) % 2 != 0]
>>> points
[(1, 2), (2, 1), (2, 3), (3, 2)]
```

We see (1, 3) is missing because $1 + 3 = 4$, and 4 is even.

Suppose we wish to build a list of factor pairs for a positive integer supplied by the user; for example, the tuple (2, 15) is a factor pair for the number 30 because $2 \times 15 = 30$. We will begin with an easier problem: create a list of all the factors for a positive integer supplied by the user. Listing 10.28 (factorsingles.py) uses a list comprehension to create a such a list of factors.

Listing 10.28: factorsingles.py

```
n = int(input("Please enter a positive integer: "))
factors = [x for x in range(1, n + 1) if n % x == 0]
print("Factors of", n, ":", factors)
```

The following shows a sample run of Listing 10.28 (factorsingles.py) with the user entering the value 100:

```
Please enter a positive integer: 100
Factors of 100 : [1, 2, 4, 5, 10, 20, 25, 50, 100]
```

Observe that the program printed all the factors of 100. We want our list to contain factor pairs; that is, we want to pair each factor with its mate so that the two values multiplied together equal the number the user entered. We already have the first elements of the pairs, and we can compute their mates easily with integer division. If x is a factor of n , then $n // x$ will be its mate because $x * n // x$ will equal n . The factor pair is thus $(x, n // x)$. Listing 10.29 (factorpairs.py) shows the resulting program which produces a list of factor pairs.

Listing 10.29: factorpairs.py

```
n = int(input("Please enter a positive integer: "))
factors = [(x, n//x) for x in range(1, n + 1) if n % x == 0]
print("Factor pairs of", n, ":", factors)
```

The following shows a sample run of Listing 10.28 (factorsingles.py) with the user entering the value 100:

```
Please enter a positive integer: 100
Factor pairs of 100 : [(1, 100), (2, 50), (4, 25), (5, 20), (10, 10),
(20, 5), (25, 4), (50, 2), (100, 1)]
```

This is fine, but it would be nice to avoid adding redundant pairs to the list; that is, we want just one of the pairs (2, 50) and (50, 2) to appear in our list. Notice that once the first element reaches the square root of the value supplied by the user, the remaining pairs are mirror images of earlier pairs. We can use this fact to limit the range expression; we will choose x s in the range $1 \dots \sqrt{n}$, inclusive. The Python implementation of this range is `range(1, round(math.sqrt(n)) + 1)`. Listing 10.30 (uniquefactorpairs.py) adds this finishing touch to avoid the redundant pairs.

Listing 10.30: uniquefactorpairs.py

```
import math
n = int(input("Please enter a positive integer: "))
factors = [(x, n//x) for x in range(1, round(math.sqrt(n)) + 1) if n % x == 0]
print("Factor pairs of", n, ":", factors)
```

The following shows a sample run of Listing 10.30 (uniquefactorpairs.py):

```
Please enter a positive integer: 100
Factor pairs of 100 : [(1, 100), (2, 50), (4, 25), (5, 20), (10, 10)]
```

We can use list comprehensions to filter elements from an existing list. In the following example we build a list of ascending integers that omits a few elements:

```
>>> L = [x for x in range(20) if x not in [12, 8, 3, 17]]
>>> L
[0, 1, 2, 4, 5, 6, 7, 9, 10, 11, 13, 14, 15, 16, 18, 19]
```

This list contains all the elements in `range(20)` in order, but it specifically excludes the values 3, 8, 12, and 17.

Compare the previous list to this one:

```
M = [x not in [12, 8, 3, 17] for x in range(0, 20)]
>>> M
[True, True, True, False, True, True, True, True, False, True,
True, True, False, True, True, True, True, False, True, True]
```

This list reports whether an element is present (**True**) or not in the list (**False**). Positions 3, 8, 12, and 17 are “marked” false.

The following list comprehension uses a different Boolean expression to build a list of Boolean values:

```
>>> [x > 2 for x in [5, 1, 0, 7, 2, 7, 3]]
[True, False, False, True, False, True, True]
```

We will see in Section 11.9 how to apply such Boolean conditions in a slightly different way to determine quickly and easily if *all* the elements in a list possess a given property or if *any* elements in a list possess a given property.

We can rewrite the following list comprehension:

```
L = [x for x in range(20) if x not in [12, 8, 3, 17]]
```

as

```
L = []
for x in range(20):
    if x not in [12, 8, 3, 17]:
        L.append(x)
```

While our expanded code is functionally equivalent to the list comprehension, in general a list comprehension will be more efficient than its equivalent expanded form.

Python list comprehensions are powerful and can be quite complex. When programming it sometimes is easier to build the list without list comprehension and later discover a way to transform the code to use list comprehension. As an example, consider the task of building a list that contains all the prime numbers less than 100. For all $n \geq 2$, the following list comprehension creates a list of all the factors of $n > 1$ up to $\sqrt{n}$:

```
[x for x in range(2, round(sqrt(n)) + 1) if n % x == 0]
```

This takes advantage of the square root optimization from Listing 7.16 (primefunc.py). The following expression evaluates to true if n is a prime number; otherwise, it evaluates to false:

```
[x for x in range(2, round(sqrt(n)) + 1) if n % x == 0] == []
```

In Python, the empty list (`[]`) is considered false, and any nonempty list is considered true, so the following expression reveals the primality of n as well:

```
not [x for x in range(2, round(sqrt(n)) + 1) if n % x == 0]
```

To make a list of all the prime numbers less than 80, we can use a list comprehension expression that begins like the following:

```
[p for p in range(2, 80) if ???]
```

We then can replace the `???` with the list comprehension from above that determines if p is a prime number:

```
[p for p in range(2, 80)
    if not [x for x in range(2, round(sqrt(p)) + 1) if p % x == 0]]
```

In the Python shell we can see the list this expression produces:

```
>>> [p for p in range(2, 80)
...   if not [x for x in range(2, round(sqrt(p)) + 1) if p % x == 0]]
[2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47, 53, 59,
61, 67, 71, 73, 79]
```

List comprehensions are a valuable, powerful tool for creating lists. While the prime numbers example demonstrates the power of Python's list comprehensions, many programmers would consider the resulting list comprehension expression above to be a bit too complex and tricky. Additionally, it is not very efficient. It creates an internal, temporary list of factors for each number it must consider. Simpler list comprehension expressions that avoid nested lists generally are efficient and readable. As a rule, you should use list comprehensions when they make your code simpler and more readable, but do not go out of your way to construct arcane list comprehension expressions just because you can.

If we replace the outer square brackets of a list comprehension with parentheses, the expression becomes a *generator expression* (sometimes called a *generator comprehension*). A generator expression creates a generator (Section 8.9) instead of a list. We can iterate over a generator with a `for` loop, as Listing 10.31 (`generatorexpression.py`) illustrates.

Listing 10.31: `generatorexpression.py`

```
for val in (2**x for x in range(10)):
    print(val, end=' ')
print()
```

Listing 10.31 (`generatorexpression.py`) prints

```
1 2 4 8 16 32 64 128 256 512
```

If you need to keep around the values of a sequence for additional processing, store them in a list and possibly use a list comprehension to make the list. If you simply need to visit the elements in a sequence once, building a list is overkill; use a generator to produce the sequence's elements as needed. The list has to store all of its elements in memory for the life of the list, but a generator produces only one element at a time. This means the list `[x for x in range(n)]` will consume a large amount of the computer's memory if `n` is large. The generator `(x for x in range(n))` uses a relatively small, constant amount of memory regardless of `n`'s value.

Section 10.16 compares lists and generators in more detail.

10.14 Multidimensional Lists

A list represents a one-dimensional (1D), linear data structure. We can display a list's elements from beginning to end in a straight line; for example:

```
[9, 17, 88, 2, 17, 6, 45]
```

We can locate an element uniquely within the list via a single integer index, its offset from the beginning of the list. In the example list above, element 88 is at index 2.

Some kinds of information are better represented two-dimensionally, in a rectangular array of elements, also known as a *matrix*; for example,

```
100  14   8  22  71
   0 243  68   1  30
  90  21   7  67 112
115 200  70 150   8
```

Such a two-dimensional (2D) matrix has a particular number of rows and columns. The values in rows are arranged horizontally, and the elements in columns are arranged vertically. The above matrix, therefore, has four rows and five columns. We say its dimension is 4×5 . The first row of the example matrix above consists of the elements 100, 14, 8, 22, and 71; the first column contains 100, 0, 90, and 115. We can locate an element uniquely within the matrix with two integer indices—the first index represents the element's offset from the first row, and the second index represents the element's offset from the first column. As with 1D lists, the index of the first row is zero, and the index of the first column is zero. The element 67 above has a row index of 2 and a column index of 3.

How are 2D matrices used? Mathematicians can represent a system of equations in a matrix and use techniques from linear algebra to solve the system. Computer scientists use 2D matrices to model the articulation of robotic arms. Computer graphics programmers mathematically manipulate 2D matrices to transform data in 3D space and project it onto a 2D screen giving users the illusion of depth, motion, location. Many classic games such as chess, checkers, Go, and Scrabble involve a 2D grid of board positions that naturally maps to a 2D matrix. A word search puzzle is a rectangular array of letters, and a maze is simply a 2D arrangement of adjoining rooms, some of which are connected and others that are not.

In Python we can express a 2D matrix as a *list of lists*. We can model the matrix shown above with the following statement:

```
matrix = [[100, 14, 8, 22, 71],
          [ 0, 243, 68, 1, 30],
          [ 90, 21, 7, 67, 112],
          [115, 200, 70, 150, 8]]
```

While this arrangement of text best visually communicates the 2D nature of the object, as far as Python is concerned it still is simply a linear list in which each element is itself a list. We can reveal this internal linear representation with the following interactive sequence:

```
>>> matrix = [[100, 14, 8, 22, 71],
...           [ 0, 243, 68, 1, 30],
...           [ 90, 21, 7, 67, 112],
...           [115, 200, 70, 150, 8]]
>>>
>>> print(matrix)
[[100, 14, 8, 22, 71], [0, 243, 68, 1, 30], [90, 21, 7, 67, 112],
 [115, 200, 70, 150, 8]]
```

Even though internally such a matrix ultimately must be linear, the good news is that we conceptually can treat our list of lists as if it were a 2D structure, and everything will work as expected. Figure 10.4 compares our conceptual notion of a 2D list with its physical layout.

We must use two indices to access an element in our 2D list. We can print the element 67 from the above 2D list named `matrix` as

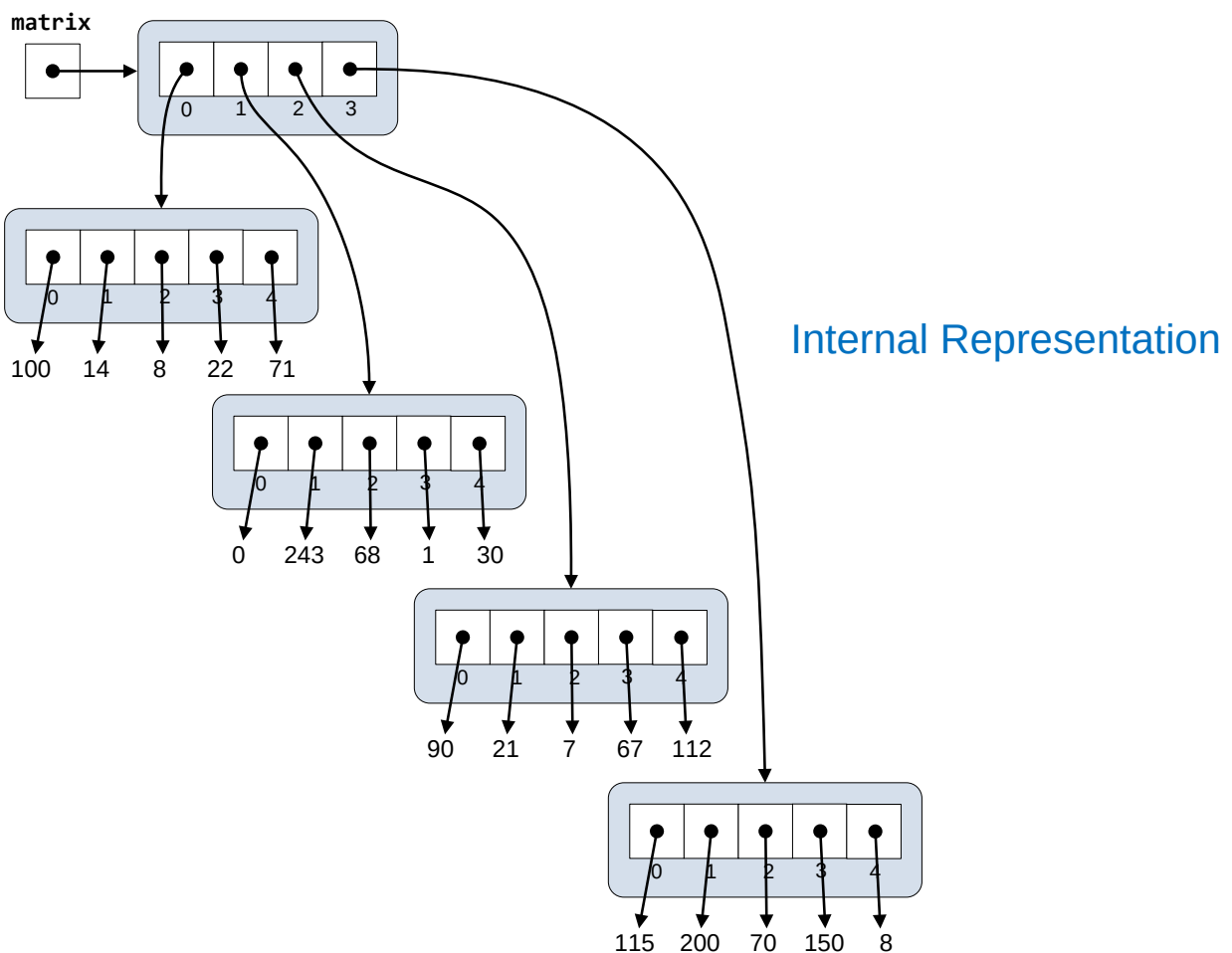
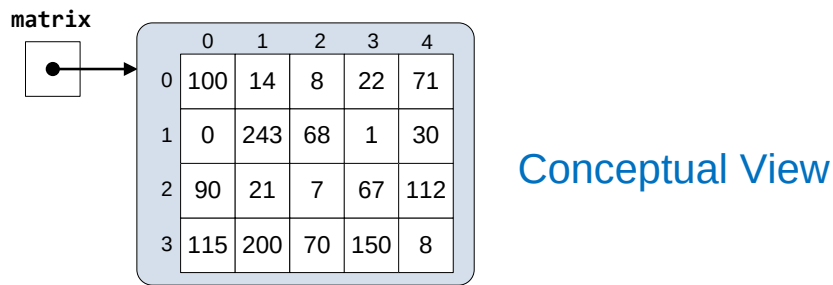
```
print(matrix[2][3])
```

Note the double square brackets. The first index (2) selects the row, and the second index (3) specifies the column. The expression `matrix` represents the whole 2D list. If we use just one index with a 2D list, it selects an entire row. The expression `matrix[x]` represents the 1D list that constitutes row `x` in `matrix`; for example, `matrix[2]` is the list `[90, 21, 7, 67, 112]`. That means we can interpret the expression `matrix[2][3]` as `(matrix[2])[3]`; that is, the element at index 3 within the row at index 2. The expression `len(matrix)` is the number of rows in `matrix`. The expression `len(matrix[2])` represents the number of elements in the row at index 2. For many applications, every row in the 2D list will have the same length; we call such a 2D list a *table*. Python does not require that all rows have the same number of elements. The following code creates what is called a *ragged array*, where each row potentially has a different length:

```
ragged = [[100, 14, 8, 22, 71],
          [ 0, 243],
          [ 90, 21, 7, 67],
          [115, 200, 70, 150, 8, 45, 60]]
```

Listing 10.32 (`prettyprint2d.py`) demonstrates how to print a 2D list more attractively with a nested loop.

Figure 10.4 A conceptual illustration of a two-dimensional list versus its physical layout. In the conceptual view the indices along the side locate the rows, and the indices across the top refer to the columns. When programming it is safe to use the conceptual view as a mental model of a 2D list. The internal representation of a 2D list's physical layout shows that a 2D list is really just a list of lists.



Listing 10.32: prettyprint2d.py

```

matrix = [[100, 14, 8, 22, 71],
          [ 0, 243, 68, 1, 30],
          [90, 21, 7, 67, 112],
          [115, 200, 70, 150, 8]]

for row in matrix:      # Process each row
    for elem in row:    # For each element in a given row
        print(f'{elem:4}', end='')
    print()

```

Listing 10.32 (prettyprint2d.py) displays the following,

```

100 14 8 22 71
 0 243 68 1 30
90 21 7 67 112
115 200 70 150 8

```

The creation of a large 2D list with identical initial values merits special attention. Section 10.3 showed how we can construct lists using the list element multiplier notation. The following statement:

```
a = [0] * 4
```

builds a 1D list named `a` that contains four zeros. This list element multiplier technique is straightforward when the elements are immutable types (see Section 9.8). All the elements refer to the same object, but since the object cannot be mutated, any reassignment of a list element will not effect any of the other elements in the list. The following code produces no surprises:

```

a = [0] * 4      # Make a small list containing zeros
print(a)         # Prints [0, 0, 0, 0]
a[1] = 5         # Replace element at index 1
print(a)         # Prints [0, 5, 0, 0]

```

It prints, as expected,

```

[0, 0, 0, 0]
[0, 5, 0, 0]

```

If we attempt to build a 2D list the same way, we get a surprising result:

```

a = [[0] * 4] * 3      # Make a 3 x 4 2D list containing zeros
print(a)               # Prints [[0, 0, 0, 0], [0, 0, 0, 0], [0, 0, 0, 0]]
a[1][2] = 5            # Replace element at row 1, column 2
print(a)               # Prints what?!

```

This code prints

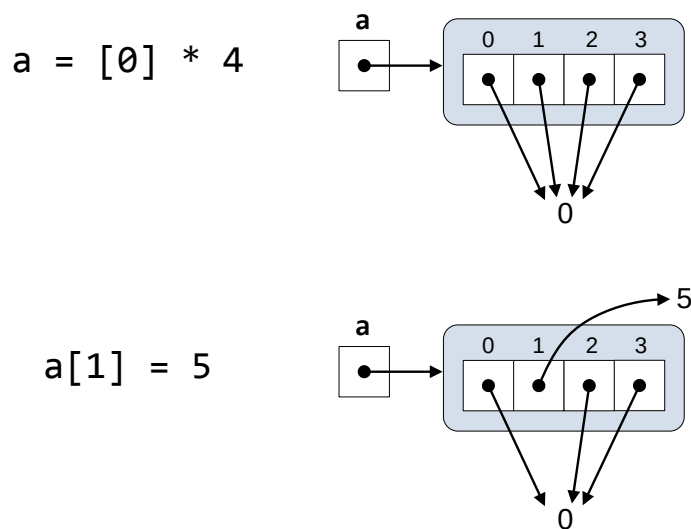
```

[[0, 0, 0, 0], [0, 0, 0, 0], [0, 0, 0, 0]]
[[0, 0, 5, 0], [0, 0, 5, 0], [0, 0, 5, 0]]

```

We expected the first line, but how did our attempt to change just one element in the list modify two other elements at the same time?

Figure 10.5 List multiplication creates a list of elements that all refer to the same object, the integer zero. Reassignment of the element at index 1 redirects the index 1 reference to a different object, in this case five.



To understand what is happening, we first will consider the simpler, 1D case that behaved as expected. Figure 10.5 illustrates list creation and element reassignment in the 1D list. Figure 10.5 shows that after the list multiplication statement that creates the list, all the elements in the list refer to the same object, the integer zero. The object zero is immutable, so it cannot change, but we can redirect a reference in the list to a different object, and that is exactly what the assignment statement does.

Figure 10.6 illustrates the attempt at 2D list creation and the subsequent element reassignment. As in the 1D case from before, the list multiplication creates a list of elements that all refer to the same object. In this case, however, instead of the integer zero, the object to which all the elements refer is a 1D list containing zeros. Since all the references in the list of lists `a` refer to the same list, all the rows in our 2D list alias the same list of zeros. Attempting the change the element at index 2 in row 1 changes the element at index 2 in all the rows.

List multiplication creation with immutable types such as integers, floating-point numbers, and strings is always well behaved. The aliasing is not a problem, since immutable objects cannot change. Programmers must be particularly careful using list multiplication when the elements are mutable types. Lists are mutable, and we will encounter more mutable types later.

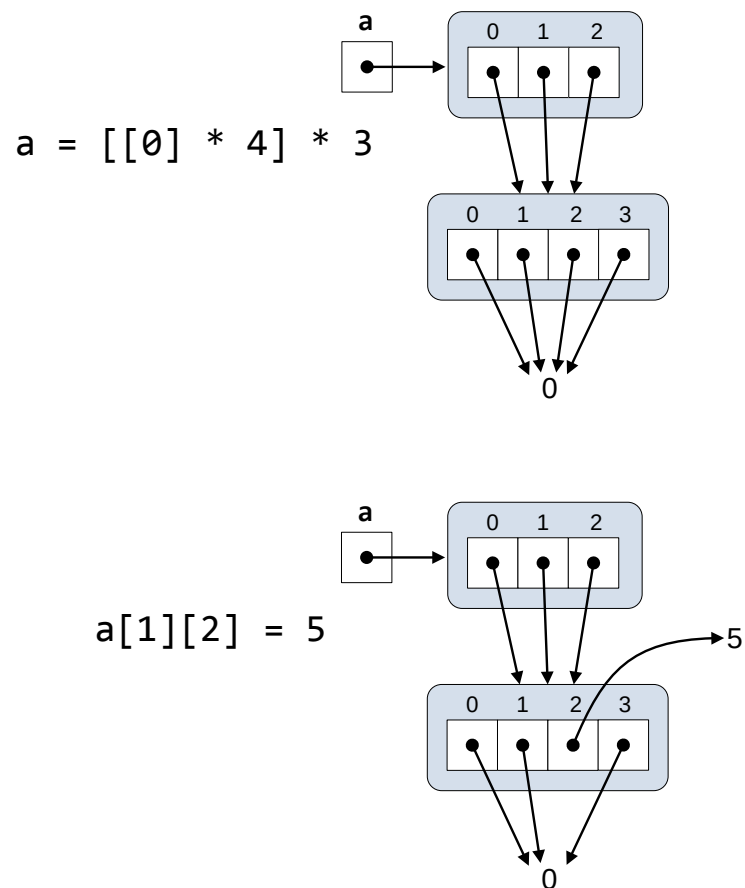
How can we easily create a 3×4 matrix of zeros? We can use a combination of list multiplication and list comprehension, as follows:

```
a = [[0] * 4 for x in range(3)]
```

The effect of this statement is similar to the following code:

```
a = []
for x in range(3):
    a.append([0] * 4)
```

Figure 10.6 List multiplication creates a list of elements that all refer to the same object, a list containing zeros. All the references in the outer list alias the same inner list, so the list actually contains just one, shared physical row. The list at row 0 is the same list at row 1 and row 2. Due to the aliasing, reassignment of the element at index 2 in row 1 affects the reference at index 2 in all the rows.



The execution of list multiplication creates a new list each time through the loop. The list comprehension example performs an optimized loop behind the scenes to achieve the result.

One thing to notice about the list comprehension example is the expression before the `for` keyword does not contain the variable `x`; it contains only the expression `[0] * 4`:

```
a = [[0] * 4 for x in range(3)]
```

The `for` expression within the list comprehension requires a variable to control `range`'s iteration. Here we gave it the name `x`. This variable `x` is local to the list comprehension. Section 2.3 emphasized that all variables should have meaningful, descriptive names. Assignment attaches a name to an object so we can access that object in the future via its name. The problem with `x` is this: we do not really do any thing with `x`. It is difficult to provide a good name for a variable that we never actually use. Some programmers address this situation by using a special variable name consisting of the single underscore (`_`). There is nothing magical about the single underscore symbol—it is a legal identifier, and, therefore, is a valid variable name. Its primary value lies in the fact that its name is ultimately non-descriptive! The name `x` may have a special meaning in some contexts, like the first element in an (x,y) ordered pair, but it is difficult to attach meaning to the name `_`.

Python programmers widely recognize the `_` name as a “throw away” variable that is convenient to use in situations in which both of the following are true:

1. the language requires a variable to appear in a particular context, and
2. the program has no use for the variable elsewhere.

We can rewrite the 2D list creation statement as

```
a = [[0] * 4 for _ in range(3)]
```

Notice that the underscore is almost invisible or certainly less obvious than the original `x` variable. This new version is less likely to prompt experienced Python programmers to ask “What does *that variable* do?”

The `_` symbol has special meaning in the interactive interpreter. The interpreter automatically assigns `_` to the value of its most recent evaluation. The following interactive sequence illustrates:



```
>>> 10 + 4
14
>>> _
14
>>> 100 - 60
40
>>> 2 + _
42
>>>
```

The `_` identifier has no special meaning in a Python program. As with any other variable, a programmer can explicitly assign `_` to any object.

Instead of mixing the list multiplication and list comprehension syntax, we can double up on list comprehension:

```
a = [[0 for _ in range(4)] for _ in range(3)]
```

Since the `_` variable as used above is local to the code within the square brackets of its list comprehension, the inner (left) `_` is separate from, and does not conflict with, the outer (right) `_`. The two `_` identifiers represent two different integer objects, and programmers are free to manipulate them independently.

Figure 10.7 illustrates the results of this double-barreled list comprehension.

We will see another good use of the `_` identifier in its role as a throw away variable in Section 11.1.

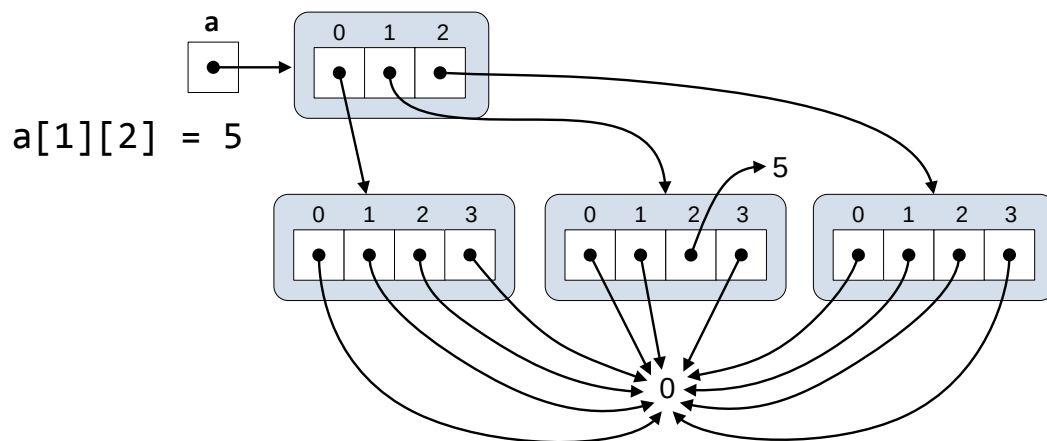
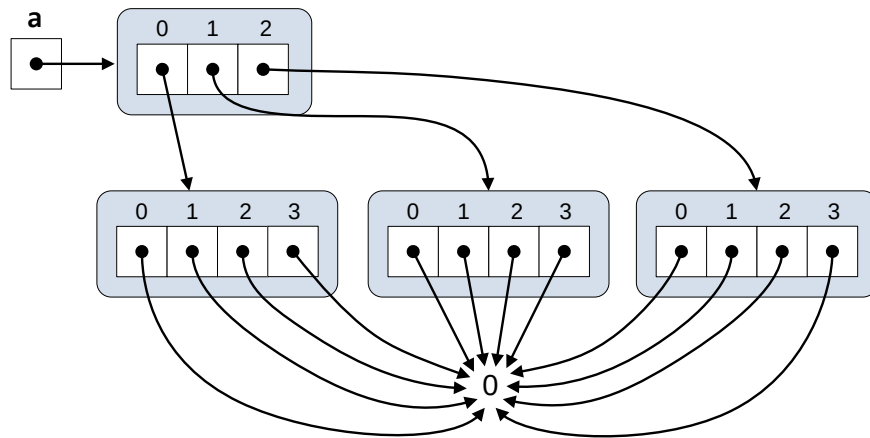
Python supports higher dimensional lists. A 2D list is a list of 1D lists. Similarly, a 3D list is a list 2D lists. You can visualize a 3D list as a rectangular solid (like a cube), where each layer of the solid is 2D plane (that is, a 2D list).

The number of list dimensions allowed by Python is limited only by available memory. Higher-dimensional lists can consume an enormous amount of memory. Fortunately, most programmers rarely need to deal with lists with dimensions greater than three. Only the most esoteric applications require lists with more than three dimensions.

Consider the children's game of Tic-Tac-Toe, sometimes called Noughts and Crosses (see <https://en.wikipedia.org/wiki/Tic-tac-toe> for more information about the game). It consists of a 3×3 playing grid in which two opposing players alternately place Xs and Os. A 3D variation uses a $4 \times 4 \times 4$ cube for placing Xs and Os (see https://en.wikipedia.org/wiki/3-D_Tic-Tac-Toe for more information). If we were to implement 3D Tic-Tac-Toe in Python, we could use a $4 \times 4 \times 4$ list. As a visual proof of concept, Listing 10.33 (`threedlist.py`) prints an intermediate state of a 3D Tic-Tac-Toe game in progress.

Figure 10.7 The double-barreled list comprehension creates independent lists for each row. Reassignment of the element at index 2 in row 1 affects the only row 1.

```
a = [[0 for _ in range(4)] for _ in range(3)]
```



Listing 10.33: threedlist.py

```

# Make a 3D matrix representing the intermediate state
# of a 3D Tic-Tac-Toe game in progress.

matrix = [[['X', '.', '.', 'O'],
            ['.', '.', '.', '.'],
            ['.', '.', '.', '.'],
            ['.', '.', '.', '.']],
          [['.', '.', '.', 'O'],
            ['.', 'X', '.', '.'],
            ['.', '.', '.', '.'],
            ['.', '.', '.', '.']],
          [['.', '.', '.', '.'],
            ['.', '.', '.', '.'],
            ['.', '.', 'X', '.'],
            ['.', '.', '.', '.']],
          [['.', '.', '.', '.'],
            ['.', '.', '.', '.'],
            ['.', '.', '.', '.'],
            ['.', '.', '.', 'O']]]

# Pretty print the matrix
# Need the length of a row in order to adjust the indentation
row_length = len(matrix[0][0])
for layer in matrix:      # Process each layer
    for row in range(len(layer)):  # For each row by its index
        # Print diminishing offset based on its index
        print(' ' * (row_length - row), end='')
        # For each element in a given row
        for column in range(len(layer[row])):
            print(f'{layer[row][column]:4}', end='')
        print()
    print()

```

Listing 10.33 (threedlist.py) prints



The dots represent empty positions. It appears that player O just moved into the bottom-right-front corner, blocking a potential win by player X (the line from the top-left-back to the bottom-right-front).

10.15 Summary of List Creation Techniques

So far we have seen a variety of ways to create lists in Python. To recap, we will look at several different ways to create the following list:

```
[2, 4, 6, 8, 10, 12, 14, 16, 18, 20]
```

- **Literal enumeration:**

```
nums = [2, 4, 6, 8, 10, 12, 14, 16, 18, 20]
```

- **Piecemeal assembly:**

```
nums = []
for i in range(2, 21, 2):
    nums.append(i)
```

- **Creation from a generator or range expression:**

```
nums = list(range(2, 21, 2))
```

- **List comprehension:**

```
nums = [x for x in range(1, 21) if x % 2 == 0]
```

- **Combination of methods with list concatenation:**

```
nums = list(range(2, 9, 2)) + [10, 12, 14] + [x for x in range(16, 21, 2)]
```

The last example using list concatenation is overcomplicated for the given list to construct, so you should use such a piecemeal approach only when the more straightforward techniques are insufficient. Chapter 11 introduces several new ways to create lists from other Python data structures.

10.16 Lists versus Generators

We now have seen two ways of expressing a sequence of values: generators and lists. How are they similar? Generators and lists share the following characteristics:

- Both generators and lists represent a sequence of values. This means the order of the values matters. There is a first element and a last element in a sequence. Except for the first element in the sequence, every element has a predecessor. Except for the last element, every element in the sequence has a successor.
- We can iterate over both generators and lists using the `for` statement.

How are generators and lists different? Generators and lists differ in the following ways:

- The elements in a list persist for the life of the list, but the elements produced by a generator become available in turn as the iteration with the generator progresses. Once the iteration moves on from the current element, previous elements are unavailable from that particular generator object.
- The memory required for a list generally will be greater than that for a generator that can produce the same sequence of values. This is because the generator manages only one element at a time, while a list must store simultaneously all the elements in the sequence.
- Lists provide *random access*. This means any value at any position in the list is available at any time. A generator serves up values one at a time, in order from the first to the last. We cannot obtain the i^{th} element from a generator without first requesting all the elements that come before it. Any element prior to the generator's current element is no longer available.
- Lists support forward and backward traversal. Generators provide only forward traversal (`reversed`, for example, does not work with a generator object).

If you need to create a sequence of values and random access is unnecessary, a generator may be the better choice. The generator's sequence behaves like a *lazy list*; that is, the sequence element exists only at the time it is needed. If, on the other hand, your program needs to have all the values of a sequence available at any time during the program's execution, a list is the necessary choice. Unlike generators, a list usually must be fully populated with all of its elements before it truly is useful to the program.

10.17 Structural Pattern Matching Revisited

Section 4.11 introduced Python's `match` statement. With our knowledge of lists, we can exploit more of the `match` statement's capabilities.

The pattern matching mechanism is *structural* and can extract information from an object. Listing 10.34 (`commandmatch.py`) allows the user to maintain a list of numbers.

Listing 10.34: commandmatch.py

```

def save(lst: list[int], filename: str) -> None:
    """ Writes the list of integers to a file named filename """
    with open(filename, 'w') as f:
        f.write(str(len(lst)) + '\n')
        for item in lst:
            f.write(str(item) + '\n')

def load(filename: str) -> list[int]:
    """ Returns the list of integers from the file named filename """
    result = []
    with open(filename, 'r') as f:
        n = int(f.readline())
        for i in range(n):
            result.append(int(f.readline().strip()))
    return result

def help() -> None:
    """ Displays information about available commands to the user """
    print('Commands:')
    print('  add <item>')
    print('  save <filename>')
    print('  load <filename>')
    print('  print')
    print('  help')
    print('  quit')

def main() -> None:
    """ Processes the user's commands from the keyboard. """
    data = []
    done = False
    while not done:
        command = input('==> ')
        match command.split():
            case ['add', argument]:
                data.append(int(argument))
            case ['save', argument]:
                save(data, argument)
            case ['load', argument]:
                data = load(argument)
            case ['print'] | ['P'] | ['p']:
                print(data)
            case ['help'] | ['?']:
                help()
            case ['quit']:
                done = True
            case ['q'] | ['Q']:
                print('Did you mean \'quit\'?')
            case _:
                print('Unrecognized command')
    print('Program finished')

```

```
if __name__ == '__main__':  
    main()
```

Suppose the user types the following command:

```
add 12
```

This input binds the `command` variable to the string

```
'add 12'
```

The expression

```
command.split()
```

in this case evaluates to the following list of strings:

```
['add', '12']
```

The `match` statement matches this list of strings to the following case:

```
case ['add', argument]:  
    data.append(int(argument))
```

The string `'add'` clearly matches the first element in the list `['add', '12']`, and so the `match` statement associates the second part of the pattern, the `argument` variable, with the string `'12'`. The `match` statement will automatically assign the string `'12'` to the variable `argument`. The action

```
data.append(int(argument))
```

then constructs the integer 12 from the string `'12'` and appends it to the list named `data`.

The *add*, *save*, and *load* commands require extra information, but the *print*, *help*, and *quit* commands do not. Notice that the structural pattern matching in the `match` statement relieves the programmer from writing the more complicated logic dealing with distinguishing between one- and two-word command entries. Note that the pattern for the *print* command matches the words *print*, *P*, and *p*. In order to quit the program, the user must type out the full word *quit*; an entry of *Q* or *q* informs the user of the need to type the full word. The catch-all wildcard pattern, `_`, alerts the user about inapplicable input.

To further demonstrate the power of Python's structural pattern matching, consider Listing 10.35 (`listlength.py`).

Listing 10.35: listlength.py

```

from typing import Any

def length(a: list[Any]) -> int:
    """ Returns the number of elements in list a """
    match a:
        case []:
            return 0
        case _:
            return 1 + length(a[1:])

if __name__ == '__main__':
    print(f'Should be 0: {length([])}')
    print(f'Should be 1: {length([3])}')
    print(f'Should be 1: {length(["A"])}')
    print(f'Should be 1: {length(["ABC"])}')
    print(f'Should be 1: {length([[]])}')
    print(f'Should be 2: {length(["ABC", "XYZ"])}')
    print(f'Should be 2: {length([2, 3])}')
    print(f'Should be 2: {length([[], []])}')
    print(f'Should be 3: {length([2, "ABC", 3])}')
    print(f'Should be 8: {length([[], [1], "ABC", 2.1, 9, 0, [1, 2], 4.5])}')

```

The `length` function in Listing 10.35 (`listlength.py`) computes the length of any list. It does so with calling the built-in `len` function and without using a loop to count the elements. (The built-in `len` function does the job quite nicely, so the purpose here of the `length` function is merely to illustrate a point.) The `[]` pattern matches only the empty list and nothing else. The `_` pattern matches anything, list or not. The type hint specifies that the parameter `a` must be a list, so the `_` wildcard is safe in a type-checked program. If the list is empty, the function returns zero; otherwise, it must contain at least one element. The recursive case counts the one element and then computes the length of the *rest* of the list using a slice.

Python's `match` statement offers several more advanced features. See <https://peps.python.org/pep-0636/> and <https://peps.python.org/pep-0622/> for more information.

10.18 Exercises

1. Can a Python list hold a mixture of integers and strings?
2. What happens if you attempt to access an element of a list using a negative index?
3. What Python statement produces a list containing the values 45, -3 , 16 and 8, in that order?
4. Given the statement

```
lst = [10, -4, 11, 29]
```

- (a) What expression represents the very first element of `lst`?
- (b) What expression represents the very last element of `lst`?
- (c) What is `lst[0]`?

- (d) What is `lst[3]`?
- (e) What is `lst[1]`?
- (f) What is `lst[-1]`?
- (g) What is `lst[-4]`?
- (h) Is the expression `lst[3.0]` legal or illegal?

5. Given the statements

```
lst = [3, 0, 1, 5, 2]
x = 2
```

evaluate the following expressions:

- (a) `lst[0]`?
- (b) `lst[3]`?
- (c) `lst[x]`?
- (d) `lst[-x]`?
- (e) `lst[x + 1]`?
- (f) `lst[x] + 1`?
- (g) `lst[lst[x]]`?
- (h) `lst[lst[lst[x]]]`?

6. What function returns the number of elements in a list?

7. What expression represents the empty list?

8. Given the list

```
lst = [20, 1, -34, 40, -8, 60, 1, 3]
```

evaluate the following expressions:

- (a) `lst`
- (b) `lst[0:3]`
- (c) `lst[4:8]`
- (d) `lst[4:33]`
- (e) `lst[-5:-3]`
- (f) `lst[-22:3]`
- (g) `lst[4:]`
- (h) `lst[:]`
- (i) `lst[:4]`
- (j) `lst[1:5]`
- (k) `-34 in lst`
- (l) `-34 not in lst`
- (m) `len(lst)`

9. An assignment statement containing the expression `a[m:n]` on the left side and a list on the right side can modify list `a`. Complete the following table by supplying the m and n values in the slice assignment statement needed to produce the indicated list from the given original list.

Original List	Target List	Slice indices	
		m	n
[2, 4, 6, 8, 10]	[2, 4, 6, 8, 10, 12, 14, 16, 18, 20]		
[2, 4, 6, 8, 10]	[-10, -8, -6, -4, -2, 0, 2, 4, 6, 8, 10]		
[2, 4, 6, 8, 10]	[2, 3, 4, 5, 6, 7, 8, 10]		
[2, 4, 6, 8, 10]	[2, 4, 6, 'a', 'b', 'c', 8, 10]		
[2, 4, 6, 8, 10]	[2, 4, 6, 8, 10]		
[2, 4, 6, 8, 10]	[]		
[2, 4, 6, 8, 10]	[10, 8, 6, 4, 2]		
[2, 4, 6, 8, 10]	[2, 4, 6]		
[2, 4, 6, 8, 10]	[6, 8, 10]		
[2, 4, 6, 8, 10]	[2, 10]		
[2, 4, 6, 8, 10]	[4, 6, 8]		

10. Write the list represented by each of the following expressions.

- (a) `[8] * 4`
- (b) `6 * [2, 7]`
- (c) `[1, 2, 3] + ['a', 'b', 'c', 'd']`
- (d) `3 * [1, 2] + [4, 2]`
- (e) `3 * ([1, 2] + [4, 2])`

11. Write the list represented by each of the following list comprehension expressions.

- (a) `[x + 1 for x in [2, 4, 6, 8]]`
- (b) `[10*x for x in range(5, 10)]`
- (c) `[x for x in range(10, 21) if x % 3 == 0]`
- (d) `[(x, y) for x in range(3) for y in range(4)]`
- (e) `[(x, y) for x in range(3) for y in range(4) if (x + y) % 2 == 0]`

12. Provide a list comprehension expression for each of the following lists.

- (a) `[1, 4, 9, 16, 25]`
- (b) `[0.25, 0.5, 0.75, 1.0, 1.25, 1.5]`
- (c) `[('a', 0), ('a', 1), ('a', 2), ('b', 0), ('b', 1), ('b', 2)]`

13. If `lst` is a list, what expression indicates whether or not `x` is a member of `lst`?

14. What does `reversed` do?

15. Complete the following function that adds up all the *positive* values in a list of integers. For example, if list `a` contains the elements 3, -3, 5, 2, -1, and 2, the call `sum_positive(a)` would evaluate to 12, since $3 + 5 + 2 + 2 = 12$. The function returns zero if the list is empty.

```
def sum_positive(a):
    # Add your code...
```

16. Complete the following function that counts the even numbers in a list of integers. For example, if list `a` contains the elements 3, 5, 4, -1, and 0, the call `count_evens(a)` would evaluate to 2, since `a` contains two even numbers: 4 and 0. The function returns zero if the list is empty. The function does not affect the contents of the list.

```
def count_evens(lst):
    # Add your code...
```

17. Write a function named `print_big_enough` that accepts two parameters, a list of numbers and a number. The function should print, in order, all the elements in the list that are at least as large as the second parameter.
18. Write a function named `next_number` that accepts a list of integer values. All the elements in the list are unique, and all elements in the list are greater than or equal to one. (The caller must ensure that these conditions are met before passing the list to `next_number`.) The `next_number` function should return the smallest positive integer **not** in the list. (Note that 1 is the smallest positive integer.)

As examples,

- `next_number([5, 3, 1])` would return 2
- `next_number([5, 4, 1, 2])` would return 3
- `next_number([2, 3])` would return 1
- `next_number([])` would return 1

19. Write a function named `reverse` that reorders the contents of a list so they are reversed from their original order. `a` is a list. Note that your function must physically rearrange the elements within the list, not just print the elements in reverse order.
20. Write a Python program that creates the matrix

```
1  1  1  1  1  1  1  1  1
1  1  1  1  1  1  1  1  1
1  1  1  1  1  1  1  1  1
1  1  1  1  1  1  1  1  1
1  1  1  1  1  1  1  1  1
1  1  1  1  1  1  1  1  1
```

and assigns it to the variable `m`. Pretty print `m` to ensure the contents are correct. Next, reassign `m[2][4]` to 0, and print `m` again to ensure your code modified the correct element.

21. Provide five different ways to create the list `[1, 2, 3, 4, 5, 6, 7, 8, 9, 10]` and assign it to the variable `lst`.
22. In a square 2D list the number of rows equals the number of columns. Write a function that accepts a square 2D list and returns `True` if the left to right contents of any row equals the top to bottom contents of any column. If no row matches any column, the function returns `False`.
23. We can represent a Tic-Tac-Toe board as a 3×3 grid in which each position can hold one of the following three strings: "X", "O", or " ". Write a function named `check_winner` that accepts a 3×3 list as a parameter. If "X" appears in a winning Tic-Tac-Toe pattern, the function should return the string "X". If "O" appears in a winning Tic-Tac-Toe pattern, the function should return the string "O". If no winning pattern exists, the function should return the string " ".

Chapter 11

Tuples, Dictionaries, and Sets

Lists, introduced in Chapter 10, are convenient data structures for representing sequences of data. In this chapter we examine several other ways that Python provides for storing aggregate data: tuples, dictionaries, and sets.

11.1 Tuples

Tuples are similar to lists, except tuples are immutable. Listing 11.1 (tupletest.py) compares the usage of lists versus tuples.

Listing 11.1: tupletest.py

```
my_list = [1, 2, 3, 4, 5, 6, 7]      # Make a list
my_tuple = (1, 2, 3, 4, 5, 6, 7)    # Make a tuple
print('The list:', my_list)         # Print the list
print('The tuple:', my_tuple)       # Print the tuple
print('The first element in the list:', my_list[0]) # Access an element
print('The first element in the tuple:', my_tuple[0]) # Access an element
print('All the elements in the list:', end=' ')
for elem in my_list:                # Iterate over the elements of a list
    print(elem, end=' ')
print()
print('All the elements in the tuple:', end=' ')
for elem in my_tuple:               # Iterate over the elements of a tuple
    print(elem, end=' ')
print()
print('List slice:', my_list[2:5])   # Slice a list
print('Tuple slice:', my_tuple[2:5]) # Slice a tuple
print('Try to modify the first element in the list . . .')
my_list[0] = 9                      # Modify the list
print('The list:', my_list)
print('Try to modify the first element in the list . . .')
my_tuple[0] = 9                     # Is tuple modification possible?
print('The tuple:', my_tuple)
```

Table 11.1 Python lists versus tuples

Feature	List	Tuple
Mutability	mutable	immutable
Creation	<code>lst = [i, j]</code>	<code>tpl = (i, j)</code>
Element access	<code>a = lst[i]</code>	<code>a = tpl[i]</code>
Element modification	<code>lst[i] = a</code>	<i>Not possible</i>
Element addition	<code>lst += [a]</code>	<i>Not possible</i>
Element removal	<code>del lst[i]</code>	<i>Not possible</i>
Slicing	<code>lst[i:j:k]</code>	<code>tpl[i:j:k]</code>
Slice assignment	<code>lst[i:j] = []</code>	<i>Not possible</i>
Iteration	<code>for elem in lst:...</code>	<code>for elem in tpl:...</code>

Listing 11.1 (tupletest.py) prints

```

The list: [1, 2, 3, 4, 5, 6, 7]
The tuple: (1, 2, 3, 4, 5, 6, 7)
The first element in the list: 1
The first element in the tuple: 1
All the elements in the list: 1 2 3 4 5 6 7
All the elements in the tuple: 1 2 3 4 5 6 7
List slice: [3, 4, 5]
Tuple slice: (3, 4, 5)
Try to modify the first element in the list . . .
The list: [9, 2, 3, 4, 5, 6, 7]
Try to modify the first element in the tuple . . .
Traceback (most recent call last):
  File "C:\code\python\tuple.py", line 21, in <module>
    my_tuple[0] = 9          # Is tuple modification possible?
    ~~~~~^~~~~
TypeError: 'tuple' object does not support item assignment

```

We see that Listing 11.1 (tupletest.py) does not run to completion. The next to the last statement in the program:

```
my_tuple[0] = 9
```

generates a run-time exception because tuples are immutable. Once we create tuple object, we cannot change that object's contents.

Table 11.1 compares lists to tuples.

Unlike with lists, we cannot modify an element within a tuple, we cannot add elements to a tuple, nor can we remove elements from a tuple. If we have a variable assigned to a tuple, we always can reassign that variable to a different tuple. Such an assignment simply binds the variable to a different tuple object—it does not modify the tuple to which the variable originally was bound.

The parentheses are optional in the following statement:

```
my_tuple = (1, 2, 3)
```

The following statement is equivalent:

```
my_tuple = 1, 2, 3
```

Lists can hold heterogeneous data types, and so too can tuples:

```
>>> t = (2, 'Fred', 41.2, [30, 20, 10])
>>> t
(2, 'Fred', 41.2, [30, 20, 10])
```

In general practice, however, many Python programmers favor storing only homogeneous types in lists and prefer tuples for holding heterogeneous types.

Section 7.1 introduced tuple unpacking. The following code unpacks a tuple into separate variables:

```
t = 3, 'A', 99
val, letter, quant = t
```

After this code executes `val` will refer to 3, `letter` will be assigned to the string `'A'`, and `quant` will be another name for the integer 99.

Tuple unpacking is convenient if you need to extract most, if not all, of the elements from a tuple. If you need only one element from a potentially large tuple, the index operator is a better choice, as shown here:

```
t = 3, 'A', 99
letter = t[1]
```

The second statement binds `letter` to the string `'A'`.

If you wish to extract several components from a tuple and ignore others, tuple extraction with “throw away” `_` variables (see Section 10.14) can be a good choice. The following code illustrates:

```
t = 3, 'A', 99, 16, 0, 42
_, letter, _, _, quant, rating = t
```

The nameless `_` variable will end up with the value 16, but this variable is meant to be ignored. The code that follows these statements would be interested only in the variables `letter`, `quant`, and `rating`.

We can convert a tuple to a list using the `list` constructor function, and the `tuple` constructor function performs the reverse conversion. The following interactive sequence demonstrates the use of the conversion functions:

```
>>> tpl = 1, 2, 3, 4, 5, 6, 7, 8
>>> tpl
(1, 2, 3, 4, 5, 6, 7, 8)
>>> list(tpl)
[1, 2, 3, 4, 5, 6, 7, 8]
>>> lst = ['a', 'b', 'c', 'd']
>>> lst
['a', 'b', 'c', 'd']
>>> tuple(lst)
('a', 'b', 'c', 'd')
```

Neither the `list` nor `tuple` function actually modifies its argument; that is, `tuple(lst)` does not modify `lst`, and `list(tpl)` does not modify `tuple` (since tuples are immutable, any modification would be impossible anyway). The `list` function makes a new list out of the contents of a tuple, and the `tuple` function makes a new tuple out of the elements in a list.

We can use the built-in `zip` function to generate a sequence of tuples from two lists. Consider the following interactive sequence:

```
>>> lst1 = [1, 2, 3, 4, 5, 6, 7, 8]
>>> lst2 = ['a', 'b', 'c', 'd', 'e', 'f', 'g', 'h']
>>> for t in zip(lst1, lst2):
...     print(t)
...
(1, 'a')
(2, 'b')
(3, 'c')
(4, 'd')
(5, 'e')
(6, 'f')
(7, 'g')
(8, 'h')
```

You can think of the `zip` function working like a physical zipper. A physical zipper pairs up two sets of interlocking (usually metal) teeth, closing an opening in a garment or bag. The Python `zip` function pairs up elements from two different sequences. The paired-up elements are tuples, and the sequences can be lists or sequences constructed from generators (see Section 8.9). If one of the sequences is shorter than the other, the `zip` function stops at the shorter sequence and ignores the remainder of the longer sequence.

Listing 11.2 (`zipseq.py`) constructs a sequence of tuples with their first elements derived from a list and their second elements obtained from a generator. Note that the generator's sequence is shorter than the list.

Listing 11.2: `zipseq.py`

```
from typing import Generator

def gen(n: int) -> Generator[int, None, None]:
    """ Generates the first n perfect squares, starting with zero:
        0, 1, 4, 9, 16,..., (n - 1)^2. """
    for i in range(n):
        yield i**2

z = zip([10, 20, 30, 40, 50, 60], gen(4))
print(z)

for p in z:
    print(p, end=' ')
print()
```

Listing 11.2 (`zipseq.py`) prints

```
<zip object at 0x0000019244C43A00>
(10, 0) (20, 1) (30, 4) (40, 9)
```

The `zip` function does not return a list; like `range`, it returns an object over which we can iterate. Like a generator, it serves its elements lazily as the `for` statement iterates over it. We can make a list from a `zip` object using the `list` conversion function:

```
>>> list(zip(range(5), range(10, 0, -1)))
[(0, 10), (1, 9), (2, 8), (3, 7), (4, 6)]
```

Unlike a physical zipper, the `zip` function can combine more than two sequences. Consider the following example:

```
>>> a = [1, 2, 3, 4, 5, 6, 7, 8]
>>> b = 'A', 'B', 'C', 'D', 'E'
>>> c = (x for x in range(14) if x % 2 == 0)
>>> a
[1, 2, 3, 4, 5, 6, 7, 8]
>>> b
('A', 'B', 'C', 'D', 'E')
>>> c
<generator object <genexpr> at 0x000001BAC073DF20>
>>> zip(a, b, c)
<zip object at 0x000001BAC08D5280>
>>> list(zip(a, b, c))
[(1, 'A', 0), (2, 'B', 2), (3, 'C', 4), (4, 'D', 6), (5, 'E', 8)]
```

We can use the `zip` function and list comprehension to build elaborate lists. Suppose we wish to make a new list from two existing lists. The first element in our new list will be the sum of the first elements from the two original lists. Similarly, the second element in our new list will be the sum of the second elements in the two original lists, and so forth. We can use `zip` to pair up the elements, as the following interactive sequence illustrates:

```
>>> for p in zip([1, 2, 3, 4, 5], [10, 11, 12, 13, 14]):
...     print(p)
...
(1, 10)
(2, 11)
(3, 12)
(4, 13)
(5, 14)
```

We want to add together the components of each tuple. To print each sum we could write

```
>>> for (x, y) in zip([1, 2, 3, 4, 5], [10, 11, 12, 13, 14]):
...     print(x + y)
...
11
13
15
17
19
```

We can reassemble these pieces into a list comprehension to build our list of sums:

```
>>> [x + y for (x, y) in zip([1, 2, 3, 4, 5], [10, 11, 12, 13, 14])]
[11, 13, 15, 17, 19]
```

When treated as a Boolean expression, the empty tuple `()` is interpreted as `False`, and any other tuple is considered `True`.

Since they are so similar, why does Python have both lists and tuples? Under some circumstances an executing program can perform optimizations on immutable objects that would be impossible with mutable objects. These optimizations can increase the program's performance. Also, it is easier in general to reason about the behavior of programs that use immutable objects. The fact that some objects cannot change makes it easier to understand how a section of code works, or, during debugging, why the section of code does not work.

11.2 Arbitrary Argument Lists

We can easily write a function that adds two numbers; consider the following `sum` function:

```
def sum(a: float, b: float) -> float:
    return a + b
```

What if we need `sum` to be flexible enough to add two *or* three numbers? We can implement such a function with default arguments (Section 8.2). Listing 11.3 (`add2or3.py`) illustrates such a function

Listing 11.3: `add2or3.py`

```
def sum(a: float, b: float, c: float=0) -> float:
    return a + b + c    # Adding zero will not affect a + b

print(sum(3, 4))
print(sum(3, 4, 5))
```

The `sum` function in Listing 11.3 (`add2or3.py`) handles two and three arguments equally well:

```
7
12
```

A function that is capable of adding up to five numbers is equally easy:

```
def sum(a: float, b: float=0, c: float=0,
        d: float=0, e: float=0) -> float:
    return a + b + c + d + e
```

Suppose we wish to write a `sum` function that can add as many numbers as the caller can supply? The default parameter approach is not practical in this situation. Our function must be able to handle 1,000 numbers or more, and these numbers must be separate arguments—not a single list containing 1,000 numbers.

When we define a function, we specify the individual parameters it accepts, providing default values as needed. In the function definitions we have seen to this point the number of parameters is fixed. We need a way to define a function in such a way so that it can accept an arbitrary number of parameters. Fortunately, Python has a mechanism for specifying that a function can accept an arbitrary number of parameters. Listing 11.4 (`addmany.py`) illustrates how to write such a function.

Listing 11.4: `addmany.py`

```
def sum(*nums: float) -> float:
    s = 0                # Initialize sum to zero
    for num in nums:     # Consider each argument passed to the function
        s += num         # Accumulate their values
    return s             # Return the sum

print(sum(3, 4))
print(sum(3, 4, 5))
print(sum(3, 3, 3, 3, 4, 1, 9, 44, -2, 8, 8))
```

The `sum` function in Listing 11.4 (`addmany.py`) handles as many actual parameters as the client can provide; the program prints

```
7
12
84
```

The single asterisk (*) before the formal parameter `nums` indicates the parameter is not necessarily a single value but potentially a collection of values. Listing 11.5 (`addmanyaugmented.py`) reveals what really is going on behind the scenes.

Listing 11.5: `addmanyaugmented.py`

```
def sum(*nums: float) -> float:
    print('-->', nums)          # See what nums really is
    print('-->', type(nums))    # See what type nums has
    s = 0                       # Initialize sum to zero
    for num in nums:            # Consider each argument passed to the function
        s += num                # Accumulate their values
    return s                    # Return the sum

print(sum(3, 4))
print(sum(3, 4, 5))
print(sum(3, 3, 3, 3, 4, 1, 9, 44, -2, 8, 8))
```

The `sum` function in Listing 11.5 (`addmanyaugmented.py`) prints the following:

```
--> (3, 4)
--> <class 'tuple'>
7
--> (3, 4, 5)
--> <class 'tuple'>
1 2
--> (3, 3, 3, 3, 4, 1, 9, 44, -2, 8, 8)
--> <class 'tuple'>
84
```

Listing 11.5 (`addmanyaugmented.py`) exposes the fact that the formal parameter `nums` is a tuple wrapping all the actual parameters sent by the caller. Since `nums` is simply a tuple, we can iterate over it with the `for` statement to extract all the actual parameters provided by the caller.

A function definition may contain at most one of these parameters that represents a tuple of arguments, and, if present, this parameter must appear after all the named, single formal parameters, if any. In the following `sum` function, callers must provide at least two parameters but may pass more:

```
def sum(num1: float, num2: float, *extranums: float) -> float:
    s = num1 + num2
    for n in extranums:
        s += n
    return s
```

Note that the formal parameters `num1` and `num2` must appear before `*extranums` in `sum`'s formal parameter list.

As we have seen, a formal parameter declared with an asterisk is really a tuple from the function's

perspective. The caller can pass individual arguments not packed within a tuple. The Python interpreter takes care of the packing the caller's arguments into a tuple during the call.

This process works in reverse also. Consider the following function that accepts four parameters:

```
def f(a, b, c, d):  
    print(f'a = {a}, b = {b}, c = {c}, d = {d}')
```

We can pass a single argument to function `f` if we express it in the proper way:

```
args = (10, 20, 30, 40)  
f(*args)
```

The variable `args` is a tuple, but `f` does not accept a single tuple; it accepts four parameters. Expressing the actual parameter as `*args` enables the interpreter to unpack the tuple into the four parameters the function expects. Note that the tuple must contain exactly the number of parameters that the function expects.

Given the definition of function `f` above, the following call is legal:

```
f(*(10, 20, 30, 40))    # Legal, unpacks the tuple into separate args
```

but the following is illegal:

```
f((10, 20, 30, 40))    # Illegal, function does not accept a tuple
```

because the actual parameter is a single object—a tuple—and the function requires four parameters, not just one. We must use the `*` tuple unpacking operator during the call. Of course the easiest way to call the function in this case is simply

```
f(10, 20, 30, 40)    # Why use a literal tuple anyway?
```

Python supports a concept known as *generalized unpacking*. It extends simple unpacking by using the asterisk to represent a collection of elements not covered by individual variables during the unpacking. The following interactive sequence shows how generalized unpacking can extract a prefix of a tuple, storing the remainder of the tuple's elements in a list:

```
>>> a = 1, 2, 3, 4, 5, 6, 7, 8  
>>> a  
(1, 2, 3, 4, 5, 6, 7, 8)  
>>> x, y, *rest = a  
>>> x  
1  
>>> y  
2  
>>> rest  
[3, 4, 5, 6, 7, 8]
```

Curiously, the elements in the remainder of the tuple are copied into a list, not a tuple. This is because once an element is unpacked from a tuple, the programmer is free to reassign it to a different object. In the above example, we can reassign `x` to a different value. If `rest` in the above example were a tuple, we could not modify its contents.

Prefer extracting a suffix of the tuple instead? Try the following:

```
>>> a = 1, 2, 3, 4, 5, 6, 7, 8
>>> a
(1, 2, 3, 4, 5, 6, 7, 8)
>>> *start, x, y = a
>>> start
[1, 2, 3, 4, 5, 6]
>>> x
7
>>> y
8
```

The next sequence unpacks a prefix and a suffix of the elements:

```
>>> a = 1, 2, 3, 4, 5, 6, 7, 8
>>> a
(1, 2, 3, 4, 5, 6, 7, 8)
>>> x1, x2, *middle, x3, x4 = a
>>> x1
1
>>> x2
2
>>> middle
[3, 4, 5, 6]
>>> x3
7
>>> x4
8
>>> a
(1, 2, 3, 4, 5, 6, 7, 8)
```

At most one `*` expression may appear in a generalized unpacking expression.

Generalized unpacking works with lists as well:

```
>>> a = [1, 2, 3, 4, 5, 6, 7, 8]
>>> a
[1, 2, 3, 4, 5, 6, 7, 8]
>>> x1, x2, *midlist, x3, x4 = a
>>> x1
1
>>> x2
2
>>> midlist
[3, 4, 5, 6]
>>> x3
7
>>> x4
8
>>> x, y, z = ['a', 'b', 'c']
>>> x
'a'
>>> y
'b'
>>> z
'c'
```

The `print` function knows how to handle a list argument, and it also accepts a variable number of arguments. Consider Listing 11.6 (`speciallistprinting.py`) which demonstrates the power of the `*` unpacking operator for flexible list printing:

Listing 11.6: `speciallistprinting.py`

```
lst = [2*i for i in range(6)]

# Typical list printing
print(lst)

# Print just the list elements
print(*lst)

# Print the list in a special way
print(*lst, sep=" and ", end="--that's all folks!\n")
```

Listing 11.6 (`speciallistprinting.py`) prints

```
[0, 2, 4, 6, 8, 10]
0 2 4 6 8 10
0 and 2 and 4 and 6 and 8 and 10--that's all folks!
```

While we are on the subject of unpacking tuples, we can unpack a tuple directly from a `range` object without the need to involve a `for` statement:

```
>>> x, y, z = range(10, 31, 10)
>>> x
10
>>> y
20
>>> z
30
```

You must ensure the number of values match exactly:

```
>>> x, y, z = range(0, 100)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
ValueError: too many values to unpack (expected 3)
>>> x, y, z = range(0, 2)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
ValueError: not enough values to unpack (expected 3, got 2)
```

Python's `match` statement recognizes tuple and list unpacking patterns. Listing 11.7 (`listcountevens.py`) counts the number of even numbers in a list of integers.

Listing 11.7: listcountevens.py

```

def count_evens(a: list[int]) -> int:
    """ Returns the number of elements in list a """
    match a:
        case []:
            return 0
        case x, *y: # Matches the first element and rest of list
            return (x + 1) % 2 + count_evens(y)
    return 0

if __name__ == '__main__':
    print(f'[] should be 0: {count_evens([])}')
    print(f'[3] should be 0: {count_evens([3])}')
    print(f'[16] should be 1: {count_evens([16])}')
    print(f'[3, -7] should be 0: {count_evens([3, -7])}')
    print(f'[-40, -7] should be 1: {count_evens([-40, -7])}')
    print(f'[11, 4] should be 1: {count_evens([11, 4])}')
    print(f'[12, 4] should be 2: {count_evens([12, 4])}')
    print(f'[12, 0, 4] should be 3: {count_evens([12, 0, 4])}')
    print(f'[11, 0, 4] should be 2: {count_evens([11, 0, 4])}')
    print(f'[12, 11, 4] should be 2: {count_evens([12, 11, 4])}')
    print(f'[12, 10, 5] should be 2: {count_evens([12, 10, 5])}')
    print(f'[1, 10, 5] should be 1: {count_evens([1, 10, 5])}')
    print(f'[10, 1, 5] should be 1: {count_evens([10, 1, 5])}')
    print(f'[1, 3, 4] should be 1: {count_evens([1, 3, 4])}')
    print(f'[1, 3, 5] should be 0: {count_evens([1, 3, 5])}')

```

Within the match statement, the pattern `[]` matches the empty list. The pattern

```

x, *rest
\end{code}

```

unpacks a list into its first element
and the rest of the list apart from its first element; for example,
given the list

```

\begin{code}
[10, 20, 30, 40]

```

the pattern `x, *rest` binds `x` to 10 and `rest` to `[20, 30, 40]`.

The expression

```
(x + 1) % 2
```

evaluates to 0 if `x` is odd because an odd number plus one is always an even number. Divide any even number by two, and the remainder is zero. In contrast, if `x` is even, the expression evaluates to one. This expression and the recursive call allows the `listcountevens` function to count the number of even integers in a given list.

11.3 Dictionaries

Lists and tuples are convenient for storing collections of data, but they have some limitations. For one, we locate an element within a list or tuple based on its position, or index. While this approach is fine for many applications, in other situations this access-by-index approach is awkward or inefficient.

A Python *dictionary* is an *associative container* which permits access based on a *key*, rather than an index. Unlike an index, a key is not restricted to an integer expression. The following interactive sequence builds a simple dictionary that uses string keys:

```
>>> d = {} # Make an empty dictionary
>>> d
{}
>>> d['Fred'] = 44 # Add an element
>>> d
{'Fred': 44}
>>> d['Ella'] = 31 # Add another element
>>> d
{'Fred': 44, 'Ella': 31}
>>> print(d)
{'Fred': 44, 'Ella': 31}
>>> d['Fred']
44
>>> d['Ella']
31
>>> d['Zach']
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
KeyError: 'Zach'
>>> d[0]
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
KeyError: 0
```

Notice that, unlike a list which uses square brackets (`[]`), the contents of a dictionary appear within curly braces (`{}`). To access an element within a dictionary, however, we use square brackets exactly as we would with a list. In a dictionary, every *key* has an associated *value*. The dictionary `d` from the interactive sequence above pairs the key `'Fred'` with the value 44. It also pairs the key `'Ella'` with the value 31.

The following assignment statement associates a value with a key:

```
d['Fred'] = 44 # Associate value 44 with key 'Fred'
```

The string `'Fred'` is the key, and 44 is its associated value. If the key within the square brackets does not exist in the dictionary, the statement adds the key and pairs it with the value on the right of the assignment operator. If the key already exists in the dictionary, the statement replaces the value previously associated with the key with the new value on the right of the assignment operator.

Accessing a value with a given key is a different story. In the statement

```
x = d['Fred']
```

`'Fred'` must be a valid key in dictionary `d`, or the program will raise an exception. A valid key is a key that is present in the dictionary. At the end of the interaction sequence above, `'Fred'` is a valid key but

'Zach' is not. We see the interpreter's reaction when we attempt to use an invalid key: the interpreter generates a `KeyError` exception.

We can check to see if a key is present in a dictionary with the `in` operator:

```
if 'Fred' in d:           # Check to see if 'Fred' is a valid key
    print(d['Fred'])      # Print the value associated with key 'Fred'
else:
    print('\n' + 'Fred' + '\n' is not a key in d') # Warn user of missing key
```

A dictionary key may be of any immutable type. This means all of the following types can serve as keys within a dictionary: integers, floating-point numbers, strings, Booleans, and tuples. Since lists are mutable objects, a list may not be a key. A dictionary is a mutable object, so a dictionary cannot use itself or another dictionary object as a key. A value within a dictionary may be any valid Python type, immutable or mutable.

The keys within a given dictionary may be of mixed types; consider the following interactive sequence:

```
>>> s = {}
>>> s[8] = 44
>>> s['Alpha'] = 'up'
>>> s['Alpha']
'up'
>>> s[True] = 'right'
>>> s[True]
'right'
>>> s[10 < 20]
'right'
>>> s['Beta'] = 100
>>> s
{8: 44, 'Alpha': 'up', True: 'right', 'Beta': 100}
>>> s[3.4] = True
>>> s
{8: 44, 'Alpha': 'up', True: 'right', 'Beta': 100, 3.4: True}
>>> s[2 == -2] = 'wrong'
>>> s[False]
'wrong'
>>> s
{8: 44, 'Alpha': 'up', True: 'right', 'Beta': 100, 3.4: True,
False: 'wrong'}
>>> x = 8
>>> s[x]
44
>>> y = 15
>>> s[y] = 'down'
>>> lst = [1, 2, 3]
>>> s[17] = lst
>>> s
{8: 44, 'Alpha': 'up', True: 'right', 'Beta': 100, 3.4: True,
False: 'wrong', 15: 'down', 17: [1, 2, 3]}
>>> print(s)
{8: 44, 'Alpha': 'up', True: 'right', 'Beta': 100, 3.4: True,
False: 'wrong', 15: 'down', 17: [1, 2, 3]}
```

This interactive sequence reveals several dictionary characteristics:

- The keys in a dictionary may have different types

- The values in a dictionary may have different types
- The values in a dictionary may be mutable objects
- The *key:value* pairs in a dictionary are ordered by their insertion order. (Note that earlier versions of Python did not guarantee element ordering within a dictionary; the key-value pairs could appear in any order.)

We can initialize a dictionary using the same syntax as the output that the `print` function displays. The following code fragment populates the dictionary `d` with four *key:value* entries:

```
d = {'Fred': 44, 'Ella': 39, 'Owen': 40, 'Zoe': 41}
print(d)
```

It prints the following:

```
{'Fred': 44, 'Ella': 39, 'Owen': 40, 'Zoe': 41}
```

The `keys` method of the dictionary class returns a sequence of all the keys in `d`. The following code demonstrates:

```
d = {'Fred': 44, 'Ella': 39, 'Owen': 40, 'Zoe': 41}
for k in d.keys():
    print(k, end=' ')
print()
```

Upon one execution of this code it prints

```
Fred Ella Owen Zoe
```

The following code behaves similarly:

```
d = {'Fred': 44, 'Ella': 39, 'Owen': 40, 'Zoe': 41}
for k in d:
    print(k, end=' ')
print()
```

This code produces the same output as iterating over `d.keys()`:

```
Fred Ella Owen Zoe
```

The `values` method of the dictionary class returns a sequence of all the values in `d`. The following code illustrates:

```
d = {'Fred': 44, 'Ella': 39, 'Owen': 40, 'Zoe': 41}
for v in d.values():
    print(v, end=' ')
print()
```

This code prints

```
44 39 40 41
```

We can obtain a sequence of tuples of *key:value* pairs of a dictionary with the `items` method:

```
d = {'Fred': 44, 'Ella': 39, 'Owen': 40, 'Zoe': 41}
for k, v in d.items():
    print(k, v)
```

This code fragment prints

```
Fred 44
Ella 39
Owen 40
Zoe 41
```

Suppose we have the list `['Fred', 'Ella', 'Owen', 'Zoe']` and the list `[4174, 2287, 5003, 2012]`. We know we can zip them together into a sequence of tuples using `zip` (Section 11.1). We can use the `dict` function to create a dictionary of *key:value* pairs formed from the tuples, as the following interactive sequence shows:

```
>>> names = ['Fred', 'Ella', 'Owen', 'Zoe']
>>> numbers = [4174, 2287, 5003, 2012]
>>> names
['Fred', 'Ella', 'Owen', 'Zoe']
>>> numbers
[4174, 2287, 5003, 2012]
>>> d = dict(zip(names, numbers))
>>> d
{'Fred': 4174, 'Ella': 2287, 'Owen': 5003, 'Zoe': 2012}
```

The elements of the list specified as the first actual parameter to `zip` become the dictionary keys, and the elements of the list specified as the second argument to `zip` form the values in the dictionary. The `zip` function pairs the first element from the `names` list with the first element of the `numbers` list, the second element from `names` with the second element of `numbers`, and so forth.

A dictionary is sometimes called an *associative array* because it associates its elements (values) with keys instead of indices. The placement and lookup of an element within a dictionary uses a process known as *hashing*. A hash function maps a key to a location within the dictionary where the key's associated value resides. A Python dictionary is a kind of *hash table*. See http://en.wikipedia.org/wiki/Hash_table for more information about hash functions and hash tables. The important thing to know about the hashing process is that it makes value lookup via a key very fast.

When treated as a Boolean expression, the empty dictionary (`{}`) is interpreted as `False`, and any other dictionary is considered `True`.

11.4 Using Dictionaries

You should use a dictionary when you need fast and convenient access to an element of a collection based on a search key rather than an index. Consider the problem of implementing a simple telephone contact list. Most people are very familiar with the names of their friends, family, and business contacts but can remember only a handful of telephone numbers. A contact list associates a name with a telephone number.

It would be inappropriate to place the names in a list and locate a name using the associated phone number as an index into the list. This look-up method is backwards—we do not want to find a name given a phone number; we want to look up a number based on a name. Besides, each phone number contains many

digits, and we would not need or want to have a list with indices with values that large—most of the space in the list would be unused.

In our situation, a person or company's name is a unique identifier for that contact. In this case the name is a *key* to that contact. A Python dictionary is the ideal data structure for mapping keys to values. A dictionary allows for the fast retrieval of a value given its associated key. Listing 11.8 (phonelist.py) uses a Python dictionary to implement a simple telephone contact database with a rudimentary command line interface.

Listing 11.8: phonelist.py

```
def save(filename: str, contact_list: dict[str, str]) -> None:
    with open(filename, 'w') as f:
        # Store number of elements
        f.write(f'{len(contact_list)}\n')
        # Store each element
        for key, value in contact_list.items():
            f.write(f'{key}\n')
            f.write(f'{value}\n')

def load(filename: str) -> dict[str, str]:
    import os
    contacts = {} # Empty current
    if os.path.exists(filename): # Does the file exist?
        with open(filename, 'r') as f:
            number = int(f.readline().strip())
            while (number):
                key = f.readline().strip()
                value = f.readline().strip()
                contacts[key] = value
                number -= 1
    else:
        print(f'File {filename} does not exist')
    return contacts

def main() -> None:
    contacts = {} # The global telephone contact list
    running = True
    while running:
        command = input('A)dd D)delete F)ind S)ave L)oad P)rint Q)uit: ')
        if command == 'A' or command == 'a':
            name = input('Enter new name:')
            print('Enter phone number for', name, end=':')
            number = input()
            contacts[name] = number
        elif command == 'D' or command == 'd':
            name = input('Enter name to delete:')
            if name in contacts:
                del contacts[name]
        elif command == 'F' or command == 'f':
            name = input('Enter name:')
            if name in contacts:
```

```

        print(name, contacts[name])
    else:
        print(f'Cannot find {name}')
    elif command == 'S' or command == 's':
        name = input('Enter filename to save contact list:')
        save(name, contacts)
    elif command == 'L' or command == 'l':
        name = input('Enter filename to load contact list:')
        contacts = load(name)
    elif command == 'P' or command == 'p':
        if len(contacts) > 0:
            for k, v in contacts.items():
                print(k, v)
        else:
            print('Contact list is empty')
    elif command == 'Q' or command == 'q':
        running = False
    elif command == 'dump': # Secret command
        print(contacts)
    else:
        print(command, 'is not a valid command')

if __name__ == '__main__':
    main()

```

The type hint for a dictionary with string keys and string values is `dict[str, str]`.

The following shows a sample run of Listing 11.8 (`phonelist.py`), where the goal of the session is to create a new contact list:

```

A)dd D)delete F)ind S)ave L)oad P)rint Q)uit: a
Enter new name:Joe
Enter phone number for Joe:1212832
A)dd D)delete F)ind S)ave L)oad P)rint Q)uit: a
Enter new name:Mary
Enter phone number for Mary:4327865
A)dd D)delete F)ind S)ave L)oad P)rint Q)uit: a
Enter new name:Sue
Enter phone number for Sue:4237856
A)dd D)delete F)ind S)ave L)oad P)rint Q)uit: s
Enter filename to save contact list:friends.txt
A)dd D)delete F)ind S)ave L)oad P)rint Q)uit: q

```

This run creates a file named `friends.txt` with the following content:

```

3
Joe
1212832
Mary
4327865
Sue
4237856

```

The first line represents the number of contacts in the file. The lines that follow contain the names and associated numbers.

The following run of Listing 11.8 (phonelist.py) loads in the contact list created earlier and adds a new contact:

```
A)dd D)delete F)ind S)ave L)oad P)rint Q)uit: p
Contact list is empty
A)dd D)delete F)ind S)ave L)oad P)rint Q)uit: l
Enter filename to load contact list:work.txt
File work.txt does not exist
A)dd D)delete F)ind S)ave L)oad P)rint Q)uit: l
Enter filename to load contact list:friends.txt
A)dd D)delete F)ind S)ave L)oad P)rint Q)uit: p
Joe 1212832
Mary 4327865
Sue 4237856
A)dd D)delete F)ind S)ave L)oad P)rint Q)uit: a
Enter new name:Zoe
Enter phone number for Zoe:3321184
A)dd D)delete F)ind S)ave L)oad P)rint Q)uit: p
Joe 1212832
Mary 4327865
Sue 4237856
Zoe 3321184
A)dd D)delete F)ind S)ave L)oad P)rint Q)uit: s
Enter filename to save contact list:friends.txt
A)dd D)delete F)ind S)ave L)oad P)rint Q)uit: q
```

Listing 11.9 (translateif.py) translates some Spanish words into English.

Listing 11.9: translateif.py

```
word = '*' # Initial word to ensure loop entry
while word != '': # Loop until user presses return by itself
    # Obtain word from the user
    word = input('Enter Spanish word:')
    if word == 'uno':
        print('one')
    elif word == 'dos':
        print('two')
    elif word == 'tres':
        print('three')
    elif word == 'cuatro':
        print('four')
    elif word == 'cinco':
        print('five')
    elif word == 'seis':
        print('six')
    elif word == 'siete':
        print('seven')
    elif word == 'ocho':
        print('eight')
    else: # Unknown word
        print('???')
```

Listing 11.9 (`translateif.py`) can successfully translate eight Spanish words into English. If we wish to increase the program's vocabulary, we must modify the program's logic by adding another `elif` block for each new word. Listing 11.10 (`translatedictionary.py`) uses a dictionary to assist the translation.

Listing 11.10: `translatedictionary.py`

```
translator = {'uno': 'one',
              'dos': 'two',
              'tres': 'three',
              'cuatro': 'four',
              'cinco': 'five',
              'seis': 'six',
              'siete': 'seven',
              'ocho': 'eight'}

word = '*'
while word != '': # Loop until user presses return by itself
    # Obtain word from the user
    word = input('Enter Spanish word:')
    if word in translator:
        print(translator[word])
    else:
        print('???') # Unknown word
```

We do not need to touch the program's logic at all to expand the program's vocabulary; all we need do is add the appropriate *key:value* item to the dictionary. This is a significant difference if wish to include enough words to make the program practical.

11.5 Counting with Dictionaries

Dictionaries are useful for counting things. We have experience using variables to count; recall Listing 5.3 (`countup.py`), Listing 5.15 (`countvowels.py`), Listing 5.35 (`startree.py`), Listing 6.7 (`measureprimespeed.py`), Listing 7.20 (`treefunc.py`), or Listing 10.25 (`timeprimes.py`). These programs all have counted one thing at a time, so they each use just one counter variable. In general, we need to use a separate variable for each count we manage. The following code counts the number of negative and nonnegative numbers in a list of numbers and returns a tuple with the results:

```
def count_neg_nonneg(nums: list[int]) -> tuple[int, int]:
    # Initialize counters
    neg_count, nonneg_count = 0, 0
    for num in nums:
        if num < 0:
            neg_count += 1
        else:
            nonneg_count += 1
    return neg_count, nonneg_count
```

Since we needed to count two different kinds of things, we had to use two separate counter variables.

What if we face a situation in which we must count multiple kinds of things, but we cannot know ahead of time how many kinds of things there will be to count? How can we determine how many counter variables to use in a program that attempts to solve such a problem?

The answer is this: We cannot know how many counter variables we will need, so we must use a different approach. If all the things we need to count are immutable objects, like numbers or strings, we can use the objects as keys in a dictionary and associate with each key a count. As a concrete example, Listing 11.11 (wordcount.py) reads the content of a text file containing words. After reading the file the program prints a count of each word. To simplify things, the text file contains only words with no punctuation. The user supplies the file name on the command line when launching the program (see Section 10.12).

Listing 11.11: wordcount.py

```
""" Uses a dictionary to count the number of occurrences of each
    word in a text file. """

import sys      # For sys.argv global command line arguments list

def main() -> None:
    """ Counts the words in a text file. """
    if len(sys.argv) < 2:  # Did the user not supply a file name?
        print('Usage: python wordcount <filename>')
        print('       where <filename> is the name of a text file.')
    else:  # User provided file name
        filename = sys.argv[1]
        counters = {}      # Initialize counting dictionary
        with open(filename, 'r') as f:  # Open the file for reading
            content = f.read()  # Read in content of the entire file
            words = content.split()  # Make list of individual words
            for word in words:
                word = word.upper()  # Make the word all caps
                if word not in counters:
                    counters[word] = 1  # First occurrence, add the counter
                else:
                    counters[word] += 1  # Increment existing counter
            # Report the counts for each word
            for word, count in counters.items():
                print(word, count)

if __name__ == '__main__':
    main()
```

The following paragraph appears in the *Declaration of Independence of the United States* (all punctuation has been removed):

When in the Course of human events it becomes necessary for one people to dissolve the political bands which have connected them with another and to assume among the powers of the earth the separate and equal station to which the Laws of Nature and of Nature's God entitle them a decent respect to the opinions of mankind requires that they should declare the causes which impel them to the separation

The following shows a sample run of Listing 11.11 (wordcount.py) when presented with the above text file:

```
WHEN 1
IN 1
THE 9
COURSE 1
OF 5
HUMAN 1
EVENTS 1
IT 1
BECOMES 1
NECESSARY 1
FOR 1
ONE 1
PEOPLE 1
TO 5
DISSOLVE 1
POLITICAL 1
BANDS 1
WHICH 3
HAVE 1
CONNECTED 1
THEM 3
WITH 1
ANOTHER 1
AND 3
ASSUME 1
AMONG 1
POWERS 1
EARTH 1
SEPARATE 1
EQUAL 1
STATION 1
LAWS 1
NATURE 1
NATURE'S 1
GOD 1
ENTITLE 1
A 1
DECENT 1
RESPECT 1
OPINIONS 1
MANKIND 1
REQUIRES 1
THAT 1
THEY 1
SHOULD 1
DECLARE 1
CAUSES 1
IMPEL 1
SEPARATION 1
```

In Listing 11.11 (`wordcount.py`), since we cannot predict what words will appear in the document, we cannot use a separate variable for each counter. Instead, we use the document's words as keys in a dictionary. For each key in the dictionary we associate an integer value that keeps track of the number of times the word appears in the file.

The following expression in Listing 11.11 (`wordcount.py`):

```
content.split()
```

exercises a method of the `str` class that separates the very long string composed of all the words in the file into separate strings. The `split` method divides the string based on whitespace (spaces, tabs, and newlines) and returns the individual words in a list. The following interactive sequence shows how the `split` method works:

```
>>> s = '   ABC   def GHI JKLM-nop, aaa  '
>>> s
'   ABC   def GHI JKLM-nop, aaa  '
>>> s.split()
['ABC', 'def', 'GHI', 'JKLM-nop,', 'aaa']
```

With no arguments, `str.split` method splits the string based on whitespace (spaces, tabs, and newlines). Whitespace separates the words to place in the list. The `split` function accepts an optional string parameter that contains the characters used to separate the words (or *tokens*); for example,

```
>>> x = 'ABC:xyz.122:prst'
>>> x
'ABC:xyz.122:prst'
>>> x.split()
['ABC:xyz.122:prst']
>>> x.split(':')
['ABC', 'xyz.122', 'prst']
```

We can put the string `split` method to good use with list comprehension (see Section 10.13). Suppose we have string containing comma-separated integer data. The string could contain spaces as well. The following represents a typical string:

```
s = "  85, 54 , 13,   17, 44  ,31, 80, 35,    30, 54,  78  "
```

Notice that the location of the spaces is arbitrary, but commas faithfully separate one integer from another. We wish to produce from this string a list containing its integers in the order they appear within the string:

```
[85, 54, 13, 17, 44, 31, 80, 35, 30, 54, 78]
```

The interactive interpreter reveals how we can split the string into its comma-separated components:

```
>>> s = "  85, 54 , 13,   17, 44  ,31, 80, 35,    30, 54,  78  "
>>> s.split(",")
['  85', ' 54 ', ' 13', '   17', ' 44  ', '31', ' 80', ' 35',
'   30', ' 54', ' 78  ']
```

This leaves us a list of strings, many containing extraneous leading or trailing spaces (or both). We need to remove the spaces with the string `strip` method and then convert the results to integers. The following interactive session shows how to use a one-line list comprehension to do exactly what we need:

```
>>> s = "  85, 54 , 13,   17, 44  ,31, 80, 35,    30, 54,  78  "
>>> s
'  85, 54 , 13,   17, 44  ,31, 80, 35,    30, 54,  78  '
>>> int_list = [int(x.strip()) for x in s.split(",")]
>>> int_list
[85, 54, 13, 17, 44, 31, 80, 35, 30, 54, 78]
```

Consider using a list comprehension expression similar to this one when writing code to read numerical data from a text file. Consider a very common task: reading comma-separated integer data from a text file. Spaces and newlines can litter the file at random, but commas definitely separate the numbers. The following file named `monitor.data` is typical:

```
92,50 , 77, 50 , 40,
36 , 52, 92,77, 77 , 93 ,
14, 42,
19, 52 ,75 ,
42, 23 ,14
```

This file contains five lines of text. Each line may have trailing spaces that are not visible in the listing above, and each line certainly has a newline (`'\n'`) at its end. We must strip the newline from each line and strip the final comma and trailing spaces. The string `rstrip` will accomplish this end-of-line clean up with the match string `' ,\n'` (space, comma, newline). Listing 11.12 (`readtextfile.py`) provides the complete code.

Listing 11.12: `readtextfile.py`

```
def readfile(filename: str) -> list[str]:
    """ Read the comma-separated integer data from the text file
        named filename and return the data in a list. """
    result = [] # List initially empty
    with open(filename, "r") as f:
        for line in f:
            # Remove any trailing spaces, comma, and newline
            result += [int(x.strip()) for x in line.rstrip(" ,\n").split(",")]
    return result

def main() -> None:
    lst = readfile("monitor.data")
    print(lst)

if __name__ == "__main__":
    main()
```

Listing 11.12 (`readtextfile.py`) reads in the file `monitor.data` and prints

```
[92, 50, 77, 50, 40, 36, 52, 92, 77, 77, 93, 14, 42, 19, 52, 75, 42, 23, 14]
```

11.6 Grouping with Dictionaries

Dictionaries are useful for grouping items. Like Listing 11.11 (`wordcount.py`), Listing 11.13 (`groupwords.py`) reads in the contents of a text file. Instead of counting the words, Listing 11.13 (`groupwords.py`) groups the words into lists based on the number of letters in the word. All the words containing only one letter are in one list, all the words containing two letters are in another list, etc.

Listing 11.13: groupwords.py

```

""" Uses a dictionary to group the words in a text file according to
    their length (number of letters). """

import sys      # For argv global command line arguments list

def main() -> None:
    """ Group the words by length in a text file. """
    if len(sys.argv) < 2:  # Did the user not supply a file name?
        print('Usage: python groupwords.py <filename>')
        print('      where <filename> is the name of a text file.')
    else:  # User provided file name
        filename = sys.argv[1]
        groups = {}      # Initialize grouping dictionary
        with open(filename, 'r') as f:  # Open the file for reading
            content = f.read()  # Read in content of the entire file
            words = content.split()  # Make list of individual words
            for word in words:
                word = word.upper()  # Make the word all caps
                # Compute the word's length
                size = len(word)
                if size in groups:
                    if word not in groups[size]:  # Avoid duplicates
                        groups[size] += [word]  # Add the word to its group
                else:
                    groups[size] = [word]  # Add the word to a new group
            # Show the groups
            for size, group in groups.items():
                print(size, ': ', group)

if __name__ == '__main__':
    main()

```

The following shows a sample run of Listing 11.13 (groupwords.py) on our snippet from the *Declaration of Independence*:

```

4 : ['WHEN', 'HAVE', 'THEM', 'WITH', 'LAWS', 'THAT', 'THEY']
2 : ['IN', 'OF', 'IT', 'TO']
3 : ['THE', 'FOR', 'ONE', 'AND', 'GOD']
6 : ['COURSE', 'EVENTS', 'PEOPLE', 'ASSUME', 'POWERS', 'NATURE', 'DECENT', 'S
5 : ['HUMAN', 'BANDS', 'WHICH', 'AMONG', 'EARTH', 'EQUAL', 'IMPEL']
7 : ['BECOMES', 'ANOTHER', 'STATION', 'ENTITLE', 'RESPECT', 'MANKIND', 'DECLA
9 : ['NECESSARY', 'POLITICAL', 'CONNECTED']
8 : ['DISSOLVE', 'SEPARATE', "NATURE'S", 'OPINIONS', 'REQUIRES']
1 : ['A']
10 : ['SEPARATION']

```

Each key represents the length of all the strings in the list it oversees.

In this example, the need for a dictionary object is not as compelling as in Listing 11.11 (wordcount.py), because in practice the length of English words is limited. We could instead use a list of 50 sets. Only rarely used, highly technical words exceed 50 letters. We could handle these “too-long” words in a special way,

perhaps storing them in the set in the last position in the list with other words of more than 49 letters. Our program could use special processing for this set of long words if it ever becomes nonempty. This approach under typical circumstances would result in a number of empty sets at higher indices because most English text contains words of at most about 20 letters. The advantage of a dictionary is that it stores only what it needs. Listing 11.14 (groupwordslst.py) is close transliteration of Listing 11.13 (groupwords.py) that uses a list in place of a dictionary.

Listing 11.14: groupwordslst.py

```

""" Uses a list to group the words from a text file according to
    their length (number of letters). """

import sys      # For argv global command line arguments list

def main() -> None:
    """ Group the words by length in a text file. """
    if len(sys.argv) < 2: # Did the user not supply a file name?
        print('Usage: python groupwordslst.py <filename>')
        print('      where <filename> is the name of a text file.')
    else: # User provided file name
        filename = sys.argv[1]
        groups = [] # Initialize grouping list
        for i in range(20):
            groups.append(set()) # Add new empty set to the list
        with open(filename, 'r') as f: # Open the file for reading
            content = f.read() # Read in content of the entire file
            words = content.split() # Make list of individual words
            for word in words:
                word = word.upper() # Make the word all caps
                # Compute the word's length
                size = len(word)
                groups[size].add(word)
            # Show the groups
            for size, group in enumerate(groups):
                print(size, ': ', group)

if __name__ == '__main__':
    main()

```

We can see the unused space in the output of Listing 11.14 (groupwordslst.py):

```

0 : set()
1 : {'A'}
2 : {'IT', 'TO', 'OF', 'IN'}
3 : {'ONE', 'AND', 'FOR', 'GOD', 'THE'}
4 : {'THEY', 'WHEN', 'HAVE', 'LAWS', 'THEM', 'WITH', 'THAT'}
5 : {'EQUAL', 'BANDS', 'HUMAN', 'EARTH', 'WHICH', 'IMPEL', 'AMONG'}
6 : {'SHOULD', 'DECENT', 'POWERS', 'NATURE', 'COURSE', 'ASSUME', 'EVENTS', 'P
7 : {'RESPECT', 'STATION', 'BECOMES', 'DECLARE', 'ANOTHER', 'ENTITLE', 'MANKI
8 : {'REQUIRES', 'NATURE'S', 'DISSOLVE', 'OPINIONS', 'SEPARATE'}
9 : {'NECESSARY', 'POLITICAL', 'CONNECTED'}
10 : {'SEPARATION'}
11 : set()

```

```
12 : set()
13 : set()
14 : set()
15 : set()
16 : set()
17 : set()
18 : set()
19 : set()
```

The expression `set()` is how Python prints the empty set (to distinguish it from an empty dictionary). In a different document there could be empty sets in the groups representing the words with fewer than 11 letters.

11.7 Keyword Arguments

The following function specifies formal parameters named `a`, `b`, and `c`:

```
def process(a, b, c) -> None:
    print(f'a={a}, b={b}, c={c}')
```

The following code uses function `process`, passing actual parameters 2, `x` (that is, 14), and 10:

```
x = 14
process(2, x, 10)
```

It prints

```
a = 2, b = 14, c = 10
```

The calling code assigns the value of the first actual parameter to the first formal parameter. It assigns the value of the second parameter to the second formal parameter. Finally, it assigns the value of the third actual parameter to the third formal parameter. By default, the association of actual parameter to formal parameter during a function invocation is strictly positional. This is the shortest, simplest way for the caller to pass parameters.

Python allows the caller to pass its actual parameters in any order using a technique known as *keyword arguments*. We introduced the `end` and `sep` keyword arguments for the `print` function in Section 2.7. In order to use keyword arguments, the caller must know the names of the function's formal parameters. Listing 11.15 (`namedparams.py`) shows how callers can use keyword parameters.

Listing 11.15: `namedparams.py`

```
def process(a, b, c) -> None:
    print(f'a={a}, b={b}, c={c}')
```



```
x = 14
process(1, 2, 3)
process(a=10, b=20, c=30)
process(b=200, c=300, a=100)
process(c=3000, a=1000, b=2000)
process(10000, c=30000, b=20000)
```

Listing 11.15 (namedparams.py) prints the following:

```
a = 1, b = 2, c = 3
a = 10, b = 20, c = 30
a = 100, b = 200, c = 300
a = 1000, b = 2000, c = 3000
a = 10000, b = 20000, c = 30000
```

The statement

```
process(10000, c=30000, b=20000)
```

shows that keywords arguments may appear in the same call as non-keyword arguments, but in such mixed-parameter calls, all non-keyword arguments must appear before any keyword arguments. The function invocation mechanism assigns the non-keyword arguments as usual: the first actual parameter to the first formal parameter, second actual parameter to the second formal parameter, etc. It assigns the keyword arguments that follow to the formal parameters of the same name.

We can define a function to require keyword parameters by prefixing a formal parameter with two asterisks (**). The following function requires the caller to pass three actual parameters named `a`, `b`, and `c`:

```
def f(**args) -> None:
    a = args['a']
    b = args['b']
    c = args['c']
    return 2*a*a + 3*b + c
```

Given this definition of `f`, a caller can invoke `f` as

```
x = f(a=4, b=11, c=3)
```

and the following statement:

```
x = f(4, 11, c)      # Illegal, f requires keyword arguments
```

is illegal.

The following function prints the names of the keyword arguments passed in by the caller:

```
def process(**args):
    for arg in args:
        print(arg)
```

This `process` function is designed to accept any keyword arguments the caller chooses to pass. Consider the following interactive sequence:

```
>>> def process(**args):
...     for arg in args:
...         print(f'{arg} --> {args[arg]}')
...     print(f'args = {args}')
...     print(type(args))
...
>>> process(num=5, x='Hello', value=True, zz=100)
num --> 5
x --> Hello
value --> True
zz --> 100
args = {'num': 5, 'x': 'Hello', 'value': True, 'zz': 100}
<class 'dict'>
```

As we can see, the formal parameter `**args` is really a dictionary. All the keyword arguments become keys and values in the dictionary. The parameter's name becomes a key, and the associated value is the caller's value assigned to the keyword argument name.

As with arbitrary argument lists, we can mix regular positional parameters with the special `**` keyword argument parameter. We actually can mix regular parameters, arbitrary argument lists, and keyword arguments as long as we use the proper order:

```
def f(x, y, z, *a, **b):
    pass
```

In this function `x`, `y`, and `z` are regular positional arguments, `a` is the arbitrary arguments tuple, and `b` is the keywords arguments dictionary. The positional arguments, if any, must appear before any arbitrary arguments and keyword arguments. The arbitrary arguments, if any, must appear after the positional arguments and before the keyword arguments. The keyword arguments, if any, must appear after the positional and arbitrary argument list parameters.

As with arbitrary argument list arguments, we can use keyword arguments on the caller side. Listing 11.16 (`callerkeyword.py`) shows how we can send a dictionary as a parameter to a function that expects regular positional parameters.

Listing 11.16: `callerkeyword.py`

```
def f(a, b, c):
    print(f'a = {a}, b = {b}, c = {c}')

f(1, 2, 3)                                # Pass three parameters
dict = {}
dict['b'] = 22
dict['a'] = 11
dict['c'] = 33
f(**dict)
f(**{'a':10, 'b':20, 'c':30})             # Pass a dictionary
```

Listing 11.16 (`callerkeyword.py`) prints

```
a = 1  b = 2  c = 3
a = 11 b = 22 c = 33
a = 10 b = 20 c = 30
```

Observe that the caller must use the `**` prefix when passing the dictionary in the place of the expected positional parameters.

Recall that we saw several methods in the `tkinter` package that accept keyword arguments; for example, the following is a statement from Listing 9.13 (`buttontester.py`):

```
b.configure(background="yellow", text="Click Count = 0", command=update)
```

Keyword arguments are very convenient for functions and methods that can accept a large number of arguments. The `configure` method in the `Button` class is defined to accept a dictionary via the `**` notation. It accepts up to 35 keyword arguments. Keyword arguments in general enable a caller to supply just a few arguments, in any order. There is no need for the programmer to remember which argument comes first, then second, etc. as with positional arguments. The function or method can accept the caller-supplied arguments and use predetermined default values for any optional arguments the caller omitted.

Section 14.4 shows how keyword arguments are invaluable for implementing multiple inheritance, an advanced object-oriented design technique.

Recall the `nullify` decorator we examined in Section 8.11. It removed the functionality from any function that accepts two parameters. Armed with our knowledge of arbitrary argument lists and keyword arguments we now can write a more comprehensive `nullify` decorator as

```
def nullify(f):
    """ A decorator that guts the functionality of a function """
    def empty_func(*args, **kwargs):
        """ This function does nothing and returns None """
        pass
    return empty_func
```

When used as a decorator, this version of `nullify` effectively removes the functionality of any function regardless of the number and kinds of parameters the function expects.

11.8 Sets

Python provides a data structure that represents a mathematical set. As with mathematical sets, we use curly braces (`{}`) in Python code to enclose the elements of a literal set. Python distinguishes between set literals and dictionary literals by the fact that all the items in a dictionary are colon-connected (`:`) key-value pairs, while the elements in a set are simply values. Unlike Python lists, sets are unordered and may contain no duplicate elements. The following interactive sequence demonstrates these set properties:

```
>>> S = {10, 3, 7, 2, 11}
>>> S
{2, 3, 7, 10, 11}
>>> T = {5, 4, 5, 2, 4, 9}
>>> T
{9, 2, 4, 5}
```

Note the element ordering of the input is different from the ordering in the output. Also observe that sets do not admit duplicate elements.

We can make a set out of a list using the `set` constructor function:

```
>>> L = [10, 13, 10, 5, 6, 13, 2, 10, 5]
```


Table 11.2 Python set operations. A and B are sets. Figure 11.1 illustrates how the set operations work.

Operation	Mathematical Notation	Python Syntax	Result Type	Meaning
Union	$A \cup B$	<code>A B</code>	<code>set</code>	Elements in A or B or both
Intersection	$A \cap B$	<code>A & B</code>	<code>set</code>	Elements common to both A and B
Set Difference	$A - B$	<code>A - B</code>	<code>set</code>	Elements in A but not in B
Symmetric Difference	$A \oplus B$	<code>A ^ B</code>	<code>set</code>	Elements in A or B , but not both
Set Membership	$x \in A$	<code>x in A</code>	<code>bool</code>	x is a member of A
Set Membership	$x \notin A$	<code>x not in A</code>	<code>bool</code>	x is not a member of A
Set Equality	$A = B$	<code>A == B</code>	<code>bool</code>	Sets A and B contain exactly the same elements
Subset	$A \subseteq B$	<code>A <= B</code>	<code>bool</code>	Every element in set A also is a member of set B
Proper Subset	$A \subset B$	<code>A < B</code>	<code>bool</code>	A is a subset B , but B contains at least one element not in A

```
>>> L
[10, 13, 10, 5, 6, 13, 2, 10, 5]
>>> S = set(L)
>>> S
{2, 5, 6, 10, 13}
```

As you can see, the element ordering of the list is not preserved, and duplicate elements appear only once in the set.

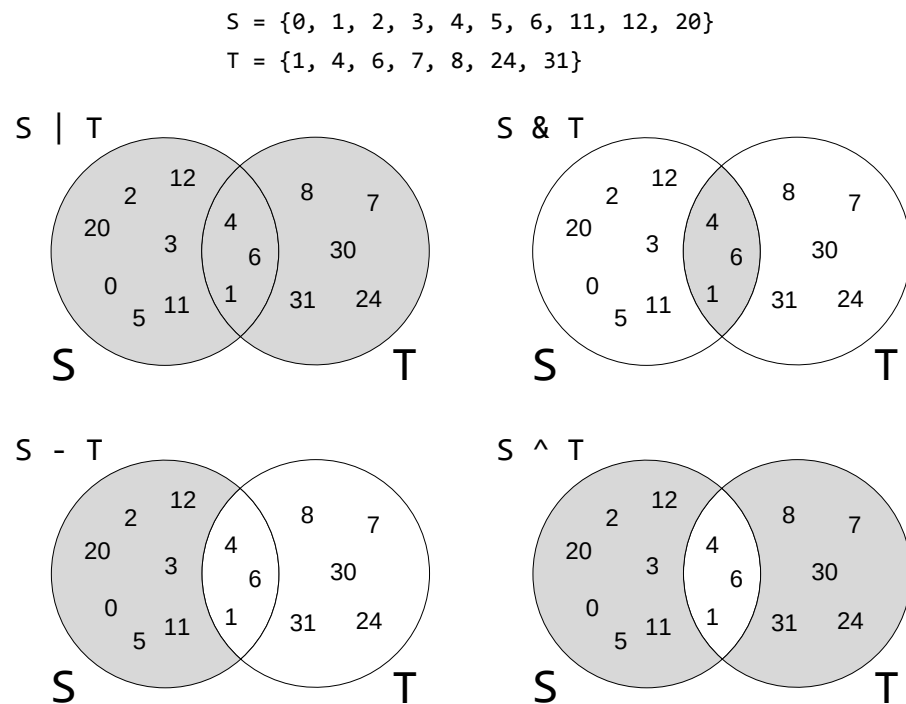
Python set notation exhibits one important difference with mathematics: the expression `{}` does not represent the empty set. In order to use the curly braces for a set, the set must contain at least one element. The expression `set()` produces a set with no elements, and thus represents the empty set. Python reserves the `{}` notation for empty *dictionaries* (see Section 11.3).

Unlike in mathematics, all sets in Python must be finite. Python supports the standard mathematical set operations of intersection, union, set difference, and symmetric difference. Table 11.2 shows the Python syntax for these operations. Figure 11.1 illustrates how the set operations work. The following interactive sequence computes the union and intersection of two sets and tests for set membership:

```
>>> S = {2, 5, 7, 8, 9, 12}
>>> T = {1, 5, 6, 7, 11, 12}
>>> S | T
{1, 2, 5, 6, 7, 8, 9, 11, 12}
>>> S & T
{12, 5, 7}
>>> 7 in S
True
>>> 11 in S
False
```

As with list comprehensions and generator expressions (Section 10.13), we can use *set comprehension* to build sets. The syntax is the same as for list comprehension, except we use curly braces rather than square brackets. The following interactive sequence constructs the set of perfect squares less than 100:

Figure 11.1 Python's set operations: union, intersection, set difference, and symmetric difference. The shaded area in each diagram indicates the elements included in the set that results from applying the indicated operator.



```
>>> S = {x**2 for x in range(10)}
>>> S
{0, 1, 64, 4, 36, 9, 16, 49, 81, 25}
```

The displayed order of elements is not as nice as the list version, but, again, element ordering is meaningless with sets.

When treated as a Boolean expression, the empty set (`set()`) is interpreted as `False`, and any other set is considered `True`.

11.9 Set Quantification with `all` and `any`

Python provides functions named `all` and `any` that respectively correspond to mathematical *universal* and *existential quantification*. These fancy mathematical terms label relatively simple concepts. Universal quantification means that a particular property is true for all the elements of a set. Existential quantification means that at least one element in the set exhibits a particular property. In mathematics the $\forall$ symbol represents universal quantification, and the $\exists$ symbol represents existential quantification. The $\forall$ symbol usually is pronounced “for all,” and the $\exists$ symbol is read as “there exists.”

To see how we can use these quantifiers in a Python program, consider the set $S = \{1, 2, 3, 4, 5, 6, 7, 8\}$. In the interactive shell we can type

```
>>> S = {1, 2, 3, 4, 5, 6, 7, 8}
>>> S
{1, 2, 3, 4, 5, 6, 7, 8}
```

To express in mathematics the fact that all the elements in set S are greater than zero, we can write

$$(\forall x \in S)(x > 0)$$

This is a statement that is either true or false, and we can see that it is a true statement. In Python, we first will use a list comprehension to see which elements in S are greater than zero. We can do this by building a list of Boolean values by using a Boolean expression in the list comprehension:

```
>>> [x > 0 for x in S]
[True, True, True, True, True, True, True, True]
```

We can see that all the entries in this list are `True`, but the best way to determine this in code is to use Python’s `all` function:

```
>>> all([x > 0 for x in S])
True
```

The `all` function returns `True` if all the elements in a list, set, or other iterable object possesses a particular quality. We do not need to create a list; a generator expression is better (note that parentheses replace the square brackets):

```
>>> all((x > 0 for x in S))
True
```

and in this case the inner parentheses are superfluous. We can write the expression as

```
>>> all(x > 0 for x in S)
True
```

The expression

```
all(x > 0 for x in S)
```

is Python's way of checking the mathematical predicate $(\forall x \in S)(x > 0)$.

The `any` function returns `True` if any element in a list, set, or other iterable object possesses a particular quality. This means the `any` function represents the mathematical existential quantifier, $\exists$:

```
>>> any(x > 0 for x in S)
True
```

The expression

```
any(x > 0 for x in S)
```

is Python's way of checking the mathematical predicate $(\exists x \in S)(x > 0)$. Certainly if the property holds for all the elements in set S , there is at least one element for which it holds.

Are all the elements of S greater than 5?

```
>>> all(x > 5 for x in S)
False
```

The answer is false, of course, because the set contains 1, 2, 3, 4, and 5, none of which are greater than 5. There are some elements in S that are greater than 5:

```
>>> any(x > 5 for x in S)
True
```

The elements 6, 7, and 8 are all greater than 5. Is it true that the set contains an element greater than 10?

```
>>> any(x > 10 for x in S)
False
```

We can see that none of the elements in S are greater than 10. If none of the set's elements possess the particular property, it certainly cannot be true for all the elements in the set:

```
>>> all(x > 10 for x in S)
False
```

The `all` and `any` functions work with any iterable object: sets, lists, dictionaries, and generated sequences.

In most Python programming, sets play a much smaller role than lists and dictionaries. Sets are most similar to lists, and the ordering of data is important in many applications. If order does not matter and all elements are unique, the `set` type does offer a big advantage over the `list` type: testing for membership using `in` is much faster on sets than lists. Listing 11.17 (`setvslistaccess.py`) creates both a set and a list, each containing the first 1,000 perfect squares. It then searches both data structures for, and does nothing with, all the integers from 0 to 9,999,999. It reports the time required for the efforts.

Listing 11.17: setvslistaccess.py

```

# Data structure size
size = 1000

# Make a big set
S = {x**2 for x in range(size)}
# Make a big list
L = [x**2 for x in range(size)]

# Verify the type of S and L
print('Set:', type(S), ' List:', type(L))

from time import perf_counter_ns

# Search size
search_size = 10000000

# Time list access
start_time = perf_counter_ns()
for i in range(search_size):
    if i in L:
        pass
stop_time = perf_counter_ns()
print(f'List elapsed: {(stop_time - start_time)/1_000_000_000} sec')

# Time set access
start_time = perf_counter_ns()
for i in range(search_size):
    if i in S:
        pass
stop_time = perf_counter_ns()
print(f'Set elapsed: {(stop_time - start_time)/1_000_000_000} sec')

```

The results of Listing 11.17 (setvslistaccess.py) are dramatic. A run on one system reports:

```

Set: <class 'set'> List: <class 'list'>
List elapsed: 155.8015458 sec
Set elapsed: 1.8636016 sec

```

The 10,000,000 list accesses required over 2.5 minutes, while the set accesses needed less than two seconds. The set membership test was over 80 times faster than the exact same test performed on the list.

Listing 11.13 (groupwords.py) grouped words from a text file according to their length. The program contained a check to avoid duplicate entries:

```

if size in groups:
    if word not in groups[size]: # Avoid duplicates
        groups[size] += [word] # Add the word to its group
else:
    groups[size] = [word] # Add the word to a new group

```

We know now that if we used sets of words rather than lists of words we could have eliminated the check for duplicate entries.

```
if size in groups:
    groups[size] += {word}  # Add the word to its group
else:
    groups[size] = {word}   # Add the word to a new group
```

By removing this extra check we also remove the application of the `in` operator on a list. We have seen that testing for membership within a list is more costly than testing for membership within a set. Eliminating this check removes the potentially costly search for an element within a large list.

11.10 Enumerating the Elements of a Data Structure

The following code prints out the contents of a list named `lst`, along with the indices of the individual elements:

```
for i in range(len(lst)):
    print(i, lst[i])
```

This code requires two function calls in order to manage the indices: one call to `len` to determine the highest index and another call to the `range` constructor to produce each index. The `__builtins__` module provides a function named `enumerate` that returns an iterable object that produces tuples. Each tuple pairs an index with its associated element. The following code uses the `enumerate` function to produce the same results as the above code:

```
for i, elem in enumerate(lst):
    print(i, elem)
```

One call to `enumerate` replaces the two calls from before. In some circumstances code that uses `enumerate` can be slightly more efficient than the code that manually manages the integer index.

The `enumerate` function accepts any type of object that supports iteration. Listing 11.18 (`enumprog.py`) demonstrates the use of `enumerate` with lists, tuples, dictionaries, sets, and generators:

Listing 11.18: enumprog.py

```

lst = [10, 20, 30, 40, 50]
t = 100, 200, 300, 400, 500
d = {'A': 4, 'B': 18, 'C': 0, 'D': 3}
s = {1000, 2000, 3000, 4000, 5000}
print(lst)
print(t)
print(d)
print(s)
for x in enumerate(lst):
    print(x, end=' ')
print()
for x in enumerate(t):
    print(x, end=' ')
print()
for x in enumerate(d):
    print(x, end=' ')
print()
for x in enumerate(s):
    print(x, end=' ')
print()

def gen(n):
    """ Generate n, n - 2, n - 3, ..., 0. """
    for i in range(n, -1, -2):
        yield i

for x in enumerate(gen(20)):
    print(x, end=' ')
print()

# Optionally specify beginning index
for x in enumerate(t, 1):
    print(x, end=' ')
print()

```

The last call to `enumerate` in Listing 11.18 (`enumprog.py`) uses an optional parameter specifying the beginning index to use in the enumeration. The default starting index is 0.

Listing 11.18 (`enumprog.py`) prints

```

[10, 20, 30, 40, 50]
(100, 200, 300, 400, 500)
{'D': 3, 'C': 0, 'A': 4, 'B': 18}
{5000, 4000, 3000, 2000, 1000}
(0, 10) (1, 20) (2, 30) (3, 40) (4, 50)
(0, 100) (1, 200) (2, 300) (3, 400) (4, 500)
(0, 'D') (1, 'C') (2, 'A') (3, 'B')
(0, 5000) (1, 4000) (2, 3000) (3, 2000) (4, 1000)
(0, 20) (1, 18) (2, 16) (3, 14) (4, 12) (5, 10) (6, 8) (7, 6) (8, 4) (9, 2) (
(1, 100) (2, 200) (3, 300) (4, 400) (5, 500)

```

11.11 Exercises

1. How are tuples different from lists?
2. How do tuples support the indexing operation (`[]`) differently from lists?
3. Are tuples mutable or immutable?
4. Are the elements in tuples ordered or unordered?
5. Rewrite the last assignment statement in the following interactive sequence so that it behaves identically but uses tuple unpacking instead of tuple slicing.

```
>>> a = 1, 2, 3, 4, 5, 6, 7, 8
>>> a
(1, 2, 3, 4, 5, 6, 7, 8)
>>> s = a[2:6]
>>> s
(3, 4, 5, 6)
```

6. Rewrite the last assignment statement in the following interactive sequence so that it behaves identically but uses tuple slicing instead of tuple unpacking.

```
>>> a = 1, 2, 3, 4, 5, 6, 7, 8
>>> a
(1, 2, 3, 4, 5, 6, 7, 8)
>>> s = _, _, _, *s, _ = a
>>> s = tuple(s)
>>> s
(4, 5, 6, 7)
```

7. Consider the tuple `tpl` defined as
`tpl = 7, 10, -3, 18, 6, 10`

Provide one assignment statement that uses tuple unpacking to assign `x` to the first element and `y` to the last element.

8. Write a function named `product` that computes the product of any number of floating-point arguments; for example, the call `product(2.5, 2, 10.0)` would evaluate to `50.0`. The function should return `1` (the identity element for multiplication) if the caller passes no arguments.
9. Write a function named `zero_sum` that accepts any number of integer arguments. The function should return `True` if the sum of its arguments is zero; otherwise, it should return `False`. The call `zero_sum(2, 3, -5)`, for example, would evaluate to `True`, since $2 + 3 + -5 = 0$. On the other hand, `zero_sum(2, 3, -10, 4)` evaluates to `False` because $2 + 3 + -10 + 4 = -1 \neq 0$. `zero_sum` should return `True` when called with no arguments.
10. Why is a dictionary considered an *associative container*?
11. What statement assigns an empty dictionary to a variable named `d`?
12. If `d` refers to a dictionary, what expression represents the value associated with the key `"Fred"`?
13. What happens when an executing program attempts to retrieve a value using a key that is not present in the dictionary?

14. What happens when an executing program attempts to associate a value with a key that is not present in the dictionary?

15. Are dictionaries mutable or immutable?

16. Given the following dictionary:

```
d = {3:0, 5:1, 10:1, 8:2, 15:4}
```

Indicate what each of the following code fragments will print:

(a) `print(d)`

(b) `for x in d:`
 `print(x)`

(c) `for x in d.keys():`
 `print(x)`

(d) `for x in d.values():`
 `print(x)`

17. Are the elements in dictionaries ordered or unordered?

18. Modify Listing 9.14 (`tkinterlight.py`) so that it models a light with a single on/off yellow lamp.

19. Write a graphical, two-player Tic-Tac-Toe game using the `tkinter` module (see <https://en.wikipedia.org/wiki/Tic-tac-toe> for more information about the game). You can use nine separate variables to track the contents of the game's squares. You must be able to draw lines and circles in the appropriate locations.

20. Explain why the statement

```
A = {}
```

does not create an empty set.

21. Provide the Python statement that assigns the variable `A` to the empty set.

22. Are sets mutable or immutable?

23. Given the following initialization statements:

```
A = {20, 19, 2, 10, 7}
B = {4, 10, 5, 6, 9, 7}
C = {10, 19}
```

evaluate the following expressions:

(a) `A`

(b) `20 in A`

(c) `20 not in A`

(d) `A & B`

(e) `A | B`

- (f) $C < A$
- (g) $C \leq A$
- (h) $C \leq B$
- (i) $A \leq A$
- (j) $A < A$
- (k) $\text{len}(A)$
- (l) $\{x + 2 \text{ for } x \text{ in range}(10)\}$
- (m) $\{x - 2 \text{ for } x \text{ in } A\}$
- (n) $\{x - 2 \text{ for } x \text{ in } A \text{ if } x < 10\}$

Chapter 12

Handling Exceptions

In our programming experience so far we have encountered several kinds of run-time exceptions, such as division by zero, accessing a list with an out-of-range index, and attempting to convert a non-number to an integer. We have seen these and other run-time exceptions immediately terminate a running program. Python provides a standard mechanism called *exception handling* that allows programmers to deal with these kinds of run-time exceptions and many more. Rather than always terminating the program's execution, an executing program can detect the problem when it arises and possibly execute code to correct the issue or mitigate it in some way. This chapter explores handling exceptions in Python.

12.1 Motivation

Algorithm design can be tricky because the details are crucial. It may be straightforward to write an algorithm to solve a problem in the general case, but the designer may have to address a number of special cases within the problem for the algorithm to be correct. Some of these special cases might occur rarely and only under the most extraordinary circumstances. The algorithm must properly handle these exceptional cases to be truly robust; however, adding the necessary details to the algorithm may render it overly complex and difficult to construct correctly. Such an overly complex algorithm would be difficult for others to read and understand, and it would be harder to debug and extend.

Ideally, a developer would write the algorithm in its general form including any common special cases. Exceptional situations that should arise rarely, along with a strategy to handle them, could appear elsewhere, perhaps as an annotation to the algorithm. This approach would focus the algorithm on its routine activity and keep its rare behavior tucked out of sight until specifically needed.

Python's exception handling infrastructure allows programmers to cleanly separate the code that implements the focused algorithm from the code that deals with exceptional situations that the algorithm may face. This approach is more modular and encourages the development of code that is cleaner and easier to maintain and debug.

An *exception* is a special object that the executing program can create when it encounters an extraordinary situation. Such a situation almost always represents a problem, usually some sort of run-time error. Examples of exceptional situations include:

- attempting to read past the end of a file

- evaluating the expression `lst[i]` where `lst` is a list, and $i \geq \text{len}(\text{lst})$.
- attempting to convert a nonnumeric string to a number, as in `int('Fred')`
- attempting to read a variable that has not been defined
- attempting to read data from the network when the connection is lost

The algorithm may be able to handle many of these potential problems itself. For example, a programmer can use an `if` statement to determine if a list index is within the bounds of a list:

```
if 0 <= i < len(lst):
    print(lst[i])    # Safely print lst[i]
```

However, if the code within a function accesses the list in many different places, the large number of conditionals required to ensure the absolute safety of all the list accesses can quickly obscure the overall logic of the function. Fortunately, programmers sometimes can avoid this scenario by checking a list index once for a large number of similar accesses within a block of code or managing the index carefully to ensure it cannot be outside the list's bounds. Other problems, however, such as the loss of a network connection, may be less straightforward for the algorithm to address directly. Fortunately, specific Python exceptions are available to cover problems such as these.

Exceptions represent a standard way to deal with run-time errors. In programming languages that do not support exception handling, programmers must devise their own ways of dealing with exceptional situations. Such ad hoc approaches produce error handling facilities developed by one programmer that can be incompatible with those used by another. Python provides a comprehensive, uniform exception handling framework. Python's exception framework provides a simple means of communicating errors between functions and short circuiting the normal function return process, effectively bypassing functions up the call chain that are unable to, or have no need to, participate in the error handling activity. The proper use of Python's exception handling infrastructure leads to code that is logically cleaner and less prone to programming errors. The standard Python library uses exceptions, and programmers can create new exceptions that address issues specific to their particular problems. These exceptions all use a common syntax and are completely compatible with each other.

12.2 Common Standard Exceptions

We have encountered a number of Python's standard exception classes. Table 12.1 lists some of the more common exception classes.

The following interactive sequence provides an example of each of the exceptions shown in Table 12.1:

```
>>> from fractions import Fraction
>>> frac = Fraction(1, 2)
>>> print(frac.numerator)
1
>>>
>>> print(frac.numertor)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
AttributeError: 'Fraction' object has no attribute 'numertor'.
>>> from fraction import Fraction
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
```

Table 12.1 Common standard exception classes

Class	Meaning
<code>AttributeError</code>	Object does not contain the specified instance variable or method
<code>ModuleError</code>	The <code>import</code> statement fails to find a specified module
<code>ImportError</code>	The <code>import</code> statement fails to find a name in a given module
<code>IndexError</code>	A sequence (list, string, tuple) index is out of range
<code>KeyError</code>	Specified key does not appear in a dictionary
<code>NameError</code>	Specified local or global name does not exist
<code>TypeError</code>	Operation or function applied to an inappropriate type
<code>ValueError</code>	Operation or function applied to correct type but inappropriate value
<code>ZeroDivisionError</code>	Second operand of division or modulus operation is zero

```

ModuleNotFoundError: No module named 'fraction'
>>> from fractions import Fractions
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
ImportError: cannot import name 'Fractions' from 'fractions'
>>> seq = [2, 5, 11]
>>> print(seq[1])
5
>>> print(seq[3])
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
IndexError: list index out of range
>>> d = {}
>>> d['Fred'] = 100
>>> print(d['Fred'])
100
>>> print(d['Freddie'])
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
KeyError: 'Freddie'
>>> print(x)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
NameError: name 'x' is not defined
>>> print(seq['Fred'])
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: list indices must be integers or slices, not str
>>> print(int('Fred'))
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
ValueError: invalid literal for int() with base 10: 'Fred'
>>> print(1/0)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
ZeroDivisionError: division by zero

```

Python contains many more standard exception classes than those shown in Table 12.1, and in Section 14.2 we will explore ways to create our own custom exception classes.

12.3 Handling Exceptions

Listing 12.1 (dividenumbers.py) computes the quotient of two integer values supplied by the user.

Listing 12.1: dividenumbers.py

```
# Get two integers from the user
print('Please enter two numbers to divide.')
num1 = int(input('Please enter the dividend: '))
num2 = int(input('Please enter the divisor: '))
print(f'{num1} divided by {num2} = {num1/num2}')
```

In Listing 12.1 (dividenumbers.py), all is well until the user attempts to divide by zero:

```
Please enter two numbers to divide.
Please enter the dividend: 4
Please enter the divisor: 0
Traceback (most recent call last):
  File "C:\Code\python\dividenumbers.py", line 5, in <module>
    print(f'{num1} divided by {num2} = {num1/num2}')
```

ZeroDivisionError: division by zero

This program execution produces a `ZeroDivisionError` exception.

We can defend against the `ZeroDivisionError` exception with a conditional statement, as Listing 12.2 (checkforzero.py) shows.

Listing 12.2: checkforzero.py

```
# Get two integers from the user
print('Please enter two numbers to divide.')
num1 = int(input('Please enter the dividend: '))
num2 = int(input('Please enter the divisor: '))
if num2 != 0:
    print(f'{num1} divided by {num2} = {num1/num2}')
```

else:

```
    print('Cannot divide by zero')
```

The solution expressed by Listing 12.2 (checkforzero.py) illustrates the concept of *look before you leap*, usually abbreviated as *LBYL* in the Python community. This programming idiom checks code that may misbehave before executing it. The LBYL idiom works well for code like that found in Listing 12.2 (checkforzero.py).

Consider Listing 12.3 (enterinteger.py) that asks the user for a small integer value.

Listing 12.3: enterinteger.py

```
val = int(input('Please enter a small positive integer: '))
print('You entered', val)
```

A typical run of Listing 12.3 (enterinteger.py) looks like

```
Please enter a small positive integer: 5
You entered 5
```

A user easily and innocently can thwart the programmer's original intentions, as the following sample run illustrates:

```
Please enter a small positive integer: five
Traceback (most recent call last):
  File "C:\Code\python\enterinteger.py", line 1, in <module>
    val = int(input("Please enter a small positive integer: "))
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
ValueError: invalid literal for int() with base 10: 'five'
```

For an English-speaking human, the response *five* should be just as acceptable as *5*. The strings acceptable to the Python `int` function, however, can contain only numeric characters and an optional leading sign character (+ or -). The user's input causes the program to produce a run-time exception. As it stands, the program reacts to the exception by printing a message and terminating itself. As shown in the exception error report, the kind of exception that this execution example produces is a `ValueError` exception.

Unfortunately, any attempt to make Listing 12.3 (`enterinteger.py`) more robust via the LBYL idiom is not as easy as it is for Listing 12.2 (`checkforzero.py`). We basically need to determine if the arbitrary string the user enters is acceptable to the `int` conversion function. The string must contain only the digit characters `'0'`, `'1'`, `'2'`, `'3'`, `'4'`, `'5'`, `'6'`, `'7'`, `'8'`, or `'9'`, and it may contain a leading `'-'` or `'+'` character indicating the number's sign. Python's *regular expression* library is ideal for this purpose, but it is somewhat complicated and deserves an entire chapter devoted to its use. Short of using the regular expression library, the logic to ensure that the string is acceptable to the `int` function would be relatively complex.

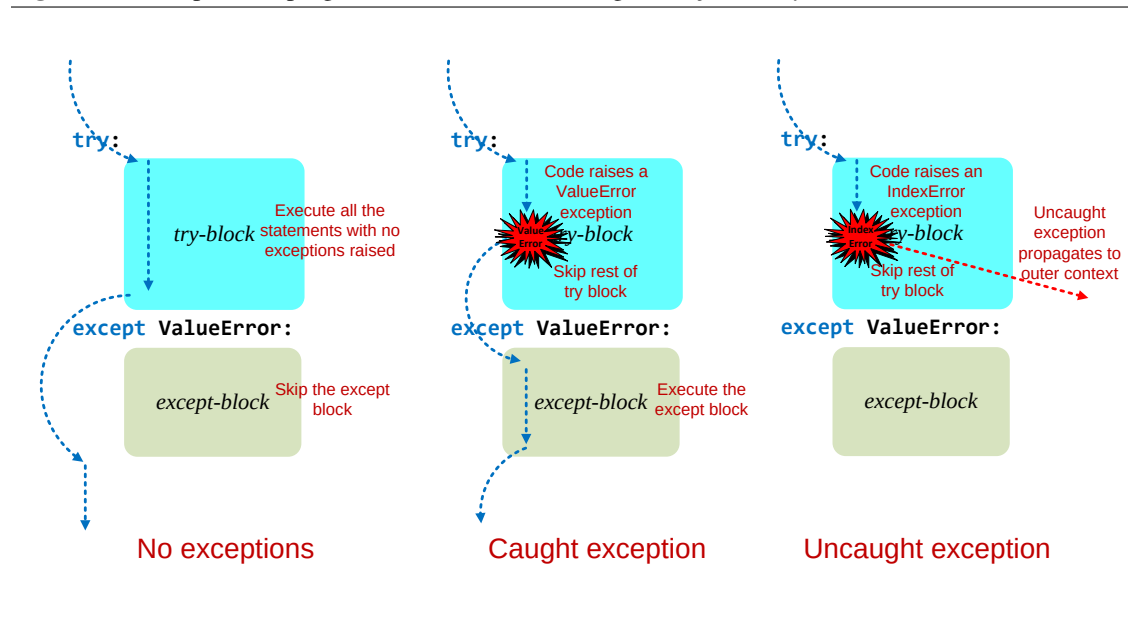
An alternative to LBYL is *EAFP*, which stands for *easier to ask for forgiveness than permission*. The EAFP approach attempts to execute the potentially problematic code within a `try` statement. If the code raises an exception, the program's execution does not necessarily terminate; instead, the program's execution jumps immediately to a different block within the `try` statement. Listing 12.4 (`enterintexcept.py`) wraps the code from Listing 12.3 (`enterinteger.py`) within a `try` statement to successfully defend against bad user input.

Listing 12.4: enterintexcept.py

```
try:
    val = int(input('Please enter a small positive integer: '))
    print('You entered', val)
except ValueError:
    print('Input not accepted')
```

The two statements between `try` and `except` constitute the `try` block. The statement after the `except` line represents an `except` block. If the user enters a string unacceptable to the `int` function, the `int` function will raise a `ValueError` exception. At this point the program will not complete the assignment statement nor will it execute the `print` statement that follows. Instead the program immediately will begin executing the code in the `except` block. This means if the user enters *five*, the program will print the message *Input not accepted*. If the user enters a convertible string like *5*, the program will complete the `try` block and ignore the code in the `except` block. Figure 12.1 contrasts the possible program execution flows within a `try/except` statement. We say the `except` block *handles* the exception raised in the `try` block. Another common terminology used to describe the excepting handling process uses the *throw/catch* metaphor: the executing program *throws* an exception that an `except` block *catches*.

Listing 12.4 (enterintexcept.py) catches only exceptions of type `ValueError`. If for some reason the code within the `try` block of Listing 12.4 (enterintexcept.py) raises a different type of exception, this `try`

Figure 12.1 The possible program execution flows through a `try/except` statement.

statement is unable to catch it. In this case the program will behave as if the `try/except` statement were not there. Unless other exception handling code is present in the calling environment, the interpreter simply will terminate the program with an error message. Listing 12.5 (`otherexcept.py`) includes a statement that attempts to assign the value at index 2 of an empty list. Unfortunately, an empty list contains no values, so any index is outside of its range of indices.

Listing 12.5: `otherexcept.py`

```
try:
    val = int(input('Please enter a small positive integer: '))
    print('You entered', val)
    [] [2] = 5  # Try to assign to a nonexistent index of the empty list
except ValueError:
    print('Input not accepted')
```

The expression `[]` represents the empty list. The expression `[] [2]` represents the element at index 2 within the empty list (there is no such element). Consider the following sample run of Listing 12.5 (`otherexcept.py`):

```
Please enter a small positive integer: 5
You entered 5
Traceback (most recent call last):
  File "C:\Code\python\otherexcept.py", line 4, in <module>
    [] [2] = 5  # Try to assign to a nonexistent index of the empty list
    ~~~~^
IndexError: list assignment index out of range
```

We see that the `except` block in Listing 12.5 (`otherexcept.py`) is unable to catch the `IndexError` exception. Figure 12.1 illustrates this possible outcome as well.

12.4 Handling Multiple Exceptions

A `try` statement can have multiple `except` blocks. Each `except` block must catch a different type of exception object. Listing 12.6 (multiexcept.py) offers three `except` blocks. Its `try` statement specifically can catch `ValueError`, `IndexError`, and `ZeroDivisionError` exceptions.

Listing 12.6: multiexcept.py

```
import random

for i in range(10):    # Loop 10 times
    print('Beginning of loop iteration', i)
    try:
        r = random.randint(1, 3)    # r is pseudorandomly 1, 2, or 3
        if r == 1:
            print(int('Fred'))    # Try to convert a non-integer
        elif r == 2:
            [][2] = 5    # Try to assign an element in an empty list
        else:
            print(3/0)    # Try to divide by zero
    except ValueError:
        print('Cannot convert integer')
    except IndexError:
        print('List index is out of range')
    except ZeroDivisionError:
        print('Division by zero not allowed')

    print('End of loop iteration', i)
```

Each time through the loop the code within the `try` block of Listing 12.6 (multiexcept.py) will raise one of three different exceptions based on the generated pseudorandom number. The program offers three `except` blocks. If the code in the `try` block raises one of the three types of exceptions, the program will execute the code in the matching `except` block. Only code in one of the three `except` blocks will execute as a result of the exception. The following shows a sample run of Listing 12.6 (multiexcept.py):

```
Beginning of loop iteration 0
List index is out of range
End of loop iteration 0
Beginning of loop iteration 1
Division by zero not allowed
End of loop iteration 1
Beginning of loop iteration 2
Cannot convert integer
End of loop iteration 2
Beginning of loop iteration 3
List index is out of range
End of loop iteration 3
Beginning of loop iteration 4
Cannot convert integer
End of loop iteration 4
Beginning of loop iteration 5
List index is out of range
End of loop iteration 5
Beginning of loop iteration 6
```

```
Division by zero not allowed
End of loop iteration 6
Beginning of loop iteration 7
List index is out of range
End of loop iteration 7
Beginning of loop iteration 8
List index is out of range
End of loop iteration 8
Beginning of loop iteration 9
Division by zero not allowed
End of loop iteration 9
```

Observe that only one `except` block executes each time through the loop.

If we need the exact code to handle more than one exception type, we can associate multiple types with a single `except` block by listing each exception type within a tuple. Listing 12.7 (`multihandle.py`) illustrates.

Listing 12.7: `multihandle.py`

```
import random

for i in range(10):    # Loop 10 times
    print('Beginning of loop iteration', i)
    try:
        r = random.randint(1, 3)    # r is pseudorandomly 1, 2, or 3
        if r == 1:
            print(int('Fred'))    # Try to convert a non-integer
        elif r == 2:
            [] [2] = 5    # Try to assign to an element in an empty list
        else:
            print(3/0)    # Try to divide by zero
    except (ValueError, ZeroDivisionError):
        print('Problem with integer detected')
    except IndexError:
        print('List index is out of range')

    print('End of loop iteration', i)
```

The first `except` block in Listing 12.7 (`multihandle.py`) will catch both `ValueError` and `ZeroDivisionError` exceptions. As shown in Listing 12.7 (`multihandle.py`), parentheses must enclose a tuple specified in an `except` block.

In general it is better to bundle exception handlers rather than duplicating code over multiple handlers.

12.5 The Catch-all Handler

Consider Listing 12.8 (`missedexception.py`), a slight modification of

Listing 12.6 (multiexcept.py).

Listing 12.8: missedexception.py

```
import random

for i in range(10):    # Loop 10 times
    print('Beginning of loop iteration', i)
    try:
        r = random.randint(1, 4)    # r is pseudorandomly 1, 2, 3, or 4
        if r == 1:
            print(int('Fred'))    # Try to convert a non-integer
        elif r == 2:
            [] [2] = 5    # Try to assign to an element in an empty list
        elif r == 3:
            print({}[1])    # Try to use a key with an empty dictionary
        else:
            print(3/0)    # Try to divide by zero
    except ValueError:
        print('Cannot convert integer')
    except IndexError:
        print('List index is out of range')
    except ZeroDivisionError:
        print('Division by zero not allowed')

    print('End of loop iteration', i)
```

In addition to raising one of the three exceptions from Listing 12.6 (multiexcept.py), Listing 12.8 (missedexception.py) can raise a `KeyError` exception. The expression `{}` represents the empty dictionary, and the expression `{}[1]` represents the value associated with the key 1 within the empty dictionary (which, of course, does not exist). Unfortunately, Listing 12.8 (missedexception.py) has no handler for the `KeyError` exception. The following output shows the results for one program run:

```
Beginning of loop iteration 0
Division by zero not allowed
End of loop iteration 0
Beginning of loop iteration 1
List index is out of range
End of loop iteration 1
Beginning of loop iteration 2
Traceback (most recent call last):
File "C:\Code\python\div.py", line 12, in <module>
    print({}[1])    # Try to use a nonexistent key to get an item from
a dictionary
KeyError: 1
```

If we want our programs not to crash, we need to handle all possible exceptions that can arise. This is particularly important when we use libraries that we did not write. A program may execute code that only under very rare circumstances raises an exception. This situation may be so rare that it evades our thorough testing and appears only after we deploy the application to users. We need a handler that can catch *any* exception.

The type `Exception` matches any exception type that a programmer would reasonably want to catch. Listing 12.9 (catchallexcept.py) shows how to use this “catch-all” exception to close the exception hole

found in Listing 12.8 (missedexception.py).

Listing 12.9: catchallexcept.py

```
import random

for i in range(10):    # Loop 10 times
    print('Beginning of loop iteration', i)
    try:
        r = random.randint(1, 4)    # r is pseudorandomly 1, 2, 3, or 4
        if r == 1:
            print(int('Fred'))    # Try to convert a non-integer
        elif r == 2:
            [][2] = 5    # Try to assign to an element in an empty list
        elif r == 3:
            print({}[1])    # Try to use a key with an empty dictionary
        else:
            print(3/0)    # Try to divide by zero
    except ValueError:
        print('Cannot convert integer')
    except IndexError:
        print('List index is out of range')
    except ZeroDivisionError:
        print('Division by zero not allowed')
    except Exception:    # Catch any other type of exception
        print('This program has encountered a problem')

    print('End of loop iteration', i)
```

A run of Listing 12.9 (catchallexcept.py) shows that it can run to completion:

```
List index is out of range
End of loop iteration 0
Beginning of loop iteration 1
Cannot convert integer
End of loop iteration 1
Beginning of loop iteration 2
This program has encountered a problem
End of loop iteration 2
Beginning of loop iteration 3
Cannot convert integer
End of loop iteration 3
Beginning of loop iteration 4
This program has encountered a problem
End of loop iteration 4
Beginning of loop iteration 5
Cannot convert integer
End of loop iteration 5
Beginning of loop iteration 6
Cannot convert integer
End of loop iteration 6
Beginning of loop iteration 7
Cannot convert integer
End of loop iteration 7
Beginning of loop iteration 8
```

```

Division by zero not allowed
End of loop iteration 8
Beginning of loop iteration 9
Division by zero not allowed
End of loop iteration 9

```

Listing 12.9 (catchallexcept.py) offers four `except` blocks. The `except` block at the end represents the catch-all handler that can catch any exception not caught by an earlier `except` block within the `try` statement. If present, the catch-all `except` block should be the last `except` block in the `try` statement. Since the `Exception` type matches *any* exception type, if it appears before another `except` block, it will intercept a specific exception before a later `except` block has a chance to see it. This is because a program executes at most one `except` block when executing a `try` statement.

There are a few exceptions that the `Exception` type does not match. These are exceptions that programmers usually do not want to handle; for example, the call `sys.exit(0)` raises an exception and terminates the executing program. Another example is the keyboard interrupt (`Ctrl C`) entered by the user. The `Exception` type does not match either of these types of exceptions. On occasion, a program may need to handle these special exceptions; for example, the program may need to perform some special clean up or properly close its connection to a remote database before terminating. The `try` statement allows us to catch *any* previously uncaught exception with an untyped `except` block. Listing 12.10 (reallycatchallexcept.py) shows how to use this super catch-all handler.

Listing 12.10: reallycatchallexcept.py

```

import random

for i in range(10):    # Loop 10 times
    print('Beginning of loop iteration', i)
    try:
        r = random.randint(1, 4)    # r is pseudorandomly 1, 2, 3, or 4
        if r == 1:
            print(int('Fred'))    # Try to convert a non-integer
        elif r == 2:
            [] [2] = 5    # Try to assign to an element in an empty list
        elif r == 3:
            print({}[1])    # Try to use a key with an empty dictionary
        else:
            print(3/0)    # Try to divide by zero
    except ValueError:
        print('Cannot convert integer')
    except IndexError:
        print('List index is out of range')
    except ZeroDivisionError:
        print('Division by zero not allowed')
    except:    # Catch absolutely any other type of exception
        print('This program has encountered a problem')

    print('End of loop iteration', i)

```

The untyped `except` block, if present, must be the last `except` block in the `try` statement. It is a syntax error for the untyped `except` block to appear before a typed `except` block.

In situations that warrant a catch-all handler, prefer to catch the `Exception` type; use the untyped catch-all handler only as a last resort.

Now that we have seen how to write a catch-all exception handler, it is important to note that its use should be limited. A catch-all handler, if used properly, is appropriate for some situations, but for beginning programmers it is tempting to provide a catch-all `except` block all the time, *just in case*. A good rule of thumb is this: code should handle specific kinds of exceptions it expects and ignore (that is, do not attempt to catch) exceptions it does not anticipate. Section 12.7 provides some direction for the proper use the catch-all exception handler.

12.6 Catching Exception Objects

The `except` blocks “catch” exception objects. We can inspect the object that an `except` block catches if we specify the object’s name with the `as` keyword. Listing 12.11 (`exceptobject.py`) shows how to use the `as` keyword to get access to the exception object raised by code in the `try` block.

Listing 12.11: `exceptobject.py`

```
import random

for i in range(10):    # Loop 10 times
    print('Beginning of loop iteration', i)
    try:
        r = random.randint(1, 4)    # r is pseudorandomly 1, 2, 3, or 4
        if r == 1:
            print(int('Fred'))    # Try to convert a non-integer
        elif r == 2:
            [] [2] = 5    # Try to assign to an element of an empty list
        elif r == 3:
            print({}[1])    # Try to use a key with an empty dictionary
        else:
            print(3/0)    # Try to divide by zero
    except ValueError as e:
        print('Problem with value ==>', type(e), e)
    except IndexError as e:
        print('Problem with list ==>', type(e), e)
    except ZeroDivisionError as e:
        print('Problem with division ==>', type(e), e)
    except Exception as e:
        print('Problem with something ==>', type(e), e)

    print('End of loop iteration', i)
```

The following sample program run reveals what the exception objects print when sent to the `print` function:

```
Beginning of loop iteration 0
Problem with division ==> <class 'ZeroDivisionError'> division by zero
End of loop iteration 0
Beginning of loop iteration 1
Problem with list ==> <class 'IndexError'> list assignment index out of
End of loop iteration 1
Beginning of loop iteration 2
Problem with division ==> <class 'ZeroDivisionError'> division by zero
End of loop iteration 2
Beginning of loop iteration 3
Problem with division ==> <class 'ZeroDivisionError'> division by zero
End of loop iteration 3
Beginning of loop iteration 4
Problem with division ==> <class 'ZeroDivisionError'> division by zero
End of loop iteration 4
Beginning of loop iteration 5
Problem with list ==> <class 'IndexError'> list assignment index out of
End of loop iteration 5
Beginning of loop iteration 6
Problem with division ==> <class 'ZeroDivisionError'> division by zero
End of loop iteration 6
Beginning of loop iteration 7
Problem with value ==> <class 'ValueError'> invalid literal for int() wit
End of loop iteration 7
Beginning of loop iteration 8
Problem with value ==> <class 'ValueError'> invalid literal for int() wit
End of loop iteration 8
Beginning of loop iteration 9
Problem with division ==> <class 'ZeroDivisionError'> division by zero
End of loop iteration 9
```

Observe how each exception object (e) prints a message that is meaningful for its particular exception.

12.7 Exception Handling Scope

Python's exception handling infrastructure is special because it can transcend the usual scoping rules for functions and objects. The exception handling examples we have seen so far have been simple programs where all the code is in the main executing module. The origin of the exception is close to the code we can see. These examples have not demonstrated the true power of Python's exceptions. To get a better idea of the scope of exceptions, consider Listing 12.12 (makeintegerlist.py).

Listing 12.12: makeintegerlist.py

```

def get_int_in_range(low: int, high: int) -> int:
    """ Obtains an integer value from the user. Acceptable values
        must fall within the specified range low...high. """
    val = int(input())      # Can raise a ValueError
    while val < low or val > high:
        print('Value out of range, please try again:', end=' ')
        val = int(input())  # Can raise a ValueError
    return val

def create_list(n: int, min: int, max: int) -> list[int]:
    """ Allows the user to create a list of n elements consisting
        of integers in the range min...max """
    result = []
    while n > 0:            # Count down to zero
        print(f'Enter integer in the range {min}...{max}:', end=' ')
        result.append(get_int_in_range(min, max))
        n -= 1
    return result

def main() -> None:
    """ Create a list of two elements supplied by the user,
        each element in the range 10...20 """
    lst = create_list(2, 10, 20)
    print(lst)

if __name__ == '__main__':
    main()                  # Invoke main

```

Listing 12.12 (makeintegerlist.py) provides two handy functions, `get_int_in_range` and `create_list`. The `get_int_in_range` function expects the user to enter an integer value that falls within the range of values specified by its parameters. It ensures the integer the user provides is in the correct range, but if the user enters a non-integer value, the function will raise an exception. The following shows the program's interaction with a well-behaved user:

```

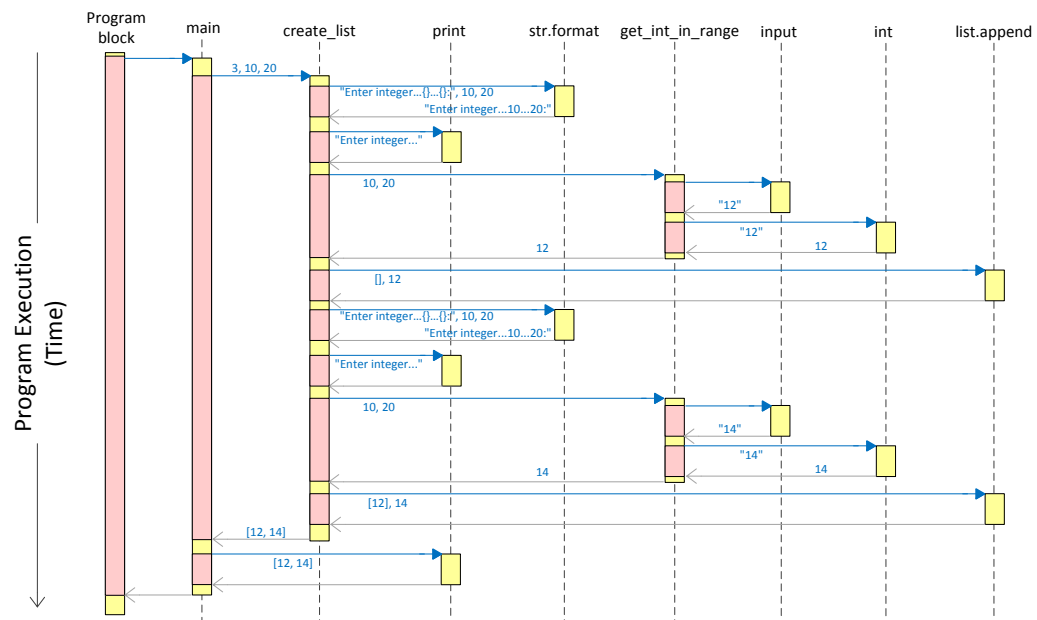
Enter integer in the range 10...20: 12
Enter integer in the range 10...20: 14
[12, 14]

```

Figure 12.2 diagrams the function call sequences of this sample program execution.

The following shows how the program runs with a more creative user:

Figure 12.2 Function call sequence diagram for Listing 12.12 (makeintegerlist.py) running with the user providing the input 12 and 14.



```

Traceback (most recent call last):
  File "C:\Code\python\makeintegerlist.py",
    line 30, in <module>
    main()      # Invoke main
    ^^^^^^^
  File "C:\Code\python\makeintegerlist.py",
    line 25, in main
    lst = create_list(2, 10, 20)
          ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "C:\Code\python\makeintegerlist.py",
    line 17, in create_list
    result.append(get_int_in_range(min, max))
                  ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "C:\Code\python\makeintegerlist.py",
    line 4, in get_int_in_range
    val = int(input())      # Can raise a ValueError
          ^^^^^^^^^^^^^^^
ValueError: invalid literal for int() with base 10: '9+7'

```

If the `get_int_in_range` function used the `eval` function instead of `int`, it would avoid this exception, but that would allow the function to accept floating-point values. Also, using the `eval` function would not help for the following input:

```

Enter integer in the range 10...20: eleven
Traceback (most recent call last):
  File "C:\Code\python\makeintegerlist.py", line 30, in <module>
    main()      # Invoke main
    ^^^^^^^
  File "C:\Code\python\makeintegerlist.py", line 25, in main
    lst = create_list(2, 10, 20)
          ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "C:\Code\python\makeintegerlist.py", line 17, in create_list
    result.append(get_int_in_range(min, max))
                  ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "C:\Code\python\makeintegerlist.py", line 4, in get_int_in_range
    val = int(input())      # Can raise a ValueError
          ^^^^^^^^^^^^^^^
ValueError: invalid literal for int() with base 10: 'eleven'

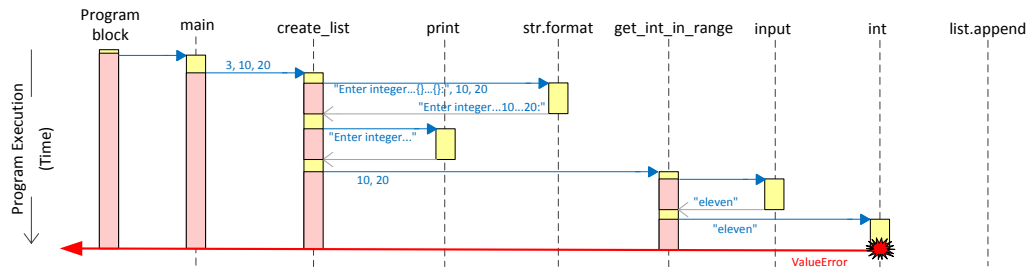
```

An uncaught exception produces a *stack trace*. Python uses an area of the computer’s memory known as the *stack* to help it control function and method invocations. The stack stores parameters, return values, and the point in the code where the program’s execution should return when a function completes. An exception stack trace provides a snapshot of the stack that enables developers to reconstruct the chain of function and/or method calls that produced the exception.

We can read the stack trace from the top down. We see that the program (referenced in the stack trace as `<module>`) called the `main` function at line 27 in the source file `makeintegerlist.py`. In turn, a statement at line 23 in the `main` function invoked the `create_list` function. Code within the `create_list` function at line 16 called the `get_int_in_range` function. Finally, line 4 in the `get_int_in_range` function raised a `ValueError` exception when it called the `int` function with an invalid string literal (the value `'eleven'` that the user provided).

We say that an exception “unwinds” the stack, because, if uncaught, an exception will propagate back up the call chain. The propagation stops when an exception handler catches the exception. If the propagation

Figure 12.3 Function call sequence diagram for Listing 12.12 (`makeintegerlist.py`) raising a `ValueError` exception when the user enters *eleven*. The exception interrupts the normal function return control flow and immediately terminates the program's execution.



progresses all the way back to the program block level and along the way encounters no exception handler with an `except` block of the required type, the interpreter will terminate the program's execution.

Figure 12.3 provides the function call sequence diagram for case of Listing 12.12 (`makeintegerlist.py`) raising an exception and thereby terminating the program's execution. In Listing 12.12 (`makeintegerlist.py`), `get_int_in_range`'s call to `int` function can raise a `ValueError` exception. To get to this point, the chain of function calls is

Program block → main → create_list → get_int_in_range → int

The red arrow in Figure 12.3 shows that the exception rises immediately up the call chain in the reverse order:

int → get_int_in_range → create_list → main → Program block

Any function in the call chain can catch the exception. In Listing 12.12 (`makeintegerlist.py`) we know that `int` can raise the exception, but which function should catch the exception? Should we handle the exception close to where it arises or farther up the call chain?

Listing 12.13 (`makeintegerlist2.py`) adds an exception handler to the `create_list` function.

Listing 12.13: makeintegerlist2.py

```

def get_int_in_range(low: int, high: int) -> int:
    """ Obtains an integer value from the user. Acceptable values
        must fall within the specified range low...high. """
    val = int(input())      # Can raise a ValueError
    while val < low or val > high:
        print('Value out of range, please try again:', end=' ')
        val = int(input())  # Can raise a ValueError
    return val

def create_list(n: int, min: int, max: int) -> list[int]:
    """ Allows the user to create a list of n elements consisting
        of integers in the range min...max """
    result = []
    try:
        while n > 0:        # Count down to zero
            print(f'Enter integer in the range {min}...{max}:', end=' ')
            result.append(get_int_in_range(min, max))
            n -= 1
    except ValueError:
        print('Disallowed user entry interrupted list creation')
    return result

def main() -> None:
    """ Create a list of two elements supplied by the user,
        each element in the range 10...20 """
    lst = create_list(2, 10, 20)
    print(lst)

if __name__ == '__main__':
    main()      # Invoke main

```

The following shows a sample run of Listing 12.13 (makeintegerlist2.py):

```

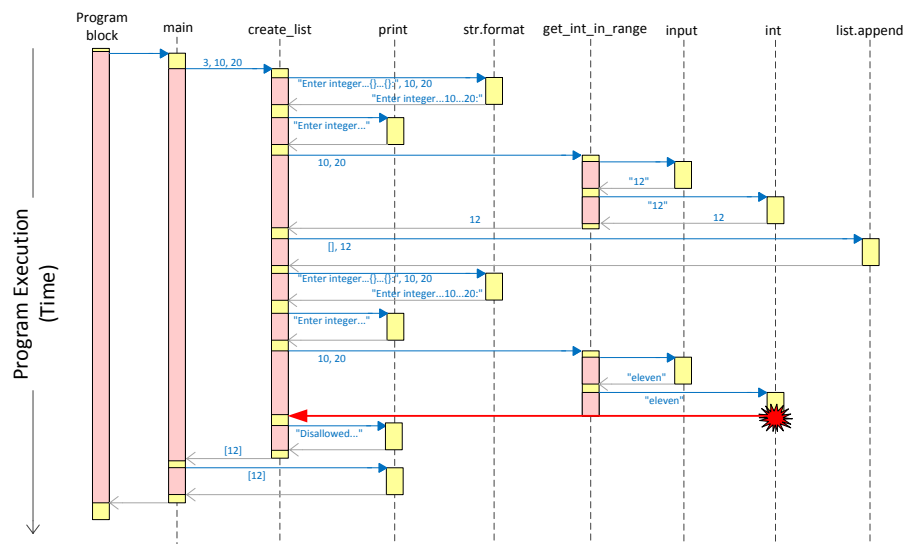
Enter integer in the range 10...20: 12
Enter integer in the range 10...20: eleven
Disallowed user entry interrupted list creation
[12]

```

Figure 12.4 diagrams the behavior of this sample run. When `get_int_in_range` calls the `int` constructor function with the argument `'eleven'`, the `int` function raises an exception. Since `get_int_in_range` has no exception handlers, the `ValueError` exception propagates back up the call chain to the nearest handler. The `create_list` function has a handler that can catch the exception, so it simply does not add any more elements to the list and returns the list as is to `main`.

Handling an exception closer to the location that raises it generally gives us more options for corrective action. Consider the following addition of an exception handler to the `get_int_in_range` function:

Figure 12.4 The execution sequence of a sample run of Listing 12.13 (makeintegerlist2.py) in which the `create_list` function handles the exception. Observe that the `int` function does not return to `get_int_in_range` as it normally would, but rather `int` bypasses `get_int_in_range` on its way to the nearest exception handler, which it locates in `create_list`.



```
def get_int_in_range(low: int, high: int) -> int:
    """ Obtains an integer value from the user. Acceptable values
        must fall within the specified range low...high. """
    need = True
    while need:
        try:
            val = int(input())          # Can raise a ValueError
            if val < low or val > high:
                print('Value out of range, please try again:', end=' ')
            else:
                need = False           # No need to continue in loop
        except ValueError:
            print('Value not a valid integer, please try again:', end=' ')
    return val
```

This version of `get_int_in_range` forces the user to enter a viable integer value, as we see in the following sample run:

```
Enter integer in the range 10...20: 12
Enter integer in the range 10...20: eleven
Value not a valid integer, please try again: eleven
Value not a valid integer, please try again: 11
[12, 11]
```

The handler is close to the source of the exception, and, therefore, can utilize local information to address the exception. This new version of `get_int_in_range` effectively eliminates the need for the exception handler in `create_list`.

We could exclude exception handlers from both `create_list` and `get_int_in_range` and instead let the `main` function handle the exception:

```
def main() -> None:
    """ Create a list of three elements supplied by the user,
        each element in the range 10...20 """
    try:
        lst = create_list(3, 10, 20)
        print(lst)
    except Exception:
        print('Error creating list')
```

The `main` function's reaction to the exception is more general than that of `create_list` and `get_int_in_range`, since it is farther away from the exception origin. Since `create_list` contains significant logic, `main` is powerless to attempt corrective action. Note that `main` cannot simply add a loop to allow the user to continue trying to add numbers each time the code within the `try` block raises an exception; doing so would call `create_list` again, and, since each call to `create_list` begins with the empty list, calling it again would lose any values the user entered before the exception appeared. The best `main` can do is report the program's inability to create the list.

Since the `main` function basically just wraps the call to `create_list`, we would not gain very much generality moving up to the program block level:

```

if __name__ == '__main__':
    try:
        main()      # Invoke main
    except Exception:
        print('Cannot create a list')

```

In general, exception handlers located closer to the code originating the exception have access to more local information that is useful for implementing more focused corrective action. As the exception is uncaught and propagates up the function-call chain, this local information is lost, and corrective action, if any, must become less focused and more generic.

In Section 8.12 we considered a program that compared the behavior of a dice simulator to that of a pair of actual, physical dice. Listing 8.36 (comparerolls.py) included a `read_file` function, reproduced here:

```

def read_file(filename: str, n: int, val: int) -> int:
    """ Reads n integers from the text file named filename.
        Returns the number of times val appears in the file. """
    count, read = 0, 0
    with open(filename, 'r') as f:
        for value in f.readlines():
            read += 1
            # Have we read enough values in yet?
            if read > n:
                break
            # Convert text integer into an actual integer
            if int(value) == val:
                count += 1
    return count

```

Notice that we used `read_file` in Listing 8.36 (comparerolls.py) to read data from the results of a dice rolling experiment, but `read_file` is not really dice-specific. The function simply reads `n` integers from any compatible text file, counting how many times the value `val` appears in the file. Given the string name of a text file, `read_file` is to open the text file and read its contents consisting of integer data.

What are some things that could go wrong with the `read_file` function? Consider the following possibilities:

- a file by the given name does not exist, at least not in the location expected by the program
- a file by the given name exists, but the program (or more precisely, the user running the program) is not authorized to read the file
- the device where the file resides is not ready for reading; for example, a DVD is not present in the drive
- the program encountered bad sectors on the disk while reading the file
- the data in the file is not of the proper format; for example, the data consists of arbitrary strings rather than text convertible to integers
- some other file error exists

As you can see, there are many ways the program can fail. The caller could misspell the file's name, or the file may reside in a different folder on the disk. A file of that name may exist in the right place but contain non-integer text. It seems the program's failure is a strong possibility.

Listing 12.14 (`comparerollsrobust.py`) takes advantage of our new knowledge to better manage the exceptions that Listing 8.36 (`comparerolls.py`) can raise.

Listing 12.14: `comparerollsrobust.py`

```
from random import randint
from functools import partial
from typing import Callable

def read_file(filename: str, n: int, val: int) -> int:
    """ Reads n integers from the text file named filename.
        Returns the number of times val appears in the file. """
    count, read = 0, 0
    with open(filename, 'r') as f:
        for value in f.readlines():
            read += 1
            # Have we read enough values in yet?
            if read > n:
                break
            # Convert text integer into an actual integer
            if int(value) == val:
                count += 1
    return count

def roll(n: int, val: int) -> int:
    """ Simulates the roll of a pair of dice n times.
        Returns the number of times a roll resulted in val. """
    count = 0
    for i in range(n):
        roll = randint(1, 6) + randint(1, 6)
        if roll == val:
            count += 1
    return count

def run_trials(func: Callable[[int, int], int], n: int) -> None:
    """ Performs n experiments using function f as the source of
        outcomes. Counts the number of occurrences of each possible
        outcome. """
    for value in range(2, 13):
        print(f'{value:3}:{func(n, value):5}')

def main() -> None:
    """ Compare the actual experiments to the simulation """
    number_of_trials = 100
    print('--- Pseudorandom number rolls ---')
    run_trials(roll, number_of_trials)
    print('--- Actual experimental data ---')
    try:
        run_trials(partial(read_file, 'dicedata.data'), number_of_trials)
    except FileNotFoundError:
        print('Cannot open the file "dicedata.data"')
```

```
except PermissionError:
    print('You do not have access to the file "dicedata.data"')
except OSError:
    print('Cannot read the file "dicedata.data"')
except ValueError:
    print('Data formatting error in "dicedata.data"')

if __name__ == '__main__':
    main()
```

In Listing 12.14 (`comparerollsrobust.py`) the `try` statement wraps the single statement that calls `read_file`, since the rest of the code should execute safely. The operating system will raise a `FileNotFoundError` exception on an attempt to open a file that does not exist in the current folder. If the OS can find the file but the user does not have permission to access the file, the OS will raise a `PermissionError` exception. The `OSError` covers all file related errors, such as attempting to process a corrupted file. In fact, just as `Exception` is the type that matches all routine exception types, the `OSError` type covers both the `FileNotFoundError` and `PermissionError` exceptions, as well as other file problems. We include the more specific exceptions to provide more helpful messages to the user. Also note that because `OSError` is more general than `FileNotFoundError` and `PermissionError`, its `except` block must appear after the `except` blocks of both `FileNotFoundError` and `PermissionError`. If `OSError`'s `except` block appears in the source earlier, it will catch the file not found and permission error exceptions before the more specific handlers get a chance. The `OSError` exception type is good to use if you need to defend against all file processing errors but do not need the finer-grained control offered by the more specific file exception types.

The `read_file` function will raise a `ValueError` if it extracts a line of text from the file that it cannot convert to an integer.

The following is a sample run of Listing 12.14 (`comparerollsrobust.py`) with a missing data file:

```
--- Pseudorandom number rolls ---
2: 5
3: 3
4: 9
5: 11
6: 13
7: 8
8: 16
9: 14
10: 8
11: 5
12: 4
--- Actual experimental data ---
Cannot open the file "dicedata.data"
```

The following shows a sample run in which the letter *R* replaces one of the integers in the data file:

```
--- Pseudorandom number rolls ---
2: 2
3: 5
4: 9
5: 8
6: 14
7: 13
8: 15
9: 13
10: 11
11: 11
12: 4
--- Actual experimental data ---
Data formatting error in "dicedata.data"
```

It is important to note the absence of a catch-all handler, introduced in Section 12.5. Any type of exception other than those specified by the `except` blocks will terminate the program with an error message. We could have added a catch-all exception handler that prints a message such as *Some other error occurred*, but such a message is no more helpful to the user than a cryptic stack trace. To the developers, however, the stack trace printed by the uncaught exception is invaluable for precisely locating the source of the problem so they can address it.

As mentioned in Section 12.5, you should limit your use of catch-all exception handlers. Catch-all handlers have the potential to “swallow” exceptions; that is, code within a function will catch an exception it did not expect and perhaps attempt some generic remedial action. The caught exception then will not propagate up to its caller, and so the caller (and its callers further up the call chain) will be cut off from the notification of the problem.

What is an appropriate use of a catch-all exception handler? A called function may need to do some sort of local damage control if it encounters any kind of exception, expected or not. Perhaps the function simply needs to log the unexpected error in its own error file. In this case the function can use the catch-all exception handler. The last action in the catch-all exception handler should be re-raising the exception (see Section 12.8). This allows one or more of its callers up the call chain to deal with the exception. In situations that warrant a catch-all handler, prefer to catch the `Exception` type; use the untyped catch-all handler only as a last resort.

12.8 Raising Exceptions

We have seen how to write code that reacts to exceptions. We know that certain functions, like `open` and `int` can raise exceptions. Also, statements that attempt to divide by zero or print an undefined variable will raise exceptions. The following statement raises a `ValueError` exception:

```
x = int('x')
```

The following statement is a more direct way to raise a `ValueError` exception:

```
raise ValueError()
```

The `raise` keyword raises an exception. Its argument, if present, must be an exception object. The class constructor for most exception objects accepts a string parameter that provides additional information to handlers:

```
raise ValueError('45')
```

In the interactive shell:

```
>>> raise ValueError()
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
ValueError
>>> raise ValueError('45')
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
ValueError: 45
>>> raise ValueError('This is a value error')
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
ValueError: This is a value error
```

This means we can write a custom version of the `int` constructor that behaves similar to the built-in `int` constructor, as Listing 12.15 (`nonnegconvert.py`) illustrates.

Listing 12.15: `nonnegconvert.py`

```
def non_neg_int(n: int) -> int:
    result = int(n)
    if result < 0:
        raise ValueError(result)
    return result

while True:
    x = non_neg_int(input('Please enter a nonnegative integer:'))
    if x == 999: # Secret number exits loop
        break
    print('You entered', x)
```

The following shows a sample run of Listing 12.15 (`nonnegconvert.py`):

```
Please enter a nonnegative integer:3
You entered 3
Please enter a nonnegative integer:-3
Traceback (most recent call last):
  File "nonnegconvert.py", line 8, in <module>
    x = nonnegint(input('Please enter a nonnegative integer:'))
  File "nonnegconvert.py", line 4, in nonnegint
    raise ValueError(result)
ValueError: -3
```

The built-in `int` function accepts well-formed negative integers with no problem, but our `non_neg_int` function is more selective. It does not accept valid negative integers; hence, it raises a value error. Listing 12.16 (`nonnegexcept.py`) catches the exception so the program does not crash.

Listing 12.16: nonnegexcept.py

```
def non_neg_int(n):  
    """ Converts argument n into a nonnegative integer, if possible.  
        Raises a ValueError if the argument is not convertible  
        to a nonnegative integer. """  
    result = int(n)  
    if result < 0:  
        raise ValueError(result)  
    return result  
  
while True:  
    try:  
        x = non_neg_int(input('Please enter a nonnegative integer:'))  
        if x == 999: # Secret number exits loop  
            break  
        print('You entered', x)  
    except ValueError:  
        print('The value you entered is not acceptable')
```

The following shows a sample run of Listing 12.16 (nonnegexcept.py):

```
Please enter a nonnegative integer:3  
You entered 3  
Please enter a nonnegative integer:-3  
The value you entered is not acceptable  
Please enter a nonnegative integer:5  
You entered 5  
Please enter a nonnegative integer:999
```

If one of Python’s built-in exception types is not appropriate to describe the exception you need to raise, you can use the generic `Exception` class and provide a descriptive message to its constructor, as in

```
raise Exception('Cannot add non-integer to restricted list')
```

If raised and uncaught, the interpreter will print the following line at the end of the stack trace:

```
Exception: Cannot add non-integer to restricted list
```

In Section 14.2 we will see a better way to customize exceptions by designing our own custom exception classes that integrate seamlessly with Python’s exception handling infrastructure.

Sometimes it is appropriate for a function to catch an exception, take some action appropriate to its local context, and then re-raise the same exception so that the function’s caller can take further action if necessary. In essence, the function that catches the exception first administers “first aid” and then passes the exception up the call chain for more advanced, application-specific treatment and care. In Listing 12.17 (reraise.py), the `count_elements` function accepts a list, `lst`, presumed to contain only integers, and a Boolean function predicate. The predicate function parameter accepts a single argument and returns true or false based on whether or not its argument parameter has a certain property. The program defines two such predicate functions: `is_prime` and `non_neg`. The `is_prime` function determines if its integer argument is prime, and the `non_neg` function determines if its argument is a nonnegative integer. Both `is_prime` and `non_neg` can raise a `TypeError` exception if the caller passes a non-integer argument. Neither the `is_prime` nor the `non_neg` function attempts to handle the `TypeError` exception itself. Listing 12.17 (reraise.py) exercises `count_elements` with the `is_prime` and `non_neg` functions.

Listing 12.17: reraise.py

```

from typing import Callable

def is_prime(n: int) -> bool:
    """ Returns True if nonnegative integer n is prime;
        otherwise, returns false.
        Raises a TypeError exception if n is not
        an integer. """
    from math import sqrt
    if n == 2:                                     # 2 is the only even prime number
        return True
    if n < 2 or n % 2 == 0:                         # Handle simple cases immediately
        return False                               # Raises a TypeError if n is not an integer
    trial_factor = 3
    root = round(sqrt(n))
    while trial_factor <= root:
        if n % trial_factor == 0:                  # Is trial factor a factor?
            return False                           # Yes, return right away
        trial_factor += 2                          # Next potential factor, skip evens
    return True                                    # Tried them all, must be prime

def non_neg(n: int) -> bool:
    """ Determines if n is nonnegative.
        Raises a TypeError if n is not an integer. """
    return n > 0

def count_elements(lst: list[int], predicate: Callable[[int], bool]):
    """ Counts the number of integers in list lst that are
        acceptable to a given predicate (Boolean function).
        Prints an error message and raises a type error if
        the list contains an element incompatible with
        the predicate. """
    count = 0
    for x in lst:
        try:
            if predicate(x):
                count += 1
        except TypeError:
            print(x, 'is a not an acceptable element')
            raise # Re-raise the caught exception
    return count

def main() -> None:
    print(count_elements([3, -71, 22, -19, 2, 9], non_neg))
    print(count_elements([2, 3, 4, 5, 6, 8, 9], is_prime))
    print(count_elements([2, 4, '6', 8, 'x', 7], is_prime))

if __name__ == '__main__':

```

```
main()
```

The `count_elements` function uses a `try` statement to defend against the possible exceptions raised by `is_prime` and `non_neg`. If it catches a `TypeError` exception, it prints a diagnostic message alerting the user that it found an element in the list that is not compatible with the predicate's expected parameter type.

Listing 12.17 (`reraise.py`) prints the following:

```
4
3
6 is a not an acceptable element
Traceback (most recent call last):
  File "reraise.py", line 48, in <module>
    main()
  File "reraise.py", line 44, in main
    print(count_elements([2, 4, '6', 8, 'x', 7], is_prime))
  File "reraise.py", line 33, in count_elements
    if predicate(x):
  File "reraise.py", line 9, in is_prime
    if n < 2 or n % 2 == 0:      # Handle simple cases immediately
TypeError: unorderable types: str() < int()
```

As we can see, the program prints the error message from the `except` block in `count_elements`, and then terminates due to handler's re-raising of the exception it caught. The printed stack trace preserves all the information about where the exception originated.

After catching the `TypeError` exception and reporting the problem, the `count_elements` function re-raises the same exception for the benefit of its caller via the statement

```
raise    # Re-raises the same exception it caught
```

Notice that this is different from the statement

```
raise TypeError()    # Raise a NEW TypeError exception
```

This latter statement creates a new exception object with its own stack trace that is different from the caught exception object. Raising the new exception results in a more complicated stack trace, as the following shows:

```

4
3
6 is a not an acceptable element
Traceback (most recent call last):
  File "reraise.py", line 33, in count_elements
    if predicate(x):
  File "reraise.py", line 9, in is_prime
    if n < 2 or n % 2 == 0:      # Handle simple cases immediately
TypeError: unorderable types: str() < int()

During handling of the above exception, another exception occurred:

Traceback (most recent call last):
  File "reraise.py", line 48, in <module>
    main()
  File "reraise.py", line 44, in main
    print(count_elements([2, 4, '6', 8, 'x', 7], is_prime))
  File "reraise.py", line 37, in count_elements
    raise TypeError() #e      # Re-raise the exception
TypeError

```

You also use the unadorned `raise` statement to re-raise an exception caught by an untyped `except` handler. This is because the `raise` keyword appearing by itself within an `except` block will re-raise the same exception object that the block caught.

If we wrap the calling code of Listing 12.17 (`reraise.py`) with a `try`, as in

```

# Wrap the calling code in a try/except statement
def main():
    try:
        print(count_elements([3, -71, 22, -19, 2, 9], non_neg))
        print(count_elements([2, 3, 4, 5, 6, 8, 9], is_prime))
        print(count_elements([2, 4, '6', 8, 'x', 7], is_prime))
    except TypeError:
        print('Error in count_elements')

```

the calling code's `except` block produces simpler output:

```

4
3
6 is a not an acceptable element
Error in count_elements

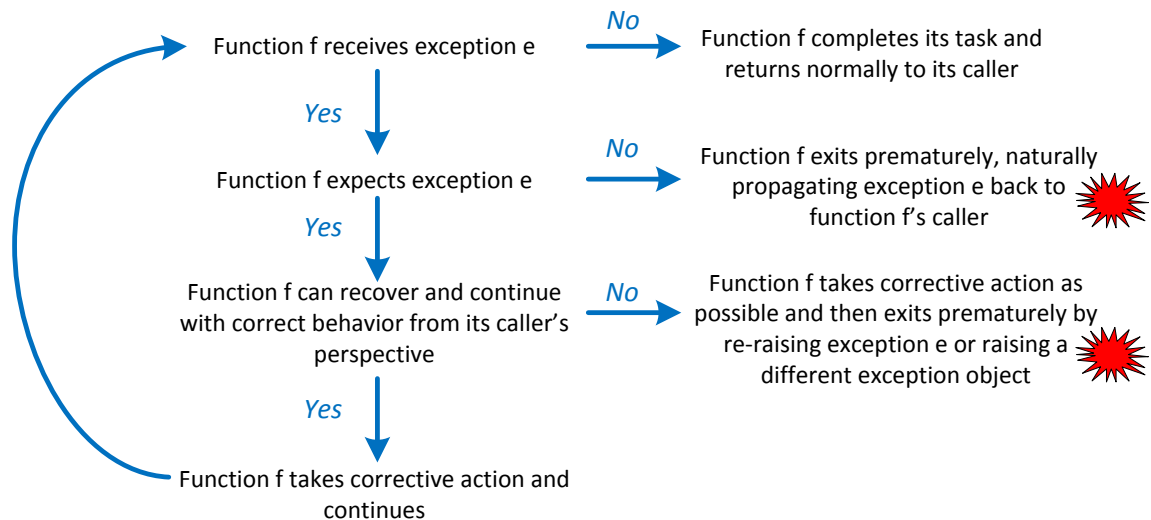
```

What is the reason for re-raising an exception? After all, the `count_elements` function could just print the message and continue. If it does so, however, the count that it eventually returns would be meaningless, and its caller would not know that `count_elements` had a problem. Re-raising the exception enables `count_elements`'s caller to be informed of the failure so the caller can react to the exception in its own way.

Note that with type hints, an IDE such as VS Code will indicate that the last `print` statement in the `main` function has a type error. We will know even before the program executes that it has a problem.

In general, suppose some function A calls function B that calls function C. The call chain thus looks like

Figure 12.5 A flowchart of the execution of a function named *f* as it encounters exceptions. Function *f* handles exceptions it expects, potentially repairing what it can and sending an exception to its caller if it cannot. Function *f* ignores exceptions it does not expect or cannot handle.



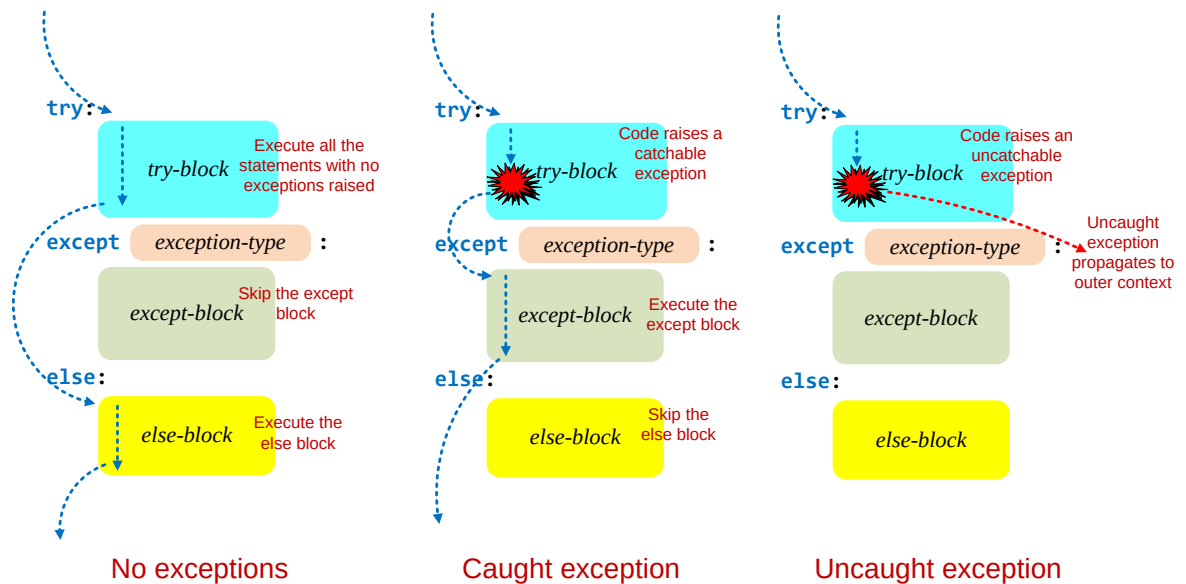
$A \rightarrow B \rightarrow C$

If *C* raises an exception, functions *A* and *B* both may need to know about it to take appropriate action. Function *B* is closer to *C* in the call chain. *B* can catch the exception raised by *C*, remedy the situation as best it can, and then ensure that its caller (*A*) receives the same exception. *A* then can take action appropriate to its own context.

The idea is that *B* is the caller in the call chain closest the exception origin (*C*), and *B* has information unique to its context that its caller (*A*) would not have. *B* should handle any exceptions it expects and can handle in some way. If the exception is such that *B* can repair the situation, continue its execution, and in the end correctly fulfill *A*'s expectations, then there is no reason for *B* to re-raise the exceptions it caught from *C*. On the other hand, if *C*'s exception renders *B* unable to meet *A*'s expectations, *B* can do local damage control but must also raise an exception that *A* can process. Often this means re-raising the same exception, but it can mean raising a different exception that is more *B*-specific.

What if *C* raises an exception type that *B* does not expect? This means *B* has no `except` block to handle that type of exception. In this case, the exception propagates naturally back up to *A*, and it then it becomes *A* responsibility to deal with it.

This technique represents a good rule of thumb for managing exceptions. A function should catch any exceptions it expects, ignore exceptions it does not expect (or cannot handle in some way), and avoid a catch-all exception handler. Figure 12.5 illustrates this basic exception handling strategy.

Figure 12.6 The possible program execution flows through a `try/else` statement.

12.9 The `try` Statement's Optional `else` Block

The Python `try` statement supports an optional `else` block. Its behavior is reminiscent of the `while` statement's `else` block (see Section 5.6). If the code within the `try` block does not produce an exception, no `except` blocks trigger, and the program's execution continues with code in the `else` block. Figure 12.6 contrasts the possible program execution flows within a `try/else` statement. The `else` block, if present, must appear after all of the `except` blocks.

Since the code in the `else` block executes only if the code in the `try` block does not raise an exception, why not just append the code in the `else` block to the end of the code within the `try` block and eliminate the `else` block altogether? The code restructured in this way may not behave identically to the original code. Consider Listing 12.18 (`trynoelse.py`) which demonstrates the different behavior.

Listing 12.18: trynoelse.py

```
def fun1() -> None:
    try:
        print('try code')
    except:
        print('exception handling code')
    else:
        print('no exception raised code')
        x = int('a') # Raises an exception

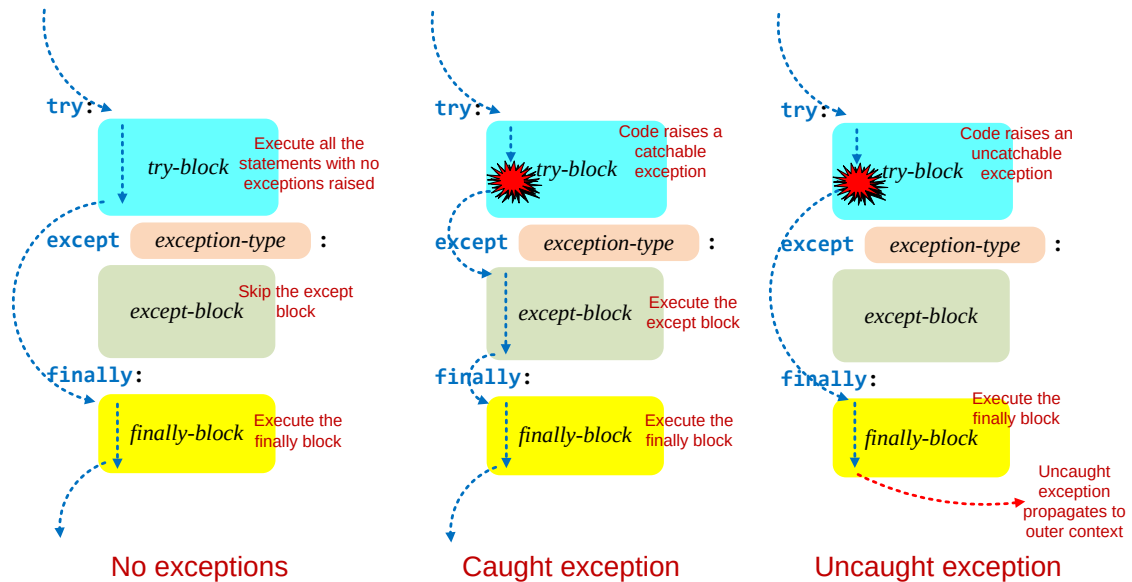
def fun2() -> None:
    try:
        print('try code')
        print('no exception raised code')
        x = int('a') # Raises an exception
    except:
        print('exception handling code')

print('Calling fun2')
fun2()
print('-----')
print('Calling fun1')
fun1()
```

The fun1 function in Listing 12.18 (trynoelse.py) uses an `else` block, and fun2 moves the `else` code up into the `try` block. The program's output is

```
Calling fun2
try code
no exception raised code
exception handling code
-----
Calling fun1
try code
no exception raised code
Traceback (most recent call last):
  File "trynoelse.py", line 22, in <module>
    fun1()
  File "trynoelse.py", line 8, in fun1
    x = int('a') # Raises an exception
ValueError: invalid literal for int() with base 10: 'a'
```

If the code in the original `else` block can raise an exception, moving it up into the `try` block means that one of the `try` statement's `except` blocks could catch that exception. Leaving the code in the `else` block means none of the `try` statement's `except` blocks will be able to handle any exceptions the code in the `else` block might raise.

Figure 12.7 The possible program execution flows through a `try/except/finally` statement.

12.10 finally block

A `try` statement may include an optional `finally` block. Code within a `finally` block always executes whether the `try` block raises an exception or not. A `finally` block usually contains “clean-up code” that must execute due to activity initiated in the `try` block. Figure 12.7 contrasts the possible program execution flows within a `try/except/finally` statement.

Consider Listing 12.19 (`riskyread.py`) which opens a file and reads its contents.

Listing 12.19: `riskyread.py`

```
# Sum the values in a text file containing integer values
sum = 0
f = open('mydata.dat')
for line in f:
    sum += int(line)
f.close() # Close the file
print(sum)
```

Listing 12.19 (`riskyread.py`) does not use a `with/as` statement as recommended in Section 9.3, and so it will have a problem should an exception arise before the program executes the `f.close()` statement. The file could be corrupted and one or more of the lines could contain text that is not convertible to a number. Either of these problems would raise an exception before the `f.close` method executes.

Listing 12.20 (`tryfileread.py`) rectifies the problems with Listing 12.19 (`riskyread.py`) by adding exception handling with nested `try` statements.

Listing 12.20: tryfileread.py

```
# Sum the values in a text file containing integer values
try:
    f = open('mydata.dat')
except OSError:
    print('Could not open file')
else: # File opened properly
    sum = 0
    try:
        for line in f:
            sum += int(line)
        f.close() # Close the file if no exception
    except Exception as er:
        print(er) # Show the problem
        f.close() # Close the file if exception
    print('sum =', sum)
```

Listing 12.20 (tryfileread.py) uses two `try` statements. The first `try` statement defends against an `OSError` exception. The operating system will prompt the `open` function to raise such an exception if it cannot satisfy the request; for example, the file may not exist in the current directory or the user may not have sufficient permissions to access the file. The program does not proceed if it cannot open the file for reading.

The code in the outer `try` statement's `else` block contains the interesting part. Once the file is open, more exceptions are possible. In particular, the text file may contain a string that does not evaluate to a number. The inner `try` statement includes a catch-all `except` block to handle any exception that may arise.

If the file `mydata.dat` contains the following:

```
5.5
2.0
6.1
```

Listing 12.20 (tryfileread.py) will print

```
sum = 13.6
```

If the file instead contains the following:

```
5.5
2.0
five
6.1
```

Listing 12.20 (tryfileread.py) will print

```
name 'five' is not defined
sum = 7.5
```

Finally, if no file named `mydata.dat` exists in the current directory, Listing 12.20 (tryfileread.py) will print

```
Could not open file
```

Observe that Listing 12.20 (tryfileread.py) contains two identical statements to close the file:

- If the execution makes it all the way to the end of the inner `try` block, it needs to close the file.
- If an exception arises in the inner `try` block, the exception handler must close the file.

This code duplication is undesirable, so we can use a `finally` block to consolidate the code, as shown in Listing 12.21 (filereadfinally.py).

Listing 12.21: filereadfinally.py

```
# Sum the values in a text file containing integers
try:
    f = open('mydata.dat')
except OSError:
    print('Could not open file')
else:
    sum = 0
    try:
        for line in f:
            sum += int(line)
    except Exception as er:
        print(er) # Show the problem
    finally:
        f.close() # Close the file
    print('sum =', sum)
```

Listing 12.21 (filereadfinally.py) behaves exactly like Listing 12.20 (tryfileread.py) contains.

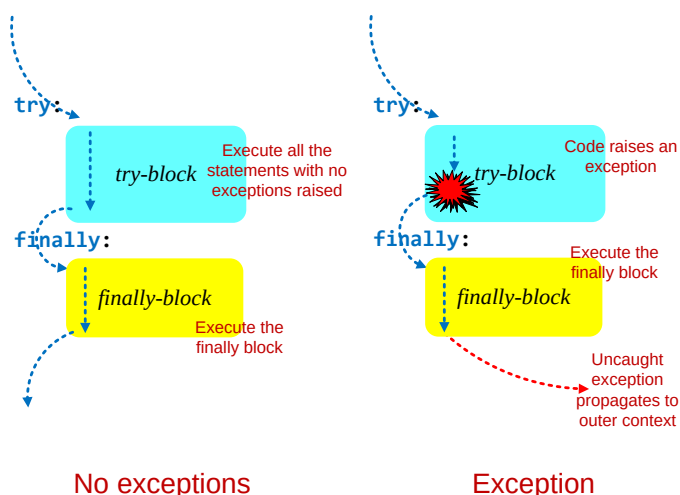
Because of the design of the file class, the `with/as` statement takes care of the details of properly closing a file should an exception arise. The `with/as` statement, however, will not automatically handle any exceptions. We can remedy this by adding a `try` statement inside the `with` block, as Listing 12.22 (betterfileread.py) shows.

Listing 12.22: betterfileread.py

```
# Sum the values in a text file containing integers
try:
    with open('mydata.dat') as f:
        sum = 0
        try:
            for line in f:
                sum += int(line)
        except Exception as er:
            print(er) # Show the problem
        print('sum =', sum)
except OSError:
    print('Could not open file')
```

Since Listing 12.22 (betterfileread.py) uses the `with/as` statement, the program does not explicitly call the file object's `close` method. The omission of the `f.close()` statement eliminates the need of the `finally` block.

The `try` keyword cannot appear without at least one of `except` or `finally`. This means the `except` blocks are optional. In fact, the following code fragment:

Figure 12.8 The possible program execution flows through a `try/finally` statement.

```
with open('data.dat') as f:      # f is a file object
    for line in f:              # Read each line as text
        print(line.strip())     # Remove trailing newline character
```

is roughly equivalent to

```
f = None      # No file object by default
try:
    f = open('data.dat')    # f is a file object
else:
    for line in f:          # Read each line as text
        print(line.strip()) # Remove trailing newline character
finally:
    if f:
        f.close()          # Close the file, if open
```

Figure 12.8 compares the possible program execution flows within a `try/finally` statement.

The `except` and `finally` blocks may not appear without an associated `try` block. An `else` block may appear only in the context of a `try`, `if`, `while`, or `for` statement.

As a final note about `finally`: the `finally` block, if present, must appear after all `except` blocks and after the `else` block.

12.11 Using Exceptions Wisely

Exceptions should be reserved for uncommon errors. For example, the following code adds up all the elements in a list of numbers named `lst`:

```

sum = 0
for elem in lst:
    sum += elem
print("Sum =", sum)

```

This loop is fairly typical. Another, much poorer, approach uses an exception:

```

sum = 0
int i = 0
try:
    while True:
        sum += lst[i]
        i += 1
except IndexError:
    pass
print("Sum =", sum)

```

Here, an `IndexError` exception interrupts the loop when the list access is out of bounds. The exception interrupts the statement

```
sum += lst[i]
```

in midstream, before it fully evaluates the expression on the right side of the `+=` operator. This prevents the program's execution from incorrectly incrementing `sum`'s value.

Both approaches compute the same result. However, the second approach *always* raises and handles an exception. The exception definitely is **not** an uncommon occurrence. You should not use exceptions to dictate normal logical flow. While very useful for its intended purpose, the exception mechanism adds some overhead to program execution, especially when an exception is raised. This overhead is reasonable when exceptions are rare but not when exceptions are part of the program's normal execution.

Exceptions are valuable aids for careless or novice programmers. A careful programmer ensures that code accessing a list does not exceed the list's bounds. Another programmer's code may accidentally attempt to access `a[len(a)]`. A novice may believe `a[len(a)]` is a valid element. Since no programmer is perfect, exceptions provide a nice safety net.

As you develop more sophisticated programs you will find exceptions more compelling. You should analyze your code carefully to determine its limitations. Exceptions can be valuable for covering these limitations. The Python standard library uses exceptions extensively, so programs that make use of library functions and classes should properly handle the exceptions they can raise.

12.12 Exercises

1. What does LBYL stand for in Python programming? What does it mean?
2. What does EAFP stand for in Python programming? What does it mean?
3. What is the maximum number of `try` blocks in a `try` statement?
4. What is the minimum number of `try` blocks in a `try` statement?
5. What is the maximum number of `except` blocks in a `try` statement?
6. What is the minimum number of `except` blocks in a `try` statement?

7. What is the maximum number of `else` blocks in a `try` statement?
8. What is the minimum number of `else` blocks in a `try` statement?
9. What is the maximum number of `finally` blocks in a `try` statement?
10. What is the minimum number of `finally` blocks in a `try` statement?
11. Wrap the following code in a `try` statement to defend against any exceptions it can raise. Do not use a catch-all handler.

```
lst = [0, 0, 0, 0]
with open('data.txt', 'r') as f:
    count = 0
    for line in f.readlines():
        lst[count] = int(line)
        count += 1
```

12. For the next set of questions show what each program will print when the user supplies the indicated input text.

Write **EXCEPTION** if and when the execution will generate an exception stack trace for an uncaught exception.

(a)

```
print('Begin')
x = int(input())
print(x)
print('End')
```

- i. User enters 22
- ii. User enters ZZ

(b)

```
print('Begin')
try:
    x = int(input())
    print(x)
except ValueError:
    print('Wrong!')
print('End')
```

- i. User enters 22
- ii. User enters ZZ

(c)

```
print('Begin')
try:
    x = int(input())
    print(x)
except IndexError:
    print('Wrong!')
print('End')
```

- i. User enters 22
- ii. User enters ZZ

```
(d) print('Begin')
    try:
        x = int(input())
        print(x)
    except Exception:
        print('Wrong!')
    print('End')
```

- i. User enters 22
- ii. User enters ZZ

```
(e) print('Begin')
    try:
        x = int(input())
        print(x)
    except ValueError:
        print('Wrong!')
    else:
        print('Wow')
    print('End')
```

- i. User enters 22
- ii. User enters ZZ

```
(f) print('Begin')
    try:
        x = int(input())
        print(x)
    except ValueError:
        print('Wrong!')
    finally:
        print('Done')
    print('End')
```

- i. User enters 22
- ii. User enters ZZ

```
(g) print('Begin')
    try:
        x = int(input())
        print(x)
    except ValueError:
        print('Wrong!')
    else:
        print('Wow')
    finally:
        print('Done')
    print('End')
```

- i. User enters 22
- ii. User enters ZZ

13. What is the problem with the following code?

```
try:
    f() # Function f can raise an exception
except Exception:
    print(1)
except ValueError:
    print(2)
```

14. What is the problem with the following code?

```
try:
    f() # Function f can raise an exception
except OSError:
    print(1)
except FileNotFoundError:
    print(2)
```

Chapter 13

Custom Types

We have examined many of Python's built-in types. Some, like, integers, floating-point numbers, and Booleans are relatively simple, while others such as lists, tuples, dictionaries, sets, and exceptions are more complex. Python's rich collection of built-in types enable us to write a wide variety of programs in diverse problem domains. Python also provides the ability for programmers to design their own custom types. This enables developers to craft data types that more closely model the problem at hand. This better alignment of software assets with the problem domain can expedite the development process.

13.1 Structured Data

Listing 13.1 (myclass.py) specifies the simplest programmer-defined type.

Listing 13.1: myclass.py

```
class MyClass:  
    pass
```

The `class` keyword along with the `MyClass` identifier introduces a new type of object to the program. Figure 13.1 shows the general form of a class definition. As in function definitions, we indent the block of code that constitutes the body of the class. The `pass` statement in the class definition indicates that the

Figure 13.1 General form of a class definition

```
class name :  
    block
```

programmer is not providing any specific information about the nature of this new type. This empty-looking class, however, is far from empty. Every class in Python by default comes with some basic functionality. Chapter 14 introduces the concept of class *inheritance*, but for now it is sufficient to know that an “empty” class contains some foundational infrastructure that is not apparent from the two lines of Python source code. The following interactive session demonstrates:

```
>>> class MyClass:
...     pass
...
>>> MyClass
<class '__main__.MyClass'>
>>> dir(MyClass)
['__class__', '__delattr__', '__dict__', '__dir__', '__doc__', '__eq__',
 '__format__', '__ge__', '__getattribute__', '__getstate__', '__gt__',
 '__hash__', '__init__', '__init_subclass__', '__le__', '__lt__',
 '__module__', '__ne__', '__new__', '__reduce__', '__reduce_ex__',
 '__repr__', '__setattr__', '__sizeof__', '__str__', '__subclasshook__',
 '__weakref__']
```

The `dir` function lists the components of the `MyClass` class.

Given the class definition, we can create an instance of the class:

```
>>> m = MyClass()
>>> m
<__main__.MyClass object at 0x0000025407E47BF0>
```

The statement

```
m = MyClass()
```

calls the `MyClass` class constructor. This creates a `MyClass` object and binds variable `m` to the newly created object.

Except for the internal implementation details that all Python objects must manage, the `MyClass` object bound to our `m` variable does not provide any value to clients. We can add data to the object via assignment:

```
>>> m.x = 45
>>> m.count = 100
>>> print(m.x)
45
>>> print(m.count)
100
```

We associate a variable with an object using the dot (`.`) operator and the assignment operator. The name `x` in this situation is known as an *instance variable*. It is a variable associated with an instance of a class. The term *attribute* is sometimes used in place of *instance variable*.

An attempt to access an instance variable before it has been defined results in an error:

```
>>> print(m.y)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
AttributeError: 'MyClass' object has no attribute 'y'
```

Listing 13.2 (multipleobjects.py) shows we can create an many instances of a class as we like.

Listing 13.2: multipleobjects.py

```

from myclass import MyClass

m1 = MyClass()
m2 = MyClass()
m1.x = 45
print(f'm1.x = {m1.x}')
print(f'm2.x = {m2.x}')    # Error, no x in m2.

```

Listing 13.2 (multipleobjects.py) prints

```

m1.x = 45
Traceback (most recent call last):
  File "C:\Code\python\twoobjects.py", line 7, in <module>
    print(f'm2.x = {m2.x}')
          ^^^^^
AttributeError: 'MyClass' object has no attribute 'x'

```

The code establishes the `x` attribute in the `MyClass` object bound to variable `m1`, but `m2`'s object has no `x` attribute. This illustrates the point that individual objects have their own instance variables. Different objects of the same class do not share instance variables.

What if we wish to define our own mathematical point type? We want all point objects to have x and y coordinates. We do not want to create a point object and then also have to separately create the x and y attributes. We need a way to automatically initialize instance variables within an object when a client creates the object. This way, the attributes are present from the beginning, and clients do not have to remember to add them later. Listing 13.3 (point0.py) shows how to ensure point objects automatically will possess x and y instance variables.

Listing 13.3: point0.py

```

class Point0:
    def __init__(self) -> None:
        self.x = 0
        self.y = 0

```

The code in Listing 13.3 (point0.py) defines an *initializer method* for the class. The initializer method must be named `__init__`, and it must specify at least one formal parameter. This parameter can be any valid identifier, but the overwhelming convention within the Python programming world names this parameter `self`. The `self` parameter refers to the object being created. There is no need to provide a type hint for the `self` parameter in a method definition, as the type always will be the class. The statement

```
p = Point0()
```

calls the `Point0` class constructor, and the constructor calls the `__init__` method behind the scenes. The following shows how a client could create the point (1,4):

```
>>> from point0 import Point0
>>>
>>> p = Point0()
>>> p.x
0
>>> p.y
0
>>> p.x, p.y = 1, 4
>>> p.x
1
>>> p.y
4
```

We refer to the `__init__` code as a *method*. A method is like a function, but a method's definition appears in a class, while a function's definition is not part of a class.

A better initializer method would allow the client to specify the desired `x` and `y` attributes when creating the object. The following `__init__` method improves the original version:

```
class Point0:
    def __init__(self, x: float=0.0, y: float=0.0) -> None:
        self.x = x
        self.y = y
```

With this change, clients can specify the initial coordinates of a `Point0` object during its creation or simply go with the default (0,0):

```
p1 = Point0(1, 4)
p2 = Point0()
print(f'p1 = ({p1.x}, {p1.y})')
print(f'p2 = ({p2.x}, {p2.y})')
```

This code prints

```
p1 = (1, 4)
p2 = (0.0, 0.0)
```

The first formal parameter to our `__init__` method still must be `self`. Without the `self` parameter, the initializer would not know which object should receive the `x` and `y` values. If the client omits the last two arguments, they default to zeros.

One weakness of our `Point0` class is that the `print` function does not treat it kindly. The following code fragment

```
p = Point0(1, 4)
print(p)
```

prints

```
<__main__.Point0 object at 0x000001B4B2B16090>
```

This output is not very helpful to human readers. To address this, we can define a method named `__repr__`. This special method provides a more human-friendly **representation** of an object. The method returns a

string, and the string it returns should be an accurate representation of the state of an object. The values of all the instance variables of an object define the object's state at any given time.

Listing 13.4 (point.py) completes a simple `Point` class by including a `__repr__` method.

Listing 13.4: point.py

```
class Point:
    def __init__(self, x: float=0.0, y: float=0.0) -> None:
        self.x = x
        self.y = y

    def __repr__(self) -> str:
        return f'({self.x}, {self.y})'
```

These simple modifications produce a programmer-defined type that behaves very much like a standard Python builtin type:

```
>>> from point import Point
>>> p1 = Point(3.4, 5.5)
>>> p1
(3.4, 5.5)
>>> p2 = Point()
>>> p2
(0.0, 0.0)
>>> p1.x = 99
>>> p1
(99, 5.5)
```

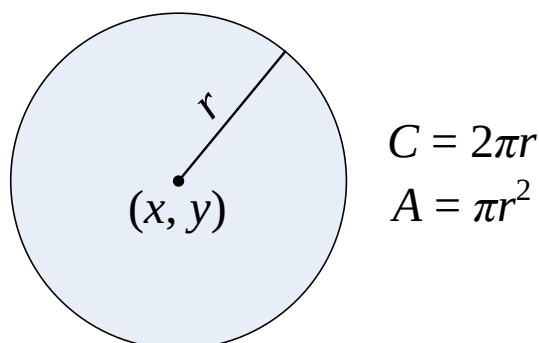
13.2 Circle Objects

A software object generally bundles together *data* (instance variables) and *functionality* (methods). The instance variables and methods of an object comprise its *members*. The class of an object defines the object's basic structure and capabilities. As a simple concrete example, consider the familiar geometric circle, shown in Figure 13.2. Given a circle's radius (r in Figure 13.2), we can compute the circle's area and circumference. The circle's center, (x,y) , establishes the circle's position. We will define a custom `Circle` class in Python from which we can create `Circle` instances (objects). The `Circle` class specifies what circle objects can do and how clients can interact with them. A client is any code outside of the `Circle` class needing the services that a `Circle` object can provide. A center and radius may be good enough for mathematicians, but in a graphical computer program circle objects may need other attributes like a fill color, fill style, edge thickness, etc. We will keep things simple for now and stick to the abstract mathematical concept of a circle.

What data must each `Circle` object maintain? Since circles can appear in various places, each circle must keep track of its own position. Just like in mathematics, we can specify the location of a `Circle` object by its (x,y) center. Also, since some circles are larger or smaller than others, each `Circle` should have its own radius. It is natural, then, for each of our `Circle` objects to have a `center` instance variable and a `radius` instance variable.

Should our circle objects have an `area` instance variable and/or a `circumference` instance variable? Both area and circumference depend solely on a circle's radius, so if we include a `radius` instance variable, both area and circumference would be redundant information—we easily can compute them as

Figure 13.2 A circle with radius r centered at (x, y) . A represents the circle's area, and C is its circumference



needed with simple formulas. Every additional instance variable we define for a class requires more memory, so if an application uses a lot of `Circle` objects, that could be a concern. We will implement `area` and `circumference` as methods in our `Circle` class.

As we finalize the design for our `Circle` class, we will add a few interesting points:

- Clients should be able to create a `Circle` object with a specified center point (a tuple of two numbers) and radius. We will require all of this information and not define any default parameters.
- Clients should be able to print a variable that refers to a `Circle` object, and its display should be meaningful to the client.
- An attempt to create a circle with a negative radius should produce a `ValueError` exception.
- Clients can reposition the circle via a `move` method.
- Clients can increase the circle's radius by one unit via a `grow` method.
- Clients can decrease the circle's radius by one unit via a `shrink` method. At no time should the circle's radius fall below zero.

Our intention is to discourage clients from changing the instance variables (`center` and `radius`) of `Circle` objects directly; thus, we offer the `move`, `grow`, and `shrink` methods. Nothing prevents clients from manipulating directly the `center` and `radius` attributes, if necessary.

Based on our criteria, we will need the following methods in our `Circle` class:

- `__init__`: The `__init__` method should contain code that initializes an object so it begins in a well-defined state. It establishes and initializes the `center` and `radius` instance variables of the newly created object. It detects and prevents an attempt to make a `Circle` object with a negative radius. The client code must supply a center (a tuple consisting of two numbers) and a radius.
- `__repr__`: This method must produce a string that accurately communicates the state of an object. In the case of the `Circle` class, the `__repr__` method returns a string containing the circle's center and radius.
- `area`: This method computes and returns the circle object's area.

- `circumference`: This method computes and returns the circumference.
- `move`: This method repositions the `Circle` object's center. The client must provide a tuple consisting of two numbers. This tuple represents the new coordinates of the object's center.
- `grow`: This method increases the `Circle` object's radius by one unit.
- `shrink`: If the `Circle` object's radius is greater than zero, this method decreases its radius by one unit. This method does not change the radius if the radius is zero before the call.

Armed with this more detailed specification, we are ready to define our `Circle` class. Listing 13.5 (`circle.py`) contains the complete definition for `Circle`.

Listing 13.5: `circle.py`

```
class Circle:
    """ Represents a geometric circle object """
    def __init__(self, center: float, radius: float) -> None:
        """ Initialize the center's center and radius """
        # Disallow a negative radius
        if radius < 0:
            raise ValueError('Negative radius')
        self.center = center
        self.radius = radius

    def area(self) -> float:
        """ Compute and return the area of the circle """
        from math import pi
        return pi*self.radius*self.radius

    def circumference(self) -> float:
        """ Compute and return the circumference of the circle """
        from math import pi
        return 2*pi*self.radius

    def move(self, pt: tuple[float, float]) -> None:
        """ Moves the center of the circle to point pt """
        self.center = pt

    def grow(self) -> None:
        """ Increases the radius of the circle """
        self.radius += 1

    def shrink(self) -> None:
        """ Decreases the radius of the circle;
            does not affect a circle with radius zero """
        if self.radius > 0:
            self.radius -= 1

    def __repr__(self) -> str:
        return f'Center:{self.center} Radius:{self.radius}'
```

The code in this class definition looks like a series of function definitions; however, since the definitions appear within the block of a class definition they are method definitions. As with functions, we can (and should) document classes and methods with docstrings.

Notice that each method definition has `self` for its first parameter. As mentioned in Section 13.1, the language does not require the first parameter's name to be `self` (it could be `x`, `obj`, or any valid identifier), but the universal convention in the Python programming world is to use the name `self`. During a method's execution, the `self` parameter references the object on whose behalf the method is executing. In Turtle graphics, for example, the `Turtle` class definition contains a `forward` method definition that begins

```
def forward(self, distance):  
    # Details omitted . . .
```

If `t` is a reference to a `Turtle` object, we can move the turtle forward 100 pixels with the call

```
t.forward(100)
```

This method invocation assigns the actual parameter `t` to the formal parameter `self` and the actual parameter 100 to the formal parameter `distance`. In fact, we can rewrite the call as

```
Turtle.forward(t, 100)
```

The call expressed this way clearly shows that `t` corresponds to `self` and 100 corresponds to `distance`.

In the case of our `Circle` class, if `circ` refers to a `Circle` object, the call

```
circ.grow()
```

is equivalent to

```
Circle.grow(circ)
```

In the `Circle` class definition, the `__init__` method initializes a `Circle` object. The following statement is an example of client code constructing a `Circle` object with its center at (10,3.4) and radius 5 and assigning it to a variable named `circ`:

```
circ = Circle((10, 3.4), 5)
```

This statement implicitly invokes the `__init__` method of the `Circle` class behind the scenes to do the initialization work. The statement then assigns the variable `circ` to this newly created `Circle` object. The initializer first ensures that the `radius` parameter is nonnegative. An attempt to make a circle with a negative radius produces an exception. Next, the `__init__` method initializes the `center` and `radius` instance variables of the object it is in the process of creating. Within method definitions (like `__init__`), we prefix instance variable names with `self`. Python interprets any variables not prefixed with `self` as normal local or global variables. Consider the following two statements in the initializer definition:

```
self.center = center  
self.radius = radius
```

The expression `self.center` refers to the `center` instance variable of the object under construction, and `center` on the right side of the assignment operator refers to the formal parameter of the `__init__` method. Similarly, `self.radius` refers to the `radius` instance variable of the object, and `radius` on the right side is the parameter.

The methods `move`, `grow`, and `shrink` can modify the state of a circle object. Note that `grow` and `shrink` do not allow arbitrary changes; they allow clients to adjust a circle's radius only by one-unit increments.

The `area` and `circumference` methods use the value of the `radius` attribute to compute a `Circle` object's area and circumference, respectively.

Figure 13.3 A conceptual illustration of a circle object. Each instance of the `Circle` class has its own `center` and `radius` instance variables.

```
circ = Circle((2.5, 6), 5)
```

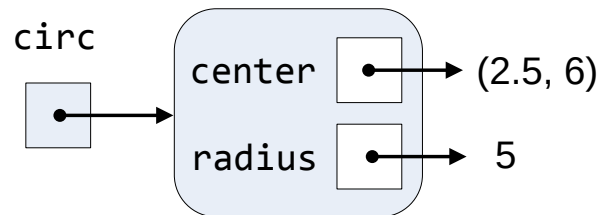


Figure 13.3 provides a conceptual view of a `Circle` object. Note that each `Circle` object needs only store a reference to its instance variables. All `Circle` instances share the code for their methods. In the following code:

```
c1 = Circle((2, 4), 5)
c2 = Circle((0, 0), 1)
```

The objects referred by `c1` and `c2` have their own `center` and `radius` instance variables. The call `print(c1)`

prints

```
Center:(2, 4) Radius:5
```

while

```
print(c2)
```

prints

```
Center:(0, 0) Radius:1
```

There is no need for each object to have its own copy of the method code.

Listing 13.6 (usecircle.py) is a simple program that makes use of our `Circle` class.

Listing 13.6: usecircle.py

```
from circle import Circle

circ = Circle((4.5, 0.2), 5)
print('Initially:', circ)
print(f'Area = {circ.area()}, Circumference = {circ.circumference()}')

circ.grow()
print('After grow:', circ)
print(f'Area = {circ.area()}, Circumference = {circ.circumference()}')

circ.move((-5, -10))
print('After move:', circ)
```

Listing 13.6 (usecircle.py) prints the following:

```
Initially: Center:(4.5, 0.2) Radius:5
Area = 78.53981633974483, Circumference = 31.41592653589793
After grow: Center:(4.5, 0.2) Radius:6
Area = 113.09733552923255, Circumference = 37.69911184307752
After move: Center:(-5, -10) Radius:6
```

The statement

```
circ = Circle((4.5, 0.2), 5)
```

calls the `Circle` constructor, which implicitly calls the `Circle` initializer method `__init__`. The `__init__` method establishes and initializes two instance variables, `center` and `radius`.

Notice that the call

```
circ.grow()
```

increases the `circ` circle object's radius from 5 to 6. It does not disturb `circ`'s center. The call

```
circ.move((-5, -10))
```

moves the `circ` circle object's center to $(-5, -10)$ but does not change the `circ`'s radius.

Listing 13.7 (twocircls.py) demonstrates that two different `Circle` objects indeed keep track of their own instance variables.

Listing 13.7: twocircls.py

```
from circle import Circle

c1 = Circle((2, 4), 5)
c2 = Circle((0, 0), 1)
print(f'c1 = {c1}, c2 = {c2}')

# Increase c1's radius by 5 units
for _ in range(5):
    c1.grow()
print(f'c1 = {c1}, c2 = {c2}')
```

Listing 13.7 (twocircls.py) prints the following:

```
c1 = Center:(2, 4) Radius:5, c2 = Center:(0, 0) Radius:1
c1 = Center:(2, 4) Radius:10, c2 = Center:(0, 0) Radius:1
```

In Listing 13.7 (twocircls.py), the circle objects `c1` and `c2` begin with different locations and sizes, and changing `c1` does not change `c2`.

As with regular functions, the methods within Python classes can accept default arguments, an arbitrary number of arguments (specified via `*`, as in Section 11.2), and keyword arguments (specified via `**`, as in Section 11.7). To illustrate default arguments, we can rewrite the `__init__` method as

```
class Circle:
    """ Represents a geometric circle object """
    def __init__(self, center, radius=1):
        """ Initialize the center's center and radius """
        # Disallow a negative radius
        if radius < 0:
            raise ValueError('Negative radius')
        self.center = center
        self.radius = radius

    # Other details of the Circle class omitted . . .
```

Under this new definition both of the following statements are valid:

```
c1 = Circle((3, 17), 4)    # Radius specified to be 4 units
c2 = Circle((22, 8))      # Radius defaults to 1 unit
```

Many of the classes in the `tkinter` package, for example, have methods that accept keyword arguments.

Listing 13.8 (circlemaker.py) exercises our new `Circle` class. It uses `Turtle` graphics (Section 9.5) to render `Circle` objects in a graphical window.

Listing 13.8: circlemaker.py

```
from turtle import Turtle, Screen, mainloop, delay
from circle import Circle

t = Turtle()                # Global turtle object
circ = Circle((0, 0), 50)   # Global circle object

def draw_circle(t: Turtle, circle: Circle) -> None:
    x, y = circle.center    # Unpack center's coordinates
    radius = circle.radius
    t.penup()               # Lift pen
    t.setposition(x, y)     # Move pen to (x,y)
    t.pendown()             # Place pen
    t.dot()                 # Draw a dot at the circle's center
    t.penup()               # Lift pen
    t.setposition(x, y - radius) # Position pen to draw rim of circle
    t.pendown()             # Place pen to draw
    t.circle(radius)        # Draw the circle
    t.penup()               # Lift pen
```

```

def do_click(x: float, y: float) -> None:
    circ.move((x, y))      # Move the circle to a new location
    redraw()

def do_up() -> None:
    circ.grow()             # Make the circle bigger
    redraw()

def do_down() -> None:
    circ.shrink()          # Make the circle smaller
    redraw()

def redraw() -> None:
    t.clear()              # Clear the drawing screen
    draw_circle(t, circ)   # Draw the circle object

def main() -> None:
    delay(0)               # Do not slowly trace drawing
    t.speed(0)             # Make turtle's actions as fast as possible
    t.hideturtle()         # Make the turtle invisible
    screen = Screen()      # Create Screen object to receive user input
    screen.listen()        # Set focus for keystrokes
    screen.onclick(do_click) # Set mouse press handler
    screen.onkey(do_up, 'Up') # Set "up" cursor key handler
    screen.onkey(do_down, 'Down') # Set "down" cursor key handler
    mainloop()

if __name__ == '__main__':
    main()

```

Listing 13.8 (circlemaker.py) is an interactive program that allows the user to place the image of a `Circle` object in the window via a mouse click. Subsequent mouse clicks move the circle to the mouse position. Users can use the up  and down  cursor keys to enlarge and reduce the size of the circle.

The `turtle` module provides the `Screen` class. In Listing 13.8 (circlemaker.py), the `screen` variable references a `Screen` object. `Screen` objects accept user input from the pointing device and keyboard. The statement

```
screen = Screen()
```

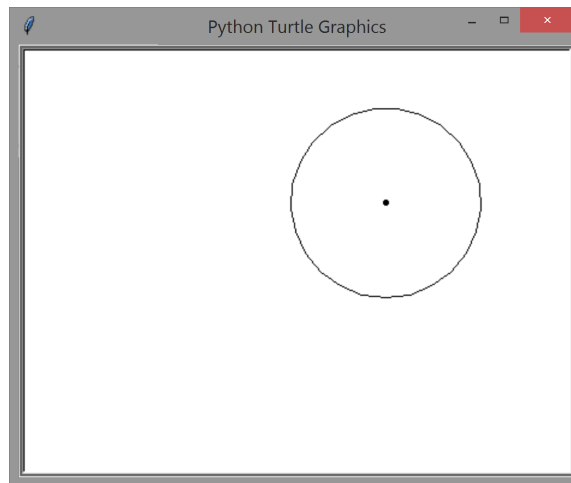
creates the `Screen` object and assigns it to the variable `screen`. The statement

```
screen.listen()
```

enables the graphical window to receive keystrokes from the user. The statement

```
screen.onclick(do_click)
```

calls the `onclick` method of the `Screen` class. Like functions (see Section 8.6), methods can accept functions and other methods as parameters. The `onclick` method accepts a function as a parameter. The

Figure 13.4 Using the custom `Circle` class in a graphical application

`turtle` module provides the `Screen.onclick` method, and we must provide the function we send to it. The only restriction on the function we send to `onclick` is that the function must accept two integer parameters. In this case we pass it our `do_click` function. The purpose of the `onclick` function is to register a *callback function* with the Turtle graphics framework. The framework will “call back” the function when a certain event occurs. If the user clicks the mouse when the pointer is over the graphics window, the framework will call the function registered with `onclick` (in this case `do_click`) and pass to it the x and y coordinates of the mouse pointer at the time of the click. That is why this callback function requires a function that expects the two integer parameters. Note that our `do_click` function packs up into a tuple the x and y coordinates it receives and passes this tuple on to the global `Circle` object `circ`’s `move` method. We must use a global variable in the `do_click` function because we cannot pass in the `circ` object—remember, *we* do not call `do_click`, the *framework* does, and the framework only passes in the mouse pointer coordinates.

Similarly, the statements

```
screen.onkey(do_up, 'Up')
screen.onkey(do_down, 'Down')
```

assign callback functions for the framework to invoke when the user presses the up  and down  cursor keys. The callback functions accepted by the `Screen.onkey` method take no parameters. Consequently, neither `do_up` nor `do_down` accept parameters. The second parameter to the `onkey` method specifies which key event to associate with the given callback function. The Turtle graphics framework defines the strings `'Up'` and `'Down'`, as well as many others, for use with the `Screen` object’s `onkey` method.

Note that the `do_click`, `do_up`, and `do_down` functions modify the global `Circle` object `circ`’s state, but the `draw_circle` method simply reads the `Circle` object’s `center` and `radius` attributes to be able to draw the circle using the Turtle object’s `circle` method. The `Turtle.circle` method draws a circle starting from the turtle’s current position—three o’clock on the circle—and it draws a counter-clockwise curve around the circle.

Figure 13.4 shows a sample run of Listing 13.8 (`circlemaker.py`).

Listing 13.8 (circlemaker.py) uses global variables to give multiple functions access to the same `Turtle` and `Circle` object. Section 8.1 exposed some of the disadvantages of using global variables. Later we will see how to use object-oriented techniques to “hide” global variables inside of classes to avoid many of the disadvantages of true globals.

13.3 Timing Objects

In 6.5 we saw how to use the `perf_counter_ns` function to measure elapsed time during a program’s execution. The following skeleton code fragment

```
start = perf_counter_ns()    # Record starting time
#
# Do something here that you wish to time
#
stop = perf_counter_ns()    # Record ending time
print((stop - start)/1_000_000_000, "seconds")
```

can be adapted to any program, but we can make it more convenient if we wrap the functionality into an object. We can wrap all the messy details of the timing code into a convenient package. Consider the following client code that uses an object to keep track of the time:

```
timer = Stopwatch()    # Create a timer object

timer.start()          # Start timing

#
# Do something here that you wish to time
#

timer.stop()           # Stop the clock
print(timer.elapsed(), " seconds")
```

This code using a `Stopwatch` object is simpler. A programmer writes code using a `Stopwatch` in a similar way to using an actual timer: push a button to start the clock (call the `start` method), push a button to stop the clock (call the `stop` method), and then read the elapsed time (use the result of the `elapsed` method). Programmers using a `Stopwatch` object in their code are much less likely to make a mistake because the details that make it work are hidden from the programmer.

Now that we are familiar with the basics of custom types in Python, we can design such a `Stopwatch` class. We will use the `time.perf_counter_ns` function as the basis for our `Stopwatch` class. Listing 13.9 (stopwatch.py) defines the structure and capabilities of our `Stopwatch` objects.

Listing 13.9: stopwatch.py

```

from time import perf_counter_ns

class Stopwatch:
    """ Initializes stopwatch objects that that programmers
        can use to time the execution time of portions of
        a program. """
    def __init__(self) -> None:
        """ Makes a new stopwatch ready for timing. """
        self.reset()

    def __repr__(self) -> str:
        """ Expresses the time in HH:MM:SS.dddd format """
        seconds = round(self._elapsed / 1_000_000_000)
        fract = self._elapsed - seconds
        hours = seconds // 3600
        seconds %= 3600
        minutes = seconds // 60
        seconds %= 60
        return f'{hours:02}:{minutes:02}:{seconds:02}.{fract}'

    def start(self) -> None:
        """ Starts the stopwatch, unless it is already running.
            This method does not affect any time that may have
            already accumulated on the timer. """
        if not self._running:
            self._start_time = perf_counter_ns() - self._elapsed
            self._running = True # Clock now running

    def stop(self) -> None:
        """ Stops the stopwatch, unless it is not running.
            Updates the accumulated elapsed time. """
        if self._running:
            self._elapsed = perf_counter_ns() - self._start_time
            self._running = False # Clock stopped

    def reset(self) -> None:
        """ Resets stopwatch to zero. """
        self._start_time = self._elapsed = 0
        self._running = False

    def elapsed(self) -> float:
        """ Reveals the stopwatch running time in seconds since
            it was last reset. """
        if not self._running:
            return self._elapsed/1_000_000_000
        else:
            return (perf_counter_ns() - self._start_time)/1_000_000_000

```

The main methods available to clients are: `start`, `stop`, `reset`, and `elapsed`. A client does not have to worry about the “messy” detail of the arithmetic to compute the elapsed time.

Note that our design allows clients to read the elapsed time of a `Stopwatch` object via the `elapsed` method without needing to call the `stop` method first. Also, this implementation allows a user to stop the

timer and resume the timing later without resetting the time in between.

Listing 13.10 (newtimeprimes.py) is a rewrite of Listing 10.25 (timeprimes.py) that uses a Stopwatch object to compare the execution speed of two different prime number algorithms.

Listing 13.10: newtimeprimes.py

```

""" Count the number of prime numbers less than
    two million and time how long it takes
    Compares the performance of two different
    algorithms. """

from stopwatch import Stopwatch
from math import sqrt

def count_primes(n: int) -> None:
    """
    Generates all the prime numbers from 2 to n - 1.
    n - 1 is the largest potential prime considered.
    """
    timer = Stopwatch()
    timer.start()           # Record start time

    count = 0
    for val in range(2, n):
        root = round(sqrt(val)) + 1
        # Try all potential factors from 2 to the square root of n
        for trial_factor in range(2, root):
            if val % trial_factor == 0: # Is it a factor?
                break                 # Found a factor
        else:
            count += 1                # No factors found

    timer.stop()             # Stop the clock
    print(f'Count = {count}, Elapsed time: {timer.elapsed()} seconds')

def sieve(n: int) -> None:
    """
    Generates all the prime numbers from 2 to n - 1.
    n - 1 is the largest potential prime considered.
    Algorithm originally developed by Eratosthenes.
    """

    timer = Stopwatch()
    timer.start()           # Record start time

    # Each position in the Boolean list indicates
    # if the number of that position is not prime:
    # false means "prime," and true means "composite."
    # Initially all numbers are prime until proven otherwise
    nonprimes = n * [False] # Global list initialized to all False

    count = 0

```

```

# First prime number is 2; 0 and 1 are not prime
nonprimes[0] = nonprimes[1] = True

# Start at the first prime number, 2.
for i in range(2, n):
    # See if i is prime
    if not nonprimes[i]:
        count += 1
        # It is prime, so eliminate all of its
        # multiples that cannot be prime
        for j in range(2*i, n, i):
            nonprimes[j] = True
# Print the elapsed time
timer.stop()
print(f'Count = {count}, Elapsed time: {timer.elapsed()} seconds')

# Time the algorithms
count_primes(2000000)
sieve(2000000)

```

The code managing the execution timing in this new, object-oriented version is simpler and more readable than our earlier version.

13.4 Traffic Light Objects

Section 9.6 introduced the Tk graphical user interface toolkit. Listing 9.14 (`tkinterlight.py`) provides a program that draws three circular lamps of various colors and a rectangular frame. It supports user interaction and simulates a standard traffic light. What if we wish to build a more sophisticated application that graphically models an intersection containing four traffic lights? We could follow the same approach, writing code to draw four rectangular frames and 12 circular lamps. Making sure all the rectangular frames and circular lamps are in the correct locations would be tedious work. Next consider an expansion of the application that is to model the scores of intersections and hundreds of traffic lights within a community. Coding all the parts of the lights in their correct positions within the graphics window would be beyond tedious.

We really are not interested in drawing rectangles and circles in our more sophisticated application; we actually need to be able to draw *traffic lights*. Further, such traffic lights ought to know their current color and automatically be able to become a different color when told to do so. In fact, a traffic light's normal operating procedure should be to change from red to green, green to yellow, and yellow to red. Other transitions, such as green to red or red to yellow should not be supported. A single program must be able to use multiple traffic lights with each light maintaining its own independent operation.

By now you probably have thought of the solution. We need a `TrafficLight` class that enables us to create `TrafficLight` objects. Each `TrafficLight` object should contain instance variables that represents the light's position and current color. The class constructor could set the light's position and initial color. A `change` method could transition the light's current color to its new color, ensuring no out-of-sequence color changes.

Recall the Tk program in Listing 9.14 (`tkinterlight.py`) that allows the user to cycle through the colors of a graphical traffic light by pressing a button. We will use that code as a starting point in designing our traffic light class. We want clients to be able to specify the position and size of traffic light objects within the graphics window. Clients also should be able to specify the light's initial color. To simplify things for this example, we will not provide to clients the ability to reposition or resize a light after its creation. After

creating a traffic light object, a clients can cycle through its color states via a change method.

Listing 13.11 (trafficlight.py) provides the class definition for traffic light objects.

Listing 13.11: trafficlight.py

```
from tkinter import Canvas

class TrafficLight:
    """ Models a traffic light. """

    def __init__(self, x: int, y: int, width: int,
                  canvas: Canvas, initial_color='red') -> None:
        """ Makes a new traffic light object. The light's
            left-top corner is anchored at the point (x, y).
            The is width pixels wide.
            The light will render on the canvas drawing surface.
            The light's initial color is initial_color. """

        # Check to see if the supplied color is valid
        if initial_color not in ('red', 'yellow', 'green'):
            raise ValueError(initial_color + ' is not a valid color')

        # Initialize instance variables
        self.canvas = canvas
        self.color = initial_color

        # Set the individual lamps initial fill colors
        red_fill = 'red' if initial_color == 'red' else 'black'
        yellow_fill = 'yellow' if initial_color == 'yellow' else 'black'
        green_fill = 'green' if initial_color == 'green' else 'black'

        # 1/5 of the width will the be the basic unit for computing the
        # height of the traffic light and positioning the individual lamps.
        unit = width//5    # Ensure integer division

        # Traffic light frame
        canvas.create_rectangle(x, y, x + width, y + 13*unit, fill='gray')

        x = x + unit
        y = y + unit
        diameter = 3*unit    # Diameter (bounding box width) of each lamp

        # Red lamp, currently on
        self.red_lamp = canvas.create_oval(x, y, x + diameter, y + diameter,
                                           fill=red_fill)

        y += 4*unit
        # Yellow lamp, currently off
        self.yellow_lamp = canvas.create_oval(x, y, x + diameter, y + diameter,
                                              fill=yellow_fill)

        y += 4*unit
        # Green lamp, currently off
        self.green_lamp = canvas.create_oval(x, y, x + diameter, y + diameter,
                                             fill=green_fill)

    def change(self) -> None:
```

```

""" Changes the traffic light's color to the next color in
    the sequence. """
if self.color == 'red':
    self.color = 'green'
    self.canvas.itemconfigure(self.red_lamp, fill='black')
    self.canvas.itemconfigure(self.green_lamp, fill='green')
elif self.color == 'green':
    self.color = 'yellow'
    self.canvas.itemconfigure(self.green_lamp, fill='black')
    self.canvas.itemconfigure(self.yellow_lamp, fill='yellow')
elif self.color == 'yellow':
    self.color = 'red'
    self.canvas.itemconfigure(self.yellow_lamp, fill='black')
    self.canvas.itemconfigure(self.red_lamp, fill='red')

```

The following are some important points from Listing 13.11 (trafficlight.py):

- Listing 13.11 (trafficlight.py) is not an executable Python program; it is module meant for other modules to import.
- The `TrafficLight` initializer accepts an (x,y) position and width for the graphical traffic light. The initializer calculates the light's height and the size and placement of its three lamps from these three parameters. The initializer also accepts a Tk canvas reference from the client. This means the client is responsible for setting up the Tk graphics environment before creating a `TrafficLight` object. Finally, the initializer can accept an initial color from the client. The color is one of the following strings: `'red'`, `'yellow'`, or `'green'`. The initializer throws a `ValueError` exception if the client attempts to pass an object as the last argument that is not one of these strings. If the client omits this parameter, the color defaults to `"red"`.

The initializer establishes five instance variables: `canvas`, `color`, `red_lamp`, `yellow_lamp`, and `green_lamp`. The following indicates their purpose:

- `canvas`: After object construction, methods must have access to the canvas object in order to recolor the lamps as necessary. This means each `TrafficLight` object must store the caller's canvas object. The `canvas` attribute of all traffic light objects will alias the the client's canvas object.
- `color`: This instance variable keeps track of the light's current color.
- `red_lamp`, `yellow_lamp`, and `green_lamp`: These instance variables allow methods to access the circle shapes the initializer created via calls to `create_oval`. The `change` method needs access to these shapes to be able change the lamps' colors.

The mathematics within the initializer ensures that all the shapes that make up a visual traffic light will appear in their correct positions regardless of a light's location and size.

- The `change` method in the `TrafficLight` class updates the `color` instance variable to the next color in the light's sequence of signals and modifies the fill colors of the two lamps affected by the change.

Given the module specified by Listing 13.11 (trafficlight.py), Listing 13.12 (trafficlightobject.py) behaves exactly like Listing 9.14 (tkinterlight.py), but the code is completely reorganized into classes.

Listing 13.12: trafficlightobject.py

```

from tkinter import Tk, Canvas
from tkinter.ttk import Button, Frame

from trafficlight import TrafficLight

class TrafficLightWindow:
    """ Graphical window that displays a traffic light and change button. """

    def __init__(self) -> None:
        root = Tk()          # Create the main window
        root.title("Traffic Light") # Set title bar text
        # Create widget container
        frame = Frame(root)
        frame.pack()
        # Create a graphical button and ensure it calls the do_button_press
        # method when the user presses it
        button = Button(frame, text='Change', command=self.do_button_press)
        # Create and add a drawing surface within the window
        canvas = Canvas(frame, width=200, height=300)
        # Make a traffic light object instance variable
        self.light = TrafficLight(50, 20, 100, canvas)
        # Position button and canvas objects
        button.grid(row=0, column=0)
        canvas.grid(row=0, column=1)
        # Start the GUI event loop
        root.mainloop()

    def do_button_press(self) -> None:
        """ The window manager calls this function when the user
            presses the graphical button. """
        self.light.change()

# Create and execute a traffic light window
TrafficLightWindow()

```

The following are some important points from Listing 13.12 (trafficlightobject.py):

- Listing 13.12 (trafficlightobject.py) imports the Listing 13.11 (trafficlight.py) module. This gives it access to our `TrafficLight` class.
- The `TrafficLightWindow` class is responsible for setting up the Tk graphical infrastructure. Its initializer creates a `Canvas` object and uses that to create a `TrafficLight` object. The initializer also creates a `tkinter.ttk Button` object, and wires it into the `do_button_press` method.
- The `do_button_press` method simply delegates its work to the `change` method of its `TrafficLight` instance variable.
- The event handling code in Listing 13.12 (trafficlightobject.py) does not access any global variables. In fact, global variables are absent from both the Listing 13.11 (trafficlight.py) and Listing 13.12 (trafficlightobject.py) modules. This is an important distinction from the code in Listing 9.14 (tkinterlight.py)

which required global variables to achieve the necessary communication with the function that changed the light's color. The instance variables within `TrafficLight` objects assume the roles formerly played by global variables in Listing 9.14 (`tkinterlight.py`).

Instance variables are like global variables confined to an particular object; any method within that class can access that object's instance variables via its `self` parameter.

Listing 13.13 (`multisizelightwindow.py`) demonstrates the flexibility of our `TrafficLight` class by creating traffic light objects of various sizes.

Listing 13.13: `multisizelightwindow.py`

```
from tkinter import Tk, Canvas, GROOVE
from tkinter.ttk import Button, Frame
from random import choice, randrange

from functools import partial

from trafficlight import TrafficLight

class MultisizeLightWindow:
    """ Graphical window that displays multiple traffic lights
        of different sizes. """

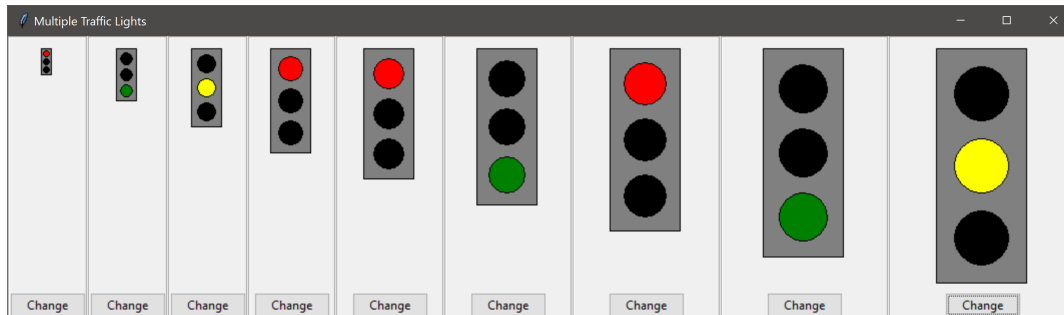
    def do_button_press(self, idx: int) -> None:
        """ The window manager calls this function when the user
            presses the graphical button. """
        self.lights[idx].change()

    def __init__(self) -> None:
        root = Tk()          # Create the main window
        root.title('Multiple Traffic Lights') # Set title bar text
        self.lights = []
        # The outer frame holds the nine inner frames that each contain
        # a canvas and a button.
        outer_frame = Frame(root)
        outer_frame.pack()
        # Create nine drawing surfaces within the window
        for i in range(1, 10):
            f = Frame(outer_frame, borderwidth=2, relief=GROOVE)
            f.grid(row=0, column=i)
            c = Canvas(f, width=20*i, height=250)
            c.grid(row=0, column=0)
            self.lights.append(TrafficLight(5*i, 10, 10*i, c))
            b = Button(f, text="Change",
                      command=partial(self.do_button_press, i - 1))
            b.grid(row=1, column=0)
        # Start the GUI event loop
        root.mainloop()

# Create and execute a traffic light window
MultisizeLightWindow()
```

Figure 13.5 shows a sample run of Listing 13.13 (`multisizelightwindow.py`). The following are some impor-

Figure 13.5 The initial display of Listing 13.13 (`multisizelightwindow.py`). The user can press buttons to change the color of their associated traffic light images.



tant points from Listing 13.13 (`multisizelightwindow.py`):

- The initializer builds a list of traffic light objects. Note that the loop variable determines the exact location and size of each light it creates.
- The initializer uses `Tk ttk.Frame` widgets. Their purpose is to organize the place of other widgets; in this case each `Frame` widget contains exactly one traffic light object and one button object.
- The widget `grid` method positions a widget within a grid of rows and columns within its parent. The `root` widget is the parent of all nine frame widgets. The `grid` method places every frame widget in row 0 (the first row), but its column depends on `i`, the loop variable. Within each frame, the `grid` method places each canvas widget at row 0, column 0 and each button widget at row 1, column 0.
- You can verify that each traffic light maintains its own color by pressing a button below a light. Each button press affects only the light appearing above itself.
- Note the application of the `functools.partial` function (see Section 8.12). The `command` keyword argument in the `tkinter.ttk.Button` class constructor must be paired with a function or method that expects no parameters. The `do_button_press` method expects two parameters—`self` and `idx` (for index). How do we make these two parameters “go away?” In the expression `partial(self.do_button_press, i - 1)` the `self` prefix for `do_button_press` takes care of the `self` parameter. That leaves the `idx` parameter. The expression `partial(self.do_button_press, i - 1)` makes a new function with the `idx` parameter hardwired in to be `i - 1`. The first time through the loop registers a callback to `self.do_button_press` with the argument 0, the second time through the loop the argument is 1, and so forth. Each call to `partial` creates a new, independent function, and each function created during a loop iterator becomes the callback of the button object created during that iteration. The net effect is our `do_button_press` method can flexibly call the `change` method of the proper traffic light object.

An alternative to the statement

```
b = Button(f, text="Change", command=partial(self.do_button_press, i - 1))
```

is

```
b = Button(f, text="Change", command=lambda x=i: lights[x - 1].change())
```

Here we use a `lambda` function (see Section 8.8) with a default argument of the current loop variable's value. As in the partial function application, this approach creates nine separate independent

functions, each associated with a particular button. This approach makes the `do_button_press` method superfluous; if we use the `lambda` functions, we can remove the `do_button_press` method from the class.

Listing 13.14 (`randomlights.py`) further demonstrates the flexibility of our `TrafficLight` class by creating 50 traffic light objects at random positions with random sizes. The program even selects the initial color of each light at random.

Listing 13.14: `randomlights.py`

```
from tkinter import Tk, Canvas
from tkinter.ttk import Button, Frame
from random import choice, randrange

from trafficlight import TrafficLight

class RandomLightsWindow:
    """ Graphical window that displays multiple traffic lights
        of different sizes. """

    def __init__(self) -> None:
        root = Tk()           # Create the main window
        root.title('Multiple Traffic Lights') # Set title bar text
        f = Frame(root)
        f.pack()
        # Create a drawing surface within the window
        canvas = Canvas(f, width=800, height=600)
        # Make a list of random traffic light objects
        color_list = ['red', 'yellow', 'green']
        self.light_list = []
        for i in range(50):
            self.light_list.append(TrafficLight(randrange(2, 700),
                                                randrange(2, 400),
                                                randrange(5, 120),
                                                canvas,
                                                choice(color_list)))

        # Create a graphical button and ensure it calls the do_button_press
        # function when the user presses it
        button = Button(f, text='Change', command=self.do_button_press)
        # Position button and canvas objects
        button.grid(row=0, column=0)
        canvas.grid(row=0, column=1)
        # Start the GUI event loop
        root.mainloop()

    def do_button_press(self) -> None:
        """ The window manager calls this function when the user
            presses the graphical button. """
        for light in self.light_list:
            light.change()

# Create and execute a traffic light window
RandomLightsWindow()
```

Figure 13.6 A sample execution of Listing 13.14 (`randomlights.py`) that creates 50 traffic light objects at random positions, with random sizes, and with random initial colors.

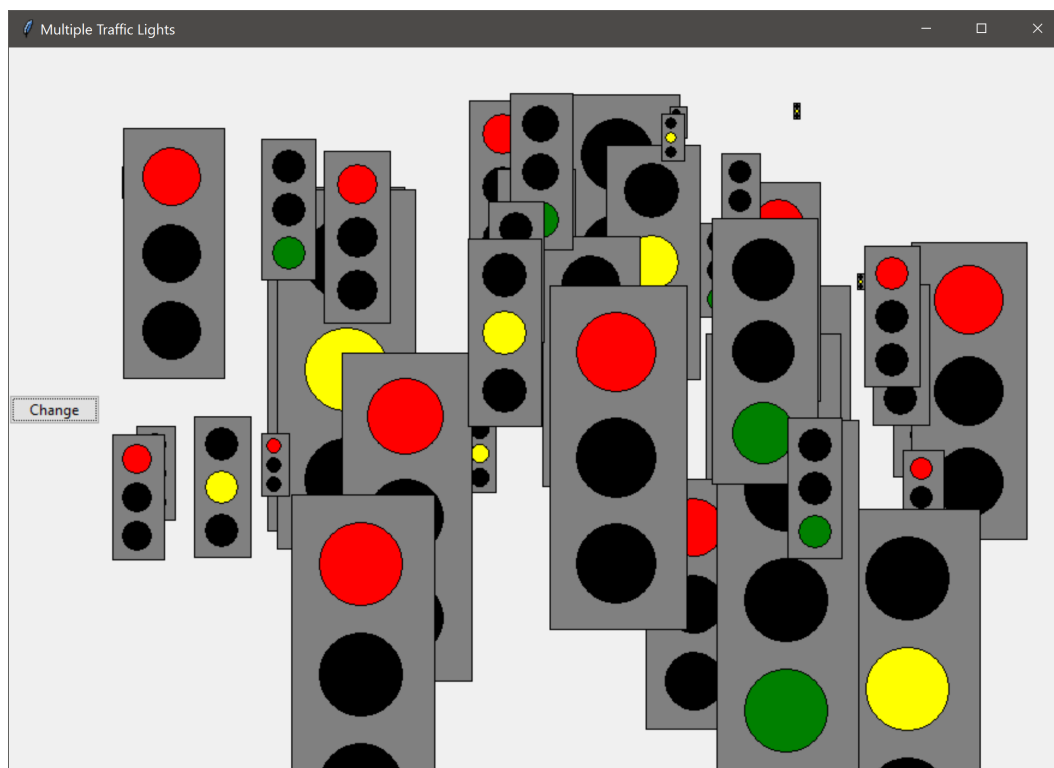


Figure 13.6 shows a sample run of Listing 13.14 (`randomlights.py`). When the user presses the *Change* button, all the lights change to their next valid state.

13.5 Automated Testing

We know that just because a program runs to completion without a run-time error it does not imply that the program works correctly. We can detect logic errors in our code as we interact with the executing program. The process of exercising code to reveal errors or demonstrate the lack thereof is called *testing*. The informal testing that we have done up to this point has been adequate, but serious software development demands a more formal approach. Good testing requires the same skills and creativity as programming itself.

Until relatively recently in the software development world, testing was often an afterthought. Testing was not perceived to be as glamorous as designing and coding. Poor testing led to buggy programs that frustrated users. Also, tests were written largely after the program's design and coding were complete. The problem with this approach is major design flaws may not be revealed until late in the development cycle. Changes late in the development process are invariably more expensive and difficult to deal with than changes earlier in the process.

Weaknesses in the standard approach to testing led to a new strategy: *test-driven development*. In test-driven development the testing is automated, and the design and implementation of good tests is just as

important as the design and development of the actual program. In pure test-driven development, test engineers develop tests based on the problem's requirements *before* developers write any application code. Testers then can subject all new application code to the pre-existing tests as soon as the code becomes available.

Listing 13.15 (functiontester.py) defines the structure of a rudimentary object that we can use to test functions.

Listing 13.15: functiontester.py

```
from typing import Any, Callable

class FunctionTester:
    """ Provides support for testing simple Python functions. """

    def __init__(self) -> None:
        """ Creates a FunctionTester object. Resets the counts to zero. """
        self.__error_count = self.__total_count = 0
        print('+-----')
        print('| Testing')
        print('+-----')

    def check(self, msg: str, expected, func, *args) -> bool:
        """ Clients pass a human-readable message (msg) to uniquely
            identify the test case, the expected value that a
            correct test would produce (expected), the function to
            test (func), and any arguments that the function under
            test requires (*args).
            Provides immediate printed feedback about the test's
            success or failure and adjusts error count accordingly.
            Returns True if the function passed the test;
            otherwise, returns False. """
        result = True # Test passed unless we determine otherwise
        print(f'    {msg} ', end=' ')
        self.__total_count += 1 # Count this test
        try:
            actual = func(*args) # Apply function to arguments
            if expected == actual:
                print('OK')
            else:
                self.__error_count += 1 # Count this failed test
                print(f'*** Failed! Expected: {expected} actual: {actual}')
                result = False # Test failed
        except Exception as e:
            self.__error_count += 1 # Count this failed test
            print(f'***** Exception {e}')
            result = False # Test failed
        return result # Notify client of test result

    def report_results(self) -> None:
        """ Prints the testing statistics. """
        print('+-----')
        print(f'| {self.__total_count} tests run')
        print(f'| {self.__total_count - self.__error_count} passed')
```

```
print(f'| {self.__error_count} failed')
print('+-----')
```

Listing 13.15 (`functiontester.py`) provides the class of simple test objects that enables a client to exercise one or more functions to see if they produce expected results. A test object keeps track of the number of tests performed and the number of failures.

The `FunctionTester.__init__` method establishes the `__total_count` and `__error_count` instance variables to handle the accounting. Clients use the `FunctionTester.check` method to test a function; callers pass up to four arguments:

- a string that uniquely identifies the test (so the tester can determine which test failed),
- the predicted return value of a correctly implemented function,
- the function to test, and
- the parameters to pass to the function, if any.

During its execution, the `FunctionTester.check` method calls the function it receives with the specified parameters and compares the actual result to the expected result. The method will indicate the test's success or failure and track the number of failed tests. Clients call the `report_results` method after performing all the tests. The `report_results` method provides the error statistics for all the tests performed.

Observe that type hints are absent from the `expected`, `func`, and `args` parameters of the `check` method. This is because we cannot predict what type the client's function to test will return, nor can we know ahead of time what types of parameters the function to test will require. In other words, we do not know the type of the function a client will pass to the `check` method.

Listing 13.16 (`testliststuff.py`) uses our `FunctionTester` class to test a few simple functions.

Listing 13.16: `testliststuff.py`

```
from functiontester import FunctionTester

def max_of_three_bad(x: int, y: int, z: int) -> int:
    """ Attempts to determine and return the maximum of three
        numeric values. """
    result = x
    if y > result:
        result = y
    elif z > result:
        result = z
    return result

def max_of_three_good(x: int, y: int, z: int) -> int:
    """ Computes and returns the maximum of three numeric values. """
    result = x
    if y > result:
        result = y
    if z > result:
        result = z
    return result
```

```

def sum(lst: list[int]) -> int:
    """ Attempts to compute and return the sum of all the elements in
        a list of integers. """
    total = 0
    for i in range(1, len(lst)):
        total += lst[i]
    return total

# maximum has a bug (it has yet to be written!)
def maximum(lst: list[int]) -> int:
    """ Computes the maximum element in a list of integers. """
    return 0 # maximum not yet implemented

def main() -> None:
    t = FunctionTester() # Make a test object

    # Some test cases to test max_of_three_bad
    t.check('max_of_three_bad test #1', 4, max_of_three_bad, 2, 3, 4)
    t.check('max_of_three_bad test #2', 4, max_of_three_bad, 4, 3, 2)
    t.check('max_of_three_bad test #3', 4, max_of_three_bad, 3, 2, 4)

    # Some test cases to test max_of_three_good
    t.check('max_of_three_good test #1', 4, max_of_three_good, 2, 3, 4)
    t.check('max_of_three_good test #2', 4, max_of_three_good, 4, 3, 2)
    t.check('max_of_three_good test #3', 4, max_of_three_good, 3, 2, 4)

    # Some test cases to test maximum
    t.check('maximum test #1', 4, maximum, [2, 3, 4, 1])
    t.check('maximum test #2', 4, maximum, [4, 3, 2, 1])
    t.check('maximum test #3', 0, maximum, [-2, -3, 0, -21])

    # Some test cases to test sum
    t.check('sum test #1', 7, sum, [0, 3, 4])
    t.check('sum test #2', 2, sum, [-3, 0, 5])

    t.report_results()

main()

```

The program's output is

```

+-----+
|  Testing
+-----+
max_of_three_bad test #1  *** Failed!  Expected: 4 actual: 3
max_of_three_bad test #2  OK
max_of_three_bad test #3  OK
max_of_three_good test #1  OK
max_of_three_good test #2  OK
max_of_three_good test #3  OK
maximum test #1  *** Failed!  Expected: 4 actual: 0
maximum test #2  *** Failed!  Expected: 4 actual: 0
maximum test #3  OK
sum test #1  OK
sum test #2  *** Failed!  Expected: 2 actual: 5
+-----+
| 11 tests run
| 7 passed
| 4 failed
+-----+

```

In the `max_of_three_bad` and `max_of_three_good` functions, the test program clearly demonstrates the difference between sequential `if` statements and multi-way `if/elif/else` statements.

In the `sum` function, the programmer was careless and used 1 as the beginning index for the list. Notice that the first test does not catch the error, since the element in the zeroth position (zero) does not affect the outcome. A human tester must be creative and even devious to try and force the code under test to demonstrate its errors.

Notice that even though we have yet to implement the `maximum` function, we can test it anyway. This is true test-driven development—design the tests first, then write the code. The `maximum` function here is an example of a *stub*. A stub is a function or method that does not provide the expected functionality but may be executed without causing a run-time error. A stub ordinarily ignores all parameters passed to it and, if necessary, returns a default value of the type expected by its caller. The yet-to-be implemented `maximum` function simply returns zero—notice that it accidentally passes one of the tests. We can define a square root function stub as

```

def sqrt(x: float) -> float:
    """ Computes the square root of x. """
    return 0.0 # TODO implement the square root function

```

Stubs are invaluable during the development process because they serve as placeholders for incomplete functionality or missing features in a fully executable and testable program. Stubs allow programmers to develop and test the overall logic and structure of a program while many of the details remain to be determined or implemented.

A `FunctionTester` object robustly handles any exceptions a function under test may raise. It records an exception as a failed test, and continues on with any remaining tests.

The `FunctionTester` class from Listing 13.15 (`functiontester.py`) has some limitations; for example, testers cannot use it to check the correctness of a function that sorts a list in place. The `FunctionTester.check` method determines only if a function returns the expected result. It also cannot test that a function does not modify a mutable parameter; for example, in the course of their execution the `sum` and `maximum` functions in Listing 13.16 (`testliststuff.py`) should not modify the list passed by a caller. We could enhance the `FunctionTester` class to check that functions handle mutable parameters properly.

Our `FunctionTester` class is limited in its testing abilities. In Section 14.5 we will examine a better way to test our code using the `unittest` module in the Python standard library.

13.6 Plotting Data

Listing 8.21 (`plotter.py`) uses Turtle graphics to plot mathematical functions. We can use a class to wrap the details of Turtle graphics and design a plotter object that is more convenient for programmers. At the same time we can add some enhancements that enable plotting of data obtained experimentally.

Listing 13.17 (`plotobj.py`) defines a class named `Plotter`. The class contains the following methods:

- `__init__`: Initializes the Turtle graphics window. It accepts parameters that define the window's physical size and ranges of x and y axes.
- `__del__`: Deletes the object from the executing program. The interpreter calls this method, also known as destructor, when the object is no longer bound to a reference with in the executing program. Among other times, this happens when the program's execution terminates.
- `draw_axes`: Draws the x and y axes on the plot. An optional Boolean parameter controls the presence of accessory reference lines that divide the plot into 20 equal sections.
- `draw_grid`: Draws horizontal and vertical accessory reference coordinate lines on the plot. Parameters control the frequency of the reference lines.
- `draw_arrow`: Draws a line segment with an attached arrow head. Expects four numeric parameters representing the (x_1, y_1) tail end point and (x_2, y_2) head end point of the arrow.
- `plot_function`: Plots the curve of a given function with a specified color. An optional Boolean parameter determines if the function renders immediately or requires the client to call the `update` method after a series of plots (defaults to `True`).
- `plot_data`: Plots the curve formed by the (x, y) points contained in a list of data. A parameter specifies the curve's color. An optional Boolean parameter determines if the function renders immediately or requires the client to call the `update` method after a series of plots (defaults to `True`).
- `setcolor`: Sets the current drawing color.
- `onclick`: Assigns a callback function that the frame should call when the user clicks the mouse button when the pointer is over the plotter window.
- `interact`: Sets the plotter to interactive mode, enabling the user to provide mouse and keyboard input.

Listing 13.17: `plotobj.py`

```
""" Provides the Plotter class that clients can use to
    draw graphs of a mathematical functions on a Cartesian
    coordinate plane. """

from turtle import Pen, Screen
from math import atan2, pi
from typing import Callable
```



```

class Plotter:
    """ A plotter object provides a graphical window for
        plotting data and mathematical functions. """

    def __init__(self, width: int, height: int,
                  min_x: float, max_x: float,
                  min_y: float, max_y: float) -> None:
        """ Initializes the plotter with a window that is
            width wide and height high. Its x-axis ranges from
            min_x to max_x, and its y-axis ranges from
            min_y to max_y. Establishes the global beginning and ending
            x values for the plot and the x_increment value.
            Draws the x- and y-axes. """

        self.pen = Pen() # The plotter object's pen
        self.screen = Screen() # The plotter object's screen

        self.pen.speed(0) # Speed up rendering
        self.screen.tracer(0, 0) # Do not draw pen while drawing
        # Establish global x and y ranges
        self.min_x, self.max_x = min_x, max_x
        self.min_y, self.max_y = min_y, max_y
        # Set up window size, in pixels
        self.screen.setup(width=width, height=height)
        # Set up screen size, in pixels
        self.screen.screensize(width, height)
        self.screen.setworldcoordinates(min_x, min_y, max_x, max_y)

        # x-axis distance that corresponds to one pixel in window distance
        self.x_increment = (max_x - min_x)/width
        self.draw_grid(20)
        self.draw_axes()
        self.screen.title('Plot')
        self.screen.update()

    def __del__(self) -> None:
        """ Called when the client no longer uses the plotter
            object. """
        print('Done plotting')

    def draw_axes(self, grid: bool=False) -> None:
        """ Draws the x and y axes within the plotting window.
            The grid parameter controls the drawing of accessory
            horizontal and vertical lines. """
        if grid:
            self.draw_grid(20)
            self.pen.hideturtle() # Make pen invisible
            prev_width = self.pen.width()
            self.pen.width(2)
            # Draw x axis
            self.pen.color('black')
            self.draw_arrow(self.min_x, 0, self.max_x, 0)
            # Draw y axis
            self.draw_arrow(0, self.min_y, 0, self.max_y)

```

```

        self.pen.width(prev_width)

def draw_grid(self, n: int) -> None:
    """ Draws horizontal and vertical accessory reference
        coordinate lines on the plot. Parameter n controls
        the frequency of the reference lines. """
    self.pen.up()
    #self.pen.setposition(self.min_x, self.min_y)
    inc = (self.max_x - self.min_x)/n
    self.pen.color('lightblue')
    x = self.min_x
    while x <= self.max_x:
        self.pen.setposition(x, self.min_y)
        self.pen.down()
        self.pen.setposition(x, self.max_y)
        self.pen.up()
        x += inc # Next x
    inc = (self.max_y - self.min_y)/n
    y = self.min_y
    while y <= self.max_y:
        self.pen.setposition(self.min_x, y)
        self.pen.down()
        self.pen.setposition(self.max_x, y)
        self.pen.up()
        y += inc # Next y

def draw_arrow(self, x1: float, y1: float, x2: float, y2: float) -> None:
    """ Draws an arrow starting at (x1, y1) and ending at
        (x2, y2). """
    # Draw arrow shaft
    self.pen.up()
    self.pen.setposition(x1, y1) # Move the pen starting point
    self.pen.down() # Draw line bottom to top
    self.pen.setposition(x2, y2) # Move the pen starting point
    # Draw arrow head
    dy = y2 - y1
    dx = x2 - x1
    angle = atan2(dy, dx) * 180/pi
    self.pen.setheading(angle)
    self.pen.stamp()

def plot_function(self, f: Callable[[float], float],
                  color: str, update: bool=True) -> None:
    """ Plots function f on the Cartesian coordinate plane
        established by initialize_plotter. Plots (x, f(x)),
        for all x in the range min_x <= x < max_x.
        The color parameter dictates the curve's color. """

    # Move pen to starting position
    self.pen.up()
    self.pen.setposition(self.min_x, f(self.min_x))
    self.pen.color(color)
    self.pen.down()
    # Iterate over the range of x values for min_x <= x < max_x
    x = self.min_x

```

```

while x < self.max_x:
    self.pen.setposition(x, f(x))
    x += self.x_increment    # Next x

if update:
    self.screen.update()

def plot_data(self, data: list[tuple[float, float]],
              color: str, update: bool=True) -> None:
    """ Plots the (x, y) pairs of values in the data list.
        The curve's color is specified by the color parameter. """
    # Move pen to starting position
    self.pen.up()
    self.pen.setposition(data[0][0], data[0][1])
    self.pen.color(color)
    self.pen.down()
    # Plot the points in the data set
    for x, y in data:
        self.pen.setposition(x, y)
    if update:
        self.screen.update()

def update(self) -> None:
    """ Draws any pending plotting actions to the screen. """
    self.screen.update()

def setcolor(self, color: str) -> None:
    """ Sets the current drawing color. """
    self.pen.color(color)

def onclick(self, fun: Callable[[float, float], None]) -> None:
    """ This method assigns a function to call when the user
        clicks the mouse over the plotting window. The
        function must accept two integer parameters that
        represent the (x, y) location of the mouse when the
        click occurred. """
    self.screen.onclick(fun)

def interact(self) -> None:
    """ Places the plotting object in interactive mode so the
        user can provide input via the mouse or keyboard. """
    self.screen.mainloop()

```

Listing 13.18 (testplotter.py) exercises the `plotobj` module, demonstrating its use within a program.

Listing 13.18: testplotter.py

```

from plotobj import Plotter
from math import sin

def quad(x: float) -> float:
    """ Quadratic function (parabola) """
    return 1/2 * x**2 + 3

def arrow_wheel(plotter, x: float, y: float, len: float,
                angle: int, color: str) -> None:
    """ Draws a collection of arrows extending radially from
        an (x, y) center point. The arrows all are len long, the
        angle parameter specifies the angle between each adjacent
        arrow, and color specifies their color. """
    from math import cos, sin, radians
    COS_theta = cos(radians(angle))
    SIN_theta = sin(radians(angle))
    plotter.setcolor(color)
    xe, ye = len, 0.0
    for i in range(360//angle):
        xe, ye = xe*COS_theta - ye*SIN_theta, xe*SIN_theta + ye*COS_theta
        plotter.draw_arrow(x, y, xe + x, ye + y)

def main() -> None:
    """ Provides a simple test of the plotting object. """
    from math import sin

    def run_test(x, y):
        """ Generate an arrow wheel centered at (x, y) with
            random size and color. """
        #test_arrow(plt)
        from random import randrange
        colors = ['red', 'green', 'blue', 'black']
        arrow_wheel(plt, x, y, randrange(10) + 1, 10, colors[randrange(4)])

    # Create a plotter object
    plt = Plotter(600, 600, -10, 10, -10, 10)
    # Plot f(x) = 1/2*x + 3, for -10 <= x < 100
    plt.plot_function(quad, 'red')
    # Plot f(x) = x, for -10 <= x < 100
    plt.plot_function(lambda x: x, 'blue')
    # Plot f(x) = 3 sin x, for -10 <= x < 100
    plt.plot_function(lambda x: 3*sin(x), 'green')

    # Execute the run_test function when the user clicks the mouse
    plt.onclick(run_test)
    #test_arrow(plt)

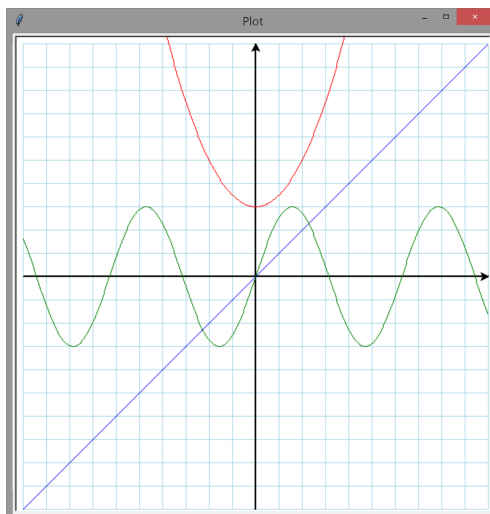
    plt.interact()

if __name__ == '__main__':
    main()

```

Listing 13.18 (testplotter.py) first draws some mathematical curves and then allows the user to place cir-

Figure 13.7 The initial plot of Listing 13.18 (`testplotter.py`) before the user clicks the mouse over the window.



cles of radiating arrows of random colors and sizes. Figure 13.7 shows the initial plot of Listing 13.18 (`testplotter.py`) before the user clicks the mouse over the window. Figure 13.8 shows a subsequent display of Listing 13.18 (`testplotter.py`) after the user creates several arrow wheels. The arrow circles give the vague appearance of exploding fireworks.

13.7 Dynamic Content

Section 13.1 showed how we can set up the instance variables of an object in the `__init__` method of its class. This technique ensures every object of that class will contain those instance variables. Python allows programmers to add instance variables dynamically to individual objects. Consider the following very simple class:

```
class Widget:
    pass
```

The following code creates a `Widget` object and adds some instance variables:

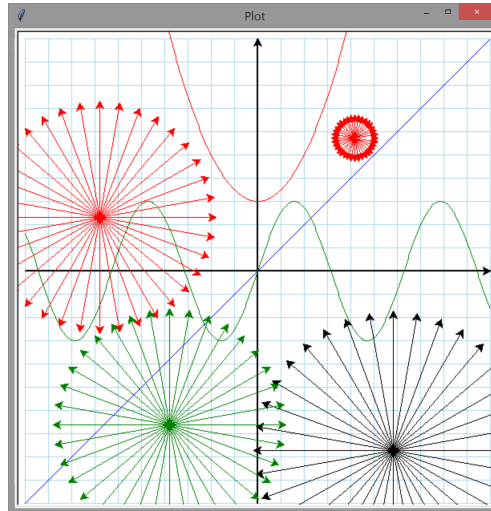
```
w = Widget()
w.a = 16
w.t = 100
print(w.a, w.t)
```

The code prints

```
16 100
```

This ability to dynamically add instance variables to objects sets Python apart from other mainstream object-oriented programming languages like C++, Java, and C#. A C++ class, for example, establishes the structure

Figure 13.8 A subsequent display of Listing 13.18 (`testplotter.py`) after the user creates several arrow wheels by clicking the mouse over various parts of the window.



(instance variables) of an object, and that structure is the same for all objects of that type. Furthermore, programmers cannot add instance variables to an object without editing the source code for its class.

Python's instance variables are flexible because each object stores its instance variables in a dictionary (see Section 11.3 for a review of dictionaries). The dictionary is located in an instance variable named `__dict__` that is common to all objects. The following experiment with the interactive interpreter demonstrates the `__dict__` instance variable in action:

```
>>> class Widget:
...     def __init__(self) -> None:
...         self.a = 10
...
>>> w = Widget()
>>> w.__dict__
{'a': 10}
>>> class Gadget:
...     pass
...
>>> g = Gadget()
>>> g.__dict__
{}
>>> g.x = 20
>>> g.y = 30
>>> g.__dict__
{'x': 20, 'y': 30}
```

Notice that whether the initializer assigns the instance variable or a statement outside the class assigns the instance variable, the variable resides in the object's dictionary.

Programmers ordinarily do not need to dynamically add instance variables to an object. The ability to add instance variables on the fly is convenient for some algorithms; for example, when traversing a list of

objects, a program could add an instance variable named `visited` to mark the object as being processed.

The `del` statement removes an instance variable:

```
del g.x    # Removes the x instance variable from object g
```

We can express the statement

```
g.x = 5
```

as

```
setattr(g, 'x', 5)
```

Similarly, we can rewrite

```
print(g.x)
```

as

```
print(getattr(g, 'x'))
```

The `setattr` function stands for *set attribute*, and `getattr` stands for *get attribute*. (Recall that the term *attribute* is another name for *instance variable*). Note that in the calls to the functions `setattr` and `getattr` the instance variable name `x` is expressed as the string `'x'`. If the name `x` did not appear in quotes, it would refer to a variable named `x`; such a variable may or may not exist, but, more importantly, it would not refer to the instance variable named `x` in the `g` object.

The ability to specify an instance variable name via a string enables users to create instance variable names at run time. Listing 13.19 (`dynamicfield.py`) demonstrates such code. `id` to an object.

Listing 13.19: `dynamicfield.py`

```
class Widget:
    def __init__(self) -> None:
        self.a = 5    # Provide a preexisting attribute

w = Widget()
print(w.__dict__)
print(f'w.a = {getattr(w, "a")}')
setattr(w, 'a', 10)
print(f'w.a = {getattr(w, "a")}')
field_name = input('Enter instance variable name: ')
setattr(w, field_name, 15)
print(getattr(w, field_name))
setattr(w, field_name, 20)
print(getattr(w, field_name))
print(w.__dict__)
```

In the following sample run of Listing 13.19 (`dynamicfield.py`), the user adds the instance variable named `my_field`:

```
{'a': 5}
w.a = 5
w.a = 10
Enter instance variable name: my_field
15
20
{'a': 10, 'my_field': 20}
```

The `setattr` and `getattr` functions are *almost* perfect alternatives to the dot (`.`) notation. The following session in the interactive interpreter reveals an interesting difference:

```
>>> class Widget:
...     pass
...
>>> w = Widget()
>>> w.__dict__
{}
>>> setattr(w, 'a', 1)
>>> w.__dict__
{'a': 1}
>>> getattr(w, 'a')
1
>>> w.a
1
>>> setattr(w, '#@&', 2)
>>> w.__dict__
{'#@&': 2, 'a': 1}
>>> getattr(w, '#@&')
2
>>> w.#@&
File "<stdin>", line 1
    w.#@&
      ^
SyntaxError: invalid syntax
>>> setattr(w, '', 3)
>>> w.__dict__
{'': 3, '#@&': 2, 'a': 1}
>>> getattr(w, '')
3
```

We may use the dot notation only for instance variable names that obey the usual identifier rules (see Section 2.3 for identifier naming rules). The instance variable names established by `setattr` have no such restrictions. Even the empty string (`''`) is a valid instance variable name for `setattr` and `getattr`!

13.8 Class Variables

In Python, a class is not simply a pattern for a type of object; a class is an object itself. Consider the following

interactive session:

```
>>> class X:
...     pass # An empty class
...
>>> x_obj = X() # Make an instance
>>> type(x_obj)
<class '__main__.X'>
>>> type(X)
<class 'type'>
```

We see the type of a class is `type`. Every object has a `__class__` field that reveals the same information as the `type` function:

```
>>> x_obj.__class__
<class '__main__.X'>
>>> X.__class__
<class 'type'>
```

We can obtain the name of any class via its `__name__` field:

```
>>> X.__name__
'X'
>>> x_obj.__class__.__name__
'X'
>>> X.__class__.__name__
'type'
```

Each individual instance of a class maintains its own instance variables. Since a class is an instance of `type`, a class can have its own variables. We reserve the term *instance variable* for variables within instances of classes; we denote variables stored in a class object as *class variables*.

A variable defined within a class but outside of all methods of that class is a class variable. Listing 13.20 (`widgetfactory.py`) uses a class variable to manage serial numbers of widget objects it manufactures. It is important that manufactured products have unique serial numbers, and the `Widget` initializer ensures the serial number of each widget object is different from any other widget object.

Listing 13.20: `widgetfactory.py`

```
class Widget:
    """ Models a manufactured item. """
    serial_number_source = 0 # Class variable

    def __init__(self) -> None:
        """ Make a widget with a unique serial number. """
        self.serial_number = Widget.serial_number_source
        Widget.serial_number_source += 1

    def get_serial_number(self) -> int:
        """ Return the widget's serial number. """
        return self.serial_number

if __name__ == '__main__':
    widget_list = []
```

```

for i in range(10):
    widget_list.append(Widget())
for w in widget_list:
    print(w.serial_number, end=' ')
print()
for w in widget_list:
    print(w.serial_number_source, end=' ')
print()

```

Listing 13.20 (widgetfactory.py) prints the following:

```

0 1 2 3 4 5 6 7 8 9
10 10 10 10 10 10 10 10 10 10

```

In Listing 13.20 (widgetfactory.py) the statement

```
serial_number_source = 0      # Class variable
```

appears in class body of `Widget` but outside the body of any method in the class. This makes `serial_number_source` a class variable. Observe that every use of the class variable within a `Widget` method definition prefixes the name with the `Widget` class qualifier. Outside of the `Widget` class, we can prefix a class variable with either the name of the class, as in `Widget.serial_number_source`, or the name of a `Widget` instance, as in `w.serial_number_source`. This means all instances share access to the same class variable. The last line of the output produced by Listing 13.20 (widgetfactory.py) shows that all the `Widget` objects print the same value. Each `Widget` object maintains its own `serial_number` attribute in its object dictionary, but no `Widget` object stores the `serial_number_source` class variable. The class variable resides in the `Widget` class dictionary.

13.9 Restricting Access to Members

The developers of the `Circle` class intend that clients should not directly manipulate a `Circle` object's `center` and `radius` instance variables. The `__init__`, `move`, `grow`, and `shrink` methods are the only methods that should be able to influence the state of a `Circle` object; however, nothing prevents client code from accessing the instance variables directly via the dot (`.`) operator. The following interactive sequence illustrates:

```

>>> from circle import Circle
>>> c = Circle((0, 0), 4)
>>> c.radius
4
>>> c.radius = 10
>>> c.radius
10

```

As we can see, clients can alter the radius of a circle outside the facilities provided by the `grow` and `shrink` methods.

The convention in Python is to use a leading underscore to name instance variables not meant for general access by clients:

```

class MyClass:
    def __init__(self, num):

```

```

self.public_var = num
self._private_var = num

```

Here, clients are encouraged to access the `public_var` instance variable of `MyClass` objects directly:

```

mc = MyClass(10)
mc.public_var = 12    # Acceptable access

```

but code like the following is discouraged:

```

mc = MyClass(10)
mc._private_var = 12  # Private variable not meant for public access

```

This statement that modifies the intended private instance variable is legal Python code. It is up to the programmer to comply with the convention of not directly manipulating or even reading an object's instance variables that begin with an underscore. It is understood that a programmer doing so assumes all the risks of any problems that may arise from the action. What could go wrong? Such access could place the object in an ill-defined state so that it does not behave as the class designer intended. The client may have the best intentions but be unaware of the unintended consequences of the unapproved action. The underscore indicates the attribute is for internal use only. This means it could disappear in a future version of the class. Working client code that depends on an internal instance variable will break if the class maintainers remove the attribute in the future.

Python does support a naming scheme that makes it more difficult to access an instance variable, in effect rendering it private to code inside the class. Python does not permit normal access with the dot (`.`) to attributes of an object with names that begin with two underscores. (The Python community often tersely refers to the *double underscore* prefix as *dunder*.) The following interactive sequence illustrates:

```

>>> class MyType:
...     def __init__(self):
...         self.__id = 2
...
>>> m = MyType()
>>> m.__id
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
AttributeError: 'MyType' object has no attribute '__id'
>>> m._MyType__id
2

```

As you can see, internally Python changes the name of an attribute that begins with double underscores by prefixing it with a single underscore followed by the class name. This process is called *name mangling*. The mangled name of `__id` is `_MyType__id`. Python thus does not provide a way to truly protect object members from the outside world. This sets Python apart from most other mainstream object-oriented languages like C++, Java, and C# which provide language features that can protect the internal details of an object. (Although to be fair, programmers have ways to circumvent the protections those languages offer, too.)

The official way to better protect data within a class is to use *properties*. Consider Listing 13.21 (`countprops.py`), a class used to make counter objects:

Listing 13.21: countprops.py

```

class Counter:
    def __init__(self, limit: int) -> None:
        if limit < 1:
            raise ValueError()
        self.__count = 0
        self.__limit = limit

    def __repr__(self) -> str:
        return f'{self.__count} of {self.__limit}'

    def increment(self) -> None:
        if self.__count < self.__limit:
            self.__count += 1

    @property
    def count(self) -> int:
        return self.__count

    @count.setter
    def count(self, value: int) -> None:
        if value <= self.__limit:
            self.__count = value

```

The dunder in `__count` and `__limit` make these attributes harder for clients to access. The first count method

```

@property
def count(self) -> int:
    return self.__count

```

has an annotation that declares it to be a property *getter*. It gives clients read access to the `__count` attribute through the name `count`. The second count method

```

@count.setter
def count(self, value: int) -> None:
    if value <= self.__limit:
        self.__count = value

```

has an annotation that declares it to be a property *setter*. It allows clients to modify the `__count` attribute if it does not violate the limit established during the object's construction. The following code fragment shows how it all works:

```

from countprops import Counter

c = Counter(10)  # Count up to 10

x = c.count      # Indirectly calls the count getter method
c.count = 5      # Okay; new value is less than established limit
c.count = 20     # Illegal; proposed value is above the limit

```

Note the client's code uses the name `count` as if it were an attribute, not a method—no parentheses are present. Executing the last statement generates a `ValueError` exception since it attempts to move the count past the predetermined limit.

Listing 13.22 (testcount.py) exercises the Counter class.

Listing 13.22: testcount.py

```
from countprops import Counter

c = Counter(10)

for _ in range(15):
    print(f'c = {c}    count = {c.count}')
    c.increment()
```

Listing 13.22 (testcount.py) prints the following:

```
c = 0 of 10    count = 0
c = 1 of 10    count = 1
c = 2 of 10    count = 2
c = 3 of 10    count = 3
c = 4 of 10    count = 4
c = 5 of 10    count = 5
c = 6 of 10    count = 6
c = 7 of 10    count = 7
c = 8 of 10    count = 8
c = 9 of 10    count = 9
c = 10 of 10   count = 10
c = 10 of 10   count = 10
c = 10 of 10   count = 10
c = 10 of 10   count = 10
c = 10 of 10   count = 10
```

Python properties make it easy for clients to access protected attributes in a natural but safe way. Clever clients still can go out of their way to modify the protected attribute in our Counter class:

```
c._Counter__count = 20
```

While Python does not provide a way to completely hide object members that are meant to be inaccessible to clients, we will follow the convention of using dundered attributes and properties that discourage clients from accessing an object's members meant to be private. We will use this technique in Section 13.10 to make a robust rational number class.

13.10 Rational Numbers

The concept of mathematical rational numbers is accessible even to elementary school students, so it presents an ideal platform for further exploration into custom class design in Python. We will define our own simple rational number class. Note that if you are developing an application that requires the use of rational numbers, you ought to use Python's standard `Fraction` class that we saw in Section 9.4. We build our own class here because our goal in this section is simply to gain more experience developing custom classes in Python.

A **rational** number is the **ratio** of two integers. There is a restriction, however—the number on the bottom of a fraction cannot be zero. The number on the top of the fraction is called the *numerator*, and the bottom number is known as the *denominator*. Consider the following beginnings of a simple `Rational` class:

```
class Rational:
    def __init__(self, num: int, den: int) -> None:
        self.numerator, self.denominator = num, den
```

There is nothing in this class definition that prevents a client from making a rational number like the following:

```
fract = Rational(1, 0)
```

In this case the variable `fract` represents an undefined integer. We can help matters with a different initializer:

```
class Rational:
    def __init__(self, num: int, den: int) -> None:
        self.numerator = num
        if den != 0:
            self.denominator = den
        else:
            raise ValueError("Attempt to make an illegal rational number")
```

While this new initializer will prevent illegal initialization by raising a run-time exception, clients still can subvert our `Rational` objects:

```
fract = RationalNum(1, 2)    # This is OK
fract.denominator = 0       # This is bad!
```

At best, the programmer made an honest mistake introducing an error into the program. Perhaps it was a careless “copy and paste” error. On the other hand, a clever programmer may be fully aware of how the program works in the larger context and intentionally write such bad code to exploit a weakness in the system that compromises its functionality or security.

We can use the Python’s dunder notation to better protect our `denominator` instance variable, as shown in Listing 13.23 (`rational.py`). Note that this form of protection defends against a conscientious but careless programmer but not a malicious one.

Listing 13.23: `rational.py`

```
class Rational:
    """
    Represents a rational number (fraction)
    """
    def __init__(self, num: int=0, den:int=1) -> None:
        """ Initializes a fraction. Defaults to 0/1. """
        self.numerator = num # Any integer numerator is okay
        if den != 0:
            self.__denominator = den
        else:
            raise ValueError("Attempt to make an illegal rational number")

    def __repr__(self) -> str:
        """
        Make a string representation of a Rational object
        """
        return f'{self.numerator}/{self.__denominator}'
```

```

@property
def denominator(self) -> int:
    return self.__denominator

@denominator.setter
def denominator(self, den: int) -> None:
    """
    Sets the denominator of the fraction to d,
    unless d is zero. If d is zero, the method
    terminates the program with an error message.
    """
    if den != 0:
        self.__denominator = den
    else:
        raise ValueError("Error: zero denominator!")

def __mul__(self, other: 'Rational') -> 'Rational':
    """
    Returns the product of this rational number object with
    the other rational object.
    """
    return Rational(self.numerator * other.numerator,
                    self.__denominator * other.__denominator)

def __add__(self, other: 'Rational') -> 'Rational':
    """
    Returns the sum of this rational number object with
    the other rational object.
    """
    return Rational() # Not yet implemented

# Client code that uses Rational objects
def main() -> None:
    fract1 = Rational(1, 2)
    fract2 = Rational(2, 3)
    print("fract1 =", fract1)
    print("fract2 =", fract2)
    fract1.numerator = 3
    fract1.denominator = 4
    fract2.numerator = 1
    fract2.denominator = 10
    print("fract1 =", fract1)
    print("fract2 =", fract2)
    fract3 = Rational(1, 2)
    fract4 = Rational(3, 5)
    print(f'{fract3} * {fract4} = {fract3 * fract4}')
    #fract5 = Rational(4, 0)
    #fract3.denominator = 0

if __name__ == '__main__':
    main()

    print('-----')
    f = Rational(1, 2)
    f.__denominator = 0 # Legal, but what does it do?

```

```
print(f.denominator)    # Prints 2, not 0
print(f.__denominator)  # Prints 0, not 2
```

Listing 13.23 (`rational.py`) does not need to protect against abuse of the `numerator` attribute since it may assume any integer value, even zero.

When run, Listing 13.23 (`rational.py`) prints

```
fract1 = 1/2
fract2 = 2/3
fract1 = 3/4
fract2 = 1/10
1/2 * 3/5 = 3/10
-----
2
0
```

If we uncomment the next to last statement in the `main` function, the program crashes at the end of its run with the following report:

```
Traceback (most recent call last):
  File "C:\Code\python\rational.py", line 77, in <module>
    main()
  File "C:\Code\python\rational.py", line 73, in main
    frac5 = Rational(4, 0)
            ^^^^^^^^^^^^^^^^^
  File "C:\Code\python\rational.py", line 11, in __init__
    raise ValueError("Attempt to make an illegal rational number")
ValueError: Attempt to make an illegal rational number
```

The anticipated exception from the next-to-last statement in the `main` function prevents the function from completing successfully.

Notice in Listing 13.23 (`rational.py`) in the `Rational` class that the `denominator` attribute's name begins with `__`. This means that client code like

```
fract = Rational(1, 2)
print(fract.__denominator)    # Error, not possible
```

will not work. Clients no longer have relatively easy direct access to the `__numerator` and `__denominator` instance variables of `Rational` objects.

Clients may appear to change a protected field as

```
fract = Rational(1, 2)
fract.__denominator = 0          # Legal, but what does it do?
print(fract.get_denominator())   # Prints 2, not 0
print(fract.__denominator)       # Prints 0, not 2
```

Surprisingly, the second statement (assignment of `fract.__denominator`) does not affect the `__denominator` attribute used by the methods in the `Rational` class; it instead adds a new, unprotected field named `__denominator`. This is because the attribute's mangled name is `_Rational__denominator`, not simply `__denominator`. The client cannot get to the protected field by merely using the dot (`.`) operator. To avoid such confusion, a client should not attempt to add instance variables to an object with names that begin with two underscores.

It is a good idea to define a `__repr__` method for all custom classes. It is one of the special methods like `__add__`, `__sub__`, `__eq__`, etc. introduced in Section 9.4 that provide syntactic sugar to the language. The interpreter calls an object's `__repr__` method when executing code that requires a string representation of an object; for example, the `print` function converts an object into a string so it can display textual output. The `str` string constructor function also uses the `__str__` method of an object behind the scenes. The following simple program demonstrates how this works:

```
# Define class X
class X:
    def __str__(self):
        return "abcdef"

# Make some X objects
a = X()
b = X()
c = X()
# Print them
print(a, b, c)
print("-----")
message = str(b)
print(message)
```

This code prints

```
abcdef abcdef abcdef
-----
abcdef
```

Our `Rational.__mul__` method allows us to multiply two `Rational` objects using the convenient `*` syntax:

```
fract3 = Rational(1, 2)
fract4 = Rational(3, 5)
print(fract3, "*", fract4, "=", fract3*fract4)
```

The expression `fract3 * fract4` is syntactic sugar for `fract3.__mul__(fract4)`. We leave the implementation of the `Rational.__add__` method to the reader; it is a bit more complicated, as rational number addition involves finding a common denominator and making adjustments to the numerators to make them compatible for adding together.

As a review of how objects and methods work in Python, consider the `main` function of Listing 13.23 (`rational.py`) which contains code that uses `Rational` objects. The statement

```
fract1.set_numerator(2)
```

calls the `set_numerator` method of the `Rational` class on behalf of the object `fract1`. During the call, from the perspective of the code in the method `self` is assigned `fract1`, and `n` is assigned 2. Recall that this statement is syntactic sugar for the more verbose

```
Rational.set_numerator(fract1, 2)
```

This means the code within `set_numerator` sees `self` as `fract1` and `n` as the integer object 2. During this particular execution of the `set_numerator` method the expression `self.__numerator`

within the method's definition refers to the `fract1` object's `__numerator` field. The method, therefore, reassigns the `__numerator` member of `fract1`.

In comparison, consider the call

```
fract2.set_numerator(1)
```

This statement calls the `set_numerator` method of the `Rational` class on behalf of the object `fract2`. Within the method's execution in this case, the expression `self.__numerator` refers to `fract2`'s numerator. The method invocation in this case will assign 1 to the `__numerator` field of the `fract2` object.

In OO-speak, we say the statement

```
fract1.set_numerator(2)
```

represents the client sending a `set_numerator` *message* to object `fract1`. In this message, it provides the value 2. In this case `fract1` is the message *receiver*. In the statement

```
fract2.set_numerator(1)
```

object `fract2` receives the `set_numerator` message with the value 1.

In the `Rational` class in Listing 13.23 (`rational.py`) clients can see the values of the instance variables via methods `get_numerator` and `get_denominator`. These methods do not change the state of a `Rational` object. Recalling the terminology from Section 13.2, these methods are often called *getter* methods because they can *get* the values of instance variables but cannot change the instance variables. In contrast, `set_numerator` and `set_denominator` are *mutator* methods because they can change the values of instance variables.

When the values of one or more instance variables in an object change, we say the object changes its state; for example, if we use an object to model the behavior of a traffic light, the object likely will contain an instance variable that represents its current color: red, yellow, or green. When that field changes, the traffic light's color changes. In the green to yellow transition, we can say the light goes from the state of being green to the state of being yellow.

13.11 Bank Account Objects

Consider the task of writing a program that manages accounts for a bank. Real bank accounts must store a large amount of information: the account owner's name, address, social security number, account number, etc. To streamline our example, we will model bank accounts that maintain just three attributes:

- Every account has a unique identifier, the account number.
- Each account's owner has a name.
- Each account has a current balance.

We can define a class of bank account objects. Each account object would have an account number instance variable (string of digits) a name instance variable (string), and a balance instance variable (integer number of cents). We use a string for the account number so it is easy to maintain a fixed number of digits, and the program will not need to perform any mathematical calculations with an account number. It is tempting to use a floating-point number for the account balance because we normally express currency as

dollars and cents with a decimal point. This approach is risky because calculations can be off by a penny due to the imprecision of floating-point numbers and rounding. Our bank account management application will store the accounts in a list and move the contents of the list to a file for persistent storage.

Listing 13.24 (bankaccounts.py) uses a BankAccount class to implement a simple database of customer bank accounts.

Listing 13.24: bankaccounts.py

```
class BankAccount:
    """ Models a bank account """
    def __init__(self, number: str, name: str, balance: int) -> None:
        """ Initialize the instance variables of a bank account object.
            Disallows a negative initial balance. """
        if balance < 0:
            raise ValueError('Negative initial balance')
        self.__number = number # Account number
        self.__name = name # Customer name
        self.__balance = balance # Funds available in the account

    def __repr__(self) -> str:
        """ Returns the string representation for this object """
        return f'[{self.__number:>5} {self.__name:10} {self.__balance:>7}]'

    def id(self) -> str:
        """ Returns the account number of this bank account object. """
        return self.__number

    def deposit(self, amount: int) -> None:
        """ Add funds to the account. There is no limit to the size
            of the deposit. """
        self.__balance += amount

    def withdraw(self, amount: int) -> bool:
        """ Remove funds from the account, if possible. Only completes
            the withdrawal successfully if there are enough funds in
            the account to fulfill the withdrawal.
            Return true if successful, false otherwise """
        result = False # Unsuccessful by default
        if self.__balance - amount >= 0: # Ensure against an overdraft
            self.__balance -= amount
            result = True # Success
        return result

# -----
# Global functions that use BankAccount objects

def open_database(filename: str, db: list[BankAccount]) -> None:
    """ Read account information from a given file and store it
        in the given list. """
    with open(filename) as lines: # Open file to read
        for line in lines:
            line = line.strip()
            line = line.replace(' ', '')
            number, name, balance = line.split(',')

```

```

        db.append(BankAccount(number, name, int(balance)))

def print_database(db: list[BankAccount]) -> None:
    """ Display the contents of the database """
    for acct in db:
        print(acct) # Calls the __repr__ method of acct

def get_account(db: list[BankAccount],
                account_number: str) -> BankAccount | None:
    """ Retrieve an account object with a given account number.
        Return None if the account does not exist. """
    for acct in db:
        if acct.id() == account_number:
            return acct
    return None

def main() -> None:
    # Simple bank account "database"
    database = []

    try:
        # Open the database of accounts
        open_database('accountdata.text', database)
        print_database(database)
        print('-----')
        customer = get_account(database, 129)
        if customer:
            print('Account 129 before deposit of $100: ', end='')
            print(customer)
            customer.deposit(100)
            print('Account 129 after deposit of $100: ', end='')
            print(customer)
            print('-----')
            print('Account 129 before withdraw of $500: ', end='')
            print(customer)
            customer.withdraw(500)
            print('Account 129 after withdraw of $500: ', end='')
            print(customer)
            print('-----')
            print('Account 129 before withdraw of $80000: ', end='')
            print(customer)
            customer.withdraw(80000)
            print('Account 129 after withdraw of $80000: ', end='')
            print(customer)

        except Exception:
            print('Error in account database')
            raise

main()

```

Given a text file named `accountdata.text` containing the data

```
324, Fred, 34523
371, Ella, 1263210
129, Zoe, 78934
120, Owen, 247702
412, Lily, 12000
420, Bert, 10354
1038, George, 6733498
966, Jan, 9923912
1210, Judy, 83497
1300, Sam, 50315
```

the program Listing 13.24 (`bankaccounts.py`) would produce the output

```
[ 324 Fred      34523]
[ 371 Ella     1263210]
[ 129 Zoe       78934]
[ 120 Owen     247702]
[ 412 Lily      12000]
[ 420 Bert      10354]
[ 1038 George   6733498]
[ 966 Jan       9923912]
[ 1210 Judy     83497]
[ 1300 Sam      50315]
-----
Account 129 before deposit of $100: [ 129 Zoe      78934]
Account 129 after  deposit of $100: [ 129 Zoe      79034]
-----
Account 129 before withdraw of $500: [ 129 Zoe      79034]
Account 129 after  withdraw of $500: [ 129 Zoe      78534]
-----
Account 129 before withdraw of $80000: [ 129 Zoe      78534]
Account 129 after  withdraw of $80000: [ 129 Zoe      78534]
```

Clients interact with these bank account objects via the methods. The intention is that clients should not access a bank account object's instance variables directly to alter the object's state.

In the `BankAccount` methods

- The initializer (`__init__`) establishes the instance variables of a `BankAccount` object, but it disallows the creation of an object with a negative account balance.
- Clients may add funds via the `deposit` method. The `deposit` method does not limit the amount of a deposit.
- The `withdraw` method prevents a client from withdrawing more money than is in the account. An overdraft attempt will not change the account's balance.

The following code will not work:

```
acct = BankAccount(31243, 'Joe', 1000)
acct.deposit(100)
acct.__balance -= 100    # Illegal
```

Clients instead should use the `withdraw` method. The `withdraw` method prevents actions such as

```
# New bank account object with $1,000.00 balance (100,000 cents)
acct = BankAccount(31243, 'Joe', 100000)
acct.withdraw(200000); # Method should disallow this operation
```

Notice that the operations of depositing and withdrawing funds are the responsibility of the object itself, not the client code. The attempt to withdraw the \$2,000 dollars will not alter the state of the object.

Consider a non-programming example. If I deposit \$1,000.00 dollars into a bank, the bank then has custody of my money. It is still my money, so I theoretically can reclaim it at any time. The bank stores money in its safe, and my money is in the safe as well. Suppose I wish to withdraw \$100 dollars from my account. Since I have \$1,000 total in my account, the transaction should be no problem. What is wrong with the following scenario:

1. Enter the bank.
2. Walk past the teller into a back room that provides access to the safe.
3. The door to the safe is open, so enter the safe and remove \$100 from a stack of \$20 bills.
4. Exit the safe and inform a teller that you got \$100 out of your account.
5. Leave the bank.

This is not the process a normal bank uses to handle withdrawals. In a perfect world where everyone is honest and makes no mistakes, all is well. In reality, many customers might be dishonest and intentionally take more money than they report. Even though I faithfully counted out my funds, perhaps some of the bills were stuck to each other and I made an honest mistake by picking up six \$20 bills instead of five. If I place the bills in my wallet with other money that is there already, I may never detect the error. Clearly a bank needs a more controlled procedure for customer withdrawals.

When working with programming objects, in many situations it is advantageous to discourage client access to the internals of an object. Client code should not be able to easily modify directly bank account instance variables for various reasons, including:

- A withdrawal should not exceed the account balance.
- Federal laws dictate that deposits above a certain amount should be reported to the Internal Revenue Service, so a bank would not want customers to be able to add funds to an account in a way to circumvent this process. We could add this functionality to our `deposit` method.
- An account number should never change for a given account for the life of that account.

We know that naming the instance variables with a dunder prefix provides only mild security (Section 13.9). One check we can perform is to search client code for dunders. Behaving client code should be dunder-free with regard to bank account instance variables.

13.12 Exercises

1. Given the definition of the geometric `Point` class, complete the function named `distance`:

```
def distance(r1, r2):
    # Details go here
```

that returns the distance between the two points passed as parameters.

- Given the definition of the `Rational` number class, complete the following function named `reduce`:

```
def reduce(r):
    # Details go here
```

that returns the rational number that represents the parameter reduced to lowest terms; for example, the fraction 10/20 would be reduced to 1/2.

- What is the purpose of the `__init__` method in a class?
- What is the parameter named `self` that appears as the first parameter of a method?
- Given the definition of the `Rational` number class, complete the following method named `reduce`:

```
class Rational:
    # Other details omitted here ...

    # Returns an object of the same value reduced
    # to lowest terms
    def reduce(self):
        # Details go here
```

that returns the rational number that represents the object reduced to lowest terms; for example, the fraction 10/20 would be reduced to 1/2.

- Given the definition of the `Rational` number class, complete the following method named `reduce`:

```
class Rational:
    # Other details omitted here ...

    # Reduces the object to lowest terms
    def reduce(self):
        # Details go here
```

that reduces the object on whose behalf the method is called to lowest terms; for example, the fraction 10/20 would be reduced to 1/2.

- Given the definition of the geometric `Point` class, add a method named `distance`:

```
class Point:
    # Other details omitted

    # Returns the distance from this point to the
    # parameter p
    def distance(self, p):
        # Details go here
```

that returns the distance between the point on whose behalf the method is called and the parameter `p`.

- Consider the following Python code:

```

class Rectangle:
    """ Represents a geometrical rectangle at a specific location and with a
        specific width and height. """

    def __init__(self, point, width, height):
        """ Establishes the rectangle's lower-left corner, width, and height.
            point: a tuple representing the (x, y) location of the rectangle's l
            width: an integer representing the rectangle's width
            height: an integer representing the rectangle's height """
        # Ensure the lower-left corner's x and y coordinates fall in the range -
        x, y = point
        x = -100 if x < -100 else 100 if x > 100 else x
        y = -100 if y < -100 else 100 if y > 100 else y
        self.corner = x, y # Pack the x and y values into a tuple
        # Ensure the rectangle's width and height are both nonnegative
        self.width = 0 if width < 0 else width
        self.height = 0 if height < 0 else height

    def get_perimeter(self):
        """ Computes the perimeter of the rectangle. """
        return 2*self.width + 2*self.height

    def get_area(self):
        """ Computes the perimeter of the rectangle. """
        return self.width * self.height

    def get_width(self):
        """ Returns the width of the rectangle. """
        return self.width

    def get_height(self):
        """ Returns the height of the rectangle. """
        return self.height

    def grow(self, n):
        """ Increases the dimensions of the rectangle.
            Both width and height increase by one. """
        self.width += 1
        self.height += 1

    def move(self, x, y):
        """ Moves the rectangle's lower-left corner to the specified
            (x, y) location. """
        self.corner = x, y

    def intersect(self, other_rec):
        """ Returns true if rectangle other_rec overlaps this
            rectangle object; otherwise, returns false. """
        # Details omitted
        pass

```



```

def diagonal(self):
    """ Returns the length of a diagonal rounded to the nearest
        integer. """
    # Details omitted
    pass

def center(self):
    """ Returns the geometric center of the rectangle with
        the (x,y) coordinates rounded to the nearest integer. """
    # Details omitted
    pass

def is_inside(self, pt):
    """ Returns true if the tuple pt represents a point within
        the bounds of the rectangle; otherwise, returns false.
        As in soccer and tennis, "on the line" is "in." """
    # Details omitted
    pass

def main():
    rect1 = Rectangle((2, 3), 5, 7)
    rect2 = Rectangle((5, 13), 1, 3)
    rect3 = Rectangle((20, 40), -5, 45)
    rect4 = Rectangle((-510, -220), 5, -4)

    print(rect1.get_width())
    print(rect1.get_height())
    print(rect2.get_width())
    print(rect2.get_height())
    print(rect3.get_width())
    print(rect3.get_height())
    print(rect4.get_width())
    print(rect4.get_height())
    print(rect1.get_perimeter())
    print(rect1.get_area())
    print(rect2.get_perimeter())
    print(rect2.get_area())
    print(rect3.get_perimeter())
    print(rect3.get_area())
    print(rect4.get_perimeter())
    print(rect4.get_area())

if __name__ == '__main__':
    main()

```

- (a) What does the program print?
- (b) With regard to a `Rectangle` object's lower-left corner, what are the minimum and maximum values allowed for the x coordinate? What are the minimum and maximum values allowed for the y coordinate?

- (c) What is a `Rectangle` object's minimum and maximum width?
- (d) What is a `Rectangle` object's minimum and maximum height?
- (e) What happens when a client attempts to create a `Rectangle` object with parameters that are outside the acceptable ranges?
- (f) Implement the `intersect` method. The method should not modify the rectangle.
- (g) Implement the `diagonal` method. The method should not modify the rectangle.
- (h) Implement the `center` method. The method should not modify the rectangle.
- (i) Implement the `intersect` method. The method should not modify the rectangle.
- (j) Implement the `is_inside` method. The method should not modify the rectangle.
- (k) Complete the following function named `corner` that returns the lower-left corner of the `Rectangle` object `r` passed to it:

```
def corner(rect)
    """ Returns a tuple representing the lower-left corner of Rectangle object
    rect. """
    pass    # Replace with your code
```

You may not modify the `Rectangle` class.

(l)

- 9. Develop a `Circle` class that, like the `Rectangle` class above, provides methods to compute perimeter and area. The `Rectangle` instance variables are not appropriate for circles; specifically, circles do not have corners, and there is no need to specify a width and height. A center point and a radius more naturally describe a circle. Build your `Circle` class appropriately.
- 10. Given the `Rectangle` and `Circle` classes from questions above, write the following `encloses` function:

```
bool encloses(rect, circ)
    # Returns true if rectangle rect is large enough to
    # completely enclose circle circ regardless of the positions of the
    # two objects; otherwise, returns false.
    pass    # Replace with your code
```

The function returns true if circle `circ`'s dimensions would allow it to fit completely within rectangle `rect`. If `circ` is too big, the function returns false. The positions of `rect` and `circ` do not influence the result.

- 11. Consider the following Python code:

```
class Widget:
    def __init__(self, v = 40):
        if v >= 40:
            self.value = v
        else:
            self.value = 0

    def get(self):
        return self.value

    def bump(self):
```

```
        if self.value < 50:
            self.value += 1

def main():
    w1 = Widget()
    w2 = Widget(5)
    print(w1.get())
    print(w2.get())
    w1.bump()
    w2.bump()
    print(w1.get())
    print(w2.get())
    for i in range(20):
        w1.bump()
        w2.bump()
    print(w1.get())
    print(w2.get())

if __name__ == '__main__':
    main()
```

- (a) What does the program print?
- (b) If `wid` is a `Widget` object, what is the minimum value the expression `wid.get()` can return?
- (c) If `wid` is a `Widget` object, what is the maximum value the expression `wid.get()` can return?

Chapter 14

Class Design: Composition and Inheritance

In object-oriented programming, class design is key. In most cases, developers do not start empty handed but build new classes from one or more existing classes. The goal is for the new class to provide capabilities unavailable in the existing classes. This chapter explores two key techniques for reuse in class design: *composition* and *inheritance*.

14.1 Composition

The typical Python class contains instance variables, usually established by the class initializer. These instance variables refer to other objects. If these instance variables are essential to the intrinsic meaning of the class, we say objects of the class are *composed* of other, more fundamental objects. This relationship represents an object-oriented design technique known as *composition*. Examples of composition abound in the classes we have designed so far:

- Listing 13.5 (circle.py): Each `Circle` object is composed of a `center` tuple object and a `radius` numeric object. Removing either of these instance variables would disqualify our `Circle` objects from modeling mathematical circles.
- Listing 13.23 (rational.py): `Rational` objects hold numeric `numerator` and `__denominator` objects. Every rational number must have both a numerator and a denominator.
- Listing 13.24 (bankaccounts.py): `BankAccount` objects contain numeric (`__balance` and string (`___number` and `___name`) objects. All of these instance variables are essential in every bank account.
- Listing 13.9 (stopwatch.py): `Stopwatch` objects hold four other numeric objects: `_start_time`, `_running`, and `_elapsed`. These components are necessary to control a timer object in the manner demonstrated.
- Listing 13.11 (trafficlight.py): `TrafficLight` objects are composed of a string object (`color`), a Tk object (`canvas`), and three integer objects (`red_lamp`, `yellow_lamp`, and `green_lamp`—the `Canvas.create_oval` method calls return these integers; a canvas object keeps track of its shapes via integer indices known as *tags*).

- Listing 13.12 (trafficlightobject.py): TrafficLightWindow objects contain TrafficLight objects. Without TrafficLight objects, the graphical window would have no traffic light image to display.
- Listing 13.15 (functiontester.py): Each FunctionTester object holds two integer objects (__error_count and __total_count).
- Listing 13.17 (plotobj.py): Plotter objects are composed of a couple of Turtle objects (Pen and Screen) and four numeric objects (min_x, max_x, min_y, max_y).

We can deduce the composite nature of our classes by examining the instance variables used within their methods. Any expression of within a method of the form `self.something`, where *something* refers to an object of a type defined outside of the class itself indicates a dependence on an external object and is evidence of composition. Unless it was superfluous or unused, removing the reference to that external object will reduce the functionality of the class or possibly make the class completely inoperable.

Some of the objects involved in Listing 13.13 (multisizelightwindow.py) form a similar but slightly different relationship. Each MultisizeLightWindow object contains a list of nine TrafficLight and Button objects. The quantity *nine* is arbitrary—we could have had five or 20 lights of different sizes, and, except for the number of traffic lights, the MultisizeLightWindow object would not behave in a fundamentally different manner. In this case we cannot claim that MultisizeLightWindow objects are composed of multiple traffic light objects, but rather we say that MultisizeLightWindow objects contain an *aggregate* of traffic light objects. This concept is called *aggregation*. In some cases the line between composition and aggregation is not so clear cut and can be open to interpretation. Both aggregation and composition model the *has a* relationship between objects, but the composition model models a stronger relationship, the *needs a* relationship. Composition is sometimes called a *part-whole* relationship.

We can illustrate the concept of composition using higher-level objects. Listing 14.1 (comptrafficlight.py) redesigns the traffic light class with more explicit composition in mind.

Listing 14.1: comptrafficlight.py

```
import tkinter as tk
import tkinter.ttk as ttk

class CompLamp:
    """ Serves as one lamp within a traffic light object. """

    def __init__(self, parent: 'CompTrafficLight', width: int, order: int,
                  color: str='red', *args, **kwargs) -> None:
        """ Creates a new lamp to be used in a traffic light object.
        parent: The traffic light owning this lamp
        width: The width of the case of the circular lamp
        order: Distance of this lamp from the top of the traffic light
        color: The lamp's initial color (defaults to "red")
        *args: Additional arguments to pass to the ttk.Frame
               superclass constructor
        **kwargs: Additional keyword arguments to pass to the
                  ttk.Frame superclass constructor """
        self.frame = ttk.Frame(parent.frame, *args, **kwargs)
        self.canvas = tk.Canvas(self.frame, width=width, height=width, bg="gray",
                                highlightthickness=0)
        self.canvas.pack()
        self.color = color
```

```

        offset = width//8
        self.lamp = self.canvas.create_oval(offset, offset,
                                             7*offset,
                                             7*offset,
                                             fill='black')
        # Place the lamp in its proper place in the traffic light
        self.frame.grid(row=order, column=0)
        self.state = "off"

    def turn_on(self) -> None:
        """ Illuminates the lamp """
        self.state = "on"
        self.canvas.itemconfigure(self.lamp, fill=self.color)

    def turn_off(self) -> None:
        """ Turns off the lamp """
        self.state = "off"
        self.canvas.itemconfigure(self.lamp, fill='black')

    def resize(self, width: int) -> None:
        self.canvas.config(width=width, height=width)
        offset = width//8
        self.canvas.coords(self.lamp, offset, offset, 7*offset, 7*offset)

class CompTrafficLight:
    """ Models a simple traffic light widget """

    def __init__(self, root: ttk.Frame, wd: int, initial_color: str='red',
                  *args, **kwargs) -> None:
        """ Makes a new traffic light object.
            root is the parent widget.
            wd is the pixels width.
            The light's initial color is initial_color.
            Clients may pass additional arguments to the constructor of the
            light's frame via *args and **kwargs. """

        # Check to see if the supplied color is valid
        if initial_color not in ('red', 'yellow', 'green'):
            raise ValueError(initial_color + " is not a valid color")

        # Create the widget frame for the light
        self.frame = ttk.Frame(root, width=wd, *args, **kwargs)

        self.frame.grid(row=0, column=0) # Only widget in a 1x1 grid
        # Set initial color
        self.color = initial_color

        # Make lamp objects and store them a dictionary keyed by their color
        self.lamps = {'red'      : CompLamp(self, wd, 0, 'red'),
                      'yellow'  : CompLamp(self, wd, 1, 'yellow'),
                      'green'   : CompLamp(self, wd, 2, 'green')}

        # Turn on lamp for initial color
        self.lamps[self.color].turn_on()

```

```

def change(self) -> None:
    """ Changes the traffic light's color to the next color in
        the sequence. """
    # First, determine which color should be next
    if self.color == 'red':
        new_color = 'green'
    elif self.color == 'green':
        new_color = 'yellow'
    else: # Current color must be 'yellow'
        new_color = 'red'
    # Next, activate and deactivate the appropriate lamps
    self.lamps[self.color].turn_off()
    self.color = new_color
    self.lamps[self.color].turn_on()

def resize(self, width) -> None:
    """ Changes the traffic light's frame width according to the
        parameter passed by the caller. """
    for lamp in self.lamps.values():
        lamp.resize(width) # Resize each lamp in the dictionary of lamps

```

In Listing 14.1 (comptrafficlight.py), objects of the class `CompLamp` class form the building blocks for traffic lights. The `CompTrafficLight` class stores three of these lamp objects in a dictionary. This may appear to be aggregation, but for a traffic signal of this kind, the quantity three is neither arbitrary nor variable. In this case we safely can classify the lamps to traffic light relationship as composition.

The initializer in the `CompLamp` class accepts four required arguments (`self`, `parent`, `width`, and `order`), one named default argument (`color`), and additional optional arguments (`*args`) and keyword arguments (`**kwargs`). The first five parameters are meant for the `CompLamp` class itself, and the `*args` and `**kwargs` arguments are meant for the `ttk.Frame` instance variable on which a `CompLamp` object relies to handle the graphics. The initializer makes these extra parameters available just in case a client requires special control over the style of the frame object.

A `CompLamp` object basically is a rectangular Tk frame that contains a Tk canvas widget that draws a circular shape. A lamp has an “on” color, and its “off” color is black. The `turn_on` and `turn_off` methods adjust the lamp’s color accordingly. The `resize` method allows clients to change the size of a lamp after its creation.

The `CompTrafficLight` class specifies traffic light objects. Conceptually, a traffic light object is composed of lamp objects. The `CompTrafficLight` initializer accepts the optional `*args` and `**kwargs` to give clients additional control over the underlying Tk widgets that display the traffic light. The statement

```

self.lamps = {'red'      : CompLamp(self, wd, 0, 'red'),
              'yellow'   : CompLamp(self, wd, 1, 'yellow'),
              'green'    : CompLamp(self, wd, 2, 'green')}

```

maps a string color to a lamp object for easy access.

Each traffic light object maintains its own color state, and clients can change a light’s color via the `change` method. The `resize` method increases or decreases the size of the light by adjusting the sizes of the individual lamps. The dimensions of the frame of the traffic light will conform to the sizes of the lamps it holds—the Tk widgets take care of such details.

Listing 14.2 (comptraffilightapp.py) uses the comptraffilight module's CompTrafficLight class in yet another version of Listing 9.14 (tkinterlight.py) and Listing 13.12 (traffilightobject.py).

Listing 14.2: comptraffilightapp.py

```
import tkinter as tk
import tkinter.ttk as ttk

from comptraffilight import CompTrafficLight

class CompTrafficLightApp:
    """ Graphical window that displays a traffic light and change button. """

    def __init__(self) -> None:
        root = tk.Tk()          # Create the main window
        root.title("Traffic Light") # Set title bar text
        # Create widget container
        frame = ttk.Frame(root)
        frame.pack()
        # Create a graphical button and ensure it calls the do_button_press
        # method when the user presses it
        button = ttk.Button(frame, text='Change', command=self.do_button_press)
        # Make a traffic light object instance variable
        self.light = CompTrafficLight(frame, 100, padding=25)
        # Position button and canvas objects
        button.grid(row=0, column=0)
        self.light.frame.grid(row=0, column=1)
        # Start the GUI event loop
        root.mainloop()

    def do_button_press(self) -> None:
        """ The window manager calls this function when the user
            presses the graphical button. """
        self.light.change()

# Create and execute a traffic light window
CompTrafficLightApp()
```

Note the following statement in the initializer:

```
self.light = CompTrafficLight(frame, 100, padding=25)
```

This creates a CompTrafficLight object and takes advantage of the `**kwargs` parameter to add a padding of 25 pixels around the border of the light. This puts some additional space between the button and the traffic light image.

Listing 14.1 (comptraffilight.py) uses Tk widgets to produce the graphical images and deal with user interaction. These widgets are themselves Python objects. Our custom lamp and traffic light objects use Tk objects, but our custom objects and the Tk objects live in two different worlds. We developed our custom classes, and someone else developed the Tk classes. We must write extra code to allow our custom objects and Tk objects to interoperate correctly to achieve the effects we desire. This extra code is the price we pay for using a library that we did not write and, therefore, have no control over its contents. Our traffic light object must have an associated Tk frame object because we want to be able to do things with Tk frames.

What if we could design a custom traffic light class that itself was a `Tk Frame` class? That could simplify our code considerably. Section 14.2 introduces the concept of *inheritance* which makes this possible.

14.2 Class Inheritance

We can base a new class on an existing class using a technique known as *inheritance*. To demonstrate how inheritance works, we begin with a simple example. Consider a class named `Widget` that offers only a `__repr__` method that identifies a `Widget` object as a widget:

```
class Widget:
    def __repr__(self) -> str:
        return "I'm a widget"
```

Given this definition of the `Widget` class, the following code

```
w = Widget()
print(f'w: {w}')
```

prints

```
I'm a widget
```

Next, consider the following class definition:

```
class Gadget(Widget):
    pass
```

This at first glance may appear to declare an empty type because of the `pass` statement in the class body. The name `Widget` within parentheses at the end of the first line indicates a special relationship that this `Gadget` class has with the existing `Widget` class. This declaration specifies that the `Gadget` class is a new class based on the existing `Widget` class as a starting point. The following code

```
g = Gadget()
print(f'g: {g}')
```

prints

```
g: I'm a widget
```

We see that a `Gadget` object behaves exactly like a `Widget` object. This is because the `Gadget` class definition adds nothing in its class body. We say that the `Gadget` class *inherits* everything from the `Widget` class, and in this case it does not add or change anything. Since we base the `Gadget` class off of the `Widget` class, we say `Widget` is the *base class* and `Gadget` is the *derived class*. Alternative terminology calls `Widget` the *superclass* and `Gadget` the *subclass*.

Ordinarily, we use inheritance to specify a new class that offers new features or redefines existing features from an existing class. Consider the following simple classes:

```
class Widget:
    def __init__(self, x: int) -> None:
        self.value = x

    def __repr__(self) -> str:
```

```

        return "I'm a widget"

class Gadget(Widget):
    pass

w = Widget(14)
g = Gadget(5)
print(f'w.value: {w.value}')
print(f'g.value: {g.value}')
```

This code prints

```
w.value: 14
g.value: 5
```

Here we see that `Gadget` objects contain a `value` instance variable. This is because the `Gadget` class inherits the `__init__` method from `Widget`. If we give `Gadget` its own initializer, as in the following code:

```
class Gadget(Widget):
    def __init__(self, x: int) -> None:
        self.count = x
```

then `Gadget` objects will not contain a `value` attribute. The following code will fail:

```
Gadget g = new Gadget(5)
print(g.value)      # Error, Gadget objects have no value attribute
```

In this case we are *overriding* in the initializer that we ordinarily would inherit from `Widget`. While it is not uncommon for a subclass to provide its own initializer method, in most cases the subclass should ensure the superclass initialization occurs as well. Object-oriented programming best practice insists that an instance of a derived class should be able to do anything that an instance of the base class can do. If the `Gadget` class inherits from the `Widget` class, `w` is a `Widget` object, `g` is a `Gadget` object, and `w.value` is a valid expression, then if `g.value` ought to be a valid expression. In this case, the `__init__` method in the `Gadget` class should invoke the `__init__` method from its base class (or superclass). The way to do this is through the `super()` method:

```
class Gadget(Widget):
    def __init__(self, x: int) -> None:
        super().__init__(0)    # value attribute will be zero
        self.count = x
```

Here, the `Gadget` initializer method calls its superclass initializer method. More likely, the client will need to give the inherited `value` instance variable a specific value, not just zero:

```
class Gadget(Widget):
    def __init__(self, x: int, y: int) -> None:
        super().__init__(x)
        self.count = y
```

This will require clients creating a `Gadget` object to provide two integer arguments, as the following code shows:

```
g = Gadget(5, 10)
```

A subclass can add new attributes and methods that do not exist in the superclass, and they can redefine the way inherited methods work.

Recall our `Stopwatch` class we defined in Listing 13.9 (`stopwatch.py`). We can use `Stopwatch` objects to measure elapsed time. Recall that we can start and stop `Stopwatch` objects as often as necessary without resetting the time.

Suppose we need a stopwatch object that records the number of times the watch has been started since its last reset. We can build our enhanced `Stopwatch` class from scratch, but it would more efficient to begin with our existing unadorned `Stopwatch` class and somehow add on the features we need. We will not merely copy the source code from our existing `Stopwatch` class and then add code to it. The inheritance mechanism does not touch the source code of our original `Stopwatch` class, and, at the same time, it allows us to write as little new code as possible. Listing 14.3 (`countingstopwatch.py`) provides a practical example of inheritance, defining the new class of our enhanced stopwatch objects.

Listing 14.3: `countingstopwatch.py`

```
from stopwatch import Stopwatch

class CountingStopwatch(Stopwatch):
    def __init__(self) -> None:
        # Allow base class to do its initialization of the
        # inherited instance variables
        super().__init__()
        # Set number of starts to zero
        self._count = 0

    def start(self) -> None:
        # Count this start message unless the watch already is running
        if not self._running:
            self._count += 1
        # Let base class do its start code
        super().start()

    def reset(self) -> None:
        # Let base class reset the inherited instance variables
        super().reset()
        # Reset new instance variable unknown to base class method
        self._count = 0

    def count(self) -> None:
        return self._count
```

The line

```
from stopwatch import Stopwatch
```

indicates the code in this module will somehow use the `Stopwatch` class from Listing 13.9 (`stopwatch.py`).

The line

```
class CountingStopwatch(Stopwatch):
```

defines a new class named `CountingStopwatch`, but this new class is based on the existing class `Stopwatch`. This single line enables the `CountingStopwatch` class to *inherit* characteristics from

the `Stopwatch` class. `CountingStopwatch` objects automatically will have `start`, `stop`, `reset`, and `elapsed` methods because the `Stopwatch` class has them.

When only one class name appears within the parentheses, the kind of inheritance is called *single inheritance*. This section focuses on single inheritance, the most commonly used form of inheritance. Section 14.4 briefly examines *multiple inheritance*.

In this case, `Stopwatch` is the superclass (or base class) of `CountingStopwatch`. `CountingStopwatch` is a subclass (or derived class) of `Stopwatch`.

Even though a subclass inherits all the methods of its superclass, a subclass may add new methods and provide new code for an inherited method. The new `CountingStopwatch` class has an initializer method definition, as usual named `__init__`. When a class has exactly one superclass, the statement

```
super().__init__()
```

within its initializer definition invokes the initializer of its superclass. The expression `super()` thus refers to code in the superclass. The superclass initializer calls the `reset` method which establishes the `_start_time`, `_elapsed`, and `_running` instance variables. After executing the superclass initializer code, the subclass initializer defines and initializes a new instance variable named `_count`.

What happens if you do not provide an initializer (`__init__` method) for the subclass? In this case when creating an instance of the subclass the interpreter automatically will invoke the superclass initializer. If you *do* provide an `__init__` initializer in a subclass, the interpreter will *not* call the superclass initializer—you must call the superclass initializer via the `super` function to ensure the initialization responsibilities of the superclass occur.

Observe that the `super()` expression appears also in the `start` and `reset` methods in `CountingStopwatch`; in these methods `super()` similarly invokes the services of their namesake counterparts in the superclass. The `count` method is a brand new method not found in the superclass.

Note the definitions of the `start` and `reset` methods. Because these methods have the same names as methods found in the superclass, we say that they *override* the inherited methods of the same name. A good rule of thumb is this: you generally should invoke the `super()` version of the method within a method you are overriding. The `super` invocation within a method of a class that has exactly one superclass allows the superclass to manage the details known to the superclass. The subclass can take care of the details specific to the subclass via the rest of the code in the overridden method.

Notice that the `CountingStopwatch` class has no apparent `stop` method. In fact, it does have a `stop` method because it inherits the `stop` method as is from the `Stopwatch` class. The `start` and `reset` methods in `CountingStopwatch` must work with the new `_count` instance variable and thus must be overridden, but the `stop` method needs no enhancement at all. Since `stop` needs no changes or additions, we omit its definition within the `CountingStopwatch` class. A derived class may inherit a method as is from its base class, or it may override the method. An overridden method may invoke the services of the base class version via `super` and add some additional functionality, or, less commonly, it may not use the base class version of the method at all and do something completely different.

Listing 14.4 (`usecountingsw.py`) provides some sample client code that uses the `CountingStopwatch` class.

Listing 14.4: usecountingsw.py

```

from countingstopwatch import CountingStopwatch
from time import sleep

timer = CountingStopwatch()
timer.start()
sleep(10) # Pause program for 10 seconds
timer.stop()
print(f'Time: {timer.elapsed()}    Number: {timer.count()}')

timer.start()
sleep(5) # Pause program for 5 seconds
timer.stop()
print(f'Time: {timer.elapsed()}    Number: {timer.count()}')

timer.start()
sleep(20) # Pause program for 20 seconds
timer.stop()
print(f'Time: {timer.elapsed()}    Number: {timer.count()}')

```

Listing 14.4 (usecountingsw.py) produces

```

Time: 10.010378278632945    Number: 1
Time: 15.016618866378108    Number: 2
Time: 35.02881993198008     Number: 3

```

For a more interactive, graphical example, consider Listing 14.5 (stopwatchplay.py).

Listing 14.5: stopwatchplay.py

```

import turtle as t
from countingstopwatch import CountingStopwatch

timer = CountingStopwatch()

def draw_watch(sw: CountingStopwatch) -> None:
    """ Renders the stopwatch object sw in a digital hr:min:sec format. """

    # Compute time in hours, minutes, and seconds
    seconds = round(sw.elapsed())
    time = ""
    hours = seconds // 3600 # Compute total hours
    seconds %= 3600 # Update seconds remaining
    minutes = seconds // 60 # Compute minutes
    seconds %= 60 # Update seconds remaining
    # Each digit occupies two spaces; pad with leading zeros, if necessary
    time = f'{hours:0>2}:{minutes:0>2}:{seconds:0>2}'

    # Draw graphical string
    t.clear()
    t.penup()
    t.setposition(-200, 0)
    t.pendown()

```

```

t.write(time, font=("Arial", 64, "normal"))
t.penup()
t.setposition(-50, -50)
t.pendown()
t.write(sw.count(), font=("Arial", 24, "normal"))

def quit() -> None:
    """ Quits the program. """
    t.bye()

def update() -> None:
    """ Updates the program's view of the global stopwatch object. """
    draw_watch(timer)      # Draw the digital display
    t.ontimer(update, 500)  # Call the update function again after one-half second

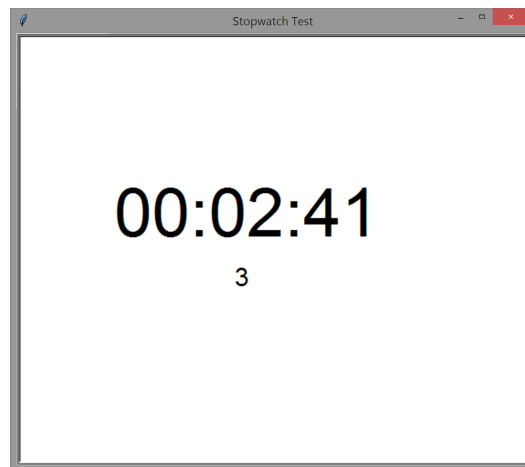
t.title("Stopwatch Test") # Set window's titlebar text
t.onkey(timer.start, "s")  # Call start method of timer if user presses s
t.onkey(timer.stop, "t")  # Call stop method of timer if user presses t
t.onkey(timer.reset, "r") # Call reset method of timer if user presses r
t.onkey(quit, "q")        # Call quit function if user presses r
t.listen()                # Ensure window receives keyboard events
t.delay(0)                # Draw as fast as possible
t.speed(0)                # Fastest turtle actions
t.ontimer(update, 500)    # Call update function every one-half second
t.hideturtle()            # Do not show the pen when drawing
t.mainloop()              # Start the show

```

Figure 14.1 shows a screenshot of Listing 14.5 (stopwatchplay.py) in the middle of a sample run. The smaller number below the digital readout represents the number of times the user has started the stopwatch from a nonrunning state. Pressing the **S** key starts the clock running. The **T** key stops the clock, and the **R** resets the clock back to 00:00:00. Attempts to start the stopwatch when it already is running will not increment the count. Resetting the stopwatch resets the time and sets the count to zero.

Section 2.1 introduced Python's `type` function. The `type` function returns the exact type of an object. A related function named `isinstance` returns `True` or `False` revealing whether or not its first argument (an expression) is an instance of the type specified by its second argument. The following interactive sequence illustrates (the `Stopwatch` type used in the sequence is from Listing 13.9 (stopwatch.py)):

Figure 14.1 The program in Listing 14.5 (stopwatchplay.py) in the middle of a sample run. The smaller number below the digital readout represents the number of times the stopwatch as been started since its creation or since it last was reset.



```
>>> type(5)
<class 'int'>
>>> isinstance(5, int)
True
>>> isinstance(5, float)
False
>>> isinstance(5, str)
False
>>> x = 2
>>> isinstance(x, int)
True
>>> type(x + 4.5)
<class 'float'>
>>> isinstance(x + 4.5, int)
False
>>> isinstance(x + 4.5, float)
True
>>> from stopwatch import Stopwatch
>>> sw = Stopwatch()
>>> isinstance(sw, Stopwatch)
True
>>> isinstance(sw, int)
False
>>> isinstance(sw, str)
False
```

This all appears to be in order. Now we will see how inheritance makes things more interesting. Consider the following new interactive session (the `Stopwatch` type is from Listing 13.9 (`stopwatch.py`), and the `CountingStopwatch` type is from Listing 14.3 (`countingstopwatch.py`)):

```
>>> from stopwatch import Stopwatch
>>> from countingstopwatch import CountingStopwatch
>>> sw = Stopwatch()
>>> csw = CountingStopwatch()
>>> type(sw)
<class 'stopwatch.Stopwatch'>
>>> type(csw)
<class 'countingstopwatch.CountingStopwatch'>
>>> isinstance(sw, Stopwatch)
True
>>> isinstance(csw, CountingStopwatch)
True
>>> isinstance(sw, CountingStopwatch)
False
>>> isinstance(csw, Stopwatch)
True
```

The `type` function reveals that the `sw` variable refers to an object of type `Stopwatch` and `csw` refers to an object of type `CountingStopwatch`. It follows that the `isinstance` function indicates that `sw` refers to an object that is an instance of `Stopwatch`. Similarly, `csw`'s object is an instance of `CountingStopwatch`. Not surprisingly, `isinstance` indicates that `sw` is not an instance of `CountingStopwatch`. Perhaps surprisingly, however, we see that `csw` is an instance of both the `CountingStopwatch` class *and* `Stopwatch` class!

The `issubclass` function determines if two classes are related by inheritance:

```
>>> issubclass(CountingStopwatch, Stopwatch)
True
>>> issubclass(Stopwatch, CountingStopwatch)
False
>>> issubclass(CountingStopwatch, CountingStopwatch)
True
>>> issubclass(Stopwatch, Stopwatch)
True
```

Note that by definition, every class technically is a subclass of itself.

Inheritance establishes a special relationship between two classes. An instance of a derived class is also an instance of the base class. In object-oriented software terminology this known as the *is a* relationship. The major benefit of this *is a* relationship is this: a client may safely use an instance of a derived class in any context meant to work with an instance of its base class. Suppose we write a function for timing a particular process:

```
def time_process(timer: Stopwatch) -> None:
    # Details omitted . . .
```

The parameter is supposed to be a reference to a `Stopwatch` object. As such, the `time_process` function can exercise any methods that a `Stopwatch` object provides: `start`, `stop`, and `reset`. Any class derived from `Stopwatch` will automatically inherit these methods and may or may not override their behavior. Regardless, the code within the `time_process` function legitimately may call the `start`, `stop`, or `reset` method on a `timer` object if its actual type is `Stopwatch` or `CountingStopwatch` (or any other class we may derive from `Stopwatch`). Listing 14.6 (`goodandbadprocess.py`) shows that the *is a* relationship is one directional.

Listing 14.6: `goodandbadprocess.py`

```
from stopwatch import Stopwatch
from countingstopwatch import CountingStopwatch

def process_1(timer: Stopwatch) -> None:
    pass

def process_2(timer: CountingStopwatch) -> None:
    pass

sw = Stopwatch()
csw = CountingStopwatch()

process_1(sw) # Okay; a stopwatch is a stopwatch
process_1(csw) # Okay; a counting stopwatch is a stopwatch

process_2(sw) # Type error
process_2(csw) # Okay; a counting stopwatch is a counting stopwatch
```

Even though Pylance may report a type error, the program will run without any errors. Since `process_2`'s formal parameter has declared type `CountingStopwatch`, it would be a type error to pass a `Stopwatch` instance as an actual parameter. Listing 14.6 (`goodandbadprocess.py`) shows that the *is a* relationship is

one directional. A CountingStopwatch instance *is a* Stopwatch, but a Stopwatch instance is not a CountingStopwatch.

Suppose we wish to use counting stopwatch objects that limit the number of times the clock can be stopped and restarted. Listing 14.7 (restrictedstopwatch.py) provides the RestrictedStopwatch class definition that leverages the capabilities of the CountingStopwatch object via inheritance:

Listing 14.7: restrictedstopwatch.py

```
from countingstopwatch import CountingStopwatch

class RestrictedStopwatch(CountingStopwatch):
    def __init__(self, n: int) -> None:
        """ Restrict the number stopwatch starts to n times. """
        # Allow superclass to do its initialization of the
        # inherited instance variables
        super().__init__()
        self._limit = n

    def start(self) -> None:
        """ If the count exceeds the limit, terminate the program's
            execution. """
        if self._count < self._limit:
            super().start() # Let superclass do its start code
        else:
            import sys
            print("Limit exceeded")
            sys.exit(1) # Limit exceeded, terminate the program
```

Listing 14.8 (testrestrictedsw.py) exercises the RestrictedStopwatch class.

Listing 14.8: testrestrictedsw.py

```
from restrictedstopwatch import RestrictedStopwatch

sw = RestrictedStopwatch(3)
print("Starting 1")
sw.start()
print("Stopping 1")
sw.stop()
print("Starting 2")
sw.start()
print("Stopping 2")
sw.stop()
print("Starting 3")
sw.start()
print("Stopping 3")
sw.stop()
print("Starting 4")
sw.start()
print("Stopping 4")
sw.stop()
print("Done")
```

The following shows the program's output:

```
Starting 1
Stopping 1
Starting 2
Stopping 2
Starting 3
Stopping 3
Starting 4
Limit exceeded
```

Trying to restart the clock past its limit of three attempts terminates the program.

Listing 14.7 (`restrictedstopwatch.py`) derives `RestrictedStopwatch` from `CountingStopwatch`. The `CountingStopwatch` class has `Stopwatch` as its base class. We say that `CountingStopwatch` is a *direct base class* of `RestrictedStopwatch` and `Stopwatch` is an *indirect base class* of `RestrictedStopwatch`. The inheritance relationship is *transitive*: since `RestrictedStopwatch` inherits from `CountingStopwatch` and `CountingStopwatch` inherits from `Stopwatch`, it is the case that `RestrictedStopwatch` inherits from `Stopwatch` as well. We can demonstrate this transitivity in the interactive interpreter:

```
>>> from stopwatch import Stopwatch
>>> from countingstopwatch import CountingStopwatch
>>> from restrictedstopwatch import RestrictedStopwatch
>>> rsw = RestrictedStopwatch(3)
>>> isinstance(rsw, RestrictedStopwatch)
True
>>> isinstance(rsw, CountingStopwatch)
True
>>> isinstance(rsw, Stopwatch)
True
```

As we can see, the `rsw` variable refers to an object that simultaneously *is a* `RestrictedStopwatch`, `CountingStopwatch`, and plain `Stopwatch`.

When we say simply “class A is a base class of B,” class A could be either a direct or indirect base class of class B. Similarly, when we say “B inherits from A,” the inheritance could be direct or indirect. Generally speaking, however, when we omit the qualifier *direct* or *indirect* in this context, we usually mean *direct*.

In Python, the built-in class `object` serves as the direct or indirect base class for all other types. All types (except `object`) inherit from `object`. The following demonstrates:

```
>>> from stopwatch import Stopwatch
>>> from countingstopwatch import CountingStopwatch
>>> from restrictedstopwatch import RestrictedStopwatch
>>> sw = Stopwatch()
>>> csw = CountingStopwatch()
>>> rsw = RestrictedStopwatch(3)
>>> isinstance(rsw, object)
True
>>> isinstance(sw, object)
True
>>> isinstance(4, object)
True
>>> isinstance(4.5, object)
True
```

```
>>> isinstance("Wow", object)
True
>>> obj = object()
>>> isinstance(obj, object)
True
```

The following class definition:

```
class Widget:
    pass
```

is an abbreviation for the following:

```
class Widget(object):
    pass
```

This means `object` is the implied direct base class for any class that does not explicitly specify a base class.

Suppose we derive another class from `Stopwatch`, one with a new method that returns a string containing the digital representation of the time, similar to the way the graphical program in Listing 14.5 (`stopwatchplay.py`) displays the elapsed time. Listing 14.9 (`digitalstopwatch.py`) provides such a class.

Listing 14.9: `digitalstopwatch.py`

```
from stopwatch import Stopwatch

class DigitalStopwatch(Stopwatch):
    def __repr__(self) -> str:
        """ Returns a string representation of the elapsed time
            in hours : minutes : seconds. """
        # Compute time in hours, minutes, and seconds
        seconds = round(self.elapsed())
        hours = seconds // 3600      # Compute total hours
        seconds %= 3600             # Update seconds remaining
        minutes = seconds // 60     # Compute minutes
        seconds %= 60               # Update seconds remaining
        # Each digit occupies two spaces; pad with leading zeros, if necessary
        return f'{hours:0>2}:{minutes:0>2}:{seconds:0>2}'
```

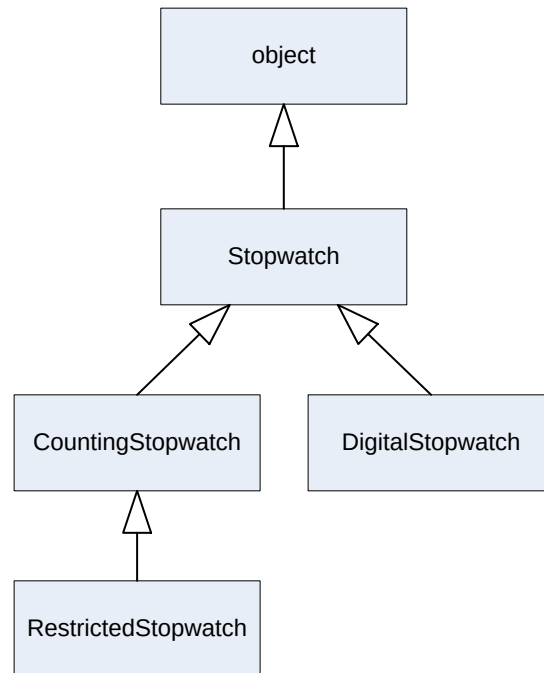
Listing 14.10 (`testdigitalsw.py`) exercises the `DigitalStopwatch` class.

Listing 14.10: `testdigitalsw.py`

```
from digitalstopwatch import DigitalStopwatch
from time import sleep

dsw = DigitalStopwatch()
print('Sleeping for about 2 minutes and 20 seconds')
dsw.start()      # Start the timer
sleep(140)       # Do nothing for 2:20
dsw.stop()       # Stop the timer
print(dsw)       # Print elapsed time in digital format
```

Figure 14.2 A Unified Modeling Language diagram representing the `Stopwatch` class hierarchy. A rectangle represents a class, and an arrow points from a derived class to its base class. As shown here, the `object` class is the ultimate superclass of all Python classes.



The `DigitalStopwatch` class does not add any new instance variables and does not modify how existing methods work; it simply adds a new method. When executed, the program prints a message, then appears to do nothing for 140 seconds (two minutes and 20 seconds), and then prints

```
Sleeping for about 2:20
00:02:20
```

At this point we have several classes derived from `Stopwatch`. A collection of classes related through inheritance is called a *class hierarchy*, or *inheritance hierarchy*. Figure 14.2 illustrates the `Stopwatch` hierarchy using a standard graphical notation known as the *Unified Modeling Language*, or *UML* (see <http://www.uml.org>). In the UML, a rectangle represents a class, and an arrow with a hollow arrowhead points from a derived class to its immediate base class. A UML class diagram such as the one in Figure 14.2 communicates to developers the relationships among the classes more quickly than the textual Python source code that potentially is spread out over multiple files.

Section 14.1 alluded to the conceptual mismatch between the custom classes from Listing 14.1 (`comptrafficlight.py`) and the standard Tk widget classes on which the custom classes depend. Inheritance provides a way to unify these two distinct sets of classes. We can build custom Tk widgets from existing Tk widget classes using inheritance. Listing 14.11 (`trafficsignal.py`) uses both composition and inheritance in the design of a new traffic light class called `TrafficSignal`.

Listing 14.11: trafficsignal.py

```

import tkinter as tk
import tkinter.ttk as ttk

class Signallamp(ttk.Frame):
    """ Serves as one lamp within a traffic light object. """

    def __init__(self, parent: ttk.Frame, width: int,
                  color: str='red', *args, **kwargs) -> None:
        """ Creates a new lamp to be used in a traffic light object.
            parent: The parent widget
            width: The width of the case of the circular lamp
            color: The lamp's initial color (defaults to "red")
            *args: Additional arguments to pass to the ttk.Frame
                   superclass initializer
            **kwargs: Additional keyword arguments to pass to the
                     ttk.Frame superclass initializer """
        super().__init__(parent, *args, **kwargs)
        self.canvas = tk.Canvas(self, width=width, height=width, bg="gray",
                                highlightthickness=0)

        self.canvas.pack()
        self.color = color
        offset = width//8
        self.lamp = self.canvas.create_oval(offset, offset,
                                             7*offset,
                                             7*offset,
                                             fill='black')

        self.status = 'off'

    def turn_on(self) -> None:
        """ Illuminates the lamp """
        self.status = 'on'
        self.canvas.itemconfigure(self.lamp, fill=self.color)

    def turn_off(self) -> None:
        """ Turns off the lamp """
        self.status = 'off'
        self.canvas.itemconfigure(self.lamp, fill='black')

    def resize(self, width: int) -> None:
        """ Adjusts the width of the lamp's frame """
        self.canvas.config(width=width, height=width)
        offset = width//8
        self.canvas.coords(self.lamp, offset, offset, 7*offset, 7*offset)

class TrafficSignal(ttk.Frame):
    """ Models a simple traffic light widget """

    def __init__(self, parent: ttk.Frame, wd: int, initial_color='red',
                  *args, **kwargs) -> None:
        """ Makes a new traffic light object.
            parent is the parent widget.

```

```

        wd is the pixels width.
        The light's initial color is initial_color. """

    super().__init__(parent, width=wd, *args, **kwargs)

    # Check to see if the supplied color is valid
    if initial_color not in ('red', 'yellow', 'green'):
        raise ValueError(initial_color + " is not a valid color")

    self.grid(row=0, column=0) # Only widget in a 1x1 grid
    # Initialize instance variables
    self.color = initial_color

    # Make lamp objects and store them a dictionary keyed by
    # their color
    self.lamps = {'red'      : SignalLamp(self, wd, 'red'),
                  'yellow'  : SignalLamp(self, wd, 'yellow'),
                  'green'   : SignalLamp(self, wd, 'green')}

    # Add the lamps to the frame
    self.lamps['red'].grid(row=0, column=0)
    self.lamps['yellow'].grid(row=1, column=0)
    self.lamps['green'].grid(row=2, column=0)

    # Turn on lamp for initial color
    self.lamps[self.color].turn_on()

def change(self) -> None:
    """ Changes the traffic light's color to the next color in
        the sequence. """
    if self.color == 'red':
        new_color = 'green'
    elif self.color == 'green':
        new_color = 'yellow'
    else:          # 'yellow':
        new_color = 'red'
    self.lamps[self.color].turn_off()
    self.color = new_color
    self.lamps[self.color].turn_on()

def resize(self, width: int) -> None:
    """ Changes the traffic light's frame width according to the
        parameter passed by the caller. """
    for lamp in self.lamps.values():
        lamp.resize(width)

```

Before, a traffic light lamp object *had* a Tk frame object; now, a traffic light lamp object *is* a Tk frame object. Similarly, a traffic light object *is* a Tk frame object. Listing 14.12 (newtrafficlightapp.py) is our final installment of the button-controlled traffic light image found in Listing 9.14 (tkinterlight.py), Listing 13.12 (trafficlightobject.py), and Listing 14.2 (comptrafficlightapp.py).

Listing 14.12: newtrafficlightapp.py

```

import tkinter as tk
import tkinter.ttk as ttk

from trafficsignal import TrafficSignal

class NewTrafficLightApp:
    """ Graphical window that displays a traffic light and change button. """

    def __init__(self) -> None:
        root = tk.Tk()          # Create the main window
        root.title("Traffic Light") # Set title bar text
        # Create widget container
        frame = ttk.Frame(root)
        frame.pack()
        # Create a graphical button and ensure it calls the do_button_press
        # method when the user presses it
        button = ttk.Button(frame, text='Change', command=self.do_button_press)
        # Create and add a drawing surface within the window
        # Make a traffic light object instance variable
        self.light = TrafficSignal(frame, 100, padding=25)
        #self.light.config(padding=25)
        # Position button and canvas objects
        button.grid(row=0, column=0)
        self.light.grid(row=0, column=1)
        # Start the GUI event loop
        root.mainloop()

    def do_button_press(self) -> None:
        """ The window manager calls this function when the user
            presses the graphical button. """
        self.light.change()

# Create and execute a traffic light window
NewTrafficLightApp()

```

Our redesigned `TrafficSignal` class and inheritance makes it easy to add functionality by adding new traffic light types. Listing 14.13 (`movablelight.py`) augments the `TrafficSignal` class, enabling clients to reposition the traffic light's center to a given (x,y) location within the graphics window.

Listing 14.13: movablelight.py

```

import tkinter as tk
import tkinter.ttk as ttk

from trafficsignal import TrafficSignal

class MovableLight(TrafficSignal):
    """ Models a graphical traffic light that the user can reposition """

    def __init__(self, parent: ttk.Frame, width: int,
                 initial_color: str='red') -> None:
        """ Makes a new movable traffic light object. The light's
            parent is the parent widget
            The width of the light is width pixels.
            The light's initial color is initial_color. """
        # Call base class initializer
        super().__init__(parent, width, initial_color)
        self.place(x=0, y=0)

    def move_to(self, x: int, y: int) -> None:
        """ Positions the light's center at point (x, y) """
        center_x_offset = self.winfo_width() // 2
        center_y_offset = self.winfo_height() // 2
        self.x = x - center_x_offset
        self.y = y - center_y_offset
        self.place(x=self.x, y=self.y)

```

Listing 14.14 (movablelighttest.py) is a small application that allows the user to move the traffic light image to various places in the window via mouse click. The user changes the light by pressing the **C** key. The program also exercises the inherited `resize` method by allowing the user to press the **<** key to shrink or **>** to grow the traffic light object.

Listing 14.14: movablelighttest.py

```

import tkinter as tk
import tkinter.ttk as ttk

from movablelight import MovableLight

class MovableLightTest:
    """ Graphical window that allows a user to move and resize
        a traffic light object. """

    def __init__(self) -> None:
        root = tk.Tk()          # Create the main window
        root.title('Movable Traffic Light Test') # Set title bar text
        width, height = 800, 600
        f = ttk.Frame(root, width=width, height=height)
        f.pack()
        # Place traffic light at the center of the window frame
        print('Frame width:', f.winfo_width())
        self.light_width = height//10
        self.light = MovableLight(f, self.light_width, 'green')

```

```

f.bind('<Button-1>', self.mouse_pressed)
f.bind('<Key>', self.key_pressed)
f.focus_set() # Allow graphical window to receive key strokes
print('width//2 =', width//2, ' height//2 =', height//2)
self.light.move_to(width//2, height//2)
# Start the GUI event loop
root.mainloop()

def mouse_pressed(self, event) -> None:
    """ The window manager calls this function when the user
        presses the graphical button. """
    self.light.move_to(event.x, event.y)

def key_pressed(self, event) -> None:
    """ The window manager calls this function when the user
        presses the graphical button. """
    ch = event.char
    if ch == '>' or ch == '.':
        self.light_width += 5
        self.light.resize(self.light_width)
    elif ch == '<' or ch == ',':
        self.light_width -= 5
        self.light.resize(self.light_width)
    elif ch == 'C' or ch == 'c':
        self.light.change()
    print('Key pressed')

# Create and execute a traffic light window
MovableLightTest()

```

As our farewell to graphical traffic light objects, Listing 14.15 (turnlight.py) defines a traffic signal that includes a left turn arrow lamp.

Listing 14.15: turnlight.py

```

from tkinter.ttk import Frame

from trafficsignal import Signallamp
from movablelight import MovableLight

class TurnLamp(Signallamp):
    def __init__(self, parent: Frame, width: int, *args, **kwargs) -> None:
        """ Creates a new lamp to be used in a traffic light object.
            parent: The parent widget
            width: The width of the case of the circular lamp
            *args: Additional arguments to pass to the ttk.Frame
                   superclass constructor
            **kwargs: Additional keyword arguments to pass to the
                     ttk.Frame superclass constructor """
        super().__init__(parent, width, *args, **kwargs)
        self.color = 'turn'
        offset = width//8
        self.arrow_shaft = self.canvas.create_line(2*offset, 4*offset,

```

```

        6*offset, 4*offset,
        fill="black", width=6)
self.arrow_top_wing = self.canvas.create_line(2*offset, 4*offset,
        4*offset, 2*offset,
        fill="black", width=6)
self.arrow_bottom_wing = self.canvas.create_line(2*offset, 4*offset,
        4*offset, 6*offset,
        fill="black", width=6)

self.status = 'off'

def turn_on(self) -> None:
    """ Illuminates the lamp """
    self.status = 'on'
    self.canvas.itemconfigure(self.arrow_shaft, fill="green")
    self.canvas.itemconfigure(self.arrow_top_wing, fill="green")
    self.canvas.itemconfigure(self.arrow_bottom_wing, fill="green")

def turn_off(self) -> None:
    """ Turns off the lamp """
    self.status = 'off'
    self.canvas.itemconfigure(self.arrow_shaft, fill="black")
    self.canvas.itemconfigure(self.arrow_top_wing, fill="black")
    self.canvas.itemconfigure(self.arrow_bottom_wing, fill="black")

def resize(self, width: int) -> None:
    super().resize(width)
    offset = width//8
    self.canvas.coords(self.arrow_shaft, 2*offset, 4*offset,
        6*offset, 4*offset)
    self.canvas.coords(self.arrow_top_wing, 2*offset, 4*offset,
        4*offset, 2*offset)
    self.canvas.coords(self.arrow_bottom_wing, 2*offset, 4*offset,
        4*offset, 6*offset)

class TurnLight(MovableLight):
    """ Models a traffic light with a left turn arrow. """

    def __init__(self, parent: Frame, width: int,
        initial_color: str="red") -> None:
        """ Makes a new traffic light object with left turn arrow.
            The is width pixels wide.
            The light's initial color is initial_color. """
        # Call superclass initializer
        super().__init__(parent, width, initial_color)

        # Add new turn lamp widget to the fourth row
        self.lamps['turn'] = TurnLamp(self, width)
        self.lamps['turn'].grid(row=3, column=0) # 4th row

    def change(self) -> None:
        """ Changes the traffic light's color to the next color in
            the sequence. """

```

```

if self.color in ('yellow', 'green'):
    super().change() # Yellow and green behave as before
else:
    if self.color == 'red':
        new_color = 'turn' # Red now changes to turn arrow
    else: # 'turn':
        new_color = 'green' # Turn arrow changes to green
    self.lamps[self.color].turn_off()
    self.color = new_color
    self.lamps[self.color].turn_on()

```

The TurnLamp class inherits from SignalLamp. The superclass can draw the circle, but the subclass must provide the code to draw the three lines that comprise a simple arrow pointing to the left indicating the permission to make a left turn. Because TurnLamps present more complicated shapes to draw, we must override the turn_on, turn_off, and resize methods. Note that the resize method invokes the superclass version of resize to take care of the parts of the lamp that do not involve repositioning the lines of the arrow.

The TurnLight class inherits from MovableLight, not directly from TrafficSignal. This means we do not need to add code that enables clients to move TurnLight objects; this functionality comes for free.

Listing 14.16 (turnlighttest.py) allows a user to experiment with TurnLight objects.

Listing 14.16: turnlighttest.py

```

import tkinter as tk
import tkinter.ttk as ttk

from turnlight import TurnLight

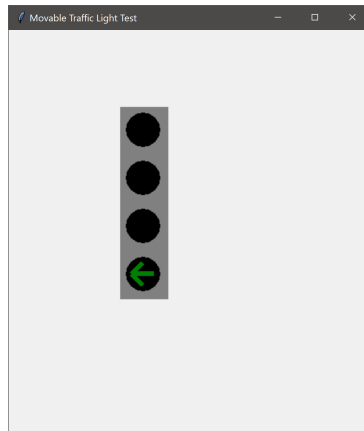
class TurnLightTest:
    """ Graphical window that displays multiple traffic lights
        of different sizes. """

    def __init__(self) -> None:
        root = tk.Tk() # Create the main window
        root.title("Movable Traffic Light Test") # Set title bar text
        width, height = 800, 600
        f = ttk.Frame(root, width=width, height=height)
        f.pack()
        # Place traffic light at the center of the window frame
        print("Frame width:", f.winfo_width())
        self.light = TurnLight(f, height//10, "green")
        f.bind("<Button-1>", self.mouse_pressed)
        f.bind("<Key>", self.key_pressed)
        f.focus_set()
        print("width//2 =", width//2, " height//2 =", height//2)
        self.light.move_to(width//2, height//2)
        self.size = height//10
        # Start the GUI event loop
        root.mainloop()

    def mouse_pressed(self, event) -> None:
        """ The window manager calls this function when the user

```

Figure 14.3 Sample execution of Listing 14.16 (turnlighttest.py). The user can move the light by clicking at a location within the window, and the keyboard controls the light's size and color state.



```

        presses the graphical button. """
        self.light.move_to(event.x, event.y)
        print("Mouse pressed at", event.x, ",", event.y)

def key_pressed(self, event) -> None:
    """ The window manager calls this function when the user
        presses the graphical button. """
    ch = event.char
    if ch == ">" or ch == ".":
        print("Resize not implemented")
        self.size += 1
        self.light.resize(self.size)
    elif ch == "<" or ch == ",":
        print("Resize not implemented")
        self.size -= 1
        self.light.resize(self.size)
    elif ch == "C" or ch == "c":
        self.light.change()
        print("Key pressed")

# Create and execute a traffic light window
TurnLightTest()

```

Figure 14.3 shows a sample execution of Listing 14.16 (turnlighttest.py).

Review the code in Listing 14.14 (movablelighttest.py) and Listing 14.16 (turnlighttest.py). Do you see anything troubling in the two applications? The two programs look almost identical. Both programs present the same interface to the user; that is, the same reactions to mouse clicks and button presses. The only difference is the exact type of traffic light the user can manipulate. Listing 14.14 (movablelighttest.py) hardwires a `MovableLight` object into the code, while Listing 14.16 (turnlighttest.py) instead uses a `TurnLight`.

Section 7.6 enumerates the disadvantages of code duplication. How can we eliminate the duplicated

code in these two applications? We can parameterize the type of traffic light the program creates. Listing 14.17 (parammovable.py) allows the user to select the type of light to manipulate.

Listing 14.17: parammovable.py

```
import tkinter as tk
import tkinter.ttk as ttk

from movablelight import MovableLight
from turnlight import TurnLight

class ParameterizedMovable:
    """ Graphical window that allows a user to move and resize
        a traffic light object. """

    def __init__(self, light_class) -> None:
        """ Creates a ParameterizedLightTester.
            light_class is the type of light to create;
            it must be MovableLight or one of its subclasses. """
        root = tk.Tk()          # Create the main window
        root.title("Movable Traffic Light Test") # Set title bar text
        width, height = 800, 600
        f = ttk.Frame(root, width=width, height=height)
        f.pack()
        # Place traffic light at the center of the window frame
        print("Frame width:", f.winfo_width())
        self.light_width = height//10
        self.light = light_class(f, self.light_width, "green")
        f.bind("<Button-1>", self.mouse_pressed)
        f.bind("<Key>", self.key_pressed)
        f.focus_set() # Allow graphical window to receive key strokes
        print("width//2 =", width//2, " height//2 =", height//2)
        self.light.move_to(width//2, height//2)
        # Start the GUI event loop
        root.mainloop()

    def mouse_pressed(self, event) -> None:
        """ The window manager calls this function when the user
            presses the graphical button. """
        self.light.move_to(event.x, event.y)

    def key_pressed(self, event) -> None:
        """ The window manager calls this function when the user
            presses the graphical button. """
        ch = event.char
        if ch == ">" or ch == ".":
            self.light_width += 5
            self.light.resize(self.light_width)
        elif ch == "<" or ch == ",":
            self.light_width -= 5
            self.light.resize(self.light_width)
        elif ch == "C" or ch == "c":
            self.light.change()
        print("Key pressed")
```

```

# Create and execute a traffic light window
print("What kind of light do you want play with?")
light = input("Standard Turn: ").upper()
if light == 'S':
    ParameterizedMovable(MovableLight) # Passing class, not an instance
elif light == 'T':
    ParameterizedMovable(TurnLight)    # Passing class, not an instance
else:
    print(light, "is not a valid choice")

```

We used a small bit of nongraphical code to allow the user to choose between the two traffic light types. The initializer of the `ParameterizedMovable` class in Listing 14.17 (`parammovable.py`) accepts a *class*, not an *instance* of a class, as an argument. We can do this because a class in Python is a real object (see Section 13.8). The type hint for a class is `Type`. We must import the `Type` type hint from the `typing` module in order hint.

14.3 Composition versus Inheritance

Often we can choose between using composition or inheritance to leverage the functionality of an existing class. The better choice is not always apparent. We can achieve the effects of inheritance using strictly composition and delegation. Our `CountingStopwatch` class in Listing 14.3 (`countingstopwatch.py`) uses inheritance to create the `CountingStopwatch` class from the simpler `Stopwatch` class. Listing 14.18 (`countingstopwatch2.py`) defines the class `CountingStopwatch2` that behaves identically to `CountingStopwatch`, but it uses composition to reuse the functionality of `Stopwatch`. Clients would not perceive any functional differences between a `CountingStopwatch` instance and a `CountingStopwatch2` instance.

Listing 14.18: `countingstopwatch2.py`

```

from stopwatch import Stopwatch

class CountingStopwatch2:
    """ This counting stopwatch uses composition instead of
        inheritance. """
    def __init__(self) -> None:
        # Create a stopwatch object to keep the time
        self.watch = Stopwatch()
        # Set number of starts to zero
        self._count = 0

    def start(self) -> None:
        # Count this start message unless the watch already is running
        if not self.watch._running:
            self._count += 1
        # Delegate other work to the stopwatch object
        self.watch.start()

    def stop(self) -> None:
        # Delegate to stopwatch object
        self.watch.stop()

```

```
def reset(self) -> None:
    # Let the stopwatch object reset the time
    watch.reset()
    # Reset the count
    self._count = 0

def elapsed(self) -> float:
    # Delegate to stopwatch object
    return self.watch.elapsed()

def count(self) -> int:
    return self._count
```

In order to retrofit Listing 14.4 (usecountingsw.py) to use our new counting stopwatch class change the import line

```
from countingstopwatch import CountingStopwatch
```

to be

```
from countingstopwatch2 import CountingStopwatch2
```

and change the statement

```
timer = CountingStopwatch()
```

to be

```
timer = CountingStopwatch2()
```

With these changes the program works exactly the same as before. Apply the same changes to Listing 14.5 (stopwatchplay.py), and you will see similar results.

While the inheritance version and the composition version both accomplish the same goals from the client's perspective, notice that the composition version requires more code. With inheritance, if a method from the superclass needs no changes, the programmer omits its definition in the subclass. With composition, however, every method meant to be used by clients in the original class must have a definition in the new class. If the new class does not need to change the method in any way, it simply delegates the work to the contained instance of the original class. We see this in the `stop` and `elapsed` methods in `CountingStopwatch2`. This can make a big difference in the work required to design a new class if the original class has many methods that need no change. We must maintain the interface of the original class if we wish to simulate the *is a* relationship without using inheritance.

Good object-oriented designs often combine composition and inheritance to achieve useful results. Suppose we are building a software system that manages a manufacturing process. A physical temperature sensor attached to a piece of equipment relays temperature information to the software via a software object. The software expects to receive an instance of the `TemperatureSensor` class from the sensor. The `TemperatureSensor` class provides only two methods: `read`, which returns the current temperature in degrees Celsius of its attached hardware, and `test`, which puts the sensor into a self test mode. The software on the physical sensor that sends the `TemperatureSensor` objects to the system is propriety, and its license forbids reverse engineering its code to change its behavior. We are stuck with sensors that provide `TemperatureSensor` objects. Fortunately, since the part of the software system that uses the information sent by the sensors is made by the same company that provides the sensors, everything works well together.

One day your company decides to replace the existing equipment management software with a new system offered by another vendor. The new system provides much greater control and monitoring capabilities, and the cost of the annual licensing fees are lower than the existing system. There is a problem, however. The new system expects different kinds of sensors on the equipment to monitor. The sensors the new software requires send information via objects of type `ThermalValue`. The `ThermalValue` class provides only a `temperature` method that returns the current temperature in degrees Fahrenheit. The new software does not provide the ability to self test a sensor.

The overall manufacturing process uses hundreds of very expensive sensors. The cost to replace all the perfectly-functioning current sensors with sensors compatible with the new software would be enormous. Does your company have to forgo the software upgrade that can reduce costs and improve the manufacturing process?

Fortunately there is a simple software solution. The following class adapts old, `TemperatureSensor` objects to work with the newer software that expects `ThermalValue` objects:

```
class TemperatureSensorAdapter(ThermalValue):
    """ Adapts a TemperatureSensor object to appear as
        a ThermalValue object. """

    def __init__(self, sensor: TemperatureSensor) -> None:
        """ sensor is the TemperatureSensor object to adapt """
        self._sensor = sensor

    def temperature(self) -> float:
        """ Converts the sensor input from Celsius to Fahrenheit """
        return 9 * self._sensor.read() / 5 + 32
```

All you need to do is intercept the `TemperatureSensor` object from an existing sensor, create a new `TemperatureSensorAdapter` object from that `TemperatureSensor` object, and then send this `TemperatureSensorAdapter` object on to the new management and monitoring software.

The new system receives a `ThermalValue` object, because a `TemperatureSensorAdapter` object *is a* `ThermalValue` object. Inheritance guarantees the *is a* relationship. The `temperature` method delegates most of its work to the contained `TemperatureSensor` object. The `TemperatureSensor` object provides the temperature, and the method merely needs to convert the temperature from Celsius to Fahrenheit to conform to the expectations of the new system.

Since `ThermalValue` objects have no `test` method, we know the new software system will never attempt to call it. It is functionality that the `TemperatureSensor` provides that will never be used, so we safely can ignore it.

Note that this solution uses both inheritance and composition. It is an example of an object-oriented *design pattern*. A design pattern provides a solution to a commonly occurring problem in software design (see https://en.wikipedia.org/wiki/Software_design_pattern). A design pattern does not provide an exact solution to a given problem; it instead shows how we can apply techniques such as inheritance and composition to solve problems of a particular kind. The link above describes over 50 design patterns.

Each design pattern has a name, and the design pattern involved in our `TemperatureSensorAdapter` class is aptly named *adapter*. Our `TemperatureSensorAdapter` class adapts both the interface (the provided method is named `read`, but the required method name is `temperature`) and computation (the provided units are degrees Celsius, but the required units are degrees Fahrenheit). The adapter pattern is sometimes called the *wrapper* pattern because object serves as a wrapper around another object. In our

example, a `TemperatureSensorAdapter` object wraps a `ThermalValue` object.

Section 14.4 provides a working example of the use of the adapter design pattern.

When it comes to choosing composition or inheritance for class reuse, which should you use? The prevailing wisdom says to prefer composition over inheritance. Composition provides greater flexibility at the cost of potentially more code to write. When you feel tempted to use inheritance, ask yourself if an instance the new class philosophically *is an* instance of the original class. If the *is a* relationship makes sense, inheritance is probably the better choice; otherwise, use composition.

Consider a motor vehicle. The drive train of a motor vehicle contains (sounds like composition) axles and wheels, among other essential parts. Each axle assembly consists of an axle and two wheels. Would it make sense to derive an axle assembly class from an axle class and add a left wheel object and a right wheel object as new instance variables? While this would work, we must ask: Is an axle assembly an axle (*is a* relationship)? No, an axle assembly has an axle and has two wheels (*has a* relationships), so composition is the better choice in this case.

14.4 Multiple Inheritance

In Python we can derive a class from multiple base classes through a technique known as *multiple inheritance*. This allows a class to blend the traits from multiple existing classes. Single inheritance is fairly straightforward, but the issues involved with multiple inheritance can be complex and subtle. One problem is ambiguity. Consider the following class definitions:

```
class A(object):
    def f(self) -> None:
        print("A's f method")

class B(object):
    def f(self) -> None:
        print("B's f method")

class C(A):
    pass

class D(A, B):
    pass
```

Now consider an instance of class C:

```
c_obj = C()
```

What does the following print, if anything?

```
c_obj.f()
```

We do not see a method named `f` defined within class C, but its superclass, A, provides an `f` method; therefore, the statement prints

```
A's f method
```

because `c_obj` executes the `f` it inherits from its superclass.

Next, we will create a D object, as shown here:

```
d_obj = D()
```

Class D does not override `f`, but it inherits method `f` from both A and B. The problem is this: `A.f` and `B.f` do two different things. What should the following statement print?

```
d_obj.f()
```

Does inheritance give favor to class A's `f` method and exclude B's `f` method? Does it instead invoke class B's `f` method to the exclusion of A's `f` method? Perhaps it executes both A's `f` method and B's `f` method? The statement actually prints

```
A's f method
```

Only one method executes, and it's A's method. If we modify the class definition of D to be

```
class D(B, A):    # Note change in base class ordering
    pass
```

and change none of the other classes, the statement

```
d_obj.f()
```

prints

```
B's f method
```

Changing the order of the listed superclasses in the class definition changes the behavior of the statement. This sensitivity to base class ordering and the ambiguity introduced when inheriting identically-named methods from multiple superclasses are just two of the issues that can make multiple inheritance tricky.

Python supports a kind of multiple inheritance called *cooperative multiple inheritance*. This means programmers must implement each class involved in a multiple inheritance hierarchy in a careful way to ensure every thing works properly. This also means it is more work (but not impossible) to involve a class in a multiple inheritance hierarchy that was not designed with multiple inheritance in mind.

Multiple inheritance is not used as much as single inheritance; in fact, several widely-used modern object-oriented programming languages provide single inheritance but do not support multiple inheritance in the way Python does. Because multiple inheritance is a useful tool in some situations we will examine it briefly.

Our foray into multiple inheritance requires a gentle warning. Python distinguishes itself from most other programming languages due to its relatively simple and straightforward way of allowing programmers to express solutions to problems. Python ordinarily presents a very low barrier for transforming thought into code. Programmers must be diligent at all times, of course, but, in order to implement multiple inheritance in a sound way, Python requires programmers to take extra care when designing a class hierarchy involving multiple inheritance.

Remember the ambiguity in subclasses introduced by multiple inheritance when superclass methods have the same name? We could be careful and make sure the methods in all our classes have unique names. Would this approach eliminate the ambiguity problem at the expense, perhaps, of creating mangled, unnatural names for some of our methods? Potentially convoluted names definitely are not a good idea, and, besides, this approach would not eliminate the ambiguity problem. Some methods have names that cannot change; for example, class initializers always are named `__init__`.

To explore the complexities of multiple inheritance with constructors, consider the following two classes:

```

class A(object):
    def __init__(self) -> None:
        print("Making an A object")
        self.value_A = 0

class B(object):
    def __init__(self) -> None:
        print("Making a B object")
        self.value_B = 1

```

Suppose now we define class C as

```

class C(A, B):
    pass

```

Our intention is that C objects should combine the capabilities of A objects and B objects; we want every C object to have both `value_A` and `value_B` instance variables. With single inheritance everything would work as expected. We do not provide a constructor for C, so, as with single inheritance, the creation of a C object would execute the superclass `__init__` method, if it exists. Given our earlier experiment with multiple inheritance, what would you expect the following code to print?

```

c_obj = C()           # Make a C object
print("=====")
print(c_obj.__dict__) # Look at its instance variables

```

Since class A was listed first in the list of superclasses, it prints

```

Making an A object
=====
{'value_A': 0}

```

The creation of `c_obj` properly initialized only the A part of `c_obj`.

As in single inheritance, adding a subclass constructor without calling `super` prevents any superclass constructors from executing. To show this, rewrite class C as

```

class C(A, B):
    def __init__(self) -> None:
        print("Making a C object")
        self.value_C = 2

```

With this change the following code:

```

c_obj = C()
print("=====")
print(c_obj.__dict__)

```

prints

```

Making a C object
=====
{'value_C': 2}

```

We see neither the A initialization nor the B initialization occurred. As with single inheritance, we need to add a call to `super`:

```
class C(A, B):
    def __init__(self) -> None:
        print("Making a C object")
        super().__init__()
        self.value_C = 2
```

but the result is still not quite what we want:

```
Making a C object
Making an A object
=====
{'value_C': 2, 'value_A': 0}
```

The initialization process still omits the B initializer.

The solution is to add a `super` call to *all* classes that are part of the multiple inheritance hierarchy:

```
class A(object):
    def __init__(self) -> None:
        print("Making an A object")
        super().__init__()
        self.value_A = 0

class B(object):
    def __init__(self) -> None:
        print("Making a B object")
        super().__init__()
        self.value_B = 1

class C(A, B):
    def __init__(self) -> None:
        print("Making a C object")
        super().__init__()
        self.value_C = 2
```

Now when we create a C object, as in

```
c_obj = C()
print("=====")
print(c_obj.__dict__)
```

we finally achieve the desired behavior:

```
Making a C object
Making an A object
Making a B object
=====
{'value_B': 1, 'value_C': 2, 'value_A': 0}
```

C objects now have all the expected instance variables and are properly initialized.

The story does not end here, unfortunately. None of the initializers in our A, B, or C classes specify any parameters. What if one superclass contains an `__init__` method that expects certain parameters, while another superclass specifies a different number and/or different types of parameters? If we change class B's definition to be

```
class B(object):
    def __init__(self, val) -> None:
        print("Making a B object")
        super().__init__()
        self.value_B = val
```

and leave the definitions for class A and class C untouched, an attempt to create a C instance produces the following exception:

```
Making a C object
Making an A object
Traceback (most recent call last):
  File "C:\Code\python\multipleinherit1.py", line 19, in <module>
    c = C()
      ^^^
  File "C:\Code\python\multipleinherit1.py", line 16, in __init__
    super().__init__()
  File "C:\Code\python\multipleinherit1.py", line 4, in __init__
    super().__init__()
TypeError: B.__init__() missing 1 required positional argument: 'val'
```

A's initializer expects no arguments, while B's initializer now requires a parameter to assign to its instance variable. How can `super().__init__()` be called once and support passing different parameters to different superclass initializers?

Python provides a couple of ways to address this dilemma. One is a “quick-and-dirty” fix, and the other is the preferred approach. The quick-and-dirty fix works like this:

1. Remove the `super` calls from the initializers in classes A and B. Without removing these calls the quick-and-dirty fix will fail in a manner similar to before.
2. Rewrite class C's definition as follows:

```
class C(A, B):
    def __init__(self) -> None:
        print("Making a C object")
        A.__init__(self)
        B.__init__(self, 5)
        self.value_C = 2
```

This approach explicitly calls the initializers for classes A and B directly.

This quick-and-dirty fix achieves the desired behavior in this case, but it makes for a far less flexible design. If we add another superclass to class C's list of superclasses, we must modify the code in C's initializer to work properly with the initializer of the additional superclass. Also, by removing the `super().__init__()` calls in A and B, we render both classes unable to work properly if we want to involve them in a better designed multiple inheritance hierarchy of classes.

The better design is more flexible and opens the door to more powerful programming techniques than would be possible with the quick-and-dirty approach. We can call multiple superclass initializers that expect a variety of different arguments by using *keyword arguments*, introduced in Section 11.7. The following points outline a working design for object construction within the context of multiple inheritance:

1. Define one dummy class that inherits directly from the built-in `object` class. Its name can be any valid identifier; here we will refer to it as `Top`. We will derive directly or indirectly all other classes in the hierarchy from `Top`. This class should contain an initializer as described below.
2. Every class we add to the multiple inheritance hierarchy should have an initializer with a definition that begins as follows:

```
def __init__(self, **kwargs) -> None:
```

(There is nothing special about the name `kwargs`, and any valid identifier is acceptable, but the convention is to use `kwargs` to indicate keyword arguments in a function or method definition.) This allows the initializer to accept any number of any types of arguments, but it requires clients to use keyword arguments when creating an instance of the class.

3. The following statement should appear in the body of every initializer, except for the `Top` class:

```
super().__init__(**kwargs)
```

This statement invokes the initializer for the next class in the inheritance hierarchy, where the exact meaning of *next class* will appear later. The `Top` initializer does **not** call `super` and, thus, ensures the chain of initializer calls terminates.

4. The initializer for each class should extract the arguments it needs, if any, from the keyword arguments parameter.
5. Clients must supply all the keyword arguments that any initializers in the call chain of superclasses expect.

Listing 14.19 (`simplemultinherit.py`) is very simple example that illustrates the general form of managing class construction in a multiple inheritance hierarchy.

Listing 14.19: `simplemultinherit.py`

```
class Top(object):
    def __init__(self, **kwargs) -> None:
        pass # Terminate the initializer call chain

class A(Top):
    def __init__(self, **kwargs) -> None:
        print("Making an A object")
        super().__init__(**kwargs)
        self.value_A = 0

class B(Top):
    def __init__(self, **kwargs) -> None:
        print("Making a B object")
        self.value_B = kwargs['val']
        kwargs.pop('val') # Remove the parameter from the kwargs
        super().__init__(**kwargs)

class C(A, B):
    def __init__(self, **kwargs) -> None:
        print("Making a C object")
        super().__init__(**kwargs)
        self.value_C = 2
```

```
c_obj = C(val=5)  # Need to supply keyword argument here
print("=====")
print(c_obj.__dict__)
```

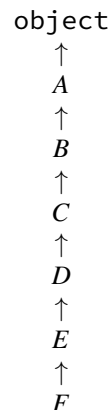
Listing 14.19 (simplemultinherit.py) prints

```
Making a C object
Making an A object
Making a B object
=====
{'value_B': 5, 'value_A': 0, 'value_C': 2}
```

Suppose we have the following single inheritance hierarchy of classes:

```
class A(object):
    def f() -> None:  # A defines method f
        pass
class B(A): pass
class C(B): pass
class D(C):
    def f() -> None:  # D overrides method f
        pass
class E(D): pass
class F(E):
    def f() -> None:  # F overrides method f
        pass
```

which we can visualize as



where the arrow (↑) points from subclass to direct superclass; for example, *E* is a direct subclass of *D*. Also note that classes *A*, *D*, and *F* define methods named *f*. This means class *D* overrides *A*'s *f* method and *F* overrides *D*'s *f* method.

The expression

```
self.f()
```

within any method of class *F* executes the *f* method defined in class *F*. Compare that to the same expression appearing in class *C*. Since class *C* does not define (override) *f*, the executing program attempts to execute

the `f` method in the superclass of `C`; that is, `B`. Class `B` does not define (override) `f`, so the executing program attempts to execute the `f` method in class `A`. Class `A` does provide a definition for method `f`, so the actual code that executes is found in class `A`, even though it is called from a method appearing within class `C`.

Now consider the expression

```
super().f()
```

within a method of class `F`. The executing program first attempts to execute the method `f` found in class `E`. Such a method does not exist, so the search continues to class `D`, which does contain an `f` method. The code within method `D.f` executes.

For single inheritance the name *super* means what it appears to say in the expression `super().f()`: find and execute the `f` method of the closest superclass upward in the inheritance hierarchy that defines an `f` method.

It is tempting to believe that the word *super* refers to superclasses. That always is true for single inheritance, but multiple inheritance clouds the meaning of the *super* function. The order in which an executing program attempts to resolve method calls is known as the *method resolution order*, or *MRO* for short. For single inheritance, the MRO is easy to determine: starting with the given class, if you cannot find the method being called within that class, walk up the hierarchy of classes until you find the nearest class that defines the method. If the method is inherited, it will be defined in a superclass and eventually located. If the method does not exist, the search terminates at class `object` and the interpreter raises an exception.

Every class has an `mro` method that reveals the MRO for that class. The method returns the list of classes in method resolution order. Given the single inheritance hierarchy above, the following code reveals the MRO for class `F`:

```
print(F.mro())
```

The statement prints

```
[<class '__main__.F'>, <class '__main__.E'>, <class '__main__.D'>, <class '__main__.C'>, <class '__main__.B'>, <class '__main__.A'>, <class 'object'>]
```

We can clean up the output considerably using a loop and taking advantage of the fact that every class has a `__name__` attribute that is the string representing the name of the class. The following code

```
for cls in F.mro():
    print(cls.__name__, end=' ')
else:
    print()
```

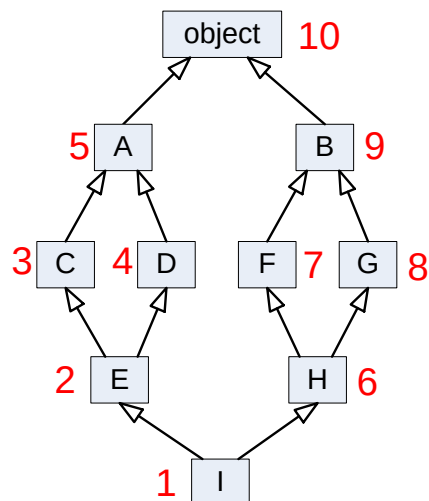
displays

```
F E D C B A object
```

(Recall that the `else` clause associated with a `while` or `for` loop executes when the loop exits normally without a `break` statement—see Section 5.6.)

Observe the ordering from left to right matches the order within the hierarchy from bottom to top. It means calling *super* within a class `D` method, for example, will direct the method search immediately to class `C`.

With multiple inheritance *super* does not always refer to a superclass. The following classes form a relatively complex multiple inheritance relationship:

Figure 14.4 The red numbers indicate the MRO for class I.

```

class A(object): pass
class B(object): pass
class C(A): pass
class D(A): pass
class E(C, D): pass
class F(B): pass
class G(B): pass
class H(F, G): pass
class I(E, H): pass

```

The MRO for class I is

```
I E C D A H F G B object
```

We can better visualize the MRO in Figure 14.4. From the MRO we see that a call to `super` in a method of class C would proceed to class D, and D is not a superclass of C. The name `super`, therefore, does not always refer to a superclass.

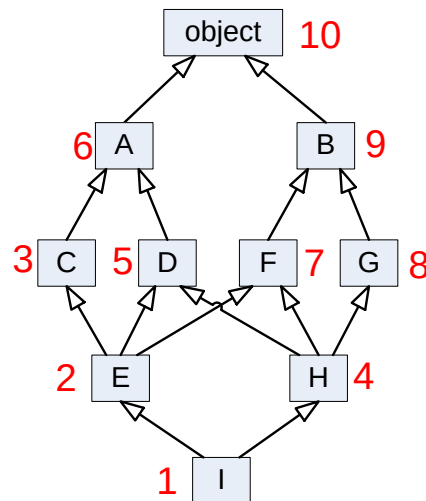
The MRO does make sense. The process is this: keep moving up the hierarchy until moving up one more time would mean visiting a class before one of its subclasses. In the example we can go from I to E to C, but we cannot next go to A since we have yet to visit D, and D is a subclass of A. If we cannot go up, we go sideways to the right. The right movement corresponds to the left-to-right ordering of superclasses in a class definition.

Pictorially, in an MRO you never should visit a class that has an arrow coming into it from a class below that you have not yet visited.

The MRO of class C is

```
C A object
```

Figure 14.5 The red numbers indicate the MRO for class I in the more complicated hierarchy.



Since this begins the MRO from C, super visits C's superclass A first, not D. This is because class C involves no multiple inheritance. The MRO depends on which class the search begins, even within the same hierarchy.

The following code complicates the hierarchy slightly:

```

class A(object): pass
class B(object): pass
class C(A): pass
class D(A): pass
class F(B): pass
class E(C, D, F): pass
class G(B): pass
class H(D, F, G): pass
class I(E, H): pass

```

The MRO for class I for this hierarchy is

```
I E C H D A F G B object
```

Figure 14.5 illustrates the MRO in this more complicated hierarchy. In this case we proceed from I to E to C. The next step up would be A, but we have yet to visit D, a subclass of A. We cannot visit D, however, because we have yet to visit H, a subclass of D. The leaves H as the next class to visit. Remember, we attempt to go up, if possible, so we visit its leftmost superclass, D. After D we visit A, but we cannot go up to object because we have yet to visit F, G, and B. These classes, in order, round out the MRO for I. No class will appear more than once in an MRO.

Armed with our relatively limited theoretical knowledge of multiple inheritance, we now can build a more interesting example. First, Listing 14.20 (top.py) establishes our Top class described earlier. Top allows us to build a cooperative multiple inheritance hierarchy that allows the various class initializers to accept varied arguments.

Listing 14.20: top.py

```
class Top(object):
    def __init__(self, **kwargs) -> None:
        pass # Terminate the initializer call chain
```

For this example we will take a break from Tk graphics and revisit Turtle graphics. Consider Listing 14.21 (graphicalobject.py), a class that represents a graphical object that a user may reposition within a Turtle graphics window.

Listing 14.21: graphicalobject.py

```
from top import Top

class GraphicalObject(Top):
    """ A graphical object that allows the user to reposition
        it anywhere within the window using the mouse.
        Contains all the code to manage the Turtle graphics
        environment. """
    def __init__(self, **kwargs) -> None:
        """ Initializes a GraphicalObject object
            Keyword args include:
                screen is the Turtle graphics screen.
                turtle is the Turtle graphics pen.
                initially appear at location (x, y). """
        super().__init__(**kwargs) # See discussion about super initializer
        print("Initializing graphical object")
        self.screen = kwargs['screen']
        self.turtle = kwargs['turtle']
        self.x = kwargs['x']
        self.y = kwargs['y']
        self.screen.delay(0) # Do not slowly trace drawing
        self.turtle.speed(0) # Make turtle's actions as fast as possible
        self.turtle.hideturtle() # Make the turtle invisible
        self.screen.onclick(self.do_click) # Set mouse press handler
        self.move_to(x=kwargs['x'], y=kwargs['y'])

    def run(self) -> None:
        """ Run the graphical program. """
        self.screen.mainloop()

    def draw(self) -> None:
        """ Renders the object within the window.
            Derived classes override this method to meet their specific
            needs. """
        pass

    def do_click(self, x, y) -> None:
        """ Called when the user presses the mouse button.
            (x, y) is the new location of the display. """
        self.move_to(x, y) # Move to a new location
        self.draw() # Redraw at new location

    def move_to(self, x, y) -> None:
        """ Relocates a Movable object to position (x, y). """
```

```
self.x, self.y = x, y
```

The line

```
class GraphicalObject(Top):
```

indicates the superclass of `GraphicalObject` is our `Top` class from before. Note the beginning of the initializer definition:

```
def __init__(self, **kwargs) -> None:
```

In the `GraphicalObject` initializer we explicitly invoke the superclass initializer via `super`:

```
super().__init__(**kwargs)
```

Every class in the multiple inheritance hierarchy, except those derived directly from `object`, must call `__init__` via `super` and pass keyword arguments. This form of the initializer invocation with `super` makes cooperative multiple inheritance possible. It enables the initializers for other classes in the hierarchy to accept different numbers of parameters. The price to pay for this versatility is this: clients must use keyword arguments for passing all parameters to class initializers.

Notice that the `draw` method is empty. This is because the `GraphicalObject` class is an abstract notion, and it is up to derived classes to specify the exact behavior needed to draw the object.

Listing 14.22 (`dot.py`) derives the `Dot` class from `GraphicalObject` and overrides the `draw` method to simply draw a red circle within the window.

Listing 14.22: `dot.py`

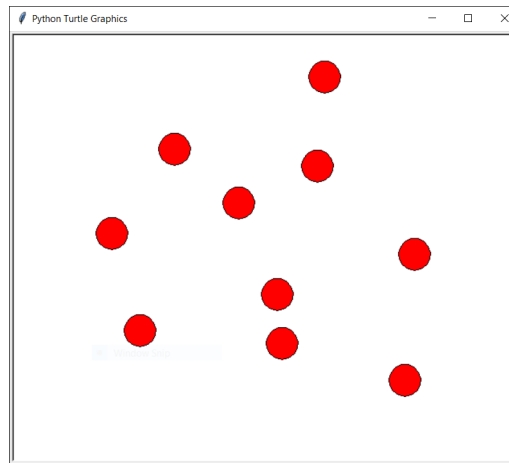
```
from graphicalobject import GraphicalObject

class Dot(GraphicalObject):
    """ A simple, round circle graphical object """
    def __init__(self, **kwargs) -> None:
        """ Initializes a dot object with a given Turtle screen, pen,
            and (x, y) position """
        super().__init__(**kwargs)

    def draw(self) -> None:
        """ Renders the dot in the Turtle graphics window """
        self.turtle.penup()      # Move pen
        self.turtle.setpos(self.x, self.y)
        self.turtle.pendown()
        self.turtle.fillcolor("red")
        self.turtle.begin_fill()
        self.turtle.circle(20)
        self.turtle.end_fill()
```

Listing 14.23 (`testdot.py`) exercises the `Dot` class.

Figure 14.6 An image of one execution of Listing 14.23 (testdot.py) after the user has clicked the mouse in various locations.



Listing 14.23: testdot.py

```
from dot import Dot
from turtle import Screen, Turtle

def main() -> None:
    # Make a Dot object
    d = Dot(screen=Screen(), turtle=Turtle(), x=10, y=20)
    # Run the graphical program
    d.run()

if __name__ == "__main__":
    main()
```

Figure 14.6 shows the state of an execution of the Listing 14.23 (testdot.py) after the user has clicked the mouse in various locations. The `draw` method does not clear the window, so each mouse click draws a new dot image in a different position without erasing it in its previous spot.

Suppose we want to combine the functionality of our nongraphical digital stopwatch (Listing 14.9 (digitalstopwatch.py)) with capability of a graphical object. The user should be able to click a location within the graphical window, and the digital stopwatch's display will appear at that point. The user still should be able to start, stop, and reset the stopwatch as desired. We also would like to leverage existing code as much as possible. One way we can do this is via multiple inheritance.

Listing 14.24 (timekeeping.py) updates Listing 13.9 (stopwatch.py) to better support multiple inheritance. The only changes are the use of `super` in the initializer and a print message to ensure the initializer executes as expected.

Listing 14.24: timekeeping.py

```

from time import perf_counter_ns
from top import Top

class Timer(Top):
    """ Provides stopwatch objects that that programmers
        can use to time the execution time of portions of
        a program. """
    def __init__(self, **kwargs) -> None:
        """ Makes a new stopwatch ready for timing. """
        super().__init__(**kwargs)      # New code here
        print("Initializing stopwatch")  # New code here
        self.reset()

    def start(self) -> None:
        """ Starts the stopwatch, unless it is already running.
            This method does not affect any time that may have
            already accumulated on the stopwatch. """
        if not self._running:
            self._start_time = perf_counter_ns() - self._elapsed
            self._running = True      # Clock now running

    def stop(self) -> None:
        """ Stops the stopwatch, unless it is not running.
            Updates the accumulated elapsed time. """
        if self._running:
            self._elapsed = perf_counter_ns() - self._start_time
            self._running = False     # Clock stopped

    def reset(self) -> None:
        """ Resets stopwatch to zero. """
        self._start_time = self._elapsed = 0
        self._running = False

    def elapsed(self) -> float:
        """ Reveals the stopwatch running time since it
            was last reset. """
        if not self._running:
            return self._elapsed/1_000_000_000
        else:
            return (perf_counter_ns() - self._start_time)/1_000_000_000

```

Listing 14.25 (digitaltimer.py) updates Listing 14.9 (digitalstopwatch.py), providing a initializer.

Listing 14.25: digitaltimer.py

```

from timekeeping import Timer

class DigitalTimer(Timer):
    def __init__(self, **kwargs) -> None:                # New code here
        super().__init__(**kwargs)                      # New code here
        print("Initializing digital stopwatch")          # New code here

    def digital_time(self) -> None:

```

```

    """ Returns a string representation of the elapsed time
        in hours : minutes : seconds. """
    # Compute time in hours, minutes, and seconds
    seconds = round(self.elapsed())
    hours = seconds // 3600      # Compute total hours
    seconds %= 3600             # Update seconds remaining
    minutes = seconds // 60     # Compute minutes
    seconds %= 60               # Update seconds remaining
    # Each digit occupies two spaces; pad with leading zeros, if necessary
    return f'{hours:0>2}:{minutes:0>2}:{seconds:0>2}'

```

Listing 14.26 (movabletimer.py) defines a class named `MovableTimer` that inherits from both `DigitalTimer` and `GraphicalObject`.

Listing 14.26: movabletimer.py

```

from digitaltimer import DigitalTimer
from graphicalobject import GraphicalObject
from turtle import Screen, Turtle

#class MovableTimer(DigitalTimer, GraphicalObject):
class MovableTimer(GraphicalObject, DigitalTimer):
    """ A graphical digital timer that allows the user to reposition
        its display anywhere within the window using the mouse.
        Contains all the code to manage the Turtle graphics
        environment. """
    def __init__(self, **kwargs) -> None:
        """ Sets the graphical environment and initial position of the
            display.
            (x, y) is the display's initial position """
        super().__init__(**kwargs)
        print("Initializing MovableTimer")
        self.screen.listen()          # Receive keypress events
        self.screen.onkey(self.start_timer, "s") # Key to start the timer
        self.screen.onkey(self.stop_timer, "t")  # Key to stop the timer
        self.screen.onkey(self.reset_timer, "r") # Key to reset the timer
        self.screen.ontimer(self.update, 500)    # Set timer event

    def draw(self) -> None:
        """ Renders the digital stopwatch within the window. """
        self.turtle.clear()          # Clear the screen
        self.turtle.penup()          # Move pen
        self.turtle.setpos(self.x, self.y)
        self.turtle.pendown()
        self.turtle.write(self.digital_time(), font=("Arial", 48, "normal"))

    def start_timer(self) -> None:
        """ Starts the timer. """
        self.start()

    def stop_timer(self) -> None:
        """ Stops the timer. """
        self.stop()

```



```

def reset_timer(self) -> None:
    """ Resets the timer to 0:00:00. """
    self.reset()

def update(self) -> None:
    """ Updates the program's view of the global stopwatch object.
        Called every one-half second. """
    self.draw()          # Draw the digital display
    # Call update again after one-half second
    self.screen.ontimer(self.update, 500)

def main() -> None:
    clock = MovableTimer(screen=Screen(), turtle=Turtle(), x=10, y=20)
    clock.run()

if __name__ == "__main__":
    main()

```

Observe that, unlike `Dot` objects, `MovableTimer` objects erase themselves when the user moves them. This means that there always will be only one stopwatch display appearing within the window at a time.

The multiple inheritance in this case means that a `MovableTimer` object *is a* `DigitalTimer` instance and *is a* `GraphicalObject` instance, as the following interactive session confirms:

```

>>> from graphicalobject import *
>>> from movabletimer import *
>>> from turtle import *
>>> x = MovableTimer(screen=Screen(), turtle=Turtle(), x=30, y=45)
Initializing stopwatch
Initializing digital stopwatch
Initializing graphical object
Initializing MovableTimer
>>> type(x)
<class 'movabletimer.MovableTimer'>
>>> isinstance(x, MovableTimer)
True
>>> isinstance(x, GraphicalObject)
True
>>> isinstance(x, DigitalTimer)
True
>>> isinstance(x, int)
False

```

This means that any function or method that works with a `GraphicalObject` as a parameter should work correctly if passed a `MovableTimer` instance. Similarly, any function or method that expects a `DigitalStopwatch` argument will work properly if passed a `MovableTimer` object.

Since Python supports cooperative multiple inheritance, how can we deal with a class that does not cooperate? Suppose we have a class from a library that we do not control; that is, we cannot change the source code of the class. Specifically, consider Listing 14.27 (`quad.py`).

Listing 14.27: quad.py

```
import turtle

class Quad:
    """ A square object """
    def __init__(self, screen: turtle._Screen, pen: turtle.Turtle,
                  x: int, y: int, width: int) -> None:
        """ Initializes a box object with a given Turtle screen, pen,
            (x, y) position, and width """
        self._screen = screen
        self._pen = pen
        self._x_val = x
        self._y_val = y
        self._width = width

    def position(self, x, y) -> None:
        """ Positions the square at (x,y) """
        self._x_val = x
        self._y_val = y

    def render(self) -> None:
        """ Renders the square in the Turtle graphics window """
        self._pen.penup() # Move pen
        self._pen.setpos(self._x_val - self._width/2,
                          self._y_val - self._width/2)
        self._pen.setheading(0)
        self._pen.pendown()
        self._pen.fillcolor("blue")
        self._pen.begin_fill()
        self._pen.forward(self._width)
        self._pen.left(90)
        self._pen.forward(self._width)
        self._pen.left(90)
        self._pen.forward(self._width)
        self._pen.left(90)
        self._pen.forward(self._width)
        self._pen.left(90)
        self._pen.end_fill()
```

The Quad class is reminiscent of the Dot class from Listing 14.22 (dot.py). The Quad class, however, is not a subclass of GraphicalObject. The initializer for Quad does not use the expected keyword arguments, and it does not have a draw method. Clearly, the Quad class does not cooperate with the classes in our GraphicalObject multiple inheritance hierarchy.

Fortunately there is a relatively easy way to make use of the capabilities the Quad class provides within our hierarchy. Section 14.3 introduced the adapter design pattern. We can create a new class that adapts Quad to meet our needs. Listing 14.28 (quadadapter.py) does exactly that.

Listing 14.28: quadadapter.py

```

from graphicalobject import GraphicalObject
from quad import Quad

class QuadAdapter(GraphicalObject):
    """ A simple, square graphical object.
        QuadAdapter is a GraphicalObject.
        QuadAdapter has a Quad. """
    def __init__(self, **kwargs) -> None:
        """ Initializes an adapted Quad object with a given Turtle
            screen, pen, and (x, y) position """
        self.box = Quad(kwargs['screen'], kwargs['turtle'],
                        kwargs['x'], kwargs['y'], kwargs['width'])
        super().__init__(**kwargs)

    def move_to(self, x, y) -> None:
        """ Repositions the quad to (x,y) """
        self.box.position(x, y)

    def draw(self) -> None:
        """ Renders the quad in the Turtle graphics window """
        self.box.render()

```

Every QuadAdapter object contains a Quad object. The QuadAdapter methods delegate their work to the equivalent methods in the Quad class. The initializer expects keyword arguments and calls `super` with keyword arguments, exactly as every good `GraphicalObject` subclass initializer should. The initializer unpacks the keyword arguments for Quad's initializer. The QuadAdapter class inherits `run` and `do_click` from `GraphicalObject`.

Listing 14.29 (`testquad.py`) demonstrates that our new QuadAdapter class is fully cooperative.

Listing 14.29: testquad.py

```

from quadadapter import QuadAdapter
from turtle import Screen, Turtle

def main() -> None:
    # Make a Quad object via the adapter
    b = QuadAdapter(screen=Screen(), turtle=Turtle(), x=20, y=20, width=30)
    # Run the graphical program
    b.run()

if __name__ == "__main__":
    main()

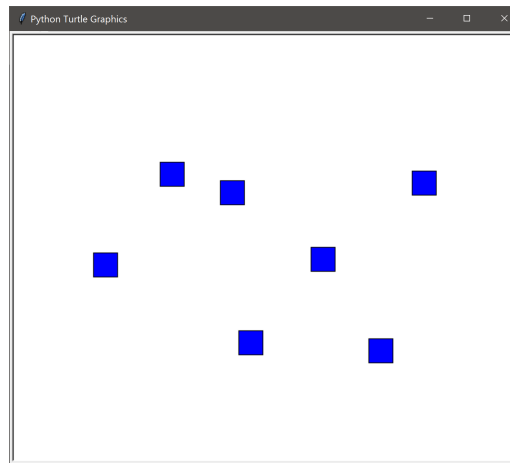
```

Figure 14.7 shows the state of an execution of Listing 14.29 (`testquad.py`) after the user has clicked the mouse in various locations.

As an aside, the type initializer enables dynamic class definitions. The following statement:

```
MyClass = type("MyClass", (object,), {})
```

Figure 14.7 An image of one execution of Listing 14.29 (testquad.py) after the user has clicked the mouse in various locations.



defines an empty class named `MyClass`. Note that this assignment statement is equivalent to the following source code:

```
class MyClass(object):
    pass
```

The second argument to the type initializer is a tuple of base classes. The expression `(object,)` represents a tuple that contains one element; the following interactive session shows why the trailing comma is necessary:

```
>>> x = (1)
>>> x
1
>>> y = (1,)
>>> y
(1,)
>>> y[0]
1
>>> x[0]
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: 'int' object is not subscriptable
```

Here we see that the expression `(1)` is just the integer 1, and the expression `(1,)` represents the tuple that contains the integer 1.

The third argument to the type initializer is a dictionary that establishes class variables for the class (not instance variables for instances of the class, see Section 13.8).

As with dynamically added instance variables (see Section 13.7), the ability to define classes on the fly is valuable but rarely needed in most applications.

14.5 Unit Testing

We created a rudimentary testing class in Section 13.5. Python provides a considerably more powerful testing framework in its `unittest` module. *Unit testing* is a well-established technique for evaluating the correctness of software components (see https://en.wikipedia.org/wiki/Unit_testing). Unit testing involves testing individual units of a software system in isolation to determine if they behave as advertised. Examples of individual software units include functions, methods, objects, and modules.

Python's `unittest` module is easy to use. The following steps explain how to use the unit testing module for simple testing:

1. Import the `unittest` module.
2. Import the code to test as necessary.
3. Derive a class from `unittest.TestCase`.
4. Add methods for each test to perform.
 - Each method's name must have the prefix `test`.
 - Each method must accept only the `self` parameter.
 - Add to the method's body any code required to perform the test.
 - Within each test method, use the `assertEqual` method to compare a computed value to an expected value.
5. Add the following code to the test program:

```
if __name__ == '__main__':  
    unittest.main()
```

Listing 13.16 (`testliststuff.py`) in Section 13.5 uses our home-grown `FunctionTester` object to test some functions. Listing 14.30 (`testlist2.py`) uses Python's standard `unittest` module to test the same functions in

Listing 14.30: `testlist2.py`

```
import unittest  
  
##### The original functions from before #####  
  
def max_of_three_bad(x: int, y: int, z: int) -> int:  
    """ Attempts to determine and return the maximum of three  
        numeric values. """  
    result = x  
    if y > result:  
        result = y  
    elif z > result:  
        result = z  
    return result  
  
def max_of_three_good(x: int, y: int, z: int) -> int:  
    """ Computes and returns the maximum of three numeric values. """
```

```

    result = x
    if y > result:
        result = y
    if z > result:
        result = z
    return result

def sum(lst: list[int]) -> int:
    """ Attempts to compute and return the sum of all the elements in
        a list of integers. """
    total = 0
    for i in range(1, len(lst)):
        total += lst[i]
    return total

# maximum has a bug (it has yet to be written!)
def maximum(lst: list[int]) -> int:
    """ Computes the maximum element in a list of integers. """
    return 0 # maximum not yet implemented

##### Some new code to test #####

class ListManager(object):
    def __init__(self, lst: list[int]) -> None:
        super().__init__()
        self.lst = lst

    def get(self, idx: int):
        return self.lst[idx] # Subject to range exception

##### The test class #####

class TestFunctionsEtc(unittest.TestCase):
    # Some test cases to test max_of_three_bad
    def test_max_of_three_bad_1(self) -> None:
        self.assertEqual(max_of_three_bad(2, 3, 4), 4)

    def test_max_of_three_bad_2(self) -> None:
        self.assertEqual(max_of_three_bad(4, 3, 2), 4)

    def test_max_of_three_bad_3(self) -> None:
        self.assertEqual(max_of_three_bad(3, 2, 4), 4)

    # Some test cases to test max_of_three_good
    def test_max_of_three_good_1(self) -> None:
        self.assertEqual(max_of_three_bad(2, 3, 4), 4)

    def test_max_of_three_good_2(self) -> None:
        self.assertEqual(max_of_three_bad(4, 3, 2), 4)

    def test_max_of_three_good_3(self) -> None:
        self.assertEqual(max_of_three_bad(3, 2, 4), 4)

```

```

# Some test cases to test maximum
def test_maximum_1(self) -> None:
    self.assertEqual(maximum([2, 3, 4, 1]), 4)

def test_maximum_2(self) -> None:
    self.assertEqual(maximum([4, 3, 2, 1]), 4)

def test_maximum_3(self) -> None:
    self.assertEqual(maximum([-2, -3, 0, -21]), 0)

# Some test cases to test sum
def test_sum_1(self) -> None:
    self.assertEqual(sum([0, 3, 4]), 7)

def test_sum_2(self) -> None:
    self.assertEqual(sum([-3, 0, 5]), 2)

# Some code that can throw an exception
def test_list_manager_1(self) -> None:
    lm = ListManager([1, 2, 3])
    self.assertEqual(lm.get(2), 3)

def test_list_manager_2(self) -> None:
    lm = ListManager([1, 2, 3])
    self.assertEqual(lm.get(3), 3)

def test_list_manager_3(self) -> None:
    lm = ListManager([1, 2, 3])
    self.assertEqual(lm.get(0), 1)

def test_list_manager_4(self) -> None:
    lm = ListManager([1, 2, 3])
    self.assertEqual(lm.get(0), 0)

if __name__ == '__main__':
    unittest.main()

```

Notice that the class `TestFunctionsEtc` inherits from `unittest.TestCase`. This means our `TestFunctionsEtc` class will have all the capabilities required to work properly in Python's unit testing framework. All the methods within the `TestFunctionsEtc` class that begin with the prefix `test` represent tests to perform. All the methods in the `TestFunctionsEtc` class happen to begin with `test`, but test classes in general can include other, non-test, methods as well. These can be helper methods that the test methods invoke.

Within each test method a statement such as

```
self.assertEqual(lm.get(0), 0)
```

compares the value of the first argument of `assertEqual` to the second argument. The first argument is the *actual* value, and the second argument is the *expected* value. The `assertEqual` method checks to see if they match. If the actual and expected value match, the test passes. If the two values do not match, the test fails. If attempting to evaluate the actual value produces an exception, the test is classified as an error.

The output of Listing 14.30 (testlist2.py) begins with the line

```
.E.FF..F..FF..F
```

This line contains 15 characters—exactly the number of methods named with the prefix `test` within our `TestFunctionsEtc` class. A dot (.) indicates a test that passed, an F represents a failed test, and an E indicates the test produced an exception (run-time error). An important point is that the test framework does **not** necessarily execute the tests in the same order that the test methods appear in the source code. The test execution order is alphabetical (actually lexicographic—the same ordering that the < operator imposes on Python string objects) by method name.

The rest of the program's output provides details about each test failure or error. The next several lines of output include

```
.E.FF..F..FF..F
=====
ERROR: test_list_manager_2 (__main__.TestFunctionsEtc.test_list_manager_2)
-----
Traceback (most recent call last):
  File "C:\Code\python\testlist2.py", line 98, in test_list_manager_2
    self.assertEqual(lm.get(3), 3)
    ^^^^^^^^^^^^^
  File "C:\Code\python\testlist2.py", line 48, in get
    return self.lst[idx] # Subject to range exception
           ~~~~~^~~~~~
IndexError: list index out of range

=====
FAIL: test_list_manager_4 (__main__.TestFunctionsEtc.test_list_manager_4)
-----
Traceback (most recent call last):
  File "C:\Code\python\testlist2.py", line 106, in test_list_manager_4
    self.assertEqual(lm.get(0), 0)
AssertionError: 1 != 0

=====
FAIL: test_max_of_three_bad_1 (__main__.TestFunctionsEtc.test_max_of_three_ba
-----
Traceback (most recent call last):
  File "C:\Code\python\testlist2.py", line 56, in test_max_of_three_bad_1
    self.assertEqual(max_of_three_bad(2, 3, 4), 4)
AssertionError: 3 != 4
```

After detailing several more test failures, the program's output concludes with

```
-----
Ran 15 tests in 0.004s

FAILED (failures=6, errors=1)
```

Here we see that out of 15 tests only eight tests passed, and seven did not pass. One of the seven tests did not pass due to a run-time exception.

The `unittest` package includes a family of over 30 methods related to `assertEqual`. These methods all have the prefix `assert` and include such tests as `assertTrue`, `assertFalse`, `assertRaises`,

`assertNotEqual`, `assertLess`, `assertLessEqual`, `assertListEqual`, `assertAlmostEqual`, `assertIs`, `assertIn`, and `assertNotIn`.

In addition to the large variety of `assert` methods, the `TestCase` class provides `setUp` and `tearDown` methods. If we override these methods in our testing class, the testing framework automatically will execute the `setUp` method before each test method runs and automatically execute the `tearDown` method after each test method executes (even if the test method raises an exception). To demonstrate how this works, add the two methods shown below to Listing 14.30 (`testlist2.py`) and see how they affect its output.

```
class TestFunctionsEtc(unittest.TestCase):
    def setUp(self) -> None:
        print("Setting up ...")

    def tearDown(self) -> None:
        print("... tearing down.")

    # Other parts of class remain the same . . .
```

The infrastructure that executes the tests is known as the *test runner*. We see the work of the test runner in the program’s output when run in the console. Graphical IDEs can visualize the test results in other ways, such as using green and red bars to indicate tests successes and failures.

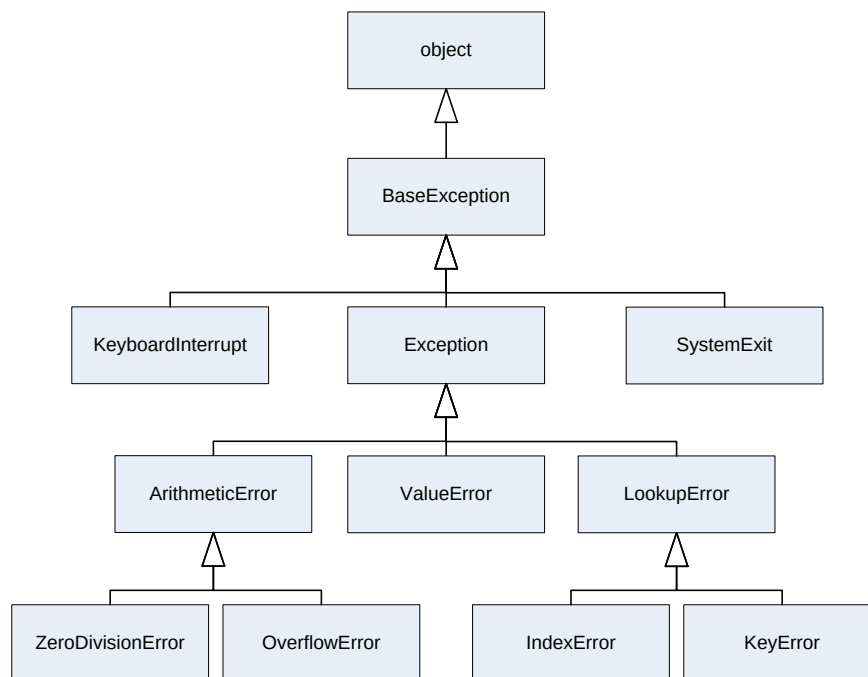
14.6 Custom Exceptions

Chapter 12 introduced Python’s exception handling infrastructure. Now that we are familiar with inheritance we can understand better how it works and even create our own custom exception types. Consider the following brief interactive session:

```
>>> v = ValueError()
>>> v
ValueError()
>>> isinstance(v, ValueError)
True
>>> isinstance(v, TypeError)
False
>>> isinstance(v, Exception)
True
```

We can see that `ValueError` is a subclass of `Exception`. Figure 14.8 shows a portion of Python’s class hierarchy of standard exceptions. The diagram omits `GeneratorExit`, which is a direct subclass of `BaseException`. It also omits 56 other standard classes derived directly or indirectly from `Exception`. Almost every standard exception class is a direct or indirect subclass of `Exception`. Among the standard exception classes, only `BaseException`, `KeyboardInterrupt`, `SystemExit`, and `GeneratorExit` are not related via inheritance to the `Exception` class. Due to the *is a* relationship imposed by inheritance, any exception object of a class other than the four excluded types mentioned above is compatible with the `Exception` type. This explains why we use `Exception` as the “catch-all” exception type used in an exception handler (see Section 12.5). The reason that we do not usually use `BaseException` as the catch-all type is we normally do not want to intervene when the user presses **Ctrl C** to exit the program or when the program itself calls `sys.exit`. The `Exception` type covers all the exceptions that we ordinarily care to handle.

Figure 14.8 A small portion of the class hierarchy of Python’s standard exceptions. This diagram omits 1 other standard class derived directly from `BaseException` and 56 other classes derived directly or indirectly from `Exception`.



Even though Python provides 68 standard exception classes, we may not find one that exactly meets our needs, especially now that we can define our own custom types via classes. Inheritance makes it easy to define our own custom exception types. Listing 13.9 (stopwatch.py) defines a simple stopwatch timer class. Suppose we wish to consider an attempt to stop a nonrunning stopwatch an error worthy of an exception. We could reuse a standard exception, but which one? The following shows an attempt with `ValueError`:

```
class Stopwatch:
    # Other details about the class omitted ...
    def stop(self) -> None:
        """ Stops the stopwatch, unless it is not running.
            Updates the accumulated elapsed time. """
        if self._running:
            self._elapsed = perf_counter_ns() - self._start_time
            self._running = False # Clock stopped
        else:
            raise ValueError("Attempt to stop a stopped clock")
```

As desired, this revised `Stopwatch.stop` method will raise an exception when a client attempts to stop a stopped stopwatch, but is this the best approach?

We can use `help` function (see Section 6.3) in the interactive interpreter to get information about `ValueError`; the first few lines it prints is

```
>>> help(ValueError)
Help on class ValueError in module builtins:

class ValueError(Exception)
|   Inappropriate argument value (of correct type).
|
|   Method resolution order:
|       ValueError
|       Exception
|       BaseException
|       object
|
```

(The remainder of `help`'s output provides information about `ValueError`'s methods.) Notice that `help` lists `ValueError`'s MRO (see Section 14.4), but, more importantly, it gives the meaning of the `ValueError` exception. `ValueError` is supposed to indicate an “inappropriate argument value.” This meaning does not match well with an attempt to stop a stopped stopwatch.

Another problem is this: What if use our `Stopwatch` class to time code that can produce its own `ValueError` exception? How can any exception handling code we might write in this situation distinguish between an error with a `Stopwatch` object and a legitimate `ValueError` that the other code may raise?

What we need is a `StopwatchError` class. Making such a class turns out to be remarkably easy; the following class meets the requirement:

```
class StopwatchError(Exception):
    """ Attempt to stop a nonrunning stopwatch """
    pass
```

The `StopwatchError` class inherits from `Exception`, but its empty class body means that `StopwatchError` adds nothing to what the `Exception` class already offers. The value that `StopwatchError` adds is this:

it defines a new exception type that can participate as a first-class citizen in Python’s exception handling infrastructure. We can rewrite the `Stopwatch.stop` method as

```
class Stopwatch:
    # Other details about the class omitted ...
    def stop(self) -> None:
        """ Stops the stopwatch, unless it is not running.
            Updates the accumulated elapsed time. """
        if self._running:
            self._elapsed = perf_counter_ns() - self._start_time
            self._running = False # Clock stopped
        else:
            raise StopwatchError()
```

Exception handling code now can distinguish between a stopwatch error and a true `ValueError`:

```
try:
    # Some code that may raise a StopwatchError or ValueError
    # exception
except ValueError:
    pass # Add code to process ValueError
except StopwatchError:
    pass # Add code to process StopwatchError
except Exception:
    pass # Add code to process all other normal exceptions
```

Because we derived `StopwatchError` from `Exception`, a “catch-all” `except` block will catch a `StopwatchError` object even if a `try` statement lacks an explicit `StopwatchError`-specific `except` block.

14.7 Exercises

1. What two major ways does Python provide for reusing classes during class design?
2. Which kind of relationship amongst classes does composition represent, *has is*, *is a*, or something else?
3. Which kind of relationship amongst classes does inheritance represent, *has is*, *is a*, or something else?
4. What is maximum of superclasses a class can have using single inheritance?
5. Some physical stopwatches allow their users to store lap times up to a certain number. Use the `Stopwatch` class from Listing 13.9 (`stopwatch.py`) and composition to design a new stopwatch class that includes a lap timer. Your class should provide a `lap` method that works as follows:
 - A newly created stopwatch object has no lap times stored in its memory.
 - Invoking the `reset` method clears all lap times from a stopwatch’s memory.
 - Invoking the `lap` method on a nonrunning stopwatch has no effect.
 - Invoking the `lap` method on a running stopwatch with no stored lap times records the elapsed time since the invocation of `start`.

- Invoking the `lap` method on a running stopwatch with stored lap times records the elapsed time since the most recent invocation of `lap`.

Your class also should provide a method named `lap_times` that returns a list stored lap times. It should return the empty list if the stopwatch has no stored lap times.

6. Implement the stopwatch with lap timer from Question 5, but use inheritance instead of composition.
7. Consider the following class `X`:

```
class X(object):  
    def identify(self) -> None:  
        print("I'm an X object")
```

Chapter 15

Algorithm Quality

The previous chapters emphasized the mechanics of the Python programming language. We have seen how to manage variables, arithmetic, conditional execution, iteration, functions, parameters, objects, lists, tuples, dictionaries, sets, exceptions, custom types, and inheritance. Our main concern has been using these features to construct programs that correctly implement algorithms to solve problems. Sometimes correct algorithms are subtly difficult to get right, and their logic errors can evade even careful testing.

Program correctness always is the primary goal of software construction, but correctness is not the only goal. Two different programs may produce the exact same results in all cases, and yet one objectively may be considered better than the other. This difference in quality has nothing to do with source code style issues, variable names, or the code's apparent complexity. The difference is this: Despite the two programs producing the same results, one program effectively works and the other does not! It turns out that it is not hard to devise and implement an algorithm that correctly solves a problem but takes too much time to complete its work. The program, therefore, does not meet the user's needs, as the user cannot wait long enough for the result.

In this chapter we examine several different kinds of algorithms, each of which deal with practical problems, and along the way we address correctness and efficiency concerns.

15.1 Good Algorithms Versus Bad Algorithms

Suppose we wish to determine if the elements in a list (Chapter 10) of integers appear in ascending order. (*Non-decreasing order* is the more precise term, as it allows for duplicate elements.) The following function correctly solves the problem:

```
def is_ascending(lst: list[int]) -> bool:
    for i in range(len(lst) - 1):
        for j in range(i + 1, len(lst)):
            if lst[i] > lst[j]:
                return False
    return True
```

The nested loop first compares the first (0th index) element to all the elements that follow. If the first element is indeed smaller than all the others, it continues by comparing the second element to all of the elements that follow the second element. The third time through the outer loop the function compares the third element

to all the elements that follow the third element. If at any time the inner loop finds an element smaller than the element at the index managed by the outer loop, the function immediately returns `False`. On the other hand, if the elements in the list are in ascending order, the nested loop must continue until it finally compares the next-to-the-last element to the last element. At that point the function returns `True`.

This `is_ascending` function is correct. If the first element is less than all the elements that follow the first element, and the second element is less than all the elements that follow the second element, and the third element is less than all the elements that follow the third element, etc., all the elements must be arranged in non-decreasing order.

Next, consider the following competing algorithm:

```
def is_ascending2(lst: list[int]) -> bool:
    for i in range(len(lst) - 1):
        if lst[i] > lst[i + 1]:
            return False
    return True
```

The `is_ascending2` function uses the mathematical principle of *transitivity*. The transitive property of inequality for integers is this: If $x \leq y$ and $y \leq z$, then $x \leq z$. The `is_ascending2` function compares the first element to the second element, the second element to the third, the third to the fourth, etc., until it finally compares the next to the last element to the last element. If the function detects any element out of order, it returns `False` immediately. If it makes it all the way to the end of the list, the function returns `True`. Because of transitivity, if the loop gets to the end of the list, we know that the first element is no larger than all the follow it; we need not compare the first element directly with each and every element that follows it.

Both the `is_ascending` and `is_ascending2` functions return the same result when presented with the same list. We say that the two functions are *functionally equivalent*. If correctness is our only criterion, neither function is better than the other.

The `is_ascending2` function is simpler than the `is_ascending` function because it uses a single loop rather than a nested loop. Apparent code complexity by itself is not a reliable criterion for judging the quality of an algorithm. We must determine which function computes its result quicker. Sometimes more complex code is faster than simpler code because the more complex code employs special tricks to speed up its processing.

Suppose we need to assess the ordering of the elements in a list containing n integers. To better evaluate the two functions we will consider using a list with its elements arranged in non-decreasing order. This prevents either function from returning early—both must make a full scan over all the elements in the list before returning `True`. The following compares the two functions:

- **`is_ascending`.** In the case of `is_ascending`, the outer loop must iterate $n - 1$ times. The inner loop scans $n - 1$ elements within the first iteration of the outer loop. During each iteration of the outer loop the inner loops scans one fewer element than it did on the previous outer loop iteration. This means the inner loop iterates $n - 1$ times on the first iteration of the outer loop, $n - 2$ times on the second iteration of the outer loop, $n - 3$ times on the third iteration of the outer loop, etc. As a result the `if` statement executes

$$(n - 1) + (n - 2) + (n - 3) + \dots + (n - [n - 3]) + (n - [n - 2]) + (n - [n - 1])$$

times. Observe that this expression simplifies to

$$(n - 1) + (n - 2) + (n - 3) + \dots + 3 + 2 + 1$$

Exactly how many times is this, in terms of the size of the list n ? We can use some algebra to simplify the above expression. Let

$$s = (n-1) + (n-2) + (n-3) + \dots + 3 + 2 + 1$$

Because of the commutative property of addition it does not matter which direction we perform the addition. We can just as well write the right side of the equation backwards as

$$s = 1 + 2 + 3 + \dots + (n-3) + (n-2) + (n-1)$$

A fact from algebra states that adding two equations results in an equation. If we add the forwards and backwards equations together, we get

$$\begin{array}{cccccccccccccccc} s & = & (n-1) & + & (n-2) & + & (n-3) & + & \dots & + & 3 & + & 2 & + & 1 \\ + & s & = & 1 & + & 2 & + & 3 & + & \dots & + & (n-3) & + & (n-2) & + & (n-1) \\ \hline 2s & = & n & + & n & + & n & + & \dots & + & n & + & n & + & n \end{array}$$

The expression on the right side of the equals sign contains $n-1$ terms—it is number of times the outer loop iterates. Each term, n , is identical. We thus are adding n to itself $n-1$ times. This is simply a multiplication, so we can rewrite the equation as

$$2s = n(n-1)$$

Solving for s , we get:

$$s = \frac{n(n-1)}{2} = \frac{n^2 - n}{2}$$

This means for a list containing n elements `is_ascending` must perform the `if` comparison up to $\frac{n^2 - n}{2}$ times.

- **is_ascending2.** The analysis of `is_ascending2` is much simpler. For a sorted list the loop must iterate $n-1$ times. This means for a list containing n elements `is_ascending2` must perform the `if` comparison up to $n-1$ times.

When the lists are small, we will not be able to detect a difference in the execution speeds of the two functions. Consider a list with 5 elements. In this case `is_ascending`'s `if` statement would execute $\frac{5^2 - 5}{2} = \frac{25 - 5}{2} = 10$ times, and `is_ascending2`'s `if` statement would execute $5 - 1 = 4$ times. While `is_ascending` executes its `if` statement $\frac{10}{4} = 2.5$ times more than does `is_ascending2`, the comparison within the `if` statement is simple and computers are fast, so we will not be able to detect the difference in execution times.

What happens if we increase the list's size by a factor of 100, from five to 500? The `is_ascending` function will execute its `if` statement $\frac{500^2 - 500}{2} = \frac{250,000 - 500}{2} = 124,750$ times. In the case of the `is_ascending2` function we get $500 - 1 = 499$. Notice that `is_ascending` is no longer executing its `if` statement 2.5 times as often as `is_ascending2`; rather the `is_ascending` function now is performing the comparison $\frac{124,750}{499} = 250$ times more often than `is_ascending2`!

As the list's length grows, the performance gap grows even more. For a list of size 5,000 the `is_ascending` function will execute its `if` statement $\frac{5000^2 - 5000}{2} = 12,497,500$ times. In the case of the `is_ascending2`

function we get $5,000 - 1 = 4,999$. The `is_ascending` function now is performing the comparison $\frac{12,497,500}{4999} = 2,500$ times more often than `is_ascending2`!

Listing 15.1 (`ascendingplot.py`) performs an experiment to test our mathematical analysis. It compares the performance of the two algorithms on lists of growing lengths. The list sizes consist of the squares of the integers from zero to 200 in increments of 20; that is, $0^2 = 0, 20^2 = 400, 40^2 = 1600, 60^2 = 3600, \dots, 200^2 = 40,000$. (This ending value of 40,000 elements was chosen to best match the speed of the program as it executes on a particular computer; you can raise this amount if running on a faster machine or lower the size for a slower computer to achieve similar results.) Besides printing the performance figures to the console, Listing 15.1 (`ascendingplot.py`) uses the `Plotter` object from Listing 13.17 (`plotobj.py`) to plot the performance curves.

Listing 15.1: `ascendingplot.py`

```
#from time import perf_counter_ns
from stopwatch import Stopwatch
from math import sqrt
from plotobj import Plotter

def is_ascending(lst: list[int]) -> bool:
    """ Returns True if lst contains elements
        in nondecreasing order as determined by
        the < operator. Returns False if the
        elements of lst are not in order. Throws
        an exception if lst is not a list or its
        elements are not compatible with the <
        operator. """
    for i in range(len(lst) - 1):
        for j in range(i + 1, len(lst)):
            if lst[i] > lst[j]:
                return False
    return True

def is_ascending2(lst: list[int]) -> bool:
    """ Returns True if lst contains elements
        in nondecreasing order as determined by
        the < operator. Returns False if the
        elements of lst are not in order. Throws
        an exception if lst is not a list or its
        elements are not compatible with the <
        operator. """
    for i in range(len(lst) - 1):
        if lst[i] > lst[i + 1]:
            return False
    return True

def compute_time(size: int,
                 data1: list[tuple[int, float]],
                 data2: list[tuple[int, float]]) -> None:
    """ Compares the performance of the is_ascending and
        is_ascending2 functions on a list of a given size.
```

```

        Prints the results of the executions and appends the
        results to the data1 and data2 lists for further processing. """

    timer = Stopwatch()
    print("List size:", size)
    my_list = list(range(size))  # Make list [0, 1, 2, 3, ..., size - 1]
    timer.start()
    ans = is_ascending(my_list)  # Compute answer
    timer.stop()
    elapsed1 = timer.elapsed()
    print(f'    is_ascending : {ans} Elapsed: {elapsed1:12.7f} sec')
    timer.reset()  # Reset timer to begin a new measurement
    timer.start()
    ans = is_ascending2(my_list)  # Compute answer
    timer.stop()
    elapsed2 = timer.elapsed()
    print(f'    is_ascending2: {ans} Elapsed: {elapsed2:12.7f} sec')
    print(f'    Speedup: {elapsed1/elapsed2:6.1f}')  # Compute speedup
    print()
    data1.append((size, elapsed1))
    data2.append((size, elapsed2))

def main():
    """ Compares the performance of the is_ascending and
        is_ascending2 functions on lists of various sizes. """
    data1, data2 = [], []

    # Compute results for sizes in the range 0...40,000
    max_size = 40000
    # Sizes used are 0**2 = 0, 20**2 = 400, 40**2 = 1600, 60**2 = 3600,
    # etc. up to 200**2 = 40,000
    for size in (x**2 for x in range(0, round(sqrt(max_size)) + 1, 20)):
        compute_time(size, data1, data2)

    # Create a plotter object
    plt = Plotter(600, 600, 0, max_size, 0, 40)

    # Plot the curves
    plt.pen.width(4)
    plt.plot_data(data1, 'blue')
    plt.plot_data(data2, 'red')

    # Wait for user interaction
    plt.interact()

if __name__ == '__main__':
    main()

```

The following shows a sample run of Listing 15.1 (ascendingplot.py):

```
List size: 0
  is_ascending : True Elapsed:    0.0000034 sec
  is_ascending2: True Elapsed:    0.0000026 sec
  Speedup:      1.3

List size: 400
  is_ascending : True Elapsed:    0.0038788 sec
  is_ascending2: True Elapsed:    0.0000370 sec
  Speedup:    104.8

List size: 1600
  is_ascending : True Elapsed:    0.0630640 sec
  is_ascending2: True Elapsed:    0.0001206 sec
  Speedup:    522.9

List size: 3600
  is_ascending : True Elapsed:    0.2839485 sec
  is_ascending2: True Elapsed:    0.0002320 sec
  Speedup:  1223.9

List size: 6400
  is_ascending : True Elapsed:    0.9222957 sec
  is_ascending2: True Elapsed:    0.0004297 sec
  Speedup:  2146.4

List size: 10000
  is_ascending : True Elapsed:    2.2293141 sec
  is_ascending2: True Elapsed:    0.0006209 sec
  Speedup:  3590.5

List size: 14400
  is_ascending : True Elapsed:    4.6402174 sec
  is_ascending2: True Elapsed:    0.0009502 sec
  Speedup:  4883.4

List size: 19600
  is_ascending : True Elapsed:    8.8314083 sec
  is_ascending2: True Elapsed:    0.0014943 sec
  Speedup:  5910.1

List size: 25600
  is_ascending : True Elapsed:   15.3933405 sec
  is_ascending2: True Elapsed:    0.0018789 sec
  Speedup:  8192.7

List size: 32400
  is_ascending : True Elapsed:   24.8598470 sec
  is_ascending2: True Elapsed:    0.0027571 sec
  Speedup:  9016.7

List size: 40000
  is_ascending : True Elapsed:   38.5650741 sec
  is_ascending2: True Elapsed:    0.0026298 sec
  Speedup: 14664.6
```

Note that during this run the `is_ascending` function requires almost 40 seconds to process a list containing 40,000 elements. This corresponds to the last argument to the `Plotter` initializer in Listing 15.1 (`ascendingplot.py`):

```
# Create a plotter object
plt = Plotter(600, 600, 0, max_size, 0, 40)
```

You should adjust the last argument (40) to the number of seconds `is_ascending` takes to process a list with 40,000 elements on your computer; for example, if your computer is faster, and `is_ascending` requires only 35 seconds to process a list of size 40,000, change the 40 to 35. This will allow the curve of the plot to better fill the plotting grid.

As Listing 15.1 (`ascendingplot.py`) demonstrates, the `is_ascending2` function consistently outperforms `is_ascending`, but both functions execute in less than one second for lists of length 3,600 or less. This means if our application deals only with smaller lists, we may not notice the difference. As the list size grows, however, the performance difference grows dramatically. The `is_ascending` function requires over 38 seconds to process a list of size 40,000, while the `is_ascending2` function takes less than $\frac{3}{1,000}$ second—the `is_ascending2` function is almost 15,000 times faster than `is_ascending`! The left window in Figure 15.1 illustrates the difference graphically. The `Plotter` object in Listing 15.1 (`ascendingplot.py`) plots the curves shown in the left window of Figure 15.1. The blue curve shows the growth of `is_ascending`'s execution time as the number of list elements grow from zero to 40,000. The x axis represents the size of the list, and the y axis represents the execution time. The red line shows the corresponding increase in `is_ascending2`'s execution time as the length of the list increases. Given the scale required to plot `is_ascending`'s curve, the curve for `is_ascending2` barely deviates from the x axis.

How do these experimental results compare with our earlier analysis? Listing 15.2 (`ascendingtheory.py`) does not call either Python function but rather plots the $f(x) = \frac{x^2 - x}{2}$ and $f(x) = x - 1$ mathematical functions over the same range of values as the list sizes tested in Listing 15.1 (`ascendingplot.py`).

Listing 15.2: `ascendingtheory.py`

```
from math import sqrt
from plotobj import Plotter

def main() -> None:
    """ Compares the theoretical performance of the is_ascending and
        is_ascending2 functions on lists of various sizes. """
    data1, data2 = [], []

    # Compute results for sizes in the range 0...40,000
    max_size = 40000
    # Sizes used are 0**2 = 0, 20**2 = 400, 40**2 = 1600, 60**2 = 3600,
    # etc. up to 200**2 = 40,000

    # Create a plotter object
    plt = Plotter(600, 600, 0, max_size, 0, 7500000000)

    # Plot the curves
    plt.pen.width(4)
    data1 = [(float(x), (x**2 - x)/2) for x in
              (x**2 for x in range(0, round(sqrt(max_size)) + 1, 20))]
```

Figure 15.1 The left window shows the graphical output of Listing 15.1 (`ascendingplot.py`). The right window shows the graphical output of Listing 15.2 (`ascendingtheory.py`). The left window plots of the experimental results comparing the performance of **`is_ascending`** to **`is_ascending2`**. The blue curve shows the growth of **`is_ascending`**'s execution time as the number of list elements grow from zero to 40,000. The red line shows the corresponding increase in **`is_ascending2`**'s execution time. Given the scale required to plot **`is_ascending`**'s curve, the curve for **`is_ascending2`** barely deviates from the x axis. The right window plots the function $\frac{n^2 - n}{2}$ with the green line and $n - 1$ with the brown line. The vertical axis scales are not the same absolute values in both graphs, as the left graph measures execution time in seconds and the right graph counts **`if`** statement executions. The shape of the curves indicate that the mathematical analysis correctly captures the relative rates of growth of execution time between the **`is_ascending`** and **`is_ascending2`** functions.



```

data2 = [(float(x), x - 1) for x in (float(x**2)
                                     for x in range(0, round(sqrt(max_size)) + 1, 20))]

plt.plot_data(data1, 'green')
plt.plot_data(data2, 'brown')

plt.interact()

if __name__ == '__main__':
    main()

```

The absolute numbers the functions compute will be very different from the numbers that Listing 15.1 (ascendingplot.py) produced. This is because the functions that Listing 15.2 (ascendingtheory.py) plots represent a count of `if` statement executions while Listing 15.1 (ascendingplot.py) plots execution time in seconds. If our analysis is correct, however, the shape of the curves should be similar. The right window of Figure 15.1 shows the graphical output of Listing 15.2 (ascendingtheory.py). Note that shapes of the curves in the left and right windows match. This means the experimental results confirm our earlier analysis. As the list size grows, the time difference between the two functions increases. While both `is_ascending` and `is_ascending2` are correct algorithms, `is_ascending2` is objectively better than `is_ascending`.

Examine again the curves in Figure 15.1 that correspond to the execution time of the `is_ascending2` Python function (red curve in the left graph) and the $f(n) = n - 1$ mathematical function that represents `if` statement execution counts (brown curve in the right graph). Both appear to be flat, but this is due to the extreme large scale of the vertical axis. If both axes used the same scale, these curves would be lines rising at a 45° angle. If we change the plotter object constructor call in Listing 15.2 (ascendingtheory.py) from

```

# Create a plotter object
plt = Plotter(600, 600, 0, max_size, 0, 7500000000)

```

to

```

# Create a plotter object (x and y axes use the same scale)
plt = Plotter(600, 600, 0, max_size, 0, max_size)

```

the scale of the vertical axis will be the same as the horizontal axis, and the program will produce the plot shown in Figure 15.2. This scale provides a more realistic view of the `is_ascending2` function, as it really does take longer to execute as the list size grows. The function $f(n) = \frac{n^2 - n}{2}$ is a parabola, but using the same scale for both axes over the full range of list sizes makes the function look almost like a vertical line! This perspective especially emphasizes the horrible performance of the bad algorithm.

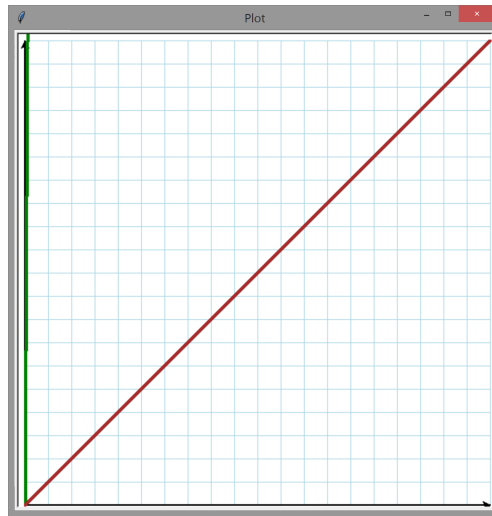
Note that both `is_ascending` and `is_ascending2` scan as few elements as necessary to return a negative result. They both return `False` immediately upon detecting an element that is out of order. As an even more extreme example of a correct but bad algorithm, `is_ascending3` always performs the same amount of work regardless of the list's ordering:

```

def is_ascending3(lst: list[int]) -> bool:
    result = True    # List is ordered unless determined otherwise
    for i in range(len(lst) - 1):
        for j in range(i + 1, len(lst)):
            if lst[i] > lst[j]:
                result = False    # Found an out-of-order element
    return result

```

Figure 15.2 Result of plotting $f(n) = \frac{n^2 - n}{2}$ (green line) versus $f(n) = n - 1$ (brown line) using the same scale for the horizontal and vertical axes. This scale provides a more realistic view of the **is_ascending2** function, as it really does take longer to execute as the list grows. It also dramatically emphasizes the horrible performance of the **is_ascending** function, which is the really steep green line that barely moves away from the y axis.



The `is_ascending3` function is just as correct as the `is_ascending` and `is_ascending2` functions because once the `is_ascending3` function sets its `result` variable to `False` it can never reset it back to `True`. Either it detects a reason to set `result` to `False` or it never changes it from its default value it assigned at the beginning. Given a list with its first element larger than its second element, the `is_ascending` and `is_ascending2` functions both will return `False` upon the first execution of their `if` statement. The `is_ascending3` function, however, will unnecessarily go through the whole list checking all the elements!

It seems computers are never fast enough or have enough memory to satisfy all our desires for software performance. Faster computers only serve to increase our expectations for applications that perform more complex tasks on larger data sets. As we have seen in the simple example above, the choice of algorithm can make a dramatic difference in the performance of a task. A correct algorithm can be so bad that even the fastest computer cannot enable it to solve a particular problem in an acceptable amount of time. A different algorithm, however, may be able to solve the same problem quickly on even a slow machine.

Computer scientists have been studying algorithms for decades and have established algorithms for common software tasks. These algorithms typically work on a collection of data. While even bad algorithms can perform acceptably on small data sets, the trick is find algorithms that continue to perform acceptably as the data size scales up.

15.2 Sorting

Lists, introduced in Chapter 10, are convenient structures for storing large amounts of data. Sorting—arranging the elements within a list into a particular order—is a common activity. For example, a list of integers may be arranged in ascending order (that is, from smallest to largest). A list of strings may be arranged in lexicographical (commonly called alphabetical) order. Many sorting algorithms exist, and some perform much better than others. We will consider several sorting algorithms that are relatively easy to describe and implement.

15.2.1 Selection Sort

Python lists have a `sort` method (Section 10.10) that orders the elements in a list. Similar to the `reversed` function (Section 10.2), the `__builtins__` module provides a `sorted` function that returns an iterable object. If the elements of a list can be ordered, we can use `sorted` to visit its elements in sorted order. The following code:

```
for elem in sorted([34, 2, 22, 70, 16, 8]):
    print(elem, end=" ")
print()
```

prints

```
2 8 16 22 34 70
```

Unlike the `list.sort` method, the `sorted` function does not disturb the contents of the original list.

Why implement a sorting algorithm when Python provides the standard `sorted` function and `list.sort` method? Writing a sort function gives us an opportunity to exercise our problem solving skills. It also gives us more practice using and manipulating lists. The experience we gain through the process better equips us to tackle problems we may encounter that have no solution in the standard library.

The *selection sort* algorithm is relatively easy to implement and easy to understand how it works. Its performance is acceptable for smaller lists. If A is a list, and i represents a list index, selection sort works as follows:

1. Set n = length of list A .
2. Set $i = 0$.
3. Examine all the elements $A[j]$, where $i < j < n$. (This simply means to consider all the elements in the list from index $i + 1$ to the last position in the list.) If any of these elements is less than $A[i]$, then exchange $A[i]$ with the smallest of these elements. (This ensures that all elements after position i are greater than or equal to $A[i]$.)
4. If i is less than $n - 1$, increase i by 1 and go to Step 2.
5. Done; list A is sorted.

The command to “go to Step 2” in Step 4 represents a loop. When the value of i in Step 4 equals n , the algorithm goes to Step 5 and terminates with a sorted list.

We can begin to translate the above description into Python as follows:

```
n = len(A)
for i in range(n - 1):
    # Examine all the elements A[j], where i < j < n.
    # If any of these A[j] is less than A[i],
    # then exchange A[i] with the smallest of these elements.
```

The first statement implements Step 1, and the `for` statement nicely packages Steps 2 and 4. The yet-to-be-implemented body of the `for` statement encompasses Step 3.

The directive at Step 3 beginning with “Examine all the elements $A[j]$, where $i < j < n$ ” also must be implemented as a loop. We continue refining our implementation with:

```
n = len(A)
for i in range(n - 1):
    # Examine all the elements A[j], where i < j < n.
    for j in range(i + 1, n):
        # Find an element smaller than A[i], if possible
        # If any A[j] is less than A[i],
        # then exchange A[i] with the smallest of these elements.
```

In order to determine if any of the elements is less than $A[i]$, we introduce a new variable named `small`. The purpose of `small` is to keep track of the position of the smallest element found so far. We will set `small` equal to i initially because we wish to locate any element less than the element located at position i .

```
n = len(A)
for i in range(n - 1):
    # small is the position of the smallest value we've seen
    # so far; we use it to find the smallest value less than A[i]
    small = i
    for j in range(i + 1, n):
        if A[j] < A[small]:
```

```

        small = j # Found a smaller element, update small
# If small changed, we found an element smaller than A[i]
    if small != i:
        # exchange A[small] and A[i]

```

Listing 15.3 (sortintegers.py) provides the complete Python implementation of the `selection_sort` function within a program that tests it out.

Listing 15.3: sortintegers.py

```

from random import randint

def random_list() -> list[int]:
    """
    Produce a list of pseudorandom integers.
    The list's length is chosen pseudorandomly in the
    range 3-20.
    The integers in the list range from -50 to 50.
    """
    result = []
    count = randint(3, 20)
    for i in range(count):
        result.append(randint(-50, 50))
    return result

def selection_sort(lst: list[int]) -> None:
    """
    Arranges the elements of list lst in ascending order.
    Physically rearranges the elements of lst.
    """
    n = len(lst)
    for i in range(n - 1):
        # Note: i, small, and j represent positions within lst
        # lst[i], lst[small], and lst[j] represent the elements at
        # those positions.
        # small is the position of the smallest value we've seen
        # so far; we use it to find the smallest value less
        # than lst[i]
        small = i
        # See if a smaller value can be found later in the list
        # Consider all the elements at position j, where i < j < n
        for j in range(i + 1, n):
            if lst[j] < lst[small]:
                small = j # Found a smaller value
        # Swap lst[i] and lst[small], if a smaller value was found
        if i != small:
            lst[i], lst[small] = lst[small], lst[i]

def main() -> None:
    """
    Tests the selection_sort function
    """
    for n in range(10):
        col = random_list()
        print(col)

```

```

        selection_sort(col)
        print(col)
        print('=====')
    main()

```

One run of Listing 15.3 (sortintegers.py) produces:

```

[-23, 47, -3, 4, 5, -46, 26, -27]
[-46, -27, -23, -3, 4, 5, 26, 47]
=====
[32, -10, -4, 41, 10, -1, -31, 3, 28, -31, -33, 46, -45, -6, 37]
[-45, -33, -31, -31, -10, -6, -4, -1, 3, 10, 28, 32, 37, 41, 46]
=====
[11, -19, 20, 43, -19, 20, -18, -17]
[-19, -19, -18, -17, 11, 20, 20, 43]
=====
[9, -22, -41, 35, 10, 48, 9, 14, -20]
[-41, -22, -20, 9, 9, 10, 14, 35, 48]
=====
[-38, -3, -7, 41, -8, -11, -23, 9, -47, 38]
[-47, -38, -23, -11, -8, -7, -3, 9, 38, 41]
=====
[-47, 1, -37, 16, -40, -14, 2, 38, 43, 19, 45]
[-47, -40, -37, -14, 1, 2, 16, 19, 38, 43, 45]
=====
[8, 39, 35, -42]
[-42, 8, 35, 39]
=====
[-8, -22, -13, 47, -28, -46, -21, -42, 27, 14, 47, -21, 2, -47]
[-47, -46, -42, -28, -22, -21, -21, -13, -8, 2, 14, 27, 47, 47]
=====
[37, -21, -32, -7]
[-32, -21, -7, 37]
=====
[33, -42, -26, 35, 37, 36, -1, 47, 24, 5, 41, -6, 48, 6, 43]
[-42, -26, -6, -1, 5, 6, 24, 33, 35, 36, 37, 41, 43, 47, 48]
=====

```

Notice that in each case the `selection_sort` function rearranges the elements in the pseudorandomly generated list into correct ascending order. To check the correctness of our sort we need to be sure that:

- the sorted list contains the same number of elements as the original, unsorted list,
- no elements in the original list are missing,
- no elements in the sorted list appear more frequently than they did in the original, unsorted list, and
- the elements appear in ascending order.

The output of Listing 15.3 (sortintegers.py) provides evidence that our `selection_sort` function is working correctly.

15.2.2 Flexible Sorting

What if we wish to change the behavior of the sorting function in Listing 15.3 (`sortintegers.py`) so that it arranges the elements in descending order instead of ascending order? It is actually an easy modification; simply change the line

```
if lst[j] < lst[small]:
```

to be

```
if lst[j] > lst[small]:
```

What if instead we want to change the sort so that it sorts the elements in ascending order except that all the even numbers in the list appear before all the odd numbers? This modification would be a little more complicated, but, with some effort, we could modify our `selection_sort` function to achieve this effect.

The next question is more intriguing: How can we rewrite the `selection_sort` function so that, by passing an additional parameter, it can sort the list in any way we want?

Consider the following comparison function:

```
def less_than(m, n):  
    return m < n
```

Ordinarily a function such as this one would not be very helpful. Rather than calling this function it is easier (and more efficient) to use the `<` operator directly. With this `less_than` function, however, we can rewrite the line

```
if lst[j] < lst[small]:
```

as

```
if less_than(lst[j], lst[small]):
```

This initially does not seem to buy us much—it appears only to make the code a bit more obscure. Notice that to change the way the `if` statement compares we need to change the name of the function. If we have a `greater_than` function, for example, we could use it in the place of `less_than`. Admittedly, changing a function name generally requires more typing than changing a single symbol (`<` to `>`); however, we will see that it gives us the ability to build a sort function that can order its elements in many different ways.

We can make our sort function more flexible by passing an ordering function as an additional parameter (see Section 8.6 for examples of functions as parameters to other functions). Listing 15.4 (`flexiblesort.py`) arranges the elements in a list two different ways using the same `selection_sort` function.

Listing 15.4: `flexiblesort.py`

```
from typing import Callable  
  
def random_list() -> list[int]:  
    """  
    Produce a list of pseudorandom integers.  
    The list's length is chosen pseudorandomly in the  
    range 3-20.  
    The integers in the list range from -50 to 50.  
    """
```

```

"""
from random import randrange
result = []
count = randrange(3, 20)
for i in range(count):
    result += [randrange(-50, 50)]
return result

def less_than(m: int, n: int) -> bool:
    """ Returns true if m is less than n; otherwise, returns false """
    return m < n

def greater_than(m: int, n: int) -> bool:
    """ Returns true if m is greater than n; otherwise, returns false """
    return m > n

def selection_sort(lst: list[int], cmp: Callable[[int, int], bool]) -> None:
    """
    Arranges the elements of list lst in ascending order.
    The comparer function cmp is used to order the elements.
    The contents of lst are physically rearranged.
    """
    n = len(lst)
    for i in range(n - 1):
        # Note: i, small, and j represent positions within lst
        # lst[i], lst[small], and lst[j] represent the elements at
        # those positions.
        # small is the position of the smallest value we've seen
        # so far; we use it to find the smallest value less
        # than lst[i]
        small = i
        # See if a smaller value can be found later in the list
        # Consider all the elements at position j, where i < j < n.
        for j in range(i + 1, n):
            if cmp(lst[j], lst[small]):
                small = j        # Found a smaller value
        # Swap lst[i] and lst[small], if a smaller value was found
        if i != small:
            lst[i], lst[small] = lst[small], lst[i]

def main() -> None:
    """
    Tests the selection_sort function
    """
    original = random_list()      # Make a random list
    working = original[:]         # Make a working copy of the list
    print('Original: ', working)
    selection_sort(working, less_than) # Sort ascending
    print('Ascending: ', working)
    working = original[:]         # Make a working copy of the list
    print('Original: ', working)
    selection_sort(working, greater_than) # Sort descending
    print('Descending: ', working)

main()

```

The output of Listing 15.4 (flexiblesort.py) is

```
Original:  [-8, 24, -46, -7, -26, -29, -44]
Ascending: [-46, -44, -29, -26, -8, -7, 24]
Original:  [-8, 24, -46, -7, -26, -29, -44]
Descending: [24, -7, -8, -26, -29, -44, -46]
```

The comparison function passed to the sort routine customizes the sort's behavior. The basic structure of the sorting algorithm does not change, but its notion of ordering is adjustable. If the second parameter to `selection_sort` is `less_than`, the function arranges the elements ascending order. If the second parameter instead is `greater_than`, the function sorts the list in descending order. More creative orderings are possible with more elaborate comparison functions.

15.2.3 Quicksort

The selection sort algorithm and implementation are relatively easy for beginning programmers to understand, but more advanced sorts exist that, on average, are much faster than selection sort, especially for large data sets. One such general purpose sort is *Quicksort*, devised by C. A. R. Hoare in 1962. Quicksort is one of the fastest known general purpose sorts.

Quicksort uses a “divide and conquer” strategy. The concept is simple:

1. Single out one of the elements in the list, often called the *pivot*. A common approach is to use the first or last element as the pivot. We will use the first element.
2. Move all the elements *less than* the pivot to positions *before* the place the pivot should go. The relative order of these elements is unimportant at this point.
3. Move all the elements *greater than or equal* to the pivot to positions *after* the place the pivot should go. The relative order of these elements is unimportant at this point.
4. Move the pivot to its proper place. It will reside between the lesser elements and greater than or equal elements. The pivot element will not move from this position during the remainder of the sort.
5. Recursively perform the Quicksort algorithm on the elements placed before the pivot, and then recursively perform the Quicksort algorithm on the elements placed after the pivot.

The algorithm sorts the list by dividing it into two sections and recursively sorting each section separately. The recursion stops when the size of the list to sort is one.

Perhaps surprisingly, Quicksort has a fairly terse Python implementation using structural pattern matching:

```
def quicksort(lst: list[int]) -> list[int]:
    """ Returns a sorted version of the callers
        list using C. A. R. Hoare's Quicksort algorithm """
    match lst:
        case []:
            return []
        case x, *xs: # Unpack: x is first element, *xs is rest of list
            return quicksort([y for y in xs if y < x]) \
```

```

        + [x] \
        + quicksort([y for y in xs if y >= x])

    return []

```

The first pattern, `[]`, matches the empty list. In this case there is nothing to do but return the same list. The second pattern is more interesting. The subject of the match statement is `lst`, a list. The following expression:

```
x, *xs
```

in the pattern unpacks the list into its first element (`x`) and the rest of the list (`*xs`). The action then concatenates three lists together:

1. `[y for y in xs if y < x]` is a list of all the elements in the rest of the list less than the first element `x`. The recursive call to `quicksort` will sort this list separately.
2. `[x]` is the list containing only the first element.
3. `[y for y in xs if y >= x]` is a list of all the elements in the rest of the list greater than or equal to the first element `x`. The recursive call to `quicksort` will sort this list separately.

When combined, this new list contains all the elements of the original list, but in sorted order.

Note that unlike our `selection_sort` function, this `quicksort` function does not modify the original list; instead it returns a new list. We say that the `selection_sort` function sorts the list “in place.” Since `quicksort` makes a new list and repeatedly concatenates smaller and smaller lists as the recursion progresses, we might expect it to be slower. The Quicksort algorithm, however, is far superior to selection sort, and the extra overhead from list creation and joining is not a significant impediment.

We can write a less elegant looking Quicksort function that sorts in place like `selection_sort`. Listing 15.5 (`comparesorts.py`) compares the run-time performances of our `selection_sort`, `quicksort`, and sort-in-place `quicksort2` functions.

Listing 15.5: `comparesorts.py`

```

from typing import Callable
from random import randint
from stopwatch import Stopwatch
from math import sqrt
from plotobj import Plotter

def random_list(n: int) -> list[int]:
    """ Produce a list of pseudorandom integers.
        The list's length is n. The integers in
        the list range from -5000 to 5000. """
    result = []
    for i in range(n):
        result.append(randint(-5000, 5000))
    return result

def selection_sort(lst: list[int]) -> list[int]:
    """ Arranges the elements of list lst in
        ascending order. Physically rearranges

```

```

        the elements of lst. Returns the sorted list """
    n = len(lst)
    for i in range(n - 1):
        small = i
        for j in range(i + 1, n):
            if lst[j] < lst[small]:
                small = j          # Found a smaller value
        if i != small:
            lst[i], lst[small] = lst[small], lst[i]
    return lst

def quicksort(lst: list[int]) -> list[int]:
    """ Returns a sorted version of the callers
        list using C. A. R. Hoare's Quicksort algorithm """
    match lst:
        case []:
            return []
        case x, *xs: # Unpack: x is first element, *xs is rest of list
            return quicksort([y for y in xs if y < x]) \
                + [x] \
                + quicksort([y for y in xs if y >= x])

    return []

def quicksort2(lst: list[int]) -> list[int]:
    """ Returns a sorted version of the callers
        list using C. A. R. Hoare's Quicksort algorithm """
    def qsort_helper(lst: list[int], front: int, back: int) -> None:
        def partition(lst: list[int], front: int, back: int) -> int:
            pivot = lst[back]
            i = front - 1
            for j in range(front, back):
                if lst[j] <= pivot:
                    i += 1
                    lst[i], lst[j] = lst[j], lst[i]
            # Move pivot to its proper place
            lst[i + 1], lst[back] = lst[back], lst[i + 1]
            return i + 1

        if front < back:
            j = partition(lst, front, back)
            qsort_helper(lst, front, j - 1)
            qsort_helper(lst, j + 1, back)

    qsort_helper(lst, 0, len(lst) - 1)
    return lst

def is_ascending(a: list[int]) -> bool:
    """ Verify that the elements of a list are in
        sorted order """
    for i in range(1, len(a)):
        if a[i - 1] > a[i]:
            return False

```



```

    return True

def time_sort(sort_name: str,
              lst: list[int],
              sort: Callable[[list[int]], list[int]],
              data: list[tuple[int, float]]) -> None:
    """ Time how long it takes to sort a list """
    # Copy the list
    working = lst[:]
    timer = Stopwatch()
    timer.start()
    working = sort(working)
    timer.stop()
    elapsed = timer.elapsed()
    if not is_ascending(working):
        print('***** FAILED SORT *****')
    data.append((len(working), elapsed))
    print(f'{sort_name}: {elapsed:10.5f} sec')

def main() -> None:
    """ Tests the selection_sort function """
    selection_data, quick_data, quick2_data = [], [], []

    # Compute results for sizes in the range 0...90,000
    max_size = 90_000
    # Sizes used are 0**2 = 0, 30**2 = 900, 60**2 = 3600, 90**2 = 8100,
    # etc. up to 300**2 = 90,000
    for size in (x**2 for x in range(0, round(sqrt(max_size)) + 1, 30)):
        seq = random_list(size)
        print(f'List size: {len(seq):,}')
        #time_sort('Selection sort', seq, selection_sort, selection_data)
        time_sort('Quicksort', seq, quicksort, quick_data)
        time_sort('In-place Quicksort', seq, quicksort2, quick2_data)
        print('=====')

    # Create a plotter object
    plt = Plotter(600, 600, 0, max_size, 0, 200)

    # Plot the curves
    plt.pen.width(4)
    #plt.plot_data(selection_data, 'blue')
    plt.plot_data(quick_data, 'red')
    plt.plot_data(quick2_data, 'green')

    # Wait for user interaction
    plt.interact()

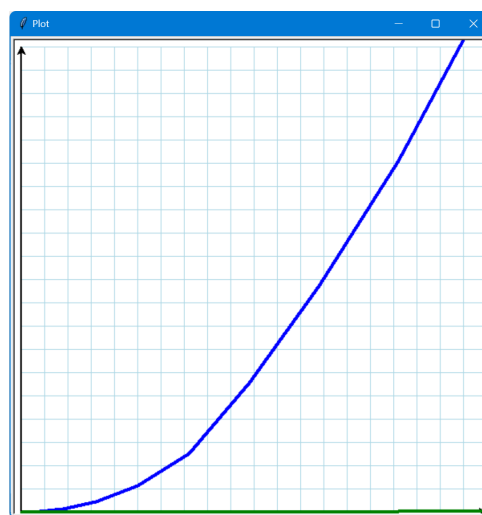
if __name__ == '__main__':
    main()

```

The results of Listing 15.5 (comparesorts.py) are dramatic. The following shows the product of one run:

```
List size: 0
Selection sort      :      0.00000 sec
Quicksort          :      0.00000 sec
In-place Quicksort:      0.00001 sec
=====
List size: 900
Selection sort      :      0.01985 sec
Quicksort          :      0.00138 sec
In-place Quicksort:      0.00134 sec
=====
List size: 3,600
Selection sort      :      0.29214 sec
Quicksort          :      0.00594 sec
In-place Quicksort:      0.00573 sec
=====
List size: 8,100
Selection sort      :      1.45044 sec
Quicksort          :      0.01358 sec
In-place Quicksort:      0.01387 sec
=====
List size: 14,400
Selection sort      :      4.61920 sec
Quicksort          :      0.02627 sec
In-place Quicksort:      0.02928 sec
=====
List size: 22,500
Selection sort      :     11.42086 sec
Quicksort          :      0.04418 sec
In-place Quicksort:      0.04816 sec
=====
List size: 32,400
Selection sort      :     25.30115 sec
Quicksort          :      0.11999 sec
In-place Quicksort:      0.10741 sec
=====
List size: 44,100
Selection sort      :     55.38944 sec
Quicksort          :      0.10911 sec
In-place Quicksort:      0.13575 sec
=====
List size: 57,600
Selection sort      :     97.17290 sec
Quicksort          :      0.14378 sec
In-place Quicksort:      0.16856 sec
=====
List size: 72,900
Selection sort      :    150.90479 sec
Quicksort          :      0.19808 sec
In-place Quicksort:      0.23475 sec
=====
List size: 90,000
Selection sort      :    221.81368 sec
Quicksort          :      0.22261 sec
In-place Quicksort:      0.25140 sec
```

Figure 15.3 The graphical output of Listing 15.5 (comparesorts.py), comparing the run times of the three sorting functions. The x -axis represents the data size, and the y -axis represents the function execution time. The x -axis ranges from 0 to 90,000, and the y -axis ranges from 0 to 200. The blue curve plots the results for `selection_sort`, the red curve plots `quicksort`, and the green curve represents the performance of the `quicksort2` function. The red curve is not visible because the program draws the green curve last, plotting it over top of the red curve. The two Quicksort functions barely rise above the x -axis at this scale.



```
=====
Done plotting
```

The `selection_sort` function requires over three and a half minutes to sort a list containing 90,000 integers, but both Quicksort functions require less than one second! The in-place `quicksort2` function is slightly slower than `quicksort`, despite the fact that `quicksort` creates and concatenates a large number of lists during its execution. Figure 15.3 shows the growth rate of the execution time of the three functions as the data size increases. Figure 15.4 adjusts the x -axis scale to range from 0 to 0.3 via the statement

```
plt = Plotter(600, 600, 0, max_size, 0, 0.3)
```

The `selection_sort` curve (blue) exceeds the upper bound for the y -axis early, but this scale better differentiates the performances of the two Quicksort functions. Notice that the in-place version, `quicksort2` (green), is slightly slower than `quicksort` (red), but the speed difference is slight. To differentiate the performances of the two Quicksort functions ever further, Figure 15.5 shows the results of a revised program that does not include the `selection_sort` function. It uses an upper bound of 1,000,000 instead of 90,000. (The `selection_sort` function would require several hours to sort a list of size 1,000,000.) The plot shows that the original `quicksort` function outperforms the in-place version. The difference is not dramatic, however.

The evidence so far indicates if we wish to sort lists, we should use our `quicksort` method. While Quicksort is much faster than selection sort, we can do even better. The Python `list` type has a `sort` method. If we need to order the elements of list `seq`, the following statement suffices:

```
seq.sort()
```

Figure 15.4 A revised plot of the results of Listing 15.5 (`comparesorts.py`), adjusting the y-axis range to 0–0.3. This scale better differentiates the performances of the two Quicksort functions. Notice that the in-place version (the green curve) is slightly slower than the original `quicksort` version (the red curve).

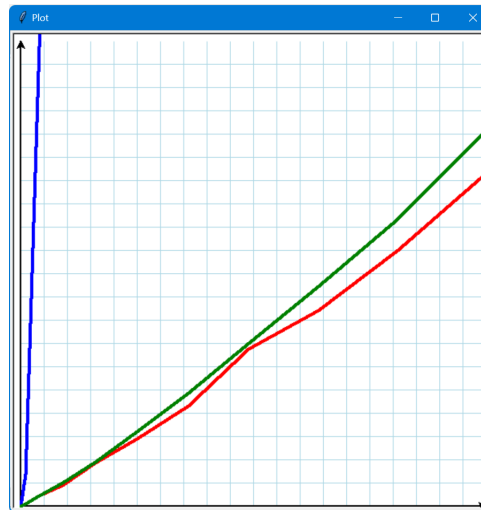


Figure 15.5 A revised version of the sort comparison program that excludes the selection sort function. It tests list sizes up to 1,000,000. Note that the original `quicksort` function, shown by the green curve, consistently outperforms the in-place `quicksort2` function, shown by the red curve, by a small margin.

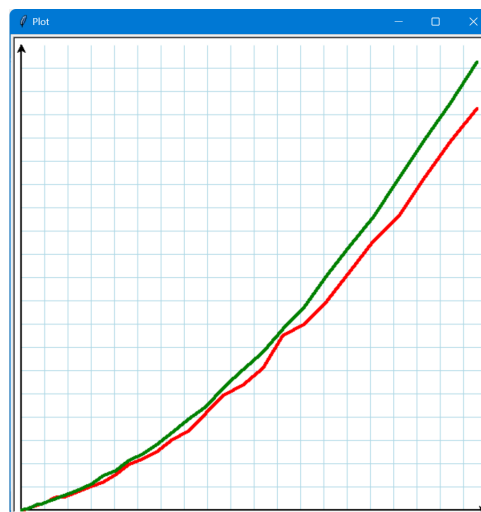
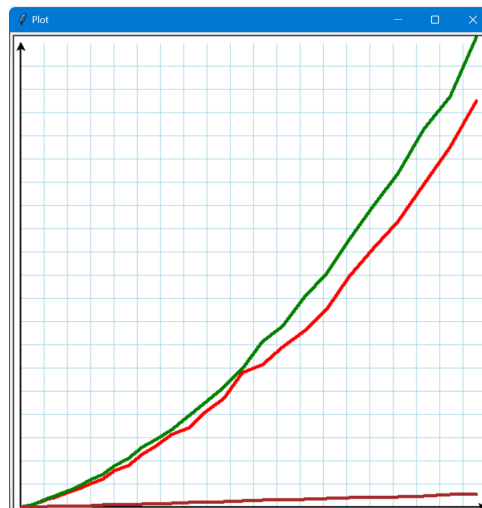


Figure 15.6 Adding the built-in `sort` method to the sort comparison. The red curve plots the execution time of `quicksort`, the green curve plots the execution time of `quicksort2`, and the brown curve at the bottom near the x -axis plots the execution time of the `list` type's built-in `sort` method.



We do not need to write any code to implement the sort. The `sort` method of the `list` class uses a fast algorithm, similar to Quicksort, but it offers an added big advantage over even our `quicksort` function. The designers of Python wrote the `sort` method in the C programming language and compiled it to machine language. This means when the interpreter encounters a statement such as

```
seq.sort()
```

it does not have to translate the code that makes up the body of the `sort` method into machine language as it executes it; instead, the machine language already is available, ready to execute directly. Removing the on-the-fly translation effort makes the execution much faster. To demonstrate the speed of the `sort` method, add the following function to the sort comparison program:

```
def builtin_sort(lst: list[int]) -> list[int]:
    lst.sort()
    return lst
```

This offers the same interface as our `selection_sort`, `quicksort`, and `quicksort2` functions, so we can pass this function to our `time_sort` function to ensure we are conducting a fair comparison.

The results are stunning. The built-in `sort` method is much faster for all list sizes. The following shows the program's output for a list containing 980,100:

```
List size: 980,100
Quicksort      :      8.97196 sec
In-place Quicksort: 10.00669 sec
Built-in sort  :      0.29589 sec
```

Figure 15.6 shows the results over the range 0 to 1,000,000.

The results make it abundantly clear. If you need to sort a list, choose the built-in `sort` method unless you have a good reason not to.

15.3 Search

Searching a list for a particular element is a common activity. We examine two basic strategies: linear search and binary search.

15.3.1 Linear Search

Listing 15.6 (linearsearch.py) uses a function named `locate` that returns the position of the first occurrence of a given element in a list; if the element is not present, the function returns `None`.

Listing 15.6: linearsearch.py

```
def locate(lst: list[int], seek: int) -> int | None:
    """
    Returns the index of element seek in list lst,
    if seek is present in lst.
    Returns None if seek is not an element of lst.
    lst is the list in which to search.
    seek is the element to find.
    """
    for i in range(len(lst)):
        if lst[i] == seek:
            return i          # Return position immediately
    return None              # Element not found

def show(lst: list[int]) -> None:
    """
    Prints the contents of list lst
    """
    for item in lst:
        # Print element right justified in 4 spaces
        print(f'{item:>4}', end='')
    print() # Print newline

def draw_arrow(value: int, spaces: str) -> None:
    """
    Print an arrow to value which is an element in a list.
    spaces specifies the horizontal offset of the arrow.
    """
    print(spaces + '^  ')
    print(spaces + '|  ')
    print(spaces + '+-- ' + str(value))

def display(lst: list[int], value: int) -> None:
    """
    Draws an ASCII art arrow showing where
    the given value is within the list.
    lst is the list.
    value is the element to locate.
    """
```

```

show(lst)                # Print contents of the list
position = locate(lst, value)
if position != None:
    position = 4*position + 3 # Compute spacing for arrow
    draw_arrow(value, ' ' * position)
else:
    print(f'({value} not in list)', sep='')
print()

def main() -> None:
    a = [100, 44, 2, 80, 5, 13, 11, 2, 110]
    display(a, 13)
    display(a, 2)
    display(a, 7)
    display(a, 100)
    display(a, 110)

if __name__ == '__main__':
    main()

```

The output of Listing 15.6 (linearssearch.py) is

```

100  44  2  80  5  13  11  2 110
                        ^
                        |
                        +-- 13

100  44  2  80  5  13  11  2 110
                ^
                |
                +-- 2

100  44  2  80  5  13  11  2 110
(7 not in list)

100  44  2  80  5  13  11  2 110
^
|
+-- 100

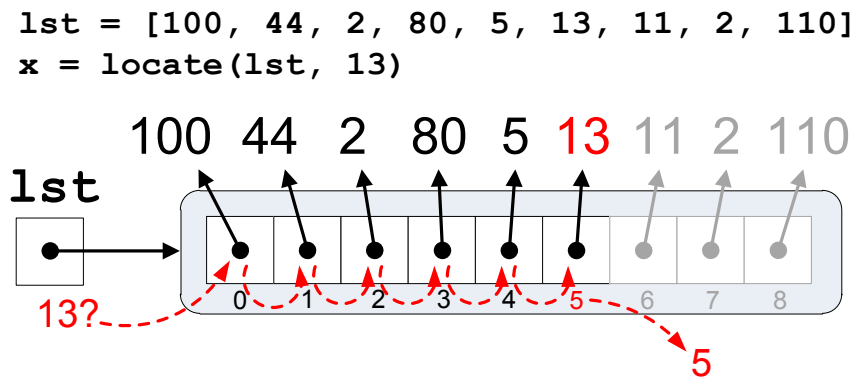
100  44  2  80  5  13  11  2 110
                        ^
                        |
                        +-- 110

```

The key function in Listing 15.6 (linearssearch.py) is `locate`; all the other functions simply lead to a more interesting display of `locate`'s results. If `locate` finds a match, the function immediately returns the position of the matching element; otherwise, if, after examining all the elements of the list, `locate` cannot find the element sought, the function returns `None`. Here `None` indicates the function could not return a valid answer. The calling code, in this example the `display` function, must ensure that `locate`'s result is not `None` before attempting to use the result as an index into a list.

The kind of search performed by `locate` is known as *linear search*, since the algorithm takes a straight

Figure 15.7 Linear search first considers the element at index 0, then index 1, then index 2, etc. until it finds the element it seeks or reaches the back of the list. The algorithm progresses through the list in a straight line without jumping around.



line path from the beginning of the list to the end of the list considering each element in order. Figure 15.7 illustrates linear search.

15.3.2 Binary Search

Linear search is acceptable for relatively small lists, but the process of examining each element in a large list is time consuming. An alternative to linear search is *binary search*. In order to perform binary search, a list must be in sorted order. Binary search exploits the sorted structure of the list using a clever but simple strategy that quickly zeros in on the element to find:

1. If the list is empty, return **None**.
2. Check the element in the middle of the list. If that element is what you are seeking, return its position. If the middle element is larger than the element you are seeking, perform a binary search on the first half of the list. If the middle element is smaller than the element you are seeking, perform a binary search on the second half of the list.

This approach is analogous to looking for a telephone number in the phone book in this manner:

1. Open the book at its center. If the name of the person is on one of the two visible pages, look at the phone number.
2. If not, and the person's last name is alphabetically less the names on the visible pages, apply the search to the left half of the open book; otherwise, apply the search to the right half of the open book.
3. Discontinue the search with failure if the person's name should be on one of the two visible pages but is not present.

We can implement the binary search algorithm as a Python function as shown in Listing 15.7 (binarysearch.py).

Listing 15.7: binarysearch.py

```

def binary_search(lst: list[int], seek: int) -> int | None:
    """
    Returns the index of element seek in list lst,
    if seek is present in lst.
    Returns None if seek is not an element of lst.
    lst is the list in which to search.
    seek is the element to find.
    """
    first = 0          # Initialize the first position in list
    last = len(lst) - 1 # Initialize the last position in list
    while first <= last:
        # mid is middle position in the list
        mid = (first + last) // 2 # Note: Integer division
        if lst[mid] == seek:
            return mid          # Found it
        elif lst[mid] > seek:
            last = mid - 1      # continue with 1st half
        else: # v[mid] < seek
            first = mid + 1     # continue with 2nd half
    return None               # Not there

def show(lst: list[int]) -> None:
    """
    Prints the contents of list lst
    """
    for item in lst:
        # Print element right justified in 4 spaces
        print(f'{item:>4}', end='')
    print() # Print newline

def draw_arrow(value: int, spaces: str) -> None:
    """
    Print an arrow to value which is an element in a list.
    n specifies the horizontal offset of the arrow.
    """
    print(spaces + '^  ')
    print(spaces + '|  ')
    print(spaces + '+-- ' + str(value))

def display(lst: list[int], value: int) -> None:
    """
    Draws an ASCII art arrow showing where
    the given value is within the list.
    lst is the list.
    value is the element to locate.
    """
    show(lst)          # Print contents of the list
    position = binary_search(lst, value)
    if position != None:
        position = 4*position + 3 # Compute spacing for arrow

```

```

        draw_arrow(value, ' ' * position)
    else:
        print(f'({value} not in list)', sep='')
        print()

def main() -> None:
    a = [100, 44, 2, 80, 5, 13, 11, 2, 110]
    a.sort()    # List must be sorted for binary search to work
    display(a, 13)
    display(a, 2)
    display(a, 7)
    display(a, 100)
    display(a, 110)

if __name__ == '__main__':
    main()

```

In the `binary_search` function:

- The initializations of `first` and `last`:

```

first = 0                # Initialize the first position in list
last = len(lst) - 1      # Initialize the last position in list

```

ensure that `first` is less than or equal to `last` for a nonempty list. If the list is empty, `first` is zero, and `last` is equal to `len(lst) - 1 = 0 - 1 = -1`. So in the case of an empty list the function will skip the loop and return `None`. This is correct behavior because an empty list cannot possibly contain any item we seek.

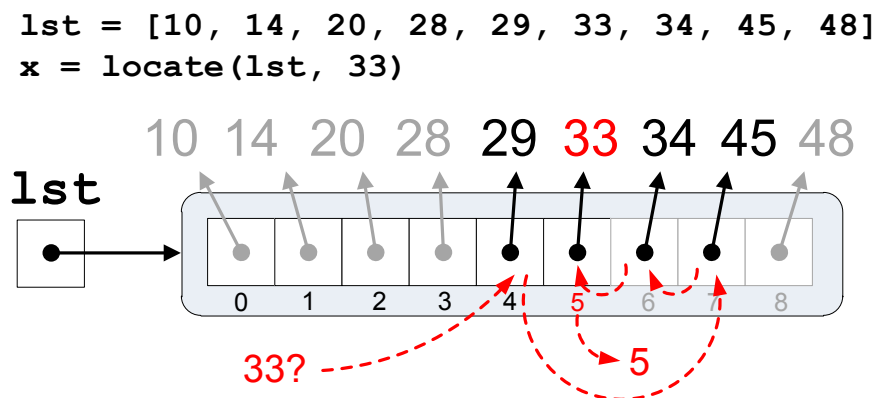
- The calculation of `mid` ensures that $\text{first} \leq \text{mid} \leq \text{last}$.
- If `mid` is the location of the sought element (checked in the first `if` statement), the loop terminates, and returns the correct position.
- The `elif` and `else` blocks ensure that either `last` decreases or `first` increases each time through the loop. Thus, if the loop does not terminate for other reasons, eventually `first` will be larger than `last`, and the loop will terminate. If the loop terminates for this reason, the function returns `None`. This is the correct behavior.
- The modification to either `first` or `last` in the `elif` and `else` blocks exclude irrelevant elements from further search. The number of elements to consider is cut in half each time through the loop.

Figure 15.8 illustrates how binary search works.

The implementation of the binary search algorithm is more complicated than the simpler linear search algorithm. Ordinarily, simpler is better, but for algorithms that process data structures that potentially hold large amounts of data, more complex algorithms employing clever tricks that exploit the structure of the data (as binary search does) often dramatically outperform simpler, easier-to-code algorithms.

For a fair comparison of linear versus binary search, suppose we want to locate an element in a sorted list. An ordered list is essential for binary search, but it can be helpful for linear search as well. The revised linear search algorithm for ordered lists is

Figure 15.8 Binary search begins in the middle of an ordered list. The algorithm jumps forward or backward as needed in decreasing stride amounts. Each successive probe reduces the number of elements in its search space by one-half.



```
# This version requires list lst to be sorted in
# ascending order.
def linear_search(lst, seek):
    i = 0                # Start at beginning
    n = len(lst)         # Length of list
    while i < n and lst[i] <= seek:
        if lst[i] == seek:
            return i     # Return position immediately
    return None          # Element not found
```

Notice that, as in the original version of linear search, the loop will terminate when it has examined all the elements, but this version will terminate early when it encounters an element larger than the sought element. Since the list is sorted, there is no need to continue the search once the search has found an element larger than the value sought; seek cannot appear after a larger element in a sorted list.

Suppose a list to search contains n elements. In the worst case—looking for an element larger than any currently in the list—the loop in linear search takes n iterations. In the best case—looking for an element smaller than any currently in the list—the function immediately returns without considering any other elements. The number of loop iterations thus ranges from 1 to n , and so on average linear search requires $\frac{n}{2}$ comparisons before the loop finishes and the function returns.

Now consider binary search. After each comparison, the size of the list remaining to consider is one-half the original size. If the binary search algorithm does not locate the element on its first probe, the number of remaining elements to search is $\frac{n}{2}$. The next time through the loop, the number of elements left to consider drops to $\frac{n}{4}$, then $\frac{n}{8}$, and so forth. The problem of determining how many times a set of things can be divided in half until only one element remains can be solved with a base-2 logarithm. For binary search, the worst case scenario of not finding the sought element requires the loop to make $\log_2 n$ iterations.

How does this analysis help us determine which search is better? The quality of an algorithm is judged by two key characteristics:

- How much time (processor cycles) does it take to run?
- How much space (memory) does it take to run?

In our situation, both search algorithms process the list with only a few extra local variables, so for large lists they both require essentially the same space. The big difference here is speed. Binary search performs more elaborate computations each time through the loop, and each operation takes time, so perhaps binary search is slower. Linear search is simpler (fewer operations through the loop), but perhaps its loop executes many more times than the loop in binary search, so overall it is slower.

We can deduce the faster algorithm in two ways: empirically and analytically. An empirical test is an experiment; we carefully implement both algorithms and then measure their execution times. The analytical approach analyzes the source code to determine how many operations the computer's processor must perform to run the program on a problem of a particular size.

Listing 15.8 (searchcompare.py) gives us some empirical results.

Listing 15.8: searchcompare.py

```
from typing import Callable
from plotobj import Plotter

"""
Compares the running times of linear search and
binary search on lists of various sizes.
"""

def binary_search(lst: list[int], seek: int) -> int | None:
    """
    Returns the index of element seek in list lst,
    if seek is present in lst.
    lst must be in sorted order.
    Returns None if seek is not an element of lst.
    lst is the list in which to search.
    seek is the element to find.
    """
    first = 0          # Initially the first element in list
    last = len(lst) - 1 # Initially the last element in list
    while first <= last:
        # mid is middle of the list
        mid = (first + last) // 2 # first + (last - first + 1)//2 # Note: Integer division
        if lst[mid] == seek:
            return mid # Found it
        elif lst[mid] > seek:
            last = mid - 1 # continue with 1st half
        else: # v[mid] < seek
            first = mid + 1 # continue with 2nd half
    return None # Not there

def ordered_linear_search(lst: list[int], seek: int) -> int | None:
    """
    Returns the index of element seek in list lst,
    if seek is present in lst.
```

```

    lst must be in sorted order.
    Returns None if seek is not an element of lst.
    lst is the list in which to search.
    seek is the element to find.
    """
    i = 0
    n = len(lst)
    while i < n and lst[i] <= seek:
        if lst[i] == seek:
            return i          # Return position immediately
        i += 1
    return None              # Element not found

def run_search(lst: list[int], seeks: list[int],
               search: Callable[[list[int], int], int | None],
               trials: int) -> float:
    """
    Searches for all the elements in an ordered list (lst)
    using search function search. Averages the running time over
    trials runs. Returns the average time.
    """
    from stopwatch import Stopwatch
    #n = len(lst)
    #elapsed = 0
    timer = Stopwatch()
    timer.start()           # Start the clock
    for i in range(trials):
        for elem in seeks:
            i = search(lst, elem)
            if i == None or i != lst[i]:
                print('error')
    timer.stop()            # Stop the clock
    return timer.elapsed()/trials    # Average time for search

def test_searches(lst: list[int], seeks: list[int], trials: int,
                  lin_data: list[tuple[int, float]],
                  bin_data: list[tuple[int, float]]) -> None:
    """
    Measures the running times of ordered linear search and binary
    search on a given list. Averages the times over n runs.
    """
    # Find each element using ordered linear search
    lin = run_search(lst, seeks, ordered_linear_search, trials)
    # Find each element using binary search
    bin = run_search(lst, seeks, binary_search, trials)
    # Print the results
    print(f'{len(lst):6} {lin:10.5f} {bin:10.5f} {lin/bin:8.1f}')
    lin_data.append((len(lst), lin))
    bin_data.append((len(lst), bin))

def make_search_set(n: int) -> list[int]:
    """

```

```

    Make a list of elements to seek
    """
    from random import randrange
    result = []
    for i in range(n):
        result += [randrange(n)]
    return result

def main() -> None:
    """
    Makes a table comparing the running times of ordered linear
    search vs. binary search on lists of various sizes.
    """
    from math import sqrt

    # Maximum list size
    max_size = 5001

    # Number of trials over which to average the results
    trials = 10

    # Stores data to plot
    linear_data = []
    binary_data = []

    # Print table header
    print('   Size   Linear   Binary   Speedup')
    print('-----')
    for size in range(0, max_size, 200):
        test_list = list(range(size))
        seek_list = make_search_set(size)
        test_searches(test_list, seek_list, trials,
                      linear_data, binary_data)

    # # Small lists: 10 to 100, in steps of 10
    # for size in range(10, 100, 10):
    #     test_list = list(range(size))
    #     seek_list = make_search_set(size)
    #     test_searches(test_list, seek_list, trials)
    # # Medium lists: 100 to 1,000, in steps of 100
    # for size in range(100, 1000, 100):
    #     test_list = list(range(size))
    #     seek_list = make_search_set(size)
    #     test_searches(test_list, seek_list, trials)
    # # Large lists: 1,000 to 5,000, in steps of 500
    # for size in range(1000, 5001, 500):
    #     test_list = list(range(size))
    #     seek_list = make_search_set(size)
    #     test_searches(test_list, seek_list, trials)

    # Create a plotter object
    plt = Plotter(600, 600, 0, max_size, 0, 1.0)

    # Plot the curves

```

```
plt.pen.width(4)
plt.plot_data(linear_data, 'red')
plt.plot_data(binary_data, 'blue')

# Wait for user interaction
plt.interact()

if __name__ == '__main__':
    main()
```

The `main` function in Listing 15.8 (`searchcompare.py`) builds lists of sequential integer values, `[1, 2, 3, ...]` of various sizes.

The program assigns each of these lists in turn to the `test_list` variable. The program also builds lists of random integer values (referenced via `seek_list`) to be used as search candidates for each `test_list`. It then passes these lists off to the `test_searches` function to measure the running times of the two search functions. The `test_searches` function, in turn, calls the `run_search` function to test a particular search function. The `run_search` function uses the elements from `main`'s `seek_list` as search candidates. The `run_search` function searches for all the elements in `seek_list` a specified number of times and averages the running times. The `main` function directs `test_searches` to average 10 runs for each of the list sizes.

On one system, Listing 15.8 (`searchcompare.py`) produces the following table:

Size	Linear	Binary	Speedup
0	0.00000	0.00000	2.4
200	0.00132	0.00018	7.4
400	0.00592	0.00042	13.9
600	0.01279	0.00066	19.4
800	0.02276	0.00095	24.1
1000	0.03700	0.00122	30.3
1200	0.05108	0.00153	33.3
1400	0.07107	0.00183	38.8
1600	0.09454	0.00214	44.2
1800	0.11553	0.00244	47.3
2000	0.14470	0.00275	52.7
2200	0.17599	0.00308	57.2
2400	0.21339	0.00344	62.0
2600	0.25041	0.00376	66.6
2800	0.28905	0.00406	71.1
3000	0.34224	0.00445	76.9
3200	0.38375	0.00476	80.7
3400	0.42832	0.00510	84.1
3600	0.48934	0.00548	89.3
3800	0.55296	0.00598	92.5
4000	0.60857	0.00615	99.0
4200	0.66639	0.00664	100.4
4400	0.72850	0.00683	106.7
4600	0.80274	0.00732	109.6
4800	0.86355	0.00757	114.0
5000	0.94850	0.00803	118.1
Done plotting			

The rightmost column of the table shows the speedup factor of binary search over ordered linear search. Notice that the speedup increases as the list length grows. For lists that contain more than 4,200 elements, binary search is more than 100 times faster than linear search.

Figure 15.9 shows the plot that Listing 15.8 (`searchcompare.py`) displays. The graphical window plots the execution times of the two different search algorithms over various list sizes.

Empirically, binary search performs dramatically better than linear search. The left side of Figure 15.10 plots the results produced by Listing 15.8 (`searchcompare.py`) for lists containing up to 1,000 elements.

In addition to empirical observations, we can judge which algorithm is better by analyzing the source code for each function. Each arithmetic operation, assignment, logical comparison, function call, and list access requires time to execute. We will assume each of these activities requires one unit of processor “time.” This assumption is not strictly true, but it will give good results for relative comparisons. Since we will follow the same rules when analyzing both search algorithms, the relative results for comparison will be close enough for our purposes.

We first consider linear search. We determined that, on average, the loop makes $\frac{n}{2}$ iterations for a list of size n . The initialization of `i` happens only one time during each call to `linear_search`. All other activity involved with the loop except the `return` statements happens $\frac{n}{2}$ times. The function returns either `i` or `None`, and it may execute at most one `return` statement during each call. Table 15.1 shows the breakdown for linear search. The results in Table 15.1 indicate the running time of the `linear_search` function can be expressed as a simple mathematical linear function: $f(n) = 3n + 4$.

Figure 15.9 Plots the execution times of linear search and binary search. The x -axis represents the number of elements in a list and ranges from 0 to 5000. The y -axis represents the function's execution time. The red curve tracks linear search, and the blue curve represents binary search. Binary search is much faster than linear search on an ordered list.

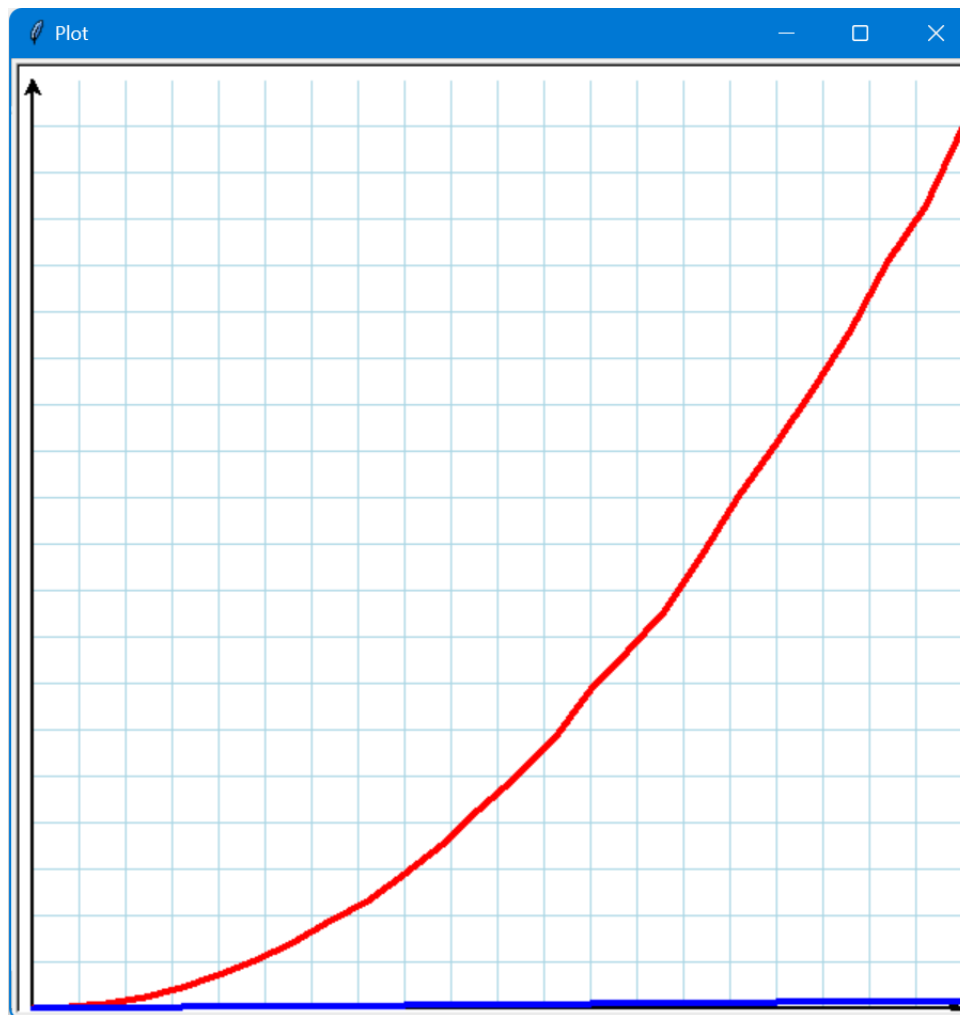


Table 15.1 Analysis of Linear Search Algorithm. The $\frac{n}{2}$ loop iterations is based on the average time to locate an element. The function will execute exactly one of the two `return` statements during a given call, so each is given a cost of $\frac{1}{2}$.

Action	Operation(s)	Operation Count	Times Executed	Total Cost
<code>i = 0</code>	<code>=</code>	1	1	1
<code>n = len(lst)</code>	<code>=, len</code>	2	1	2
<code>while i < n and lst[i] <= seek:</code>	<code><, and, <=, []</code>	4	$\frac{n}{2}$	$2n$
<code>if lst[i] == seek:</code>	<code>[], ==</code>	2	$\frac{n}{2}$	n
<code>return i</code>	<code>return</code>	1	$\frac{1}{2}$	$\frac{1}{2}$
<code>return None</code>	<code>return</code>	1	$\frac{1}{2}$	$\frac{1}{2}$
Total time units				$3n + 4$

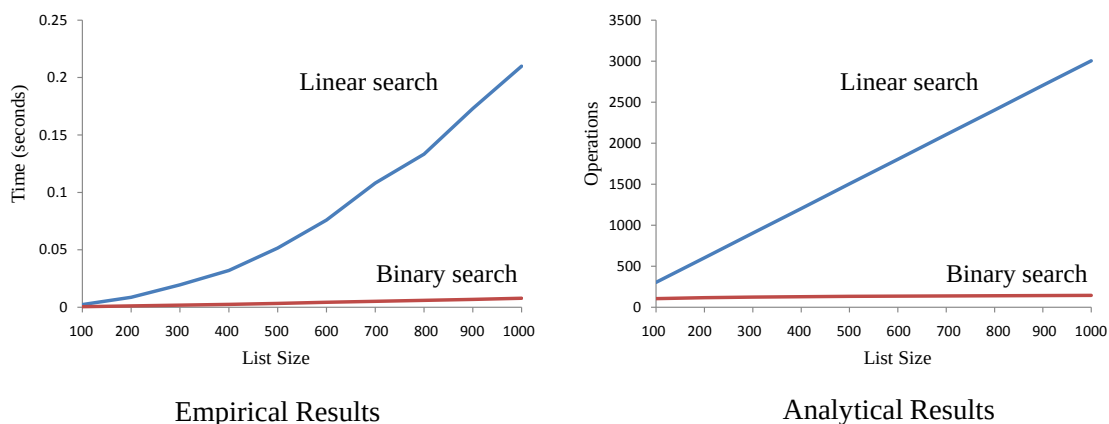
Table 15.2 Analysis of Binary Search Algorithm. Each time through the loop the function executes either the `elif` or `else` statement, so each one is charged $\frac{1}{2}$ its actual cost.

Action	Operation(s)	Operation Count	Times Executed	Total Cost
<code>first = 0</code>	<code>=</code>	1	1	1
<code>last = len(lst) - 1</code>	<code>=, len, -</code>	3	1	3
<code>while first <= last:</code>	<code><=</code>	1	$\log_2 n$	$\log_2 n$
<code>mid=(first+last)//2</code>	<code>=, +, //</code>	3	$\log_2 n$	$3\log_2 n$
<code>if lst[mid] == seek:</code>	<code>[], ==</code>	2	$\log_2 n$	$2\log_2 n$
<code>return mid</code>	<code>return</code>	1	1	1
<code>elif lst[mid] > seek:</code>	<code>[], ></code>	2	$\log_2 n$	$2\log_2 n$
<code>last = mid - 1</code>	<code>=, -</code>	2	$\frac{1}{2}\log_2 n$	$\log_2 n$
<code>else:</code>		0		0
<code>first = mid + 1</code>	<code>=, +</code>	2	$\frac{1}{2}\log_2 n$	$\log_2 n$
<code>return None</code>	<code>return</code>	1	1	1
Total time units				$10\log_2 n + 6$

Next, we consider binary search. We determined that in the worst case the loop in `binary_search` iterates $\log_2 n$ times if the list contains n elements. The `binary_search` function performs the two initializations before the loop just once per call. Most of the actions within the loop occur $\log_2 n$ times, except that only one `return` statement can be executed per call, and in the `if/elif/else` statement only one path can be chosen per loop iteration. Table 15.2 shows the complete analysis of binary search. We see that the execution time for binary search can be expressed as the logarithmic function $10\log_2 n + 6$.

Figure 15.10 compares the empirical results with the analytical results for lists containing 100 to 1,000 elements. The left side of Figure 15.10 plots the values produced by Listing 15.8 (`searchcompare.py`), and the right side of Figure 15.10 plots the two functions $3n + 4$ and $10\log_2 n + 6$. In these two graphs we can compare the growth rates of the two search techniques by examining the shapes of the curves. Notice how closely the two graphs compare to each other. In both graphs the gap between the linear search curve and binary search curve increasingly widens at the same rate as the list size increases. The binary search curve appears to be effectively flat, although it really is growing very slowly, much more slowly than the linear search curve. The bottom line is that binary search is fast even for large lists.

Figure 15.10 Linear search versus binary search. The two graphs plot the execution speeds of ordered linear search and binary search on lists with 100 to 1,000 elements. The graph on the left plots data from timing the program’s execution. The graph on the right plots the functions derived from analyzing the Python source code. Notice how closely the two graphs correspond.



15.4 Recursion Revisited

In Section 8.3 we saw recursive functions for factorial and greatest common divisor. Suppose we have a recursive function named `f` that accepts a single parameter. Recall that recursion works in the following manner:

1. If function `f`’s argument selects the base case of the recursion, return the default answer;
2. otherwise, do something with the argument and invoke `f` with an argument that is closer to the base case.

We can restate this in a more informal way:

1. If `f`’s argument represents a trivial problem, return the default, easy answer.
2. If `f`’s argument represents a nontrivial problem, return the result of computing part of the problem and combining it with the solution to a smaller or simpler problem.

The phrase “the solution to a smaller or simpler problem” is the recursive call. As the recursion progresses, the function works on smaller and/or simpler problems until it reaches a trivial problem, at which point the recursive process is over.

Many data structures lend themselves to recursive algorithms. Consider the task of counting the occurrences of a particular element within a list. We will name the function `count`, and it will accept a list and an additional parameter that represents the element to count. Given a correctly implemented `count` function, the following code fragment,

```
lst1 = [21, 19, 31, 22, 14, 31, 22, 6, 31]
```

```

print(count(lst1, 31))
lst2 = ['FRED', [2, 3], 44, 'WILMA', 'FRED', 8, 'BARNEY']
print(count(lst2, 'FRED'))
print(count(lst2, 'BETTY'))
print(count([], 16))

```

should print

```

3
2
0
0

```

We will think about the problem *recursively*. What should the function do if the list is empty? Is this a trivial problem? What should the function do if the list is not empty?

If the list is empty, the problem is trivial. No matter what we are trying to count, it will not appear in an empty list. In this case the count function simply can return zero.

What if the list is not empty? We can look at the first element. If the first element is equal to the value we wish to count, we know we have one occurrence, so the answer is one plus the number of times the value appears in the rest of the list. If the first element is not equal to the value we wish to count, the answer is just the number of times the value appears in the rest of the list.

Do you see that “the number of times the value appears in the rest of the list” is a recursive call on a shorter list? Each recursive call processes a shorter and shorter list until the list is empty (the trivial case). All along the chain of recursive function calls the function is either adding one or not adding one to count’s ultimate answer. Listing 15.9 (recursivecount.py) implements our recursive count function.

Listing 15.9: recursivecount.py

```

from typing import Any

def count(lst: list[Any], item: Any) -> int:
    """ Counts the number of occurrences of item within the list lst """
    if len(lst) == 0:      # Is the list empty?
        return 0          # Nothing can appear in an empty list
    else:
        # Count the occurrences in the rest of the list
        # (all but the first element)
        count_rest = count(lst[1:], item)
        if lst[0] == item:
            return 1 + count_rest
        else:
            return count_rest

def main() -> None:
    lst1 = [21, 19, 31, 22, 14, 31, 22, 6, 31]
    print(count(lst1, 31))
    lst2 = ['FRED', [2, 3], 44, 'WILMA', 'FRED', 8, 'BARNEY']
    print(count(lst2, 'FRED'))
    print(count(lst2, 'BETTY'))
    print(count([], 16))

```

```
if __name__ == '__main__':
    main()
```

The expression `count([21, 19, 31, 14, 31, 6, 31], 31)` would evaluate as

```
count([21, 19, 31, 14, 31, 6, 31], 31) = count([19, 31, 14, 31, 6, 31], 31)
                                         = count([31, 14, 31, 6, 31], 31)
                                         = 1 + count([14, 31, 6, 31], 31)
                                         = 1 + count([31, 6, 31], 31)
                                         = 1 + 1 + count([6, 31], 31)
                                         = 1 + 1 + count([31], 31)
                                         = 1 + 1 + 1 + count([], 31)
                                         = 1 + 1 + 1 + 0
                                         = 1 + 1 + 1
                                         = 1 + 2
                                         = 3
```

While the `count` function in Listing 15.9 (`recursivecount.py`) works properly, it has one, potentially big disadvantage. Each time the function's execution selects the recursive route it slices the list. Slicing a list creates a copy of the list (see Section 10.8). This means every call to `count` in the recursive call chain makes a complete copy of the list, except for the first element of each successive list. If the list is long, this can unnecessarily consume a large amount of the computer's memory. How big can it get? Consider an initial call to `count` that passes a list of 1,000 elements. The first recursive call passes a new list of 999 elements. The second recursive call passes a new list of 998 elements, and so forth. By the time it completes, the `count` will have created 999 extra lists, holding a combined total of 499,500 elements. The space required to process the list is about 500 times the size of original list. Not only does the recursion use more memory, copying the list takes time. This excessive list copying slows down the program's execution.

It would be better to implement the `count` function so that it does not make any copies. Listing 15.10 (`inplacecount.py`) implements a recursive `count` function that makes no copies of the list.

Listing 15.10: `inplacecount.py`

```
from typing import Any

def count_helper(lst: list[Any], pos: int, item: Any) -> int:
    """ Counts the number of occurrences of item within the list
        lst. pos represents the current position under
        examination within the list. """
    if pos == len(lst):    # Are we past the end of the list?
        return 0          # Nothing can appear past the end
    else:
        # Count the occurrences in the rest of the list
        # (all but the first element)
        count_rest = count_helper(lst, pos + 1, item)
        if lst[pos] == item:
            return 1 + count_rest
        else:
            return count_rest

def count(lst: list[Any], item: Any) -> int:
```

```

    """ Counts the number of occurrences of item within the list
        lst. Delegates the work to the recursive count_helper
        function, passing zero as the initial position (which is
        the index of the first element in the list). """
    return count_helper(lst, 0, item)

def main() -> None:
    lst1 = [21, 19, 31, 22, 14, 31, 22, 6, 31]
    print(count(lst1, 31))
    lst2 = ['FRED', [2, 3], 44, 'WILMA', 'FRED', 8, 'BARNEY']
    print(count(lst2, 'FRED'))
    print(count(lst2, 'BETTY'))
    print(count([], 16))

if __name__ == '__main__':
    main()

```

Listing 15.10 (inplacecount.py) uses two functions to do the counting. Its `count` function merely calls `count_helper` with the proper initial parameters. The `count_helper` function does all the interesting work. Instead of creating copies of the list, `count_helper` accepts an additional parameter, an index, for the recursion to keep track of its position within the list. The list parameter is an alias of the original list, not a copy. When a function can process a list without making a copy, we say the function processes the list *in place*.

Listing 15.10 (inplacecount.py) exposes the `count_helper` function, making it available to the `main` function and other code. If the programmer's intention is for only the `count` function to use `count_helper`, we can make it local to `count`, as Listing 15.11 (inplacecountlocal.py) illustrates (see Section 8.10 for more information about local function definitions).

Listing 15.11: inplacecountlocal.py

```

from typing import Any

def count(lst: list[Any], item: Any) -> int:
    """ Counts the number of occurrences of item within the list
        lst. Delegates the work to the recursive count_helper
        function, passing zero as the initial position (which is
        the index of the first element in the list). """

    def count_helper(pos: int) -> int:
        """ Local function counts the number of occurrences of an
            item within a list.
            pos represents the current position under examination
            within the list.
            lst and item are nonlocal variables from the outer context. """
        if pos == len(lst): # Are we past the end of the list?
            return 0 # Nothing can appear past the end
        else:
            # Count the occurrences in the rest of the list
            # (all but the first element)
            count_rest = count_helper(pos + 1)

```

```

        if lst[pos] == item:
            return 1 + count_rest
        else:
            return count_rest

    # Body of count function
    return count_helper(0)

def main() -> None:
    lst1 = [21, 19, 31, 22, 14, 31, 22, 6, 31]
    print(count(lst1, 31))
    lst2 = ['FRED', [2, 3], 44, 'WILMA', 'FRED', 8, 'BARNEY']
    print(count(lst2, 'FRED'))
    print(count(lst2, 'BETTY'))
    print(count([], 16))

if __name__ == '__main__':
    main()

```

In Listing 15.11 (inplacecountlocal.py), the `count_helper` function is inaccessible outside of the `count` function. Note also that the `count_helper` function can access `count`'s `lst` and `item` parameters directly. This reduces `count_helper`'s parameter `count` to one.

You may be thinking that it would be simpler to implement our counting function with a loop in a manner similar to linear search:

```

def count(lst: list[Any], item: Any) -> int:
    cnt = 0    # Initialize item count
    for elem in lst:
        if elem == item:
            cnt += 1    # Found an item, count it
    return cnt

```

This processes the list in place and does not use recursion. This version of `count` actually is superior to both recursive versions. As we saw in Section 8.3, every function call requires a little extra time and memory. If two functions—one iterative and one recursive—faithfully implement the same algorithm, the iterative version will be more efficient.

A recursive function does have one distinct advantage over a non-recursive function, though. A recursive function does not just call itself; the self-call eventually returns back to the site of its invocation. Each recursive invocation has its own parameters and creates its own local variables. When the function returns to itself, it “remembers” its original parameters and local variables the way they were before the recursive invocation. We say the function *unwinds* back to its previous state.

Listing 15.12 (recursivememory.py) demonstrates the unwinding power of recursion.

Listing 15.12: recursivememory.py

```

from random import randint

def rec(n: int, depth: int) -> None:
    """ Prints the value of the parameter n and local variable
        rand before and after the recursive call.
        n is the parameter of interest.
        depth represents the depth of the recursion. """
    rand = randint(0, 1000) # Make a random number
    print(' ' * depth, 'Entering: n =', n, ' rand =', rand)
    if n == 0:
        print(' ' * depth, ' *** Recursion over ***')
    else:
        rec(n - 1, depth + 1)
    print(' ' * depth, 'Exiting: n =', n, ' rand =', rand)

rec(10, 0)

```

The `rec` function in Listing 15.12 (`recursivememory.py`) is recursive. The first parameter, `n`, is the parameter of interest. The second parameter, `depth`, reflects the depth of the recursion. The `depth` variable controls the indentation level of each printed line.

Listing 15.12 (`recursivememory.py`) prints

```

Entering: n = 10   rand = 716
  Entering: n = 9   rand = 970
    Entering: n = 8   rand = 21
      Entering: n = 7   rand = 835
        Entering: n = 6   rand = 11
          Entering: n = 5   rand = 759
            Entering: n = 4   rand = 168
              Entering: n = 3   rand = 65
                Entering: n = 2   rand = 86
                  Entering: n = 1   rand = 238
                    Entering: n = 0   rand = 991
                      *** Recursion over ***
                        Exiting: n = 0   rand = 991
                          Exiting: n = 1   rand = 238
                            Exiting: n = 2   rand = 86
                              Exiting: n = 3   rand = 65
                                Exiting: n = 4   rand = 168
                                  Exiting: n = 5   rand = 759
                                    Exiting: n = 6   rand = 11
                                      Exiting: n = 7   rand = 835
                                        Exiting: n = 8   rand = 21
                                          Exiting: n = 9   rand = 970
                                            Exiting: n = 10   rand = 716

```

Observe that the unwinding of the recursive calls restores the original values of the parameter `n` and local variable `rand`. Since `rand` is assigned pseudo-probabilistically, it is not possible to restore its original value without first storing it somewhere for later retrieval. The function-call-and-return process automatically takes care of saving and restoring the previous values of local variables.

It is more difficult to write a non-recursive function that has this ability to unwind itself to a previous program state. Such non-recursive implementations usually offer no efficiency advantages. Listing 15.13 (`nonrecursivememory.py`) provides a non-recursive version of the `rec` function.

Listing 15.13: `nonrecursivememory.py`

```
from random import randint

def nonrec(n: int, depth: int) -> None:
    """ Prints the value of the parameter n and local variable
        rand before and after the recursive call.
        n is the parameter of interest.
        depth represents the depth of the recursion. """
    history = []
    while n != 0:
        rand = randint(0, 1000) # Make a random number
        history += [(n, depth, rand)] # Remember original values of n and depth
        print(' ' * depth, 'Entering: n =', n, ' rand =', rand)
        n -= 1
        depth += 1
    print(' ' * depth, ' *** Loop over ***')
    while len(history) > 0:
        n, depth, rand = history[-1]
        del history[-1]
        print(' ' * depth, 'Exiting: n =', n, ' rand =', rand)

nonrec(10, 0)
```

The output of Listing 15.13 (`nonrecursivememory.py`) is identical to the output of Listing 15.12 (`recursivememory.py`), except for the text before the unwinding. Listing 15.13 (`nonrecursivememory.py`) uses two separate, sequential loops and an accessory list to simulate the recursive behavior of Listing 15.12 (`recursivememory.py`). The purpose of the local `history` list is to remember the current state of the variables `n`, `depth`, and `rand` so the executing program can restore their values after the simulated recursion “returns.” The extra space for the `history` list and extra time spent managing the `history` list is comparable to the space and time the recursive function requires. It is much easier to allow the magic of recursion to automatically take care of saving and restoring the function’s local variables and parameters.

The ability of a function to remember its current state, call itself on a subproblem, and return to its previous state is essential to some algorithms. Section 15.5 explores one such algorithm.

15.5 List Permutations

Sometimes it is useful to consider all the possible arrangements of the elements within a list. A sorting algorithm, for example, must work correctly on any initial arrangement of elements in a list. To test a sort function, a programmer could check to see if it produces the correct result for all arrangements of a relatively small list. We saw in Section 5.4 that an arrangement of a sequence of ordered items is called a *permutation*. Listing 5.20 (`permuteabc.py`) *prints* all the permutations of the sequence *ABC*. We need something more flexible: a function that *generates* all the possible permutations of any list. The function will accept a list as a parameter and return a list containing all the permutations of the parameter. (Note that the return value is a list of lists.) Listing 15.14 (`listpermutations.py`) contains functions that build a new list

containing all the permutations of a given list.

Listing 15.14: listpermutations.py

```
from typing import Any

def perm(lst: list[Any], begin: int, result: list[list[Any]]) -> None:
    """ Creates a list (result) containing all the permutations of the
        elements of a given list (lst), beginning with a
        specified index (begin).
        This is a helper function for the permutations function. """
    end = len(lst) - 1 # Index of the last element
    if begin == end:
        result += [lst[:]] # Copy lst into result
    else:
        for i in range(begin, end + 1): # Consider all indices
            # Interchange the element at the first position
            # with the element at position i
            lst[begin], lst[i] = lst[i], lst[begin]
            # Recursively permute the rest of the list
            perm(lst, begin + 1, result)
            # Undo the earlier interchange
            lst[begin], lst[i] = lst[i], lst[begin]

def permutations(lst: list[Any]) -> list[list[Any]]:
    """ Returns a list containing all the permutations of the
        orderings of the elements of a given list (lst).
        Delegates the hard work to the perm function. """
    result = []
    perm(lst, 0, result)
    return result

def main() -> None:
    """ Tests the permutations function. """
    a = list(range(3)) # Make list [0, 1, 2]
    print('List:', a) # Print the list
    # Generate and print all permutations of the list
    print('Permutations:', permutations(a))

if __name__ == '__main__':
    main()
```

Listing 15.14 (listpermutations.py) prints

```
List: [0, 1, 2]
Permutations: [[0, 1, 2], [0, 2, 1], [1, 0, 2], [1, 2, 0], [2, 1, 0], [2, 0,
```

Examining program's output closely, we see it is a list that contains all the permutations of the list `[0, 1, 2]`.

The `perm` function in Listing 15.14 (listpermutations.py) is a recursive function, as it calls itself inside of its definition. We have seen how recursion can be an alternative to iteration; however, the `perm` function here uses *both* iteration *and* recursion together to generate all the arrangements of a list. At first glance,

the combination of these two algorithm design techniques as used here may be difficult to follow, but we actually can understand the process better if we *ignore* some of the details of the code.

First, notice that in the recursive call the argument `begin` is one larger. This means as the recursion progresses the beginning index keeps increasing until it reaches the index of the last element in the list. The recursion terminates when `begin` becomes equal to the last index.

In its simplest form the function looks like this:

```
def perm(lst: list[Any], begin: int, result: list[list[Any]]) -> None:
    end = len(lst) - 1 # Index of the last element
    if begin == end:
        # Add the current list to the list of permutations
    else:
        # Do the interesting part of the algorithm
```

Let us zoom in on the interesting part of the algorithm (less the comments):

```
for i in range(begin, end + 1):
    lst[begin], lst[i] = lst[i], lst[begin]
    perm(lst, begin + 1, result)
    lst[begin], lst[i] = lst[i], lst[begin]
```

If the mixture of iteration and recursion is confusing, eliminate iteration! If a loop iterates a fixed number of times, you may replace the loop with the statements in its body duplicated that number times; for example, we can rewrite the code

```
for i in range(5):
    print(i)
```

as

```
print(0)
print(1)
print(2)
print(3)
print(4)
```

Notice that the loop is gone. This process of transforming a loop into the series of statements that the loop would perform is known as *loop unrolling*. Compilers and interpreters can unroll loops behind the scenes to make the code's execution faster. After unrolling the loop, the loop control variable (in this case `i`) is gone, so there is no need to initialize `i` (done once) and, more importantly, no need to check and update `i` during each iteration of the loop. Eliminating these tasks, especially the check and update within the loop, reduces the work the program must do, thereby speeding up its execution.

Our purpose for unrolling the loop in `perm` is not to optimize it. Instead we are trying to understand better how the algorithm works. In order to unroll `perm`'s loop, we will consider the case for lists containing exactly three elements. In this case we would hardcode the `for` statement in the `perm` function as

```
for i in range(begin, 3):
    lst[begin], lst[i] = lst[i], lst[begin]
    perm(lst, begin + 1, result)
    lst[begin], lst[i] = lst[i], lst[begin]
```

and we can unroll this code into

```

lst[begin], lst[0] = lst[0], lst[begin]    # Swap
perm(lst, begin + 1, result)
lst[begin], lst[0] = lst[0], lst[begin]    # Swap back

lst[begin], lst[1] = lst[1], lst[begin]    # Swap
perm(lst, begin + 1, result)
lst[begin], lst[1] = lst[1], lst[begin]    # Swap back

lst[begin], lst[2] = lst[2], lst[begin]    # Swap
perm(lst, begin + 1, result)
lst[begin], lst[2] = lst[2], lst[begin]    # Swap back

```

Once the loop is gone, we see we have simply a series of recursive calls of `perm` sandwiched by element swaps. The first swap interchanges an element in the list with the first element. The second swap reverses the effects of the first swap. This series of *swap-call perm-swap back* operations allows each element in the list to have its turn being the first element in the permuted list. The `perm` recursive call generates all the permutations of the rest of the list. Figure 15.11 traces the recursive process of generating all the permutations of the list `[0, 1, 2]`. The leftmost third of Figure 15.11 shows the original contents of the list and the initial call of `perm`. The three branches represent the three iterations of the `for` loop: `i` varying from `begin` (0) to the last index (2). The lists indicate the state of the list after the first swap but before the recursive call to `perm`.

The middle third of Figure 15.11 shows the state of the list during the first recursive call to `perm`. The two branches represent the two iterations of the `for` loop: `i` varying from `begin` (1) to the last index (2). The lists indicate the state of the list after the first swap but before the next recursive call to `perm`. At this level of recursion the element at index zero is fixed, and the remainder of the processing during this chain of recursion is restricted to indices greater than zero.

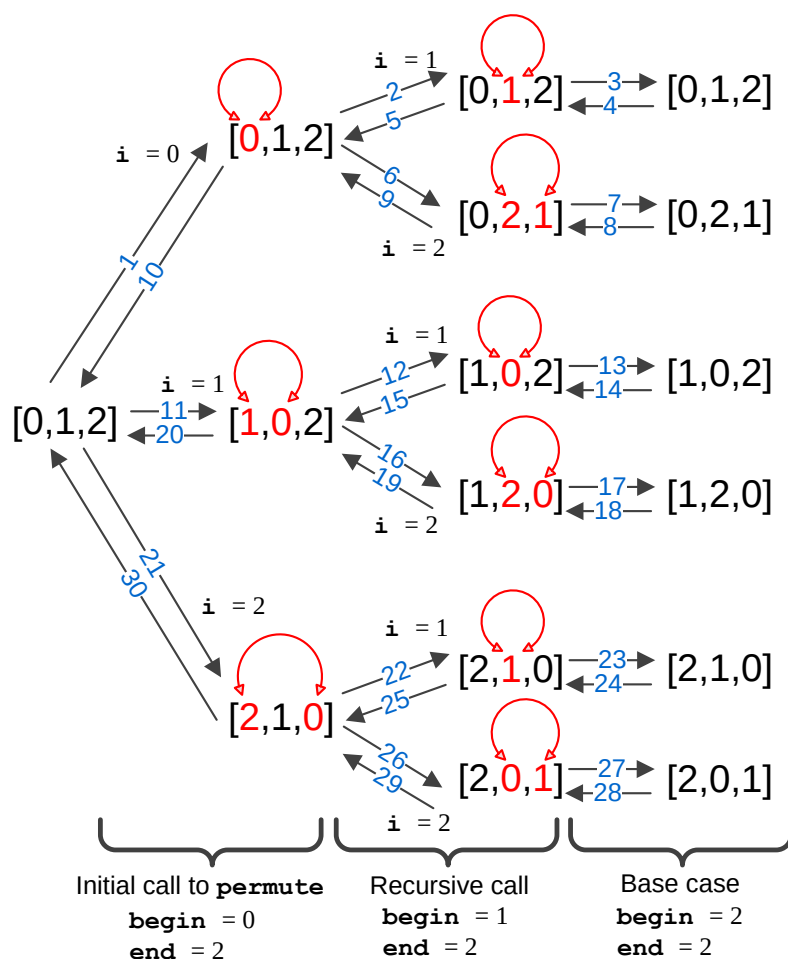
The rightmost third of Figure 15.11 shows the state of the list during the second recursive call to `perm`. At this level of recursion the elements at indices zero and one are fixed, and the remainder of the processing during this chain of recursion is restricted to indices greater than one. This leaves the element at index two, but this represents the base case of the recursion because `begin` (2) equals the index of the last element (2). In this case the function makes no more recursive calls to itself. The function merely adds a copy of the current list to the list of permutations.

The arrows in Figure 15.11 represent a call to, or a return from, `perm`. They illustrate the recursive call chain. The arrows pointing left to right represent a call, and the arrows pointing from right to left represent a return from the function. The numbers associated with arrow indicate the order in which the calls and returns occur during the execution of `perm`.

We can augment the `perm` function to better illustrate the iterative and recursive processes. With a technique known as *code instrumentation*, we will add statements that provide insight into the algorithm's progression. The term *instrumentation* mirrors its meaning outside the realm of programming. A motor vehicle, for example, has an instrument panel containing several different instruments. The speedometer indicates the vehicle's current speed, and the tachometer provides the vehicle's engine's RPMs. Neither of these devices is absolutely essential for driving the vehicle, but they do give the driver more precise information about the state of the driving experience.

Listing 15.15 (`perminstrumented.py`) instruments the `perm` function by adding `print` statements that indicate state of its list as it loops and calls itself recursively.

Figure 15.11 A tree mapping out the recursive process of the `perm` function operating on the list `[0, 1, 2]`. The second column from the left shows the original contents of the list after the first swap but before the first recursive call to `perm`. The swapped elements appear in red. The third column shows the contents of the list at the second level of recursion. In the third column the elements at index zero are fixed, as this recursion level is using `begin` with a value of one instead of zero. The `for` loop within this recursive call swaps the elements highlighted in red. The rightmost column is the point where `begin` equals the index of the last element, and so the `perm` function does not call itself, effectively terminating the recursion. The numbers on the arrows trace the order in which the program makes the calls to, and returns from, the `perm` function.



Listing 15.15: perminstrumented.py

```

from typing import Any

def perm(lst: list[Any], begin: int, result: list[list[Any]],
        depth: int) -> None:
    """ Creates a list (result) containing all the permutations of the
        elements of a given list (lst), beginning with a
        specified index (begin).
        Printing statements report the progression of the
        function's recursion. The depth parameter indicates the
        depth of the recursion.
        This is a helper function for the permutations function. """
    print(' ' * depth, 'begin =', begin)
    end = len(lst) - 1 # Index of the last element
    if begin == end:
        result += [lst[:]] # Copy lst into result
        print(' ' * depth, ' *')
    else:
        for i in range(begin, end + 1):
            # Interchange the element at the first position
            # with the element at position i
            lst[begin], lst[i] = lst[i], lst[begin]
            print(' ' * depth, ' ', lst[begin], '<-->', lst[i], ' ', lst)
            # Recursively permute the rest of the list
            perm(lst, begin + 1, result, depth + 1)
            # Undo the earlier interchange
            lst[begin], lst[i] = lst[i], lst[begin]

def permutations(lst: list[Any]) -> list[list[Any]]:
    """ Returns a list containing all the permutations of the
        orderings of the elements of a given list (lst).
        Delegates the hard work to the perm function. """
    result = []
    perm(lst, 0, result, 0) # Initial call with depth = 0
    return result

def main() -> None:
    a = list(range(3))
    print(permutations(a))

if __name__ == '__main__':
    main()

```

Figure 15.12 shows the output of Listing 15.15 (perminstrumented.py).

Now that we have a function that produces a list containing all the permutations of a given list in a regular fashion, we must admit that the function is practical only for relatively small lists. Consider a list that contains just 25 elements. How many distinct permutations are there of this list? The factorial function

Figure 15.12 Output of Listing 15.15 (perminstrumented.py). The nested, inner sections show the recursive executions that place the element at index one. The outer sections represent the initial recursive calls that establish the element at index zero. The asterisk (*) indicates the end of the recursion. Note that the recursion ends exactly when `begin` is 2. Since 2 is the last index, there is no need to continue.

```

begin = 0
0 <--> 0   [0, 1, 2]
begin = 1
  1 <--> 1   [0, 1, 2]
  begin = 2
    *
  2 <--> 1   [0, 2, 1]
  begin = 2
    *
1 <--> 0   [1, 0, 2]
begin = 1
  0 <--> 0   [1, 0, 2]
  begin = 2
    *
  2 <--> 0   [1, 2, 0]
  begin = 2
    *
2 <--> 0   [2, 1, 0]
begin = 1
  1 <--> 1   [2, 1, 0]
  begin = 2
    *
  0 <--> 1   [2, 0, 1]
  begin = 2
    *
[[0, 1, 2], [0, 2, 1], [1, 0, 2], [1, 2, 0], [2, 1, 0], [2, 0, 1]]

```

counts the number of ways of arranging the elements in a sequence:

$$25! = 15,511,210,043,330,985,984,000,000$$

Thus, our lowly list containing just 25 elements has 15,511,210,043,330,985,984,000,000 distinct permutations. Computers are fast and easily deal with large numbers, so this should not be a problem, should it? If each element of the list occupied just one byte of storage (the actual size is more than one byte), one permutation of the list containing 25 elements would require 25 bytes or memory. The list containing all the permutations would require

$$\begin{aligned} 25 \text{ bytes} \cdot 25! &= 387,780,251,083,274,649,600,000,000 \text{ bytes} \\ &\approx 387,780 \text{ zettabytes} \end{aligned}$$

(1 zettabyte = 1 billion terabytes.) 387,780 zettabytes is about 140,000 times greater than 2.7 zettabytes, the estimated total data storage space in all media (hard drives, solid state drives, CDs, DVDs, tape, etc.) found on the planet (see <http://en.wikipedia.org/wiki/Zettabyte>). It is safe to assume that your laptop or desktop would not have enough RAM to hold the list of all permutations. Besides, if your program could generate one permutation each nanosecond (an unreasonably fast rate even with today's fastest processors), the program would require

$$\begin{aligned} 25! \text{ nanoseconds} &= 15,511,210,043,330,985,984,000,000 \text{ nanoseconds} \\ &\approx 15,511,210,043,330,986 \text{ seconds} \\ &\approx 4,308,669,456,481 \text{ hours} \\ &\approx 179,527,894,020 \text{ days} \\ &\approx 491,857,244 \text{ years} \end{aligned}$$

Most users would be unwilling to wait almost *five million centuries* for the list of permutations that, by the way, is too large to store on any computer system on the planet!

Listing 15.14 (`listpermutations.py`) is impractical for all but relatively small lists because the `perm` function does not return until after building the list containing all the permutations. The basic algorithm is sound, however, and fortunately we can salvage it nicely using generators. (We first explored generators in Section 8.9.) Instead of producing the entire list of permutations, our function will yield each permutation one at a time.

We know that a function that produces a generator must use a `yield` statement rather than `return`. What we have yet to see is how this works with recursion.

The `perm` function in Listing 15.14 (`listpermutations.py`) adds a new list permutation to its `result` list with the following statement in its base case:

```
result += [lst[:]]    # Copy lst into result
```

The expression `lst[:]` makes a copy of the `lst`. We want to `yield` a copy of the list instead of adding a copy of it to another list. This is an easy change, as we rewrite the statement to be

```
yield lst[:]    # Yield a copy of lst
```

We covered the base case perfectly, what happens in the recursive case? Recursive generators are a little different from the iterative generators we saw earlier. Since the `if` block contains a `yield` statement,

the `else` block needs one as well. What we want to yield is what the recursive call to `perm` eventually yields when it reaches its base case. When we need to yield a value from a recursive call we must use the `yield from` statement. The `yield from` statement indicates the generator should yield the result that the chain of recursive calls ultimately yields when it reaches its terminal base case. Listing 15.16 (`generatepermutations.py`) shows how to use `yield` and `yield from` in a recursive generator.

Listing 15.16: `generatepermutations.py`

```
from typing import Any, Generator

def perm(lst: list[Any], begin: int) -> Generator[Any, Any, Any]:
    """ Generates the sequence of all the permutations of the
        elements of a given list (lst), beginning with a
        specified index (begin).
        This is a helper function for the permutations function. """
    end = len(lst) - 1 # Index of the last element
    if begin == end:
        yield lst[:] # Yield a copy of lst
    else:
        for i in range(begin, end + 1): # Consider all indices
            # Interchange the element at the first position
            # with the element at position i
            lst[begin], lst[i] = lst[i], lst[begin]
            # Recursively permute the rest of the list
            yield from perm(lst, begin + 1)
            # Undo the earlier interchange
            lst[begin], lst[i] = lst[i], lst[begin]

def permutations(lst: list[Any]) -> Generator[Any, Any, Any]:
    """ Generates the sequence of all the permutations of the
        elements of list lst.
        Delegates the hard work to the perm function. """
    yield from perm(lst, 0)

def main() -> None:
    """ Tests the permutations function. """
    a = list(range(3)) # Make list [0, 1, 2]
    print('List:', a) # Print the list
    # Generate and print all permutations of the list
    for p in permutations(a):
        print(p, end=' ')
    print()

if __name__ == '__main__':
    main()
```

Listing 15.16 (`generatepermutations.py`) prints

```
List: [0, 1, 2]
[0, 1, 2] [0, 2, 1] [1, 0, 2] [1, 2, 0] [2, 1, 0] [2, 0, 1]
```

The `permutations` function must yield the result of the call to `perm`, so the `yield from` statement appears there as well. This is because the `permutations` function itself does not create the value; instead, `permutations` relies on the `perm` function to create the value. A function that relies on another function to create the yielded value must use `yield from`. Note that this is consistent with the way `yield from` works with recursion.

Note that since we are no longer building a physical list, we do not need the extra `result` parameter in the `perm` function.

The `perm` function in our generator code creates and yields just one permutation at a time. If the caller does not store each generated list but merely prints it out or uses it in some other way and discards it before obtaining the next list, the program will not run into the memory limitations of the original version. This does not help the time it takes to produce all the permutations; however, the generator `perm` function “returns” a permutation immediately, thus avoiding the problems with the original version that tried to make all the permutations before returning. This means the caller can get into and out of the function quickly. While the program still would require centuries to complete its execution if asked to print all the permutations of a list with 25 elements, it could print the first 100 permutations very quickly:

```
lst = list(range(25))
count = 0
for p in permutations(lst):    # Too many to see them all!
    print(p, end=' ')
    count += 1
    if count == 100:
        break                 # Just print the first 100 permutations
print()
```

Note that we can always build a list of permutations with our generator version of the `permutations` function by using the `list` conversion function: The statement

```
lst = list(permutations([0, 1, 2]))
```

builds such a list. Be aware, however, that just as in the non-generator version, memory and time constraints limit the size of the list used in a statement like this one.

While Listing 15.16 (`generatepermutations.py`) is a good exercise in recursive list processing, the Python standard library provides a generator-like object named `permutations` in the `itertools` module that works almost like our `permutations` function. Surprisingly, the standard `permutations` generator produces tuples instead of lists, as Listing 15.17 (`stdpermutations.py`) demonstrates.

Listing 15.17: `stdpermutations.py`

```
# Use the standard permutations function to list
# the possible arrangements of elements in a list.

from itertools import permutations

a = [0, 1, 2]
for p in permutations(a):
    print(p, end=' ')
print()
```

Listing 15.17 (`stdpermutations.py`) produces

```
(0, 1, 2) (0, 2, 1) (1, 0, 2) (1, 2, 0) (2, 0, 1) (2, 1, 0)
```

In order to better match the behavior of Listing 15.16 (`generatepermutations.py`), we must convert each tuple to a list. Listing 15.18 (`stdpermutations2list.py`) uses the `list` constructor function to perform the conversion.

Listing 15.18: `stdpermutations2list.py`

```
# Use the standard permutations function to list
# the possible arrangements of elements in a list.

from itertools import permutations

a = [0, 1, 2]
for p in permutations(a):
    print(list(p), end=' ')
print()
```

Listing 15.18 (`stdpermutations2list.py`) produces

```
[0, 1, 2] [0, 2, 1] [1, 0, 2] [1, 2, 0] [2, 0, 1] [2, 1, 0]
```

It is evident that the standard `permutations` object uses a different algorithm because it orders the results differently from Listing 15.16 (`generatepermutations.py`).

15.6 Randomly Permuting a List

We have seen that generating all the permutations of a large list is computationally intractable. Often, however, we merely need to produce one permutation chosen at random. For example, we may need to randomly rearrange the contents of an ordered list so that we can test a sort function to see if it will produce the original list. We could generate all the permutations, put each one in a list, and select a permutation at random from that list. This approach is inefficient, especially as the length of the list to permute grows larger. Fortunately, we can randomly permute the contents of a list easily and quickly. Listing 15.19 (`randompermute.py`) contains a function named `permute` that randomly permutes the elements of a list.

Listing 15.19: `randompermute.py`

```
from random import randrange
from typing import Any

def permute(lst: list[Any]) -> None:
    """
    Randomly permutes the contents of list lst
    """
    n = len(lst)
    for i in range(n - 1):
        pos = randrange(i, n)    # i <= pos < n
        lst[i], lst[pos] = lst[pos], lst[i]

def main() -> None:
    """
    Tests the permute function that randomly permutes the
```

```

    contents of a list
    """
    a = [1, 2, 3, 4, 5, 6, 7, 8]
    print('Before:', a)
    permute(a)
    print('After :', a)

main()

```

Notice that the `permute` function in Listing 15.19 (`randompermute.py`) uses a simple un-nested loop and no recursion. The `permute` function varies the `i` index variable from 0 to the index of the next to last element in the list. The function pseudorandomly chooses an index greater than or equal to `i` using `randrange` (see Section 6.6), and the function then exchanges the elements at position `i` and the random position. At this point all the elements at positions less than `i` are fixed and will not change as the function's execution continues. The function then increments index `i` for the next iteration and continues its work until it has considered all acceptable values for `i`.

To be correct, our `permute` function must be able to generate any valid permutation of the list. It is important also that our `permute` function is able produce all possible permutations with equal probability; said another way, we do not want our `permute` function to generate some permutations more often than others. The `permute` function in Listing 15.19 (`randompermute.py`) is fine, but consider a slight variation of the algorithm:

```

def faulty_permute(lst: list[Any]) -> None:
    """
    An attempt to randomly permute the contents of list lst
    """
    n = len(lst)
    for i in range(n - 1):
        pos = randrange(0, n)    # 0 <= pos < n
        lst[i], lst[pos] = lst[pos], lst[i]

```

Do you see the difference between `faulty_permute` and `permute`? In `faulty_permute`, the random index is chosen from all valid list indices, whereas `permute` restricts the random index to valid indices greater than or equal to `i`. This means the function can exchange any element within `lst` with the element at position `i` during any loop iteration. While this approach may superficially appear to be just as good as `permute`, it in fact produces an uneven distribution of permutations. Listing 15.20 (`comparepermutations.py`) exercises each permutation function 1,000,000 times on the list `[1, 2, 3]` and tallies each permutation. There are exactly six possible permutations of this three-element list.

Listing 15.20: `comparepermutations.py`

```

from random import randrange
from typing import Any, Callable

# Randomly permute a list
def permute(lst: list[Any]) -> None:
    """
    Randomly permutes the contents of list lst
    """
    n = len(lst)
    for i in range(n - 1):

```

```

        pos = randrange(i, n)    # i <= pos < n
        lst[i], lst[pos] = lst[pos], lst[i]

# Randomly permute a list?
def faulty_permute(lst: list[Any]) -> None:
    """
    An attempt to randomly permute the contents of list lst
    """
    n = len(lst)
    for i in range(n):
        pos = randrange(0, n)    # 0 <= pos < n
        lst[i], lst[pos] = lst[pos], lst[i]

def classify(a: list[int]) -> int:
    """
    Classify a list as one of the six permutations
    """
    sum = 100*a[0] + 10*a[1] + a[2]
    if sum == 123: return 0
    elif sum == 132: return 1
    elif sum == 213: return 2
    elif sum == 231: return 3
    elif sum == 312: return 4
    elif sum == 321: return 5
    else: return -1

def report(a: list[int]) -> None:
    """
    Report the accumulated statistics
    """
    print("1,2,3: ", a[0])
    print("1,3,2: ", a[1])
    print("2,1,3: ", a[2])
    print("2,3,1: ", a[3])
    print("3,1,2: ", a[4])
    print("3,2,1: ", a[5])

def run_test(perm: Callable[[list[Any]], None], runs: int) -> None:
    """
    Uses a permutation function to generate the permutations
    of the list [1,2,3]
    perm: the permutation function to test
    runs: the number permutations to perform
    """
    # The list to permute
    original = [1, 2, 3]

    # permutation_tally list keeps track of each permutation pattern
    # permutation_tally[0] counts {1,2,3}
    # permutation_tally[1] counts {1,3,2}
    # permutation_tally[2] counts {2,1,3}

```

```

# permutation_tally[3] counts {2,3,1}
# permutation_tally[4] counts {3,1,2}
# permutation_tally[5] counts {3,2,1}
permutation_tally = 6 * [0] # Clear all the counters
for i in range(runs):      # Run runs times
    # working holds a copy of original is gets permuted and tallied
    working = original[:]
    # Permute the list with the permutation algorithm
    perm(working)
    # Count this permutation
    permutation_tally[classify(working)] += 1
report(permutation_tally) # Report results

def main() -> None:
    # Each test performs one million permutations
    runs = 1000000

    print("--- Random permute #1 -----")
    run_test(permute, runs)

    print("--- Random permute #2 -----")
    run_test(faulty_permute, runs)

main()

```

In Listing 15.20 (comparepermutations.py)'s output, permute #1 corresponds to our original permute function, and permute #2 is the `faulty_permute` function. The output of Listing 15.20 (comparepermutations.py) reveals that the faulty permutation function favors some permutations over others:

```

--- Random permute #1 -----
1,2,3: 166176
1,3,2: 166957
2,1,3: 167012
2,3,1: 166668
3,1,2: 166182
3,2,1: 167005
--- Random permute #2 -----
1,2,3: 148811
1,3,2: 184870
2,1,3: 185251
2,3,1: 184763
3,1,2: 148200
3,2,1: 148105

```

In one million runs, the `permute` function provides a fairly even distribution of the six possible permutations of `[1, 2, 3]`. The distributions are not all exactly the same, but we would expect minor differences due to the variations that randomness introduces. On the other hand, the `faulty_permute` function generates the permutations `[1, 3, 2]`, `[2, 1, 3]`, and `[2, 3, 1]` more often than the permutations `[1, 2, 3]`, `[3, 1, 2]`, and `[3, 2, 1]`.

To see why `faulty_permute` misbehaves, we need to examine all the permutations it can produce during one call. Figure 15.13 shows a hierarchical structure that maps out how `faulty_permute`

Figure 15.13 A tree mapping out the ways in which `faulty_permute` can transform the list `[1, 2, 3]` at each iteration of its `for` loop

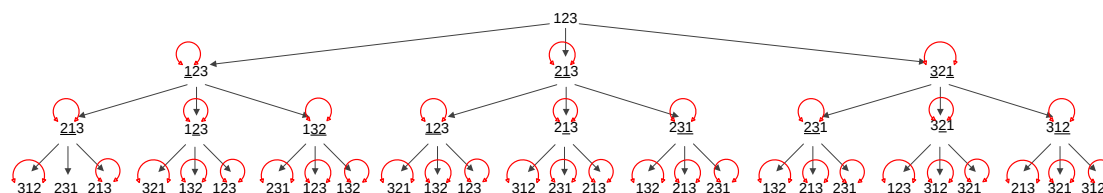
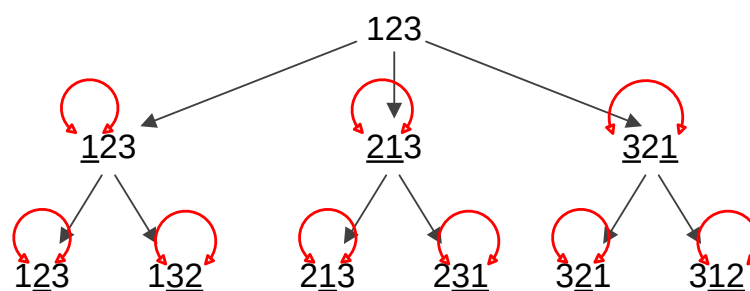


Figure 15.14 A tree mapping out the ways in which `permute` can transform the list `[1, 2, 3]` at each iteration of its `for` loop



transforms its list parameter each time through the `for` loop. The top of the tree shows the original list, `[1, 2, 3]`. The second row shows the three possible resulting lists after the first iteration of the `for` loop. The leftmost list represents the element at index zero swapped with the element at index zero (effectively no change). The second list on the second row represents the interchange of the elements at index 0 and index 1. The third list on the second row results from the interchange of the elements at positions 0 and 2. The underlined elements represent the elements most recently swapped. If only one item in the list is underlined, the function merely swapped the item with itself. The bottom row contains all the possible outcomes of the `faulty_permute` function given the list `[1, 2, 3]`.

As Figure 15.13 shows, the lists `[1, 3, 2]`, `[2, 1, 3]`, and `[2, 3, 1]` each appear five times in the last row, while `[1, 2, 3]`, `[3, 1, 2]`, and `[3, 2, 1]` each appear only four times. There are a total of 27 possible outcomes, so some permutations appear $\frac{4}{27} = 14.815\%$ of the time, while

the others appear $\frac{5}{27} = 18.519\%$ of the time. When generating one million permutations we would get $14.815\% \cdot 1,000,000 = 148150$, and $18.519 \cdot 1,000,000 = 185190$. Notice that these numbers agree with our experimental results from Listing 15.20 (comparepermutations.py).

Compare Figure 15.13 to Figure 15.14. The second row of the tree for `permute` is identical to the second row of the tree for `faulty_permute`, but the third rows are different. The second time through its loop the `permute` function does not attempt to exchange the element at index zero with any other elements. We see that none of the first elements in the lists in row three are underlined. The third row contains exactly one instance of each of the possible permutations of `[1, 2, 3]`. This means that the correct `permute`

function is not biased towards any of the individual permutations, and so the function can generate all the permutations with equal probability. The `permute` function has a $\frac{1}{6} = 16.667\%$ probability of generating a particular permutation. Over 1,000,000 trials, we would expect each permutation to occur 166667 times. This number agrees with our the experimental results of Listing 15.20 (`comparepermutations.py`).

15.7 Reversing a List

Listing 15.21 (`listreverse.py`) contains a recursive function named `rev` that accepts a list as a parameter and returns a new list with all the elements of the original list in reverse order.

Listing 15.21: `listreverse.py`

```
def rev(lst):
    return [] if len(lst) == 0 else rev(lst[1:]) + lst[0:1]

print(rev([1, 2, 3, 4, 5, 6, 7]))
```

Python has a standard function, `reversed`, that accepts a list parameter. The `reversed` function does not return a list but instead returns an iterable object that works like a generator or `range` within a `for` loop (see Section 5.3). Listing 15.22 (`reversedexample.py`) uses `reversed` to print the contents of a list backwards.

Listing 15.22: `reversedexample.py`

```
for item in reversed([1, 2, 3, 4, 5, 6, 7]):
    print(item, end=" ")
print()
```

Listing 15.22 (`reversedexample.py`) prints

```
7 6 5 4 3 2 1
```

We can use the `list` conversion function to make a new list object out of the iterator object that `reversed` creates:

```
rev = list(reversed([1, 2, 3, 4, 5, 6, 7]))
```

Section 10.10 introduced the `reverse` method that modifies a list by reversing its elements in place. The following code

```
x = [1, 2, 3, 4]
print(x)
x.reverse()
print(x)
```

prints

```
[1, 2, 3, 4]
[4, 3, 2, 1]
```


The `reverse` of the list class method does not use any extra space since it does not create a new list but merely repositions the elements in the existing list object. You can use the `reverse` method if you need an actual reversed list, as opposed to using the iterator `reversed` that provides merely a backwards traversal of the list. Note that after calling the `reverse` method on a list, calling `reverse` a second time restores the list to its original ordering.

15.8 Memoization

We know a program can use variables to remember values as it executes. A programmer must be able to predict the number of values the program must manage in order to write enough variables in the code. A dictionary provides an opportunity to create an arbitrary amount of new storage during a program's execution. We will consider a simple problem that demonstrates the value of the dynamic storage provided by dictionaries.

Section 8.3 introduced the Fibonacci number sequence. That section provided a function to compute the n^{th} Fibonacci number, reproduced here with added type hints:

```
def fibonacci(n: int) -> int:
    """ Returns the nth Fibonacci number. """
    if n <= 0:
        return 0
    elif n == 1:
        return 1
    else:
        return fibonacci(n - 2) + fibonacci(n - 1)
```

This `fibonacci` function is correct, but it does not scale well—its execution time grows significantly as its parameter, n , increases. The problem is this: when computing a solution for a particular Fibonacci number the function can repeat exactly the same work multiple times. Figure 15.15 illustrates the repetitive work performed by the call `fibonacci(5)`. As we can see from the figure, the function's recursive execution calls `fibonacci(1)` five times during the computation of `fibonacci(5)`. We can verify the results shown in Figure 15.15 by augmenting our `fibonacci` function with a global dictionary that counts function calls. Listing 15.23 (`fibonacciinstrumented.py`) provides the complete program.

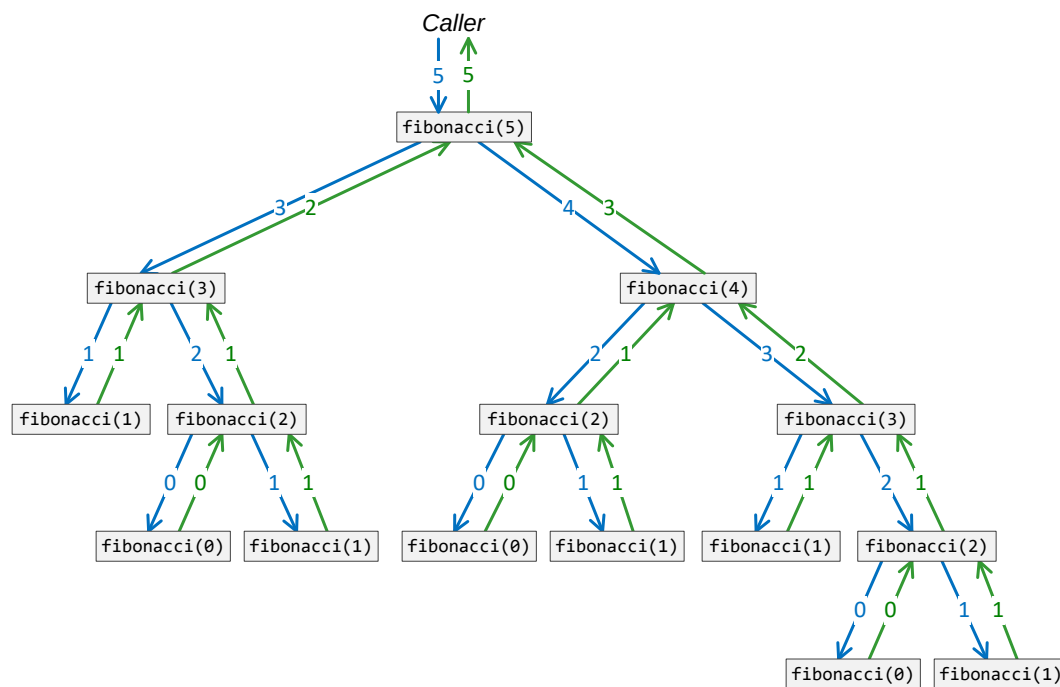
Listing 15.23: `fibonacciinstrumented.py`

```
# This dictionary will keep track of the number of calls to the
# fibonacci function.
call_counter = {}

def fibonacci(n: int) -> int:
    """ Returns the nth Fibonacci number. """
    # Count the call
    if n not in call_counter:
        call_counter[n] = 1
    else:
        call_counter[n] += 1

    if n <= 0:
        return 0
    elif n == 1:
        return 1
```

Figure 15.15 The hierarchy of recursive function calls that result from the call `fibonacci(5)`. Note that the recursion process calls `fibonacci(1)` five times.



```

    else:
        return fibonacci(n - 2) + fibonacci(n - 1)

# Call fibonacci(5)
print("fibonacci(5) = ", fibonacci(5))
print()

# Report the total number of calls to the fibonacci function
print("Argument    Calls")
print("-----")
for args, calls in sorted(call_counter.items()):
    print("    ", args, "    ", calls);

```

Listing 15.23 (fibonacciinstrumented.py) prints

```

fibonacci(5) = 5

Argument    Calls
-----
0           3
1           5
2           3
3           2
4           1
5           1

```

The results that Listing 15.23 (fibonacciinstrumented.py) prints agree exactly with the call count shown in Figure 15.15. As we compute larger Fibonacci numbers, the amount of repeated work worsens quickly; for example, the call `fibonacci(20)` recursively calls `fibonacci(1)` 6,765 times! For additional emphasis, the call `fibonacci(35)` recursively calls `fibonacci(1)`, `fibonacci(2)`, `fibonacci(3)`, `fibonacci(4)`, and `fibonacci(5)` over one million times *each*! We may be tempted to care less about the program’s repeated work—after all, it is the computer doing the work, not us. Unfortunately, the computer, even though it is very fast, requires some amount of time to perform any task. As we multiply the number of tasks a program must do to solve a problem, the time to compute the solution increases, and, in the case of `factorial`, the time increases dramatically.

We can improve the performance of our `fibonacci` function using a technique known as *memoization* (not to be confused with the word *memorization* which means to commit something to memory). Memoization is an algorithm design technique that records the result of a specific computation so that result can be used as needed at a later time during the algorithm’s execution. It is as if the executing program “makes a note to itself” or “stores the result in a memo.” When the program needs the result of an identical computation in the future, it simply reads the memo with the answer it stored earlier. In this way the program avoids repeating the work. Memoization is especially useful for problems that consist of subproblems that overlap and appear to require multiple computations with identical input.

Listing 15.24 (fibonacci.py) uses a dictionary to cache previously computed Fibonacci numbers. It demonstrates the dramatic speedup that memoization affords.

Listing 15.24: fibonacci.py

```

from stopwatch import Stopwatch
from random import randrange
from typing import Callable

```

```

def fibonacci(n: int) -> int:
    """ Returns the nth Fibonacci number recursively. """
    if n <= 0:
        return 0
    elif n == 1:
        return 1
    else:
        return fibonacci(n - 2) + fibonacci(n - 1)

# Dictionary for caching the results of the fibonacci2 function
ans = {0:0, 1:1} # Precompute the results for 0 and 1

def fibonacci2(n: int) -> int:
    """ Returns the nth Fibonacci number. Caches a
        recursively computed result to be used when needed
        in the future. Provides a huge performance improvement
        over the recursive version. """
    if n not in ans.keys():
        result = fibonacci2(n - 2) + fibonacci2(n - 1)
        ans[n] = result
    return ans[n]

def time_it(f: Callable[[int], int], ns: list[int]) -> float:
    """ f is a function that accepts a single parameter.
        ns is a list.
        Measures the time for function f to process each element in ns.
        Returns the cumulative elapsed time. """
    timer = Stopwatch()
    timer.start()
    for i in ns:
        print(f(i), end=' ')
    timer.stop()
    return timer.elapsed() # Return elapsed time

def main() -> None:
    """ Tests the performance of the fibonacci and fibonacci2 functions. """

    # Make a list of pseudorandom integers in the range 1...50
    numbers = []
    for i in range(10):
        numbers.append(randrange(40) + 1)

    # Print the numbers
    print(numbers)

    # Compare the two Fibonacci functions
    print(f'Time: {time_it(fibonacci, numbers):.8}')
    print('-----')
    print(f'Time: {time_it(fibonacci2, numbers):.8}')

```

```
if __name__ == "__main__":
    main()
```

One run of Listing 15.24 (fibonacci.py) produces the following output:

```
[29, 30, 32, 2, 3, 32, 29, 2, 38, 19]
514229 832040 2178309 1 2 2178309 514229 1 39088169 4181 Time: 13.800178
-----
514229 832040 2178309 1 2 2178309 514229 1 39088169 4181 Time: 0.0001108
```

Listing 15.24 (fibonacci.py) clearly demonstrates the power of memoization. The non-memoized version requires almost 14 seconds to compute the Fibonacci numbers for a list of 10 relatively small values, while the memoized version takes approximately $\frac{1}{10,000}$ of a second for the same list of values. The memoized version is over 124,000 times faster than the non-memoized version.

Figure 15.16 shows the recursion tree for our memoized Fibonacci function computing the fifth Fibonacci number. It assumes the `fibonacci2` has not previously been called, and so at the initial call at the top of the tree the global `ans` dictionary contains only the 0 and 1 keys. The figure shows only nine invocations of `fibonacci2`, compared to 15 invocations of the non-memoized `fibonacci` function. If you add the function call counting instrumentation used in Listing 15.23 (fibonacciinstrumented.py) to `fibonacci2`, you will find the numbers it reports agrees with Figure 15.16.

If our goal is simply to remove the redundant computations during recursion, we can make the caching dictionary local to a function, as shown here:

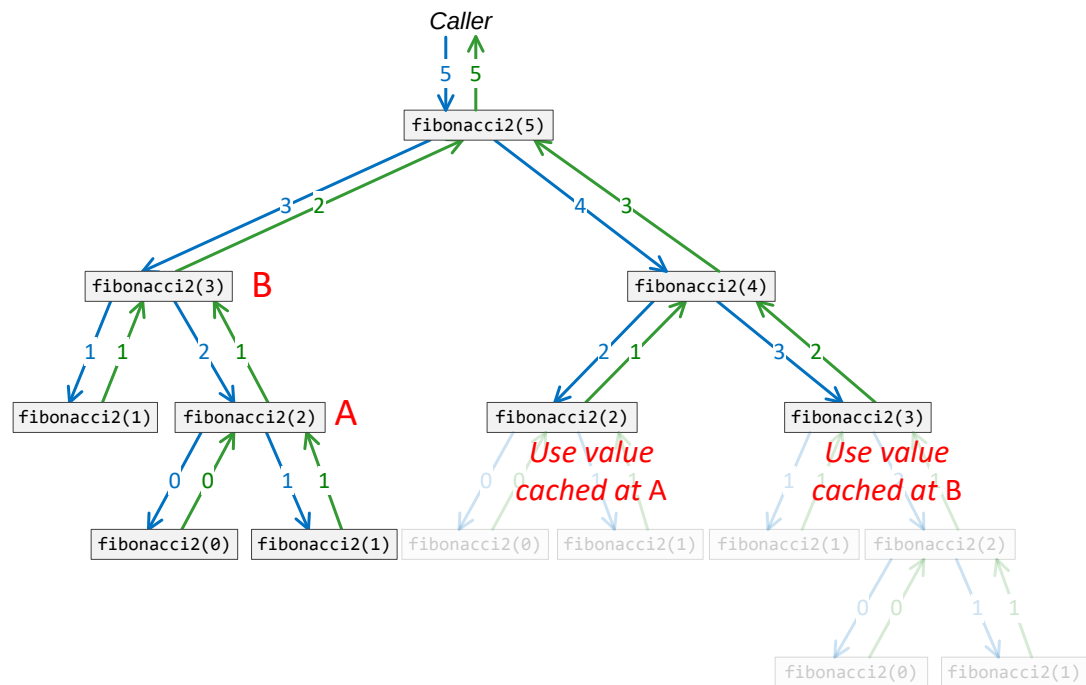
```
def fibonacci3(n: int) -> int:
    # Local dictionary for caching the results of the fib function
    ans = {0:0, 1:1} # Precompute the results for 0 and 1

    # Local function
    def fib(n: int) -> int:
        """ Returns the nth Fibonacci number. Caches a
            recursively computed result to be used when needed
            in the future. Provides a huge performance improvement
            over the recursive version. """
        nonlocal ans
        if n not in ans.keys():
            result = fib(n - 2) + fib(n - 1)
            ans[n] = result
        return ans[n]

    return fib(n)
```

Each call to `fibonacci3` starts with a fresh `ans` dictionary containing only the base case keys 0 and 1. It calls a local helper function, `fib`, which performs the recursion. The `fib` function avoids redundant computations using the `ans` dictionary to which it has access. If a program needs to compute only one Fibonacci number in the course of its execution, `fibonacci3` will be just as fast as `fibonacci2` when computing the same number. If, however, a program needs to compute Fibonacci numbers multiple times, `fibonacci2` may be the better choice. The global dictionary that `fibonacci2` uses maintains its contents between function calls, and so it is more likely to have the desired result memoized from an earlier invocation.

Figure 15.16 The hierarchy of recursive function calls that result from the call `fibonacci2(5)`. Note that the calls of `fibonacci(2)` and `fibonacci(3)` on the right side of the tree need no recursive calls; this is because their earlier calls on the left side of the tree stored their results in the global `ans` dictionary for instant retrieval.



One disadvantage of the global `ans` dictionary is that other functions within the program may accidentally or maliciously manipulate its contents causing `fibonacci2`'s results to be unreliable. To help avoid this problem you can put `fibonacci2` and `ans` into a separate module. Rename `ans` to `_ans` to make a point to callers that `_ans` is off limits to their code.

Another application of memoization is finding the *longest common subsequence* of two sequences. A *sequence* is simply an ordered list of elements, as in a Python list, tuple, or string. Most typically the elements of a sequence all will have the same type. A *subsequence* is a sequence formed from another sequence by removing elements without changing the relative order of the remaining elements. As an example, consider the sequence of characters in the Python list `['A', 'B', 'C', 'D']`. The complete set of subsequences is

```
[ ]
['A']
['B']
['A', 'B']
['C']
['A', 'C']
['B', 'C']
['A', 'B', 'C']
['D']
['A', 'D']
['B', 'D']
['A', 'B', 'D']
['C', 'D']
['A', 'C', 'D']
['B', 'C', 'D']
['A', 'B', 'C', 'D']
```

Note that the empty sequence is a subsequence of any sequence (even itself), and any sequence is its own subsequence.

The *longest common subsequence* (LCS) problem is this: given two sequences of symbols, find the longest subsequence that is common to both sequences. This problem is related to genome sequencing problems in computational biology. The LCS problem has a relatively simple recursive solution. We will use a Python string to represent a sequence, where each character is a symbol in the sequence. With this representation, the string `'ACD'` is a subsequence of `'ABCD'`. The following function computes the LCS of two strings:

```
def LCS(X: str, Y: str) -> str:
    """ Computes the longest common subsequence of strings X and Y. """
    result = '' # No symbols in common unless determined otherwise
    if len(X) > 0 and len(Y) > 0: # No symbols in common if either
        # string is empty
        Xrest = X[1:] # String X without its first symbol
        Yrest = Y[1:] # String Y without its first symbol
        if X[0] == Y[0]: # Do the first symbols of both strings match?
            # An LCS will include this shared symbol plus the LCS of rest
            # of both strings
            result = X[0] + LCS(Xrest, Yrest)
        else:
            # Compare string X to the string Y, excluding Y's first symbol
```

```

    X_Y1 = LCS(X, Yrest)
    # Compare string X, excluding X's first symbol, to string Y
    X1_Y = LCS(Xrest, Y)
    # Choose longer of the two computed sequences
    result = X_Y1 if len(X_Y1) > len(X1_Y) else X1_Y
return result

```

The LCS function is correct, but its recursive invocations suffer a similar fate to our original fibonacci function—they unnecessarily recompute multiple times the LCS of identical string pairs. As an example, the call `LCS('ABCBADAB', 'BDCABA')` will recompute recursively the LCS of the subsequences 'BDAB' and 'ABA' three times. These multiple, superfluous recomputations render this LCS function impractical for all but relatively short sequences.

The following `LCS_memoized` function adapts the original LCS function to use memoization:

```

def LCS_memoized(X: str, Y: str) -> str:
    """ Computes the longest common subsequence of strings X and Y.
        Caches intermediate results in a memoization dictionary so they
        do not need to be recomputed over and over again. """

    memo = {} # Start with empty memoization storage

    def LCS(X: str, Y: str) -> str:
        """ Computes the longest common subsequence of strings X and Y. """
        nonlocal memo # Memoization storage for computed results
        # Check first to see if we already have computed
        if (X, Y) in memo:
            result = memo[X, Y] # LCS of X and Y already computed,
                                # so use previous value
        else:
            result = '' # No symbols in common unless determined
                        # otherwise
            if len(X) > 0 and len(Y) > 0: # No symbols in common if
                                         # either string is empty
                Xrest = X[1:] # String X without its first symbol
                Yrest = Y[1:] # String Y without its first symbol
                if X[0] == Y[0]: # First symbols of both strings match
                    result = X[0] + LCS(Xrest, Yrest)
                else:
                    X_Y1 = LCS(X, Yrest)
                    X1_Y = LCS(Xrest, Y)
                    # Choose longest
                    result = X_Y1 if len(X_Y1) > len(X1_Y) else X1_Y
                # Remember this result for X and Y to avoid recomputing
                # it in the future
                memo[X, Y] = result
            return result

    result = LCS(X, Y)
    print(f'***** Stored memos: {len(memo)}')
    return result

```


Note that the memoization dictionary is local to `LCS_memoized`. Each time a caller invokes `LCS_memoized` the memoization dictionary begins empty. The memoization dictionary is valid only for the two specific strings that the caller passes. It would be unwise to make the memoization dictionary global; otherwise, if an application ran a long time repeatedly calling `LCS_memoized` with many different strings, a global dictionary would grow continuously. This unbounded growth potentially would exhaust the memory available to the application. The `LCS_memoized` function uses the dictionary merely to eliminate the redundant recursive recomputations.

Listing 15.25 (`lcsmemo.py`) tests out both the memoized and non-memoized versions of our LCS functions. It performs multiple runs on the same data to achieve more accurate measurements. It then plots the results using Turtle graphics.

Listing 15.25: `lcsmemo.py`

```
from stopwatch import Stopwatch # Our Stopwatch class from before
from random import choice
import turtle

def LCS(X: str, Y: str) -> str:
    """ Computes the longest common subsequence of strings X and Y. """

    result = '' # No symbols in common unless determined otherwise
    if len(X) > 0 and len(Y) > 0: # No symbols in common if
        # either string is empty
        Xrest = X[1:] # String X without its first symbol
        Yrest = Y[1:] # String Y without its first symbol
        if X[0] == Y[0]: # Do the first symbols of both strings match?
            # An LCS will include this shared symbol plus the LCS of rest
            # of both strings
            result = X[0] + LCS(Xrest, Yrest)
        else:
            # Compare string X to the string Y, excluding Y's first symbol
            X_Y1 = LCS(X, Yrest)
            # Compare string X, excluding X's first symbol, to string Y
            X1_Y = LCS(Xrest, Y)
            # Choose longer of the two computed sequences
            result = X_Y1 if len(X_Y1) > len(X1_Y) else X1_Y
    return result

def LCS_memoized(X: str, Y: str) -> str:
    """ Computes the longest common subsequence of strings X and Y.
        Caches intermediate results in a memoization dictionary so they
        do not need to be recomputed over and over again. """

    memo = {} # Start with empty memoization storage

    def LCS(X: str, Y: str) -> str:
        """ Computes the longest common subsequence of strings X and Y. """
        nonlocal memo # Memoization storage for computed results
        # Check first to see if we already have computed
        if (X, Y) in memo:
            result = memo[X, Y] # LCS of X and Y already computed,
                                # so use previous value
        else:
```

```

        result = '' # No symbols in common unless determined
                     # otherwise
        if len(X) > 0 and len(Y) > 0: # No symbols in common if
                                     # either string is empty
            Xrest = X[1:] # String X without its first symbol
            Yrest = Y[1:] # String Y without its first symbol
            if X[0] == Y[0]: # First symbols of both strings match
                result = X[0] + LCS(Xrest, Yrest)
            else:
                X_Y1 = LCS(X, Yrest)
                X1_Y = LCS(Xrest, Y)
                # Choose longest
                result = X_Y1 if len(X_Y1) > len(X1_Y) else X1_Y
        # Remember this result for X and Y to avoid recomputing
        # it in the future
        memo[X, Y] = result
    return result

result = LCS(X, Y)
print(f'***** Stored memos: {len(memo)}')
return result

def highlight(seq1: str, seq2: str) -> None:
    """ Highlights the characters of seq2 within seq1. """
    pos1, pos2 = 0, 0
    print(seq1) # Print the string on one line
    while pos1 < len(seq1):
        if pos2 < len(seq2) and seq1[pos1] == seq2[pos2]:
            print('^', end='') # Print '^' if characters match
            pos2 += 1
        else:
            print(' ', end='') # Print a space to move to the
                               # next position
        pos1 += 1
    print()

def compare_LCS(seq1: str, seq2: str) -> tuple[float, float]:
    """ Computes the longest common subsequence of seq 1 and seq2 in
        two different ways and compares the execution times of the two
        approaches. """
    timer_std = Stopwatch() # Each function has its own stopwatch
    timer_memo = Stopwatch() # to keep track of elapsed time independently
    timer_std.start() # Time the standard LCS function
    subseq_std = LCS(seq1, seq2)
    timer_std.stop()
    timer_memo.start() # Time the memoized LCS function
    subseq_memo = LCS_memoized(seq1, seq2)
    timer_memo.stop()
    # Report results
    print(seq1)
    print(seq2)
    print(f'{subseq_std} {len(subseq_std)} (Standard: {timer_std.elapsed():f})')
    print(f'{subseq_memo} {len(subseq_memo)} (Memoized: {timer_memo.elapsed():f})')

```

```

    print()
    # Show the computed longest common subsequences
    highlight(seq1, subseq_std)
    highlight(seq2, subseq_std)
    print()
    highlight(seq1, subseq_memo)
    highlight(seq2, subseq_memo)

    return timer_std.elapsed(), timer_memo.elapsed()

def build_random_string(n: int, symbols: str) -> str:
    """ Builds a string of length n with characters selected
        pseudorandomly from the string parameter symbols. """
    result = ''
    while len(result) < n:
        result = result + choice(symbols) # Add a random character
                                           # from symbols
    return result

def plot(data: list[tuple[float, float]]) -> None:
    """ Plots the data from the LCS vs. Memoized LCS experiments. """

    # Compute bounds for the graphing
    w = turtle.window_width()
    h = turtle.window_height()
    x_origin = -w/2 + 15
    y_origin = -h/2 + 15

    xrange = len(data) # Number of data points
    # Determine the range of y values
    yrange = max(max(data, key=lambda d0: d0[0])[0],
                 max(data, key=lambda d1: d1[1])[1])

    # These scaling factors ensure the points will fit onto the plot
    xscale = 0.9*w/xrange
    yscale = 0.9*h/yrange

    def plot_curve(index: int, color: str) -> None:
        """ Local function to plot data from one of the functions.
            index refers to the slice of the data.
            color is the line and point color. """
        turtle.penup()
        turtle.pencolor(color)
        turtle.setposition(x_origin, y_origin)
        turtle.pendown()
        for length, times in enumerate(data):
            turtle.setposition(x_origin + xscale*length,
                              y_origin + yscale*times[index])

            turtle.dot()
            #print('Plotting', length, x_origin + xscale*length,
            #      y_origin + yscale*times[index])

```

```

turtle.hideturtle()  # Do not show the pen
turtle.delay(0)      # No need to animate the drawing

turtle.title('Longest Common Subsequence vs. Memoized LCS')
# Draw axes
turtle.pencolor('black')
turtle.pensize(3)
turtle.penup()

# x axis
turtle.setposition(x_origin, y_origin)
turtle.pendown()
turtle.forward(w - 30)
turtle.penup()
# y axis
turtle.setposition(x_origin, y_origin)
turtle.left(90)
turtle.pendown()
turtle.forward(h - 30)
turtle.pensize(2)

# Draw first curve
plot_curve(0, 'blue')

# Draw second curve
plot_curve(1, 'red')

turtle.exitonclick()

def main() -> None:
    N = 10          # Number of times to perform a series
                   # of experiments
    max_length = 18 # Maximum string length for experiments

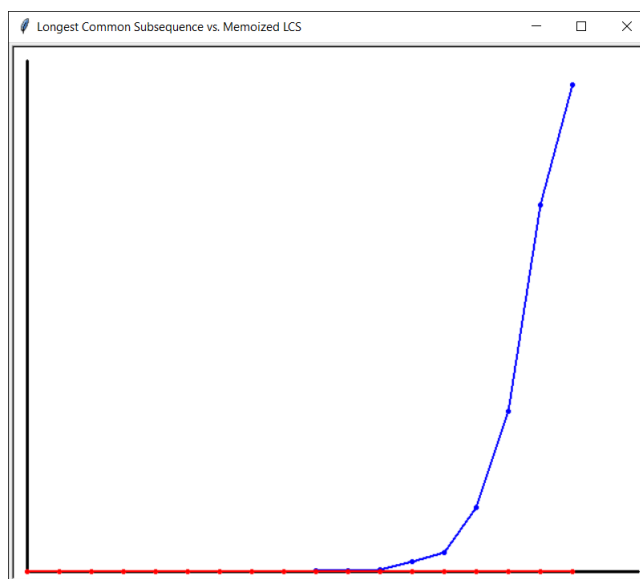
    # Make a list of tuples for measuring the time to compute the
    # LCS two different ways; initialize all the times to zero
    data = [(0.0, 0.0) for _ in range(max_length)]

    # Perform the series of experiments N times to produce
    # more reliable results
    for runs in range(N):
        # The series of experiments consists of computing the LCS
        # of two strings of increasing length
        for i in range(max_length):
            print(f'>>> Run {runs} String length {i}')
            # Construct two random DNA base sequences
            seq1 = build_random_string(i, 'ACGT')
            seq2 = build_random_string(i, 'ACGT')

            # Perform an experiment to time the two LCS functions
            lcs_time, memo_time = compare_LCS(seq1, seq2)
            # Accumulate the times
            data[i] = data[i][0] + lcs_time, data[i][1] + memo_time

```

Figure 15.17 A graph produced by one run of Listing 15.25 (lcsmemo.py). The horizontal axis represents the length of the strings, increasing to the right. The vertical axis represents the program's running time. The blue line plots the accumulated time consumed by the non-memoized function, and the red line plots the accumulated time of the memoized version of the function. Note that difference in execution times increases dramatically as strings become longer, and the memoized version continues to perform well even with longer strings.



```
print(data)
plot(data)

if __name__ == '__main__':
    main()
```

Figure 15.17 shows the graph plotted by one run of Listing 15.25 (lcsmemo.py).

15.9 Exercises

1. Complete the following function that reorders the contents of a list so they are reversed from their original order. For example, a list containing the elements 2, 6, 2, 5, 0, 1, 2, 3 would be transformed into 3, 2, 1, 0, 5, 2, 6, 2. Note that your function must physically rearrange the elements within the list, not just print the elements in reverse order.

```
def reverse(lst):
    # Add your code...
```

2. Complete the following function that reorders the contents of a list of integers so that all the even numbers appear before any odd number. The even values are sorted in ascending order with respect to themselves, and the odd numbers that follow are also sorted in ascending order with respect to

themselves. For example, a list containing the elements 2, 1, 10, 4, 3, 6, 7, 9, 8, 5 would be transformed into 2, 4, 6, 8, 10, 1, 3, 5, 7, 9. Note that your function must physically rearrange the elements within the list, not just print the elements in the desired order.

```
def special_sort(lst):
    # Add your code...
```

3. Create a special comparison function to be passed to our flexible selection sort function. The special comparison function should enable the sort function to arrange the elements of a list in the order specified in Exercise 2.
4. Complete the following function that filters negative elements out of a list. The function returns the filtered list and the original list is unchanged. For example, if a list containing the elements 2, -16, 2, -5, 0, 1, -2, -3 is passed to the function, the function would return the list containing 2, 2, 0, 1. Note the original ordering of the nonnegative values is unchanged in the result.

```
def filter(a):
    # Add your code...
```

5. Complete the following function that shifts all the elements of a list backward one place. The last element that gets shifted off the back end of the list is copied into the first (0th) position. For example, if a list containing the elements 2, 1, 10, 4, 3, 6, 7, 9, 8, 5 is passed to the function, it would be transformed into 5, 2, 1, 10, 4, 3, 6, 7, 9, 8. Note that your function must physically rearrange the elements within the list, not just print the elements in the shifted order.

```
def rotate(lst):
    # Add your code...
```

6. Complete the following function that determines if the number of even and odd values in an integer list is the same. The function would return true if the list contains 5, 1, 0, 2 (two evens and two odds), but it would return false for the list containing 5, 1, 0, 2, 11 (too many odds). The function should return true if the list is empty, since an empty list contains the same number of evens and odds (0 for both). The function does not affect the contents of the list.

```
def balanced(a):
    # Add your code...
```

7. Complete the following function that returns true if a list `lst` contains duplicate elements; it returns false if all the elements in `lst` are unique. For example, the list `[2, 3, 2, 1, 9]` contains duplicates (2 appears more than once), but the list `[2, 1, 0, 3, 8, 4]` does not (none of the elements appear more than once).

An empty list has no duplicates. The function does not affect the contents of the list.

```
def has_duplicates(lst):
    # Add your code...
```

8. Can linear search be used on an unsorted list? Why or why not?
9. Can binary search be used on an unsorted list? Why or why not?
10. How many different orderings are there for the list `[4, 3, 8, 1, 10]`?

11. Complete the following function that determines if two lists contain the same elements, but not necessarily in the same order. The function would return true if the first list contains 5, 1, 0, 2 and the second list contains 0, 5, 2, 1. The function would return false if one list contains elements the other does not or if the number of elements differ. This function could be used to determine if one list is a permutation of another list. The function does not affect the contents of either list.

```
def is_permutation(a, b):  
    # Add your code...
```

12. Listing 15.14 (listpermutations.py) contains the following statement:

```
result += [lst[:]]
```

- (a) What happens if you change the statement to the following?

```
result += [lst]
```

- (b) Explain why this modified statement produces the result that it does.

13. Write a nonrecursive version of the `rev` function shown in Listing 15.21 (listreverse.py).
14. Write an function that reverses a list in place. Do not use the `reverse` method.

Chapter 16

Representing Relationships with Graphs

Python dictionaries are ideal for representing relationships among things. Much of what modern civilization uses computers to do involves managing relationships at some level. Mathematical *graph theory* models such relationships. This chapter introduces graph algorithms useful to computer scientists.

16.1 Introduction to Graphs

Figure 16.1 shows the routes between cities for a particular airline. The circles in the figure represent airports, and the associated labels represent airport code names (for example, ATL is Hartsfield-Jackson Atlanta International Airport near Atlanta, Georgia). A line connecting two circles indicates a direct nonstop flight between two airports.

The circles and lines make up a *mathematical graph*, or simply *graph*. A graph represents relationships among things. A circle on the graph is known as a *vertex* (plural: *vertices*), or *node*. A line connecting two vertices is called an *edge*, or *arc*. Two vertices connected directly by an edge are *adjacent*; two vertices not connected directly by an edge are *non-adjacent*. Two adjacent vertices are related to each other in some way; for example, in the airline routes graph two airports are related if a direct flight between them exists.

Graph theory is a distinct area within mathematics that studies graphs. Graph theory is concerned with connections amongst vertices. Figure 16.2 shows the bare graph with the background map removed. From the mathematician's point of view, the positioning of the vertices in a graph visualization is irrelevant, so the graph in Figure 16.3 is identical to the graph in Figure 16.2.

In a *directed graph* (or *digraph* for short) the edges have a direction; that is, vertex *a* may be adjacent to vertex *b* with vertex *b* not being adjacent to vertex *a*. Figure 16.4 visualizes a directed graph. In the figure the arrow pointing from vertex *d* to vertex *f* indicates that *d* is related to *f*. This means *f* is adjacent to *d*, but observe that *d* is not adjacent to *f*. Since edges connect *c* and *d* both ways, *c* is adjacent to *d*, and *d* is adjacent to *c*.

Mathematical graphs are useful for representing a large number of practical, real-world problems. Developers can model these problems in software using concepts from graph theory and implement solutions based on graph theoretic algorithms. For this reason, graph theory is of particular interest to computer scientists.

Figure 16.1 A map showing the routes between cities for an airline. The circles in the figure represent airports, and the associated labels are airport code names (for example, ATL is Hartsfield-Jackson Atlanta International Airport near Atlanta, Georgia). A line connecting two circles indicates a direct nonstop flight between two airports. (The image of the United States map is adapted from https://commons.wikimedia.org/wiki/File:Blank_US_Map.svg.)

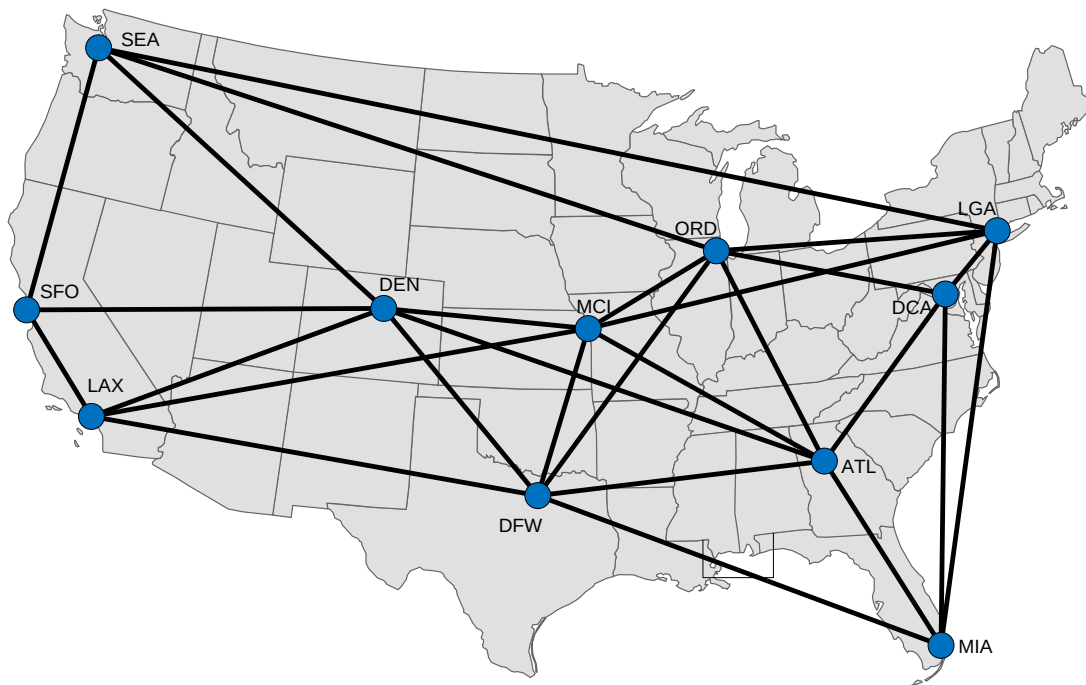


Figure 16.2 The airline routes graph with the background map removed.

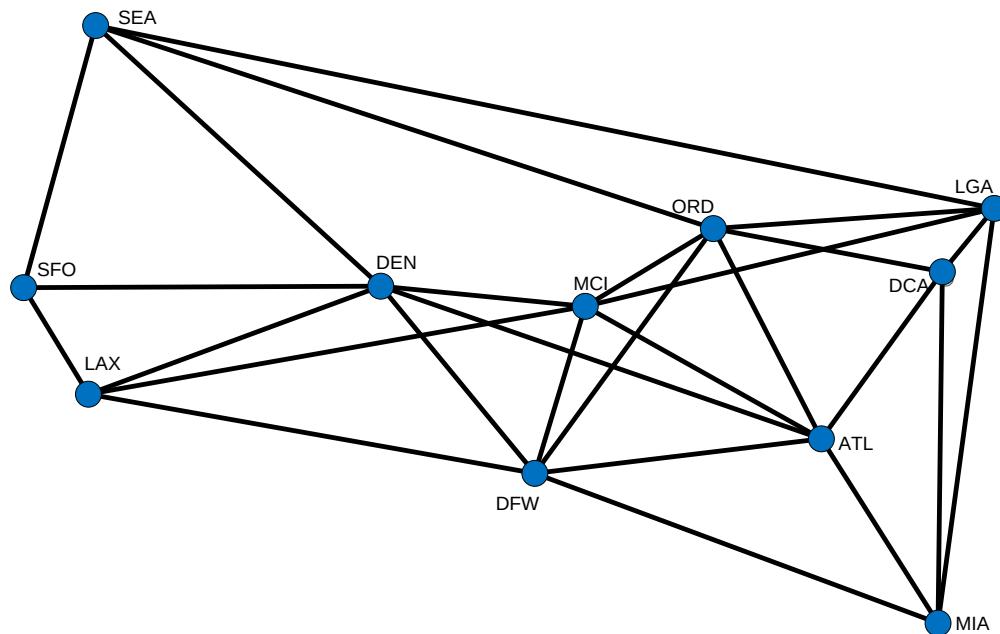


Figure 16.3 In mathematic graph theory, the positioning of vertices in a visualization of the graph is irrelevant. This graph is equivalent to the graph shown in Figure 16.2.

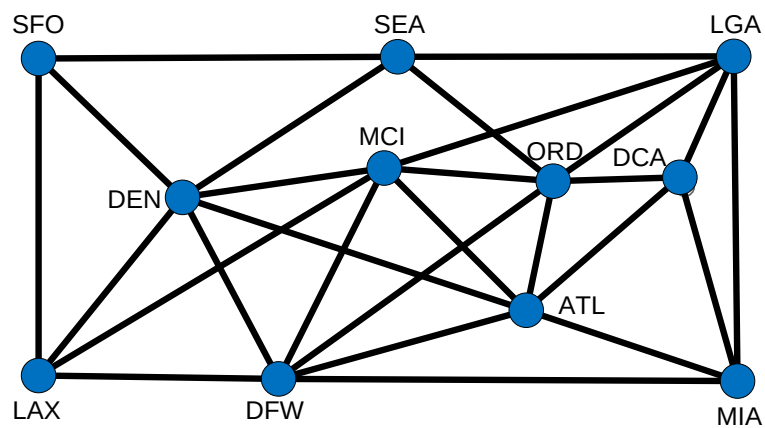
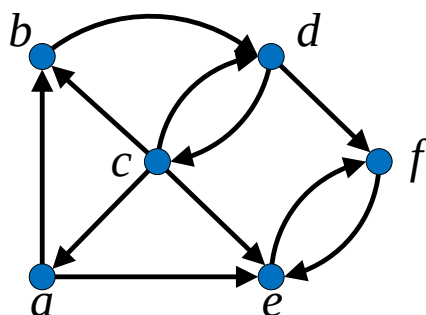


Figure 16.4 A visualization of a directed graph. Note that arrows represent directed edges.



Some applications of graph theory include:

- **Transportation.** Besides airline routes, a global positioning system (GPS) can compute a driving route from one location to another. An intersection is a vertex in this case, and a road connecting one intersection to another is an edge. Graph algorithms can determine the best route, where best can mean shortest traveling time, shortest distance (mileage), or fewest traffic hazards.
- **Communication.** A person uses a telephone to call another person. The caller and callee are vertices, and the call that connects them is an edge.
- **Games.** A game such as chess or checkers involves moving a playing piece from one square to another. Only certain moves are legal. When representing a game configuration as a graph, a vertex represents a square, and an edge represents a legal move to another square (to occupy an empty square or to capture an opponent's piece).

On a larger scale, the complete board configuration (the locations of all the pieces for both players) can be a vertex, and a legal move (edge) will result in a different board configuration (vertex). A computer program can consider all valid moves from a particular board configuration many turns ahead to determine the best current move for a player.

- **Degree requirements.** A university degree for a particular major requires a number of courses. Some of the courses may have prerequisite courses; for example, typically a student wishing to enroll in a second, more advanced computer programming course must first complete an introductory computer programming course. In a prerequisite graph, every course is a vertex, and a directed edge connects a course to its prerequisite course(s).
- **Social media.** A vertex represents a person, and an edge represents *friendship*, connecting one person to another.
- **Web.** A web page is a vertex, and a hyperlink is an edge connecting one web page to another. The content of the World Wide Web is an immense directed graph.
- **Chemistry.** Atoms comprise molecules. The vertices of a graph can model the atoms of a molecule, and the graph's edges can represent the bonds between the atoms.

- **Computer programming.** A flowchart like the one shown in Figure 5.1 is a directed graph. Each geometric shape is a vertex, and each arrow that connects one shape to another is an edge.

These examples are only a few of the many problem domains that mathematical graphs naturally model. Graph algorithms are a significant resource to software developers faced with the task of tackling problems in these areas and many others.

16.2 Implementing Graphs in Python

Python does not have a native graph data structure; however, we can easily model a mathematical graph in Python using a dictionary and sets. The keys in the dictionary represent the vertices of the graph, and each key maps to a set. The set contains the vertices adjacent to the key. Using this technique we can express the graph in Figure 16.3 as

```
# Dictionary representing the graph of airline routes
routes = {
    "ATL": {"MIA", "DCA", "ORD", "MCI", "DFW", "DEN"},
    "MIA": {"LGA", "DCA", "ATL", "DFW"},
    "DFW": {"LAX", "DEN", "MCI", "ORD", "ATL", "MIA"},
    "LAX": {"SFO", "DEN", "MCI", "DFW"},
    "DEN": {"SFO", "LAX", "MCI", "DFW", "SEA", "ATL"},
    "SEA": {"SFO", "DEN", "ORD", "LGA"},
    "MCI": {"DEN", "LAX", "DFW", "ATL", "ORD", "LGA"},
    "ORD": {"SEA", "MCI", "DFW", "ATL", "DCA", "LGA"},
    "DCA": {"ORD", "ATL", "MIA", "LGA"},
    "LGA": {"SEA", "MCI", "ORD", "DCA", "MIA"},
    "SFO": {"SEA", "DEN", "LAX"}
}
```

Given this representation we can print all the airports with a direct connection from ATL as

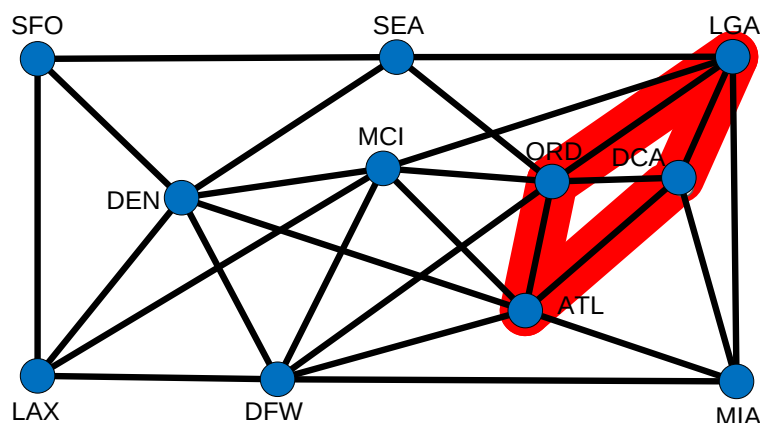
```
for airport in routes["ATL"]:
    print(airport)
```

Since a Python `set` holds the collection of adjacent airports, the statement above most likely will print the airports out in a different order than they appear in the source code assigning `routes`. If in a particular application the ordering of adjacent vertices is important, use a Python list instead of a set. For most problems, however, a set is the preferred adjacency structure. Counterintuitively, the formal name for this kind of graph representation is *adjacency list* (see https://en.wikipedia.org/wiki/Adjacency_list), where *list* as used here means an unordered collection, or set. Python reserves the name `list` for an ordered linear collection.

16.3 Path Finding

Computer scientists and mathematicians have developed a number of algorithms that solve common problems in graph theory. One problem is finding a path from one vertex to another within a graph. From our airline example graph in Figure 16.3, ATL–DEN–SFO–SEA represents a path from ATL to SEA. Another path from ATL to SEA is ATL–MIA–DFW–MCI–LGA–ORD–SEA. Since no direct flight exists between ATL and SFO, the sequence ATL–SFO–SEA is *not* a path in the Figure 16.3 graph.

Figure 16.5 A cycle in the airline graph. Note the path ATL–DCA–LGA–ORD–ATL. This path begins and ends at the same vertex.



Generally, shorter paths are desirable, as a shorter path usually translates into some kind of savings, such as shorter travel time or less fuel consumed. Some graphs have values attached to each edge that represent a cost such as time or distance, making some edges more expensive than others. We will keep things simple here and treat all edges equally. This means one path is shorter than another path if it visits fewer vertices. The length of a path is the number of edges used in the path, so the path length of ATL–MIA–DFW–MCI–LGA–ORD–SEA is 6. So, while ATL–MIA–DFW–MCI–LGA–ORD–SEA surely gets a passenger from from ATL to SEA, the path ATL–DEN–SFO–SEA is shorter. (Can you find an even shorter path from ATL to SEA in Figure 16.3?)

Our goal is to find a shortest path from one vertex to another in a graph. Notice that we say *a* shortest path rather than *the* shortest path, as several different paths may tie for the shortest length; for example, consider the following paths from MIA to SFO:

MIA–ATL–DEN–SFO
 MIA–DFW–LAX–SFO
 MIA–DFW–DEN–SFO
 MIA–LGA–SEA–SFO

All of these paths have length 3, and no path from MIA to SFO is shorter than 3. In general, there should be no paths connecting two vertices shorter than a shortest path between the vertices.

In order to find a shortest path in a graph, the algorithm must traverse the graph. Graph traversal is more complicated than list traversal. Traversing a list is easy: begin at the first element (index 0) and visit each successive element (element at current index plus one), in order, until locating the desired element or running out of elements to visit. A graph, however, is not a linear structure; it is a network of interconnected elements, and an element (vertex) can have multiple successors (zero or more adjacent vertices). If our traversal algorithm is not careful, it could, for example, begin at vertex ATL and follow the path ATL–DCA–LGA–ORD–ATL. Figure 16.5 shows a graph with a cycle.

Such a path that begins and ends at the same vertex is known as a *cycle*. A naïve algorithm could continue the traversal following the exact same path as shown here:

ATL–DCA–LGA–ORD–ATL–DCA–LGA–ORD–ATL–DCA–LGA–ORD–ATL–...

The algorithm effectively becomes stuck in the cycle, revisiting the same vertices over and over. Unable to escape the cycle, the algorithm cannot visit any vertices outside of this cycle.

To avoid getting stuck in a cycle our traversal algorithm must have a way to remember which vertices it has visited so it does not attempt to revisit them. Our path finding algorithm will add a visited vertex to a dictionary. The vertex serves as a key, and its associated value will be the vertex that immediately precedes it on a shortest path from the starting vertex to the destination vertex. During its traversal of the graph, the algorithm will check this dictionary before attempting to visit a vertex. If the vertex is not in the dictionary, the algorithm “visits” the vertex by adding it (with its predecessor on a shortest path) to the dictionary. If the vertex is already present in the dictionary, the algorithm ignores the vertex and continues its traversal through other available vertices in the graph.

The second problem for our path finding algorithm is not merely to find *a* path from the starting vertex to the ending vertex but to find a *shortest* path. Remember that an algorithm cannot see the graph globally as we can when we look at its visual representation. The algorithm must be clever in its choice of which vertex to visit next as it traverses the graph. This choice is crucial because once the algorithm adds a vertex to the dictionary of visited vertices it can never go back and revisit the vertex to determine if choosing a different adjacent vertex would result in a shorter path. (As noted above, if it attempts to revisit a vertex, the algorithm could get stuck in a cycle, retracing the same path over and over again.)

16.4 Breadth-first Search

There are a number path finding algorithms, and Listing 16.1 (`airlineroute.py`) uses one such algorithm, *breadth-first search* (BFS), to compute a shortest path from one vertex to another (see https://en.wikipedia.org/wiki/Breadth-first_search). The BFS algorithm uses a *queue* to guide the algorithm’s progress. A queue is a first-come, first-serve data structure. It is similar to a list, in that a queue is a linear sequence of elements, and clients can add elements to, and remove elements from, the sequence. Unlike with a list, however, element addition and removal in a queue is restricted: clients may remove only the most recent element added to the queue. A queue thus works like a line of customers waiting at a checkout counter in a store. The cashier serves the person at the front of the line first, and customers who are ready to check out but not in line must join the end of the line and wait for their turn. Conceptually, we can place items on the back of a queue only, and we can remove items only from the front of a queue.

Fortunately, the Python standard library offers a `Queue` class in a module named `queue`. Our implementation of the BFS algorithm will use a `Queue` object, and it will use three methods that the `Queue` class provides:

- `put` places an element onto the rear of the queue,
- `get` serves an element from the front of the queue, and
- `empty` returns `True` if the queue is empty and `False` if it contains any elements.

The `put` and `get` methods modify the queue, but the `empty` method does not disturb the contents, if any, of a queue object.

We can practice with a `Queue` object in the interactive interpreter:

```

>>> from queue import Queue
>>> q = Queue()
>>> q.empty()
True
>>> q.put(34)
>>> q.empty()
False
>>> q.put("Hello")
>>> q.put(12)
>>> q.put([10, 20, 30])
>>> q.get()
34
>>> q.get()
'Hello'
>>> q.get()
12
>>> q.empty()
False
>>> q.get()
[10, 20, 30]
>>> q.empty()
True

```

As you can see, what goes into the queue first comes out first, and what goes in last comes out last.

Just as a queue organizes the service of customers at a grocery checkout so that the cashier serves earlier arrivals before later ones, the proper use of a queue ensures the BFS algorithm visits vertices in a graph in an order that guarantees it will find a shortest path between two vertices. BFS proceeds as follows:

Inputs: A starting vertex and destination vertex within a graph.

Begin with an empty dictionary and an empty queue.

Add the starting vertex to the queue.

while the queue is not empty and the dictionary does not contain the destination vertex as a key:

 Serve the next vertex, v , from the queue.

for each vertex w adjacent to v :

if w is not in the dictionary:

 Add w to the queue.

 Add key w with its value v to the dictionary.

Output: The dictionary with the keys consisting vertices reachable from the starting vertex; the value of a key is the vertex that immediately precedes it on a shortest path from the starting vertex to the destination vertex.

The first time through its **while** loop the BFS algorithm extracts the starting vertex from the queue. The **for** loop then considers every vertex adjacent to the starting vertex. These adjacent vertices go into the queue. Note that all of these vertices added to the queue are a distance of 1 away from the starting vertex and record the starting vertex as their predecessor. On the second iteration of its **while** loop the algorithm removes one vertex from the queue. This has to be one of the vertices it added on its first iteration—a vertex adjacent to the starting vertex. Call this vertex v . The algorithm adds any nonvisited vertices adjacent to v to the queue. Note that all these nonvisited vertices adjacent to v are a distance of 2 from the starting vertex. Because it is using a queue the algorithm cannot visit these newly added vertices until it processes

the vertices added to the queue earlier. This means the algorithm visits all the vertices that are a distance of 1 from the starting vertex before it attempts to visit any vertices at a distance of 2 away from the starting vertex. Unless it encounters the destination vertex beforehand, the algorithm eventually will extract from the queue a vertex at a distance of 2 from the starting vertex, adding all of that vertex's unvisited vertices (distance 3) to the back of the queue. Consequently, all distance 2 vertices will be ahead of distance 3 vertices in the queue, and so the algorithm visits all distance 2 vertices before visiting any distance 3 vertex. The `while` loop continues until the queue is empty or the algorithm reaches the destination vertex. In this way the algorithm visits all distance 1 vertices before any distance 2 vertex, all distance 2 vertices before any distance 3 vertex, all distance 3 vertices before any distance 4 vertex, and so forth. Figure 16.6 illustrates with an example BFS traversal. The queue ensures that the next vertex visited in the `while` loop is no farther away from the starting vertex than any other yet-to-be visited vertex. Expanding the search to a more distant vertex only after all nearer vertices have been considered is the trick to discovering a shortest path in the graph from the starting vertex to the destination vertex.

From the dictionary that this BFS algorithm produces we can construct a shortest path by following the predecessor links back from the destination vertex as described in the following procedure:

Inputs: A starting vertex and destination vertex within a graph.

Begin with an empty path.

Use the BFS algorithm to compute the dictionary containing vertex predecessors on a shortest path from the starting vertex to the destination vertex.

`if` the destination vertex is in the dictionary:

 Add the destination vertex to the path.

 Set v to be the destination vertex.

`while` the starting vertex is not in the path:

 Update v to be v 's predecessor from the dictionary.

 Add v to the front of the path.

Output: The path that contains, in order, the vertices found on a shortest path from the starting vertex to the destination vertex; an empty path indicates that the destination vertex is not reachable from the starting vertex.

Listing 16.1 (`airlineroute.py`) implements the BFS and path building algorithms in Python. The `bfs` function faithfully implements the breadth-first search traversal algorithm, building and returning a dictionary containing vertices, each paired with its immediate predecessor on a shortest path from the starting vertex. The `find_path` function implements the second algorithm from above that uses the result of `bfs` to construct a list of vertices, in order, on a shortest path from the starting vertex to the destination vertex.

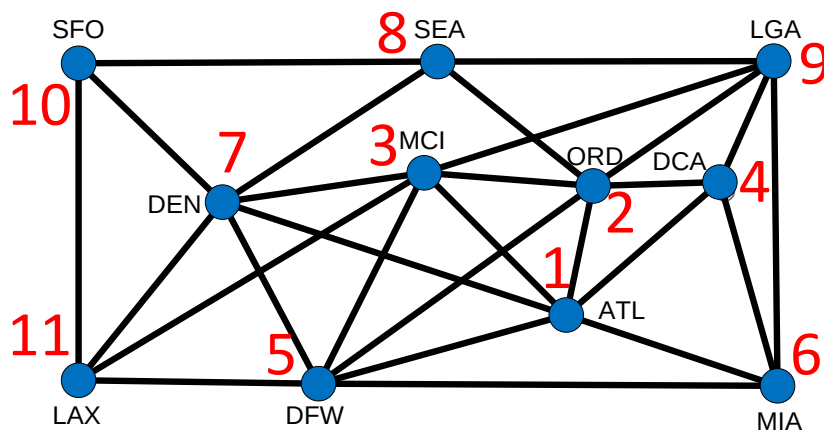
The graph processed by Listing 16.1 (`airlineroute.py`) is shown in Figure 16.7. This graph adds three new airports to Figure 16.3: BNA, CHA, and CLT. These new vertices are interconnected to each other, but they are not connected to any of the vertices from the original graph. (In the real world you can get from CHA [Chattanooga, Tennessee], for example, to DEN, but this contrived graph illustrates the general case where a path may not exist between two vertices in a graph.)

Listing 16.1: `airlineroute.py`

```
from queue import Queue

# A graph is a dictionary mapping a vertex label to a set
# of adjacent vertex labels
```


Figure 16.6 The numbers indicate a possible order in which BFS visits vertices starting at vertex ATL. Since a Python program uses a dictionary to represent the graph, the exact order of the visitation is unpredictable, even from one execution to the next on the same machine. What is predictable, however, is that BFS will visit all vertices at a distance of n from the starting vertex before it visits any vertex at a distance greater than or equal to $n + 1$ from the starting vertex. For example, the shortest distance between ATL and DEN is 1 and the shortest distance between ATL and LAX is 2; this means the number associated with DEN must be less than the number associated with LAX. Consequently, BFS from ATL necessarily would visit DEN before visiting LAX.



```

Graph = dict[str, set[str]]

def bfs(graph: Graph, start_vertex: str, end_vertex: str) -> dict[str, str]:
    """ Performs a breadth-first traversal of graph originating
        at start_vertex and ending when the traversal encounters
        end_vertex. Builds and returns a dictionary with
        vertices mapped to their immediate predecessors on the
        breadth-first traversal. """

    # A dictionary of vertex predecessors encountered on a breadth-first
    # traversal from start_vertex to end_vertex.
    # The dictionary key is a vertex, and the associated value is
    # the vertex that comes immediately before the key on a
    # breadth-first traversal.
    # This dictionary initially is empty, and we will add vertices as
    # we "visit" them during the breadth-first traversal.
    predecessor = {}

    # Make an empty queue and insert the starting vertex
    # The algorithm will extract and visit vertices from this queue
    q = Queue()
    q.put(start_vertex)

    # Keep searching while the queue holds vertices yet to visit
    # and we have yet to visit our destination vertex
    while not q.empty() and end_vertex not in predecessor:
        vertex = q.get() # Get vertex on the front of the queue
        for adjacent_vertex in graph[vertex]: # Consider all adjacents
            # Has the predecessor of this vertex been established?
            if adjacent_vertex not in predecessor:
                q.put(adjacent_vertex) # Enqueue the vertex
                # Register which vertex should come immediately before this
                # one on a shortest path (this "visits" the vertex)
                predecessor[adjacent_vertex] = vertex

    # At this point we exited the while loop because either the queue
    # was empty or the destination vertex now is in the predecessor
    # dictionary (or both). If the queue is empty but the destination
    # vertex is not in the predecessor dictionary, the destination vertex
    # is unreachable from the starting vertex. If the queue is not empty
    # but the destination vertex is in the predecessor dictionary, the
    # path from the starting vertex to the destination vertex exists and
    # excludes one or more vertices in the graph. If the queue is empty
    # and the destination vertex is in the predecessor dictionary, the
    # shortest path from the starting vertex to the destination vertex
    # includes all vertices reachable from the starting vertex.
    return predecessor

def find_path(graph: Graph, start_vertex: str, end_vertex: str) -> list[str]:
    """ Builds a list of vertices in order along the shortest path
        from a starting vertex to an ending vertex in a graph.
        Returns an empty list if no such exists.
        graph: The graph to traverse.
        start_vertex: The starting vertex.

```

```

    end_vertex: The vertex to locate. """

    # Compute predecessor of each vertex on a shortest path
    # Call the bfs function to build the predecessor dictionary
    predecessor = bfs(graph, start_vertex, end_vertex)

    path = []          # Path initially empty

    # Check that we were able to reach the destination vertex
    if end_vertex in predecessor:
        # Start at the end and work backwards
        path = [end_vertex]
        vertex = end_vertex
        # Keep going until we get to the beginning of the path
        while vertex != start_vertex:
            # Get vertex that comes immediately before on a shortest path
            vertex = predecessor[vertex]
            # Prepend the predecessor vertex to the front of the path list
            path = [vertex] + path

    return path

def main() -> None:
    """ Tests the find_path function by searching for routes between
        airports. """

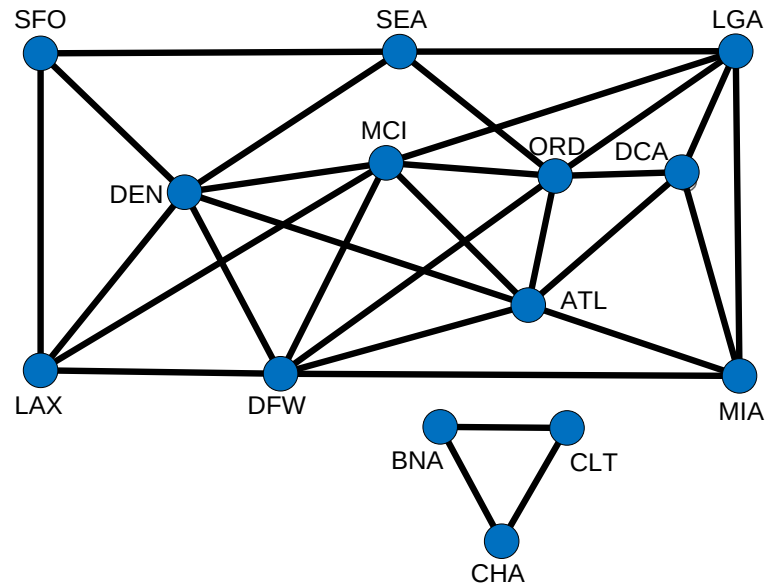
    # Dictionary representing the graph of airline routes
    routes = {
        'ATL': {'MIA', 'DCA', 'ORD', 'MCI', 'DFW', 'DEN'},
        'MIA': {'LGA', 'DCA', 'ATL', 'DFW'},
        'DFW': {'LAX', 'DEN', 'MCI', 'ORD', 'ATL', 'MIA'},
        'LAX': {'SFO', 'DEN', 'MCI', 'DFW'},
        'DEN': {'SFO', 'LAX', 'MCI', 'DFW', 'SEA', 'ATL'},
        'SEA': {'SFO', 'DEN', 'ORD', 'LGA'},
        'MCI': {'DEN', 'LAX', 'DFW', 'ATL', 'ORD', 'LGA'},
        'ORD': {'SEA', 'MCI', 'DFW', 'ATL', 'DCA', 'LGA'},
        'DCA': {'ORD', 'ATL', 'MIA', 'LGA'},
        'LGA': {'SEA', 'MCI', 'ORD', 'DCA', 'MIA'},
        'SFO': {'SEA', 'DEN', 'LAX'},
        'CLT': {'BNA', 'CHA'},
        'BNA': {'CLT', 'CHA'},
        'CHA': {'CLT', 'BNA'}
    }

    # Attempt to find a route from one airport to another
    print(find_path(routes, 'LAX', 'DCA'))
    print(find_path(routes, 'MIA', 'SFO'))
    print(find_path(routes, 'ATL', 'MIA'))
    print(find_path(routes, 'LGA', 'LGA'))
    print(find_path(routes, 'CLT', 'BNA'))
    print(find_path(routes, 'BNA', 'ATL'))

if __name__ == "__main__":

```

Figure 16.7 An extension of the graph in Figure 16.2 that includes three extra vertices. None of the new vertices connect to any the vertices in the original graph.



```
main()
```

One run of Listing 16.1 (airlineroute.py) prints

```
['LAX', 'DEN', 'ATL', 'DCA']
['MIA', 'LGA', 'SEA', 'SFO']
['ATL', 'MIA']
['LGA']
['CLT', 'BNA']
[]
```

The final line of the program's output prints an empty list indicating that no path exists from BNA to ATL. A graph in which a path exists between any two vertices is known as a *connected graph*. The graph shown in Figure 16.3 is a connected graph. A graph that is not connected, like Figure 16.7, is a *disconnected graph*.

Suppose we wish to determine if a graph is connected; that is, does there exist a path between any two vertices in the graph? Given our `bfs` function from Listing 16.1 (airlineroute.py), the following function would work:

```
def is_connected(G):
    """ G is a dictionary that represents a graph. """
    for v in G: # Examine each dictionary key (vertex)
        for w in G: # Examine each dictionary key (vertex)
            if v != w and w not in bfs(G, v, w):
                return False # Vertex w not reachable from vertex v
```

```
return True # All vertices in G are reachable from any vertex in G
```

This function essentially attempts to find a path from all vertices to all other vertices (except from a vertex to itself). While this code works, it is very inefficient. It performs a large amount of unnecessary work. As an example, given the graph in Figure 16.3 the `is_connected` function will find a path between ATL and SFO and also during its processing find a path between SFO and ATL. Since the graph is undirected, attempting to find the reverse path is superfluous. If the number of vertices in the graph is n , the `is_connected` function calls the `bfs` function $n^2 - n$ times. (The n^2 is the number of possibilities resulting from pairing every vertex with every other vertex, and we subtract n so we do not count pairing a vertex with itself.) Using this technique on a graph with 50 vertices produces 2,450 calls of `bfs`.

Section 15.3 showed how two different search algorithms both can be correct and yet perform dramatically differently. If we make a minor tweak to `bfs`, we can rewrite the `is_connected` function so that it can do its job with only *one* call to `bfs`. Listing 16.2 (`checkconnected.py`) extends Listing 16.1 (`airlineroute.py`) by adding an `is_connected` function. It makes two small changes to the original `bfs` function:

- The `start_vertex` and `end_vertex` parameters now are optional (default to `None`).
- The function will choose an arbitrary starting vertex from the graph if the caller does not supply one.

If the caller omits the `end_vertex` parameter, the `while` loop within `bfs` will compute predecessors for all reachable vertices within the graph. This is because `end_vertex` defaults to `None`, and the function never adds the `None` key to its initially empty predecessor dictionary. If the caller omits both the `start_vertex` and `end_vertex`, the function will choose an arbitrary starting vertex from the graph. Due to the nature of default arguments, the caller has no way to omit only the `start_vertex` parameter (see Section 8.2).

Listing 16.2: `checkconnected.py`

```
from queue import Queue

# A graph is a dictionary mapping a vertex label to a set
# of adjacent vertex labels
Graph = dict[str, set[str]]

def bfs(graph: Graph,
        start_vertex: str | None=None,
        end_vertex: str | None=None) -> dict[str, str]:
    """ Performs a breadth-first traversal of graph originating
    at start_vertex and ending when the traversal encounters
    end_vertex. If the caller omits the end_vertex parameter, the
    BFS continues to all vertices in the graph. If the caller
    omits the start_vertex parameter, the BFS begins with an
    arbitrary vertex in the graph.
    Builds and returns a dictionary with
    vertices mapped to their immediate predecessors on the
    breadth-first traversal. """

    # A dictionary of vertex predecessors encountered on a breadth-first
    # traversal from start_vertex to end_vertex.
    # The dictionary key is a vertex, and the associated value is
```

```

# the vertex that comes immediately before the key on a
# breadth-first traversal.
# This dictionary initially is empty, and we will add vertices as
# we "visit" them during the breadth-first traversal.
predecessor = {}

if graph:
    if not start_vertex:          # Caller provided a starting vertex?
        start_vertex = list(graph.keys())[0] # Grab an arbitrary vertex

    # Make an empty queue and insert the starting vertex
    # The algorithm will extract and visit vertices from this queue
    q = Queue()
    q.put(start_vertex)

    # Keep searching while the queue holds vertices yet to visit
    # and we have yet to visit our destination vertex
    while not q.empty() and end_vertex not in predecessor:
        vertex = q.get() # Get vertex on the front of the queue
        for adjacent_vertex in graph[vertex]: # Consider all adjacents
            # Has the predecessor of this vertex been established?
            if adjacent_vertex not in predecessor:
                q.put(adjacent_vertex) # Enqueue the vertex
                # Register which vertex should come immediately before this
                # one on a shortest path (this "visits" the vertex)
                #print(adjacent_vertex, "<--", vertex)
                predecessor[adjacent_vertex] = vertex

    # At this point we exited the while loop because either the queue was
    # empty or the destination vertex now is in the predecessor dictionary
    # (or both). If the queue is empty but the destination vertex is not
    # in the predecessor dictionary, the destination vertex is unreachable
    # from the starting vertex. If the queue is not empty but the
    # destination vertex is in the predecessor dictionary, the path from
    # the starting vertex to the destination vertex exists and excludes
    # one or more vertices in the graph. If the queue is empty and the
    # destination vertex is in the predecessor dictionary, the shortest
    # path from the starting vertex to the destination vertex
    # includes all vertices reachable from the starting vertex.

    return predecessor

def find_path(graph: Graph,
              start_vertex: str,
              end_vertex: str) -> list[str]:
    """ Builds a list of vertices in order along the shortest path
        from a starting vertex to an ending vertex in a graph.
        graph: The graph to traverse.
        start_vertex: The starting vertex.
        end_vertex: The vertex to locate. """

    # Compute predecessor of each vertex on a shortest path
    # Call the bfs function to build the predecessor dictionary
    predecessor = bfs(graph, start_vertex, end_vertex)

```

```

path = []          # Path initially empty

# Check that we were able to reach the destination vertex
if end_vertex in predecessor:
    # Start at the end and work backwards
    path = [end_vertex]
    vertex = end_vertex
    # Keep going until we get to the beginning of the path
    while vertex != start_vertex:
        # Get vertex that comes immediately before on a shortest path
        vertex = predecessor[vertex]
        # Prepend the predecessor vertex to the front of the path list
        path = [vertex] + path

    return path

def is_connected(G: Graph) -> bool:
    """ Returns True if G is a dictionary representing a connected graph;
        otherwise, returns False. A graph containing no vertices
        (and, therefore, no edges) is considered connected. """

    # Use bfs to compute the predecessor of each vertex on a
    # shortest path from an arbitrary vertex within G.
    predecessor = bfs(G)
    for vertex in G:
        if vertex not in predecessor:
            return False
    return True

def main():
    """ Tests the find_path and is_connected functions. """

    # Dictionary representing the graph of airline routes
    routes = {
        "ATL": {"MIA", "DCA", "ORD", "MCI", "DFW", "DEN"},
        "MIA": {"LGA", "DCA", "ATL", "DFW"},
        "DFW": {"LAX", "DEN", "MCI", "ORD", "ATL", "MIA"},
        "LAX": {"SFO", "DEN", "MCI", "DFW"},
        "DEN": {"SFO", "LAX", "MCI", "DFW", "SEA", "ATL"},
        "SEA": {"SFO", "DEN", "ORD", "LGA"},
        "MCI": {"DEN", "LAX", "DFW", "ATL", "ORD", "LGA"},
        "ORD": {"SEA", "MCI", "DFW", "ATL", "DCA", "LGA"},
        "DCA": {"ORD", "ATL", "MIA", "LGA"},
        "LGA": {"SEA", "MCI", "ORD", "DCA", "MIA"},
        "SFO": {"SEA", "DEN", "LAX"},
        "CLT": {"BNA", "CHA"},
        "BNA": {"CLT", "CHA"},
        "CHA": {"CLT", "BNA"}
    }

    r1 = {
        "ATL": {"MIA", "DCA", "ORD", "MCI", "DFW", "DEN"},

```

```

        "MIA": {"LGA", "DCA", "ATL", "DFW"},
        "DFW": {"LAX", "DEN", "MCI", "ORD", "ATL", "MIA"},
        "LAX": {"SFO", "DEN", "MCI", "DFW"},
        "DEN": {"SFO", "LAX", "MCI", "DFW", "SEA", "ATL"},
        "SEA": {"SFO", "DEN", "ORD", "LGA"},
        "MCI": {"DEN", "LAX", "DFW", "ATL", "ORD", "LGA"},
        "ORD": {"SEA", "MCI", "DFW", "ATL", "DCA", "LGA"},
        "DCA": {"ORD", "ATL", "MIA", "LGA"},
        "LGA": {"SEA", "MCI", "ORD", "DCA", "MIA"},
        "SFO": {"SEA", "DEN", "LAX"}
    }

    r2 = {
        "CLT": {"BNA", "CHA"},
        "BNA": {"CLT", "CHA"},
        "CHA": {"CLT", "BNA"}
    }

    # Find shortest paths, as before
    print(find_path(routes, "LAX", "DCA"))
    print(find_path(routes, "MIA", "SFO"))
    print(find_path(routes, "ATL", "MIA"))
    print(find_path(routes, "LGA", "LGA"))
    print(find_path(routes, "CLT", "BNA"))
    print(find_path(routes, "BNA", "ATL"))

    # Check connectivity
    print("Connected: ", is_connected(routes))
    print("Connected: ", is_connected(r1))
    print("Connected: ", is_connected(r2))

if __name__ == "__main__":
    main()

```

In Listing 16.2 (checkconnected.py) the graph `routes` is not connected, but `r1` and `r2` are both connected (not to each other, though). The execution of Listing 16.2 (checkconnected.py) prints

```

['LAX', 'MCI', 'ATL', 'DCA']
['MIA', 'LGA', 'SEA', 'SFO']
['ATL', 'MIA']
['LGA']
['CLT', 'BNA']
[]
Connected:  False
Connected:  True
Connected:  True

```

Observe that we made no changes to `find_path`, and yet it still works correctly with our modified `bfs` function.

16.5 Depth-first Search

An alternative to breadth-first search graph traversal is *depth-first search* (DFS). BFS conservatively visits vertices at increasing distances, never venturing farther away until all closer options are exhausted. DFS, on the other hand, traces a path that moves as far away from the starting vertex as possible. DFS backs up to consider other paths only when it can go no further. It can proceed no further when it reaches a vertex connected only to already visited vertices. At this point, it must back up to its most recently visited vertex and consider another path. It then continues its effort to move as far as possible away from the starting vertex.

The BFS algorithm uses a queue to guide its traversal. DFS uses a different accessory data structure—a *stack*. A stack is a linear data structure with restricted access like a queue, but, unlike a queue, a stack permits removal of only the most recently added element. A stack, therefore, is a *last-in, first out (LIFO)* container. Unlike the Queue class, Python does not have a standard stack class. Fortunately, we easily can design a stack class built from a normal Python list. Listing 16.3 (stack.py) implements a stack data type.

Listing 16.3: stack.py

```
class Stack:
    """ Implements a stack; that is, a last-in, first-out
        (LIFO) data structure. """
    def __init__(self) -> None:
        """ Initiailizes the internal storage for a stack
            object. """
        self._data = []

    def push(self, item: str) -> None:
        """ Places an item on the top of the stack. """
        self._data.append(item)

    def pop(self) -> str:
        """ Removes and returns the item on the top
            of the stack. """
        return self._data.pop()

    def top(self) -> str:
        """ Returns the item on the top of the stack
            without removing it. """
        return self._data[-1]

    def empty(self) -> bool:
        """ Returns true if the stack is empty. """
        return len(self._data) < 1
```

Lists support random access; that is, we can access an element at any index. In order to model a stack, the Stack class limits its efforts to modify a list to two `list` methods:

- `append` places an element onto the top (end) of the stack.
- `pop` removes and returns the last list element.

The `list` class has no `empty` method to determine if the list contains any elements, but we can use the `len` function and compare its result to zero.

We can practice with our stack in the interactive interpreter:

```
>>> from stack import Stack
>>> s = new Stack()
      File "<stdin>", line 1
        s = new Stack()
            ^^^^^
SyntaxError: invalid syntax
>>> s = Stack()
>>> s.empty()
True
>>> s.push('A')
>>> s.empty()
False
>>> s.push('B')
>>> s.push('C')
>>> s.pop()
'C'
>>> s.pop()
'B'
>>> s.pop()
'A'
>>> s.empty()
True
```

As you can see, what goes onto the stack last comes out first, and what goes in first comes out last.

DFS proceeds as follows:

Inputs: A starting vertex and destination vertex within a graph.

Begin with an empty set and an empty stack.

Push the starting vertex onto the stack.

while the stack is not empty

 Pop the top vertex, v , from the stack.

if v is not in the set:

 Add v to the set:

for each vertex w adjacent to v :

if w is not in the set:

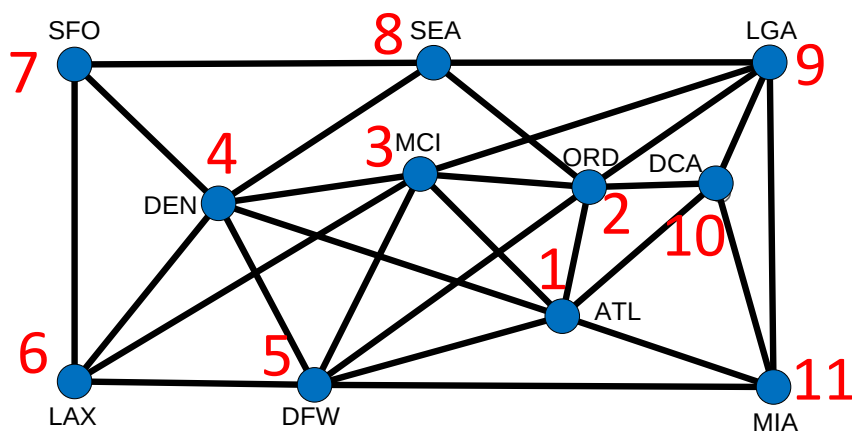
 Push w onto the stack

Output: The set containing vertices reachable from the starting vertex.

The first time through its **while** loop, the DFS algorithm extracts the starting vertex from the stack. Since the visited set begins empty, the algorithm adds the starting vertex to the set as its only element. It then will add all of the starting vertex's adjacent vertices to the stack. On the second iteration of its **while** loop the algorithm removes from the stack the last vertex it added on the previous iteration, one of the vertices adjacent to the starting vertex. Call this vertex v . The algorithm adds any nonvisited vertices adjacent to v to the stack. Because it uses a stack, the algorithm will visit these newly added vertices before it visits any vertices it added to the stack earlier. This allows the algorithm to propagate to vertices farther away from the starting vertex before it considers closer vertices. Figure 16.8 illustrates one such DFS traversal.

Listing 16.4 (dfs.py) implements the DFS algorithm.

Figure 16.8 The numbers indicate a possible order in which DFS visits vertices starting at vertex ATL. Since a Python program uses a dictionary to represent the graph, the exact order of the visitation is unpredictable, even from one execution to the next on the same machine. What is predictable, however, is that the path traced by DFS will tend to move farther from the starting vertex before visiting all vertices closer to the starting vertex. Observe that this traversal visits MIA last, even though MIA is adjacent to the starting vertex.



Listing 16.4: dfs.py

```

from stack import Stack

# A graph is a dictionary mapping a vertex label to a set
# of adjacent vertex labels
Graph = dict[str, set[str]]

def dfs(graph: Graph, start_vertex: str) -> set[str]:
    """ Performs a depth-first traversal of graph originating
    at start_vertex. Returns the set of vertices
    reachable from the start vertex. Uses an explicit
    stack object to guide the search. """

    # A set of vertices encountered on a depth-first
    # traversal from the start vertex.
    # This set initially is empty, and we will add vertices as
    # we "visit" them during the depth-first traversal.
    visited = set()

    # Make an empty stack and insert the starting vertex
    # The algorithm will extract and visit vertices from this stack
    s = Stack()
    s.push(start_vertex)

    # Keep searching while the stack holds vertices yet to visit
    # and we have yet to visit our destination vertex
    while not s.empty():
        vertex = s.pop() # Get vertex on the top of the stack
        if vertex not in visited:
            visited.add(vertex) # Visit the vertex
            for adjacent_vertex in graph[vertex]: # Consider all adjacents
                # Has the predecessor of this vertex been established?
                if adjacent_vertex not in visited:
                    s.push(adjacent_vertex) # Push the vertex
    return visited

def is_connected(G: Graph) -> bool:
    """ Returns True if G is a dictionary representing a connected
    graph; otherwise, returns False. A graph containing no
    vertices (and, therefore, no edges) is considered connected. """
    # Choose an arbitrary starting vertex
    v = list(G.keys())[0]
    # Use depth-first search to build a set of reachable vertices.
    predecessor_list = dfs(G, v)
    for vertex in G:
        if vertex not in predecessor_list:
            return False
    return True

def main():
    """ Tests is_connected function with DFS. """

```

```

# Dictionary representing the graph of airline routes
routes = {
    "ATL": {"MIA", "DCA", "ORD", "MCI", "DFW", "DEN"},
    "MIA": {"LGA", "DCA", "ATL", "DFW"},
    "DFW": {"LAX", "DEN", "MCI", "ORD", "ATL", "MIA"},
    "LAX": {"SFO", "DEN", "MCI", "DFW"},
    "DEN": {"SFO", "LAX", "MCI", "DFW", "SEA", "ATL"},
    "SEA": {"SFO", "DEN", "ORD", "LGA"},
    "MCI": {"DEN", "LAX", "DFW", "ATL", "ORD", "LGA"},
    "ORD": {"SEA", "MCI", "DFW", "ATL", "DCA", "LGA"},
    "DCA": {"ORD", "ATL", "MIA", "LGA"},
    "LGA": {"SEA", "MCI", "ORD", "DCA", "MIA"},
    "SFO": {"SEA", "DEN", "LAX"},
    "CLT": {"BNA", "CHA"},
    "BNA": {"CLT", "CHA"},
    "CHA": {"CLT", "BNA"}
}

r1 = {
    "ATL": {"MIA", "DCA", "ORD", "MCI", "DFW", "DEN"},
    "MIA": {"LGA", "DCA", "ATL", "DFW"},
    "DFW": {"LAX", "DEN", "MCI", "ORD", "ATL", "MIA"},
    "LAX": {"SFO", "DEN", "MCI", "DFW"},
    "DEN": {"SFO", "LAX", "MCI", "DFW", "SEA", "ATL"},
    "SEA": {"SFO", "DEN", "ORD", "LGA"},
    "MCI": {"DEN", "LAX", "DFW", "ATL", "ORD", "LGA"},
    "ORD": {"SEA", "MCI", "DFW", "ATL", "DCA", "LGA"},
    "DCA": {"ORD", "ATL", "MIA", "LGA"},
    "LGA": {"SEA", "MCI", "ORD", "DCA", "MIA"},
    "SFO": {"SEA", "DEN", "LAX"}
}

r2 = {
    "CLT": {"BNA", "CHA"},
    "BNA": {"CLT", "CHA"},
    "CHA": {"CLT", "BNA"}
}

# Check connectivity
print("Connected: ", is_connected(routes))
print("Connected: ", is_connected(r1))
print("Connected: ", is_connected(r2))

if __name__ == "__main__":
    main()

```

Listing 16.4 (dfs.py) correctly displays the following:

```

Connected:  False False
Connected:  True  True
Connected:  True  True

```

DFS uses a stack to guide the graph traversal, but we can rewrite the function so that the stack is “invisible:”

```
def dfs(graph: Graph, start_vertex: str) -> set[str]:
    """ Performs a depth-first traversal of graph originating
        at start_vertex. Returns the set of vertices
        reachable from the start vertex. Uses the implicit
        stack resulting from recursion rather than an
        explicit stack object to guide the traversal. """

    # A set of vertices encountered on a depth-first
    # traversal from the start vertex.
    # This set initially is empty, and we will add vertices as
    # we "visit" them during the depth-first traversal.
    visited = set()

    def dfs_helper(start_vertex: str) -> None:
        """ Recursive depth-first search. """
        visited.add(start_vertex)
        for adjacent_vertex in graph[start_vertex]:
            if adjacent_vertex not in visited:
                dfs_helper(adjacent_vertex)

    dfs_helper(start_vertex) # Call the local function
    return visited
```

This `dfs` function uses recursion via its inner helper function. A function invocation uses a portion of memory that is not coincidentally called the *stack*. This memory stack enables a function call to return to the location in the code from where it was called.

Consider the loop in the inner function, `dfs_helper`:

```
for adjacent_vertex in graph[start_vertex]:
    if adjacent_vertex not in visited:
        dfs_helper(adjacent_vertex)
```

The loop considers only the vertices adjacent to the start vertex. Suppose the start vertex has three neighboring vertices. It makes the recursive invocation, passing the start vertex's first neighbor. This call is a new DFS with this vertex as the start vertex. This recursion can visit vertices much farther away than the start vertex's other neighbors. This recursion must complete and return *before* the loop considers the next neighbor of the start vertex. The implicit function-call stack ensures that the recursion eventually will return back to the loop. The loop then will consider the next neighbor, and the recursion during this iteration will do a depth-first search from there. The overall effect accomplishes a DFS of a graph in a manner similar to the `dfs` function in Listing 16.4 (`dfs.py`) with its explicit `Stack` object.

Because of the way DFS operates, is not as useful for finding the shortest path between two vertices, especially if the graph is large. DFS may follow the wrong path very far before backing up to consider other branches. DFS is the better choice for solving a number of other graph problems. One of these problems is determining if a graph contains a cycle. Recall from Section 16.4 that a cycle is a path that begins at one vertex and passes through the initial vertex a second time. Figure 16.9 shows a graph with a cycle. Figure 16.10 shows a graph that does not contain a cycle. Listing 16.5 (`cycledetect.py`) uses DFS to detect a cycle within an undirected graph.

Listing 16.5: `cycledetect.py`

Figure 16.9 A graph that contains a cycle. Note the path 0-1-2-0. This path begins and ends at the same vertex.

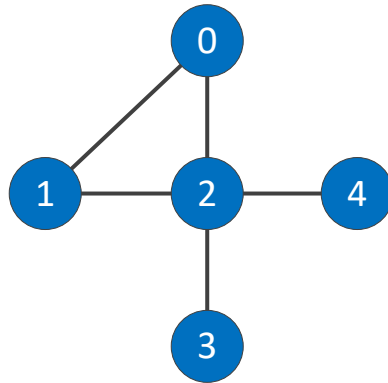
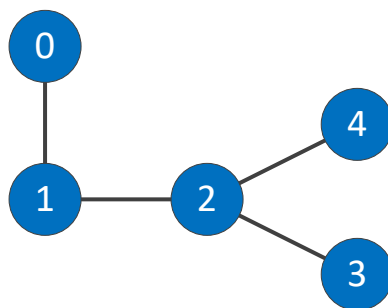


Figure 16.10 A graph that is cycle free. No simple path begins and ends at the same vertex.



```

from stack import Stack

# A graph is a dictionary mapping a vertex label to a set
# of adjacent vertex labels
Graph = dict[int, set[int]]

def cycle(G: Graph) -> bool:
    """ Returns True if G is a dictionary representing an acyclic
        undirected graph; otherwise, returns False. """
    visited = [False for _ in range(len(G))]
    open = [True for _ in range(len(G))]
    cycle_detected = False

    def dfs(vertex: int, prev: int | None=None) -> None:
        """ Performs a depth-first traversal of the graph,
            looking for a cycle. """
        nonlocal cycle_detected
        """ Recursive depth-first search. """
        if open[vertex]:
            if visited[vertex]:
                cycle_detected = True
            else:
                visited[vertex] = True
                for adj in G[vertex]:
                    # Exclude recursing on previous vertex
                    # from parent recursive call
                    if adj != prev:
                        dfs(adj, vertex)
                open[vertex] = False

        # Use depth-first search to detect cycle
    for vertex in G:
        dfs(vertex)
    return cycle_detected

def main():
    """ Tests cycle detection. """
    # Dictionary representing the graph
    G_1 = { # Graph Contains a cycle
        0: {1, 2},
        1: {0, 2},
        2: {0, 1, 3, 4},
        3: {2},
        4: {2}
    }

    G_2 = { # Graph is cycle free
        0: {1},
        1: {0, 2},
        2: {1, 3, 4},
        3: {2},
        4: {2}
    }

```



```
# Check for cycle
print("Cycle: ", cycle(G_1))
print("Cycle: ", cycle(G_2))

if __name__ == "__main__":
    main()
```

Listing 16.5 (cycledetect.py) displays the following:

```
Cycle:  True
Cycle:  False
```

Many useful graph algorithms involve variations of either BFS or DFS.

16.6 Exercises

CAUTION! SECTION UNDER CONSTRUCTION

Add item.

Index

needs a relationship, 574

absolute value, 112
 accumulator, 127
 actual, 183
 adapter design pattern, 602
 adjacency list, 709
 adjacent vertices, 705
 aggregate, 574
 aggregation, 574
 algorithm, 64
 aliases, 205
 aliasing, 397
 aliasing, object, 372
 arc, 705
 ascending ordering of list elements, 631
 associative array, 451
 associative container, 448
 attribute, 518
 attributes, 350

base case, 269
 base class, 578
 benchmarking, 207
 BFS, 711
 binary search, 657
 block, 12
 body, 76
 Boolean, 71
 breadth-first search, 711
 bugs, 60

callback function, 284, 529
 calling code, 173
 catch-all exception handler, 485, 500
 catching an exception, 481
 chained assignment, 54
 class, 17, 349
 class constructor, 361
 class hierarchy, 590
 class variables, 554
 client, 350

client code, 173
 closure, 306
 code duplication, 248
 code instrumentation, 677
 comma-separated list, 22
 command prompt, 411
 commutative, 390
 compiler, 3
 composition, 573
 concatenation, 19
 conditional expression, 111
 connected graph, 717
 constructor, 18
 context manager, 356
 control codes, 33
 cooperative multiple inheritance, 604
 cycle in a graph, 711

data, 349
 data persistence, 354
 debugger, 3
 decorator, 335
 default argument, 264
 default parameters, 264
 definite loop, 132
 delimiter, 16
 denominator, 361, 558
 dependent function, 263
 depth-first search, 722
 derivative, 323
 derived class, 578
 design pattern, 602
 DFS, 722
 dictionary, 448
 dictionary key, 448
 differentiation, 324
 digraph, 705
`dir` function, 179
 direct base class, 588
 directed graph, 705
 directory function, 179
 disconnected graph, 717

- docstring, 232
- documentation string, 232
- double negative, 82
- double underscore, 556
- dunder, 556
- dynamically-typed, 233
- EAFP, 481
- easier to ask for forgiveness than permission, 481
- edge, 705
- elapsed time, 186
- end keyword argument in `print`, 38
- Eratosthenes, 409
- escape symbol, 33
- `except` block, 481
- exception, 477
- exception `else` block, 507
- exception classes, common standard, 478
- exception handling, 477
- exception, re-raising, 502
- exception, swallowing, 500
- existential quantification, 468
- expression, 15
- external documentation, 232
- factorial, 267
- Fibonacci sequence, 272
- fields, 350
- files, 354
- `finally` block, 509
- floating-point numbers, 29
- `for` statement, 133
- formal, 183
- function, 172
- function definition, 212
- function invocation, 212
- function `time.perf_counter_ns`, 186
- function `time.sleep`, 188
- function call, 173
- function coherence, 238
- function composition, 226, 244
- function definition parts, 212
- function invocation, 173
- functional composition, 36
- functional decomposition, 212
- functional independence, 263
- functionally equivalent, 632
- garbage, 374
- garbage collection, 374
- generalized unpacking, 444
- generator, 312
- generator comprehension, 418
- generator expression, 418
- get attribute, 552
- `getattr`, 552
- getter, 557
- global variable, 259
- graph, 705
- graph theory, 705
- handling an exception, 481
- has a* relationship, 574
- hash table, 451
- hashing, 451
- Hoare, C. A. R., 647
- `id` function, 179
- identifier, 27
- identifiers beginning with two underscores, 556
- `if` statement, 73
- immutable, 229
- in place list processing, 671
- indefinite loop, 133
- index, 353, 379
- indirect base class, 588
- inheritance, 578
- inheritance hierarchy, 590
- inheritance, single, 581
- initializer method, 519
- instance variable, 350, 518
- instance variable, object, 521
- integer division, 48
- internal documentation, 232
- interpreter, 2, 3
- `is a`, 586
- key, 448
- keyword argument, 38
- keyword arguments, 462
- keywords, 28
- lambda calculus, 306
- lambda expression, 305
- last-in, first out, 722
- lazy list, 429
- LBYL, 480
- LCS, 696
- `len` function, 384
- length, 384
- LIFO, 722
- linear search, 656

- list aliasing, 397
- list comprehension, 413
- `list` function, 388
- list of lists, 419
- list slicing, 402
- local variables, 259
- longest common subsequence, 696
- look before you leap, 480
- loop unrolling, 676

- mathematical graph, 705
- matrix, 418
- member, object, 521
- members, 350
- memoization, 692
- Mersenne Twister, 189
- method call, 350
- method override, 579
- method resolution order, 610
- method, object, 521
- methods, 349
- midpoint, 220
- module, 177
- modules, 171, 275
- modulus, 48
- monolithic code, 211
- MRO, 610
- multiple inheritance, 603
- multiplication table, 138
- mutable, 383, 397

- name collision, 202
- name mangling, 556
- namespace, 202
- namespace pollution, 204
- nested, 87
- nested loop, 138
- newline, 33
- node, 705
- non-decreasing order of list elements, 631
- `None` object reference, 374
- numerator, 361, 558
- numerical differentiation, 324

- object aliasing, 372
- object oriented, 349
- operations, 350
- override, 581

- Papert, Seymour, 195
- partial application, 340

- pass statement, 84
- permutation, 143, 674
- precedence, 53
- predicate, 71, 469
- private object attributes, 556
- profiler, 3
- properties, 556
- pure function, 264

- qualified name, 178
- quantification, 468
- queue, 711
- Quicksort, 647

- ragged array, 419
- random access, 429
- re-raising an exception, 502
- read, eval, print loop, 16
- recursive, 267
- recursive case, 269
- refactoring, 248
- refactoring code, 248
- reference counting, 374
- regular expression, 481
- relational operators, 72
- remainder, 48
- reserved words, 28
- run-time errors, 58
- run-time exceptions, 58

- scripting languages, 3
- selection sort, 642
- `sep` keyword argument in `print`, 39
- sequence, 696
- set attribute, 552
- set comprehension, 466
- `setattr`, 552
- setter, 557
- short-circuit evaluation, 83
- Sieve of Eratosthenes, 409
- single inheritance, 581
- slice assignment, 404
- slicing, 93, 402
- soft keywords, 109
- stack, 492, 722
- stack trace, 235, 492
- stack unwinding, 492
- statement, 5
- statically-typed language, 234
- string, 16
- stub, 544

subclass, 578
subscript, 380
subsequence, 696
super function, 581
superclass, 578
swallowing an exception, 500
symbolic differentiation, 324
syntactic sugar, 362
syntax error, 57

tab stop, 77
table, 419
tags, Tk, 573
terminal, 411
test runner, 626
test-driven development, 540
testing, 540
text files, 359
texttttry statement, 481
throw away variable, 424
throwing an exception, 481
times table, 138
Tk tags, 573
Tk toolkit, 364
tkinter, 364
token, 458
transitivity, 632
transitivity of class inheritance, 588
translation phase, 57
try block, 481
tuple, 23
tuple assignment, 23
tuple unpacking, 221
Turtle graphics, 195
type, 17
type aliasing, 280
type hints, 235

UML, 590
unbound variable, 26
undefined variable, 26
underscore, beginning an identifier, 556
Unified Modeling Language, 590
unit testing, 622
universal quantification, 468
unpacking a tuple, 221
unwinding the stack, 492

value, 448
vertex, 705

while statement, 124
whitespace, 12
wrapper, 602

yield from, 682
yield statement, 312